

Mathematics for Computer Science

revised Wednesday 4th January, 2012, 13:53

Eric Lehman

Google Inc.

F Thomson Leighton

Department of Mathematics
and the Computer Science and AI Laboratory,
Massachusetts Institute of Technology;
Akamai Technologies

Albert R Meyer

Department of Electrical Engineering and Computer Science
and the Computer Science and AI Laboratory,
Massachusetts Institute of Technology

Contents

I Proofs

1	What is a Proof?	5
1.1	Propositions	5
1.2	Predicates	7
1.3	The Axiomatic Method	8
1.4	Our Axioms	9
1.5	Proving an Implication	11
1.6	Proving an “If and Only If”	13
1.7	Proof by Cases	15
1.8	Proof by Contradiction	16
1.9	<i>Good</i> Proofs in Practice	17
2	The Well Ordering Principle	25
2.1	Well Ordering Proofs	25
2.2	Template for Well Ordering Proofs	26
2.3	Factoring into Primes	28
3	Logical Formulas	35
3.1	Propositions from Propositions	36
3.2	Propositional Logic in Computer Programs	39
3.3	Equivalence and Validity	41
3.4	The Algebra of Propositions	44
3.5	The SAT Problem	49
3.6	Predicate Formulas	50
4	Mathematical Data Types	69
4.1	Sets	69
4.2	Sequences	72
4.3	Functions	73
4.4	Binary Relations	75
5	Infinite Sets	89
5.1	Finite Cardinality	90
5.2	Infinite Cardinality	92
5.3	The Halting Problem	97
5.4	The Logic of Sets	101
5.5	Does All This Really Work?	104

6	Induction	115
6.1	Ordinary Induction	115
6.2	Strong Induction	124
6.3	Strong Induction vs. Induction vs. Well Ordering	128
6.4	State Machines	129
7	Recursive Data Types	161
7.1	Recursive Definitions and Structural Induction	161
7.2	Strings of Matched Brackets	165
7.3	Recursive Functions on Nonnegative Integers	168
7.4	Arithmetic Expressions	171
7.5	Induction in Computer Science	176
8	Number Theory	187
8.1	Divisibility	187
8.2	The Greatest Common Divisor	193
8.3	The Fundamental Theorem of Arithmetic	199
8.4	Alan Turing	202
8.5	Modular Arithmetic	205
8.6	Arithmetic with a Prime Modulus	208
8.7	Arithmetic with an Arbitrary Modulus	213
8.8	The RSA Algorithm	219
8.9	What has SAT got to do with it?	221

II Structures

9	Directed graphs & Partial Orders	245
9.1	Digraphs & Vertex Degrees	247
9.2	Digraph Walks and Paths	248
9.3	Adjacency Matrices	251
9.4	Walk Relations	254
9.5	Directed Acyclic Graphs & Partial Orders	255
9.6	Weak Partial Orders	258
9.7	Representing Partial Orders by Set Containment	260
9.8	Path-Total Orders	261
9.9	Product Orders	262
9.10	Scheduling	263
9.11	Equivalence Relations	269
9.12	Summary of Relational Properties	270

10 Communication Networks 295

- 10.1 Complete Binary Tree 295
- 10.2 Routing Problems 295
- 10.3 Network Diameter 296
- 10.4 Switch Count 297
- 10.5 Network Latency 298
- 10.6 Congestion 298
- 10.7 2-D Array 299
- 10.8 Butterfly 301
- 10.9 Beneš Network 303

11 Simple Graphs 315

- 11.1 Vertex Adjacency and Degrees 315
- 11.2 Sexual Demographics in America 317
- 11.3 Some Common Graphs 319
- 11.4 Isomorphism 321
- 11.5 Bipartite Graphs & Matchings 323
- 11.6 The Stable Marriage Problem 328
- 11.7 Coloring 335
- 11.8 Getting from u to v in a Graph 339
- 11.9 Connectivity 341
- 11.10 Odd Cycles and 2-Colorability 345
- 11.11 Forests & Trees 346

12 Planar Graphs 381

- 12.1 Drawing Graphs in the Plane 381
- 12.2 Definitions of Planar Graphs 381
- 12.3 Euler’s Formula 392
- 12.4 Bounding the Number of Edges in a Planar Graph 393
- 12.5 Returning to K_5 and $K_{3,3}$ 394
- 12.6 Coloring Planar Graphs 395
- 12.7 Classifying Polyhedra 397
- 12.8 Another Characterization for Planar Graphs 400

13 State Machines 407

- 13.1 The Alternating Bit Protocol 407
- 13.2 Reasoning About **While** Programs 410

III Counting

14 Sums and Asymptotics 421

- 14.1 The Value of an Annuity 422
- 14.2 Sums of Powers 428
- 14.3 Approximating Sums 430
- 14.4 Hanging Out Over the Edge 434
- 14.5 Products 446
- 14.6 Double Trouble 448
- 14.7 Asymptotic Notation 451

15 Cardinality Rules 471

- 15.1 Counting One Thing by Counting Another 471
- 15.2 Counting Sequences 472
- 15.3 The Generalized Product Rule 475
- 15.4 The Division Rule 479
- 15.5 Counting Subsets 482
- 15.6 Sequences with Repetitions 483
- 15.7 The Binomial Theorem 485
- 15.8 A Word about Words 487
- 15.9 Counting Practice: Poker Hands 487
- 15.10 The Pigeonhole Principle 492
- 15.11 A Magic Trick 496
- 15.12 Inclusion-Exclusion 501
- 15.13 Combinatorial Proofs 507

16 Generating Functions 541

- 16.1 Operations on Generating Functions 542
- 16.2 The Fibonacci Sequence 547
- 16.3 Counting with Generating Functions 550
- 16.4 An “Impossible” Counting Problem 554

IV Probability

17 Events and Probability Spaces 571

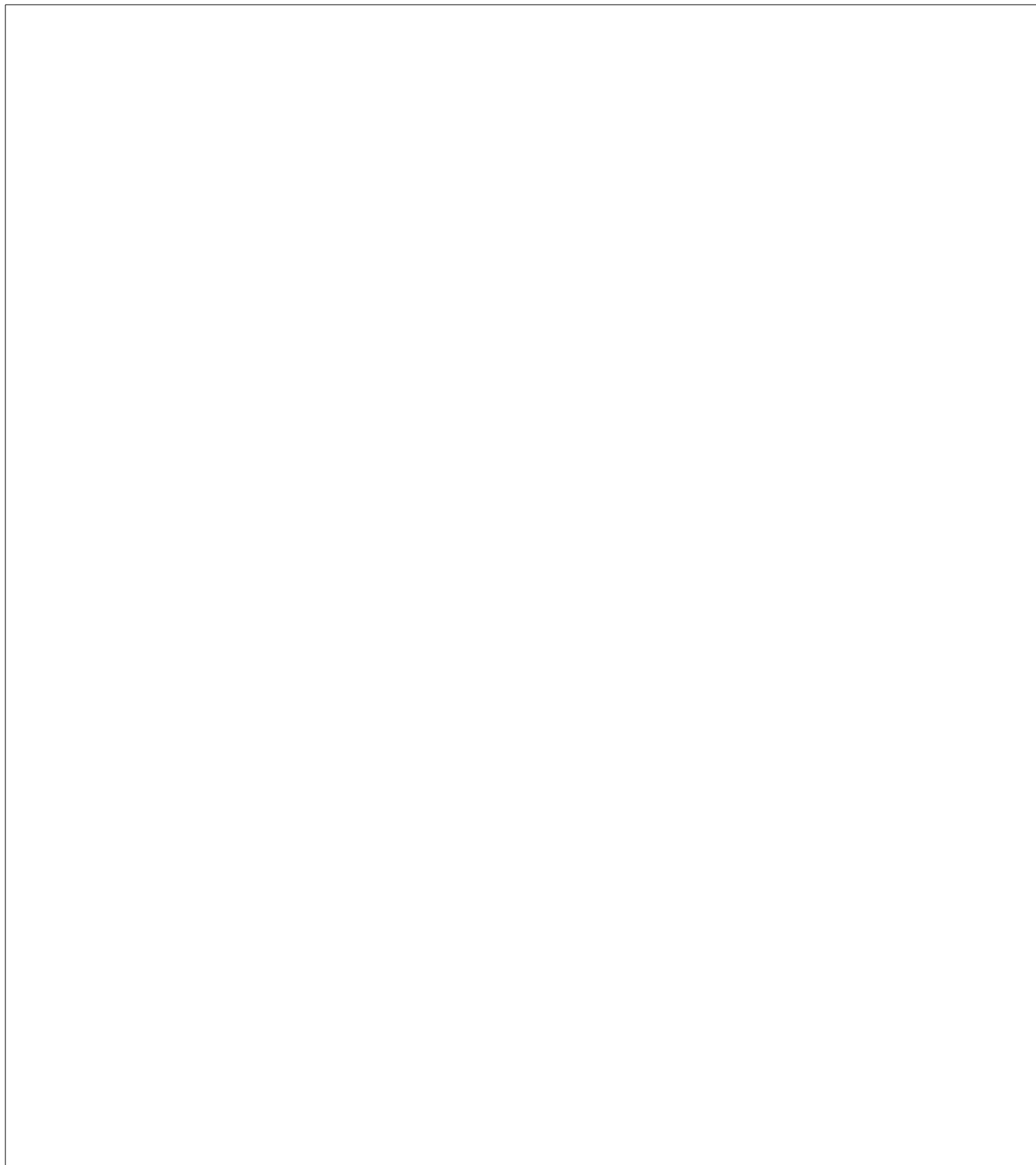
- 17.1 Let’s Make a Deal 571
- 17.2 The Four Step Method 572
- 17.3 Strange Dice 581
- 17.4 Set Theory and Probability 589

17.5	Conditional Probability	593
17.6	Independence	605
18	Random Variables	635
18.1	Random Variable Examples	635
18.2	Independence	637
18.3	Distribution Functions	638
18.4	Great Expectations	646
18.5	Linearity of Expectation	658
19	Deviation from the Mean	679
19.1	Why the Mean?	679
19.2	Markov’s Theorem	680
19.3	Chebyshev’s Theorem	682
19.4	Properties of Variance	686
19.5	Estimation by Random Sampling	690
19.6	Confidence versus Probability	695
19.7	Sums of Random Variables	696
19.8	Really Great Expectations	706
20	Random Processes	725
20.1	Gamblers’ Ruin	725
20.2	Random Walks on Graphs	734

V Recurrences

21	Recurrences	753
21.1	The Towers of Hanoi	753
21.2	Merge Sort	760
21.3	Linear Recurrences	764
21.4	Divide-and-Conquer Recurrences	771
21.5	A Feel for Recurrences	778
Index		780

I Proofs



Introduction

This text explains how to use mathematical models and methods to analyze problems that arise in computer science. Proofs play a central role in this work because the authors share a belief with most mathematicians that proofs are essential for genuine understanding. Proofs also play a growing role in computer science; they are used to certify that software and hardware will *always* behave correctly, something that no amount of testing can do.

Simply put, a proof is a method of establishing truth. Like beauty, “truth” sometimes depends on the eye of the beholder, and it should not be surprising that what constitutes a proof differs among fields. For example, in the judicial system, *legal* truth is decided by a jury based on the allowable evidence presented at trial. In the business world, *authoritative* truth is specified by a trusted person or organization, or maybe just your boss. In fields such as physics or biology, *scientific* truth¹ is confirmed by experiment. In statistics, *probable* truth is established by statistical analysis of sample data.

Philosophical proof involves careful exposition and persuasion typically based on a series of small, plausible arguments. The best example begins with “Cogito ergo sum,” a Latin sentence that translates as “I think, therefore I am.” It comes from the beginning of a 17th century essay by the mathematician/philosopher, René Descartes, and it is one of the most famous quotes in the world: do a web search on the phrase and you will be flooded with hits.

¹Actually, only scientific *falsehood* can be demonstrated by an experiment—when the experiment fails to behave as predicted. But no amount of experiment can confirm that the *next* experiment won’t fail. For this reason, scientists rarely speak of truth, but rather of *theories* that accurately predict past, and anticipated future, experiments.

Deducing your existence from the fact that you’re thinking about your existence is a pretty cool and persuasive-sounding idea. However, with just a few more lines of argument in this vein, Descartes [goes on](#) to conclude that there is an infinitely beneficent God. Whether or not you believe in an infinitely beneficent God, you’ll probably agree that any very short “proof” of God’s infinite beneficence is bound to be far-fetched. So even in masterful hands, this approach is not reliable.

Mathematics has its own specific notion of “proof.”

Definition. A *mathematical proof* of a *proposition* is a chain of *logical deductions* leading to the proposition from a base set of *axioms*.

The three key ideas in this definition are highlighted: *proposition*, *logical deduction*, and *axiom*. Chapter [1](#) examines these three ideas along with some basic ways of organizing proofs. Chapter [2](#) introduces proofs using the Well Ordering Principle; later Chapter [6](#) introduces the closely related proof method of Induction.

If you’re going to prove a proposition, you better have a precise understanding of what the proposition means. To avoid ambiguity and uncertain definitions in ordinary language, mathematicians use language very precisely, and they often express propositions using logical formulas; these are the subject of Chapter [3](#).

The first three Chapters assume the reader is familiar with a few mathematical concepts like sets and functions. Chapters [4](#) and [5](#) offer a more careful look at such mathematical data types, examining in particular properties and methods for proving things about infinite sets. Chapter [7](#) goes on to examine recursive data types.

Number theory is the study of properties of the integers. This part of the text ends with Chapter [8](#) on Number theory because there are lots of easy-to-state and interesting-to-prove properties of numbers. This subject was once thought to have few, if any, practical applications, but it has turned out to have multiple applications in Computer Science. For example, most modern data encryption methods are based on Number theory.

1 What is a Proof?

1.1 Propositions

Definition. A *proposition* is a statement that is either true or false.

For example, both of the following statements are propositions. The first is true and the second is false.

Proposition 1.1.1. $2 + 3 = 5$.

Proposition 1.1.2. $1 + 1 = 3$.

Being true or false doesn’t sound like much of a limitation, but it does exclude statements such as, “Wherefore art thou Romeo?” and “Give me an A!” It also excludes statements whose truth varies with circumstance such as, “It’s five o’clock,” or “the stock market will rise tomorrow.”

Unfortunately it is not always easy to decide if a proposition is true or false:

Proposition 1.1.3. *For every nonnegative integer, n , the value of $n^2 + n + 41$ is prime.*

(A *prime* is an integer greater than one that is not divisible by any other integer greater than 1, for example, 2, 3, 5, 7, 11, ...) Let’s try some numerical experimentation to check this proposition. Let ¹

$$p(n) ::= n^2 + n + 41. \quad (1.1)$$

We begin with $p(0) = 41$ which is prime; then

$$p(1) = 43, p(2) = 47, p(3) = 53, \dots, p(20) = 461$$

are each prime. Hmmm, starts to look like a plausible claim. In fact we can keep checking through $n = 39$ and confirm that $p(39) = 1601$ is prime.

But $p(40) = 40^2 + 40 + 41 = 41 \cdot 41$, which is not prime. So it’s not true that the expression is prime *for all* nonnegative integers. In fact, it’s not hard to show that *no* polynomial with integer coefficients can map all nonnegative numbers into prime numbers, unless it’s a constant (see Problem 1.6). The point is that in general

¹The symbol $::=$ means “equal by definition.” It’s always ok simply to write “=” instead of $::=$, but reminding the reader that an equality holds by definition can be helpful.

you can’t check a claim about an infinite set by checking a finite set of its elements, no matter how large the finite set.

By the way, propositions like this about *all* numbers or all items of some kind are so common that there is a special notation for them. With this notation, Proposition 1.1.3 would be

$$\forall n \in \mathbb{N}. p(n) \text{ is prime.} \quad (1.2)$$

Here the symbol $\forall$ is read “for all.” The symbol $\mathbb{N}$ stands for the set of *nonnegative integers*, namely, 0, 1, 2, 3, ... (ask your instructor for the complete list). The symbol “ $\in$ ” is read as “is a member of,” or “belongs to,” or simply as “is in.” The period after the $\mathbb{N}$ is just a separator between phrases.

Here are two even more extreme examples:

Proposition 1.1.4. *[Euler’s Conjecture] The equation*

$$a^4 + b^4 + c^4 = d^4$$

has no solution when a, b, c, d are positive integers.

Euler (pronounced “oiler”) conjectured this in 1769. But the proposition was proven false 218 years later by Noam Elkies at a liberal arts school up Mass Ave. The solution he found was $a = 95800, b = 217519, c = 414560, d = 422481$.

In logical notation, Euler’s Conjecture could be written,

$$\forall a \in \mathbb{Z}^+ \forall b \in \mathbb{Z}^+ \forall c \in \mathbb{Z}^+ \forall d \in \mathbb{Z}^+. a^4 + b^4 + c^4 \neq d^4.$$

Here, $\mathbb{Z}^+$ is a symbol for the positive integers. Strings of $\forall$ ’s like this are usually abbreviated for easier reading:

$$\forall a, b, c, d \in \mathbb{Z}^+. a^4 + b^4 + c^4 \neq d^4.$$

Proposition 1.1.5. $313(x^3 + y^3) = z^3$ *has no solution when $x, y, z \in \mathbb{Z}^+$.*

This proposition is also false, but the smallest counterexample has more than 1000 digits!

Proposition 1.1.6. *Every map can be colored with 4 colors so that adjacent² regions have different colors.*

²Two regions are adjacent only when they share a boundary segment of positive length. They are not considered to be adjacent if their boundaries meet only at a few points.

This proposition is true and is known as the “*Four-Color Theorem*.” However, there have been many incorrect proofs, including one that stood for 10 years in the late 19th century before the mistake was found. A laborious proof was finally found in 1976 by mathematicians Appel and Haken, who used a complex computer program to categorize the four-colorable maps; the program left a few thousand maps uncategorized, and these were checked by hand by Haken and his assistants—including his 15-year-old daughter. There was a lot of debate about whether this was a legitimate proof: the proof was too big to be checked without a computer, and no one could guarantee that the computer calculated correctly, nor did anyone have the energy to recheck the four-colorings of thousands of maps that were done by hand. Within the past decade a mostly [intelligible proof](#) of the Four-Color Theorem was found, though a computer is still needed to check colorability of several hundred special maps.³

Proposition 1.1.7 (Goldbach’s Conjecture). *Every even integer greater than 2 is the sum of two primes.*

Goldbach’s Conjecture dates back to 1742, and to this day, no one knows whether it’s true or false.

For a computer scientist, some of the most important things to prove are the correctness of programs and systems—whether a program or system does what it’s supposed to. Programs are notoriously buggy, and there’s a growing community of researchers and practitioners trying to find ways to prove program correctness. These efforts have been successful enough in the case of CPU chips that they are now routinely used by leading chip manufacturers to prove chip correctness and avoid mistakes like the notorious Intel division bug in the 1990’s.

Developing mathematical methods to verify programs and systems remains an active research area. We’ll illustrate some of these methods in [Chapter 6](#).

1.2 Predicates

A *predicate* is a proposition whose truth depends on the value of one or more variables.

Most of the propositions above were defined in terms of predicates. For example,

“ n is a perfect square”

³The story of the Four-Color Proof is told in a well-reviewed popular (non-technical) book: “Four Colors Suffice. How the Map Problem was Solved.” *Robin Wilson*. Princeton Univ. Press, 2003, 276pp. ISBN 0-691-11533-8.

is a predicate whose truth depends on the value of n . The predicate is true for $n = 4$ since four is a perfect square, but false for $n = 5$ since five is not a perfect square.

Like other propositions, predicates are often named with a letter. Furthermore, a function-like notation is used to denote a predicate supplied with specific variable values. For example, we might name our earlier predicate P :

$$P(n) ::= \text{“}n \text{ is a perfect square”}.$$

So $P(4)$ is true, and $P(5)$ is false.

This notation for predicates is confusingly similar to ordinary function notation. If P is a predicate, then $P(n)$ is either *true* or *false*, depending on the value of n . On the other hand, if p is an ordinary function, like $n^2 + 1$, then $p(n)$ is a *numerical quantity*. **Don’t confuse these two!**

1.3 The Axiomatic Method

The standard procedure for establishing truth in mathematics was invented by Euclid, a mathematician working in Alexandria, Egypt around 300 BC. His idea was to begin with five *assumptions* about geometry, which seemed undeniable based on direct experience. (For example, “There is a straight line segment between every pair of points.”) Propositions like these that are simply accepted as true are called *axioms*.

Starting from these axioms, Euclid established the truth of many additional propositions by providing “proofs.” A *proof* is a sequence of logical deductions from axioms and previously-proved statements that concludes with the proposition in question. You probably wrote many proofs in high school geometry class, and you’ll see a lot more in this text.

There are several common terms for a proposition that has been proved. The different terms hint at the role of the proposition within a larger body of work.

- Important propositions are called *theorems*.
- A *lemma* is a preliminary proposition useful for proving later propositions.
- A *corollary* is a proposition that follows in just a few logical steps from a theorem.

The definitions are not precise. In fact, sometimes a good lemma turns out to be far more important than the theorem it was originally used to prove.

Euclid’s axiom-and-proof approach, now called the *axiomatic method*, remains the foundation for mathematics today. In fact, just a handful of axioms, called the axioms Zermelo-Frankel with Choice (*ZFC*), together with a few logical deduction rules, appear to be sufficient to derive essentially all of mathematics. We’ll examine these in Chapter 5.

1.4 Our Axioms

The ZFC axioms are important in studying and justifying the foundations of mathematics, but for practical purposes, they are much too primitive. Proving theorems in ZFC is a little like writing programs in byte code instead of a full-fledged programming language—by one reckoning, a formal proof in ZFC that $2 + 2 = 4$ requires more than 20,000 steps! So instead of starting with ZFC, we’re going to take a *huge* set of axioms as our foundation: we’ll accept all familiar facts from high school math!

This will give us a quick launch, but you may find this imprecise specification of the axioms troubling at times. For example, in the midst of a proof, you may find yourself wondering, “Must I prove this little fact or can I take it as an axiom?” There really is no absolute answer, since what’s reasonable to assume and what requires proof depends on the circumstances and the audience. A good general guideline is simply to be up front about what you’re assuming, and don’t try to evade needed work by declaring everything an axiom!

1.4.1 Logical Deductions

Logical deductions or *inference rules* are used to prove new propositions using previously proved ones.

A fundamental inference rule is *modus ponens*. This rule says that a proof of P together with a proof that P IMPLIES Q is a proof of Q .

Inference rules are sometimes written in a funny notation. For example, *modus ponens* is written:

Rule.

$$\frac{P, \quad P \text{ IMPLIES } Q}{Q}$$

When the statements above the line, called the *antecedents*, are proved, then we can consider the statement below the line, called the *conclusion* or *consequent*, to also be proved.

A key requirement of an inference rule is that it must be *sound*: an assignment of truth values to the letters, P , Q , ..., that makes all the antecedents true must also make the consequent true. So if we start off with true axioms and apply sound inference rules, everything we prove will also be true.

There are many other natural, sound inference rules, for example:

Rule.

$$\frac{P \text{ IMPLIES } Q, \quad Q \text{ IMPLIES } R}{P \text{ IMPLIES } R}$$

Rule.

$$\frac{\text{NOT}(P) \text{ IMPLIES NOT}(Q)}{Q \text{ IMPLIES } P}$$

On the other hand,

Non-Rule.

$$\frac{\text{NOT}(P) \text{ IMPLIES NOT}(Q)}{P \text{ IMPLIES } Q}$$

is not sound: if P is assigned **T** and Q is assigned **F**, then the antecedent is true and the consequent is not.

Note that a propositional inference rule is sound precisely when the conjunction (AND) of all its antecedents implies its consequent.

As with axioms, we will not be too formal about the set of legal inference rules. Each step in a proof should be clear and “logical”; in particular, you should state what previously proved facts are used to derive each new conclusion.

1.4.2 Patterns of Proof

In principle, a proof can be *any* sequence of logical deductions from axioms and previously proved statements that concludes with the proposition in question. This freedom in constructing a proof can seem overwhelming at first. How do you even *start* a proof?

Here’s the good news: many proofs follow one of a handful of standard templates. Each proof has its own details, of course, but these templates at least provide you with an outline to fill in. We’ll go through several of these standard patterns, pointing out the basic idea and common pitfalls and giving some examples. Many of these templates fit together; one may give you a top-level outline while others help you at the next level of detail. And we’ll show you other, more sophisticated proof techniques later on.

The recipes below are very specific at times, telling you exactly which words to write down on your piece of paper. You’re certainly free to say things your own

way instead; we’re just giving you something you *could* say so that you’re never at a complete loss.

1.5 Proving an Implication

Propositions of the form “If P , then Q ” are called *implications*. This implication is often rephrased as “ P IMPLIES Q .”

Here are some examples:

- (Quadratic Formula) If $ax^2 + bx + c = 0$ and $a \neq 0$, then

$$x = \left(-b \pm \sqrt{b^2 - 4ac} \right) / 2a.$$

- (Goldbach’s Conjecture) If n is an even integer greater than 2, then n is a sum of two primes.
- If $0 \leq x \leq 2$, then $-x^3 + 4x + 1 > 0$.

There are a couple of standard methods for proving an implication.

1.5.1 Method #1

In order to prove that P IMPLIES Q :

1. Write, “Assume P .”
2. Show that Q logically follows.

Example

Theorem 1.5.1. *If $0 \leq x \leq 2$, then $-x^3 + 4x + 1 > 0$.*

Before we write a proof of this theorem, we have to do some scratchwork to figure out why it is true.

The inequality certainly holds for $x = 0$; then the left side is equal to 1 and $1 > 0$. As x grows, the $4x$ term (which is positive) initially seems to have greater magnitude than $-x^3$ (which is negative). For example, when $x = 1$, we have $4x = 4$, but $-x^3 = -1$ only. In fact, it looks like $-x^3$ doesn’t begin to dominate until $x > 2$. So it seems the $-x^3 + 4x$ part should be nonnegative for all x between 0 and 2, which would imply that $-x^3 + 4x + 1$ is positive.

So far, so good. But we still have to replace all those “seems like” phrases with solid, logical arguments. We can get a better handle on the critical $-x^3 + 4x$ part by factoring it, which is not too hard:

$$-x^3 + 4x = x(2 - x)(2 + x)$$

Aha! For x between 0 and 2, all of the terms on the right side are nonnegative. And a product of nonnegative terms is also nonnegative. Let’s organize this blizzard of observations into a clean proof.

Proof. Assume $0 \leq x \leq 2$. Then x , $2 - x$, and $2 + x$ are all nonnegative. Therefore, the product of these terms is also nonnegative. Adding 1 to this product gives a positive number, so:

$$x(2 - x)(2 + x) + 1 > 0$$

Multiplying out on the left side proves that

$$-x^3 + 4x + 1 > 0$$

as claimed. ■

There are a couple points here that apply to all proofs:

- You’ll often need to do some scratchwork while you’re trying to figure out the logical steps of a proof. Your scratchwork can be as disorganized as you like—full of dead-ends, strange diagrams, obscene words, whatever. But keep your scratchwork separate from your final proof, which should be clear and concise.
- Proofs typically begin with the word “Proof” and end with some sort of doohickey like $\square$ or “q.e.d.” The only purpose for these conventions is to clarify where proofs begin and end.

1.5.2 Method #2 - Prove the Contrapositive

An implication (“ P IMPLIES Q ”) is logically equivalent to its *contrapositive*

$$\text{NOT}(Q) \text{ IMPLIES } \text{NOT}(P).$$

Proving one is as good as proving the other, and proving the contrapositive is sometimes easier than proving the original statement. If so, then you can proceed as follows:

1. Write, “We prove the contrapositive:” and then state the contrapositive.
2. Proceed as in Method #1.

Example

Theorem 1.5.2. *If r is irrational, then $\sqrt{r}$ is also irrational.*

A number is *rational* when it equals a quotient of integers, that is, if it equals m/n for some integers m and n . If it's not rational, then it's called *irrational*. So we must show that if r is *not* a ratio of integers, then $\sqrt{r}$ is also *not* a ratio of integers. That's pretty convoluted! We can eliminate both *not*'s and make the proof straightforward by using the contrapositive instead.

Proof. We prove the contrapositive: if $\sqrt{r}$ is rational, then r is rational.

Assume that $\sqrt{r}$ is rational. Then there exist integers m and n such that:

$$\sqrt{r} = \frac{m}{n}$$

Squaring both sides gives:

$$r = \frac{m^2}{n^2}$$

Since m^2 and n^2 are integers, r is also rational. ■

1.6 Proving an “If and Only If”

Many mathematical theorems assert that two statements are logically equivalent; that is, one holds if and only if the other does. Here is an example that has been known for several thousand years:

Two triangles have the same side lengths if and only if two side lengths and the angle between those sides are the same.

The phrase “if and only if” comes up so often that it is often abbreviated “iff.”

1.6.1 Method #1: Prove Each Statement Implies the Other

The statement “ P IFF Q ” is equivalent to the two statements “ P IMPLIES Q ” and “ Q IMPLIES P .” So you can prove an “iff” by proving *two* implications:

1. Write, “We prove P implies Q and vice-versa.”
2. Write, “First, we show P implies Q .” Do this by one of the methods in Section 1.5.
3. Write, “Now, we show Q implies P .” Again, do this by one of the methods in Section 1.5.

1.6.2 Method #2: Construct a Chain of Iffs

In order to prove that P is true iff Q is true:

1. Write, “We construct a chain of if-and-only-if implications.”
2. Prove P is equivalent to a second statement which is equivalent to a third statement and so forth until you reach Q .

This method sometimes requires more ingenuity than the first, but the result can be a short, elegant proof.

Example

The *standard deviation* of a sequence of values $x_1, x_2, \dots, x_n$ is defined to be:

$$\sqrt{\frac{(x_1 - \mu)^2 + (x_2 - \mu)^2 + \dots + (x_n - \mu)^2}{n}} \quad (1.3)$$

where μ is the *mean* of the values:

$$\mu ::= \frac{x_1 + x_2 + \dots + x_n}{n}$$

Theorem 1.6.1. *The standard deviation of a sequence of values $x_1, \dots, x_n$ is zero iff all the values are equal to the mean.*

For example, the standard deviation of test scores is zero if and only if everyone scored exactly the class average.

Proof. We construct a chain of “iff” implications, starting with the statement that the standard deviation (1.3) is zero:

$$\sqrt{\frac{(x_1 - \mu)^2 + (x_2 - \mu)^2 + \dots + (x_n - \mu)^2}{n}} = 0. \quad (1.4)$$

Now since zero is the only number whose square root is zero, equation (1.4) holds iff

$$(x_1 - \mu)^2 + (x_2 - \mu)^2 + \dots + (x_n - \mu)^2 = 0. \quad (1.5)$$

Now squares of real numbers are always nonnegative, so every term on the left hand side of equation (1.5) is nonnegative. This means that (1.5) holds iff

$$\text{Every term on the left hand side of (1.5) is zero.} \quad (1.6)$$

But a term $(x_i - \mu)^2$ is zero iff $x_i = \mu$, so (1.6) is true iff

Every x_i equals the mean.

■

1.7 Proof by Cases

Breaking a complicated proof into cases and proving each case separately is a useful, common proof strategy. Here’s an amusing example.

Let’s agree that given any two people, either they have met or not. If every pair of people in a group has met, we’ll call the group a *club*. If every pair of people in a group has not met, we’ll call it a group of *strangers*.

Theorem. *Every collection of 6 people includes a club of 3 people or a group of 3 strangers.*

Proof. The proof is by case analysis⁴. Let x denote one of the six people. There are two cases:

1. Among 5 other people besides x , at least 3 have met x .
2. Among the 5 other people, at least 3 have not met x .

Now we have to be sure that at least one of these two cases must hold,⁵ but that’s easy: we’ve split the 5 people into two groups, those who have shaken hands with x and those who have not, so one of the groups must have at least half the people.

Case 1: Suppose that at least 3 people did meet x .

This case splits into two subcases:

Case 1.1: No pair among those people met each other. Then these people are a group of at least 3 strangers. So the Theorem holds in this subcase.

Case 1.2: Some pair among those people have met each other. Then that pair, together with x , form a club of 3 people. So the Theorem holds in this subcase.

This implies that the Theorem holds in Case 1.

Case 2: Suppose that at least 3 people did not meet x .

This case also splits into two subcases:

⁴Describing your approach at the outset helps orient the reader.

⁵Part of a case analysis argument is showing that you’ve covered all the cases. Often this is obvious, because the two cases are of the form “ P ” and “not P .” However, the situation above is not stated quite so simply.

Case 2.1: Every pair among those people met each other. Then these people are a club of at least 3 people. So the Theorem holds in this subcase.

Case 2.2: Some pair among those people have not met each other. Then that pair, together with x , form a group of at least 3 strangers. So the Theorem holds in this subcase.

This implies that the Theorem also holds in Case 2, and therefore holds in all cases. ■

1.8 Proof by Contradiction

In a *proof by contradiction* or *indirect proof*, you show that if a proposition were false, then some false fact would be true. Since a false fact can’t be true, the proposition had better not be false. That is, the proposition really must be true.

Proof by contradiction is *always* a viable approach. However, as the name suggests, indirect proofs can be a little convoluted. So direct proofs are generally preferable as a matter of clarity.

Method: In order to prove a proposition P by contradiction:

1. Write, “We use proof by contradiction.”
2. Write, “Suppose P is false.”
3. Deduce something known to be false (a logical contradiction).
4. Write, “This is a contradiction. Therefore, P must be true.”

Example

Remember that a number is *rational* if it is equal to a ratio of integers. For example, $3.5 = 7/2$ and $0.1111\ldots = 1/9$ are rational numbers. On the other hand, we’ll prove by contradiction that $\sqrt{2}$ is irrational.

Theorem 1.8.1. $\sqrt{2}$ is irrational.

Proof. We use proof by contradiction. Suppose the claim is false; that is, $\sqrt{2}$ is rational. Then we can write $\sqrt{2}$ as a fraction n/d in *lowest terms*.

Squaring both sides gives $2 = n^2/d^2$ and so $2d^2 = n^2$. This implies that n is a multiple of 2. Therefore n^2 must be a multiple of 4. But since $2d^2 = n^2$, we know

$2d^2$ is a multiple of 4 and so d^2 is a multiple of 2. This implies that d is a multiple of 2.

So the numerator and denominator have 2 as a common factor, which contradicts the fact that n/d is in lowest terms. So $\sqrt{2}$ must be irrational. ■

1.9 Good Proofs in Practice

One purpose of a proof is to establish the truth of an assertion with absolute certainty. Mechanically checkable proofs of enormous length or complexity can accomplish this. But humanly intelligible proofs are the only ones that help someone understand the subject. Mathematicians generally agree that important mathematical results can't be fully understood until their proofs are understood. That is why proofs are an important part of the curriculum.

To be understandable and helpful, more is required of a proof than just logical correctness: a good proof must also be clear. Correctness and clarity usually go together; a well-written proof is more likely to be a correct proof, since mistakes are harder to hide.

In practice, the notion of proof is a moving target. Proofs in a professional research journal are generally unintelligible to all but a few experts who know all the terminology and prior results used in the proof. Conversely, proofs in the first weeks of a beginning course like 6.042 would be regarded as tediously long-winded by a professional mathematician. In fact, what we accept as a good proof later in the term will be different from what we consider good proofs in the first couple of weeks of 6.042. But even so, we can offer some general tips on writing good proofs:

State your game plan. A good proof begins by explaining the general line of reasoning, for example, “We use case analysis” or “We argue by contradiction.”

Keep a linear flow. Sometimes proofs are written like mathematical mosaics, with juicy tidbits of independent reasoning sprinkled throughout. This is not good. The steps of an argument should follow one another in an intelligible order.

A proof is an essay, not a calculation. Many students initially write proofs the way they compute integrals. The result is a long sequence of expressions without explanation, making it very hard to follow. This is bad. A good proof usually looks like an essay with some equations thrown in. Use complete sentences.

Avoid excessive symbolism. Your reader is probably good at understanding words,

but much less skilled at reading arcane mathematical symbols. So use words where you reasonably can.

Revise and simplify. Your readers will be grateful.

Introduce notation thoughtfully. Sometimes an argument can be greatly simplified by introducing a variable, devising a special notation, or defining a new term. But do this sparingly since you’re requiring the reader to remember all that new stuff. And remember to actually *define* the meanings of new variables, terms, or notations; don’t just start using them!

Structure long proofs. Long programs are usually broken into a hierarchy of smaller procedures. Long proofs are much the same. Facts needed in your proof that are easily stated, but not readily proved are best pulled out and proved in preliminary lemmas. Also, if you are repeating essentially the same argument over and over, try to capture that argument in a general lemma, which you can cite repeatedly instead.

Be wary of the “obvious.” When familiar or truly obvious facts are needed in a proof, it’s OK to label them as such and to not prove them. But remember that what’s obvious to you, may not be—and typically is not—obvious to your reader.

Most especially, don’t use phrases like “clearly” or “obviously” in an attempt to bully the reader into accepting something you’re having trouble proving. Also, go on the alert whenever you see one of these phrases in someone else’s proof.

Finish. At some point in a proof, you’ll have established all the essential facts you need. Resist the temptation to quit and leave the reader to draw the “obvious” conclusion. Instead, tie everything together yourself and explain why the original claim follows.

Creating a good proof is a lot like creating a beautiful work of art. In fact, mathematicians often refer to really good proofs as being “elegant” or “beautiful.” It takes a practice and experience to write proofs that merit such praises, but to get you started in the right direction, we will provide templates for the most useful proof techniques.

Throughout the text there are also examples of *bogus proofs*—arguments that look like proofs but aren’t. Sometimes a bogus proof can reach false conclusions because of missteps or mistaken assumptions. More subtle bogus proofs reach correct conclusions, but in improper ways, for example by circular reasoning, by

leaping to unjustified conclusions, or by saying that the hard part of “the proof is left to the reader.” Learning to spot the flaws in improper proofs will hone your skills at seeing how each proof step follows logically from prior steps. It will also enable you to spot flaws in your own proofs.

The analogy between good proofs and good programs extends beyond structure. The same rigorous thinking needed for proofs is essential in the design of critical computer systems. When algorithms and protocols only “mostly work” due to reliance on hand-waving arguments, the results can range from problematic to catastrophic. An early example was the Therac 25, a machine that provided radiation therapy to cancer victims, but occasionally killed them with massive overdoses due to a software race condition. A more recent (August 2004) example involved a single faulty command to a computer system used by United and American Airlines that grounded the entire fleet of both companies—and all their passengers!

It is a certainty that we’ll all one day be at the mercy of critical computer systems designed by you and your classmates. So we really hope that you’ll develop the ability to formulate rock-solid logical arguments that a system actually does what you think it does!

Problems for Section 1.1

Class Problems

Problem 1.1.

Identify exactly where the bugs are in each of the following bogus proofs.⁶

(a) **Bogus Claim:** $1/8 > 1/4$.

Bogus proof.

$$\begin{aligned} 3 &> 2 \\ 3 \log_{10}(1/2) &> 2 \log_{10}(1/2) \\ \log_{10}(1/2)^3 &> \log_{10}(1/2)^2 \\ (1/2)^3 &> (1/2)^2, \end{aligned}$$

and the claim now follows by the rules for multiplying fractions. ■

(b) *Bogus proof:* $1¢ = \$0.01 = (\$0.1)^2 = (10¢)^2 = 100¢ = \1 . ■

(c) **Bogus Claim:** If a and b are two equal real numbers, then $a = 0$.

⁶From Stueben, Michael and Diane Sandford. *Twenty Years Before the Blackboard*, Mathematical Association of America, ©1998.

Bogus proof.

$$\begin{aligned} a &= b \\ a^2 &= ab \\ a^2 - b^2 &= ab - b^2 \\ (a - b)(a + b) &= (a - b)b \\ a + b &= b \\ a &= 0. \end{aligned}$$

■

Problem 1.2.

It's a fact that the Arithmetic Mean is at least as large the Geometric Mean, namely,

$$\frac{a + b}{2} \geq \sqrt{ab}$$

for all nonnegative real numbers a and b . But there's something objectionable about the following proof of this fact. What's the objection, and how would you fix it?

Bogus proof.

$$\begin{aligned} \frac{a + b}{2} &\stackrel{?}{\geq} \sqrt{ab}, && \text{so} \\ a + b &\stackrel{?}{\geq} 2\sqrt{ab}, && \text{so} \\ a^2 + 2ab + b^2 &\stackrel{?}{\geq} 4ab, && \text{so} \\ a^2 - 2ab + b^2 &\stackrel{?}{\geq} 0, && \text{so} \\ (a - b)^2 &\geq 0 && \text{which we know is true.} \end{aligned}$$

The last statement is true because $a - b$ is a real number, and the square of a real number is never negative. This proves the claim. ■

Problem 1.3.

Albert announces to his class that he plans to surprise them with a quiz sometime next week.

His students first wonder if the quiz could be on Friday of next week. They reason that it can't: if Albert didn't give the quiz *before* Friday, then by midnight Thursday, they would know the quiz had to be on Friday, and so the quiz wouldn't be a surprise any more.

Next the students wonder whether Albert could give the surprise quiz Thursday. They observe that if the quiz wasn't given *before* Thursday, it would have to be given *on* the Thursday, since they already know it can't be given on Friday. But having figured that out, it wouldn't be a surprise if the quiz was on Thursday either. Similarly, the students reason that the quiz can't be on Wednesday, Tuesday, or Monday. Namely, it's impossible for Albert to give a surprise quiz next week. All the students now relax, having concluded that Albert must have been bluffing.

And since no one expects the quiz, that's why, when Albert gives it on Tuesday next week, it really is a surprise!

What do you think is wrong with the students' reasoning?

Problems for Section 1.5

Homework Problems

Problem 1.4.

Show that $\log_7 n$ is either an integer or irrational, where n is a positive integer. Use whatever familiar facts about integers and primes you need, but explicitly state such facts.

Problems for Section 1.7

Class Problems

Problem 1.5.

If we raise an irrational number to an irrational power, can the result be rational? Show that it can by considering $\sqrt{2}^{\sqrt{2}}$ and arguing by cases.

Homework Problems

Problem 1.6.

For $n = 40$, the value of polynomial $p(n) ::= n^2 + n + 41$ is not prime, as noted in Section 1.1. But we could have predicted based on general principles that no nonconstant polynomial can generate only prime numbers.

In particular, let $q(n)$ be a polynomial with integer coefficients, and let $c ::= q(0)$ be the constant term of q .

- (a) Verify that $q(cm)$ is a multiple of c for all $m \in \mathbb{Z}$.

(b) Show that if q is nonconstant and $c > 1$, then as n ranges over the nonnegative integers, $\mathbb{N}$, there are infinitely many $q(n) \in \mathbb{Z}$ that are not primes.

Hint: You may assume the familiar fact that the magnitude of any nonconstant polynomial, $q(n)$, grows unboundedly as n grows.

(c) Conclude immediately that for every nonconstant polynomial, q , there must be an $n \in \mathbb{N}$ such that $q(n)$ is not prime.

Problems for Section 1.8

Class Problems

Problem 1.7.

Generalize the proof of Theorem 1.8.1 that $\sqrt{2}$ is irrational. For example, how about $\sqrt[3]{2}$?

Problem 1.8.

Here is a different proof that $\sqrt{2}$ is irrational, taken from the American Mathematical Monthly, v.116, #1, Jan. 2009, p.69:

Proof. Suppose for the sake of contradiction that $\sqrt{2}$ is rational, and choose the least integer, $q > 0$, such that $(\sqrt{2} - 1)q$ is a nonnegative integer. Let $q' ::= (\sqrt{2} - 1)q$. Clearly $0 < q' < q$. But an easy computation shows that $(\sqrt{2} - 1)q'$ is a nonnegative integer, contradicting the minimality of q . ■

(a) This proof was written for an audience of college teachers, and at this point it is a little more concise than desirable. Write out a more complete version which includes an explanation of each step.

(b) Now that you have justified the steps in this proof, do you have a preference for one of these proofs over the other? Why? Discuss these questions with your teammates for a few minutes and summarize your team’s answers on your whiteboard.

Problem 1.9.

Here is a generalization of Problem 1.7 that you may not have thought of:

Lemma 1.9.1. *Let the coefficients of the polynomial $a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} + x^n$ be integers. Then any real root of the polynomial is either integral or irrational.*

(a) Explain why Lemma 1.9.1 immediately implies that $\sqrt[m]{k}$ is irrational whenever k is not an m th power of some integer.

(b) Collaborate with your tablemates to write a clear, textbook quality proof of Lemma 1.9.1 on your whiteboard. (Besides clarity and correctness, textbook quality requires good English with proper punctuation. When a real textbook writer does this, it usually takes multiple revisions; if you’re satisfied with your first draft, you’re probably misjudging.) You may find it helpful to appeal to the following:

Lemma 1.9.2. *If a prime, p , is a factor of some power of an integer, then it is a factor of that integer.*

You may assume Lemma 1.9.2 without writing down its proof, but see if you can explain why it is true.

Homework Problems

Problem 1.10.

The fact that there are irrational numbers a, b such that a^b is rational was proved in Problem 1.5. Unfortunately, that proof was *nonconstructive*: it didn’t reveal a specific pair, a, b , with this property. But in fact, it’s easy to do this: let $a ::= \sqrt{2}$ and $b ::= 2 \log_2 3$.

We know $\sqrt{2}$ is irrational, and obviously $a^b = 3$. Finish the proof that this a, b pair works, by showing that $2 \log_2 3$ is irrational.

Exam Problems

Problem 1.11.

Prove that $\log_9 12$ is irrational. *Hint:* Proof by contradiction.



2 The Well Ordering Principle

Every *nonempty* set of *nonnegative integers* has a *smallest* element.

This statement is known as The *Well Ordering Principle*. Do you believe it? Seems sort of obvious, right? But notice how tight it is: it requires a *nonempty* set—it’s false for the empty set which has *no* smallest element because it has no elements at all! And it requires a set of *nonnegative* integers—it’s false for the set of *negative* integers and also false for some sets of nonnegative *rational*s—for example, the set of positive rationals. So, the Well Ordering Principle captures something special about the nonnegative integers.

2.1 Well Ordering Proofs

While the Well Ordering Principle may seem obvious, it’s hard to see offhand why it is useful. But in fact, it provides one of the most important proof rules in discrete mathematics.

In fact, looking back, we took the Well Ordering Principle for granted in proving that $\sqrt{2}$ is irrational. That proof assumed that for any positive integers m and n , the fraction m/n can be written in *lowest terms*, that is, in the form m'/n' where m' and n' are positive integers with no common factors. How do we know this is always possible?

Suppose to the contrary that there were $m, n \in \mathbb{Z}^+$ such that the fraction m/n cannot be written in lowest terms. Now let C be the set of positive integers that are numerators of such fractions. Then $m \in C$, so C is nonempty. Therefore, by Well Ordering, there must be a smallest integer, $m_0 \in C$. So by definition of C , there is an integer $n_0 > 0$ such that

the fraction $\frac{m_0}{n_0}$ cannot be written in lowest terms.

This means that m_0 and n_0 must have a common factor, $p > 1$. But

$$\frac{m_0/p}{n_0/p} = \frac{m_0}{n_0},$$

so any way of expressing the left hand fraction in lowest terms would also work for

m_0/n_0 , which implies

the fraction $\frac{m_0/p}{n_0/p}$ cannot be written in lowest terms either.

So by definition of C , the numerator, m_0/p , is in C . But $m_0/p < m_0$, which contradicts the fact that m_0 is the smallest element of C .

Since the assumption that C is nonempty leads to a contradiction, it follows that C must be empty. That is, that there are no numerators of fractions that can't be written in lowest terms, and hence there are no such fractions at all.

We've been using the Well Ordering Principle on the sly from early on!

2.2 Template for Well Ordering Proofs

More generally, there is a standard way to use Well Ordering to prove that some property, $P(n)$ holds for every nonnegative integer, n . Here is a standard way to organize such a well ordering proof:

To prove that “ $P(n)$ is true for all $n \in \mathbb{N}$ ” using the Well Ordering Principle:

- Define the set, C , of *counterexamples* to P being true. Namely, define¹

$$C ::= \{n \in \mathbb{N} \mid P(n) \text{ is false}\}.$$

- Assume for proof by contradiction that C is nonempty.
- By the Well Ordering Principle, there will be a smallest element, n , in C .
- Reach a contradiction (somehow) —often by showing how to use n to find another member of C that is smaller than n . (This is the open-ended part of the proof task.)
- Conclude that C must be empty, that is, no counterexamples exist. QED

2.2.1 Summing the Integers

Let's use this template to prove

Theorem 2.2.1.

$$1 + 2 + 3 + \cdots + n = n(n + 1)/2 \tag{2.1}$$

for all nonnegative integers, n .

First, we better address of a couple of ambiguous special cases before they trip us up:

- If $n = 1$, then there is only one term in the summation, and so $1 + 2 + 3 + \cdots + n$ is just the term 1. Don’t be misled by the appearance of 2 and 3 and the suggestion that 1 and n are distinct terms!
- If $n \leq 0$, then there are no terms at all in the summation. By convention, the sum in this case is 0.

So while the dots notation is convenient, you have to watch out for these special cases where the notation is misleading! (In fact, whenever you see the dots, you should be on the lookout to be sure you understand the pattern, watching out for the beginning and the end.)

We could have eliminated the need for guessing by rewriting the left side of (2.1) with *summation notation*:

$$\sum_{i=1}^n i \quad \text{or} \quad \sum_{1 \leq i \leq n} i.$$

Both of these expressions denote the sum of all values taken by the expression to the right of the sigma as the variable, i , ranges from 1 to n . Both expressions make it clear what (2.1) means when $n = 1$. The second expression makes it clear that when $n = 0$, there are no terms in the sum, though you still have to know the convention that a sum of no numbers equals 0 (the *product* of no numbers is 1, by the way).

OK, back to the proof:

Proof. By contradiction. Assume that Theorem 2.2.1 is *false*. Then, some nonnegative integers serve as *counterexamples* to it. Let’s collect them in a set:

$$C ::= \{n \in \mathbb{N} \mid 1 + 2 + 3 + \cdots + n \neq \frac{n(n+1)}{2}\}.$$

Assuming there are counterexamples, C is a nonempty set of nonnegative integers. So, by the Well Ordering Principle, C has a minimum element, call it c . That is, among the nonnegative integers, c is the *smallest counterexample* to equation (2.1).

Since c is the smallest counterexample, we know that (2.1) is false for $n = c$ but true for all nonnegative integers $n < c$. But (2.1) is true for $n = 0$, so $c > 0$. This means $c - 1$ is a nonnegative integer, and since it is less than c , equation (2.1) is true for $c - 1$. That is,

$$1 + 2 + 3 + \cdots + (c - 1) = \frac{(c - 1)c}{2}.$$

But then, adding c to both sides we get

$$1 + 2 + 3 + \cdots + (c - 1) + c = \frac{(c - 1)c}{2} + c = \frac{c^2 - c + 2c}{2} = \frac{c(c + 1)}{2},$$

which means that (2.1) does hold for c , after all! This is a contradiction, and we are done. ■

2.3 Factoring into Primes

We’ve previously taken for granted the *Prime Factorization Theorem* that every integer greater than one has a unique² expression as a product of prime numbers. This is another of those familiar mathematical facts which are not really obvious. We’ll prove the uniqueness of prime factorization in a later chapter, but well ordering gives an easy proof that every integer greater than one can be expressed as *some* product of primes.

Theorem 2.3.1. *Every positive integer greater than one can be factored as a product of primes.*

Proof. The proof is by Well Ordering.

Let C be the set of all integers greater than one that cannot be factored as a product of primes. We assume C is not empty and derive a contradiction.

If C is not empty, there is a least element, $n \in C$, by Well Ordering. The n can’t be prime, because a prime by itself is considered a (length one) product of primes and no such products are in C .

So n must be a product of two integers a and b where $1 < a, b < n$. Since a and b are smaller than the smallest element in C , we know that $a, b \notin C$. In other words, a can be written as a product of primes $p_1 p_2 \cdots p_k$ and b as a product of primes $q_1 \cdots q_l$. Therefore, $n = p_1 \cdots p_k q_1 \cdots q_l$ can be written as a product of primes, contradicting the claim that $n \in C$. Our assumption that C is not empty must therefore be false. ■

Problems for Section 2.2

Practice Problems

Problem 2.1.

For practice using the Well Ordering Principle, fill in the template of an easy to

²... unique up to the order in which the prime factors appear

2.3. Factoring into Primes

29

prove fact: every amount of postage that can be assembled using only 10 cent and 15 cent stamps is divisible by 5.

In particular, Let $S(n)$ mean that exactly n cents postage can be assembled using only 10 and 15 cent stamps. Then the proof shows that

$$S(n) \text{ IMPLIES } 5 \mid n, \quad \text{for all nonnegative integers } n. \quad (*)$$

Fill in the missing portions (indicated by “...”) of the following proof of (*).

Let C be the set of *counterexamples* to (*), namely

$$C ::= \{n \mid \dots\}$$

Assume for the purpose of obtaining a contradiction that C is nonempty. Then by the WOP, there is a smallest number, $m \in C$. This m must be positive because

But if $S(m)$ holds and m is positive, then $S(m - 10)$ or $S(m - 15)$ must hold, because

So suppose $S(m - 10)$ holds. Then $5 \mid (m - 10)$, because...

But if $5 \mid (m - 10)$, then obviously $5 \mid m$, contradicting the fact that m is a counterexample.

Next, if $S(m - 15)$ holds, we arrive at a contradiction in the same way.

Since we get a contradiction in both cases, we conclude that...

which proves that (*) holds.

Problem 2.2.

The Fibonacci numbers

$$0, 1, 1, 2, 3, 5, 8, 13, \dots$$

are defined as follows. Let $F(n)$ be the n th Fibonacci number. Then

$$F(0) ::= 0, \quad (2.2)$$

$$F(1) ::= 1, \quad (2.3)$$

$$F(n) ::= F(n - 1) + F(n - 2) \quad \text{for } n \geq 2. \quad (2.4)$$

Indicate exactly which sentence(s) in the following bogus proof contain logical errors? Explain.

False Claim. *Every Fibonacci number is even.*

Bogus proof. Let all the variables n, m, k mentioned below be nonnegative integer valued.

1. The proof is by the WOP.
2. Let $\text{Even}(n)$ mean that $F(n)$ is even.
3. Let C be the set of counterexamples to the assertion that $\text{Even}(n)$ holds for all $n \in \mathbb{N}$, namely,

$$C ::= \{n \in \mathbb{N} \mid \text{NOT}(\text{Even}(n))\}.$$

4. We prove by contradiction that C is empty. So assume that C is not empty.
5. By WOP, there is a least nonnegative integer, $m \in C$,
6. Then $m > 0$, since $F(0) = 0$ is an even number.
7. Since m is the minimum counterexample, $F(k)$ is even for all $k < m$.
8. In particular, $F(m-1)$ and $F(m-2)$ are both even.
9. But by the defining equation (2.4), $F(m)$ equals the sum $F(m-1) + F(m-2)$ of two even numbers, and so it is also even.
10. That is, $\text{Even}(m)$ is true.
11. This contradicts the condition in the definition of m that $\text{NOT}(\text{Even}(m))$ holds.
12. This contradiction implies that C must be empty. Hence, $F(n)$ is even for all $n \in \mathbb{N}$.

■

Problem 2.3.

In Chapter 2, the Well Ordering was used to show that all positive rational numbers can be written in “lowest terms,” that is, as a ratio of positive integers with no common factor prime factor. Below is a different proof which also arrives at this correct conclusion, but this proof is bogus. Identify every step at which the proof makes an unjustified inference.

Bogus proof. Suppose to the contrary that there was positive rational, q , such that q cannot be written in lowest terms. Now let C be the set of such rational numbers that cannot be written in lowest terms. Then $q \in C$, so C is nonempty. So there must be a smallest rational, $q_0 \in C$. So since $q_0/2 < q_0$, it must be possible to express $q_0/2$ in lowest terms, namely,

$$\frac{q_0}{2} = \frac{m}{n} \quad (2.5)$$

for positive integers m, n with no common prime factor. Now we consider two cases:

Case 1: [n is odd]. Then $2m$ and n also have no common prime factor, and therefore

$$q_0 = 2 \cdot \left(\frac{m}{n}\right) = \frac{2m}{n}$$

expresses q_0 in lowest terms, a contradiction.

Case 2: [n is even]. Any common prime factor of m and $n/2$ would also be a common prime factor of m and n . Therefore m and $n/2$ have no common prime factor, and so

$$q_0 = \frac{m}{n/2}$$

expresses q_0 in lowest terms, a contradiction.

Since the assumption that C is nonempty leads to a contradiction, it follows that C is empty—that is, there are no counterexamples. ■

Class Problems

Problem 2.4.

Use the Well Ordering Principle to prove that

$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6}. \quad (2.6)$$

for all nonnegative integers, n .

Problem 2.5.

The proof below uses the Well Ordering Principle to prove that every amount of postage that can be assembled using only 6 cent and 15 cent stamps, is divisible by 3. Let the notation “ $j \mid k$ ” indicate that integer j is a divisor of integer k , and let

$S(n)$ mean that exactly n cents postage can be assembled using only 6 and 15 cent stamps. Then the proof shows that

$$S(n) \text{ IMPLIES } 3 \mid n, \quad \text{for all nonnegative integers } n. \quad (*)$$

Fill in the missing portions (indicated by “...”) of the following proof of (*).

Let C be the set of *counterexamples* to (*), namely³

$$C ::= \{n \mid \dots\}$$

Assume for the purpose of obtaining a contradiction that C is nonempty. Then by the WOP, there is a smallest number, $m \in C$. This m must be positive because...

But if $S(m)$ holds and m is positive, then $S(m - 6)$ or $S(m - 15)$ must hold, because...

So suppose $S(m - 6)$ holds. Then $3 \mid (m - 6)$, because...

But if $3 \mid (m - 6)$, then obviously $3 \mid m$, contradicting the fact that m is a counterexample.

Next, if $S(m - 15)$ holds, we arrive at a contradiction in the same way. Since we get a contradiction in both cases, we conclude that...

which proves that (*) holds.

Homework Problems

Problem 2.6.

Use the Well Ordering Principle to prove that any integer greater than or equal to 8 can be represented as the sum of integer multiples of 3 and 5.

Problem 2.7.

Euler's Conjecture in 1769 was that there are no positive integer solutions to the equation

$$a^4 + b^4 + c^4 = d^4.$$

Integer values for a, b, c, d that do satisfy this equation, were first discovered in 1986. So Euler guessed wrong, but it took more two hundred years to prove it.

³The notation “ $\{n \mid \dots\}$ ” means “the set of elements, n , such that ...”

Now let's consider Lehman's equation, similar to Euler's but with some coefficients:

$$8a^4 + 4b^4 + 2c^4 = d^4 \quad (2.7)$$

Prove that Lehman's equation (2.7) really does not have any positive integer solutions.

Hint: Consider the minimum value of a among all possible solutions to (2.7).

Exam Problems

Problem 2.8.

The (flawed) proof below uses the Well Ordering Principle to prove that every amount of postage that can be paid exactly, using only 10 cent and 15 cent stamps, is divisible by 5. Let $S(n)$ mean that exactly n cents postage can be paid using only 10 and 15 cent stamps. Then the proof shows that

$$S(n) \text{ IMPLIES } 5 \mid n, \quad \text{for all nonnegative integers } n. \quad (*)$$

Fill in the missing portions (indicated by "...") of the following proof of (*), and at the final line point out where the error in the proof is.

Let C be the set of *counterexamples* to (*), namely

$$C ::= \{n \mid S(n) \text{ and } \text{NOT}(5 \mid n)\}$$

Assume for the purpose of obtaining a contradiction that C is nonempty. Then by the WOP, there is a smallest number, $m \in C$. Then $S(m - 10)$ or $S(m - 15)$ must hold, because the m cents postage is made from 10 and 15 cent stamps, so we remove one.

So suppose $S(m - 10)$ holds. Then $5 \mid (m - 10)$, because...

But if $5 \mid (m - 10)$, then $5 \mid m$, because...

contradicting the fact that m is a counterexample.

Next suppose $S(m - 15)$ holds. Then the proof for $m - 10$ carries over directly for $m - 15$ to yield a contradiction in this case as well. Since we get a contradiction in both cases, we conclude that C must be empty. That is, there are no counterexamples to (*), which proves that (*) holds.

What was wrong/missing in the argument? Your answer should fit in the line below.

Problem 2.9.

We'll prove that for every positive integer, n , the sum of the first n odd numbers is n^2 , that is,

$$\sum_{i=1}^n (2(i-1) + 1) = n^2, \quad (2.8)$$

Assume to the contrary that equation (2.8) failed for some positive integer, n . Let m be the least such number.

- (a) Why must there be such an m ?
- (b) Explain why $m \geq 2$.
- (c) Explain why part (b) implies that

$$\sum_{i=1}^{m-1} (2(i-1) + 1) = (m-1)^2. \quad (2.9)$$

- (d) What term should be added to the left hand side of (2.9) so the result equals

$$\sum_{i=1}^m (2(i-1) + 1)?$$

- (e) Now prove that

$$\sum_{i=1}^m (2(i-1) + 1) = m^2. \quad (2.10)$$

- (f) Conclude that equation (2.8) holds for all positive integers, n .

3 Logical Formulas

It is amazing that people manage to cope with all the ambiguities in the English language. Here are some sentences that illustrate the issue:

- “You may have cake, or you may have ice cream.”
- “If pigs can fly, then you can understand the Chebyshev bound.”
- “If you can solve any problem we come up with, then you get an *A* for the course.”
- “Every American has a dream.”

What *precisely* do these sentences mean? Can you have both cake and ice cream or must you choose just one dessert? Pigs can’t fly, so does the second sentence say anything about your understanding the Chebyshev bound? If you can solve some problems we come up with, can you get an *A* for the course? And if you can’t solve a single one of the problems, does it mean you can’t get an *A*? Finally, does the last sentence imply that all Americans have the same dream—say of owning a house—or might different Americans may have different dreams—say, Eric dreams of designing a “killer” software application, Tom of being a tennis champion, Albert of being able to sing?

Some uncertainty is tolerable in normal conversation. But when we need to formulate ideas precisely—as in mathematics and programming—the ambiguities inherent in everyday language can be a real problem. We can’t hope to make an exact argument if we’re not sure exactly what the statements mean. So before we start into mathematics, we need to investigate the problem of how to talk about mathematics.

To get around the ambiguity of English, mathematicians have devised a special language for talking about logical relationships. This language mostly uses ordinary English words and phrases such as “or,” “implies,” and “for all.” But mathematicians give these words precise and unambiguous definitions.

Surprisingly, in the midst of learning the language of logic, we’ll come across the most important open problem in computer science—a problem whose solution could change the world.

3.1 Propositions from Propositions

In English, we can modify, combine, and relate propositions with words such as “not,” “and,” “or,” “implies,” and “if-then.” For example, we can combine three propositions into one like this:

If all humans are mortal **and** all Greeks are human, **then** all Greeks are mortal.

For the next while, we won’t be much concerned with the internals of propositions—whether they involve mathematics or Greek mortality—but rather with how propositions are combined and related. So we’ll frequently use variables such as P and Q in place of specific propositions such as “All humans are mortal” and “ $2 + 3 = 5$.” The understanding is that these *propositional variables*, like propositions, can take on only the values **T** (true) and **F** (false). Propositional variables are also called *Boolean variables* after their inventor, the nineteenth century mathematician George—you guessed it—Boole.

3.1.1 NOT, AND, and OR

Mathematicians use the words NOT, AND, and OR for operations that change or combine propositions. The precise mathematical meaning of these special words can be specified by *truth tables*. For example, if P is a proposition, then so is “NOT(P),” and the truth value of the proposition “NOT(P)” is determined by the truth value of P according to the following truth table:

P	NOT(P)
T	F
F	T

The first row of the table indicates that when proposition P is true, the proposition “NOT(P)” is false. The second line indicates that when P is false, “NOT(P)” is true. This is probably what you would expect.

In general, a truth table indicates the true/false value of a proposition for each possible set of truth values for the variables. For example, the truth table for the proposition “ P AND Q ” has four lines, since there are four settings of truth values for the two variables:

P	Q	P AND Q
T	T	T
T	F	F
F	T	F
F	F	F

According to this table, the proposition “ P AND Q ” is true only when P and Q are both true. This is probably the way you ordinarily think about the word “and.”

There is a subtlety in the truth table for “ P OR Q ”:

P	Q	P OR Q
T	T	T
T	F	T
F	T	T
F	F	F

The first row of this table says that “ P OR Q ” is true even if *both* P and Q are true. This isn’t always the intended meaning of “or” in everyday speech, but this is the standard definition in mathematical writing. So if a mathematician says, “You may have cake, or you may have ice cream,” he means that you *could* have both.

If you want to exclude the possibility of both having and eating, you should combine them with the *exclusive-or* operation, XOR:

P	Q	P XOR Q
T	T	F
T	F	T
F	T	T
F	F	F

3.1.2 IMPLIES

The combining operation with the least intuitive technical meaning is “implies.” Here is its truth table, with the lines labeled so we can refer to them later.

P	Q	P IMPLIES Q	
T	T	T	(tt)
T	F	F	(tf)
F	T	T	(ft)
F	F	T	(ff)

Let’s experiment with this definition. For example, is the following proposition true or false?

“If Goldbach’s Conjecture is true, then $x^2 \geq 0$ for every real number x .”

Now, we already mentioned that no one knows whether Goldbach’s Conjecture, Proposition 1.1.7, is true or false. But that doesn’t prevent you from answering the question! This proposition has the form P IMPLIES Q where the *hypothesis*, P , is “Goldbach’s Conjecture is true” and the *conclusion*, Q , is “ $x^2 \geq 0$ for every

real number x .” Since the conclusion is definitely true, we’re on either line (tt) or line (ft) of the truth table. Either way, the proposition as a whole is *true*!

One of our original examples demonstrates an even stranger side of implications.

“If pigs fly, then you can understand the Chebyshev bound.”

Don’t take this as an insult; we just need to figure out whether this proposition is true or false. Curiously, the answer has *nothing* to do with whether or not you can understand the Chebyshev bound. Pigs do not fly, so we’re on either line (ft) or line (ff) of the truth table. In both cases, the proposition is *true*!

In contrast, here’s an example of a false implication:

“If the moon shines white, then the moon is made of white cheddar.”

Yes, the moon shines white. But, no, the moon is not made of white cheddar cheese. So we’re on line (tf) of the truth table, and the proposition is false.

The truth table for implications can be summarized in words as follows:

An implication is true exactly when the if-part is false or the then-part is true.

This sentence is worth remembering; a large fraction of all mathematical statements are of the if-then form!

3.1.3 If and Only If

Mathematicians commonly join propositions in one additional way that doesn’t arise in ordinary speech. The proposition “ P if and only if Q ” asserts that P and Q have the same truth value, that is, either both are true or both are false.

P	Q	P IFF Q
T	T	T
T	F	F
F	T	F
F	F	T

For example, the following if-and-only-if statement is true for every real number x :

$$x^2 - 4 \geq 0 \text{ IFF } |x| \geq 2.$$

For some values of x , *both* inequalities are true. For other values of x , *neither* inequality is true. In every case, however, the IFF proposition as a whole is true.

3.2 Propositional Logic in Computer Programs

Propositions and logical connectives arise all the time in computer programs. For example, consider the following snippet, which could be either C, C++, or Java:

```
if ( x > 0 || (x <= 0 && y > 100) )
    :
    (further instructions)
```

Java uses The symbol `||` for “OR,” and the symbol `&&` for “AND.” The *further instructions* are carried out only if the proposition following the word `if` is true. On closer inspection, this big expression is built from two simpler propositions. Let A be the proposition that $x > 0$, and let B be the proposition that $y > 100$. Then we can rewrite the condition as

$$A \text{ OR } (\text{NOT}(A) \text{ AND } B). \quad (3.1)$$

3.2.1 Truth Table Calculation

A truth table calculation reveals that the more complicated expression 3.1 always has the same truth value as

$$A \text{ OR } B. \quad (3.2)$$

Namely, we begin with a table with just the truth values of A and B :

A	B	$A \text{ OR } (\text{NOT}(A) \text{ AND } B)$	$A \text{ OR } B$
T	T		
T	F		
F	T		
F	F		

These values are enough to fill in two more columns:

A	B	$A \text{ OR } (\text{NOT}(A) \text{ AND } B)$	$A \text{ OR } B$
T	T	F	T
T	F	F	T
F	T	T	T
F	F	T	F

Now we have the values needed to fill in the AND column:

<i>A</i>	<i>B</i>	<i>A</i>	OR	(NOT(<i>A</i>))	AND	<i>B</i>)	<i>A</i> OR <i>B</i>
T	T			F	F		T
T	F			F	F		T
F	T			T	T		T
F	F			T	F		F

and this provides the values needed to fill in the remaining column for the first OR:

<i>A</i>	<i>B</i>	<i>A</i>	OR	(NOT(<i>A</i>))	AND	<i>B</i>)	<i>A</i> OR <i>B</i>
T	T	T		F	F		T
T	F	T		F	F		T
F	T	T		T	T		T
F	F	F		T	F		F

Expression whose truth values always match are called *equivalent*. Since the (highlighted) columns of truth values of the two expressions are the same, they are equivalent. So we can simplify the code snippet without changing the program’s behavior by replacing the complicated expression with an equivalent simpler one:

```
if ( x > 0 || y > 100 )
    :
    (further instructions)
```

The equivalence of (3.1) and (3.2) can also be confirmed reasoning by cases:

A is **T**. An expression of the form (**T** OR anything) is equivalent to **T**. Since *A* is **T** both (3.1) and (3.2) in this case are of this form, so they have the same truth value, namely, **T**.

A is **F**. An expression of the form (**F** OR anything) will have same truth value as anything. Since *A* is **F**, (3.2) has the same truth value as *B*.

An expression of the form (**T** AND anything) is equivalent to anything, as is any expression of the form **F** OR anything. So in this case *A* OR (NOT(*A*) AND *B*) is equivalent to (NOT(*A*) AND *B*), which in turn is equivalent to *B*.

Therefore both (3.1) and (3.2) will have the same truth value in this case, namely, the value of *B*.

Simplifying logical expressions has real practical importance in computer science. Expression simplification in programs like the one above can make a program easier to read and understand, and can also make it faster since fewer operations

are needed. In hardware, simplifying expressions can decrease the number of logic gates on a chip. That’s because digital circuits can be described by logical formulas (see Problems 3.5 and 3.6), and minimizing the logical formulas corresponds to reducing the number of gates in the circuit. The payoff of gate minimization is potentially enormous: a chip with fewer gates is smaller, consumes less power, has a lower defect rate, and is cheaper to manufacture.

3.2.2 Cryptic Notation

Java uses symbols like “&&” and “||” in place of AND and OR. Circuit designers use “.” and “+,” and actually refer to AND as a product and OR as a sum. Mathematicians use still other symbols given in the table below.

English	Symbolic Notation
NOT(P)	$\neg P$ (alternatively, $\overline{P}$)
P AND Q	$P \wedge Q$
P OR Q	$P \vee Q$
P IMPLIES Q	$P \longrightarrow Q$
if P then Q	$P \longrightarrow Q$
P IFF Q	$P \longleftrightarrow Q$
P XOR Q	$P \oplus Q$

For example, using this notation, “If P AND NOT(Q), then R ” would be written:

$$(P \wedge \overline{Q}) \longrightarrow R.$$

The mathematical notation is concise but cryptic. Words such as “AND” and “OR” are easier to remember and won’t get confused with operations on numbers. We will often use $\overline{P}$ as an abbreviation for NOT(P), but aside from that, we mostly stick to the words—except when formulas would otherwise run off the page.

3.3 Equivalence and Validity

3.3.1 Implications and Contrapositives

Do these two sentences say the same thing?

If I am hungry, then I am grumpy.
If I am not grumpy, then I am not hungry.

We can settle the issue by recasting both sentences in terms of propositional logic. Let P be the proposition “I am hungry,” and Q be “I am grumpy.” The first sentence says “ P IMPLIES Q ” and the second says “ $\text{NOT}(Q)$ IMPLIES $\text{NOT}(P)$.” Once more, we can compare these two statements in a truth table:

P	Q	$(P \text{ IMPLIES } Q)$	$(\text{NOT}(Q) \text{ IMPLIES } \text{NOT}(P))$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Sure enough, the highlighted columns showing the truth values of these two statements are the same. A statement of the form “ $(\text{NOT } Q) \text{ IMPLIES } (\text{NOT } P)$ ” is called the *contrapositive* of the implication “ $P \text{ IMPLIES } Q$.” The truth table shows that an implication and its contrapositive are equivalent—they are just different ways of saying the same thing.

In contrast, the *converse* of “ $P \text{ IMPLIES } Q$ ” is the statement “ $Q \text{ IMPLIES } P$.” In terms of our example, the converse is:

If I am grumpy, then I am hungry.

This sounds like a rather different contention, and a truth table confirms this suspicion:

P	Q	$P \text{ IMPLIES } Q$	$Q \text{ IMPLIES } P$
T	T	T	T
T	F	F	T
F	T	T	F
F	F	T	T

Now the highlighted columns differ in the second and third row, confirming that an implication is generally *not* equivalent to its converse.

One final relationship: an implication and its converse together are equivalent to an iff statement, specifically, to these two statements together. For example,

If I am grumpy then I am hungry, and if I am hungry then I am grumpy.

are equivalent to the single statement:

I am grumpy iff I am hungry.

Once again, we can verify this with a truth table.

P	Q	$(P \text{ IMPLIES } Q)$	AND	$(Q \text{ IMPLIES } P)$	$P \text{ IFF } Q$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	T	F	F	F
F	F	T	T	T	T

The fourth column giving the truth values of

$$(P \text{ IMPLIES } Q) \text{ AND } (Q \text{ IMPLIES } P)$$

is the same as the sixth column giving the truth values of $P \text{ IFF } Q$, which confirms that the AND of the implications is equivalent to the IFF statement.

3.3.2 Validity and Satisfiability

A *valid* formula is one which is always true. The simplest example is

$$P \text{ OR NOT}(P).$$

You can think about valid formulas as capturing fundamental logical truths. For example, a property of implication that we take for granted is that if one statement implies a second one, and the second one implies a third, then the first implies the third. The following valid formula confirms the truth of this property of implication.

$$[(P \text{ IMPLIES } Q) \text{ AND } (Q \text{ IMPLIES } R)] \text{ IMPLIES } (P \text{ IMPLIES } R).$$

Equivalence of formulas is really a special case of validity. Namely, statements F and G are equivalent iff the statement $(F \text{ IFF } G)$ is valid. For example, the equivalence of the expressions 3.2 and 3.1 means that

$$(A \text{ OR } B) \text{ IFF } (A \text{ OR } (\text{NOT}(A) \text{ AND } B))$$

is valid. Of course validity can also be viewed as an aspect of equivalence. Namely, a formula is valid iff it is equivalent to **T**.

A *satisfiable* formula is one which can sometimes be true. One way satisfiability comes up is when there are a collection of system specifications. The job of the system designer is to come up with a system that follows all the specs. This means that the AND of all the specs had better be satisfiable or the system will be impossible (see Problem 3.10).

There is also a close relationship between validity and satisfiability, namely, a statement P is valid iff its negation $\text{NOT}(P)$ is *not* satisfiable.

3.4 The Algebra of Propositions

3.4.1 Propositions in Normal Form

Every propositional formula is equivalent to a “sum-of-products” or *disjunctive form*. More precisely, a disjunctive form is simply an OR of AND-terms, where each AND-term is a AND of variables or negations of variables, for example,

$$(A \text{ AND } B) \text{ OR } (A \text{ AND } C). \quad (3.3)$$

You can read a disjunctive form for any propositional formula directly from its truth table. For example, the formula

$$A \text{ AND } (B \text{ OR } C) \quad (3.4)$$

has truth table:

A	B	C	$A \text{ AND } (B \text{ OR } C)$
T	T	T	T
T	T	F	T
T	F	T	T
T	F	F	F
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

The formula (3.4) is true in the first row when A , B , and C are all true, that is, where $A \text{ AND } B \text{ AND } C$ is true. It is also true in the second row where $A \text{ AND } B \text{ AND } \overline{C}$ is true, and in the third row when $A \text{ AND } \overline{B} \text{ AND } C$ is true, and that’s all. So (3.4) is true exactly when

$$(A \text{ AND } B \text{ AND } C) \text{ OR } (A \text{ AND } B \text{ AND } \overline{C}) \text{ OR } (A \text{ AND } \overline{B} \text{ AND } C) \quad (3.5)$$

is true. So (3.4) and (3.5) are equivalent.

The expression (3.5) is a disjunctive form where each AND-term is an AND of *every one* of the variables or their negations in turn. An expression of this form is called a *disjunctive normal form (DNF)*. A DNF formula can often be simplified into a smaller disjunctive form. For example, the DNF (3.5) further simplifies to the equivalent disjunctive form (3.3) above.

Incidentally, this equivalence of $A \text{ AND } (B \text{ OR } C)$ and $(A \text{ AND } B) \text{ OR } (A \text{ AND } C)$ is called the *distributive law* of AND over OR because of its obvious resemblance to the distributivity of multiplication over addition for numbers.

Applying the same reasoning to the **F** entries of a truth table yields a *conjunctive form* for any formula, namely a **AND** of **OR**-terms, where the **OR**-terms are **OR**’s only of variables or their negations. For example, formula (3.4) is false in the fourth row of its truth table (3.4.1) where A is **T**, B is **F** and C is **F**. But this is exactly the one row where $(\overline{A} \text{ OR } B \text{ OR } C)$ is **F**! Likewise, the (3.4) is false in the fifth row which is exactly where $(A \text{ OR } \overline{B} \text{ OR } \overline{C})$ is **F**. This means that (3.4) will be **F** whenever the **AND** of these two **OR**-terms is false. Continuing in this way with the **OR**-terms corresponding to the remaining three rows where (3.4) is false, we get a *conjunctive normal form* (CNF) that is equivalent to (3.4), namely,

$$(\overline{A} \text{ OR } B \text{ OR } C) \text{ AND } (A \text{ OR } \overline{B} \text{ OR } \overline{C}) \text{ AND } (A \text{ OR } \overline{B} \text{ OR } C) \text{ AND } (A \text{ OR } B \text{ OR } \overline{C}) \text{ AND } (A \text{ OR } B \text{ OR } C)$$

The methods above can obviously be applied to any truth table, which implies

Theorem 3.4.1. *Every propositional formula is equivalent to both a disjunctive normal form and a conjunctive normal form.*

3.4.2 Proving Equivalences

A check of equivalence or validity by truth table runs out of steam pretty quickly: a proposition with n variables has a truth table with 2^n lines, so the effort required to check a proposition grows exponentially with the number of variables. For a proposition with just 30 variables, that’s already over a billion lines to check!

An alternative approach that *sometimes* helps is to use algebra to prove equivalence. A lot of different operators may appear in a propositional formula, so a useful first step is to get rid of all but three: **AND**, **OR**, and **NOT**. This is easy because each of the operators is equivalent to a simple formula using only these three. For example, $A \text{ IMPLIES } B$ is equivalent to $\text{NOT}(A) \text{ OR } B$. **AND**, **OR**, **NOT** formulas for the remaining operators are left to Problem 3.11.

We list below a bunch of equivalence axioms with the symbol “ $\longleftrightarrow$ ” between equivalent formulas. These axioms are important because they are all that’s needed to prove every possible equivalence. We’ll start with some equivalences for **AND**’s that look like the familiar ones for multiplication of numbers:

$$A \text{ AND } B \longleftrightarrow B \text{ AND } A \quad \text{commutativity of AND} \quad (3.6)$$

$$(A \text{ AND } B) \text{ AND } C \longleftrightarrow A \text{ AND } (B \text{ AND } C) \quad \text{associativity of AND} \quad (3.7)$$

$$\mathbf{T} \text{ AND } A \longleftrightarrow A \quad \text{identity for AND} \quad (3.8)$$

$$\mathbf{F} \text{ AND } A \longleftrightarrow \mathbf{F} \quad \text{zero for AND} \quad (3.9)$$

Three axioms that don't directly correspond to number properties are

$$A \text{ AND } A \longleftrightarrow A \quad \text{idempotence for AND} \quad (3.10)$$

$$A \text{ AND } \overline{A} \longleftrightarrow \mathbf{F} \quad \text{contradiction for AND} \quad (3.11)$$

$$\text{NOT}(\overline{A}) \longleftrightarrow A \quad \text{double negation} \quad (3.12)$$

$$(3.13)$$

It is associativity (3.7) that justifies writing $A \text{ AND } B \text{ AND } C$ without specifying whether it is parenthesized as $A \text{ AND } (B \text{ AND } C)$ or $(A \text{ AND } B) \text{ AND } C$. That's because both ways of inserting parentheses yield equivalent formulas.

There are a corresponding set of equivalences for OR which we won't bother to list, except for the OR rule corresponding to contradiction for AND:

$$A \text{ OR } \overline{A} \longleftrightarrow \mathbf{T} \quad \text{validity for OR} \quad (3.14)$$

$$(3.15)$$

There is also a familiar rule connecting AND and OR:

$$A \text{ AND } (B \text{ OR } C) \longleftrightarrow (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \quad \text{distributivity of AND over OR} \quad (3.16)$$

Finally, there are *DeMorgan's Laws* which explain how to distribute NOT's over AND's and OR's:

$$\text{NOT}(A \text{ AND } B) \longleftrightarrow \overline{A} \text{ OR } \overline{B} \quad \text{DeMorgan for AND} \quad (3.17)$$

$$\text{NOT}(A \text{ OR } B) \longleftrightarrow \overline{A} \text{ AND } \overline{B} \quad \text{DeMorgan for OR} \quad (3.18)$$

All these axioms can be verified easily with truth tables.

These axioms are all that's needed to convert any formula to a disjunctive normal form. We can illustrate how they work by applying them to turn the negation of formula (3.4), namely,

$$\text{NOT}((A \text{ AND } B) \text{ OR } (A \text{ AND } C)). \quad (3.19)$$

into disjunctive normal form.

We start by applying DeMorgan's Law for OR to (3.19) in order to move the NOT deeper into the formula. This gives

$$\text{NOT}(A \text{ AND } B) \text{ AND } \text{NOT}(A \text{ AND } C).$$

Now applying Demorgan's Law for AND to the two innermost AND-terms, gives

$$(\overline{A} \text{ OR } \overline{B}) \text{ AND } (\overline{A} \text{ OR } \overline{C}). \quad (3.20)$$

At this point NOT only applies to variables, and we won’t need Demorgan’s Laws any further.

Now we will repeatedly apply the distributivity of AND over OR to turn (3.20) into a disjunctive form. To start, we’ll distribute $(\bar{A} \text{ OR } \bar{B})$ over OR to get

$$((\bar{A} \text{ OR } \bar{B}) \text{ AND } \bar{A}) \text{ OR } ((\bar{A} \text{ OR } \bar{B}) \text{ AND } \bar{C}).$$

Using distributivity over both AND’s we get

$$((\bar{A} \text{ AND } \bar{A}) \text{ OR } (\bar{B} \text{ AND } \bar{A})) \text{ OR } ((\bar{A} \text{ AND } \bar{C}) \text{ OR } (\bar{B} \text{ AND } \bar{C})).$$

By the way, we’ve implicitly used commutativity (3.6) here to justify distributing over an AND from the right. Now applying idempotence to remove the duplicate occurrence of $\bar{A}$ we get

$$(\bar{A} \text{ OR } (\bar{B} \text{ AND } \bar{A})) \text{ OR } ((\bar{A} \text{ AND } \bar{C}) \text{ OR } (\bar{B} \text{ AND } \bar{C})).$$

Associativity now allows dropping the parentheses around the terms being OR’d to yield the following disjunctive form for (3.19):

$$\bar{A} \text{ OR } (\bar{B} \text{ AND } \bar{A}) \text{ OR } (\bar{A} \text{ AND } \bar{C}) \text{ OR } (\bar{B} \text{ AND } \bar{C}). \quad (3.21)$$

The last step is to turn each of these AND-terms into a disjunctive normal form with all three variables A , B , and C . We’ll illustrate how to do this for the second AND-term $(\bar{B} \text{ AND } \bar{A})$. This term needs to mention C to be in normal form. To introduce C , we use validity for OR and identity for AND to conclude that

$$(\bar{B} \text{ AND } \bar{A}) \longleftrightarrow (\bar{B} \text{ AND } \bar{A}) \text{ AND } (C \text{ OR } \bar{C}).$$

Now distributing $(\bar{B} \text{ AND } \bar{A})$ over the OR yields the disjunctive normal form

$$(\bar{B} \text{ AND } \bar{A} \text{ AND } C) \text{ OR } (\bar{B} \text{ AND } \bar{A} \text{ AND } \bar{C}).$$

Doing the same thing to the other AND-terms in (3.21) finally gives a disjunctive normal form for (3.4):

$$\begin{aligned} &(\bar{A} \text{ AND } B \text{ AND } C) \text{ OR } (\bar{A} \text{ AND } B \text{ AND } \bar{C}) \text{ OR} \\ &(\bar{A} \text{ AND } \bar{B} \text{ AND } C) \text{ OR } (\bar{A} \text{ AND } \bar{B} \text{ AND } \bar{C}) \text{ OR} \\ &(\bar{B} \text{ AND } \bar{A} \text{ AND } C) \text{ OR } (\bar{B} \text{ AND } \bar{A} \text{ AND } \bar{C}) \text{ OR} \\ &(\bar{A} \text{ AND } \bar{C} \text{ AND } B) \text{ OR } (\bar{A} \text{ AND } \bar{C} \text{ AND } \bar{B}) \text{ OR} \\ &(\bar{B} \text{ AND } \bar{C} \text{ AND } A) \text{ OR } (\bar{B} \text{ AND } \bar{C} \text{ AND } \bar{A}). \end{aligned}$$

Using commutativity to sort the term and OR-idempotence to remove duplicates, finally yields a unique sorted DNF:

$$\begin{aligned} & (A \text{ AND } \overline{B} \text{ AND } \overline{C}) \text{ OR} \\ & (\overline{A} \text{ AND } B \text{ AND } C) \text{ OR} \\ & (\overline{A} \text{ AND } B \text{ AND } \overline{C}) \text{ OR} \\ & (\overline{A} \text{ AND } \overline{B} \text{ AND } C) \text{ OR} \\ & (\overline{A} \text{ AND } \overline{B} \text{ AND } \overline{C}). \end{aligned}$$

This example illustrates a strategy for applying these equivalences to convert any formula into disjunctive normal form, and conversion to conjunctive normal form works similarly, which explains:

Theorem 3.4.2. *Any propositional formula can be transformed into disjunctive normal form or a conjunctive normal form using the equivalences listed above.*

What has this got to do with equivalence? That’s easy: to prove that two formulas are equivalent, convert them both to disjunctive normal form over the set of variables that appear in the terms. Then use commutativity to sort the variables and AND-terms so they all appear in some standard order. We claim the formulas are equivalent iff they have the same sorted disjunctive normal form. This is obvious if they do have the same disjunctive normal form. But conversely, the way we read off a disjunctive normal form from a truth table shows that two different sorted DNF’s over the same set of variables correspond to different truth tables and hence to inequivalent formulas. This proves

Theorem 3.4.3 (Completeness of the propositional equivalence axioms). *Two propositional formula are equivalent iff they can be proved equivalent using the equivalence axioms listed above.*

The benefit of the axioms is that they leave room for ingeniously applying them to prove equivalences with less effort than the truth table method. Theorem 3.4.3 then adds the reassurance that the axioms are guaranteed to prove every equivalence, which is a great punchline for this section. But we don’t want to mislead you: it’s important to realize that using the strategy we gave for applying the axioms involves essentially the same effort it would take to construct truth tables, and there is no guarantee that applying the axioms will generally be any easier than using truth tables.

3.5 The SAT Problem

Determining whether or not a more complicated proposition is satisfiable is not so easy. How about this one?

$$(P \text{ OR } Q \text{ OR } R) \text{ AND } (\overline{P} \text{ OR } \overline{Q}) \text{ AND } (\overline{P} \text{ OR } \overline{R}) \text{ AND } (\overline{R} \text{ OR } \overline{Q})$$

The general problem of deciding whether a proposition is satisfiable is called *SAT*. One approach to SAT is to construct a truth table and check whether or not a **T** ever appears, but as for validity, this approach quickly bogs down for formulas with many variables because truth tables grow exponentially with the number of variables.

Is there a more efficient solution to SAT? In particular, is there some, presumably very ingenious, procedure that determines in a number of steps that grows *polynomially*—like n^2 or n^{14} —instead of exponentially, whether any given proposition is satisfiable or not? No one knows. And an awful lot hangs on the answer. It turns out that an efficient solution to SAT would immediately imply efficient solutions to many, many other important problems involving packing, scheduling, routing, and circuit verification, among other things. This would be wonderful, but there would also be worldwide chaos. Decrypting coded messages would also become an easy task, so online financial transactions would be insecure and secret communications could be read by everyone. Why this would happen is explained in Section 8.9.

Of course, the situation is the same for validity checking, since you can check for validity by checking for satisfiability of negated formula. This also explains why the simplification of formulas mentioned in Section 3.2 would be hard—validity testing is a special case of determining if a formula simplifies to **T**.

Recently there has been exciting progress on *SAT-solvers* for practical applications like digital circuit verification. These programs find satisfying assignments with amazing efficiency even for formulas with millions of variables. Unfortunately, it’s hard to predict which kind of formulas are amenable to SAT-solver methods, and for formulas that are *unsatisfiable*, SAT-solvers generally get nowhere.

So no one has a good idea how to solve SAT in polynomial time, or how to prove that it can’t be done—researchers are completely stuck. The problem of determining whether or not SAT has a polynomial time solution is known as the “**P** vs. **NP**” problem.¹ It is the outstanding unanswered question in theoretical

¹**P** stands for problems whose instances can be solved in time that grows polynomially with the size of the instance. **NP** stands for **n**ondeterministic **p**olynomial time, but we’ll leave an explanation of what that is to texts on the theory of computational complexity.

computer science. It is also one of the seven [Millenium Problems](#): the Clay Institute will award you \$1,000,000 if you solve the **P** vs. **NP** problem.

3.6 Predicate Formulas

3.6.1 Quantifiers

The “for all” notation, $\forall$, already made an early appearance in Section 1.1. For example, the predicate

$$“x^2 \geq 0”$$

is always true when x is a real number. That is,

$$\forall x \in \mathbb{R}. x^2 \geq 0$$

is a true statement. On the other hand, the predicate

$$“5x^2 - 7 = 0”$$

is only sometimes true; specifically, when $x = \pm\sqrt{7/5}$. There is a “there exists” notation, $\exists$, to indicate that a predicate is true for at least one, but not necessarily all objects. So

$$\exists x \in \mathbb{R}. 5x^2 - 7 = 0$$

is true, while

$$\forall x \in \mathbb{R}. 5x^2 - 7 = 0$$

is not true.

There are several ways to express the notions of “always true” and “sometimes true” in English. The table below gives some general formats on the left and specific examples using those formats on the right. You can expect to see such phrases hundreds of times in mathematical writing!

Always True

For all $x \in D$, $P(x)$ is true.

$P(x)$ is true for every x in the set, D .

For all $x \in \mathbb{R}$, $x^2 \geq 0$.

$x^2 \geq 0$ for every $x \in \mathbb{R}$.

Sometimes True

There is an $x \in D$ such that $P(x)$ is true.

$P(x)$ is true for some x in the set, D .

$P(x)$ is true for at least one $x \in D$.

There is an $x \in \mathbb{R}$ such that $5x^2 - 7 = 0$.

$5x^2 - 7 = 0$ for some $x \in \mathbb{R}$.

$5x^2 - 7 = 0$ for at least one $x \in \mathbb{R}$.

All these sentences quantify how often the predicate is true. Specifically, an assertion that a predicate is always true is called a *universal* quantification, and an assertion that a predicate is sometimes true is an *existential* quantification. Sometimes the English sentences are unclear with respect to quantification:

If you can solve any problem we come up with,
then you get an A for the course. (3.22)

The phrase “you can solve any problem we can come up with” could reasonably be interpreted as either a universal or existential quantification:

you can solve *every* problem we come up with, (3.23)

or maybe

you can solve *at least one* problem we come up with. (3.24)

To be precise, let Probs be the set of problems we come up with, $\text{Solves}(x)$ be the predicate “You can solve problem x ,” and G be the proposition, “You get an A for the course.” Then the two different interpretations of (3.22) can be written as follows:

$(\forall x \in \text{Probs. Solves}(x)) \text{ IMPLIES } G,$

for (3.23), and

$(\exists x \in \text{Probs. Solves}(x)) \text{ IMPLIES } G.$

for (3.24).

3.6.2 Mixing Quantifiers

Many mathematical statements involve several quantifiers. For example, we already described

Goldbach’s Conjecture 1.1.7: Every even integer greater than 2 is the sum of two primes.

Let’s write this out in more detail to be precise about the quantification:

For every even integer n greater than 2, there exist primes p and q such that $n = p + q$.

Let Evens be the set of even integers greater than 2, and let Primes be the set of primes. Then we can write Goldbach’s Conjecture in logic notation as follows:

$$\underbrace{\forall n \in \text{Evens.}}_{\text{for every even integer } n > 2} \underbrace{\exists p \in \text{Primes. } \exists q \in \text{Primes.}}_{\text{there exist primes } p \text{ and } q \text{ such that}} n = p + q.$$

3.6.3 Order of Quantifiers

Swapping the order of different kinds of quantifiers (existential or universal) usually changes the meaning of a proposition. For example, let’s return to one of our initial, confusing statements:

“Every American has a dream.”

This sentence is ambiguous because the order of quantifiers is unclear. Let A be the set of Americans, let D be the set of dreams, and define the predicate $H(a, d)$ to be “American a has dream d .” Now the sentence could mean there is a single dream that every American shares—such as the dream of owning their own home:

$$\exists d \in D. \forall a \in A. H(a, d)$$

Or it could mean that every American has a personal dream:

$$\forall a \in A. \exists d \in D. H(a, d)$$

For example, some Americans may dream of a peaceful retirement, while others dream of continuing practicing their profession as long as they live, and still others may dream of being so rich they needn’t think about work at all.

Swapping quantifiers in Goldbach’s Conjecture creates a patently false statement that every even number ≥ 2 is the sum of *the same* two primes:

$$\underbrace{\exists p \in \text{Primes}. \exists q \in \text{Primes.}}_{\substack{\text{there exist primes} \\ p \text{ and } q \text{ such that}}} \underbrace{\forall n \in \text{Evens.}}_{\substack{\text{for every even} \\ \text{integer } n > 2}} n = p + q.$$

3.6.4 Variables Over One Domain

When all the variables in a formula are understood to take values from the same nonempty set, D , it’s conventional to omit mention of D . For example, instead of $\forall x \in D. \exists y \in D. Q(x, y)$ we’d write $\forall x \exists y. Q(x, y)$. The unnamed nonempty set that x and y range over is called the *domain of discourse*, or just plain *domain*, of the formula.

It’s easy to arrange for all the variables to range over one domain. For example, Goldbach’s Conjecture could be expressed with all variables ranging over the domain $\mathbb{N}$ as

$$\forall n. n \in \text{Evens} \text{ IMPLIES } (\exists p. \exists q. p \in \text{Primes AND } q \in \text{Primes AND } n = p + q).$$

3.6.5 Negating Quantifiers

There is a simple relationship between the two kinds of quantifiers. The following two sentences mean the same thing:

Not everyone likes ice cream.

There is someone who does not like ice cream.

The equivalence of these sentences is an instance of a general equivalence that holds between predicate formulas:

$$\text{NOT}(\forall x. P(x)) \text{ is equivalent to } \exists x. \text{NOT}(P(x)). \quad (3.25)$$

Similarly, these sentences mean the same thing:

There is no one who likes being mocked.

Everyone dislikes being mocked.

The corresponding predicate formula equivalence is

$$\text{NOT}(\exists x. P(x)) \text{ is equivalent to } \forall x. \text{NOT}(P(x)). \quad (3.26)$$

The general principle is that *moving a NOT across a quantifier changes the kind of quantifier*. Note that (3.26) follows from negating both sides of (3.25).

3.6.6 Validity for Predicate Formulas

The idea of validity extends to predicate formulas, but to be valid, a formula now must evaluate to true no matter what values its variables may take over any unspecified domain, and no matter what interpretation a predicate variable may be given. For example, we already observed that the rule for negating a quantifier is captured by the valid assertion (3.26).

Another useful example of a valid assertion is

$$\exists x \forall y. P(x, y) \text{ IMPLIES } \forall y \exists x. P(x, y). \quad (3.27)$$

Here's an explanation why this is valid:

Let D be the domain for the variables and P_0 be some binary predicate² on D . We need to show that if

$$\exists x \in D. \forall y \in D. P_0(x, y) \quad (3.28)$$

²That is, a predicate that depends on two variables.

holds under this interpretation, then so does

$$\forall y \in D. \exists x \in D. P_0(x, y). \quad (3.29)$$

So suppose (3.28) is true. Then by definition of $\exists$, this means that some element $d_0 \in D$ has the property that

$$\forall y \in D. P_0(d_0, y).$$

By definition of $\forall$, this means that

$$P_0(d_0, d)$$

is true for all $d \in D$. So given any $d \in D$, there is an element in D , namely, d_0 , such that $P_0(d_0, d)$ is true. But that’s exactly what (3.29) means, so we’ve proved that (3.29) holds under this interpretation, as required.

We hope this is helpful as an explanation, but we don’t really want to call it a “proof.” The problem is that with something as basic as (3.27), it’s hard to see what more elementary axioms are ok to use in proving it. What the explanation above did was translate the logical formula (3.27) into English and then appeal to the meaning, in English, of “for all” and “there exists” as justification.

In contrast to (3.27), the formula

$$\forall y \exists x. P(x, y) \text{ IMPLIES } \exists x \forall y. P(x, y). \quad (3.30)$$

is *not* valid. We can prove this just by describing an interpretation where the hypothesis, $\forall y \exists x. P(x, y)$, is true but the conclusion, $\exists x \forall y. P(x, y)$, is not true. For example, let the domain be the integers and $P(x, y)$ mean $x > y$. Then the hypothesis would be true because, given a value, n , for y we could choose the value of x to be $n + 1$, for example. But under this interpretation the conclusion asserts that there is an integer that is bigger than all integers, which is certainly false. An interpretation like this which falsifies an assertion is called a *counter model* to the assertion.

Problems for Section 3.1

Practice Problems

Problem 3.1.

Suppose you are taking a class, and that class has a textbook and a final exam. Let the propositional variables P , Q , and R have the following meanings:

P = You get an A on the final exam.

Q = You do every exercise in the book.

R = You get an A in the class.

Write the following propositions using P , Q , and R and logical connectives.

- (a) You get an A in the class, but you do not do every exercise in the book.
- (b) You get an A on the final, you do every exercise in the book, and you get an A in the class.
- (c) To get an A in the class, it is necessary for you to get an A on the final.
- (d) You get an A on the final, but you don't do every exercise in this book; nevertheless, you get an A in this class.

Problem 3.2.

Which of the following are *valid*?

- (a) $\exists x \exists y. P(x, y) \text{ IMPLIES } \exists y \exists x. P(x, y)$
- (b) $\forall x \exists y. Q(x, y) \text{ IMPLIES } \exists y \forall x. Q(x, y)$
- (c) $\exists x \forall y. R(x, y) \text{ IMPLIES } \forall y \exists x. R(x, y)$
- (d) $\text{NOT}(\exists x S(x)) \text{ IFF } \forall x \text{ NOT}(S(x))$

Class Problems

Problem 3.3.

When the mathematician says to his student, “If a function is not continuous, then it is not differentiable,” then letting D stand for “differentiable” and C for continuous, the only proper translation of the mathematician's statement would be

$$\text{NOT}(C) \text{ IMPLIES } \text{NOT}(D),$$

or equivalently,

$$D \text{ IMPLIES } C.$$

But when a mother says to her son, “If you don't do your homework, then you can't watch TV,” then letting T stand for “can watch TV” and H for “do your homework,” a reasonable translation of the mother's statement would be

$$\text{NOT}(H) \text{ IFF } \text{NOT}(T),$$

or equivalently,

$$H \text{ IFF } T.$$

Explain why it is reasonable to translate these two IF-THEN statements in different ways into propositional formulas.

Homework Problems

Problem 3.4.

Describe a simple recursive procedure which, given a positive integer argument, n , produces a truth table whose rows are all the assignments of truth values to n propositional variables. For example, for $n = 2$, the table might look like:

T	T
T	F
F	T
F	F

Your description can be in English, or a simple program in some familiar language (say Scheme or Java), but if you do write a program, be sure to include some sample output.

Problems for Section 3.2

Class Problems

Problem 3.5.

Propositional logic comes up in digital circuit design using the convention that **T** corresponds to 1 and **F** to 0. A simple example is a 2-bit *half-adder* circuit. This circuit has 3 binary inputs, a_1, a_0 and b , and 3 binary outputs, c, s_1, s_0 . The 2-bit word a_1a_0 gives the binary representation of an integer, k , between 0 and 3. The 3-bit word cs_1s_0 gives the binary representation of $k + b$. The third output bit, c , is called the final *carry bit*.

So if k and b were both 1, then the value of a_1a_0 would be 01 and the value of the output cs_1s_0 would 010, namely, the 3-bit binary representation of $1 + 1$.

In fact, the final carry bit equals 1 only when all three binary inputs are 1, that is, when $k = 3$ and $b = 1$. In that case, the value of cs_1s_0 is 100, namely, the binary representation of $3 + 1$.

This 2-bit half-adder could be described by the following formulas:

$$\begin{aligned} c_0 &= b \\ s_0 &= a_0 \text{ XOR } c_0 \\ c_1 &= a_0 \text{ AND } c_0 && \text{the carry into column 1} \\ s_1 &= a_1 \text{ XOR } c_1 \\ c_2 &= a_1 \text{ AND } c_1 && \text{the carry into column 2} \\ c &= c_2. \end{aligned}$$

(a) Generalize the above construction of a 2-bit half-adder to an $n + 1$ bit half-adder with inputs $a_n, \dots, a_1, a_0$ and b for arbitrary $n \geq 0$. That is, give simple formulas for s_i and c_i for $0 \leq i \leq n + 1$, where c_i is the carry into column i and $c = c_{n+1}$.

(b) Write similar definitions for the digits and carries in the sum of two $n + 1$ -bit binary numbers $a_n \dots a_1 a_0$ and $b_n \dots b_1 b_0$.

Visualized as digital circuits, the above adders consist of a sequence of single-digit half-adders or adders strung together in series. These circuits mimic ordinary pencil-and-paper addition, where a carry into a column is calculated directly from the carry into the previous column, and the carries have to ripple across all the columns before the carry into the final column is determined. Circuits with this design are called *ripple-carry* adders. Ripple-carry adders are easy to understand and remember and require a nearly minimal number of operations. But the higher-order output bits and the final carry take time proportional to n to reach their final values.

(c) How many of each of the propositional operations does your adder from part (b) use to calculate the sum?

Homework Problems

Problem 3.6.

There are adder circuits that are much faster than the ripple-carry circuits of Problem 3.5. They work by computing the values in later columns for both a carry of 0 and a carry of 1, *in parallel*. Then, when the carry from the earlier columns finally arrives, the pre-computed answer can be quickly selected. We'll illustrate this idea by working out the equations for an $(n + 1)$ -bit parallel half-adder.

Parallel half-adders are built out of parallel *add1* modules. An $(n + 1)$ -bit *add1* module takes as input the $(n + 1)$ -bit binary representation, $a_n \dots a_1 a_0$, of an integer, s , and produces as output the binary representation, $c p_n \dots p_1 p_0$, of $s + 1$.

(a) A 1-bit *add1* module just has input a_0 . Write propositional formulas for its outputs c and p_0 .

(b) Explain how to build an $(n + 1)$ -bit parallel half-adder from an $(n + 1)$ -bit *add1* module by writing a propositional formula for the half-adder output, o_i , using only the variables a_i , p_i , and b .

We can build a double-size *add1* module with $2(n + 1)$ inputs using two single-size *add1* modules with $n + 1$ inputs. Suppose the inputs of the double-size module are $a_{2n+1}, \dots, a_1, a_0$ and the outputs are $c, p_{2n+1}, \dots, p_1, p_0$. The setup is illustrated in Figure 3.1.

Namely, the first single size *add1* module handles the first $n + 1$ inputs. The inputs to this module are the low-order $n + 1$ input bits $a_n, \dots, a_1, a_0$, and its outputs will serve as the first $n + 1$ outputs $p_n, \dots, p_1, p_0$ of the double-size module. Let $c_{(1)}$ be the remaining carry output from this module.

The inputs to the second single-size module are the higher-order $n + 1$ input bits $a_{2n+1}, \dots, a_{n+2}, a_{n+1}$. Call its first $n + 1$ outputs $r_n, \dots, r_1, r_0$ and let $c_{(2)}$ be its carry.

(c) Write a formula for the carry, c , in terms of $c_{(1)}$ and $c_{(2)}$.

(d) Complete the specification of the double-size module by writing propositional formulas for the remaining outputs, p_i , for $n + 1 \leq i \leq 2n + 1$. The formula for p_i should only involve the variables a_i , $r_{i-(n+1)}$, and $c_{(1)}$.

(e) Parallel half-adders are exponentially faster than ripple-carry half-adders. Confirm this by determining the largest number of propositional operations required to compute any one output bit of an n -bit add module. (You may assume n is a power of 2.)

Exam Problems

Problem 3.7.

Show that there are exactly two truth assignments for the variables P,Q,R,S that satisfy the following formula:

$$(\overline{P} \text{ OR } Q) \text{ AND } (\overline{Q} \text{ OR } R) \text{ AND } (\overline{R} \text{ OR } S) \text{ AND } (\overline{S} \text{ OR } P)$$

Hint: A truth table will do the job, but it will have a bunch of rows. A proof by cases can be quicker; if you do use cases, be sure each one is clearly specified.

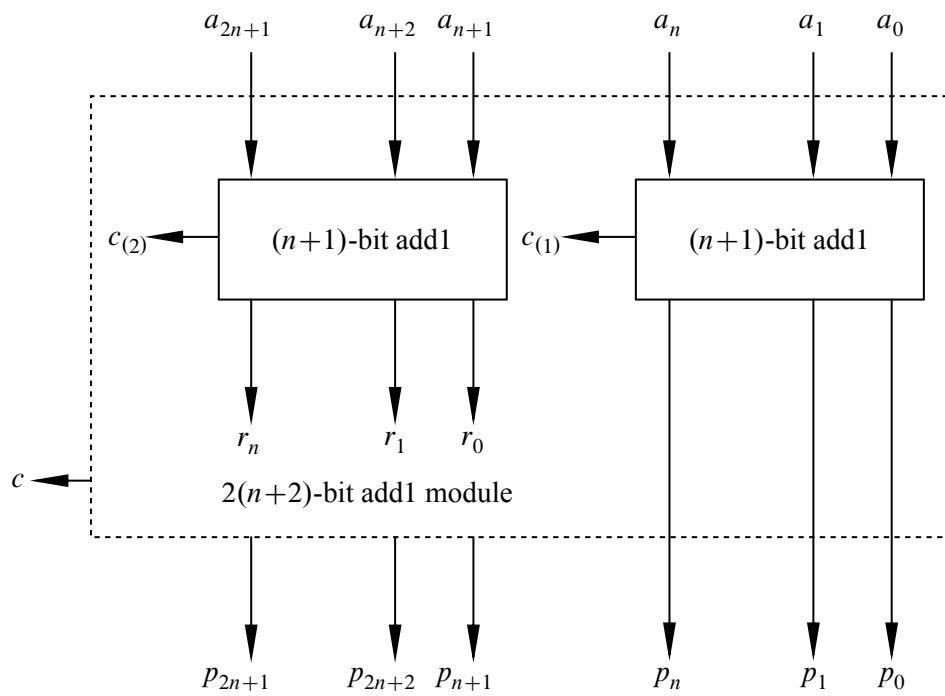


Figure 3.1 Structure of a Double-size *add1* Module.

Problems for Section 3.3

Practice Problems

Problem 3.8.

Indicate whether each of the following propositional formulas is valid (V), satisfiable but not valid (S), or not satisfiable (N). For the satisfiable ones, indicate a satisfying truth assignment.

$M \text{ IMPLIES } Q$

$M \text{ IMPLIES } (\overline{P} \text{ OR } \overline{Q})$

$M \text{ IMPLIES } [M \text{ AND } (P \text{ IMPLIES } M)]$

$(P \text{ OR } Q) \text{ IMPLIES } Q$

$(P \text{ OR } Q) \text{ IMPLIES } (\overline{P} \text{ AND } \overline{Q})$

$(P \text{ OR } Q) \text{ IMPLIES } [M \text{ AND } (P \text{ IMPLIES } M)]$

$(P \text{ XOR } Q) \text{ IMPLIES}$

Q

$(P \text{ XOR } Q) \text{ IMPLIES}$

$(\overline{P} \text{ OR } \overline{Q})$

$(P \text{ XOR } Q) \text{ IMPLIES}$

$[M \text{ AND } (P \text{ IMPLIES } M)]$

Class Problems

Problem 3.9. (a) Verify by truth table that

$(P \text{ IMPLIES } Q) \text{ OR } (Q \text{ IMPLIES } P)$

is valid.

(b) Let P and Q be propositional formulas. Describe a single formula, R , using AND's, OR's, and NOT's such that R is valid iff P and Q are equivalent.

Problem 3.10.

This problem³ examines whether the following specifications are *satisfiable*:

1. If the file system is not locked, then

(a) new messages will be queued.

(b) new messages will be sent to the messages buffer.

³From Rosen, 5th edition, Exercise 1.1.36

- (c) the system is functioning normally, and conversely, if the system is functioning normally, then the file system is not locked.
- 2. If new messages are not queued, then they will be sent to the messages buffer.
- 3. New messages will not be sent to the message buffer.
- (a) Begin by translating the five specifications into propositional formulas using four propositional variables:

$L ::=$ file system locked,
 $Q ::=$ new messages are queued,
 $B ::=$ new messages are sent to the message buffer,
 $N ::=$ system functioning normally.

- (b) Demonstrate that this set of specifications is satisfiable by describing a single truth assignment for the variables L , Q , B , N and verifying that under this assignment, all the specifications are true.
- (c) Argue that the assignment determined in part (b) is the only one that does the job.

Problems for Section 3.4

Practice Problems

Problem 3.11.

A half dozen different operators may appear in propositional formulas, but just AND, OR, and NOT are enough to do the job. That is because each of the operators is equivalent to a simple formula using only these three operators. For example, A IMPLIES B is equivalent to $\text{NOT}(A) \text{ OR } B$. So all occurrences of IMPLIES in a formula can be replaced using just NOT and OR.

- (a) Write formulas using only AND, OR, NOT that are equivalent to each of A IFF B and A XOR B . Conclude that every propositional formula is equivalent to an AND-OR-NOT formula.
- (b) Explain why you don’t even need AND.
- (c) Explain how to get by with the single operator NAND where A NAND B is equivalent by definition to $\text{NOT}(A \text{ AND } B)$.

Class Problems

Problem 3.12.

Explain how to find a conjunctive form for a propositional formula directly from a disjunctive form for its complement.

Homework Problems

Problem 3.13.

Use the equivalence axioms of Section 3.4.2 to convert the following formula to disjunctive form:

$$A \text{ XOR } B \text{ XOR } C.$$

Problems for Section 3.5

Homework Problems

Problem 3.14.

A 3-conjunctive form (3CF) formula is a conjunctive form formula in which each OR-term is a ORof at most 3 variables or negations of variables. Although it may be hard to tell if a propositional formula, F , is satisfiable, it is always easy to construct a formula, $\mathcal{C}(F)$, that is

- in 3-conjunctive form,
- has at most 24 times as many occurrences of variables as F , and
- is satisfiable iff F is satisfiable.

To construct $\mathcal{C}(F)$, introduce a different new variables, one for each operator that occurs in F . For example, if F was

$$((P \text{ XOR } Q) \text{ XOR } R) \text{ OR } (\overline{P} \text{ AND } S) \tag{3.31}$$

we might use new variables X_1 , X_2 , O , and A corresponding to the the operator occurrences as follows:

$$((P \underbrace{\text{ XOR } Q}_{X_1}) \underbrace{\text{ XOR } R}_{X_2}) \underbrace{\text{ OR } (\overline{P} \underbrace{\text{ AND } S}_{A})}_O.$$

Next we write a formula that constrains each new variable to have the same truth value as the subformula determined by its corresponding operator. For the example

above, these constraining formulas would be

$$\begin{aligned} X_1 &\text{ IFF } (P \text{ XOR } Q), \\ X_2 &\text{ IFF } (X_1 \text{ XOR } R), \\ A &\text{ IFF } (\overline{P} \text{ AND } S), \\ [O &\text{ IFF } (X_2 \text{ XOR } A)] \text{ AND } O \end{aligned}$$

- (a) Explain why the AND of the four constraining formulas above will be satisfiable iff (3.31) is satisfiable.
- (b) Explain why any constraining formula will be equivalent to a 3CF formula with at most 24 occurrences of variables.
- (c) Using the ideas illustrated in the previous parts, explain how to construct $\mathcal{C}(F)$ for an arbitrary propositional formula, F .

Problems for Section 3.6

Practice Problems

Problem 3.15.

For each of the following propositions:

1. $\forall x \exists y. 2x - y = 0$
2. $\forall x \exists y. x - 2y = 0$
3. $\forall x. x < 10 \text{ IMPLIES } (\forall y. y < x \text{ IMPLIES } y < 9)$
4. $\forall x \exists y. [y > x \wedge \exists z. y + z = 100]$

determine which propositions are true when the variables range over:

- (a) the nonnegative integers.
- (b) the integers.
- (c) the real numbers.

Problem 3.16.

Let $Q(x, y)$ be the statement

“ x has been a contestant on television show y .”

The universe of discourse for x is the set of all students at your school and for y is the set of all quiz shows on television.

Determine whether or not each of the following expressions is logically equivalent to the sentence:

“No student at your school has ever been a contestant on a television quiz show.”

(a) $\forall x \forall y. \text{NOT}(Q(x, y))$

(b) $\exists x \exists y. \text{NOT}(Q(x, y))$

(c) $\text{NOT}(\forall x \forall y. Q(x, y))$

(d) $\text{NOT}(\exists x \exists y. Q(x, y))$

Problem 3.17.

Find a counter model showing the following is not valid.

$$\exists x. P(x) \text{ IMPLIES } \forall x. P(x)$$

(Just define your your counter model. You do not need to verify that it is correct.)

Problem 3.18.

Find a counter model showing the following is not valid.

$$[\exists x. P(x) \text{ AND } \exists x. Q(x)] \text{ IMPLIES } \exists x. [P(x) \text{ AND } Q(x)]$$

(Just define your your counter model. You do not need to verify that it is correct.)

Class Problems

Problem 3.19.

A media tycoon has an idea for an all-news television network called LNN: The Logic News Network. Each segment will begin with a definition of the domain of discourse and a few predicates. The day’s happenings can then be communicated concisely in logic notation. For example, a broadcast might begin as follows:

THIS IS LNN. The domain of discourse is

$\{\text{Albert, Ben, Claire, David, Emily}\}.$

Let $D(x)$ be a predicate that is true if x is deceitful. Let $L(x, y)$ be a predicate that is true if x likes y . Let $G(x, y)$ be a predicate that is true if x gave gifts to y .

Translate the following broadcasted logic notation into (English) statements.

(a)

$$(\neg(D(\text{Ben}) \vee D(\text{David}))) \longrightarrow (L(\text{Albert}, \text{Ben}) \wedge L(\text{Ben}, \text{Albert}))$$

(b)

$$\begin{aligned} &\forall x (x = \text{Claire} \wedge \neg L(x, \text{Emily})) \vee (x \neq \text{Claire} \wedge L(x, \text{Emily})) \wedge \\ &\forall x (x = \text{David} \wedge L(x, \text{Claire})) \vee (x \neq \text{David} \wedge \neg L(x, \text{Claire})) \end{aligned}$$

(c)

$$\neg D(\text{Claire}) \longrightarrow (G(\text{Albert}, \text{Ben}) \wedge \exists x G(\text{Ben}, x))$$

(d)

$$\forall x \exists y \exists z (y \neq z) \wedge L(x, y) \wedge \neg L(x, z)$$

(e) How could you express “Everyone except for Claire likes Emily” using just propositional connectives *without* using any quantifiers ($\forall, \exists$)? Can you generalize to explain how *any* logical formula over this domain of discourse can be expressed without quantifiers? How big would the formula in the previous part be if it was expressed this way?

Problem 3.20.

The goal of this problem is to translate some assertions about binary strings into logic notation. The domain of discourse is the set of all finite-length binary strings: $\lambda, 0, 1, 00, 01, 10, 11, 000, 001, \dots$ (Here λ denotes the empty string.) In your translations, you may use all the ordinary logic symbols (including $=$), variables, and the binary symbols $0, 1$ denoting $0, 1$.

A string like $01x0y$ of binary symbols and variables denotes the *concatenation* of the symbols and the binary strings represented by the variables. For example, if the value of x is 011 and the value of y is 1111 , then the value of $01x0y$ is the binary string 0101101111 .

Here are some examples of formulas and their English translations. Names for these predicates are listed in the third column so that you can reuse them in your solutions (as we do in the definition of the predicate NO-1S below).

Meaning	Formula	Name
x is a prefix of y	$\exists z (xz = y)$	PREFIX(x, y)
x is a substring of y	$\exists u \exists v (uxv = y)$	SUBSTRING(x, y)
x is empty or a string of 0's	NOT(SUBSTRING($1, x$))	NO-1S(x)

- (a) x consists of three copies of some string.
- (b) x is an even-length string of 0's.
- (c) x does not contain both a 0 and a 1.
- (d) x is the binary representation of $2^k + 1$ for some integer $k \geq 0$.
- (e) An elegant, slightly trickier way to define NO-1S(x) is:

$$\text{PREFIX}(x, 0x). \quad (*)$$

Explain why (*) is true only when x is a string of 0's.

Problem 3.21.

For each of the logical formulas, indicate whether or not it is true when the domain of discourse is $\mathbb{N}$, (the nonnegative integers 0, 1, 2, ...), $\mathbb{Z}$ (the integers), $\mathbb{Q}$ (the rationals), $\mathbb{R}$ (the real numbers), and $\mathbb{C}$ (the complex numbers). Add a brief explanation to the few cases that merit one.

$$\begin{aligned} \exists x. x^2 = 2 \\ \forall x. \exists y. x^2 = y \\ \forall y. \exists x. x^2 = y \\ \forall x \neq 0. \exists y. xy = 1 \\ \exists x. \exists y. x + 2y = 2 \text{ AND } 2x + 4y = 5 \end{aligned}$$

Problem 3.22.

Show that

$$(\forall x \exists y. P(x, y)) \longrightarrow \forall z. P(z, z)$$

is not valid by describing a counter-model.

Homework Problems

Problem 3.23.

Express each of the following predicates and propositions in formal logic notation. The domain of discourse is the nonnegative integers, $\mathbb{N}$. Moreover, in addition to the propositional operators, variables and quantifiers, you may define predicates

using addition, multiplication, and equality symbols, and nonnegative integer *constants* $0, 1, \dots$), but no *exponentiation* (like x^y). For example, the predicate “ n is an even number” could be defined by either of the following formulas:

$$\exists m. (2m = n), \quad \exists m. (m + m = n).$$

- (a) m is a divisor of n .
- (b) n is a prime number.
- (c) n is a power of a prime.

Problem 3.24.

Translate the following sentence into a predicate formula:

There is a student who has emailed exactly two other people in the class, besides possibly herself.

The domain of discourse should be the set of students in the class; in addition, the only predicates that you may use are

- equality, and
- $E(x, y)$, meaning that “ x has sent e-mail to y .”

Exam Problems

Problem 3.25.

The following predicate logic formula is invalid:

$$\forall x, \exists y. P(x, y) \longrightarrow \exists y, \forall x. P(x, y)$$

Which of the following are counter models for the implication above?

1. The predicate $P(x, y) = 'yx = 1'$ where the domain of discourse is $\mathbb{Q}$.
2. The predicate $P(x, y) = 'y < x'$ where the domain of discourse is $\mathbb{R}$.
3. The predicate $P(x, y) = 'yx = 2'$ where the domain of discourse is $\mathbb{R}$ without 0.
4. The predicate $P(x, y) = 'yxy = x'$ where the domain of discourse is the set of all binary strings, including the empty string.

Problem 3.26.

Some students from a large class will be lined up left to right. Translate each of the following assertions into predicate formulas with the set of students in the class as the domain of discourse. The only predicates you may use are

- equality and,
- $F(x, y)$, meaning that “ x is somewhere to the left of y in the line.” For example, in the line “CDA”, both $F(C, A)$ and $F(C, D)$ are true.

Once you have defined a formula for a predicate P you may use the abbreviation “ P ” in further formulas.

- (a) Student x is in the line.
- (b) Student x is first in line.
- (c) Student x is immediately to the right of student y .
- (d) Student x is second.

4 Mathematical Data Types

4.1 Sets

We’ve been assuming that the concepts of sets, sequences, and functions are already familiar ones, and we’ve mentioned them repeatedly. Now we’ll do a quick review of the definitions.

Informally, a *set* is a bunch of objects, which are called the *elements* of the set. The elements of a set can be just about anything: numbers, points in space, or even other sets. The conventional way to write down a set is to list the elements inside curly-braces. For example, here are some sets:

$$\begin{array}{ll} A = \{\text{Alex, Tippy, Shells, Shadow}\} & \text{dead pets} \\ B = \{\text{red, blue, yellow}\} & \text{primary colors} \\ C = \{\{a, b\}, \{a, c\}, \{b, c\}\} & \text{a set of sets} \end{array}$$

This works fine for small finite sets. Other sets might be defined by indicating how to generate a list of them:

$$D = \{1, 2, 4, 8, 16, \dots\} \quad \text{the powers of 2}$$

The order of elements is not significant, so $\{x, y\}$ and $\{y, x\}$ are the same set written two different ways. Also, any object is, or is not, an element of a given set —there is no notion of an element appearing more than once in a set.¹ So writing $\{x, x\}$ is just indicating the same thing twice, namely, that x is in the set. In particular, $\{x, x\} = \{x\}$.

The expression $e \in S$ asserts that e is an element of set S . For example, $32 \in D$ and $\text{blue} \in B$, but $\text{Tailspin} \notin A$ —yet.

Sets are simple, flexible, and everywhere. You’ll find some set mentioned in nearly every section of this text.

4.1.1 Some Popular Sets

Mathematicians have devised special symbols to represent some common sets.

¹It’s not hard to develop a notion of *multisets* in which elements can occur more than once, but multisets are not ordinary sets.

symbol	set	elements
$\emptyset$	the empty set	none
$\mathbb{N}$	nonnegative integers	$\{0, 1, 2, 3, \dots\}$
$\mathbb{Z}$	integers	$\{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$
$\mathbb{Q}$	rational numbers	$\frac{1}{2}, -\frac{5}{3}, 16$, etc.
$\mathbb{R}$	real numbers	$\pi, e, -9, \sqrt{2}$, etc.
$\mathbb{C}$	complex numbers	$i, \frac{19}{2}, \sqrt{2} - 2i$, etc.

A superscript “+” restricts a set to its positive elements; for example, $\mathbb{R}^+$ denotes the set of positive real numbers. Similarly, $\mathbb{Z}^-$ denotes the set of negative integers.

4.1.2 Comparing and Combining Sets

The expression $S \subseteq T$ indicates that set S is a *subset* of set T , which means that every element of S is also an element of T (it could be that $S = T$). For example, $\mathbb{N} \subseteq \mathbb{Z}$ and $\mathbb{Q} \subseteq \mathbb{R}$ (every rational number is a real number), but $\mathbb{C} \not\subseteq \mathbb{Z}$ (not every complex number is an integer).

As a memory trick, notice that the $\subseteq$ points to the smaller set, just like a $\leq$ sign points to the smaller number. Actually, this connection goes a little further: there is a symbol $\subset$ analogous to $<$. Thus, $S \subset T$ means that S is a subset of T , but the two are *not* equal. So $A \subseteq A$, but $A \not\subset A$, for every set A .

There are several ways to combine sets. Let’s define a couple of sets for use in examples:

$$X ::= \{1, 2, 3\}$$

$$Y ::= \{2, 3, 4\}$$

- The *union* of sets X and Y (denoted $X \cup Y$) contains all elements appearing in X or Y or both. Thus, $X \cup Y = \{1, 2, 3, 4\}$.
- The *intersection* of X and Y (denoted $X \cap Y$) consists of all elements that appear in *both* X and Y . So $X \cap Y = \{2, 3\}$.
- The *set difference* of X and Y (denoted $X - Y$) consists of all elements that are in X , but not in Y . Therefore, $X - Y = \{1\}$ and $Y - X = \{4\}$.

4.1.3 Complement of a Set

Sometimes we are focused on a particular domain, D . Then for any subset, A , of D , we define $\overline{A}$ to be the set of all elements of D *not* in A . That is, $\overline{A} ::= D - A$. The set $\overline{A}$ is called the *complement* of A .

For example, when the domain we’re working with is the real numbers, the complement of the positive real numbers is the set of negative real numbers together with zero. That is,

$$\overline{\mathbb{R}^+} = \mathbb{R}^- \cup \{0\}.$$

It can be helpful to rephrase properties of sets using complements. For example, two sets, A and B , are said to be *disjoint* iff they have no elements in common, that is, $A \cap B = \emptyset$. This is the same as saying that A is a subset of the complement of B , that is, $A \subseteq \overline{B}$.

4.1.4 Power Set

The set of all the subsets of a set, A , is called the *power set*, $\mathcal{P}(A)$, of A . So $B \in \mathcal{P}(A)$ iff $B \subseteq A$. For example, the elements of $\mathcal{P}(\{1, 2\})$ are $\emptyset$, $\{1\}$, $\{2\}$ and $\{1, 2\}$.

More generally, if A has n elements, then there are 2^n sets in $\mathcal{P}(A)$. For this reason, some authors use the notation 2^A instead of $\mathcal{P}(A)$.

4.1.5 Set Builder Notation

An important use of predicates is in *set builder notation*. We’ll often want to talk about sets that cannot be described very well by listing the elements explicitly or by taking unions, intersections, etc., of easily-described sets. Set builder notation often comes to the rescue. The idea is to define a *set* using a *predicate*; in particular, the set consists of all values that make the predicate true. Here are some examples of set builder notation:

$$A ::= \{n \in \mathbb{N} \mid n \text{ is a prime and } n = 4k + 1 \text{ for some integer } k\}$$

$$B ::= \{x \in \mathbb{R} \mid x^3 - 3x + 1 > 0\}$$

$$C ::= \{a + bi \in \mathbb{C} \mid a^2 + 2b^2 \leq 1\}$$

The set A consists of all nonnegative integers n for which the predicate

$$“n \text{ is a prime and } n = 4k + 1 \text{ for some integer } k”$$

is true. Thus, the smallest elements of A are:

$$5, 13, 17, 29, 37, 41, 53, 57, 61, 73, \dots$$

Trying to indicate the set A by listing these first few elements wouldn’t work very well; even after ten terms, the pattern is not obvious! Similarly, the set B consists of all real numbers x for which the predicate

$$x^3 - 3x + 1 > 0$$

is true. In this case, an explicit description of the set B in terms of intervals would require solving a cubic equation. Finally, set C consists of all complex numbers $a + bi$ such that:

$$a^2 + 2b^2 \leq 1$$

This is an oval-shaped region around the origin in the complex plane.

4.1.6 Proving Set Equalities

Two sets are defined to be equal if they contain the same elements. That is, $X = Y$ means that $z \in X$ if and only if $z \in Y$, for all elements, z . (This is actually the first of the ZFC axioms.) So set equalities can be formulated and proved as “iff” theorems. For example:

Theorem 4.1.1 (*Distributive Law for Sets*). *Let A , B , and C be sets. Then:*

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \quad (4.1)$$

Proof. The equality (4.1) is equivalent to the assertion that

$$z \in A \cap (B \cup C) \quad \text{iff} \quad z \in (A \cap B) \cup (A \cap C) \quad (4.2)$$

for all z . Now we’ll prove (4.2) by a chain of iff’s.

Now we have

$$\begin{aligned} z \in A \cap (B \cup C) & \\ \text{iff } (z \in A) \text{ AND } (z \in B \cup C) & \quad (\text{def of } \cap) \\ \text{iff } (z \in A) \text{ AND } (z \in B \text{ OR } z \in C) & \quad (\text{def of } \cup) \\ \text{iff } (z \in A \text{ AND } z \in B) \text{ OR } (z \in A \text{ AND } z \in C) & \quad (\text{AND distributivity (3.16)}) \\ \text{iff } (z \in A \cap B) \text{ OR } (z \in A \cap C) & \quad (\text{def of } \cap) \\ \text{iff } z \in (A \cap B) \cup (A \cap C) & \quad (\text{def of } \cup) \end{aligned}$$

■

4.2 Sequences

Sets provide one way to group a collection of objects. Another way is in a *sequence*, which is a list of objects called *terms* or *components*. Short sequences are commonly described by listing the elements between parentheses; for example, (a, b, c) is a sequence with three terms.

While both sets and sequences perform a gathering role, there are several differences.

- The elements of a set are required to be distinct, but terms in a sequence can be the same. Thus, (a, b, a) is a valid sequence of length three, but $\{a, b, a\}$ is a set with two elements —not three.
- The terms in a sequence have a specified order, but the elements of a set do not. For example, (a, b, c) and (a, c, b) are different sequences, but $\{a, b, c\}$ and $\{a, c, b\}$ are the same set.
- Texts differ on notation for the *empty sequence*; we use λ for the empty sequence.

The product operation is one link between sets and sequences. A *product of sets*, $S_1 \times S_2 \times \cdots \times S_n$, is a new set consisting of all sequences where the first component is drawn from S_1 , the second from S_2 , and so forth. For example, $\mathbb{N} \times \{a, b\}$ is the set of all pairs whose first element is a nonnegative integer and whose second element is an a or a b :

$$\mathbb{N} \times \{a, b\} = \{(0, a), (0, b), (1, a), (1, b), (2, a), (2, b), \dots\}$$

A product of n copies of a set S is denoted S^n . For example, $\{0, 1\}^3$ is the set of all 3-bit sequences:

$$\{0, 1\}^3 = \{(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$$

4.3 Functions

A *function* assigns an element of one set, called the *domain*, to an element of another set, called the *codomain*. The notation

$$f : A \rightarrow B$$

indicates that f is a function with domain, A , and codomain, B . The familiar notation “ $f(a) = b$ ” indicates that f assigns the element $b \in B$ to a . Here b would be called the *value* of f at *argument* a .

Functions are often defined by formulas as in:

$$f_1(x) ::= \frac{1}{x^2}$$

where x is a real-valued variable, or

$$f_2(y, z) ::= y10yz$$

where y and z range over binary strings, or

$$f_3(x, n) ::= \text{the pair } (n, x)$$

where n ranges over the nonnegative integers.

A function with a finite domain could be specified by a table that shows the value of the function at each element of the domain. For example, a function $f_4(P, Q)$ where P and Q are propositional variables is specified by:

P	Q	$f_4(P, Q)$
T	T	T
T	F	F
F	T	T
F	F	T

Notice that f_4 could also have been described by a formula:

$$f_4(P, Q) ::= [P \text{ IMPLIES } Q].$$

A function might also be defined by a procedure for computing its value at any element of its domain, or by some other kind of specification. For example, define $f_5(y)$ to be the length of a left to right search of the bits in the binary string y until a 1 appears, so

$$\begin{aligned} f_5(0010) &= 3, \\ f_5(100) &= 1, \\ f_5(0000) &\text{ is undefined.} \end{aligned}$$

Notice that f_5 does not assign a value to any string of just 0's. This illustrates an important fact about functions: they need not assign a value to every element in the domain. In fact this came up in our first example $f_1(x) = 1/x^2$, which does not assign a value to 0. So in general, functions may be *partial functions*, meaning that there may be domain elements for which the function is not defined. If a function is defined on every element of its domain, it is called a *total function*.

It's often useful to find the set of values a function takes when applied to the elements in a *set* of arguments. So if $f : A \rightarrow B$, and S is a subset of A , we define $f(S)$ to be the set of all the values that f takes when it is applied to elements of S . That is,

$$f(S) ::= \{b \in B \mid f(s) = b \text{ for some } s \in S\}.$$

For example, if we let $[r, s]$ denote the interval from r to s on the real line, then $f_1([1, 2]) = [1/4, 1]$.

For another example, let’s take the “search for a 1” function, f_5 . If we let X be the set of binary words which start with an even number of 0’s followed by a 1, then $f_5(X)$ would be the odd nonnegative integers.

Applying f to a set, S , of arguments is referred to as “applying f pointwise to S ”, and the set $f(S)$ is referred to as the *image* of S under f .² The set of values that arise from applying f to all possible arguments is called the *range* of f . That is,

$$\text{range}(f) ::= f(\text{domain}(f)).$$

Some authors refer to the codomain as the range of a function, but they shouldn’t. The distinction between the range and codomain will be important later in Sections 5.1 when we relate sizes of sets to properties of functions between them.

4.3.1 Function Composition

Doing things step by step is a universal idea. Taking a walk is a literal example, but so is cooking from a recipe, executing a computer program, evaluating a formula, and recovering from substance abuse.

Abstractly, taking a step amounts to applying a function, and going step by step corresponds to applying functions one after the other. This is captured by the operation of *composing* functions. Composing the functions f and g means that first f applied is to some argument, x , to produce $f(x)$, and then g is applied to that result to produce $g(f(x))$.

Definition 4.3.1. For functions $f : A \rightarrow B$ and $g : B \rightarrow C$, the *composition*, $g \circ f$, of g with f is defined to be the function from A to C defined by the rule:

$$(g \circ f)(x) ::= g(f(x)),$$

for all $x \in A$.

Function composition is familiar as a basic concept from elementary calculus, and it plays an equally basic role in discrete mathematics.

4.4 Binary Relations

Binary relations define relations between two objects. For example, “less-than” on the real numbers relates every real number, a , to a real number, b , precisely when

²There is a picky distinction between the function f which applies to elements of A and the function which applies f pointwise to subsets of A , because the domain of f is A , while the domain of pointwise- f is $\mathcal{P}(A)$. It is usually clear from context whether f or pointwise- f is meant, so there is no harm in overloading the symbol f in this way.

$a < b$. Similarly, the subset relation relates a set, A , to another set, B , precisely when $A \subseteq B$. A function $f : A \rightarrow B$ is a special case of binary relation in which an element $a \in A$ is related to an element $b \in B$ precisely when $b = f(a)$.

In this section we’ll define some basic vocabulary and properties of binary relations.

Definition 4.4.1. A *binary relation*, R , consists of a set, A , called the *domain* of R , a set, B , called the *codomain* of R , and a subset of $A \times B$ called the *graph* of R .

A relation whose domain is A and codomain is B is said to be “between A and B ”, or “from A to B .” As with functions, we write $R : A \rightarrow B$ to indicate that R is a relation from A to B . When the domain and codomain are the same set, A , we simply say the relation is “on A .” It’s common to use infix notation “ $a R b$ ” to mean that the pair (a, b) is in the graph of R .

Notice that Definition 4.4.1 is exactly the same as the definition in Section 4.3 of a *function*, except that it doesn’t require the functional condition that, for each domain element, a , there is *at most* one pair in the graph whose first coordinate is a . As we said, a function is a special case of a binary relation.

The “in-charge of” relation, `chrg`, for MIT in Spring ’10 subjects and instructors is a handy example of a binary relation. Its domain, `Fac`, is the names of all the MIT faculty and instructional staff, and its codomain is the set, `SubNums`, of subject numbers in the Fall ’09–Spring ’10 MIT subject listing. The graph of `chrg` contains precisely the pairs of the form

$$(\langle \text{instructor-name} \rangle, \langle \text{subject-num} \rangle)$$

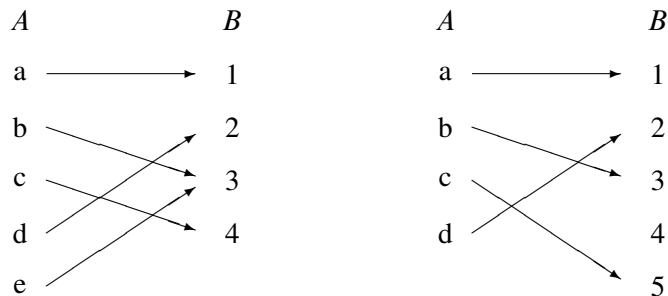
such that the faculty member named $\langle \text{instructor-name} \rangle$ is in charge of the subject with number $\langle \text{subject-num} \rangle$ that was offered in Spring ’10. So `graph(chrg)` contains pairs like

```
(A. R. Meyer, 6.042),
(A. R. Meyer, 18.062),
(A. R. Meyer, 6.844),
(T. Leighton, 6.042),
(T. Leighton, 18.062),
(G. Freeman, 6.011),
(G. Freeman, 6.UAT),
(G. Freeman, 6.881)
(G. Freeman, 6.882)
(T. Eng, 6.UAT)
(J. Guttag, 6.00)
⋮
```

Some subjects in the codomain, SubNums, do not appear among this list of pairs—that is, they are not in $\text{range}(\text{chrg})$. These are the Fall term-only subjects. Similarly, there are instructors in the domain, Fac, who do not appear in the list because all their in-charge subjects are Fall term-only.

4.4.1 Relation Diagrams

Some standard properties of a relation can be visualized in terms of a diagram. The diagram for a binary relation, R , has points corresponding to the elements of the domain appearing in one column (a very long column if $\text{domain}(R)$ is infinite). All the elements of the codomain appear in another column which we’ll usually picture as being to the right of the domain column. There is an arrow going from a point, a , in the lefthand, domain column to a point, b , in the righthand, codomain column, precisely when the corresponding elements are related by R . For example, here are diagrams for two functions:



Being a function is certainly an important property of a binary relation. What it means is that every point in the domain column has *at most one arrow coming out of it*. So we can describe being a function as the “ ≤ 1 arrow out” property. There are four more standard properties of relations that come up all the time. Here are all five properties defined in terms of arrows:

Definition 4.4.2. A binary relation, R is

- is a *function* when it has the [≤ 1 arrow **out**] property.
- is *surjective* when it has the [≥ 1 arrows **in**] property. That is, every point in the righthand, codomain column has at least one arrow pointing to it.
- is *total* when it has the [≥ 1 arrows **out**] property.
- is *injective* when it has the [≤ 1 arrow **in**] property.

- is *bijective* when it has both the $[= 1 \text{ arrow out}]$ and the $[= 1 \text{ arrow in}]$ property.

From here on, we’ll stop mentioning the arrows in these properties and for example, just write $[\leq 1 \text{ in}]$ instead of $[\leq 1 \text{ arrows in}]$.

So in the diagrams above, the relation on the left has the $[= 1 \text{ out}]$ and $[\geq 1 \text{ in}]$ properties, which means it is a total, surjective, function. But it does not have the $[\leq 1 \text{ in}]$ property because element 3 has two arrows going into it; in other words, it is not injective.

The relation on the right has the $[= 1 \text{ out}]$ and $[\leq 1 \text{ in}]$ properties, which means it is a total, injective function. But it does not have the $[\geq 1 \text{ in}]$ property because element 4 has no arrow going into it; in other words, it is not surjective.

Of course the arrows in a diagram for R correspond precisely to the pairs in the graph of R . Notice that knowing just where the arrows are is not enough to determine, for example, if R has the $[\geq 1 \text{ out}]$, total, property. If all we know is the arrows, we wouldn’t know about any points in the domain column that had no arrows out. In other words, $\text{graph}(R)$ alone does not determine whether R is total: we also need to know what $\text{domain}(R)$ is.

Example 4.4.3. The function defined by the formula $1/x^2$ has the $[\geq 1 \text{ out}]$ property if its domain is $\mathbb{R}^+$, but not if its domain is some set of real numbers including 0. It has the $[= 1 \text{ in}]$ and $[= 1 \text{ out}]$ property if its domain and codomain are both $\mathbb{R}^+$, but it has neither the $[\leq 1 \text{ in}]$ nor the $[\geq 1 \text{ out}]$ property if its domain and codomain are both $\mathbb{R}$.

4.4.2 Relational Images

The idea of the image of a set under a function extends directly to relations.

Definition 4.4.4. The *image* of a set, Y , under a relation, R , written $R(Y)$, is the set of elements of the codomain, B , of R that are related to some element in Y . In terms of the relation diagram, $R(Y)$ is the set of points with an arrow coming in that starts from some point in Y .

For example, the subject numbers that Meyer is in charge of in Spring ’10 is exactly $\text{chrg}(\text{A. Meyer})$. To figure out what this is, we look for all the arrows in the chrg diagram that start at “A. Meyer,” and see which subject-numbers are at the other end of these arrows. The set of these subject-numbers happened to be $\{6.042, 18.062, 6.844\}$. Similarly, to find the subject numbers that either Freeman or Eng are in charge of, we can collect all the arrows that start at either “G. Freeman,” or “T. Eng” and, again, see which subject-numbers are at the other end of these arrows. This, by definition, is $\text{chrg}(\{\text{G. Freeman}, \text{T. Eng}\})$. The partial list of

pairs in $\text{chrg}(\text{chrg})$ given above implies that

$$\{6.011, 6.881, 6.882, 6.\text{UAT}\} \subseteq \text{chrg}(\{\text{G. Freeman}, \text{T. Eng}\}).$$

Finally, Fac is the set of all in-charge instructors, so $\text{chrg}(\text{Fac})$ is the set of all the subjects listed for Spring '10.

Inverse Relations and Images

Definition 4.4.5. The *inverse*, R^{-1} of a relation $R : A \rightarrow B$ is the relation from B to A defined by the rule

$$b R^{-1} a \text{ IFF } a R b.$$

In other words, R^{-1} is the relation you get by reversing the direction of the arrows in the diagram for R .

Definition 4.4.6. The image of a set under the relation, R^{-1} , is called the *inverse image* of the set. That is, the inverse image of a set, X , under the relation, R , is defined to be $R^{-1}(X)$.

Continuing with the in-charge example above, the instructors in charge of 6.UAT in Spring '10 is exactly the inverse image of $\{6.\text{UAT}\}$ under the chrg relation. They turn out to be Eng and Freeman. That is,

$$\text{chrg}^{-1}(\{6.\text{UAT}\}) = \{\text{T. Eng}, \text{D. Freeman}\}.$$

Now let Intro be the set of introductory course 6 subject numbers. These are the subject numbers that start with “6.0.” So the names of the instructors who were in-charge of introductory course 6 subjects in Spring '10, is $\text{chrg}^{-1}(\text{Intro})$. From the part of the graph of chrg shown above, we can see that Meyer, Leighton, Freeman, and Gutttag were among the instructors in charge of introductory subjects in Spring '10. That is,

$$\{\text{Meyer}, \text{Leighton}, \text{Freeman}, \text{Gutttag}\} \subseteq \text{chrg}^{-1}(\text{Intro}).$$

Finally, $\text{chrg}^{-1}(\text{SubNums})$, is the set of all instructors who were in charge of a subject listed for Spring '10.

Problems for Section 4.1

Homework Problems

Problem 4.1.

Let A , B , and C be sets. Prove that:

$$A \cup B \cup C = (A - B) \cup (B - C) \cup (C - A) \cup (A \cap B \cap C). \quad (4.3)$$

Hint: $P \text{ OR } Q \text{ OR } R$ is equivalent to

$$(P \text{ AND } \overline{Q}) \text{ OR } (Q \text{ AND } \overline{R}) \text{ OR } (R \text{ AND } \overline{P}) \text{ OR } (P \text{ AND } Q \text{ AND } R).$$

Class Problems

Problem 4.2.

Set Formulas and Propositional Formulas.

(a) Verify that the propositional formula $(P \text{ AND } \overline{Q}) \text{ OR } (P \text{ AND } Q)$ is equivalent to P .

(b) Prove that³

$$A = (A - B) \cup (A \cap B)$$

for all sets, A, B , by using a chain of iff's to show that

$$x \in A \text{ IFF } x \in (A - B) \cup (A \cap B)$$

for all elements, x .

Problem 4.3.

Subset take-away⁴ is a two player game involving a fixed finite set, A . Players alternately choose nonempty subsets of A with the conditions that a player may not choose

- the whole set A , or
- any set containing a set that was named earlier.

The first player who is unable to move loses the game.

For example, if A is $\{1\}$, then there are no legal moves and the second player wins. If A is $\{1, 2\}$, then the only legal moves are $\{1\}$ and $\{2\}$. Each is a good reply to the other, and so once again the second player wins.

The first interesting case is when A has three elements. This time, if the first player picks a subset with one element, the second player picks the subset with the other two elements. If the first player picks a subset with two elements, the

³The *set difference*, $A - B$, of sets A and B is

$$A - B ::= \{a \in A \mid a \notin B\}.$$

⁴From Christenson & Tilford, *David Gale's Subset Takeaway Game*, *American Mathematical Monthly*, Oct. 1997

second player picks the subset whose sole member is the third element. Both cases produce positions equivalent to the starting position when A has two elements, and thus leads to a win for the second player.

Verify that when A has four elements, the second player still has a winning strategy.⁵

Practice Problems

Problem 4.4 (Power Sets).

For any set A , let $\mathcal{P}(A)$ be its *power set*, the set of all its subsets; note that A is itself a member of $\mathcal{P}(A)$. Let $\emptyset$ denote the empty set.

- (a) The elements of $\mathcal{P}(\{1, 2\})$ are:
- (b) The elements of $\mathcal{P}(\{\emptyset, \{\emptyset\}\})$ are:
- (c) How many elements are there in $\mathcal{P}(\{1, 2, \dots, 8\})$?

Problem 4.5.

How many relations are there on a set of size n when:

- (a) $n = 1$?
- (b) $n = 2$?
- (c) $n = 3$?

Exam Problems

Problem 4.6.

Below is a familiar “chain of IFF’s” proof of the set equality

$$A \cup (B \cap A) = A. \tag{4.4}$$

Proof.

$$\begin{aligned} x \in A \cup (B \cap A) &\text{ IFF } x \in A \text{ OR } x \in (B \cap A) && \text{(def of } \cup) \\ &\text{ IFF } x \in A \text{ OR } (x \in B \text{ AND } x \in A) && \text{(def of } \cap) \\ &\text{ IFF } x \in A, \end{aligned}$$

where the last IFF follows from the fact that

⁵David Gale worked out some of the properties of this game and conjectured that the second player wins the game for any set A . This remains an open problem.

the propositional formulas $P \text{ OR } (Q \text{ AND } P)$ and P are equivalent.

■

State a similar propositional equivalence that would justify the key step in a chain of IFF’s proof for the following set equality.

$$\overline{A - B} = (\overline{A} - \overline{C}) \cup (B \cap C) \cup ((\overline{A} \cup B) \cap \overline{C}) \quad (4.5)$$

(You are *not* being asked to write out a IFF proof of the equality or a proof of the propositional equivalence. Just state the equivalence.)

Problems for Section 4.2

Homework Problems

Problem 4.7.

Prove that for any sets A, B, C , and D , if $A \times B$ and $C \times D$ are disjoint, then either A and C are disjoint or B and D are disjoint.

Problem 4.8. (a) Give an example where the following result fails:

False Theorem. For sets A, B, C , and D , let

$$\begin{aligned} L &::= (A \cup B) \times (C \cup D), \\ R &::= (A \times C) \cup (B \times D). \end{aligned}$$

Then $L = R$.

(b) Identify the mistake in the following proof of the False Theorem.

Bogus proof. Since L and R are both sets of pairs, it’s sufficient to prove that $(x, y) \in L \iff (x, y) \in R$ for all x, y .

The proof will be a chain of iff implications:

$$\begin{aligned} &(x, y) \in R \\ \text{iff } &(x, y) \in (A \times C) \cup (B \times D) \\ \text{iff } &(x, y) \in A \times C, \text{ or } (x, y) \in B \times D \\ \text{iff } &(x \in A \text{ and } y \in C) \text{ or else } (x \in B \text{ and } y \in D) \\ \text{iff } &\text{either } x \in A \text{ or } x \in B, \text{ and either } y \in C \text{ or } y \in D \\ \text{iff } &x \in A \cup B \text{ and } y \in C \cup D \\ \text{iff } &(x, y) \in L. \end{aligned}$$



(c) Fix the proof to show that $R \subseteq L$.

Problems for Section 4.4

Practice Problems

Problem 4.9.

For a binary relation, $R : A \rightarrow B$, some properties of R can be determined from just the arrows of R , that is, from $\text{graph}(R)$, and others require knowing if there are elements in domain, A , or the codomain, B , that don't show up in $\text{graph}(R)$. For each of the following possible properties of R , indicate whether it is always determined by

1. $\text{graph}(R)$ alone,
2. $\text{graph}(R)$ and A alone,
3. $\text{graph}(R)$ and B alone,
4. all three parts of R .

Properties:

- (a) surjective
- (b) injective
- (c) total
- (d) function
- (e) bijection

Problem 4.10.

The *inverse*, R^{-1} , of a binary relation, R , from A to B , is the relation from B to A defined by:

$$b R^{-1} a \quad \text{iff} \quad a R b.$$

In other words, you get the diagram for R^{-1} from R by “reversing the arrows” in the diagram describing R . Now many of the relational properties of R correspond to different properties of R^{-1} . For example, R is an *total* iff R^{-1} is a *surjection*.

Fill in the remaining entries in this table:

R is	iff R^{-1} is
total	a surjection
a function	
a surjection	
an injection	
a bijection	

Hint: Explain what’s going on in terms of “arrows” from A to B in the diagram for R .

Problem 4.11.

For each of the following real-valued functions on the real numbers, indicate whether it is a bijection, a surjection but not a bijection, an injection but not a bijection, or neither an injection nor a surjection.

(a) $x \rightarrow x + 2$

(b) $x \rightarrow 2x$

(c) $x \rightarrow x^2$

(d) $x \rightarrow x^3$

(e) $x \rightarrow \sin x$

(f) $x \rightarrow x \sin x$

(g) $x \rightarrow e^x$

Problem 4.12.

Let

$$A ::= \{1, 2, 3\}$$

$$B ::= \{4, 5, 6\}$$

$$R ::= \{(1, 4), (1, 5), (2, 5), (3, 6)\}$$

$$S ::= \{(4, 5), (4, 6), (5, 4)\}.$$

Note that R is a relation from A to B and S is a relation from B to B .

List the pairs in each of the relations below.

- (a) $S \circ R$.
- (b) $S \circ S$.
- (c) $S^{-1} \circ R$.

Problem 4.13.

For any function $f : A \rightarrow B$ and subset, $A' \subset A$, we define

$$f(A') ::= \{f(a) \mid a \in A'\}$$

For example, if $f(x)$ is the doubling function, $2x$, with domain and codomain equal to the real numbers, then $f(\mathbb{Z})$ defines the set of even integers (here $\mathbb{Z}$ stands for the integers).

Now assume f is total and A is finite, and replace the $\star$ with one of $\leq, =, \geq$ to produce the *strongest* correct version of the following statements:

- (a) $|f(A)| \star |B|$.
- (b) If f is a surjection, then $|A| \star |B|$.
- (c) If f is a surjection, then $|f(A)| \star |B|$.
- (d) If f is an injection, then $|f(A)| \star |A|$.
- (e) If f is a bijection, then $|A| \star |B|$.

Problem 4.14.

Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be functions and $h : A \rightarrow C$ be their composition, namely, $h(a) ::= g(f(a))$ for all $a \in A$.

- (a) Prove that if f and g are surjections, then so is h .
- (b) Prove that if f and g are bijections, then so is h .
- (c) If f is a bijection, then so is f^{-1} .

Class Problems

Problem 4.15.

Define a *surjection relation*, surj , on sets by the rule

Definition. $A \text{ surj } B$ iff there is a surjective **function** from A to B .

Define the *injection relation*, inj , on sets by the rule

Definition. $A \text{ inj } B$ iff there is a total injective *relation* from A to B .

- (a) Prove that if $A \text{ surj } B$ and $B \text{ surj } C$, then $A \text{ surj } C$.
- (b) Explain why $A \text{ surj } B$ iff $B \text{ inj } A$.
- (c) Conclude from (a) and (b) that if $A \text{ inj } B$ and $B \text{ inj } C$, then $A \text{ inj } C$.

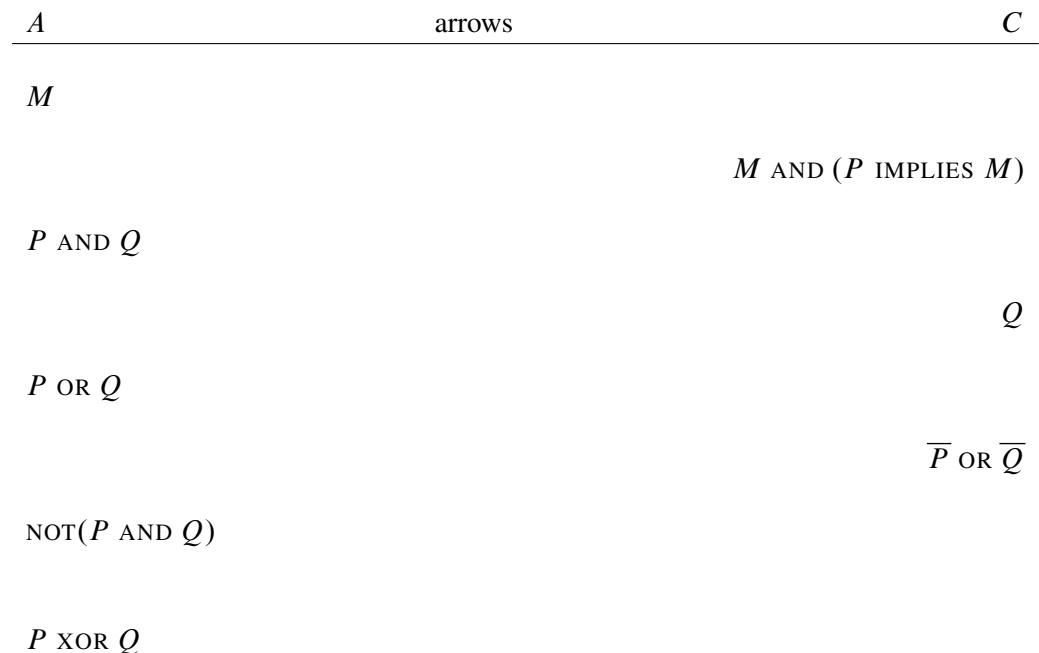
Problem 4.16.

Let A be the following set of five propositional formulas shown below on the left, and let C be the set of three propositional formulas on the right. The “implies” binary relation, I , from A to C is defined by the rule

$$F I G \quad \text{iff} \quad [\text{the formula } (F \text{ IMPLIES } G) \text{ is valid}].$$

For example, $(P \text{ AND } Q) I P$, because the formula $(P \text{ AND } Q)$ does imply P . Also, it is not true that $(P \text{ OR } Q) I P$ since $(P \text{ OR } Q) \text{ IMPLIES } P$ is not valid.

- (a) Fill in the arrows so the following figure describes the graph of the relation, I :



(b) Circle the properties below possessed by the relation I :

FUNCTION TOTAL INJECTIVE SURJECTIVE BIJECTIVE

(c) Circle the properties below possessed by the relation I^{-1} :

FUNCTION TOTAL INJECTIVE SURJECTIVE BIJECTIVE

Homework Problems

Problem 4.17.

Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be functions.

(a) Prove that if the composition $g \circ f$ is a bijection, then f is an injection and g is a surjection.

(b) If f is an injection and g is a surjection, then is $g \circ f$ necessarily a bijection?

Problem 4.18.

Let A , B , and C be nonempty sets, and let $f : B \rightarrow C$ and $g : A \rightarrow B$ be functions. Let $h ::= f \circ g$ be the composition function of f and g , namely, the function with domain A and range C such that $h(x) = f(g(x))$.

(a) Prove that if h is surjective and f is total and injective, then g must be surjective.

Hint: contradiction.

(b) Suppose that h is injective and f is total. Prove that g must be injective and provide a counterexample showing how this claim could fail if f was *not* total.

Problem 4.19.

Let A , B , and C be sets, and let $f : B \rightarrow C$ and $g : A \rightarrow B$ be functions. Let $h : A \rightarrow C$ be the composition, $f \circ g$, that is, $h(x) ::= f(g(x))$ for $x \in A$. Prove or disprove the following claims:

(a) If h is surjective, then f must be surjective.

(b) If h is surjective, then g must be surjective.

(c) If h is injective, then f must be injective.

(d) If h is injective and f is total, then g must be injective.

Problem 4.20.

There is a simple and useful way to extend composition of functions to composition of relations. Namely, let $R : B \rightarrow C$ and $S : A \rightarrow B$ be relations. Then the composition of R with S is the binary relation $(R \circ S) : A \rightarrow C$ defined by the rule

$$a (R \circ S) c ::= \exists b \in B. (b R c) \text{ AND } (a S b).$$

This agrees with the Definition 4.3.1 of composition in the special case when R and S are functions.

We can represent a relation, S , between two sets $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_m\}$ as an $n \times m$ matrix, M_S , of zeroes and ones, with the elements of M_S defined by the rule

$$M_S(i, j) = 1 \quad \text{IFF} \quad a_i S b_j.$$

If we represent relations as matrices in this fashion, then we can compute the composition of two relations R and S by a “boolean” matrix multiplication, $\otimes$, of their matrices. Boolean matrix multiplication is the same as matrix multiplication except that addition is replaced by OR and multiplication is replaced by AND. Namely, suppose $R : B \rightarrow C$ is a binary relation with $C = \{c_1, \dots, c_p\}$. So M_R is an $m \times p$ matrix. Then $M_S \otimes M_R$ is an $n \times p$ matrix defined by the rule:

$$[M_S \otimes M_R](i, j) ::= \text{OR}_{k=1}^m [M_S(i, k) \text{ AND } M_R(k, j)]. \quad (4.6)$$

Prove that the matrix representation, $M_{R \circ S}$, of $R \circ S$ equals $M_S \otimes M_R$ (note the reversal of R and S).

5 Infinite Sets

This chapter is about infinite sets and some challenges in proving things about them.

Wait a minute! Why bring up infinity in a Mathematics for *Computer Science* text? After all, any data set in a computer memory is limited by the size of memory, and there is a bound on the possible size of computer memory, for the simple reason that the universe is (or at least appears to be) bounded. So why not stick with *finite* sets of some (maybe pretty big) bounded size? This is a good question, but let’s see if we can persuade you that dealing with infinite sets is inevitable.

You may not have noticed, but up to now you’ve already accepted the routine use of the integers, the rationals and irrationals, and sequences of these—infinite sets all. Further, do you really want Physics or the other sciences to give up the real numbers on the grounds that only a bounded number of bounded measurements can be made in a bounded size universe? It’s pretty convincing and a lot simpler to ignore such big and uncertain bounds (the universe seems to be getting bigger all the time) and accept theories using real numbers.

Likewise in computer science, it simply isn’t plausible that writing a program to add nonnegative integers with up to as many digits as, say, the stars in the sky (billions of galaxies each with billions of stars), would be any different than writing a program that would add *any* two integers no matter how many digits they had. The same is true in designing a compiler: it’s neither useful nor sensible to make use of the fact that in a bounded universe, only a bounded number of programs will ever be compiled.

Infinite sets also provide a nice setting to practice proof methods, because it’s harder to sneak in unjustified steps under the guise of intuition. And there has been a truly astonishing outcome of studying infinite sets. It led to the discovery of widespread logical limits on what computers can possibly do. For example, in section 5.3, we’ll use reasoning developed for infinite sets to prove that it’s impossible to have a perfect type-checker for a programming language.

So in this chapter we ask you to bite the bullet and start learning to cope with infinity. But as a warmup, we’ll first examine some basic properties of *finite* sets.

5.1 Finite Cardinality

A finite set is one that has only a finite number of elements. This number of elements is the “size” or *cardinality* of the set:

Definition 5.1.1. If A is a finite set, the *cardinality* of A , written $|A|$, is the number of elements in A .

A finite set may have no elements (the empty set), or one element, or two elements, . . . , so the cardinality of finite sets is always a nonnegative integer.

Now suppose $R : A \rightarrow B$ is a function. This means that every element of A contributes at most one arrow to the diagram for R , so the number of arrows is at most the number of elements in A . That is, if R is a function, then

$$|A| \geq \# \text{arrows}.$$

If R is also surjective, then every element of B has an arrow into it, so there must be at least as many arrows in the diagram as the size of B . That is,

$$\# \text{arrows} \geq |B|.$$

Combining these inequalities implies that if R is a surjective function, then $|A| \geq |B|$.

In short, if we write $A \text{ surj } B$ to mean that there is a surjective function from A to B , then we’ve just proved a lemma: if $A \text{ surj } B$, then $|A| \geq |B|$. The following definition and lemma lists this statement and three similar rules relating domain and codomain size to relational properties.

Definition 5.1.2. Let A, B be (not necessarily finite) sets. Then

1. $A \text{ surj } B$ iff there is a surjective *function* from A to B .
2. $A \text{ inj } B$ iff there is a total, injective *relation* from A to B .
3. $A \text{ bij } B$ iff there is a bijection from A to B .
4. $A \text{ strict } B$ iff $B \text{ surj } A$, but not $A \text{ surj } B$.

Lemma 5.1.3. 1. If $A \text{ surj } B$, then $|A| \geq |B|$.

2. If $A \text{ inj } B$, then $|A| \leq |B|$.

3. If $A \text{ bij } B$, then $|A| = |B|$.

Proof. We’ve already given an “arrow” proof of implication 1. Implication 2. follows immediately from the fact that if R has the $[\leq 1 \text{ out}]$, function property, and the $[\geq 1 \text{ in}]$, surjective property, then R^{-1} is total and injective, so $A \text{ surj } B$ iff $B \text{ inj } A$. Finally, since a bijection is both a surjective function and a total injective relation, implication 3. is an immediate consequence of the first two. ■

Lemma 5.1.3.1. has a converse: if the size of a finite set, A , is greater than or equal to the size of another finite set, B , then it’s always possible to define a surjective function from A to B . In fact, the surjection can be a total function. To see how this works, suppose for example that

$$\begin{aligned} A &= \{a_0, a_1, a_2, a_3, a_4, a_5\} \\ B &= \{b_0, b_1, b_2, b_3\}. \end{aligned}$$

Then define a total function $f : A \rightarrow B$ by the rules

$$f(a_0) ::= b_0, \quad f(a_1) ::= b_1, \quad f(a_2) ::= b_2, \quad f(a_3) = f(a_4) = f(a_5) ::= b_3.$$

More concisely,

$$f(a_i) ::= b_{\min(i,3)},$$

for $0 \leq i \leq 5$. Since $5 \geq 3$, this f is a surjection. So we have figured out that if A and B are finite sets, then $|A| \geq |B|$ if and only if $A \text{ surj } B$. So it follows that $A \text{ strict } B$ iff $|A| < |B|$. All told, this argument wraps up the proof of the Theorem that summarizes the whole finite cardinality story:

Theorem 5.1.4. [Mapping Rules] For finite sets, A, B ,

$$|A| \geq |B| \quad \text{iff} \quad A \text{ surj } B, \tag{5.1}$$

$$|A| \leq |B| \quad \text{iff} \quad A \text{ inj } B, \tag{5.2}$$

$$|A| = |B| \quad \text{iff} \quad A \text{ bij } B, \tag{5.3}$$

$$|A| < |B| \quad \text{iff} \quad A \text{ strict } B. \tag{5.4}$$

5.1.1 How Many Subsets of a Finite Set?

As an application of the bijection mapping rule (5.3), we can give an easy proof of:

Theorem 5.1.5. There are 2^n subsets of an n -element set. That is,

$$|A| = n \quad \text{implies} \quad |\mathcal{P}(A)| = 2^n.$$

For example, the three-element set $\{a_1, a_2, a_3\}$ has eight different subsets:

$$\begin{array}{cccc} \emptyset & \{a_1\} & \{a_2\} & \{a_1, a_2\} \\ \{a_3\} & \{a_1, a_3\} & \{a_2, a_3\} & \{a_1, a_2, a_3\} \end{array}$$

Theorem 5.1.5 follows from the fact that there is a simple bijection from subsets of A to $\{0, 1\}^n$, the n -bit sequences. Namely, let $a_1, a_2, \dots, a_n$ be the elements of A . The bijection maps each subset of $S \subseteq A$ to the bit sequence $(b_1, \dots, b_n)$ defined by the rule that

$$b_i = 1 \quad \text{iff} \quad a_i \in S.$$

For example, if $n = 10$, then the subset $\{a_2, a_3, a_5, a_7, a_{10}\}$ maps to a 10-bit sequence as follows:

$$\begin{array}{lcl} \text{subset: } \{ & a_2, & a_3, & a_5, & a_7, & a_{10} & \} \\ \text{sequence: } (& 0, & 1, & 1, & 0, & 1, & 0, & 1, & 0, & 0, & 1 &) \end{array}$$

Now by bijection case of the Mapping Rules 5.1.4.(5.3),

$$|\mathcal{P}(A)| = |\{0, 1\}^n|.$$

But every computer scientist knows¹ that there are 2^n n -bit sequences! So we’ve proved Theorem 5.1.5!

5.2 Infinite Cardinality

In the late nineteenth century, the mathematician Georg Cantor was studying the convergence of Fourier series and found some series that he wanted to say converged “most of the time,” even though there were an infinite number of points where they didn’t converge. So Cantor needed a way to compare the size of infinite sets. To get a grip on this, he got the idea of extending Theorem 5.1.4 to infinite sets, by regarding two infinite sets as having the “same size” when there was a bijection between them. Likewise, an infinite set A is considered “as big as” a set B when $A \text{ surj } B$, and “strictly smaller” than B when $A \text{ strict } B$. Cantor got diverted from his study of Fourier series by his effort to develop a theory of infinite sizes based on these ideas. His theory ultimately had profound consequences for the foundations of mathematics and computer science. But Cantor made a lot of

¹In case you’re someone who doesn’t know how many n -bit sequences there are, you’ll find the 2^n explained in Section 15.2.2.

enemies in his own time because of his work: the general mathematical community doubted the relevance of what they called “Cantor’s paradise” of unheard-of infinite sizes.

A nice technical feature of Cantor’s idea is that it avoids the need for a definition of what the “size” of an infinite set might be—all it does is compare “sizes.”

Warning: We haven’t, and won’t, define what the “size” of an infinite set is. The definition of infinite “sizes” is cumbersome and technical, and we can get by just fine without it. All we need are the “as big as” and “same size” relations, surj and bij , between sets.

But there’s something else to **watch out for**: we’ve referred to surj as an “as big as” relation and bij as a “same size” relation on sets. Of course most of the “as big as” and “same size” properties of surj and bij on finite sets do carry over to infinite sets, but *some important ones don’t*—as we’re about to show. So you have to be careful: don’t assume that surj has any particular “as big as” property on *infinite* sets until it’s been proved.

Let’s begin with some familiar properties of the “as big as” and “same size” relations on finite sets that do carry over exactly to infinite sets:

Lemma 5.2.1. *For any sets, A, B, C ,*

1. $A \text{ surj } B \text{ iff } B \text{ inj } A$.
2. *If $A \text{ surj } B$ and $B \text{ surj } C$, then $A \text{ surj } C$.*
3. *If $A \text{ bij } B$ and $B \text{ bij } C$, then $A \text{ bij } C$.*
4. $A \text{ bij } B \text{ iff } B \text{ bij } A$.

Part 1. follows from the fact that R has the $[\leq 1 \text{ out}, \geq 1 \text{ in}]$ surjective function property iff R^{-1} has the $[\geq 1 \text{ out}, \leq 1 \text{ in}]$ total, injective property. Part 2. follows from the fact that compositions of surjections are surjections. Parts 3. and 4. follow from the first two parts because R is a bijection iff R and R^{-1} are surjective functions. We’ll leave verification of these facts to Problem 4.15.

Another familiar property of finite sets carries over to infinite sets, but this time it’s not so obvious:

Theorem 5.2.2. *[Schröder-Bernstein] For any sets A, B , if $A \text{ surj } B$ and $B \text{ surj } A$, then $A \text{ bij } B$.*

That is, the Schröder-Bernstein Theorem says that if A is at least as big as B and conversely, B is at least as big as A , then A is the same size as B . Phrased this way, you might be tempted to take this theorem for granted, but that would be a mistake. For infinite sets A and B , the Schröder-Bernstein Theorem is actually

pretty technical. Just because there is a surjective function $f : A \rightarrow B$ —which need not be a bijection —and a surjective function $g : B \rightarrow A$ —which also need not be a bijection —it’s not at all clear that there must be a bijection $e : A \rightarrow B$. The idea is to construct e from parts of both f and g . We’ll leave the actual construction to Problem 5.7.

Another familiar property similar to the one resolved by the Schröder-Bernstein Theorem is that if a set is *not* as big another, then it must be strictly smaller, that is,

$$\text{NOT}(A \text{ surj } B) \text{ IMPLIES } A \text{ strict } B.$$

This property of finite sets indeed also holds for infinite sets, but proving it requires methods that go well beyond the scope of this text.

5.2.1 Infinity is different

A basic property of finite sets that does *not* carry over to infinite sets is that adding something new makes a set bigger. That is, if A is a finite set and $b \notin A$, then $|A \cup \{b\}| = |A| + 1$, and so A and $A \cup \{b\}$ are not the same size. But if A is infinite, then these two sets *are* the same size!

Lemma 5.2.3. *Let A be a set and $b \notin A$. Then A is infinite iff $A \text{ bij } A \cup \{b\}$.*

Proof. Since A is *not* the same size as $A \cup \{b\}$ when A is finite, we only have to show that $A \cup \{b\}$ is the same size as A when A is infinite.

That is, we have to find a bijection between $A \cup \{b\}$ and A when A is infinite. Here’s how: since A is infinite, it certainly has at least one element; call it a_0 . But since A is infinite, it has at least two elements, and one of them must not be equal to a_0 ; call this new element a_1 . But since A is infinite, it has at least three elements, one of which must not equal a_0 or a_1 ; call this new element a_2 . Continuing in this way, we conclude that there is an infinite sequence $a_0, a_1, a_2, \dots, a_n, \dots$ of different elements of A . Now it’s easy to define a bijection $e : A \cup \{b\} \rightarrow A$:

$$\begin{aligned} e(b) &::= a_0, \\ e(a_n) &::= a_{n+1} && \text{for } n \in \mathbb{N}, \\ e(a) &::= a && \text{for } a \in A - \{b, a_0, a_1, \dots\}. \end{aligned}$$

■

5.2.2 Countable Sets

A set, C , is *countable* iff its elements can be listed in order, that is, the distinct elements in C are precisely

$$c_0, c_1, \dots, c_n, \dots$$

This means that if we defined a function, f , on the nonnegative integers by the rule that $f(i) ::= c_i$, then f would be a bijection from $\mathbb{N}$ to C . More formally,

Definition 5.2.4. A set, C , is *countably infinite* iff $\mathbb{N} \text{ bij } C$. A set is *countable* iff it is finite or countably infinite.

For example, the most basic countably infinite set is the set, $\mathbb{N}$, itself. But the set, $\mathbb{Z}$, of *all* integers is also countably infinite, because the integers can be listed in the order,

$$0, -1, 1, -2, 2, -3, 3, \dots \quad (5.5)$$

In this case, there is a simple formula for the n th element of the list (5.5). That is, the bijection $f : \mathbb{N} \rightarrow \mathbb{Z}$ such that $f(n)$ is the n th element of the list can be defined as:

$$f(n) ::= \begin{cases} n/2 & \text{if } n \text{ is even,} \\ -(n+1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

There is also a simple way to list all *pairs* of nonnegative integers, which shows that $(\mathbb{N} \times \mathbb{N})$ is also countably infinite. From that it's a small step to reach the conclusion that the set, $\mathbb{Q}^{\geq 0}$, of nonnegative rational numbers is countable. This may be a surprise —after all, the rationals densely fill up the space between integers, and for any two, there's another in between, so it might seem as though you couldn't write them all out in a list, but Problem 5.6 illustrates how to do it. More generally, it is easy to show that countable sets are closed under unions and products (Problems 5.1 and 5.12) which implies the countability of a bunch of familiar sets:

Corollary 5.2.5. *The following sets are countably infinite:*

$$\mathbb{Z}^+, \mathbb{Z}, \mathbb{N} \times \mathbb{N}, \mathbb{Q}^+, \mathbb{Z} \times \mathbb{Z}, \mathbb{Q}.$$

A small modification of the proof of Lemma 5.2.3 shows that countably infinite sets are the “smallest” infinite sets, namely, if A is an infinite set, and B is countable, then $A \text{ surj } B$ (see Problem 5.4).

Since adding one new element to an infinite set doesn't change its size, it's obvious that neither will adding any *finite* number of elements. It's a common mistake to think that this proves that you can throw in infinitely many new elements. But just because it's ok to do something any finite number of times doesn't make it OK to do an infinite number of times. For example, starting from 3, you can increment by 1 any finite number of times and the result will be some integer greater than or equal to 3. But if you increment an infinite number of times, you don't get an integer at all.

The good news is that you really can add a *countably* infinite number of new elements to an infinite set and still wind up with just a set of the same size, see Problem 5.9.

5.2.3 Power sets are strictly bigger

Cantor’s astonishing discovery was that *not all infinite sets are the same size*. In particular, he proved that for any set, A , the power set, $\mathcal{P}(A)$, is “strictly bigger” than A . That is

In particular,

Theorem 5.2.6. [Cantor] For any set, A ,

$$A \text{ strict } \mathcal{P}(A).$$

Proof. First of all, $\mathcal{P}(A)$ is as big as A : for example, the partial function $f : \mathcal{P}(A) \rightarrow A$, where $f(\{a\}) ::= a$ for $a \in A$ and f is only defined on one-element sets, is a surjection.

To show that $\mathcal{P}(A)$ is strictly bigger than A , we have to show that if g is a function from A to $\mathcal{P}(A)$, then g is not a surjection. To do this, we’ll simply find a subset, $A_g \subseteq \mathcal{P}(A)$ that is not in the range of g . The idea is, for any element $a \in A$, to look at the set $g(a) \subseteq A$ and ask whether or not a happens to be in $g(a)$. Namely define

$$A_g ::= \{a \in A \mid a \notin g(a)\}.$$

Now A_g is a well-defined subset of A , which means it is a member of $\mathcal{P}(A)$. But A_g can’t be in the range of g , because if it were, we would have

$$A_g = g(a_0)$$

for some $a_0 \in A$, so by definition of A_g ,

$$a \in g(a_0) \text{ iff } a \in A_g \text{ iff } a \notin g(a)$$

for all $a \in A$. Now letting $a = a_0$ yields the contradiction

$$a_0 \in g(a_0) \text{ iff } a_0 \notin g(a_0).$$

So g is not a surjection, because there is an element in the power set of A , namely the set A_g , that is not in the range of g . ■

Cantor’s Theorem immediately implies:

Corollary 5.2.7. $\mathcal{P}(\mathbb{N})$ is uncountable.

The bijection between subsets of an n -element set and the length n bit-strings, $\{0, 1\}^n$, used to prove Theorem 5.1.5, carries over to a bijection between subsets of a countably infinite set and the infinite bit-strings, $\{0, 1\}^\omega$. That is,

$$\mathcal{P}(\mathbb{N}) \text{ bij } \{0, 1\}^\omega.$$

This immediately implies

Corollary 5.2.8. $\{0, 1\}^\omega$ is uncountable.

Larger Infinities

There are lots of different sizes of infinite sets. For example, starting with the infinite set, $\mathbb{N}$, of nonnegative integers, we can build the infinite sequence of sets

$$\mathbb{N} \text{ strict } \mathcal{P}(\mathbb{N}) \text{ strict } \mathcal{P}(\mathcal{P}(\mathbb{N})) \text{ strict } \mathcal{P}(\mathcal{P}(\mathcal{P}(\mathbb{N}))) \text{ strict } \dots$$

By Theorem 5.2.6, each of these sets is strictly bigger than all the preceding ones. But that’s not all: the union of all the sets in the sequence is strictly bigger than each set in the sequence (see Problem 5.16). In this way you can keep going indefinitely, building “bigger” infinities all the way.

5.3 The Halting Problem

Granted that towers of larger and larger infinite sets are at best just a romantic concern for a computer scientist, the *reasoning* that leads to these conclusions plays a critical role in the theory of computation. Cantor’s proof embodies the simplest form of what is known as a “diagonal argument.” Diagonal arguments are used to show that lots of problems logically just can’t be solved by computation, and there is no getting around it.

This story begins with a reminder that having procedures operate on programs is a basic part of computer science technology. For example, *compilation* refers to taking any given program text written in some “high level” programming language like Java, C++, Python, . . . , and then generating a program of low-level instructions that does the same thing but is targeted to run well on available hardware. Similarly, *interpreters* or *virtual machines* are procedures that take a program text designed to be run on one kind of computer and simulate it on another kind of computer. Routine features of compilers involve “type-checking” programs to ensure that certain kinds of run-time errors won’t happen, and “optimizing” the generated programs so they run faster or use less memory.

Now the fundamental thing that logically just can’t be done by computation is a *perfect* job of type-checking, optimizing, or any kind of analysis of the overall run time behavior of programs. In this section we’ll illustrate this with a basic example known as the *Halting Problem*. The general Halting Problem for some programming language is, given an arbitrary program, recognize when running the program will not finish successfully —halt— because it aborts with some kind of error, or because it simply never stops. Of course it’s easy to detect when any given program *will* halt: just run it on a virtual machine and wait. The problem is what if the given program does *not* halt —how do you recognize that? We will use

a diagonal argument to prove that if an analysis program tries to recognize non-halting programs, it is bound to give wrong answers, or no answers, for an infinite number of programs it might have to analyze!

To be precise about this, let’s call a programming procedure —written in your favorite programming language such as C++, or Java, or Python —a *string procedure* when it is applicable to strings over a standard alphabet —say the 256 character ASCII alphabet ASCII. When a string procedure applied to an ASCII string returns the boolean value **True**, we’ll say the procedure *recognizes* the string. If the procedure does anything else —returns a value other than **True**, aborts with an error, runs forever, . . . —then it doesn’t recognize the string.

As a simple example, you might think about how to write a string procedure that recognizes precisely those *double letter* ASCII strings in which every character occurs twice in a row. For example, aaCC33, and zz++ccBB are double letter ASCII strings, but aa;bb, b33, and AAAAA are not. Even better, how about actually writing a recognizer for the double letter ASCII strings in your favorite programming language?

We’ll call a set of strings *recognizable* if there is a procedure that recognizes precisely that set of strings. So the set of double letter strings is recognizable.

Let ASCII* be the set of (finite) strings of ASCII characters. There is no harm in assuming that every program can be written using only the ASCII characters; they usually are anyway. When a string $s \in \text{ASCII}^*$ is actually the ASCII description of some string procedure, we’ll refer to that string procedure as P_s . You can think of P_s as the result of compiling s .² It’s technically helpful to treat *every* ASCII string as a program for a string procedure. So when a string $s \in \text{ASCII}^*$ doesn’t parse as a proper string procedure, we’ll define P_s to be some default string procedure —say one that always returns **False**.

Now we can define the precise set of strings that describe non-halting programs:

Definition 5.3.1.

$$\text{No-halt} ::= \{s \in \text{ASCII}^* \mid P_s \text{ does not recognize } s\}. \quad (5.6)$$

Recognizing the strings in No-halt is a special case of the Halting Problem. We’ll blow away any chance of having a program solve the general problem by showing that no program can solve this special case. In particular, we’re going to prove

Theorem 5.3.2. *No-halt is not recognizable.*

²The string, $s \in \text{ASCII}^*$, and the procedure, P_s , have to be distinguished to avoid a type error: you can’t apply a string to string. For example, let s be the string that you wrote as your program to recognize the double letter strings. Applying s to a string argument, say aabbccdd, should throw a type exception; what you need to do is compile s to the procedure P_s and then apply P_s to aabbccdd.

We’ll use an argument just like Cantor’s in the proof of Theorem 5.2.6.

Proof. Namely for any string $s \in \text{ASCII}^*$, let $f(s)$ be the set of strings recognized by P_s :

$$f(s) ::= \{t \in \text{ASCII}^* \mid P_s \text{ recognizes } t\}.$$

By convention, we associated a string procedure, P_s , with every string, $s \in \text{ASCII}^*$, which makes f a total function, and by definition,

$$s \in \text{No-halt} \text{ IFF } s \notin f(s), \quad (5.7)$$

for all strings, $s \in \text{ASCII}^*$.

Now suppose to the contrary that No-halt was recognizable. This means there is some procedure P_{s_0} that recognizes No-halt, which is the same as saying that

$$\text{No-halt} = f(s_0).$$

Combined with (5.7), we get

$$s \in f(s_0) \text{ iff } s \notin f(s) \quad (5.8)$$

for all $s \in \text{ASCII}^*$. Now letting $s = s_0$ in (5.8) yields the immediate contradiction

$$s_0 \in f(s_0) \text{ iff } s_0 \notin f(s_0).$$

This contradiction implies that No-halt cannot be recognized by any string procedure. ■

So that does it: it’s logically impossible for programs in any particular language to solve just this special case of the general Halting Problem for programs in that language. And having proved that it’s impossible to have a procedure that figures out whether an arbitrary program returns **True**, it’s easy to show that it’s impossible to have a procedure that is a perfect recognizer for *any* overall run time property.³

For example, most compilers do “static” type-checking at compile time to ensure that programs won’t make run-time type errors. A program that type-checks is guaranteed not to cause a run-time type-error. But since it’s impossible to recognize perfectly when programs won’t cause type-errors, it follows that the type-checker must be rejecting programs that really wouldn’t cause a type-error. The conclusion is that no type-checker is perfect—you can always do better!

³The weasel word “overall” creeps in here to rule out some run time properties that are easy to recognize because they depend only on part of the run time behavior. For example, the set of programs that halt after executing at most 100 instructions is recognizable.

It’s a different story if we think about the *practical* possibility of writing programming analyzers. The fact that it’s logically impossible to analyze perfectly arbitrary programs does not mean that you can’t do a very good job analyzing interesting programs that come up in practice. In fact these “interesting” programs are commonly *intended* to be analyzable in order to confirm that they do what they’re supposed to do.

So it’s not clear how much of a hurdle this theoretical limitation implies in practice. What the theory does provide is some perspective on claims about general analysis methods for programs. The theory tells us that people who make such claims either

- are exaggerating the power (if any) of their methods —say to make a sale or get a grant, or
- are trying to keep things simple by not going into technical limitations they’re aware of, or
- perhaps most commonly, are so excited about some useful practical successes of their methods that they haven’t bothered to think about the limitations which you know must be there.

So from now on, if you hear people making claims about having general program analysis/verification/optimization methods, you’ll know they can’t be telling the whole story.

One more important point: there’s no hope of getting around this by switching programming languages. Our proof covered programs written in some given programming language like Java, for example, and concluded that no Java program can perfectly analyze all Java programs. Could there be a C++ analysis procedure that successfully takes on all Java programs? After all, C++ does allow more intimate manipulation of computer memory than Java does. But there is no loophole here: it’s possible to write a virtual machine for C++ in Java, so if there were a C++ procedure that analyzed Java programs, the Java virtual machine would be able to do it too, and that’s impossible. These logical limitations on the power of computation apply no matter what kinds of programs or computers you use.

5.4 The Logic of Sets

5.4.1 Russell’s Paradox

Reasoning naively about sets turns out to be risky. In fact, one of the earliest attempts to come up with precise axioms for sets in the late nineteenth century by the logician Gotlob Frege, was shot down by a three line argument known as *Russell’s Paradox*⁴ which reasons in nearly the same way as the proof of Cantor’s Theorem 5.2.6. This was an astonishing blow to efforts to provide an axiomatic foundation for mathematics:

Russell’s Paradox

Let S be a variable ranging over all sets, and define

$$W ::= \{S \mid S \notin S\}.$$

So by definition,

$$S \in W \text{ iff } S \notin S,$$

for every set S . In particular, we can let S be W , and obtain the contradictory result that

$$W \in W \text{ iff } W \notin W.$$

So the simplest reasoning about sets crashes mathematics! Russell and his colleague Whitehead spent years trying to develop a set theory that was not contradictory, but would still do the job of serving as a solid logical foundation for all of mathematics.

Actually, a way out of the paradox was clear to Russell and others at the time: *it’s unjustified to assume that W is a set*. So the step in the proof where we let S be W has no justification, because S ranges over sets, and W may not be a set. In fact, the paradox implies that W had better not be a set!

⁴Bertrand *Russell* was a mathematician/logician at Cambridge University at the turn of the Twentieth Century. He reported that when he felt too old to do mathematics, he began to study and write about philosophy, and when he was no longer smart enough to do philosophy, he began writing about politics. He was jailed as a conscientious objector during World War I. For his extensive philosophical and political writing, he won a Nobel Prize for Literature.

But denying that W is a set means we must *reject* the very natural axiom that every mathematically well-defined collection of sets is actually a set. The problem faced by Frege, Russell and their fellow logicians was how to specify *which* well-defined collections are sets. Russell and his Cambridge University colleague Whitehead immediately went to work on this problem. They spent a dozen years developing a huge new axiom system in an even huger monograph called *Principia Mathematica*, but basically their approach failed. It was so cumbersome no one ever used it, and it was subsumed by a much simpler, and now widely accepted, axiomatization of set theory due to the logicians Zermelo and Frankel.

5.4.2 The ZFC Axioms for Sets

It’s generally agreed that, using some simple logical deduction rules, essentially all of mathematics can be derived from some axioms about sets called the Axioms of Zermelo-Frankel Set Theory with Choice (ZFC).

We’re *not* going to be studying these axioms in this text, but we thought you might like to see them –and while you’re at it, get some practice reading quantified formulas:

Extensionality. Two sets are equal if they have the same members. In a logic formula of set theory, this would be stated as:

$$(\forall z. z \in x \text{ IFF } z \in y) \text{ IMPLIES } x = y.$$

Pairing. For any two sets x and y , there is a set, $\{x, y\}$, with x and y as its only elements:

$$\forall x, y. \exists u. \forall z. [z \in u \text{ IFF } (z = x \text{ OR } z = y)]$$

Union. The union, u , of a collection, z , of sets is also a set:

$$\forall z. \exists u. \forall x. (\exists y. x \in y \text{ AND } y \in z) \text{ IFF } x \in u.$$

Infinity. There is an infinite set. Specifically, there is a nonempty set, x , such that for any set $y \in x$, the set $\{y\}$ is also a member of x .

Subset. Given any set, x , and any definable property of sets, there is a set containing precisely those elements $y \in x$ that have the property.

$$\forall x. \exists z. \forall y. y \in z \text{ IFF } [y \in x \text{ AND } \phi(y)]$$

where $\phi(y)$ is any assertion about y definable in the notation of set theory.

Power Set. All the subsets of a set form another set:

$$\forall x. \exists p. \forall u. u \subseteq x \text{ IFF } u \in p.$$

Replacement. Suppose a formula, ϕ , of set theory defines the graph of a function, that is,

$$\forall x, y, z. [\phi(x, y) \text{ AND } \phi(x, z)] \text{ IMPLIES } y = z.$$

Then the image of any set, s , under that function is also a set, t . Namely,

$$\forall s \exists t \forall y. [\exists x. \phi(x, y) \text{ IFF } y \in t].$$

Foundation. There cannot be an infinite sequence

$$\cdots \in x_n \in \cdots \in x_1 \in x_0$$

of sets each of which is a member of the previous one. This is equivalent to saying every nonempty set has a “member-minimal” element. Namely, define

$$\text{member-minimal}(m, x) ::= [m \in x \text{ AND } \forall y \in x. y \notin m].$$

Then the Foundation axiom is

$$\forall x. x \neq \emptyset \text{ IMPLIES } \exists m. \text{member-minimal}(m, x).$$

Choice. Given a set, s , whose members are nonempty sets no two of which have any element in common, then there is a set, c , consisting of exactly one element from each set in s . The formula is given in Problem 5.20.

5.4.3 Avoiding Russell’s Paradox

These modern ZFC axioms for set theory are much simpler than the system Russell and Whitehead first came up with to avoid paradox. In fact, the ZFC axioms are as simple and intuitive as Frege’s original axioms, with one technical addition: the Foundation axiom. Foundation captures the intuitive idea that sets must be built up from “simpler” sets in certain standard ways. And in particular, Foundation implies that no set is ever a member of itself. So the modern resolution of Russell’s paradox goes as follows: since $S \notin S$ for all sets S , it follows that W , defined above, contains every set. This means W can’t be a set—or it would be a member of itself.

5.5 Does All This Really Work?

So this is where mainstream mathematics stands today: there is a handful of ZFC axioms from which virtually everything else in mathematics can be logically derived. This sounds like a rosy situation, but there are several dark clouds, suggesting that the essence of truth in mathematics is not completely resolved.

- The ZFC axioms weren’t etched in stone by God. Instead, they were mostly made up by Zermelo, who may have been a brilliant logician, but was also a fallible human being —probably some days he forgot his house keys. So maybe Zermelo, just like Frege, didn’t get his axioms right and will be shot down by some successor to Russell who will use his axioms to prove a proposition P and its negation $\overline{P}$. Then math would be broken. This sounds crazy, but after all, it has happened before.

In fact, while there is broad agreement that the ZFC axioms are capable of proving all of standard mathematics, the axioms have some further consequences that sound paradoxical. For example, the Banach-Tarski Theorem says that, as a consequence of the Axiom of Choice, a solid ball can be divided into six pieces and then the pieces can be rigidly rearranged to give *two* solid balls of the same size as the original!

- Some basic questions about the nature of sets remain unresolved. For example, Cantor raised the question whether there is a set whose size is strictly between the smallest infinite set, $\mathbb{N}$ (see Problem 5.4), and the strictly larger set, $\mathcal{P}(\mathbb{N})$? Cantor guessed not:

Cantor’s Continuum Hypothesis: There is no set, A , such that

$$\mathbb{N} \text{ strict } A \text{ strict } \mathcal{P}(\mathbb{N}).$$

The Continuum Hypothesis remains an open problem a century later. Its difficulty arises from one of the deepest results in modern Set Theory —discovered in part by Gödel in the 1930’s and Paul Cohen in the 1960’s —namely, the ZFC axioms are not sufficient to settle the Continuum Hypothesis: there are two collections of sets, each obeying the laws of ZFC, and in one collection the Continuum Hypothesis is true, and in the other it is false. So settling the Continuum Hypothesis requires a new understanding of what Sets should be to arrive at persuasive new axioms that extend ZFC and are strong enough to determine the truth of the Continuum Hypothesis one way or the other.

- But even if we use more or different axioms about sets, there are some unavoidable problems. In the 1930’s, Gödel proved that, assuming that an axiom system like ZFC is consistent —meaning you can’t prove both P and $\overline{P}$ for any proposition, P —then the very proposition that the system is consistent (which is not too hard to express as a logical formula) cannot be proved in the system. In other words, no consistent system is strong enough to verify itself.

5.5.1 Large Infinities in Computer Science

If the romance of different size infinities and continuum hypotheses doesn’t appeal to you, not knowing about them is not going to limit you as a computer scientist. These abstract issues about infinite sets rarely come up in mainstream mathematics, and they don’t come up at all in computer science, where the focus is generally on “countable,” and often just finite, sets. In practice, only logicians and set theorists have to worry about collections that are “too big” to be sets. That’s part of the reason that the 19th century mathematical community made jokes about “Cantor’s paradise” of obscure infinite sets. But the challenge of reasoning correctly about this far out stuff led directly to the profound discoveries about the logical limits of computation described in Section 5.3, and that really is something every computer scientist should understand.

Problems for Section 5.2

Practice Problems

Problem 5.1.

Prove that if A and B are countable sets, then so is $A \cup B$.

Problem 5.2.

Let $A = \{a_0, a_1, \dots, a_{n-1}\}$ be a set of size n , and $B = \{b_0, b_1, \dots, b_{m-1}\}$ a set of size m . Prove that $|A \times B| = mn$ by defining a simple bijection from $A \times B$ to the nonnegative integers from 0 to $mn - 1$.

Problem 5.3.

Show that the set $\{0, 1\}^*$ of finite binary strings is countable.

Class Problems

Problem 5.4. (a) Several students felt the proof of Lemma 5.2.3 was worrisome, if not circular. What do you think?

Lemma 5.2.3. Let A be a set and $b \notin A$. If A is infinite, then there is a bijection from $A \cup \{b\}$ to A .

Proof. Here’s how to define the bijection: since A is infinite, it certainly has at least one element; call it a_0 . But since A is infinite, it has at least two elements, and one of them must not be equal to a_0 ; call this new element a_1 . But since A is infinite, it has at least three elements, one of which must not equal a_0 or a_1 ; call this new element a_2 . Continuing in the way, we conclude that there is an infinite sequence $a_0, a_1, a_2, \dots, a_n, \dots$ of different elements of A . Now we can define a bijection $f : A \cup \{b\} \rightarrow A$:

$$\begin{aligned} f(b) &::= a_0, \\ f(a_n) &::= a_{n+1} && \text{for } n \in \mathbb{N}, \\ f(a) &::= a && \text{for } a \in A - \{a_0, a_1, \dots\}. \end{aligned}$$

■

(b) Use the proof of Lemma 5.2.3 to show that if A is an infinite set, then $A \text{ surj } \mathbb{N}$, that is, every infinite set is “as big as” the set of nonnegative integers.

Problem 5.5.

Let $R : A \rightarrow B$ be a binary relation. Use an arrow counting argument to prove the following generalization of the Mapping Rule 1.

Lemma. If R is a function, and $X \subseteq A$, then

$$|X| \geq |R(X)|.$$

Problem 5.6.

The rational numbers fill the space between integers, so a first thought is that there must be more of them than the integers, but it’s not true. In this problem you’ll show that there are the same number of positive rationals as positive integers. That is, the positive rationals are countable.

(a) Define a bijection between the set, $\mathbb{Z}^+$, of positive integers, and the set, $(\mathbb{Z}^+ \times \mathbb{Z}^+)$, of all pairs of positive integers:

$$\begin{array}{l} (1, 1), (1, 2), (1, 3), (1, 4), (1, 5), \dots \\ (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), \dots \\ (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), \dots \\ (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), \dots \\ (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), \dots \\ \vdots \end{array}$$

(b) Conclude that the set, $\mathbb{Q}^+$, of all positive rational numbers is countable.

Problem 5.7.

This problem provides a proof of the [Schröder-Bernstein] Theorem:

$$\text{If } A \text{ surj } B \text{ and } B \text{ surj } A, \text{ then } A \text{ bij } B. \quad (5.9)$$

(a) It is OK to assume that A and B are disjoint. Why?

(b) Explain why there are total injective functions $f : A \rightarrow B$, and $g : B \rightarrow A$.

Picturing the diagrams for f and g , there is *exactly one* arrow *out* of each element—a left-to-right f -arrow if the element is in A and a right-to-left g -arrow if the element is in B . This is because f and g are total functions. Also, there is *at most one* arrow *into* any element, because f and g are injections.

So starting at any element, there is a unique, and unending path of arrows going forwards. There is also a unique path of arrows going backwards, which might be unending, or might end at an element that has no arrow into it. These paths are completely separate: if two ran into each other, there would be two arrows into the element where they ran together.

This divides all the elements into separate paths of four kinds:

- i. paths that are infinite in both directions,
- ii. paths that are infinite going forwards starting from some element of A .
- iii. paths that are infinite going forwards starting from some element of B .
- iv. paths that are unending but finite.

(c) What do the paths of the last type (iv) look like?

(d) Show that for each type of path, either

- the f -arrows define a bijection between the A and B elements on the path, or
- the g -arrows define a bijection between B and A elements on the path, or
- both sets of arrows define bijections.

For which kinds of paths do both sets of arrows define bijections?

(e) Explain how to piece these bijections together to prove that A and B are the same size.

Problem 5.8.

If there is a surjective function (≤ 1 out, ≥ 1 in mapping) $f : \mathbb{N} \rightarrow S$, then S is countable.

Hint: A Computer Science proof involves filtering for duplicates.

Homework Problems

Problem 5.9.

Prove that if A is an infinite set and C is a countable set, then

$$A \text{ bij } A \cup C.$$

Hint: See Problem 5.4.

Problem 5.10.

In this problem you will prove a fact that may surprise you—or make you even more convinced that set theory is nonsense: the half-open unit interval is actually the *same size* as the nonnegative quadrant of the real plane!⁵ Namely, there is a bijection from $(0, 1]$ to $[0, \infty)^2$.

(a) Describe a bijection from $(0, 1]$ to $[0, \infty)$.

Hint: $1/x$ almost works.

(b) An infinite sequence of the decimal digits $\{0, 1, \dots, 9\}$ will be called *long* if it has infinitely many occurrences of some digit other than 0. Let L be the set of all such long sequences. Describe a bijection from L to the half-open real interval $(0, 1]$.

Hint: Put a decimal point at the beginning of the sequence.

⁵The half open unit interval, $(0, 1]$, is $\{r \in \mathbb{R} \mid 0 < r \leq 1\}$. Similarly, $[0, \infty) ::= \{r \in \mathbb{R} \mid r \geq 0\}$.

(c) Describe a surjective function from L to L^2 that involves alternating digits from two long sequences. a *Hint*: The surjection need not be total.

(d) Prove the following lemma and use it to conclude that there is a bijection from L^2 to $(0, 1]^2$.

Lemma 5.5.1. *Let A and B be nonempty sets. If there is a bijection from A to B , then there is also a bijection from $A \times A$ to $B \times B$.*

(e) Conclude from the previous parts that there is a surjection from $(0, 1]$ and $(0, 1]^2$. Then appeal to the Schröder-Bernstein Theorem to show that there is actually a bijection from $(0, 1]$ and $(0, 1]^2$.

(f) Complete the proof that there is a bijection from $(0, 1]$ to $[0, \infty)^2$.

Exam Problems

Problem 5.11.

Prove that if $A_0, A_1, \dots, A_n, \dots$ is an infinite sequence of countable sets, then so is

$$\bigcup_{n=0}^{\infty} A_n$$

Problem 5.12.

Let A and B denote two countably infinite sets:

$$A = \{a_0, a_1, a_2, a_3, \dots\}$$

$$B = \{b_0, b_1, b_2, b_3, \dots\}$$

Show that their product, $A \times B$, is also a countable set by showing how to list the elements of $A \times B$. You need only show enough of the initial terms in your sequence to make the pattern clear—a half dozen or so terms usually suffice.

Problem 5.13. (a) Prove that if A and B are countable sets, then so is $A \cup B$.

(b) Prove that if C is a countable set and D is infinite, then there is a bijection between D and $C \cup D$.

a

Problem 5.14.

Let $\{0, 1\}^\omega$ be the uncountable set of infinite binary sequences, and let $F_n \subset \{0, 1\}^\omega$ be the set of infinite binary sequences whose bits are all 0 after the n th bit. That is, if $\mathbf{s} ::= (s_0, s_1, s_2, \dots) \in \{0, 1\}^\omega$, then

$$\mathbf{s} \in F_n \text{ IFF } \forall i > n. s_i = 0.$$

For example, the sequence $\mathbf{t}$ that starts 001101 with 0's after that is in F_5 , since by definition $t_i = 0$ for all $i > 5$. In fact, $\mathbf{t}$ is by definition also in $F_6, F_7, \dots$

(a) What is the size, $|F_n|$, of F_n ?

(b) Explain why the set $F \subset \{0, 1\}^\omega$ of sequences with only finitely many 1's, is a countable set. (You may assume without proof any results from class about countability.)

(c) Prove that the set of infinite binary sequences with infinitely many 1's is uncountable. *Hint:* Use parts (a) and (b); a direct proof by diagonalization is tricky.

Problems for Section 5.3

Class Problems

Problem 5.15.

Let $\mathbb{N}^\omega$ be the set of infinite sequences of nonnegative integers. For example, some sequences of this kind are:

$$\begin{aligned} (0, 1, 2, 3, 4, \dots), \\ (2, 3, 5, 7, 11, \dots), \\ (3, 1, 4, 5, 9, \dots). \end{aligned}$$

Prove that this set of sequences is uncountable.

Problem 5.16.

There are lots of different sizes of infinite sets. For example, starting with the infinite set, $\mathbb{N}$, of nonnegative integers, we can build the infinite sequence of sets

$$\mathbb{N} \text{ strict } \mathcal{P}(\mathbb{N}) \text{ strict } \mathcal{P}(\mathcal{P}(\mathbb{N})) \text{ strict } \mathcal{P}(\mathcal{P}(\mathcal{P}(\mathbb{N}))) \text{ strict } \dots$$

where each set is “strictly smaller” than the next one by Theorem 5.2.6. Let $\mathcal{P}^n(\mathbb{N})$ be the n th set in the sequence, and

$$U ::= \bigcup_{n=0}^{\infty} \mathcal{P}^n(\mathbb{N}).$$

Prove that

$$\mathcal{P}^n(\mathbb{N}) \text{ strict } U$$

for all $n \in \mathbb{N}$.

Now of course, we could take $U, \mathcal{P}(U), \mathcal{P}(\mathcal{P}(U)), \dots$ and keep on in this way building still bigger infinities indefinitely.

Problem 5.17.

The method used to prove Cantor’s Theorem that the power set is “bigger” than the set, leads to many important results in logic and computer science. In this problem we’ll apply that idea to describe a set of binary strings that can’t be described by ordinary logical formulas. To be provocative, we could say that we will describe an undescribable set of strings!

The following logical formula illustrates how a formula can describe a set of strings. The formula

$$\text{NOT}[\exists y. \exists z. s = y1z], \quad (\text{no-1s}(s))$$

where the variables range over the set, $\{0, 1\}^*$, of finite binary strings, says that the binary string, s , does not contain a 1.

We’ll call such a predicate formula, $G(s)$, about strings a *string formula*, and we’ll use the notation $\text{strings}(G)$ for the set of binary strings with the property described by G . That is,

$$\text{strings}(G) ::= \{s \in \{0, 1\}^* \mid G(s)\}.$$

A set of binary strings is *describable* if it equals $\text{strings}(G)$ for some string formula, G . So the set, 0^* , of finite strings of 0’s is describable because it equals $\text{strings}(\text{no-1s})$.⁶

The idea of representing data in binary is a no-brainer for a computer scientist, so it won’t be a stretch to agree that any string formula can be represented by a binary string. We’ll use the notation G_x for the string formula with binary representation

⁶no-1s and similar formulas were examined in Problem 3.20, but it is not necessary to have done that problem to do this one.

$x \in \{0, 1\}^*$. The details of the representation don’t matter, except that there ought to be a display procedure that can actually display G_x given x .

Standard binary representations of formulas are often based on character-by-character translation into binary, which means that only a sparse set of binary strings actually represent string formulas. It will be technically convenient to have *every* binary string represent some string formula. This is easy to do: tweak the display procedure so it displays some default formula, say no-1s, when it gets a binary string that isn’t a standard representation of a string formula. With this tweak, *every* binary string, x , will now represent a string formula, G_x .

Now we have just the kind of situation where a Cantor-style diagonal argument can be applied, namely, we’ll ask whether a string describes a property of *itself*! That may sound like a mind-bender, but all we’re asking is whether $x \in \text{strings}(G_x)$.

For example, using character-by-character translations of formulas into binary, neither the string 0000 nor the string 10 would be the binary representation of a formula, so the display procedure applied to either of them would display no-1s. That is, $G_{0000} = G_{10} = \text{no-1s}$ and so $\text{strings}(G_{0000}) = \text{strings}(G_{10}) = 0^*$. This means that

$$0000 \in \text{strings}(G_{0000}) \quad \text{and} \quad 10 \notin \text{strings}(G_{10}).$$

Now we are in a position to give a precise mathematical description of an “undescribable” set of binary strings, namely, let

Theorem. *Define*

$$U ::= \{x \in \{0, 1\}^* \mid x \notin \text{strings}(G_x)\}. \quad (5.10)$$

The set U is not describable.

Use reasoning similar to Cantor’s Theorem 5.2.6 to prove this Theorem.

Homework Problems

Problem 5.18.

For any sets, A , and B , let $[A \rightarrow B]$ be the set of total functions from A to B . Prove that if A is not empty and B has more than one element, then $\text{NOT}(A \text{ surj } [A \rightarrow B])$.

Hint: Suppose that σ is a function from A to $[A \rightarrow B]$ mapping each element $a \in A$ to a function $\sigma_a : A \rightarrow B$. Pick any two elements of B ; call them 0 and 1. Then define

$$\text{diag}(a) ::= \begin{cases} 0 & \text{if } \sigma_a(a) = 1, \\ 1 & \text{otherwise.} \end{cases}$$

Exam Problems

Problem 5.19.

Let $\{1, 2, 3\}^\omega$ be the set of infinite sequences containing only the numbers 1, 2, and 3. For example, some sequences of this kind are:

(1, 1, 1, 1...),

(2, 2, 2, 2...),

(3, 2, 1, 3...).

Prove that $\{1, 2, 3\}^\omega$ is uncountable.

Hint: One approach is to define a surjective function from $\{1, 2, 3\}^\omega$ to the power set $\mathcal{P}(\mathbb{N})$.

Problems for Section 5.4

Class Problems

Problem 5.20.

The Axiom of Choice says that if s is a set whose members are nonempty sets that are *pairwise disjoint* —that is no two sets in s have an element in common —then there is a set, c , consisting of exactly one element from each set in s .

In formal logic, we could describe s with the formula,

$\text{pairwise-disjoint}(s) ::= \forall x \in s. x \neq \emptyset \text{ AND } \forall x, y \in s. x \neq y \text{ IMPLIES } x \cap y = \emptyset.$

Similarly we could describe c with the formula

$\text{choice-set}(c, s) ::= \forall x \in s. \exists! z. z \in c \cap x.$

Here “ $\exists! z.$ ” is fairly standard notation for “there exists a *unique* $z.$ ”

Now we can give the formal definition:

Definition (Axiom of Choice).

$\forall s. \text{pairwise-disjoint}(s) \text{ IMPLIES } \exists c. \text{choice-set}(c, s).$

The only issue here is that Set Theory is technically supposed to be expressed in terms of *pure* formulas in the language of sets, which means formula that uses only the membership relation, $\in$, propositional connectives, the two quantifiers $\forall$ and $\exists$, and variables ranging over all sets. Verify that the Axiom of Choice can be expressed as a pure formula, by explaining how to replace all impure subformulas above with equivalent pure formulas.

For example, the formula $x = y$ could be replaced with the pure formula $\forall z. z \in x \text{ IFF } z \in y.$

Problem 5.21.

Let $R : A \rightarrow A$ be a binary relation on a set, A . If $a_1 R a_0$, we'll say that a_1 is “ R -smaller” than a_0 . R is called *well founded* when there is no infinite “ R -decreasing” sequence:

$$\cdots R a_n R \cdots R a_1 R a_0, \quad (5.11)$$

of elements $a_i \in A$.

For example, if $A = \mathbb{N}$ and R is the $<$ -relation, then R is well founded because if you keep counting down with nonnegative integers, you eventually get stuck at zero:

$$0 < \cdots < n - 1 < n.$$

But you can keep counting up forever, so the $>$ -relation is not well founded:

$$\cdots > n > \cdots > 1 > 0.$$

Also, the $\leq$ -relation on $\mathbb{N}$ is not well founded because a *constant* sequence of, say, 2's, gets $\leq$ -smaller forever:

$$\cdots \leq 2 \leq \cdots \leq 2 \leq 2.$$

(a) If B is a subset of A , an element $b \in B$ is defined to be *R -minimal in B* iff there is no R -smaller element in B . Prove that $R : A \rightarrow A$ is well founded iff every nonempty subset of A has an R -minimal element.

A logic *formula of set theory* has only predicates of the form “ $x \in y$ ” for variables x, y ranging over sets, along with quantifiers and propositional operations. For example,

$$\text{isempty}(x) ::= \forall w. \text{NOT}(w \in x)$$

is a formula of set theory that means that “ x is empty.”

(b) Write a formula, $\text{member-minimal}(u, v)$, of set theory that means that u is $\in$ -minimal in v .

(c) The Foundation axiom of set theory says that $\in$ is a well founded relation on sets. Express the Foundation axiom as a formula of set theory. You may use “member-minimal” and “isempty” in your formula as abbreviations for the formulas defined above.

(d) Explain why the Foundation axiom implies that no set is a member of itself.

6 Induction

Induction is a powerful method for showing a property is true for all nonnegative integers. Induction plays a central role in discrete mathematics and computer science, and in fact, its use is a defining characteristic of *discrete* —as opposed to *continuous* —mathematics. This chapter introduces two versions of induction — Ordinary and Strong —and explains why they work and how to use them in proofs. It also introduces the Invariant Principle, which is a version of induction specially adapted for reasoning about step-by-step processes.

6.1 Ordinary Induction

To understand how induction works, suppose there is a professor who brings to class a bottomless bag of assorted miniature candy bars. She offers to share the candy in the following way. First, she lines the students up in order. Next she states two rules:

1. The student at the beginning of the line gets a candy bar.
2. If a student gets a candy bar, then the following student in line also gets a candy bar.

Let’s number the students by their order in line, starting the count with 0, as usual in computer science. Now we can understand the second rule as a short description of a whole sequence of statements:

- If student 0 gets a candy bar, then student 1 also gets one.
- If student 1 gets a candy bar, then student 2 also gets one.
- If student 2 gets a candy bar, then student 3 also gets one.

⋮

Of course this sequence has a more concise mathematical description:

If student n gets a candy bar, then student $n + 1$ gets a candy bar, for all nonnegative integers n .

So suppose you are student 17. By these rules, are you entitled to a miniature candy bar? Well, student 0 gets a candy bar by the first rule. Therefore, by the second rule, student 1 also gets one, which means student 2 gets one, which means student 3 gets one as well, and so on. By 17 applications of the professor’s second rule, you get your candy bar! Of course the rules actually guarantee a candy bar to *every* student, no matter how far back in line they may be.

6.1.1 A Rule for Ordinary Induction

The reasoning that led us to conclude that every student gets a candy bar is essentially all there is to induction.

The Principle of Induction.

Let P be a predicate on nonnegative integers. If

- $P(0)$ is true, and
- $P(n)$ IMPLIES $P(n + 1)$ for all nonnegative integers, n ,

then

- $P(m)$ is true for all nonnegative integers, m .

Since we’re going to consider several useful variants of induction in later sections, we’ll refer to the induction method described above as *ordinary induction* when we need to distinguish it. Formulated as a proof rule as in Section 1.4.1, this would be

Rule. Induction Rule

$$\frac{P(0), \quad \forall n \in \mathbb{N}. P(n) \text{ IMPLIES } P(n + 1)}{\forall m \in \mathbb{N}. P(m)}$$

This general induction rule works for the same intuitive reason that all the students get candy bars, and we hope the explanation using candy bars makes it clear why the soundness of the ordinary induction can be taken for granted. In fact, the rule is so obvious that it’s hard to see what more basic principle could be used to justify it.¹ What’s not so obvious is how much mileage we get by using it.

¹But see Section 6.3.

6.1.2 A Familiar Example

The formula (6.1) below for the sum of the nonnegative integers up to n is the kind of statement about all nonnegative integers to which induction applies directly. We already proved it (Theorem 2.2.1) using the Well Ordering Principle, but now we’ll prove it using induction.

Theorem. For all $n \in \mathbb{N}$,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} \quad (6.1)$$

To use the Induction Principle to prove the Theorem, define predicate $P(n)$ to be the equation (6.1). Now the theorem can be restated as the claim that $P(n)$ is true for all $n \in \mathbb{N}$. This is great, because the induction principle lets us reach precisely that conclusion, provided we establish two simpler facts:

- $P(0)$ is true.
- For all $n \in \mathbb{N}$, $P(n)$ IMPLIES $P(n+1)$.

So now our job is reduced to proving these two statements. The first is true because $P(0)$ asserts that a sum of zero terms is equal to $0(0+1)/2 = 0$, which is true by definition.

The second statement is more complicated. But remember the basic plan from Section 1.5 for proving the validity of any implication: *assume* the statement on the left and then *prove* the statement on the right. In this case, we assume $P(n)$ —namely, equation (6.1)—in order to prove $P(n+1)$, which is the equation

$$1 + 2 + 3 + \cdots + n + (n+1) = \frac{(n+1)(n+2)}{2}. \quad (6.2)$$

These two equations are quite similar; in fact, adding $(n+1)$ to both sides of equation (6.1) and simplifying the right side gives the equation (6.2):

$$\begin{aligned} 1 + 2 + 3 + \cdots + n + (n+1) &= \frac{n(n+1)}{2} + (n+1) \\ &= \frac{(n+2)(n+1)}{2} \end{aligned}$$

Thus, if $P(n)$ is true, then so is $P(n+1)$. This argument is valid for every nonnegative integer n , so this establishes the second fact required by the induction principle. Therefore, the induction principle says that the predicate $P(m)$ is true for all nonnegative integers, m , so the theorem is proved.

6.1.3 A Template for Induction Proofs

The proof of equation (6.1) was relatively simple, but even the most complicated induction proof follows exactly the same template. There are five components:

1. **State that the proof uses induction.** This immediately conveys the overall structure of the proof, which helps your reader follow your argument.
2. **Define an appropriate predicate $P(n)$.** The predicate $P(n)$ is called the *induction hypothesis*. The eventual conclusion of the induction argument will be that $P(n)$ is true for all nonnegative n . Clearly stating the induction hypothesis is often the most important part of an induction proof, and omitting it is the largest source of confused proofs by students.

In the simplest cases, the induction hypothesis can be lifted straight from the proposition you are trying to prove, as we did with equation (6.1). Sometimes the induction hypothesis will involve several variables, in which case you should indicate which variable serves as n .

3. **Prove that $P(0)$ is true.** This is usually easy, as in the example above. This part of the proof is called the *base case* or *basis step*.
4. **Prove that $P(n)$ implies $P(n + 1)$ for every nonnegative integer n .** This is called the *inductive step*. The basic plan is always the same: assume that $P(n)$ is true and then use this assumption to prove that $P(n + 1)$ is true. These two statements should be fairly similar, but bridging the gap may require some ingenuity. Whatever argument you give must be valid for every nonnegative integer n , since the goal is to prove the implications $P(0) \rightarrow P(1)$, $P(1) \rightarrow P(2)$, $P(2) \rightarrow P(3)$, etc. all at once.
5. **Invoke induction.** Given these facts, the induction principle allows you to conclude that $P(n)$ is true for all nonnegative n . This is the logical capstone to the whole argument, but it is so standard that it's usual not to mention it explicitly.

Always be sure to explicitly label the *base case* and the *inductive step*. It will make your proofs clearer, and it will decrease the chance that you forget a key step (such as checking the base case).

6.1.4 A Clean Writeup

The proof of the Theorem given above is perfectly valid; however, it contains a lot of extraneous explanation that you won't usually see in induction proofs. The

writup below is closer to what you might see in print and should be prepared to produce yourself.

Revised proof of the Theorem. We use induction. The induction hypothesis, $P(n)$, will be equation (6.1).

Base case: $P(0)$ is true, because both sides of equation (6.1) equal zero when $n = 0$.

Inductive step: Assume that $P(n)$ is true, where n is any nonnegative integer. Then

$$\begin{aligned} 1 + 2 + 3 + \cdots + n + (n + 1) &= \frac{n(n + 1)}{2} + (n + 1) \quad (\text{by induction hypothesis}) \\ &= \frac{(n + 1)(n + 2)}{2} \quad (\text{by simple algebra}) \end{aligned}$$

which proves $P(n + 1)$.

So it follows by induction that $P(n)$ is true for all nonnegative n . ■

Induction was helpful for *proving the correctness* of this summation formula, but not helpful for *discovering* it in the first place. Tricks and methods for finding such formulas will be covered in Part III of the text.

6.1.5 A More Challenging Example

During the development of MIT’s famous Stata Center, as costs rose further and further beyond budget, there were some radical fundraising ideas. One rumored plan was to install a big courtyard with dimensions $2^n \times 2^n$ with one of the central squares² occupied by a statue of a wealthy potential donor—who we will refer to as “Bill”, for the purposes of preserving anonymity. The $n = 3$ case is shown in Figure 6.1.

A complication was that the building’s unconventional architect, Frank Gehry, was alleged to require that only special L-shaped tiles (shown in Figure 6.2) be used for the courtyard. For $n = 2$, a courtyard meeting these constraints is shown in Figure 6.3. But what about for larger values of n ? Is there a way to tile a $2^n \times 2^n$ courtyard with L-shaped tiles around a statue in the center? Let’s try to prove that this is so.

Theorem 6.1.1. *For all $n \geq 0$ there exists a tiling of a $2^n \times 2^n$ courtyard with Bill in a central square.*

²In the special case $n = 0$, the whole courtyard consists of a single central square; otherwise, there are four central squares.

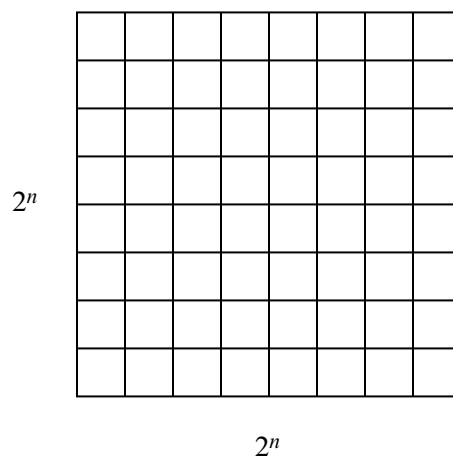


Figure 6.1 A $2^n \times 2^n$ courtyard for $n = 3$.

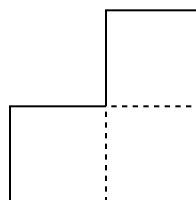


Figure 6.2 The special L-shaped tile.

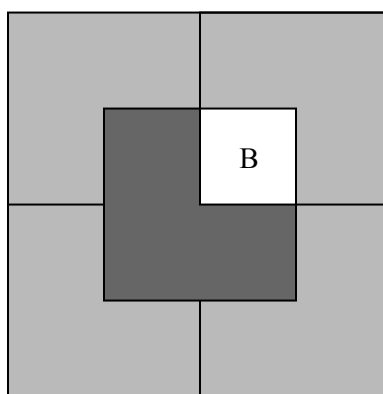


Figure 6.3 A tiling using L-shaped tiles for $n = 2$ with Bill in a center square.

Proof. (doomed attempt) The proof is by induction. Let $P(n)$ be the proposition that there exists a tiling of a $2^n \times 2^n$ courtyard with Bill in the center.

Base case: $P(0)$ is true because Bill fills the whole courtyard.

Inductive step: Assume that there is a tiling of a $2^n \times 2^n$ courtyard with Bill in the center for some $n \geq 0$. We must prove that there is a way to tile a $2^{n+1} \times 2^{n+1}$ courtyard with Bill in the center . . . ■

Now we’re in trouble! The ability to tile a smaller courtyard with Bill in the center isn’t much help in tiling a larger courtyard with Bill in the center. We haven’t figured out how to bridge the gap between $P(n)$ and $P(n + 1)$.

So if we’re going to prove Theorem 6.1.1 by induction, we’re going to need some *other* induction hypothesis than simply the statement about n that we’re trying to prove.

When this happens, your first fallback should be to look for a *stronger* induction hypothesis; that is, one which implies your previous hypothesis. For example, we could make $P(n)$ the proposition that for *every* location of Bill in a $2^n \times 2^n$ courtyard, there exists a tiling of the remainder.

This advice may sound bizarre: “If you can’t prove something, try to prove something grander!” But for induction arguments, this makes sense. In the inductive step, where you have to prove $P(n)$ IMPLIES $P(n + 1)$, you’re in better shape because you can *assume* $P(n)$, which is now a more powerful statement. Let’s see how this plays out in the case of courtyard tiling.

Proof (successful attempt). The proof is by induction. Let $P(n)$ be the proposition that for every location of Bill in a $2^n \times 2^n$ courtyard, there exists a tiling of the remainder.

Base case: $P(0)$ is true because Bill fills the whole courtyard.

Inductive step: Assume that $P(n)$ is true for some $n \geq 0$; that is, for every location of Bill in a $2^n \times 2^n$ courtyard, there exists a tiling of the remainder. Divide the $2^{n+1} \times 2^{n+1}$ courtyard into four quadrants, each $2^n \times 2^n$. One quadrant contains Bill (**B** in the diagram below). Place a temporary Bill (**X** in the diagram) in each of the three central squares lying outside this quadrant as shown in Figure 6.4.

Now we can tile each of the four quadrants by the induction assumption. Replacing the three temporary Bills with a single L-shaped tile completes the job. This proves that $P(n)$ implies $P(n + 1)$ for all $n \geq 0$. Thus $P(m)$ is true for all $m \in \mathbb{N}$, and the theorem follows as a special case where we put Bill in a central square. ■

This proof has two nice properties. First, not only does the argument guarantee that a tiling exists, but also it gives an algorithm for finding such a tiling. Second,

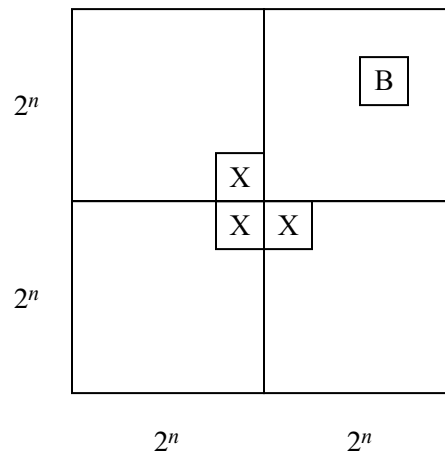


Figure 6.4 Using a stronger inductive hypothesis to prove Theorem 6.1.1.

we have a stronger result: if Bill wanted a statue on the edge of the courtyard, away from the pigeons, we could accommodate him!

Strengthening the induction hypothesis is often a good move when an induction proof won’t go through. But keep in mind that the stronger assertion must actually be *true*; otherwise, there isn’t much hope of constructing a valid proof! Sometimes finding just the right induction hypothesis requires trial, error, and insight. For example, mathematicians spent almost twenty years trying to prove or disprove the conjecture that “Every planar graph is 5-choosable”³. Then, in 1994, Carsten Thomassen gave an induction proof simple enough to explain on a napkin. The key turned out to be finding an extremely clever induction hypothesis; with that in hand, completing the argument was easy!

6.1.6 A Faulty Induction Proof

If we have done a good job in writing this text, right about now you should be thinking, “Hey, this induction stuff isn’t so hard after all —just show $P(0)$ is true and that $P(n)$ implies $P(n + 1)$ for any number n .” And, you would be right, although sometimes when you start doing induction proofs on your own, you can run into trouble. For example, we will now attempt to ruin your day by using induction to “prove” that all horses are the same color. And just when you thought it was safe to skip class and work on your robot program instead. Bummer!

³5-choosability is a slight generalization of 5-colorability. Although every planar graph is 4-colorable and therefore 5-colorable, not every planar graph is 4-choosable. If this all sounds like nonsense, don’t panic. We’ll discuss graphs, planarity, and coloring in Part II of the text.

False Theorem. *All horses are the same color.*

Notice that no n is mentioned in this assertion, so we’re going to have to re-formulate it in a way that makes an n explicit. In particular, we’ll (falsely) prove that

False Theorem 6.1.2. *In every set of $n \geq 1$ horses, all the horses are the same color.*

This is a statement about all integers $n \geq 1$ rather ≥ 0 , so it’s natural to use a slight variation on induction: prove $P(1)$ in the base case and then prove that $P(n)$ implies $P(n + 1)$ for all $n \geq 1$ in the inductive step. This is a perfectly valid variant of induction and is *not* the problem with the proof below.

Bogus proof. The proof is by induction on n . The induction hypothesis, $P(n)$, will be

$$\text{In every set of } n \text{ horses, all are the same color.} \quad (6.3)$$

Base case: ($n = 1$). $P(1)$ is true, because in a set of horses of size 1, there’s only one horse, and this horse is definitely the same color as itself.

Inductive step: Assume that $P(n)$ is true for some $n \geq 1$. That is, assume that in every set of n horses, all are the same color. Now suppose we have a set of $n + 1$ horses:

$$h_1, h_2, \dots, h_n, h_{n+1}.$$

We need to prove these $n + 1$ horses are all the same color.

By our assumption, the first n horses are the same color:

$$\underbrace{h_1, h_2, \dots, h_n}_{\text{same color}}, h_{n+1}$$

Also by our assumption, the last n horses are the same color:

$$h_1, \underbrace{h_2, \dots, h_n, h_{n+1}}_{\text{same color}}$$

So h_1 is the same color as the remaining horses besides h_{n+1} —that is, $h_2, \dots, h_n$. Likewise, h_{n+1} is the same color as the remaining horses besides h_1 —that is, $h_2, \dots, h_n$, again. Since h_1 and h_{n+1} are the same color as $h_2, \dots, h_n$, all $n + 1$ horses must be the same color, and so $P(n + 1)$ is true. Thus, $P(n)$ implies $P(n + 1)$.

By the principle of induction, $P(n)$ is true for all $n \geq 1$. ■

We’ve proved something false! Is math broken? Should we all become poets? No, this proof has a mistake.

The mistake in this argument is in the sentence that begins “So h_1 is the same color as the remaining horses besides h_{n+1} —that is $h_2, \dots, h_n, \dots$.” The “ $\dots$ ” notation in the expression “ $h_1, h_2, \dots, h_n, h_{n+1}$ ” creates the impression that there are some remaining horses—namely $h_2, \dots, h_n$ —besides h_1 and h_{n+1} . However, this is not true when $n = 1$. In that case, $h_1, h_2, \dots, h_n, h_{n+1}$ is just h_1, h_2 and there are no “remaining” horses for h_1 to share a color with. And of course in this case h_1 and h_2 really don’t need to be the same color.

This mistake knocks a critical link out of our induction argument. We proved $P(1)$ and we *correctly* proved $P(2) \rightarrow P(3)$, $P(3) \rightarrow P(4)$, etc. But we failed to prove $P(1) \rightarrow P(2)$, and so everything falls apart: we can not conclude that $P(2)$, $P(3)$, etc., are true. And, of course, these propositions are all false; there are sets of n horses of different colors for all $n \geq 2$.

Students sometimes explain that the mistake in the proof is because $P(n)$ is false for $n \geq 2$, and the proof assumes something false, namely, $P(n)$, in order to prove $P(n + 1)$. You should think about how to explain to such a student why this explanation would get no credit on a Math for Computer Science exam.

6.2 Strong Induction

A useful variant of induction is called *Strong Induction*. Strong induction and ordinary induction are used for exactly the same thing: proving that a predicate is true for all nonnegative integers. Strong induction is useful when a simple proof that the predicate holds for $n + 1$ does not follow just from the fact that it holds at n , but from the fact that it holds for other values $\leq n$.

6.2.1 A Rule for Strong Induction

Principle of Strong Induction.

Let P be a predicate on nonnegative integers. If

- $P(0)$ is true, and
- for all $n \in \mathbb{N}$, $P(0), P(1), \dots, P(n)$ together imply $P(n + 1)$,

then $P(m)$ is true for all $m \in \mathbb{N}$.

The only change from the ordinary induction principle is that strong induction allows you to assume more stuff in the inductive step of your proof! In an ordinary induction argument, you assume that $P(n)$ is true and try to prove that $P(n + 1)$ is also true. In a strong induction argument, you may assume that $P(0), P(1), \dots$, and $P(n)$ are *all* true when you go to prove $P(n + 1)$. These extra assumptions can only make your job easier. Hence the name: *strong* induction.

Formulated as a proof rule, strong induction is

Rule. Strong Induction Rule

$$\frac{P(0), \quad \forall n \in \mathbb{N}. (P(0) \text{ AND } P(1) \text{ AND } \dots \text{ AND } P(n)) \text{ IMPLIES } P(n + 1)}{\forall m \in \mathbb{N}. P(m)}$$

Stated more succinctly, the rule is

Rule.

$$\frac{P(0), \quad [\forall k \leq n \in \mathbb{N}. P(k)] \text{ IMPLIES } P(n + 1)}{\forall m \in \mathbb{N}. P(m)}$$

The template for strong induction proofs is identical to the template given in Section 6.1.3 for ordinary induction except for two things:

- you should state that your proof is by strong induction, and
- you can assume that $P(0), P(1), \dots, P(n)$ are all true instead of only $P(n)$ during the inductive step.

6.2.2 Products of Primes

As a first example, we’ll use strong induction to re-prove Theorem 2.3.1 which we previously proved using Well Ordering.

Theorem. *Every integer greater than 1 is a product of primes.*

Proof. We will prove the Theorem by strong induction, letting the induction hypothesis, $P(n)$, be

n is a product of primes.

So the Theorem will follow if we prove that $P(n)$ holds for all $n \geq 2$.

Base Case: ($n = 2$): $P(2)$ is true because 2 is prime, so it is a length one product of primes by convention.

Inductive step: Suppose that $n \geq 2$ and that k is a product of primes for every integer k where $2 \leq k \leq n$. We must show that $P(n + 1)$ holds, namely, that $n + 1$ is also a product of primes. We argue by cases:

If $n + 1$ is itself prime, then it is a length one product of primes by convention, and so $P(n + 1)$ holds in this case.

Otherwise, $n + 1$ is not prime, which by definition means $n + 1 = km$ for some integers k, m such that $2 \leq k, m \leq n$. Now by the strong induction hypothesis, we know that k is a product of primes. Likewise, m is a product of primes. By multiplying these products, it follows immediately that $km = n + 1$ is also a product of primes. Therefore, $P(n + 1)$ holds in this case as well.

So $P(n + 1)$ holds in any case, which completes the proof by strong induction that $P(n)$ holds for all $n \geq 2$. ■

6.2.3 Making Change

The country Inductia, whose unit of currency is the Strong, has coins worth 3Sg (3 Strong) and 5Sg. Although the Inductians have some trouble making small change like 4Sg or 7Sg, it turns out that they can collect coins to make change for any number that is at least 8 Strong.

Strong induction makes this easy to prove for $n + 1 \geq 11$, because then $(n + 1) - 3 \geq 8$, so by strong induction the Inductians can make change for exactly $(n + 1) - 3$ Strong, and then they can add a 3Sg coin to get $(n + 1)$ Sg. So the only thing to do is check that they can make change for all the amounts from 8 to 10Sg, which is not too hard to do.

Here’s a detailed writeup using the official format:

Proof. We prove by strong induction that the Inductians can make change for any amount of at least 8Sg. The induction hypothesis, $P(n)$ will be:

There is a collection of coins whose value is $n + 8$ Strong.

We now proceed with the induction proof:

Base case: $P(0)$ is true because a 3Sg coin together with a 5Sg coin makes 8Sg.

Inductive step: We assume $P(k)$ holds for all $k \leq n$, and prove that $P(n + 1)$ holds. We argue by cases:

Case $(n + 1 = 1)$: We have to make $(n + 1) + 8 = 9$ Sg. We can do this using three 3Sg coins.

Case $(n + 1 = 2)$: We have to make $(n + 1) + 8 = 10$ Sg. Use two 5Sg coins.

Case $(n + 1 \geq 3)$: Then $0 \leq n - 2 \leq n$, so by the strong induction hypothesis, the Inductians can make change for $n - 2$ Strong. Now by adding a 3Sg coin, they can make change for $(n + 1)$ Sg.

Since $n \geq 0$, we know that $n + 1 \geq 1$ and thus that the three cases cover every possibility. Since $P(n + 1)$ is true in every case, we can conclude by strong

Stack Heights										Score
<u>10</u>										
5	<u>5</u>									25 points
<u>5</u>	3	2								6
<u>4</u>	3	2	1							4
2	<u>3</u>	2	1	2						4
<u>2</u>	2	2	1	2	1					2
1	<u>2</u>	2	1	2	1	1				1
1	1	<u>2</u>	1	2	1	1	1			1
1	1	1	1	<u>2</u>	1	1	1	1		1
1	1	1	1	1	1	1	1	1	1	1
Total Score										= 45 points

Figure 6.5 An example of the stacking game with $n = 10$ boxes. On each line, the underlined stack is divided in the next step.

induction that for all $n \geq 0$, the Inductionians can make change for $n + 8$ Strong. That is, they can make change for any number of eight or more Strong. ■

6.2.4 The Stacking Game

Here is another exciting game that’s surely about to sweep the nation!

You begin with a stack of n boxes. Then you make a sequence of moves. In each move, you divide one stack of boxes into two nonempty stacks. The game ends when you have n stacks, each containing a single box. You earn points for each move; in particular, if you divide one stack of height $a + b$ into two stacks with heights a and b , then you score ab points for that move. Your overall score is the sum of the points that you earn for each move. What strategy should you use to maximize your total score?

As an example, suppose that we begin with a stack of $n = 10$ boxes. Then the game might proceed as shown in Figure 6.5. Can you find a better strategy?

Analyzing the Game

Let’s use strong induction to analyze the unstacking game. We’ll prove that your score is determined entirely by the number of boxes —your strategy is irrelevant!

Theorem 6.2.1. *Every way of unstacking n blocks gives a score of $n(n - 1)/2$ points.*

There are a couple technical points to notice in the proof:

- The template for a strong induction proof mirrors the one for ordinary induction.
- As with ordinary induction, we have some freedom to adjust indices. In this case, we prove $P(1)$ in the base case and prove that $P(1), \dots, P(n)$ imply $P(n + 1)$ for all $n \geq 1$ in the inductive step.

Proof. The proof is by strong induction. Let $P(n)$ be the proposition that every way of unstacking n blocks gives a score of $n(n - 1)/2$.

Base case: If $n = 1$, then there is only one block. No moves are possible, and so the total score for the game is $1(1 - 1)/2 = 0$. Therefore, $P(1)$ is true.

Inductive step: Now we must show that $P(1), \dots, P(n)$ imply $P(n + 1)$ for all $n \geq 1$. So assume that $P(1), \dots, P(n)$ are all true and that we have a stack of $n + 1$ blocks. The first move must split this stack into substacks with positive sizes a and b where $a + b = n + 1$ and $0 < a, b \leq n$. Now the total score for the game is the sum of points for this first move plus points obtained by unstacking the two resulting substacks:

$$\begin{aligned}
 \text{total score} &= (\text{score for 1st move}) \\
 &\quad + (\text{score for unstacking } a \text{ blocks}) \\
 &\quad + (\text{score for unstacking } b \text{ blocks}) \\
 &= ab + \frac{a(a - 1)}{2} + \frac{b(b - 1)}{2} && \text{by } P(a) \text{ and } P(b) \\
 &= \frac{(a + b)^2 - (a + b)}{2} = \frac{(a + b)((a + b) - 1)}{2} \\
 &= \frac{(n + 1)n}{2}
 \end{aligned}$$

This shows that $P(1), P(2), \dots, P(n)$ imply $P(n + 1)$.

Therefore, the claim is true by strong induction. ■

6.3 Strong Induction vs. Induction vs. Well Ordering

Strong induction looks genuinely “stronger” than ordinary induction —after all, you can assume a lot more when proving the induction step. Since ordinary induction is a special case of strong induction, you might wonder why anyone would bother with the ordinary induction.

But strong induction really isn’t any stronger, because a simple text manipulation program can automatically reformat any proof using strong induction into a proof using ordinary induction —just by decorating the induction hypothesis with a universal quantifier in a standard way. Still, it’s worth distinguishing these two kinds of induction, since which you use will signal whether the inductive step for $n + 1$ follows directly from the case for n or requires cases smaller than n , and that is generally good for your reader to know.

The template for the two kinds of induction rules looks nothing like the one for the Well Ordering Principle, but this chapter included a couple of examples where induction was used to prove something already proved using Well Ordering. In fact, this can always be done. As the examples may suggest, any Well Ordering proof can automatically be reformatted into an Induction proof. So theoretically, no one need bother with the Well Ordering Principle either.

But wait a minute —it’s equally easy to go the other way, and automatically reformat any Strong Induction proof into a Well Ordering proof. The three proof methods —Well Ordering, Induction, and Strong Induction, are simply different formats for presenting the same mathematical reasoning!

So why three methods? Well, sometimes induction proofs are clearer because they don’t require proof by contradiction. Also, induction proofs often provide recursive procedures that reduce handling large inputs to handling smaller ones. On the other hand, Well Ordering can come out slightly shorter and sometimes seem more natural (and less worrisome to beginners).

So which method should you use? —whichever you find easier! But whichever method you choose, be sure to state the method up front to help a reader follow your proof.

6.4 State Machines

State machines are a simple abstract model of step-by-step processes. Since computer programs can be understood as defining step-by-step computational processes, it’s not surprising that state machines come up regularly in computer science. They also come up in many other settings such as digital circuit design and modeling of probabilistic processes. This section introduces *Floyd’s Invariant Principle* which is a version of induction tailored specifically for proving properties of state machines.

One of the most important uses of induction in computer science involves proving one or more desirable properties continues to hold at every step in a process. A property that is preserved through a series of operations or steps is known as an

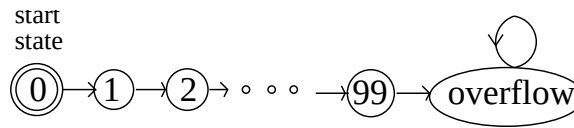


Figure 6.6 State transitions for the 99-bounded counter.

invariant. Examples of desirable invariants include properties such as a variable never exceeding a certain value, the altitude of a plane never dropping below 1,000 feet without the wingflaps being deployed, and the temperature of a nuclear reactor never exceeding the threshold for a meltdown.

6.4.1 States and Transitions

Formally, a state machine is nothing more than a binary relation on a set, except that the elements of the set are called “states,” the relation is called the *transition relation*, and an arrow in the graph of the transition relation is called a *transition*. A transition from state q to state r will be written $q \longrightarrow r$. The transition relation is also called the *state graph* of the machine. A state machine also comes equipped with a designated *start state*.

A simple example is a bounded counter, which counts from 0 to 99 and overflows at 100. This state machine is pictured in Figure 6.6, with states pictured as circles, transitions by arrows, and with start state 0 indicated by the double circle. To be precise, what the picture tells us is that this bounded counter machine has

$$\begin{aligned} \text{states} &::= \{0, 1, \dots, 99, \text{overflow}\}, \\ \text{start state} &::= 0, \\ \text{transitions} &::= \{n \longrightarrow n + 1 \mid 0 \leq n < 99\} \\ &\quad \cup \{99 \longrightarrow \text{overflow}, \text{overflow} \longrightarrow \text{overflow}\}. \end{aligned}$$

This machine isn’t much use once it overflows, since it has no way to get out of its overflow state.

State machines for digital circuits and string pattern matching algorithms, for example, usually have only a finite number of states. Machines that model continuing computations typically have an infinite number of states. For example, instead of the 99-bounded counter, we could easily define an “unbounded” counter that just keeps counting up without overflowing. The unbounded counter has an infinite state set, namely, the nonnegative integers, which makes its state diagram harder to draw. : –)

State machines are often defined with labels on states and/or transitions to indicate such things as input or output values, costs, capacities, or probabilities. Our

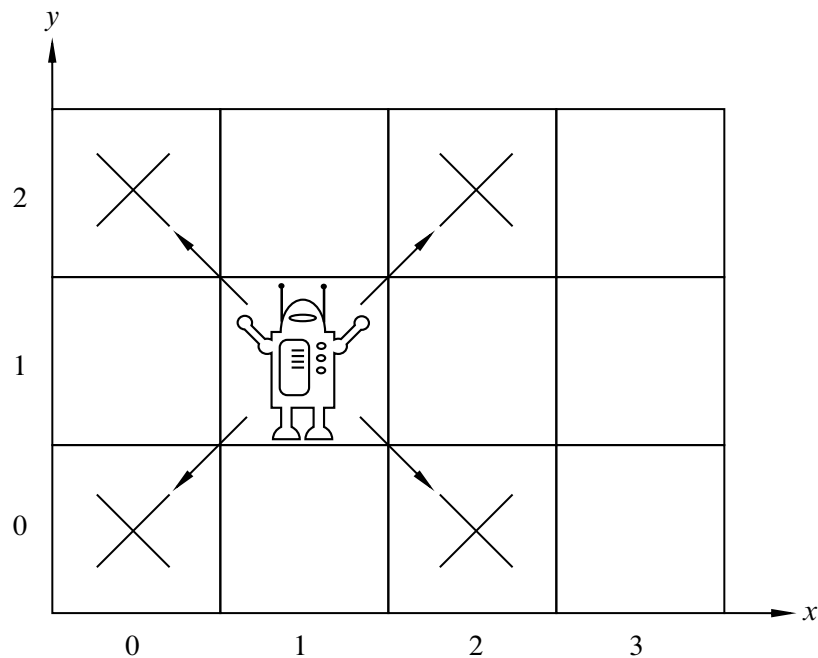


Figure 6.7 *The Diagonally Moving Robot.*

state machines don’t include any such labels because they aren’t needed for our purposes. We do name states, as in Figure 6.6, so we can talk about them, but the names aren’t part of the state machine.

6.4.2 Invariant for a Diagonally-Moving Robot

Suppose we have a robot that moves on an infinite 2-dimensional integer grid. The *state* of the robot at any time can be specified by the integer coordinates (x, y) of the robot’s current position. The *start state* is $(0, 0)$ since it is given that the robot starts at that position. At each step, the robot may move to a diagonally adjacent grid point, as illustrated in Figure 6.7.

To be precise, the robot’s transitions are:

$$\{(m, n) \longrightarrow (m \pm 1, n \pm 1) \mid m, n \in \mathbb{Z}\}.$$

For example, after the first step, the robot could be in states $(1, 1)$, $(1, -1)$, $(-1, 1)$, or $(-1, -1)$. After two steps, there are 9 possible states for the robot, including $(0, 0)$. We ask can the robot ever reach position $(1, 0)$?

If you play around with the robot a bit, you’ll probably notice that the robot can only reach positions (m, n) for which $m + n$ is even, which means, of course, that

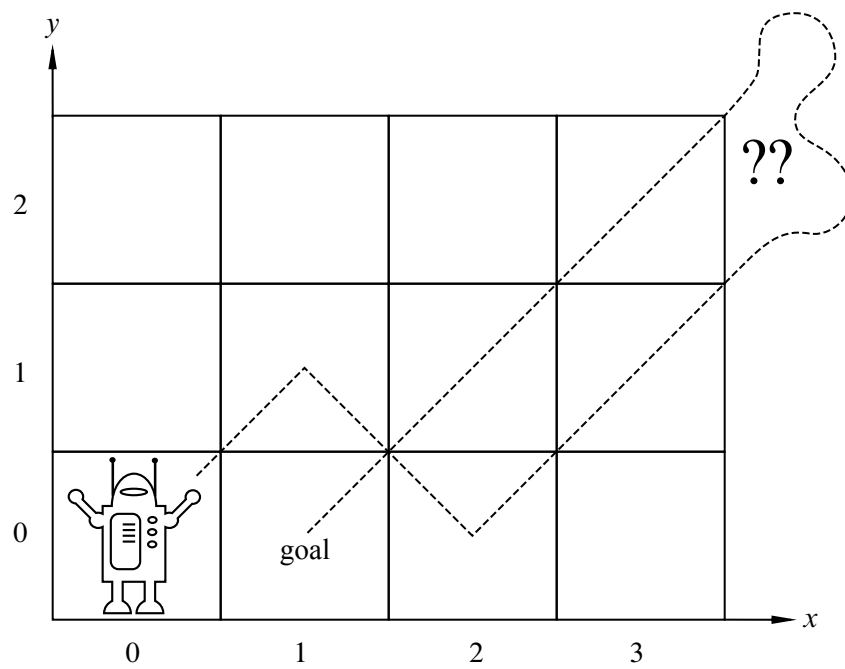


Figure 6.8 *Can the Robot get to (1,0)?*

it can't reach $(1, 0)$. This all follows because evenness of the sum of coordinates is preserved by transitions.

This once, let's go through this preserved-property argument again carefully highlighting where induction comes in. Namely, define the even-sum property of states to be:

$$\text{Even-sum}((m, n)) ::= [m + n \text{ is even}].$$

Lemma 6.4.1. *For any transition, $q \rightarrow r$, of the diagonally-moving robot, if $\text{Even-sum}(q)$, then $\text{Even-sum}(r)$.*

This lemma follows immediately from the definition of the robot's transitions: $(m, n) \rightarrow (m \pm 1, n \pm 1)$. After a transition, the sum of coordinates changes by $(\pm 1) + (\pm 1)$, that is, by 0, 2, or -2. Of course, adding 0, 2 or -2 to an even number gives an even number. So by a trivial induction on the number of transitions, we can prove:

Theorem 6.4.2. *The sum of the coordinates of any state reachable by the diagonally-moving robot is even.*

Proof. The proof is induction on the number of transitions the robot has made. The induction hypothesis is

$$P(n) ::= \text{if } q \text{ is a state reachable in } n \text{ transitions, then } \text{Even-sum}(q).$$

base case: $P(0)$ is true since the only state reachable in 0 transitions is the start state $(0, 0)$, and $0 + 0$ is even.

inductive step: Assume that $P(n)$ is true, and let r be any state reachable in $n + 1$ transitions. We need to prove that $\text{Even-sum}(r)$ holds.

Since r is reachable in $n + 1$ transitions, there must be a state, q , reachable in n transitions such that $q \rightarrow r$. Since $P(n)$ is assumed to be true, $\text{Even-sum}(q)$ holds, and so by Lemma 6.4.1, $\text{Even-sum}(r)$ also holds. This proves that $P(n)$ IMPLIES $P(n + 1)$ as required, completing the proof of the inductive step.

We conclude by induction that for all $n \geq 0$, if q is reachable in n transitions, then $\text{Even-sum}(q)$. This implies that every reachable state has the Even-sum property. ■

Corollary 6.4.3. *The robot can never reach position $(1, 0)$.*

Proof. By Theorem 6.4.2, we know the robot can only reach positions with coordinates that sum to an even number, and thus it cannot reach position $(1, 0)$. ■

6.4.3 The Invariant Principle

Using the Even-sum invariant to understand the diagonally-moving robot is a simple example of a basic proof method called The Invariant Principle. The Principle summarizes how induction on the number of steps to reach a state applies to invariants. To formulate it precisely, we need a definition of *reachability*.

Definition 6.4.4. The *reachable states* of a state machine, M , are defined recursively as follows:

- the start state is reachable, and
- if p is a reachable state of M , and $p \longrightarrow q$ is a transition of M , then q is also a reachable state of M .

Definition 6.4.5. A *preserved invariant* of a state machine is a predicate, P , on states, such that whenever $P(q)$ is true of a state, q , and $q \longrightarrow r$ for some state, r , then $P(r)$ holds.

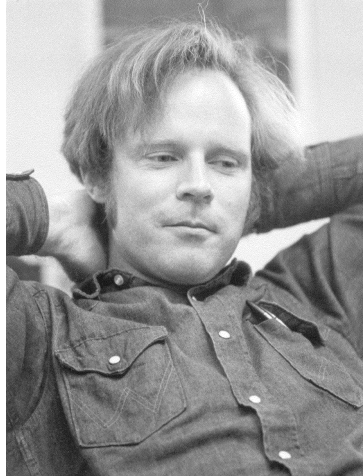
The Invariant Principle

If a preserved invariant of a state machine is true for the start state, then it is true for all reachable states.

The Invariant Principle is nothing more than the Induction Principle reformulated in a convenient form for state machines. Showing that a predicate is true in the start state is the base case of the induction, and showing that a predicate is a preserved invariant corresponds to the inductive step.⁴

⁴Preserved invariants are commonly just called “invariants” in the literature on program correctness, but we decided to throw in the extra adjective to avoid confusion with other definitions. For example, other texts (as well as another subject at MIT) use “invariant” to mean “predicate true of all reachable states.” Let’s call this definition “invariant-2.” Now invariant-2 seems like a reasonable definition, since unreachable states by definition don’t matter, and all we want to show is that a desired property is invariant-2. But this confuses the *objective* of demonstrating that a property is invariant-2 with the *method* of finding a *preserved* invariant to *show* that it is invariant-2.

Robert W Floyd



The Invariant Principle was formulated by Robert W Floyd at Carnegie Tech⁵ in 1967. Floyd was already famous for work on formal grammars that transformed the field of programming language parsing; that was how he got to be a professor even though he never got a Ph.D. (He was admitted to a PhD program as a teenage prodigy, but flunked out and never went back.)

In that same year, Albert R Meyer was appointed Assistant Professor in the Carnegie Tech Computer Science Department where he first met Floyd. Floyd and Meyer were the only theoreticians in the department, and they were both delighted to talk about their shared interests. After just a few conversations, Floyd’s new junior colleague decided that Floyd was the smartest person he had ever met.

Naturally, one of the first things Floyd wanted to tell Meyer about was his new, as yet unpublished, Invariant Principle. Floyd explained the result to Meyer, and Meyer wondered (privately) how someone as brilliant as Floyd could be excited by such a trivial observation. Floyd had to show Meyer a bunch of examples before Meyer understood Floyd’s excitement—not at the truth of the utterly obvious Invariant Principle, but rather at the insight that such a simple method could be so widely and easily applied in verifying programs.

Floyd left for Stanford the following year. He won the Turing award—the “Nobel prize” of computer science—in the late 1970’s, in recognition both of his work on grammars and on the foundations of program verification. He remained at Stanford from 1968 until his death in September, 2001. You can learn more about Floyd’s life and work by reading the [eulogy](#) written by his closest colleague, Don Knuth.

6.4.4 The Die Hard Example

The movie *Die Hard 3: With a Vengeance* includes an amusing example of a state machine. The lead characters played by Samuel L. Jackson and Bruce Willis have to disarm a bomb planted by the diabolical Simon Gruber:

Simon: On the fountain, there should be 2 jugs, do you see them? A 5-gallon and a 3-gallon. Fill one of the jugs with exactly 4 gallons of water and place it on the scale and the timer will stop. You must be precise; one ounce more or less will result in detonation. If you’re still alive in 5 minutes, we’ll speak.

Bruce: Wait, wait a second. I don’t get it. Do you get it?

Samuel: No.

Bruce: Get the jugs. Obviously, we can’t fill the 3-gallon jug with 4 gallons of water.

Samuel: Obviously.

Bruce: All right. I know, here we go. We fill the 3-gallon jug exactly to the top, right?

Samuel: Uh-huh.

Bruce: Okay, now we pour this 3 gallons into the 5-gallon jug, giving us exactly 3 gallons in the 5-gallon jug, right?

Samuel: Right, then what?

Bruce: All right. We take the 3-gallon jug and fill it a third of the way...

Samuel: No! He said, “Be precise.” Exactly 4 gallons.

Bruce: Sh - -. Every cop within 50 miles is running his a - - off and I’m out here playing kids games in the park.

Samuel: Hey, you want to focus on the problem at hand?

Fortunately, they find a solution in the nick of time. You can work out how.

The Die Hard 3 State Machine

The jug-filling scenario can be modeled with a state machine that keeps track of the amount, b , of water in the big jug, and the amount, l , in the little jug. With the 3 and 5 gallon water jugs, the states formally will be pairs, (b, l) of real numbers

such that $0 \leq b \leq 5, 0 \leq l \leq 3$. (We can prove that the reachable values of b and l will be nonnegative integers, but we won't assume this.) The start state is $(0, 0)$, since both jugs start empty.

Since the amount of water in the jug must be known exactly, we will only consider moves in which a jug gets completely filled or completely emptied. There are several kinds of transitions:

1. Fill the little jug: $(b, l) \longrightarrow (b, 3)$ for $l < 3$.
2. Fill the big jug: $(b, l) \longrightarrow (5, l)$ for $b < 5$.
3. Empty the little jug: $(b, l) \longrightarrow (b, 0)$ for $l > 0$.
4. Empty the big jug: $(b, l) \longrightarrow (0, l)$ for $b > 0$.
5. Pour from the little jug into the big jug: for $l > 0$,

$$(b, l) \longrightarrow \begin{cases} (b + l, 0) & \text{if } b + l \leq 5, \\ (5, l - (5 - b)) & \text{otherwise.} \end{cases}$$

6. Pour from big jug into little jug: for $b > 0$,

$$(b, l) \longrightarrow \begin{cases} (0, b + l) & \text{if } b + l \leq 3, \\ (b - (3 - l), 3) & \text{otherwise.} \end{cases}$$

Note that in contrast to the 99-counter state machine, there is more than one possible transition out of states in the Die Hard machine. Machines like the 99-counter with at most one transition out of each state are called *deterministic*. The Die Hard machine is *nondeterministic* because some states have transitions to several different states.

The Die Hard 3 bomb gets disarmed successfully because the state $(4, 3)$ is reachable.

Die Hard Once and For All

The *Die Hard* series is getting tired, so we propose a final *Die Hard Once and For All*. Here Simon's brother returns to avenge him, and he poses the same challenge, but with the 5 gallon jug replaced by a 9 gallon one. The state machine has the same specification as in Die Hard 3, with all occurrences of “5” replaced by “9.”

Now reaching any state of the form $(4, l)$ is impossible. We prove this using the Invariant Principle. Namely, we define the preserved invariant predicate, $P((b, l))$, to be that b and l are nonnegative integer multiples of 3.

To prove that P is a preserved invariant of Die-Hard-Once-and-For-All machine, we assume $P(q)$ holds for some state $q ::= (b, l)$ and that $q \longrightarrow r$. We have to show that $P(r)$ holds. The proof divides into cases, according to which transition rule is used.

One case is a “fill the little jug” transition. This means $r = (b, 3)$. But $P(q)$ implies that b is an integer multiple of 3, and of course 3 is an integer multiple of 3, so $P(r)$ still holds.

Another case is a “pour from big jug into little jug” transition. For the subcase when there isn’t enough room in the little jug to hold all the water, namely, when $b + l > 3$, we have $r = (b - (3 - l), 3)$. But $P(q)$ implies that b and l are integer multiples of 3, which means $b - (3 - l)$ is too, so in this case too, $P(r)$ holds.

We won’t bother to crank out the remaining cases, which can all be checked just as easily. Now by the Invariant Principle, we conclude that every reachable state satisfies P . But since no state of the form $(4, l)$ satisfies P , we have proved rigorously that Bruce dies once and for all!

By the way, notice that the state $(1, 0)$, which satisfies $\text{NOT}(P)$, has a transition to $(0, 0)$, which satisfies P . So the negation of a preserved invariant may not be a preserved invariant.

6.4.5 Fast Exponentiation

Partial Correctness & Termination

Floyd distinguished two required properties to verify a program. The first property is called *partial correctness*; this is the property that the final results, if any, of the process must satisfy system requirements.

You might suppose that if a result was only partially correct, then it might also be partially incorrect, but that’s not what Floyd meant. The word “partial” comes from viewing a process that might not terminate as computing a *partial relation*. Partial correctness means that *when there is a result*, it is correct, but the process might not always produce a result, perhaps because it gets stuck in a loop.

The second correctness property called *termination* is that the process does always produce some final value.

Partial correctness can commonly be proved using the Invariant Principle. Termination can commonly be proved using the Well Ordering Principle. We’ll illustrate this by verifying a Fast Exponentiation procedure.

Exponentiating

The most straightforward way to compute the b th power of a number, a , is to multiply a by itself $b - 1$ times. There is another way to do it using considerably fewer

multiplications called *Fast Exponentiation*. The register machine program below defines the fast exponentiation algorithm. The letters x, y, z, r denote registers that hold numbers. An *assignment statement* has the form “ $z := a$ ” and has the effect of setting the number in register z to be the number a .

A Fast Exponentiation Program

Given inputs $a \in \mathbb{R}, b \in \mathbb{N}$, initialize registers x, y, z to $a, 1, b$ respectively, and repeat the following sequence of steps until termination:

- if $z = 0$ **return** y and terminate
- $r := \text{remainder}(z, 2)$
- $z := \text{quotient}(z, 2)$
- if $r = 1$, then $y := xy$
- $x := x^2$

We claim this program always terminates and leaves $y = a^b$.

To begin, we’ll model the behavior of the program with a state machine:

1. states $::= \mathbb{R} \times \mathbb{R} \times \mathbb{N}$,
2. start state $::= (a, 1, b)$,
3. transitions are defined by the rule

$$(x, y, z) \longrightarrow \begin{cases} (x^2, y, \text{quotient}(z, 2)) & \text{if } z \text{ is nonzero and even,} \\ (x^2, xy, \text{quotient}(z, 2)) & \text{if } z \text{ is nonzero and odd.} \end{cases}$$

The preserved invariant, $P((x, y, z))$, will be

$$z \in \mathbb{N} \text{ AND } yx^z = a^b. \tag{6.4}$$

To prove that P is preserved, assume $P((x, y, z))$ holds and that $(x, y, z) \longrightarrow (x_t, y_t, z_t)$. We must prove that $P((x_t, y_t, z_t))$ holds, that is,

$$z_t \in \mathbb{N} \text{ AND } y_t x_t^{z_t} = a^b. \tag{6.5}$$

Since there is a transition from (x, y, z) , we have $z \neq 0$, and since $z \in \mathbb{N}$ by (6.4), we can consider just two cases:

If z is even, then we have that $x_t = x^2$, $y_t = y$, $z_t = z/2$. Therefore, $z_t \in \mathbb{N}$ and

$$\begin{aligned} y_t x_t^{z_t} &= y(x^2)^{z/2} \\ &= yx^{2 \cdot z/2} \\ &= yx^z \\ &= a^b \end{aligned} \quad \text{(by (6.4))}$$

If z is odd, then we have that $x_t = x^2$, $y_t = xy$, $z_t = (z-1)/2$. Therefore, $z_t \in \mathbb{N}$ and

$$\begin{aligned} y_t x_t^{z_t} &= xy(x^2)^{(z-1)/2} \\ &= yx^{1+2 \cdot (z-1)/2} \\ &= yx^{1+(z-1)} \\ &= yx^z \\ &= a^b \end{aligned} \quad \text{(by (6.4))}$$

So in both cases, (6.5) holds, proving that P is a preserved invariant.

Now it's easy to prove partial correctness, namely, if the Fast Exponentiation program terminates, it does so with a^b in register y . This works because obviously $1 \cdot a^b = a^b$, which means that the start state, $(a, 1, b)$, satisfies P . By the Invariant Principle, P holds for all reachable states. But the program only stops when $z = 0$, so if a terminated state, $(x, y, 0)$ is reachable, then $y = yx^0 = a^b$ as required.

Ok, it's partially correct, but what's fast about it? The answer is that the number of multiplications it performs to compute a^b is roughly the length of the binary representation of b . That is, the Fast Exponentiation program uses roughly $\log_2 b$ multiplications compared to the naive approach of multiplying by a a total of $b-1$ times.

More precisely, it requires at most $2(\lceil \log_2 b \rceil + 1)$ multiplications for the Fast Exponentiation algorithm to compute a^b for $b > 1$. The reason is that the number in register z is initially b , and gets at least halved with each transition. So it can't be halved more than $\lceil \log_2 b \rceil + 1$ times before hitting zero and causing the program to terminate. Since each of the transitions involves at most two multiplications, the total number of multiplications until $z = 0$ is at most $2(\lceil \log_2 b \rceil + 1)$ for $b > 0$ (see Problem 6.26).

Problems for Section 6.1

Class Problems

Problem 6.1.

Use induction to prove that

$$1^3 + 2^3 + \cdots + n^3 = \left(\frac{n(n+1)}{2} \right)^2. \quad (6.6)$$

for all $n \geq 1$.

Remember to formally

1. Declare proof by induction.
2. Identify the induction hypothesis $P(n)$.
3. Establish the base case.
4. Prove that $P(n) \Rightarrow P(n+1)$.
5. Conclude that $P(n)$ holds for all $n \geq 1$.

as in the five part template.

Problem 6.2.

Prove by induction on n that

$$1 + r + r^2 + \cdots + r^n = \frac{r^{n+1} - 1}{r - 1}$$

for all $n \in \mathbb{N}$ and numbers $r \neq 1$.

Problem 6.3.

Prove by induction:

$$1 + \frac{1}{4} + \frac{1}{9} + \cdots + \frac{1}{n^2} < 2 - \frac{1}{n}, \quad (6.7)$$

for all $n > 1$.

Problem 6.4. (a) Prove by induction that a $2^n \times 2^n$ courtyard with a 1×1 statue of Bill in *a corner* can be covered with L-shaped tiles. (Do not assume or reprove the (stronger) result of Theorem 6.1.1 that Bill can be placed anywhere. The point of this problem is to show a different induction hypothesis that works.)

(b) Use the result of part (a) to prove the original claim that there is a tiling with Bill in the middle.

Problem 6.5.

The Fibonacci numbers

0 1 1 2 3 5 8 13 ...

are defined as follows. Let

$$F(0) ::= 0, \\ F(1) ::= 1, F(n) ::= F(n-1) + F(n-2) \text{ for } n \geq 2.$$

$F(n)$ be the n th Fibonacci number.

Which sentences in the proof contain logical errors?

False Claim 6.4.6. *Every Fibonacci number is even.*

False proof. 1. We use strong induction.

2. The induction hypothesis is that $F(n)$ is even.
3. We will first show that this hypothesis holds for $n = 0$.
4. This is true, since $F(0) = 0$, which is an even number.
5. Now, suppose $n \geq 2$. We will show that $F(n)$ is even, assuming that $F(k)$ is even for all $k < n$.
6. By assumption, both $F(n-1)$ and $F(n-2)$ are even.
7. Therefore, $F(n)$ is even, since $F(n) = F(n-1) + F(n-2)$ and the sum of two even numbers is even.
8. Thus, the strong induction principle implies that $F(n)$ is even for all $n > 0$. ■

Problem 6.6.

We’ve proved in two different ways that

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

But now we’re going to prove a *contradictory* theorem!

False Theorem. For all $n \geq 0$,

$$2 + 3 + 4 + \cdots + n = \frac{n(n+1)}{2}$$

Proof. We use induction. Let $P(n)$ be the proposition that $2 + 3 + 4 + \cdots + n = n(n+1)/2$.

Base case: $P(0)$ is true, since both sides of the equation are equal to zero. (Recall that a sum with no terms is zero.)

Inductive step: Now we must show that $P(n)$ implies $P(n+1)$ for all $n \geq 0$. So suppose that $P(n)$ is true; that is, $2 + 3 + 4 + \cdots + n = n(n+1)/2$. Then we can reason as follows:

$$\begin{aligned} 2 + 3 + 4 + \cdots + n + (n+1) &= [2 + 3 + 4 + \cdots + n] + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) \\ &= \frac{(n+1)(n+2)}{2} \end{aligned}$$

Above, we group some terms, use the assumption $P(n)$, and then simplify. This shows that $P(n)$ implies $P(n+1)$. By the principle of induction, $P(n)$ is true for all $n \in \mathbb{N}$. ■

Where exactly is the error in this proof?

Problem 6.7.

Alice wants to prove by induction that a predicate, P , holds for certain nonnegative integers. She has proven that for all nonnegative integers $n = 0, 1, \dots$

$$P(n) \text{ IMPLIES } P(n+3).$$

(a) Suppose Alice also proves that $P(5)$ holds. Which of the following propositions can she infer?

1. $P(n)$ holds for all $n \geq 5$

2. $P(3n)$ holds for all $n \geq 5$
3. $P(n)$ holds for $n = 8, 11, 14, \dots$
4. $P(n)$ does not hold for $n < 5$
5. $\forall n. P(3n + 5)$
6. $\forall n > 2. P(3n - 1)$
7. $P(0)$ IMPLIES $\forall n. P(3n + 2)$
8. $P(0)$ IMPLIES $\forall n. P(3n)$

(b) Which of the following could Alice prove in order to conclude that $P(n)$ holds for all $n \geq 5$?

1. $P(0)$
2. $P(5)$
3. $P(5)$ and $P(6)$
4. $P(0)$, $P(1)$, and $P(2)$
5. $P(5)$, $P(6)$, and $P(7)$
6. $P(2)$, $P(4)$, and $P(5)$
7. $P(2)$, $P(4)$, and $P(6)$
8. $P(3)$, $P(5)$, and $P(7)$

Problem 6.8.

Some fundamental principles for reasoning about nonnegative integers are:

1. The Induction Principle,
2. The Strong Induction Principle,
3. The Well-ordering Principle.

Identify which, if any, of the above principles is captured by each of the following inference rules.

(a)

$$\frac{P(0), \forall m. (\forall k \leq m. P(k)) \text{ IMPLIES } P(m + 1)}{\forall n. P(n)}$$

(b)

$$\frac{P(b), \forall k \geq b. P(k) \text{ IMPLIES } P(k + 1)}{\forall k \geq b. P(k)}$$

(c)

$$\frac{\exists n. P(n)}{\exists m. [P(m) \text{ AND } (\forall k. P(k) \text{ IMPLIES } k \geq m)]}$$

(d)

$$\frac{P(0), \forall k > 0. P(k) \text{ IMPLIES } P(k + 1)}{\forall n. P(n)}$$

(e)

$$\frac{\forall m. (\forall k < m. P(k)) \text{ IMPLIES } P(m)}{\forall n. P(n)}$$

Homework Problems

Problem 6.9.

Use strong induction to prove that $n \leq 3^{n/3}$ for every integer $n \geq 0$.

Problem 6.10.

For any binary string, α , let $\text{num}(\alpha)$ be the nonnegative integer it represents in binary notation. For example, $\text{num}(10) = 2$, and $\text{num}(0101) = 5$.

An $n + 1$ -bit adder adds two $n + 1$ -bit binary numbers. More precisely, an $n + 1$ -bit adder takes two length $n + 1$ binary strings

$$\begin{aligned}\alpha_n &::= a_n \dots a_1 a_0, \\ \beta_n &::= b_n \dots b_1 b_0,\end{aligned}$$

and a binary digit, c_0 , as inputs, and produces a length $n + 1$ binary string

$$\sigma_n ::= s_n \dots s_1 s_0,$$

and a binary digit, c_{n+1} , as outputs, and satisfies the specification:

$$\text{num}(\alpha_n) + \text{num}(\beta_n) + c_0 = 2^{n+1}c_{n+1} + \text{num}(\sigma_n). \quad (6.8)$$

There is a straightforward way to implement an $n + 1$ -bit adder as a digital circuit: an $n + 1$ -bit *ripple-carry circuit* has $1 + 2(n + 1)$ binary inputs

$$a_n, \dots, a_1, a_0, b_n, \dots, b_1, b_0, c_0,$$

and $n + 2$ binary outputs,

$$c_{n+1}, s_n, \dots, s_1, s_0.$$

As in Problem 3.5, the ripple-carry circuit is specified by the following formulas:

$$s_i ::= a_i \text{ XOR } b_i \text{ XOR } c_i \quad (6.9)$$

$$c_{i+1} ::= (a_i \text{ AND } b_i) \text{ OR } (a_i \text{ AND } c_i) \text{ OR } (b_i \text{ AND } c_i), \quad (6.10)$$

for $0 \leq i \leq n$.

(a) Verify that definitions (6.9) and (6.10) imply that

$$a_n + b_n + c_n = 2c_{n+1} + s_n. \quad (6.11)$$

for all $n \in \mathbb{N}$.

(b) Prove by induction on n that an $n + 1$ -bit ripple-carry circuit really is an $n + 1$ -bit adder, that is, its outputs satisfy (6.8).

Hint: You may assume that, by definition of binary representation of integers,

$$\text{num}(\alpha_{n+1}) = a_{n+1}2^{n+1} + \text{num}(\alpha_n). \quad (6.12)$$

Problem 6.11.

The Math for Computer Science mascot, Theory Hippotamus, made a startling discovery while playing with his prized collection of unit squares over the weekend. Here is what happened.

First, Theory Hippotamus put his favorite unit square down on the floor as in Figure 6.9 (a). He noted that the length of the periphery of the resulting shape was 4, an even number. Next, he put a second unit square down next to the first so that the two squares shared an edge as in Figure 6.9 (b). He noticed that the length of the periphery of the resulting shape was now 6, which is also an even number. (The periphery of each shape in the figure is indicated by a thicker line.) Theory Hippotamus continued to place squares so that each new square shared an edge with at least one previously-placed square and no squares overlapped. Eventually, he arrived at the shape in Figure 6.9 (c). He realized that the length of the periphery of this shape was 36, which is again an even number.

Our plucky porcine pal is perplexed by this peculiar pattern. Use induction on the number of squares to prove that the length of the periphery is always even, no matter how many squares Theory Hippotamus places or how he arranges them.

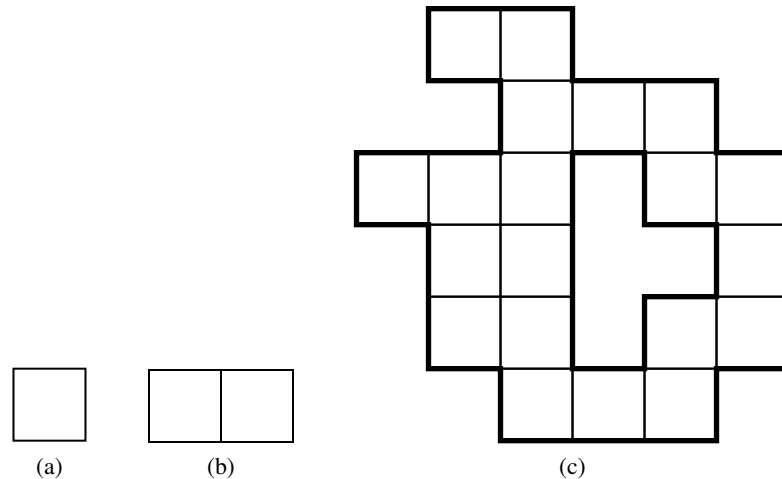


Figure 6.9 Some shapes that Theory Hippotamus created.

Problems for Section 6.2

Class Problems

Problem 6.12.

A sequence of numbers is *weakly decreasing* when each number in the sequence is $\geq$ the numbers after it. (This implies that a sequence of just one number is weakly decreasing.)

Here’s a bogus proof of a very important true fact, every integer greater than 1 is a *product of a unique weakly decreasing sequence of primes* —a pusp, for short.

Explain what’s bogus about the proof.

Lemma 6.4.7. *Every integer greater than 1 is a pusp.*

For example, $252 = 7 \cdot 3 \cdot 3 \cdot 2 \cdot 2$, and no other weakly decreasing sequence of primes will have a product equal to 252.

Bogus proof. We will prove Lemma 6.4.7 by strong induction, letting the induction hypothesis, $P(n)$, be

$$n \text{ is a pusp.}$$

So Lemma 6.4.7 will follow if we prove that $P(n)$ holds for all $n \geq 2$.

Base Case ($n = 2$): $P(2)$ is true because 2 is prime, and so it is a length one product of primes, and this is obviously the only sequence of primes whose product can equal 2.

Inductive step: Suppose that $n \geq 2$ and that i is a pusp for every integer i where $2 \leq i < n + 1$. We must show that $P(n + 1)$ holds, namely, that $n + 1$ is also a pusp. We argue by cases:

If $n + 1$ is itself prime, then it is the product of a length one sequence consisting of itself. This sequence is unique, since by definition of prime, $n + 1$ has no other prime factors. So $n + 1$ is a pusp, that is $P(n + 1)$ holds in this case.

Otherwise, $n + 1$ is not prime, which by definition means $n + 1 = km$ for some integers k, m such that $2 \leq k, m < n + 1$. Now by the strong induction hypothesis, we know that k and m are pusps. It follows immediately that by merging the unique prime sequences for k and m , in sorted order, we get a unique weakly decreasing sequence of primes whose product equals $n + 1$. So $n + 1$ is a pusp, in this case as well.

So $P(n + 1)$ holds in any case, which completes the proof by strong induction that $P(n)$ holds for all $n \geq 2$. ■

Problem 6.13.

Define the *potential*, $p(S)$, of a stack of blocks, S , to be $k(k - 1)/2$ where k is the number of blocks in S . Define the potential, $p(A)$, of a set of stacks, A , to be the sum of the potentials of the stacks in A .

Generalize Theorem 6.2.1 about scores in the stacking game to show that for any set of stacks, A , if a sequence of moves starting with A leads to another set of stacks, B , then $p(A) \geq p(B)$, and the score for this sequence of moves is $p(A) - p(B)$.

Hint: Try induction on the number of moves to get from A to B .

Homework Problems

Problem 6.14.

A group of $n \geq 1$ people can be divided into teams, each containing either 4 or 7 people. What are all the possible values of n ? Use induction to prove that your answer is correct.

Problem 6.15.

The following Lemma is true, but the *proof* given for it below is defective. Pin-point *exactly* where the proof first makes an unjustified step and explain why it is unjustified.

Lemma 6.4.8. For any prime p and positive integers $n, x_1, x_2, \dots, x_n$, if $p \mid$

$x_1 x_2 \dots x_n$, then $p \mid x_i$ for some $1 \leq i \leq n$.

Bogus proof. Proof by strong induction on n . The induction hypothesis, $P(n)$, is that Lemma holds for n .

Base case $n = 1$: When $n = 1$, we have $p \mid x_1$, therefore we can let $i = 1$ and conclude $p \mid x_i$.

Induction step: Now assuming the claim holds for all $k \leq n$, we must prove it for $n + 1$.

So suppose $p \mid x_1 x_2 \dots x_{n+1}$. Let $y_n = x_n x_{n+1}$, so $x_1 x_2 \dots x_{n+1} = x_1 x_2 \dots x_{n-1} y_n$. Since the righthand side of this equality is a product of n terms, we have by induction that p divides one of them. If $p \mid x_i$ for some $i < n$, then we have the desired i . Otherwise $p \mid y_n$. But since y_n is a product of the two terms x_n, x_{n+1} , we have by strong induction that p divides one of them. So in this case $p \mid x_i$ for $i = n$ or $i = n + 1$. ■

Problem 6.16.

The Fibonacci numbers

$$0, 1, 1, 2, 3, 5, 8, 13, \dots$$

are defined as follows. Let $F(n)$ be the n th Fibonacci number. Then

$$F(0) ::= 0, \tag{6.13}$$

$$F(1) ::= 1, \tag{6.14}$$

$$F(m) ::= F(m - 1) + F(m - 2) \quad \text{for } m \geq 2. \tag{6.15}$$

Indicate exactly which sentence(s) in the following bogus proof contain logical errors? Explain.

False Claim. *Every Fibonacci number is even.*

Bogus proof. Let all the variables n, m, k mentioned below be nonnegative integer valued. Let $\text{Even}(n)$ mean that $F(n)$ is even. The proof is by strong induction with induction hypothesis $\text{Even}(n)$.

base case: $F(0) = 0$ is an even number, so $\text{Even}(0)$ is true.

inductive step: We assume may assume the strong induction hypothesis

$$\text{Even}(k) \text{ for } 0 \leq k \leq n,$$

and we must prove $\text{Even}(n + 1)$.

So assume $n + 1 \geq 2$. Then by strong induction hypothesis, $\text{Even}(n)$ and $\text{Even}(n - 1)$ are true, that is, $F(n)$ and $F(n - 1)$ are both even. But by the defining equation (6.15), $F(n + 1)$ equals the sum, $F(n) + F(n - 1)$, of two even numbers, and so it is also even. This proves $\text{Even}(n + 1)$ as required.

Hence, $F(m)$ is even for all $m \in \mathbb{N}$ by the Strong Induction Principle. ■

Exam Problems

Problem 6.17.

The n th Fibonacci number, F_n , is defined recursively as follows:

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2 \end{cases}$$

These numbers satisfy many unexpected identities, such as

$$F_0^2 + F_1^2 + \cdots + F_n^2 = F_n F_{n+1} \quad (6.16)$$

Equation (6.16) can be proved to hold for all $n \in \mathbb{N}$ by induction, using the equation itself as the induction hypothesis, $P(n)$.

(a) Prove the **base case** ($n = 0$).

(b) Now prove the **inductive step**.

Problem 6.18.

Let $S(n)$ mean that exactly n cents of postage can be paid using only 4 and 7 cent stamps. Use strong induction to prove that

$$\forall n. (n \geq 18) \text{ IMPLIES } S(n).$$

Problems for Section 6.4

Practice Problems

Problem 6.19.

Which states of the Die Hard 3 machine below have transitions to exactly two states?

Die Hard Transitions

1. Fill the little jug: $(b, l) \longrightarrow (b, 3)$ for $l < 3$.
2. Fill the big jug: $(b, l) \longrightarrow (5, l)$ for $b < 5$.
3. Empty the little jug: $(b, l) \longrightarrow (b, 0)$ for $l > 0$.
4. Empty the big jug: $(b, l) \longrightarrow (0, l)$ for $b > 0$.
5. Pour from the little jug into the big jug: for $l > 0$,

$$(b, l) \longrightarrow \begin{cases} (b + l, 0) & \text{if } b + l \leq 5, \\ (5, l - (5 - b)) & \text{otherwise.} \end{cases}$$

6. Pour from big jug into little jug: for $b > 0$,

$$(b, l) \longrightarrow \begin{cases} (0, b + l) & \text{if } b + l \leq 3, \\ (b - (3 - l), 3) & \text{otherwise.} \end{cases}$$

Problem 6.20.

Prove that every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.

Homework Problems

Problem 6.21.

Here is a *very, very fun* game. We start with two distinct, positive integers written on a blackboard. Call them a and b . You and I now take turns. (I’ll let you decide who goes first.) On each player’s turn, he or she must write a new positive integer on the board that is the difference of two numbers that are already there. If a player can not play, then he or she loses.

For example, suppose that 12 and 15 are on the board initially. Your first play must be 3, which is $15 - 12$. Then I might play 9, which is $12 - 3$. Then you might play 6, which is $15 - 9$. Then I can not play, so I lose.

(a) Show that every number on the board at the end of the game is a multiple of $\gcd(a, b)$.

(b) Show that every positive multiple of $\gcd(a, b)$ up to $\max(a, b)$ is on the board at the end of the game.

(c) Describe a strategy that lets you win this game every time.

Problem 6.22.

In the late 1960s, the military junta that ousted the government of the small republic of Nerdia completely outlawed built-in multiplication operations, and also forbade division by any number other than 3. Fortunately, a young dissident found a way to help the population multiply any two nonnegative integers without risking persecution by the junta. The procedure he taught people is:

procedure *multiply*(x, y : nonnegative integers)

$r := x$;

$s := y$;

$a := 0$;

while $s \neq 0$ **do**

if $3 \mid s$ **then**

$r := r + r + r$;

$s := s/3$;

else if $3 \mid (s - 1)$ **then**

$a := a + r$;

$r := r + r + r$;

$s := (s - 1)/3$;

else

$a := a + r + r$;

$r := r + r + r$;

$s := (s - 2)/3$;

return a ;

We can model the algorithm as a state machine whose states are triples of nonnegative integers (r, s, a) . The initial state is $(x, y, 0)$. The transitions are given by the rule that for $s > 0$:

$$(r, s, a) \rightarrow \begin{cases} (3r, s/3, a) & \text{if } 3 \mid s \\ (3r, (s - 1)/3, a + r) & \text{if } 3 \mid (s - 1) \\ (3r, (s - 2)/3, a + 2r) & \text{otherwise.} \end{cases}$$

(a) List the sequence of steps that appears in the execution of the algorithm for inputs $x = 5$ and $y = 10$.

(b) Use the Invariant Method to prove that the algorithm is partially correct—that is, if $s = 0$, then $a = xy$.

(c) Prove that the algorithm terminates after at most $1 + \log_3 y$ executions of the body of the **do** statement.

Problem 6.23.

A robot named Wall-E wanders around a two-dimensional grid. He starts out at $(0, 0)$ and is allowed to take four different types of step:

1. $(+2, -1)$
2. $(+1, -2)$
3. $(+1, +1)$
4. $(-3, 0)$

Thus, for example, Wall-E might walk as follows. The types of his steps are listed above the arrows.

$$(0, 0) \xrightarrow{1} (2, -1) \xrightarrow{3} (3, 0) \xrightarrow{2} (4, -2) \xrightarrow{4} (1, -2) \rightarrow \dots$$

Wall-E’s true love, the fashionable and high-powered robot, Eve, awaits at $(0, 2)$.

(a) Describe a state machine model of this problem.

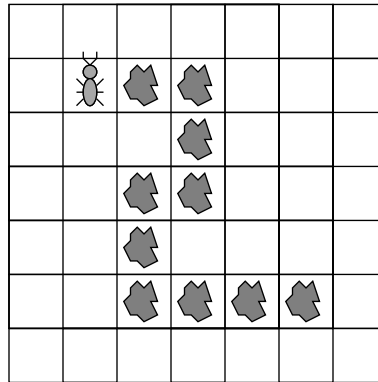
(b) Will Wall-E ever find his true love? Either find a path from Wall-E to Eve or use the Invariant Principle to prove that no such path exists.

Problem 6.24.

A hungry ant is placed on an unbounded grid. Each square of the grid either contains a crumb or is empty. The squares containing crumbs form a path in which, except at the ends, every crumb is adjacent to exactly two other crumbs. The ant is placed at one end of the path and on a square containing a crumb. For example, the figure below shows a situation in which the ant faces North, and there is a trail of food leading approximately Southeast. The ant has already eaten the crumb upon which it was initially placed.

The ant can only smell food directly in front of it. The ant can only remember a small number of things, and what it remembers after any move only depends on what it remembered and smelled immediately before the move. Based on smell and memory, the ant may choose to move forward one square, or it may turn right or left. It eats a crumb when it lands on it.

The above scenario can be nicely modelled as a state machine in which each state is a pair consisting of the “ant’s memory” and “everything else”—for example, information about where things are on the grid. Work out the details of such a model state machine; design the ant-memory part of the state machine so the ant will eat all the crumbs on *any* finite path at which it starts and then signal when it



is done. Be sure to clearly describe the possible states, transitions, and inputs and outputs (if any) in your model. Briefly explain why your ant will eat all the crumbs.

Note that the last transition is a self-loop; the ant signals done for eternity. One could also add another end state so that the ant signals done only once.

Problem 6.25.

Suppose that you have a regular deck of cards arranged as follows, from top to bottom:

$$A\heartsuit 2\heartsuit \dots K\heartsuit A\spadesuit 2\spadesuit \dots K\spadesuit A\clubsuit 2\clubsuit \dots K\clubsuit A\diamondsuit 2\diamondsuit \dots K\diamondsuit$$

Only two operations on the deck are allowed: *inshuffling* and *outshuffling*. In both, you begin by cutting the deck exactly in half, taking the top half into your right hand and the bottom into your left. Then you shuffle the two halves together so that the cards are perfectly interlaced; that is, the shuffled deck consists of one card from the left, one from the right, one from the left, one from the right, etc. The top card in the shuffled deck comes from the right hand in an outshuffle and from the left hand in an inshuffle.

(a) Model this problem as a state machine.

(b) Use the Invariant Principle to prove that you can not make the entire first half of the deck black through a sequence of inshuffles and outshuffles.

Note: Discovering a suitable invariant can be difficult! The standard approach is to identify a bunch of reachable states and then look for a pattern, some feature that they all share.⁶

⁶If this does not work, consider twitching and drooling until someone takes the problem away.

Problem 6.26.

Prove that the fast exponentiation state machine of Section 6.4.5 will halt after

$$\lceil \log_2 n \rceil + 1 \quad (6.17)$$

transitions starting from any state where the value of z is $n \in \text{integers}^+$.

Hint: Strong induction.

Class Problems

Problem 6.27.

In this problem you will establish a basic property of a puzzle toy called the *Fifteen Puzzle* using the method of invariants. The Fifteen Puzzle consists of sliding square tiles numbered $1, \dots, 15$ held in a 4×4 frame with one empty square. Any tile adjacent to the empty square can slide into it.

The standard initial position is

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

We would like to reach the target position (known in the oldest author’s youth as “the impossible”):

15	14	13	12
11	10	9	8
7	6	5	4
3	2	1	

A state machine model of the puzzle has states consisting of a 4×4 matrix with 16 entries consisting of the integers $1, \dots, 15$ as well as one “empty” entry—like each of the two arrays above.

The state transitions correspond to exchanging the empty square and an adjacent numbered tile. For example, an empty at position $(2, 2)$ can exchange position with tile above it, namely, at position $(1, 2)$:

n_1	n_2	n_3	n_4		n_1		n_3	n_4
n_5		n_6	n_7	→	n_5	n_2	n_6	n_7
n_8	n_9	n_{10}	n_{11}		n_8	n_9	n_{10}	n_{11}
n_{12}	n_{13}	n_{14}	n_{15}		n_{12}	n_{13}	n_{14}	n_{15}

We will use the invariant method to prove that there is no way to reach the target state starting from the initial state.

We begin by noting that a state can also be represented as a pair consisting of two things:

1. a list of the numbers $1, \dots, 15$ in the order in which they appear—reading rows left-to-right from the top row down, ignoring the empty square, and
2. the coordinates of the empty square—where the upper left square has coordinates $(1, 1)$, the lower right $(4, 4)$.

(a) Write out the “list” representation of the start state and the “impossible” state.

Let L be a list of the numbers $1, \dots, 15$ in some order. A pair of integers is an *out-of-order pair* in L when the first element of the pair both comes *earlier* in the list and *is larger*, than the second element of the pair. For example, the list $1, 2, 4, 5, 3$ has two out-of-order pairs: $(4, 3)$ and $(5, 3)$. The increasing list $1, 2, \dots, n$ has no out-of-order pairs.

Let a state, S , be a pair $(L, (i, j))$ described above. We define the *parity* of S to be the mod 2 sum of the number, $p(L)$, of out-of-order pairs in L and the row-number of the empty square, that is the parity of S is $p(L) + i \pmod{2}$.

(b) Verify that the parity of the start state and the target state are different.

(c) Show that the parity of a state is preserved under transitions. Conclude that “the impossible” is impossible to reach.

By the way, if two states have the same parity, then in fact there *is* a way to get from one to the other. If you like puzzles, you’ll enjoy working this out on your own.

Problem 6.28.

A robot moves on the two-dimensional integer grid. It starts out at $(0, 0)$, and is allowed to move in any of these four ways:

1. $(+2, -1)$: right 2, down 1
2. $(-2, +1)$: left 2, up 1
3. $(+1, +3)$
4. $(-1, -3)$

Prove that this robot can never reach $(1, 1)$.

Problem 6.29.

The Massachusetts Turnpike Authority is concerned about the integrity of the new Zakim bridge. Their consulting architect has warned that the bridge may collapse if more than 1000 cars are on it at the same time. The Authority has also been warned by their traffic consultants that the rate of accidents from cars speeding across bridges has been increasing.

Both to lighten traffic and to discourage speeding, the Authority has decided to make the bridge *one-way* and to put tolls at *both* ends of the bridge (don’t laugh, this is Massachusetts). So cars will pay tolls both on entering and exiting the bridge, but the tolls will be different. In particular, a car will pay \$3 to enter onto the bridge and will pay \$2 to exit. To be sure that there are never too many cars on the bridge, the Authority will let a car onto the bridge only if the difference between the amount of money currently at the entry toll booth minus the amount at the exit toll booth is strictly less than a certain threshold amount of $\$T_0$.

The consultants have decided to model this scenario with a state machine whose states are triples of nonnegative integers, (A, B, C) , where

- A is an amount of money at the entry booth,
- B is an amount of money at the exit booth, and
- C is a number of cars on the bridge.

Any state with $C > 1000$ is called a *collapsed* state, which the Authority dearly hopes to avoid. There will be no transition out of a collapsed state.

Since the toll booth collectors may need to start off with some amount of money in order to make change, and there may also be some number of “official” cars already on the bridge when it is opened to the public, the consultants must be ready to analyze the system started at *any* uncollapsed state. So let A_0 be the initial number of dollars at the entrance toll booth, B_0 the initial number of dollars at the exit toll booth, and $C_0 \leq 1000$ the number of official cars on the bridge when it is opened. You should assume that even official cars pay tolls on exiting or entering the bridge after the bridge is opened.

(a) Give a mathematical model of the Authority’s system for letting cars on and off the bridge by specifying a transition relation between states of the form (A, B, C) above.

The Authority has asked their engineering consultants to determine T and to verify that this policy will keep the number of cars from exceeding 1000.

The consultants reason that if C_0 is the number of official cars on the bridge when it is opened, then an additional $1000 - C_0$ cars can be allowed on the bridge.

So as long as $A - B$ has not increased by $3(1000 - C_0)$, there shouldn't more than 1000 cars on the bridge. So they recommend defining

$$T_0 ::= 3(1000 - C_0) + (A_0 - B_0), \quad (6.18)$$

where A_0 is the initial number of dollars at the entrance toll booth, B_0 is the initial number of dollars at the exit toll booth.

(b) Let $D_0 ::= 2A_0 - 3B_0 - 6C_0$ and define

$$P(A, B, C) ::= [2A - 3B - 6C = D_0] \text{ AND } [C \leq 1000].$$

Verify that P is a preserved invariant of the state machine.

(c) Conclude that the traffic won't cause the bridge to collapse.

(d) A clever MIT intern working for the Turnpike Authority agrees that the Turnpike's bridge management policy will be *safe*: the bridge will not collapse. But she warns her boss that the policy will lead to *deadlock*—a situation where traffic can't move on the bridge even though the bridge has not collapsed.

Explain more precisely in terms of system transitions what the intern means, and briefly, but clearly, justify her claim.

Problem 6.30.

Start with 102 coins on a table, 98 showing heads and 4 showing tails. There are two ways to change the coins:

- (i) flip over any ten coins, or
- (ii) let n be the number of heads showing. Place $n + 1$ additional coins, all showing tails, on the table.

For example, you might begin by flipping nine heads and one tail, yielding 90 heads and 12 tails, then add 91 tails, yielding 90 heads and 103 tails.

(a) Model this situation as a state machine, carefully defining the set of states, the start state, and the possible state transitions.

(b) Explain how to reach a state with exactly one tail showing.

(c) Define the following derived variables:

$C ::=$ the number of coins on the table,	$H ::=$ the number of heads,
$T ::=$ the number of tails,	$C_2 ::=$ remainder($C/2$),
$H_2 ::=$ remainder($H/2$),	$T_2 ::=$ remainder($T/2$).

Which of these variables is

1. strictly increasing
2. weakly increasing
3. strictly decreasing
4. weakly decreasing
5. constant

(d) Prove that it is not possible to reach a state in which there is exactly one head showing.

Problem 6.31.

A classroom is designed so students sit in a square arrangement. An outbreak of beaver flu sometimes infects students in the class; beaver flu is a rare variant of bird flu that lasts forever, with symptoms including a yearning for more quizzes and the thrill of late night problem set sessions.

Here is an illustration of a 6×6 -seat classroom with seats represented by squares. The locations of infected students are marked with an asterisk.

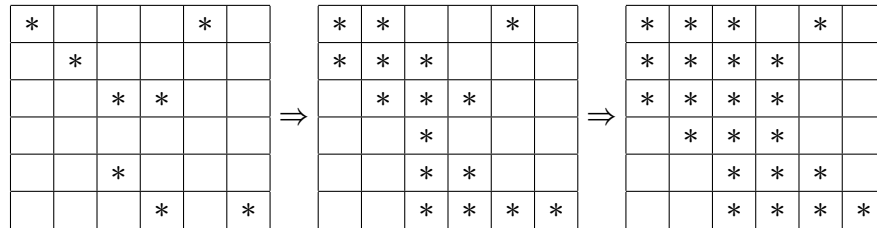
*				*	
	*				
		*	*		
		*			
			*		*

Outbreaks of infection spread rapidly step by step. A student is infected after a step if either

- the student was infected at the previous step (since beaver flu lasts forever), or
- the student was adjacent to *at least two* already-infected students at the previous step.

Here *adjacent* means the students' individual squares share an edge (front, back, left or right); they are not adjacent if they only share a corner point. So each student is adjacent to 2, 3 or 4 others.

In the example, the infection spreads as shown below.



In this example, over the next few time-steps, all the students in class become infected.

Theorem. *If fewer than n students among those in an $n \times n$ arrangement are initially infected in a flu outbreak, then there will be at least one student who never gets infected in this outbreak, even if students attend all the lectures.*

Prove this theorem.

Hint: Think of the state of an outbreak as an $n \times n$ square above, with asterisks indicating infection. The rules for the spread of infection then define the transitions of a state machine. Show that

$R(q) ::=$ The “perimeter” of the “infected region”
of state q is at most k ,

is a preserved invariant.

7 Recursive Data Types

Recursive data types play a central role in programming, and induction is really all about them.

Recursive data types are specified by *recursive definitions* that say how to construct new data elements from previous ones. Along with each recursive data type there are recursive definitions of properties or functions on the data type. Most importantly, based on a recursive definition, there is a *structural induction* method for proving that all data of the given type have some property.

This chapter examines a few examples of recursive data types and recursively defined functions on them:

- strings of characters,
- the “balanced” strings of brackets,
- the nonnegative integers, and
- arithmetic expressions.

7.1 Recursive Definitions and Structural Induction

We’ll start off illustrating recursive definitions and proofs using the example of character strings. Normally we’d take strings of characters for granted, but it’s informative to treat them as a recursive data type. In particular, strings are a nice first example because you will see recursive definitions of things that are easy to understand or you already know, so you can focus on how the definitions work without having to figure out what they are for.

Definitions of recursive data types have two parts:

- **Base case(s)** specifying that some known mathematical elements are in the data type, and
- **Constructor case(s)** that specify how to construct new data elements from previously constructed elements or from base elements.

The definition of strings over a given character set, A , follows this pattern:

Definition 7.1.1. Let A be a nonempty set called an *alphabet*, whose elements are referred to as *characters*, *letters*, or *symbols*. The recursive data type, A^* , of strings over alphabet, A , are defined as follows:

- **Base case:** the empty string, λ , is in A^* .
- **Constructor case:** If $a \in A$ and $s \in A^*$, then the pair $\langle a, s \rangle \in A^*$.

So $\{0, 1\}^*$ are supposed to be the binary strings.

The usual way to treat binary strings is as sequences of 0’s and 1’s. For example, we have identified the length-4 binary string 1011 as a sequence of bits, of a 4-tuple, namely, $(1, 0, 1, 1)$. But according to the recursive Definition 7.1.1, this string would be represented by nested pairs, namely

$$\langle 1, \langle 0, \langle 1, \langle 1, \lambda \rangle \rangle \rangle \rangle .$$

These nested pairs are definitely cumbersome, and may also seem bizarre, but they actually reflect the way lists of characters would be represented in programming languages like Scheme or Python, where $\langle a, s \rangle$ would correspond to $\text{cons}(a, s)$.

Notice that we haven’t said exactly how the empty string is represented. It really doesn’t matter as long as we can recognize the empty string and not confuse it with any nonempty string.

Continuing the recursive approach, let’s define the length of a string.

Definition 7.1.2. The length, $|s|$, of a string, s , is defined recursively based on the definition of $s \in A^*$:

Base case: $|\lambda| ::= 0$.

Constructor case: $|\langle a, s \rangle| ::= 1 + |s|$.

This definition of length follows a standard pattern: functions on recursive data types can be defined recursively using the same cases as the data type definition. Namely, to define a function, f , on a recursive data type, define the value of f for the base cases of the data type definition, and then define the value of f in each constructor case in terms of the values of f on the component data items.

Let’s do another example: the *concatenation* $s \cdot t$ of the strings s and t is the string consisting of the letters of s followed by the letters of t . This is a perfectly clear mathematical definition of concatenation (except maybe for what to do with the empty string), and in terms of Scheme/Python lists, $s \cdot t$ would be the list $\text{append}(s, t)$. Here’s a recursive definition of concatenation.

Definition 7.1.3. The *concatenation* $s \cdot t$ of the strings $s, t \in A^*$ is defined recursively based on the definition of $s \in A^*$:

Base case:

$$\lambda \cdot t ::= t.$$

Constructor case:

$$\langle a, s \rangle \cdot t ::= \langle a, s \cdot t \rangle.$$

Structural induction is a method for proving that all the elements of a recursively defined data type have some property. A structural induction proof has two parts corresponding to the recursive definition:

- Prove that each base case element has the property.
- Prove that each constructor case element has the property, when the constructor is applied to elements that have the property.

For example, we can verify the familiar fact that the length of the concatenation of two strings is the sum of their lengths using structural induction:

Theorem 7.1.4. For all $s, t \in A^*$,

$$|s \cdot t| = |s| + |t|.$$

Proof. By structural induction on the definition of $s \in A^*$. The induction hypothesis is

$$P(s) ::= \forall t \in A^*. |s \cdot t| = |s| + |t|.$$

Base case ($s = \lambda$):

$$\begin{aligned} |s \cdot t| &= |\lambda \cdot t| \\ &= |t| && \text{(def } \cdot, \text{ base case)} \\ &= 0 + |t| \\ &= |s| + |t| && \text{(def length, base case)} \end{aligned}$$

Constructor case: Suppose $s ::= \langle a, r \rangle$ and assume the induction hypothesis, $P(r)$.

We must show that $P(s)$ holds:

$$\begin{aligned}
 |s \cdot t| &= |\langle a, r \rangle \cdot t| \\
 &= |\langle a, r \cdot t \rangle| && \text{(concat def, constructor case)} \\
 &= 1 + |r \cdot t| && \text{(length def, constructor case)} \\
 &= 1 + (|r| + |t|) && \text{since } P(r) \text{ holds} \\
 &= (1 + |r|) + |t| \\
 &= |\langle a, r \rangle| + |t| && \text{(length def, constructor case)} \\
 &= |s| + |t|.
 \end{aligned}$$

This proves that $P(s)$ holds as required, completing the constructor case. By structural induction we conclude that $P(s)$ holds for all strings $s \in A^*$. ■

This proof illustrates the general principle:

The Principle of Structural Induction.

Let P be a predicate on a recursively defined data type R . If

- $P(b)$ is true for each base case element, $b \in R$, and
- for all two argument constructors, $\mathbf{c}$,

$$[P(r) \text{ AND } P(s)] \text{ IMPLIES } P(\mathbf{c}(r, s))$$

for all $r, s \in R$,

and likewise for all constructors taking other numbers of arguments,

then

$$P(r) \text{ is true for all } r \in R.$$

The number, $\#_c(s)$, of occurrences of the character $c \in A$ in the string s has a simple recursive definition based on the definition of $s \in A^*$:

Definition 7.1.5.

Base case: $\#_c(\lambda) ::= 0$.

Constructor case:

$$\#_c(\langle a, s \rangle) ::= \begin{cases} \#_c(s) & \text{if } a \neq c, \\ 1 + \#_c(s) & \text{if } a = c. \end{cases}$$

We’ll need the following lemma in the next section:

Lemma 7.1.6.

$$\#_c(s \cdot t) = \#_c(s) + \#_c(t).$$

The easy proof by structural induction is an exercise (Problem 7.7).

7.2 Strings of Matched Brackets

Let $\{[], []^*\}$ be the set of all strings of square brackets. For example, the following two strings are in $\{[], []^*\}$:

$$[][[[]]] \quad \text{and} \quad [[[]]][] \quad (7.1)$$

A string, $s \in \{[], []^*\}$, is called a *matched string* if its brackets “match up” in the usual way. For example, the left hand string above is not matched because its second right bracket does not have a matching left bracket. The string on the right is matched.

We’re going to examine several different ways to define and prove properties of matched strings using recursively defined sets and functions. These properties are pretty straightforward, and you might wonder whether they have any particular relevance in computer science. The honest answer is “not much relevance, *any more*.” The reason for this is one of the great successes of computer science as explained in the text box below.

Expression Parsing

During the early development of computer science in the 1950’s and 60’s, creation of effective programming language compilers was a central concern. A key aspect in processing a program for compilation was expression parsing. One significant problem was to take an expression like

$$x + y * z^2 \div y + 7$$

and *put in* the brackets that determined how it should be evaluated —should it be

$$\begin{aligned} & [[x + y] * z^2 \div y] + 7, \text{ or,} \\ & x + [y * z^2 \div [y + 7]], \text{ or,} \\ & [x + [y * z^2]] \div [y + 7], \text{ or } \dots? \end{aligned}$$

The Turing award (the “Nobel Prize” of computer science) was ultimately bestowed on Robert W Floyd, for, among other things, being discoverer of a simple program that would insert the brackets properly.

In the 70’s and 80’s, this parsing technology was packaged into high-level compiler-compilers that automatically generated parsers from expression grammars. This automation of parsing was so effective that the subject no longer demanded attention. It largely disappeared from the computer science curriculum by the 1990’s.

The matched strings can be nicely characterized as a recursive data type:

Definition 7.2.1. Recursively define the set, `RecMatch`, of strings as follows:

- **Base case:** $\lambda \in \text{RecMatch}$.
- **Constructor case:** If $s, t \in \text{RecMatch}$, then

$$[s]t \in \text{RecMatch}.$$

Here $[s]t$ refers to the concatenation of strings which would be written in full as

$$[\cdot (s \cdot ([\cdot t))).$$

From now on, we’ll usually omit the “.”s.”

Using this definition, $\lambda \in \text{RecMatch}$ by the Base case, so letting $s = t = \lambda$ in the constructor case implies

$$[\lambda]\lambda = [] \in \text{RecMatch}.$$

Now,

$$\begin{aligned} [\lambda][\] &= [\][\] \in \text{RecMatch} && (\text{letting } s = \lambda, t = [\]) \\ [[\]]\lambda &= [[\]]\lambda \in \text{RecMatch} && (\text{letting } s = [\], t = \lambda) \\ [[\]][\] &\in \text{RecMatch} && (\text{letting } s = [\], t = [\]) \end{aligned}$$

are also strings in RecMatch by repeated applications of the Constructor case; and so on.

It’s pretty obvious that in order for brackets to match, there better be an equal number of left and right ones. For further practice, let’s carefully prove this from the recursive definitions.

Lemma. *Every string in RecMatch has an equal number of left and right brackets.*

Proof. The proof is by structural induction with induction hypothesis

$$P(s) ::= \#_{\text{[}}(s) = \#_{\text{]}}(s).$$

Base case: $P(\lambda)$ holds because

$$\#_{\text{[}}(\lambda) = 0 = \#_{\text{]}}(\lambda)$$

by the base case of Definition 7.1.5 of $\#_c()$.

Constructor case: By structural induction hypothesis, we assume $P(s)$ and $P(t)$ and must show $P([\ s\]t)$:

$$\begin{aligned} \#_{\text{[}}([\ s\]t) &= \#_{\text{[}}([\]) + \#_{\text{[}}(s) + \#_{\text{[}}([\]) + \#_{\text{[}}(t) && (\text{Lemma 7.1.6}) \\ &= 1 + \#_{\text{[}}(s) + 0 + \#_{\text{[}}(t) && (\text{def } \#_{\text{[}}()) \\ &= 1 + \#_{\text{]}}(s) + 0 + \#_{\text{]}}(t) && (\text{by } P(s) \text{ and } P(t)) \\ &= 0 + \#_{\text{]}}(s) + 1 + \#_{\text{]}}(t) \\ &= \#_{\text{]}}([\]) + \#_{\text{]}}(s) + \#_{\text{]}}([\]) + \#_{\text{]}}(t) && (\text{def } \#_{\text{]}}()) \\ &= \#_{\text{]}}([\ s\]t) && (\text{Lemma 7.1.6}) \end{aligned}$$

This completes the proof of the constructor case. We conclude by structural induction that $P(s)$ holds for all $s \in \text{RecMatch}$. ■

Warning: When a recursive definition of a data type allows the same element to be constructed in more than one way, the definition is said to be *ambiguous*. We were careful to choose an *unambiguous* definition of RecMatch to ensure that functions defined recursively on its definition would always be well-defined. Recursively defining a function on an ambiguous data type definition usually will not work. To illustrate the problem, here’s another definition of the matched strings.

Definition 7.2.2. Define the set, $\text{AmbRecMatch} \subseteq \{[], []\}^*$ recursively as follows:

- **Base case:** $\lambda \in \text{AmbRecMatch}$,
- **Constructor cases:** if $s, t \in \text{AmbRecMatch}$, then the strings $[s]$ and st are also in AmbRecMatch .

It’s pretty easy to see that the definition of AmbRecMatch is just another way to define RecMatch , that is $\text{AmbRecMatch} = \text{RecMatch}$ (see Problem 7.14). The definition of AmbRecMatch is arguably easier to understand, but we didn’t use it because it’s ambiguous, while the trickier definition of RecMatch is unambiguous. Here’s why this matters. Let’s define the number of operations, $f(s)$, to construct a matched string s recursively on the definition of $s \in \text{AmbRecMatch}$:

$$\begin{aligned} f(\lambda) &::= 0, & (f \text{ base case}) \\ f([s]) &::= 1 + f(s), \\ f(st) &::= 1 + f(s) + f(t). & (f \text{ concat case}) \end{aligned}$$

This definition may seem ok, but it isn’t: $f(\lambda)$ winds up with two values, and consequently:

$$\begin{aligned} 0 &= f(\lambda) & (f \text{ base case}) \\ &= f(\lambda \cdot \lambda) & (\text{concat def, base case}) \\ &= 1 + f(\lambda) + f(\lambda) & (f \text{ concat case}), \\ &= 1 + 0 + 0 = 1 & (f \text{ base case}). \end{aligned}$$

This is definitely not a situation we want to be in!

7.3 Recursive Functions on Nonnegative Integers

The nonnegative integers can be understood as a recursive data type.

Definition 7.3.1. The set, $\mathbb{N}$, is a data type defined recursively as:

- $0 \in \mathbb{N}$.
- If $n \in \mathbb{N}$, then the *successor*, $n + 1$, of n is in $\mathbb{N}$.

This of course makes it clear that ordinary induction is simply the special case of structural induction on the recursive Definition 7.3.1. This also justifies the familiar recursive definitions of functions on the nonnegative integers.

7.3.1 Some Standard Recursive Functions on $\mathbb{N}$

The Factorial function. This function is often written “ $n!$.” You will see a lot of it in later chapters. Here we’ll use the notation $\text{fac}(n)$:

- $\text{fac}(0) ::= 1$.
- $\text{fac}(n + 1) ::= (n + 1) \cdot \text{fac}(n)$ for $n \geq 0$.

The Fibonacci numbers. Fibonacci numbers arose out of an effort 800 years ago to model population growth. They have a continuing fan club of people captivated by their extraordinary properties. The n th Fibonacci number, fib , can be defined recursively by:

$$\begin{aligned} F(0) &::= 0, \\ F(1) &::= 1, \\ F(n) &::= F(n - 1) + F(n - 2) \quad \text{for } n \geq 2. \end{aligned}$$

Here the recursive step starts at $n = 2$ with base cases for 0 and 1. This is needed since the recursion relies on two previous values.

What is $F(4)$? Well, $F(2) = F(1) + F(0) = 1$, $F(3) = F(2) + F(1) = 2$, so $F(4) = 3$. The sequence starts out 0, 1, 1, 2, 3, 5, 8, 13, 21, . . .

Sum-notation. Let “ $S(n)$ ” abbreviate the expression “ $\sum_{i=1}^n f(i)$.” We can recursively define $S(n)$ with the rules

- $S(0) ::= 0$.
- $S(n + 1) ::= f(n + 1) + S(n)$ for $n \geq 0$.

7.3.2 Ill-formed Function Definitions

There are some other blunders to watch out for when defining functions recursively. The main problems come when recursive definitions don’t follow the recursive definition of the underlying data type. Below are some function specifications that resemble good definitions of functions on the nonnegative integers, but they aren’t.

$$f_1(n) ::= 2 + f_1(n - 1). \tag{7.2}$$

This “definition” has no base case. If some function, f_1 , satisfied (7.2), so would a function obtained by adding a constant to the value of f_1 . So equation (7.2) does not uniquely define an f_1 .

$$f_2(n) ::= \begin{cases} 0, & \text{if } n = 0, \\ f_2(n + 1) & \text{otherwise.} \end{cases} \quad (7.3)$$

This “definition” has a base case, but still doesn’t uniquely determine f_2 . Any function that is 0 at 0 and constant everywhere else would satisfy the specification, so (7.3) also does not uniquely define anything.

In a typical programming language, evaluation of $f_2(1)$ would begin with a recursive call of $f_2(2)$, which would lead to a recursive call of $f_2(3)$, ... with recursive calls continuing without end. This “operational” approach interprets (7.3) as defining a *partial* function, f_2 , that is undefined everywhere but 0.

$$f_3(n) ::= \begin{cases} 0, & \text{if } n \text{ is divisible by 2,} \\ 1, & \text{if } n \text{ is divisible by 3,} \\ 2, & \text{otherwise.} \end{cases} \quad (7.4)$$

This “definition” is inconsistent: it requires $f_3(6) = 0$ and $f_3(6) = 1$, so (7.4) doesn’t define anything.

Mathematicians have been wondering about this function specification for a while:

$$f_4(n) ::= \begin{cases} 1, & \text{if } n \leq 1, \\ f_4(n/2) & \text{if } n > 1 \text{ is even,} \\ f_4(3n + 1) & \text{if } n > 1 \text{ is odd.} \end{cases} \quad (7.5)$$

For example, $f_4(3) = 1$ because

$$f_4(3) ::= f_4(10) ::= f_4(5) ::= f_4(16) ::= f_4(8) ::= f_4(4) ::= f_4(2) ::= f_4(1) ::= 1.$$

The constant function equal to 1 will satisfy (7.5) (why?), but it’s not known if another function does too. The problem is that the third case specifies $f_4(n)$ in terms of f_4 at arguments larger than n , and so cannot be justified by induction on $\mathbb{N}$. It’s known that any f_4 satisfying (7.5) equals 1 for all n up to over a billion.

A final example is Ackermann’s function, which is an extremely fast-growing function of two nonnegative arguments. Its inverse is correspondingly slow-growing—it grows slower than $\log n$, $\log \log n$, $\log \log \log n$, ..., but it does grow unboundly. This inverse actually comes up analyzing a useful, highly efficient procedure known as the *Union-Find algorithm*. This algorithm was conjectured to run in a number of steps that grew linearly in the size of its input, but turned out to be

“linear” but with a slow growing coefficient nearly equal to the inverse Ackermann function. This means that pragmatically *Union-Find* is linear since the theoretically growing coefficient is less than 5 for any input that could conceivably come up.

Ackermann’s function can be defined recursively as the function, A , given by the following rules:

$$A(m, n) = 2n, \quad \text{if } m = 0 \text{ or } n \leq 1, \quad (7.6)$$

$$A(m, n) = A(m - 1, A(m, n - 1)), \quad \text{otherwise.} \quad (7.7)$$

Now these rules are unusual because the definition of $A(m, n)$ involves an evaluation of A at arguments that may be a lot bigger than m and n . The definitions of f_2 above showed how definitions of function values at small argument values in terms of larger one can easily lead to nonterminating evaluations. The definition of Ackermann’s function is actually ok, but proving this takes some ingenuity (see Problem 7.16).

7.4 Arithmetic Expressions

Expression evaluation is a key feature of programming languages, and recognition of expressions as a recursive data type is a key to understanding how they can be processed.

To illustrate this approach we’ll work with a toy example: arithmetic expressions like $3x^2 + 2x + 1$ involving only one variable, “ x .” We’ll refer to the data type of such expressions as *Aexp*. Here is its definition:

Definition 7.4.1.

- **Base cases:**
 - The variable, x , is in *Aexp*.
 - The arabic numeral, k , for any nonnegative integer, k , is in *Aexp*.
- **Constructor cases:** If $e, f \in \text{Aexp}$, then
 - $[e + f] \in \text{Aexp}$. The expression $[e + f]$ is called a *sum*. The *Aexp*’s e and f are called the *components* of the sum; they’re also called the *summands*.

- $[e * f] \in \text{Aexp}$. The expression $[e * f]$ is called a *product*. The Aexp’s e and f are called the *components* of the product; they’re also called the *multiplier* and *multiplicand*.
- $-[e] \in \text{Aexp}$. The expression $-[e]$ is called a *negative*.

Notice that Aexp’s are fully bracketed, and exponents aren’t allowed. So the Aexp version of the polynomial expression $3x^2 + 2x + 1$ would officially be written as

$$[[3 * [x * x]] + [[2 * x] + 1]]. \quad (7.8)$$

These brackets and *’s clutter up examples, so we’ll often use simpler expressions like “ $3x^2 + 2x + 1$ ” instead of (7.8). But it’s important to recognize that $3x^2 + 2x + 1$ is not an Aexp; it’s an *abbreviation* for an Aexp.

7.4.1 Evaluation and Substitution with Aexp’s

Evaluating Aexp’s

Since the only variable in an Aexp is x , the value of an Aexp is determined by the value of x . For example, if the value of x is 3, then the value of $3x^2 + 2x + 1$ is obviously 34. In general, given any Aexp, e , and an integer value, n , for the variable, x , we can evaluate e to find its value, $\text{eval}(e, n)$. It’s easy, and useful, to specify this evaluation process with a recursive definition.

Definition 7.4.2. The *evaluation function*, $\text{eval} : \text{Aexp} \times \mathbb{Z} \rightarrow \mathbb{Z}$, is defined recursively on expressions, $e \in \text{Aexp}$, as follows. Let n be any integer.

- **Base cases:**

$$\text{eval}(x, n) ::= n, \quad (\text{value of variable } x \text{ is } n.) \quad (7.9)$$

$$\text{eval}(k, n) ::= k, \quad (\text{value of numeral } k \text{ is } k, \text{ regardless of } x.) \quad (7.10)$$

- **Constructor cases:**

$$\text{eval}([e_1 + e_2], n) ::= \text{eval}(e_1, n) + \text{eval}(e_2, n), \quad (7.11)$$

$$\text{eval}([e_1 * e_2], n) ::= \text{eval}(e_1, n) \cdot \text{eval}(e_2, n), \quad (7.12)$$

$$\text{eval}(-[e_1], n) ::= -\text{eval}(e_1, n). \quad (7.13)$$

For example, here’s how the recursive definition of `eval` would arrive at the value of $3 + x^2$ when x is 2:

$$\begin{aligned}
 \text{eval}([3 + [x * x]], 2) &= \text{eval}(3, 2) + \text{eval}([x * x], 2) && \text{(by Def 7.4.2.7.11)} \\
 &= 3 + \text{eval}([x * x], 2) && \text{(by Def 7.4.2.7.10)} \\
 &= 3 + (\text{eval}(x, 2) \cdot \text{eval}(x, 2)) && \text{(by Def 7.4.2.7.12)} \\
 &= 3 + (2 \cdot 2) && \text{(by Def 7.4.2.7.9)} \\
 &= 3 + 4 = 7.
 \end{aligned}$$

Substituting into Aexp’s

Substituting expressions for variables is a standard operation used by compilers and algebra systems. For example, the result of substituting the expression $3x$ for x in the expression $x(x - 1)$ would be $3x(3x - 1)$. We’ll use the general notation $\text{subst}(f, e)$ for the result of substituting an Aexp, f , for each of the x ’s in an Aexp, e . So as we just explained,

$$\text{subst}(3x, x(x - 1)) = 3x(3x - 1).$$

This substitution function has a simple recursive definition:

Definition 7.4.3. The *substitution function* from $\text{Aexp} \times \text{Aexp}$ to Aexp is defined recursively on expressions, $e \in \text{Aexp}$, as follows. Let f be any Aexp.

- **Base cases:**

$$\text{subst}(f, x) ::= f, \quad \text{(subbing } f \text{ for variable, } x, \text{ just gives } f) \quad (7.14)$$

$$\text{subst}(f, k) ::= k \quad \text{(subbing into a numeral does nothing.)} \quad (7.15)$$

- **Constructor cases:**

$$\text{subst}(f, [e_1 + e_2]) ::= [\text{subst}(f, e_1) + \text{subst}(f, e_2)] \quad (7.16)$$

$$\text{subst}(f, [e_1 * e_2]) ::= [\text{subst}(f, e_1) * \text{subst}(f, e_2)] \quad (7.17)$$

$$\text{subst}(f, -[e_1]) ::= -[\text{subst}(f, e_1)]. \quad (7.18)$$

Here’s how the recursive definition of the substitution function would find the result of substituting $3x$ for x in the $x(x - 1)$:

$$\begin{aligned}
 & \text{subst}(3x, x(x - 1)) \\
 &= \text{subst}([3 * x], [x * [x + -[1]]]) && \text{(unabbreviating)} \\
 &= [\text{subst}([3 * x], x) * \text{subst}([3 * x], [x + -[1]])] && \text{(by Def 7.4.3 7.17)} \\
 &= [[3 * x] * \text{subst}([3 * x], [x + -[1]])] && \text{(by Def 7.4.3 7.14)} \\
 &= [[3 * x] * [\text{subst}([3 * x], x) + \text{subst}([3 * x], -[1])]] && \text{(by Def 7.4.3 7.16)} \\
 &= [[3 * x] * [[3 * x] + -[\text{subst}([3 * x], 1)]]] && \text{(by Def 7.4.3 7.14 \& 7.18)} \\
 &= [[3 * x] * [[3 * x] + -[1]]] && \text{(by Def 7.4.3 7.15)} \\
 &= 3x(3x - 1) && \text{(abbreviation)}
 \end{aligned}$$

Now suppose we have to find the value of $\text{subst}(3x, x(x - 1))$ when $x = 2$. There are two approaches.

First, we could actually do the substitution above to get $3x(3x - 1)$, and then we could evaluate $3x(3x - 1)$ when $x = 2$, that is, we could recursively calculate $\text{eval}(3x(3x - 1), 2)$ to get the final value 30. In programming jargon, this would be called evaluation using the *Substitution Model*. Because the formula $3x$ appears twice after substitution, the multiplication $3 \cdot 2$ that computes its value gets performed twice.

The other approach is called evaluation using the *Environment Model*. Namely, to compute

$$\text{eval}(\text{subst}(3x, x(x - 1)), 2) \tag{7.19}$$

we evaluate $3x$ when $x = 2$ using just 1 multiplication to get the value 6. Then we evaluate $x(x - 1)$ when x has this value 6 to arrive at the value $6 \cdot 5 = 30$. So the Environment Model only computes the value of $3x$ once and so requires one fewer multiplication than the Substitution model to compute (7.19). But how do we know that these final values reached by these two approaches always agree? We can prove this easily by structural induction on the definitions of the two approaches. More precisely, what we want to prove is

Theorem 7.4.4. *For all expressions $e, f \in \text{Aexp}$ and $n \in \mathbb{Z}$,*

$$\text{eval}(\text{subst}(f, e), n) = \text{eval}(f, \text{eval}(e, n)). \tag{7.20}$$

Proof. The proof is by structural induction on e .¹

Base cases:

¹This is an example of why it’s useful to notify the reader what the induction variable is—in this case it isn’t n .

- Case[x]

The left hand side of equation (7.20) equals $\text{eval}(f, n)$ by this base case in Definition 7.4.3 of the substitution function, and the right hand side also equals $\text{eval}(f, n)$ by this base case in Definition 7.4.2 of eval .

- Case[k].

The left hand side of equation (7.20) equals k by this base case in Definitions 7.4.3 and 7.4.2 of the substitution and evaluation functions. Likewise, the right hand side equals k by two applications of this base case in the Definition 7.4.2 of eval .

Constructor cases:

- Case[$[e_1 + e_2]$]

By the structural induction hypothesis (7.20), we may assume that for all $f \in \text{Aexp}$ and $n \in \mathbb{Z}$,

$$\text{eval}(\text{subst}(f, e_i), n) = \text{eval}(e_i, \text{eval}(f, n)) \quad (7.21)$$

for $i = 1, 2$. We wish to prove that

$$\text{eval}(\text{subst}(f, [e_1 + e_2]), n) = \text{eval}([e_1 + e_2], \text{eval}(f, n)) \quad (7.22)$$

But the left hand side of (7.22) equals

$$\text{eval}([\text{subst}(f, e_1) + \text{subst}(f, e_2)], n)$$

by Definition 7.4.3.7.16 of substitution into a sum expression. But this equals

$$\text{eval}(\text{subst}(f, e_1), n) + \text{eval}(\text{subst}(f, e_2), n)$$

by Definition 7.4.2.(7.11) of eval for a sum expression. By induction hypothesis (7.21), this in turn equals

$$\text{eval}(e_1, \text{eval}(f, n)) + \text{eval}(e_2, \text{eval}(f, n)).$$

Finally, this last expression equals the right hand side of (7.22) by Definition 7.4.2.(7.11) of eval for a sum expression. This proves (7.22) in this case.

- Case[$[e_1 * e_2]$] Similar.

- Case[$[-e_1]$] Even easier.

This covers all the constructor cases, and so completes the proof by structural induction. ■

7.5 Induction in Computer Science

Induction is a powerful and widely applicable proof technique, which is why we’ve devoted two entire chapters to it. Strong induction and its special case of ordinary induction are applicable to any kind of thing with nonnegative integer sizes –which is an awful lot of things, including all step-by-step computational processes.

Structural induction then goes beyond number counting, and offers a simple, natural approach to proving things about recursive data types and recursive computation. This makes it a technique every computer scientist should embrace.

Problems for Section 7.1

Class Problems

Problem 7.1.

Prove that for all strings $r, s, t \in A^*$

$$(r \cdot s) \cdot t = r \cdot (s \cdot t).$$

Problem 7.2.

The *reversal* of a string is the string written backwards, for example, $\text{rev}(abcde) = edcba$.

(a) Give a simple recursive definition of $\text{rev}(s)$ based on the recursive definition 7.1.1 of $s \in A^*$ and using the concatenation operation 7.1.3.

(b) Prove that

$$\text{rev}(s \cdot t) = \text{rev}(t) \cdot \text{rev}(s),$$

for all strings $s, t \in A^*$.

Problem 7.3.

The Elementary 18.01 Functions (F18’s) are the set of functions of one real variable defined recursively as follows:

Base cases:

- The identity function, $\text{id}(x) ::= x$ is an F18,
- any constant function is an F18,
- the sine function is an F18,

Constructor cases:

If f, g are F18's, then so are

1. $f + g, fg, 2^g$,
2. the inverse function $f^{(-1)}$,
3. the composition $f \circ g$.

(a) Prove that the function $1/x$ is an F18.

Warning: Don't confuse $1/x = x^{-1}$ with the inverse, $\text{id}^{(-1)}$ of the identity function $\text{id}(x)$. The inverse $\text{id}^{(-1)}$ is equal to id .

(b) Prove by Structural Induction on this definition that the Elementary 18.01 Functions are *closed under taking derivatives*. That is, show that if $f(x)$ is an F18, then so is $f' ::= df/dx$. (Just work out 2 or 3 of the most interesting constructor cases; you may skip the less interesting ones.)

Problem 7.4.

Here is a simple recursive definition of the set, E , of even integers:

Definition. Base case: $0 \in E$.

Constructor cases: If $n \in E$, then so are $n + 2$ and $-n$.

Provide similar simple recursive definitions of the following sets:

- (a) The set $S ::= \{2^k 3^m 5^n \mid k, m, n \in \mathbb{N}\}$.
- (b) The set $T ::= \{2^k 3^{2k+m} 5^{m+n} \mid k, m, n \in \mathbb{N}\}$.
- (c) The set $L ::= \{(a, b) \in \mathbb{Z}^2 \mid 3 \mid (a - b)\}$.

Let L' be the set defined by the recursive definition you gave for L in the previous part. Now if you did it right, then $L' = L$, but maybe you made a mistake. So let's check that you got the definition right.

(d) Prove by structural induction on your definition of L' that

$$L' \subseteq L.$$

(e) Confirm that you got the definition right by proving that

$$L \subseteq L'.$$

(f) See if you can give an *unambiguous* recursive definition of L .

Problem 7.5.

Definition. The recursive data type, binary-2PTG, of *binary trees* with leaf labels, L , is defined recursively as follows:

- **Base case:** $\langle \text{leaf}, l \rangle \in \text{binary-2PTG}$, for all labels $l \in L$.
- **Constructor case:** If $G_1, G_2 \in \text{binary-2PTG}$, then

$$\langle \text{bintree}, G_1, G_2 \rangle \in \text{binary-2PTG}.$$

The *size*, $|G|$, of $G \in \text{binary-2PTG}$ is defined recursively on this definition by:

- **Base case:**

$$|\langle \text{leaf}, l \rangle| ::= 1, \quad \text{for all } l \in L.$$

- **Constructor case:**

$$|\langle \text{bintree}, G_1, G_2 \rangle| ::= |G_1| + |G_2| + 1.$$

For example, for the size of the binary-2PTG, G , pictured in Figure 7.1, is 7.

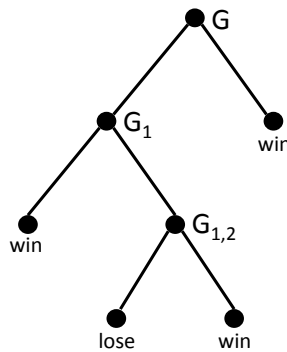


Figure 7.1 A picture of a binary tree w .

(a) Write out (using angle brackets and labels `bintree`, `leaf`, etc.) the binary-2PTG, G , pictured in Figure 7.1.

The value of $\text{flatten}(G)$ for $G \in \text{binary-2PTG}$ is the sequence of labels in L of the leaves of G . For example, for the binary-2PTG, G , pictured in Figure 7.1,

$$\text{flatten}(G) = (\text{win}, \text{lose}, \text{win}, \text{win}).$$

(b) Give a recursive definition of flatten . (You may use the operation of *concatenation* (append) of two sequences.)

(c) Prove by structural induction on the definitions of flatten and size that

$$2 \cdot \text{length}(\text{flatten}(G)) = |G| + 1. \quad (7.23)$$

Homework Problems

Problem 7.6.

Let m, n be integers, not both zero. Define a set of integers, $L_{m,n}$, recursively as follows:

- **Base cases:** $m, n \in L_{m,n}$.
- **Constructor cases:** If $j, k \in L_{m,n}$, then
 1. $-j \in L_{m,n}$,
 2. $j + k \in L_{m,n}$.

Let L be an abbreviation for $L_{m,n}$ in the rest of this problem.

(a) Prove *by structural induction* that every common divisor of m and n also divides every member of L .

(b) Prove that any integer multiple of an element of L is also in L .

(c) Show that if $j, k \in L$ and $k \neq 0$, then $\text{rem}(j, k) \in L$.

(d) Show that there is a positive integer $g \in L$ which divides every member of L .
Hint: The least positive integer in L .

(e) Conclude that $g = \text{GCD}(m, n)$ for g from part (d).

Problem 7.7.

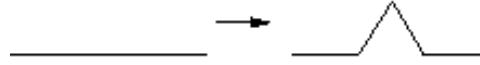


Figure 7.2 Constructing the Koch Snowflake.

Definition. Define the number, $\#_c(s)$, of occurrences of the character $c \in A$ in the string s recursively on the definition of $s \in A^*$:

base case: $\#_c(\lambda) ::= 0$.

constructor case:

$$\#_c(\langle a, s \rangle) ::= \begin{cases} \#_c(s) & \text{if } a \neq c, \\ 1 + \#_c(s) & \text{if } a = c. \end{cases}$$

Prove by structural induction that for all $s, t \in A^*$ and $c \in A$

$$\#_c(s \cdot t) = \#_c(s) + \#_c(t).$$

Problem 7.8.

Fractals are example of a mathematical object that can be defined recursively. In this problem, we consider the Koch snowflake. Any Koch snowflake can be constructed by the following recursive definition.

- **base case:** An equilateral triangle with a positive integer side length is a Koch snowflake.
- **constructor case:** Let K be a Koch snowflake, and let l be a line segment on the snowflake. Remove the middle third of l , and replace it with two line segments of the same length as is done in Figure 7.2

The resulting figure is also a Koch snowflake.

Prove by structural induction that the area inside any Koch snowflake is of the form $q\sqrt{3}$, where q is a rational number.

Problem 7.9.

Let L be some convenient set whose elements will be called *labels*. The labeled binary trees, LBT's, are defined recursively as follows:

Definition. If l is a label,

Base case: $\langle l, \text{leaf} \rangle$ is an LBT, and

Constructor case: if B and C are LBT's, then $\langle l, B, C \rangle$ is an LBT.

The *leaf-labels* and *internal-labels* of an LBT, are defined recursively in the obvious way:

Definition. Base case: The set of leaf-labels of the LBT $\langle l, \text{leaf} \rangle$ is $\{l\}$ and its set of internal-labels is the empty set.

Constructor case: The set of leaf labels of the LBT $\langle l, B, C \rangle$ is the union of the leaf-labels of B and of C ; the set of internal-labels is the union of $\{l\}$ and the sets of internal-labels of B and of C .

The set of *labels* of an LBT is the union of its leaf- and internal-labels.

The LBT's with *unique* labels are also defined recursively:

Definition. Base case: The LBT $\langle l, \text{leaf} \rangle$ has *unique labels*.

Constructor case: The LBT $\langle l, B, C \rangle$ has *unique labels* iff l is not a label of B or C , and no label is a label of both B and C .

If B is an LBT, let n_B be the number of internal-labels appearing in B and f_B be the number of leaf labels of B .

Prove by structural induction that

$$f_B = n_B + 1 \quad (7.24)$$

for all LBT's *with unique labels*. This equation can obviously fail if labels are not unique, so your proof had better use uniqueness of labels at some point; be sure to indicate where.

Exam Problems

Problem 7.10.

The Arithmetic Trig Functions (*Atrig*'s) are the set of functions of one real variable defined recursively as follows:

Base cases:

- The identity function, $\text{id}(x) ::= x$ is an *Atrig*,
- any constant function is an *Atrig*,
- the sine function is an *Atrig*,

Constructor cases:

If f, g are *Atrig*'s, then so are

1. $f + g$
2. $f \cdot g$
3. the composition $f \circ g$.

Prove by Structural Induction on this definition that if $f(x)$ is an *Atrig*, then so is $f' ::= df/dx$.

Problem 7.11.

The *Limited* 18.01 Functions (LF18's) are defined similarly to the F18 functions from class problem 7.3, but they don't have function composition or inverse as a constructor. Namely,

Definition. LF18 is the set of functions of one complex variable defined recursively as follows:

Base cases:

- The identity function, $\text{id}(z) ::= z$ for $z \in \mathbb{C}$, is an LF18,
- any constant function is an LF18.

Constructor cases: If f, g are LF18's, then so are

1. $f + g$, fg , and 2^f .

Prove by structural induction that LF18 is closed under composition. That is, using the induction hypothesis,

$$P(f) ::= \forall g \in \text{LF18}. f \circ g \in \text{LF18},$$

prove that $P(f)$ holds for all $f \in \text{LF18}$. Make sure to indicate explicitly

- each of the base cases, and
- each of the constructor cases.

Problems for Section 7.2

Practice Problems

Problem 7.12. (a) To prove that the set *RecMatch*, of matched strings of Definition 7.2.1 equals the set *AmbRecMatch* of ambiguous matched strings of Definition 7.2.2, you could first prove that

$$\forall r \in \text{RecMatch}. r \in \text{AmbRecMatch},$$

and then prove that

$$\forall u \in \text{AmbRecMatch}. u \in \text{RecMatch}.$$

Of these two statements, circle the one that would be simpler to prove by structural induction directly from the definitions.

(b) Suppose structural induction was being used to prove that $\text{AmbRecMatch} \subseteq \text{RecMatch}$. Circle the one predicate below that would fit the format for a structural induction hypothesis in such a proof.

- $P_0(n) ::= |s| \leq n \text{ IMPLIES } s \in \text{RecMatch}.$
- $P_1(n) ::= |s| \leq n \text{ IMPLIES } s \in \text{AmbRecMatch}.$
- $P_2(s) ::= s \in \text{RecMatch}.$
- $P_3(s) ::= s \in \text{AmbRecMatch}.$
- $P_4(s) ::= (s \in \text{RecMatch} \text{ IMPLIES } s \in \text{AmbRecMatch}).$

(c) The recursive definition AmbRecMatch is ambiguous because it allows the $s \cdot t$ constructor to apply when s or t is the empty string. But even fixing that, ambiguity remains. Demonstrate this by giving two different derivations for the string $[][][]$ according to AmbRecMatch but only using the $s \cdot t$ constructor when $s \neq \lambda$ and $t \neq \lambda$.

Class Problems

Problem 7.13.

Let p be the string $[]$. A string of brackets is said to be *erasable* iff it can be reduced to the empty string by repeatedly erasing occurrences of p . For example, here’s how to erase the string $[[[]][[]]$:

$$[[[]][[]] \rightarrow [[]] \rightarrow [] \rightarrow \lambda.$$

On the other hand the string $[]][[[[[]]$ is not erasable because when we try to erase, we get stuck:

$$[]][[[[[]] \rightarrow][[[[[]] \rightarrow][[[\not\rightarrow$$

Let Erasable be the set of erasable strings of brackets. Let RecMatch be the recursive data type of strings of *matched* brackets given in Definition 7.2.1.

(a) Use structural induction to prove that

$$\text{RecMatch} \subseteq \text{Erasable}.$$

(b) Supply the missing parts (labeled by “(*)”) of the following proof that

$$\text{Erasable} \subseteq \text{RecMatch}.$$

Proof. We prove by strong induction that every length- n string in Erasable is also in RecMatch. The induction hypothesis is

$$P(n) ::= \forall x \in \text{Erasable}. |x| = n \text{ IMPLIES } x \in \text{RecMatch}.$$

Base case:

(*) What is the base case? Prove that P is true in this case.

Inductive step: To prove $P(n + 1)$, suppose $|x| = n + 1$ and $x \in \text{Erasable}$. We need to show that $x \in \text{RecMatch}$.

Let’s say that a string y is an *erase* of a string z iff y is the result of erasing a *single* occurrence of p in z .

Since $x \in \text{Erasable}$ and has positive length, there must be an erase, $y \in \text{Erasable}$, of x . So $|y| = n - 1 \geq 0$, and since $y \in \text{Erasable}$, we may assume by induction hypothesis that $y \in \text{RecMatch}$.

Now we argue by cases:

Case (y is the empty string):

(*) Prove that $x \in \text{RecMatch}$ in this case.

Case ($y = [s]t$ for some strings $s, t \in \text{RecMatch}$): Now we argue by subcases.

- **Subcase** ($x = py$):

(*) Prove that $x \in \text{RecMatch}$ in this subcase.

- **Subcase** (x is of the form $[s']t$ where s is an erase of s'):

Since $s \in \text{RecMatch}$, it is erasable by part (b), which implies that $s' \in \text{Erasable}$. But $|s'| < |x|$, so by induction hypothesis, we may assume that $s' \in \text{RecMatch}$. This shows that x is the result of the constructor step of RecMatch, and therefore $x \in \text{RecMatch}$.

- **Subcase** (x is of the form $[s]t'$ where t is an erase of t'):

(*) Prove that $x \in \text{RecMatch}$ in this subcase.

(*) Explain why the above cases are sufficient.

This completes the proof by strong induction on n , so we conclude that $P(n)$ holds for all $n \in \mathbb{N}$. Therefore $x \in \text{RecMatch}$ for every string $x \in \text{Erasable}$. That is, $\text{Erasable} \subseteq \text{RecMatch}$. Combined with part (a), we conclude that

$$\text{Erasable} = \text{RecMatch}.$$



Problem 7.14. (a) Prove that the set RecMatch , of matched strings of Definition 7.2.1 is closed under string concatenation. Namely, if $s, t \in \text{RecMatch}$, then $s \cdot t \in \text{RecMatch}$.

(b) Prove $\text{AmbRecMatch} \subseteq \text{RecMatch}$, where AmbRecMatch is the set of ambiguous matched strings of Definition 7.2.2.

(c) Prove that $\text{RecMatch} = \text{AmbRecMatch}$.

Problem 7.15.

One way to determine if a string has matching brackets, that is, if it is in the set, RecMatch , of Definition 7.2.1 is to start with 0 and read the string from left to right, adding 1 to the count for each left bracket and subtracting 1 from the count for each right bracket. For example, here are the counts for two sample strings:

	[	]		[		[	[	[	[	]	]	]	]
0	1	0	-1	0	1	2	3	4	3	2	1	0	

	[	[		[	]	]	[	]	]	[	]
0	1	2	3	2	1	2	1	0	1	0	

A string has a *good count* if its running count never goes negative and ends with 0. So the second string above has a good count, but the first one does not because its count went negative at the third step. Let

$$\text{GoodCount} ::= \{s \in \{[, [\}^* \mid s \text{ has a good count}\}.$$

The empty string has a length 0 running count we'll take as a good count by convention, that is, $\lambda \in \text{GoodCount}$. The matched strings can now be characterized precisely as this set of strings with good counts.

(a) Prove that GoodCount contains RecMatch by structural induction on the definition of RecMatch .

(b) Conversely, prove that RecMatch contains GoodCount .

Hint: By induction on the length of strings in GoodCount . Consider when the running count equals 0 for the second time.

(c) Give an example of a small string $s \in \text{RecMatch}$ such that $\llbracket s \rrbracket \neq \text{erase}(e)$ for any $e \in \text{Aexp}$.

8 Number Theory

Number theory is the study of the integers. *Why* anyone would want to study the integers is not immediately obvious. First of all, what’s to know? There’s 0, there’s 1, 2, 3, and so on, and, oh yeah, -1, -2, Which one don’t you understand? Second, what practical value is there in it?

The mathematician G. H. Hardy expressed pleasure in its impracticality when he wrote:

[Number theorists] may be justified in rejoicing that there is one science, at any rate, and that their own, whose very remoteness from ordinary human activities should keep it gentle and clean.

Hardy was specially concerned that number theory not be used in warfare; he was a pacifist. You may applaud his sentiments, but he got it wrong: number theory underlies modern cryptography, which is what makes secure online communication possible. Secure communication is of course crucial in war—which may leave poor Hardy spinning in his grave. It’s also central to online commerce. Every time you buy a book from Amazon, use a certificate to access a web page, or use a PayPal account, you are relying on number theoretic algorithms.

Number theory also provides an excellent environment for us to practice and apply the proof techniques that we developed in previous chapters. We’ll work out properties of greatest common divisors (gcd’s) and use them to prove that integers factor uniquely into primes. Then we’ll introduce modular arithmetic and work out enough of its properties to explain the RSA public key crypto-system.

Since we’ll be focusing on properties of the integers, we’ll adopt the default convention in this chapter that *variables range over the set, $\mathbb{Z}$, of integers*.

8.1 Divisibility

The nature of number theory emerges as soon as we consider the *divides* relation, where

Definition 8.1.1.

$$a \mid b ::= [ak = b \text{ for some } k].$$

The divides relation comes up so frequently that multiple synonyms for it are used all the time. The following phrases all say the same thing:

- $a \mid b$,
- a divides b ,
- a is a *divisor* of b ,
- a is a *factor* of b ,
- b is *divisible* by a ,
- b is a *multiple* of a .

Some immediate consequences of Definition 8.1.1 are that $n \mid 0$, $n \mid n$, and $1 \mid n$ for all $n \neq 0$.

Dividing seems simple enough, but let’s play with this definition. The Pythagoreans, an ancient sect of mathematical mystics, said that a number is *perfect* if it equals the sum of its positive integral divisors, excluding itself. For example, $6 = 1 + 2 + 3$ and $28 = 1 + 2 + 4 + 7 + 14$ are perfect numbers. On the other hand, 10 is not perfect because $1 + 2 + 5 = 8$, and 12 is not perfect because $1 + 2 + 3 + 4 + 6 = 16$. Euclid characterized all the *even* perfect numbers around 300 BC. But is there an *odd* perfect number? More than two thousand years later, we still don’t know! All numbers up to about 10^{300} have been ruled out, but no one has proved that there isn’t an odd perfect number waiting just over the horizon.

So a half-page into number theory, we’ve strayed past the outer limits of human knowledge! This is pretty typical; number theory is full of questions that are easy to pose, but incredibly difficult to answer.¹

Some of the greatest insights and mysteries in number theory concern properties of *prime* numbers:

Definition 8.1.2. A *prime* is a number greater than 1 that is divisible only by itself and 1.

Several such problems are included in the box on the following page. Interestingly, we’ll see that computer scientists have found ways to turn some of these difficulties to their advantage.

8.1.1 Facts about Divisibility

The following lemma collects some basic facts about divisibility.

Lemma 8.1.3.

¹*Don’t Panic*—we’re going to stick to some relatively benign parts of number theory. These super-hard unsolved problems rarely get put on problem sets.

Famous Conjectures in Number Theory

Goldbach Conjecture Every even integer greater than two is equal to the sum of two primes. For example, $4 = 2 + 2$, $6 = 3 + 3$, $8 = 3 + 5$, etc. The conjecture holds for all numbers up to 10^{16} . In 1939 Schnirelman proved that every even number can be written as the sum of not more than 300,000 primes, which was a start. Today, we know that every even number is the sum of at most 6 primes.

Twin Prime Conjecture There are infinitely many primes p such that $p + 2$ is also a prime. In 1966 Chen showed that there are infinitely many primes p such that $p + 2$ is the product of at most two primes. So the conjecture is known to be *almost* true!

Primality Testing There is an efficient way to determine whether a number is prime. A naive search for factors of an integer n takes a number of steps proportional to $\sqrt{n}$, which is exponential in the *size* of n in decimal or binary notation. All known procedures for prime checking blew up like this on various inputs. Finally in 2002, an amazingly simple, new method was discovered by Agrawal, Kayal, and Saxena, which showed that prime testing only required a polynomial number of steps. Their paper began with a quote from Gauss emphasizing the importance and antiquity of the problem even in his time—two centuries ago. So prime testing is definitely not in the category of infeasible problems requiring an exponentially growing number of steps in bad cases.

Factoring Given the product of two large primes $n = pq$, there is no efficient way to recover the primes p and q . The best known algorithm is the “number field sieve,” which runs in time proportional to:

$$e^{1.9(\ln n)^{1/3}(\ln \ln n)^{2/3}}$$

This is infeasible when n has 300 digits or more.

Fermat’s Last Theorem There are no positive integers x , y , and z such that

$$x^n + y^n = z^n$$

for some integer $n > 2$. In a book he was reading around 1630, Fermat claimed to have a proof but not enough space in the margin to write it down. Wiles finally gave a proof of the theorem in 1994, after seven years of working in secrecy and isolation in his attic. His proof did not fit in any margin.

1. If $a \mid b$ and $b \mid c$, then $a \mid c$.
2. If $a \mid b$ and $a \mid c$, then $a \mid sb + tc$ for all s and t .
3. For all $c \neq 0$, $a \mid b$ if and only if $ca \mid cb$.

Proof. These facts all follow directly from Definition 8.1.1, and we’ll just prove part 2. for practice:

Given that $a \mid b$, there is some $k_1 \in \mathbb{Z}$ such that $ak_1 = b$. Likewise, $ak_2 = c$, so

$$sb + tc = s(k_1a) + t(k_2a) = (sk_1 + tk_2)a.$$

Therefore $sb + tc = k_3a$ where $k_3 ::= (sk_1 + tk_2)$, which means that

$$a \mid sb + tc.$$

■

A number of the form $sb + tc$ is called an *integer linear combination* of b and c , or a plain *linear combination*, since in this chapter we’re only talking about integers. So Lemma 8.1.3.2 can be rephrased as

If a divides b and c , then a divides every linear combination of b and c .

We’ll be making good use of linear combinations, so let’s get the general definition on record:

Definition 8.1.4. An integer n is a *linear combination* of numbers $b_0, \dots, b_n$ iff

$$n = s_0b_0 + s_1b_1 + \dots + s_nb_n$$

for some integers $s_0, \dots, s_n$.

8.1.2 When Divisibility Goes Bad

As you learned in elementary school, if one number does *not* evenly divide another, you get a “quotient” and a “remainder” left over. More precisely:

Theorem 8.1.5. [Division Theorem]² Let n and d be integers such that $d > 0$. Then there exists a unique pair of integers q and r , such that

$$n = q \cdot d + r \text{ AND } 0 \leq r < d. \tag{8.1}$$

²This theorem is often called the “Division Algorithm,” even though it is not what we would call an algorithm. We will take this familiar result for granted without proof.

The number q is called the *quotient* and the number r is called the *remainder* of n divided by d . We use the notation $\text{qcnt}(n, d)$ for the quotient and $\text{rem}(n, d)$ for the remainder.

For example, $\text{qcnt}(2716, 10) = 271$ and $\text{rem}(2716, 10) = 6$, since $2716 = 271 \cdot 10 + 6$. Similarly, $\text{rem}(-11, 7) = 3$, since $-11 = (-2) \cdot 7 + 3$. There is a remainder operator built into many programming languages. For example, “32 % 5” will be familiar as remainder notation to programmers in Java, C, and C++; it evaluates to $\text{rem}(32, 5) = 2$ in all three languages. On the other hand, these languages treat quotients involving negative numbers idiosyncratically, so if you program in one those languages, remember to stick to the definition according to the Division Theorem 8.1.5.

The remainder on division by n is a number in the interval from 0 to $n - 1$. Such intervals come up so often that it is useful to have a simple notation for them.

$$(k, n) ::= \{i \mid k < i < n\}$$

$$[k, n) ::= (k, n) \cup \{k\}$$

$$(k, n] ::= (k, n) \cup \{n\}$$

$$[k, n] ::= (k, n) \cup \{k, n\}$$

8.1.3 Die Hard

Die Hard 3 is just a B-grade action movie, but we think it has an inner message: everyone should learn at least a little number theory. In Section 6.4.4, we formalized a state machine for the Die Hard jug-filling problem using 3 and 5 gallon jugs, and also with 3 and 9 gallon jugs, and came to different conclusions about bomb explosions. What’s going on in general? For example, how about getting 4 gallons from 12- and 18-gallon jugs, getting 32 gallons with 899- and 1147-gallon jugs, or getting 3 gallons into a jug using just 21- and 26-gallon jugs?

It would be nice if we could solve all these silly water jug questions at once. This is where number theory comes in handy.

Finding an Invariant Property

Suppose that we have water jugs with capacities a and b with $b \geq a$. Let’s carry out some sample operations of the state machine and see what happens, assuming

the b -jug is big enough:

$(0, 0) \rightarrow (a, 0)$	fill first jug
$\rightarrow (0, a)$	pour first into second
$\rightarrow (a, a)$	fill first jug
$\rightarrow (2a - b, b)$	pour first into second (assuming $2a \geq b$)
$\rightarrow (2a - b, 0)$	empty second jug
$\rightarrow (0, 2a - b)$	pour first into second
$\rightarrow (a, 2a - b)$	fill first
$\rightarrow (3a - 2b, b)$	pour first into second (assuming $3a \geq 2b$)

What leaps out is that at every step, the amount of water in each jug is of a linear combination of a and b . This is easy to prove by induction on the number of transitions:

Lemma 8.1.6 (Water Jugs). *In the Die Hard state machine of Section 6.4.4 with jugs of sizes a and b , the amount of water in each jug is always a linear combination of a and b .*

Proof. The induction hypothesis, $P(n)$, is the proposition that after n transitions, the amount of water in each jug is a linear combination of a and b .

Base case ($n = 0$): $P(0)$ is true, because both jugs are initially empty, and $0 \cdot a + 0 \cdot b = 0$.

Inductive step: Suppose the machine is in state (x, y) after n steps, that is, the little jug contains x gallons and the big one contains y gallons. There are two cases:

- If we fill a jug from the fountain or empty a jug into the fountain, then that jug is empty or full. The amount in the other jug remains a linear combination of a and b . So $P(n + 1)$ holds.
- Otherwise, we pour water from one jug to another until one is empty or the other is full. By our assumption, the amount x and y in each jug is a linear combination of a and b before we begin pouring. After pouring, one jug is either empty (contains 0 gallons) or full (contains a or b gallons). Thus, the other jug contains either $x + y$ gallons, $x + y - a$, or $x + y - b$ gallons, all of which are linear combinations of a and b since x and y are. So $P(n + 1)$ holds in this case as well.

Since $P(n + 1)$ holds in any case, this proves the inductive step, completing the proof by induction. ■

So we have established that the jug problem has an invariant property, namely that the amount of water in every jug is always a linear combination of the capacities of the jugs. Lemma 8.1.6 has an important corollary:

Corollary. *Getting 4 gallons from 12- and 18-gallon jugs, and likewise getting 32 gallons from 899- and 1147-gallon jugs,*

Bruce dies!

Proof. By the Water Jugs Lemma 8.1.6, with 12- and 18-gallon jugs, the amount in any jug is a linear combination of 12 and 18. This is always a multiple of 6 by Lemma 8.1.3.2, so Bruce can’t get 4 gallons. Likewise, the amount in any jug using 899- and 1147-gallon jugs is a multiple of 31, so he can’t get 32 either. ■

But the Water Jugs Lemma isn’t very satisfying. One problem is that it leaves the question of getting 3 gallons into a jug using just 21- and 26-gallon jugs unresolved, since the only positive factor of both 21 and 26 is 1, and of course 1 divides 3. A bigger problem is that we’ve just managed to recast a pretty understandable question about water jugs into a complicated question about linear combinations. This might not seem like a lot of progress. Fortunately, linear combinations are closely related to something more familiar, namely greatest common divisors, and these will help us solve the general water jug problem.

8.2 The Greatest Common Divisor

A *common divisor* of a and b is a number that divides them both. The *greatest common divisor* (*gcd*) of a and b is written $\gcd(a, b)$. For example, $\gcd(18, 24) = 6$. The gcd turns out to be a very valuable piece of information about the relationship between a and b and for reasoning about integers in general. So we’ll be making lots of arguments about gcd’s in what follows.

8.2.1 Euclid’s Algorithm

The first thing to figure out is how to find gcd’s. A good way called *Euclid’s Algorithm* has been known for several thousand years. It is based on the following elementary observation.

Lemma 8.2.1.

$$\gcd(a, b) = \gcd(b, \text{rem}(a, b)).$$

Proof. By the Division Theorem 8.1.5,

$$a = qb + r \tag{8.2}$$

where $r = \text{rem}(a, b)$. So a is a linear combination of b and r , which implies that any divisor of b and r is a divisor of a by Lemma 8.1.3.2. Likewise, r is a linear combination, $a - qb$, of a and b , so any divisor of a and b is a divisor of r . This means that a and b have the same common divisors as b and r , and so they have the same *greatest* common divisor. ■

Lemma 8.2.1 is useful for quickly computing the greatest common divisor of two numbers. For example, we could compute the greatest common divisor of 1147 and 899 by repeatedly applying it:

$$\begin{aligned} \gcd(1147, 899) &= \gcd(899, \underbrace{\text{rem}(1147, 899)}_{=248}) \\ &= \gcd(248, \underbrace{\text{rem}(899, 248)}_{=155}) \\ &= \gcd(155, \underbrace{\text{rem}(248, 155)}_{=93}) \\ &= \gcd(93, \underbrace{\text{rem}(155, 93)}_{=62}) \\ &= \gcd(62, \underbrace{\text{rem}(93, 62)}_{=31}) \\ &= \gcd(31, \underbrace{\text{rem}(62, 31)}_{=0}) \\ &= \gcd(31, 0) \\ &= 31 \end{aligned}$$

The last equation might look wrong, but 31 is a divisor of both 31 and 0 since every integer divides 0. This calculation that $\gcd(1147, 899) = 31$ was how we figured out that with water jugs of sizes 1147 and 899, Bruce dies trying to get 32 gallons.

Euclid’s algorithm can easily be formalized as a state machine. The set of states is $\mathbb{N}^2$ and there is one transition rule:

$$(x, y) \longrightarrow (y, \text{rem}(x, y)), \tag{8.3}$$

for $y > 0$. By Lemma 8.2.1, the gcd stays the same from one state to the next. That means the predicate

$$\gcd(x, y) = \gcd(a, b)$$

is a preserved invariant on the states (x, y) . This preserved invariant is, of course, true in the start state (a, b) . So by the Invariant Principle, if y ever becomes 0, the invariant will be true and so

$$x = \gcd(x, 0) = \gcd(a, b).$$

Namely, the value of x will be the desired $\gcd$.

What's more, x , and therefore also y , gets to be 0 pretty fast. To see why, note that after two transitions (8.3), the first coordinate of the state is $\text{rem}(x, y)$. But

$$\text{rem}(x, y) \leq x/2 \quad \text{for } 0 < y \leq x. \quad (8.4)$$

This is immediate if $y \leq x/2$ since the $\text{rem}(x, y) < y$ by definition. On the other hand, if $y > x/2$, then $\text{rem}(x, y) = x - y < x/2$. So x gets halved or smaller every two steps, which implies that after at most $2 \log a$ transitions, x will reach its minimum possible value, and at most one more transition will be possible. It follows that Euclid's algorithm terminates after at most $1 + 2 \log a$ transitions.³

But applying Euclid's algorithm to 26 and 21 gives

$$\gcd(26, 21) = \gcd(21, 5) = \gcd(5, 1) = 1,$$

which is why we left the 21- and 26-gallon jug problem unresolved. To resolve the matter, we will need more number theory.

8.2.2 The Pulverizer

We will get a lot of mileage out of the following key fact:

Theorem 8.2.2. *The greatest common divisor of a and b is a linear combination of a and b . That is,*

$$\gcd(a, b) = sa + tb,$$

for some integers s and t .

We already know from Lemma 8.1.3.2 that every linear combination of a and b is divisible by any common factor of a and b , so it is certainly divisible by the greatest of these common divisors. Since any constant multiple of a linear combination is also a linear combination, Theorem 8.2.2 implies that any multiple of the $\gcd$ is a linear combination. So we have the immediate corollary:

Corollary 8.2.3. *An integer is a linear combination of a and b iff it is a multiple of $\gcd(a, b)$.*

³A tighter analysis shows that at most $\log_\varphi(a)$ transitions are possible where φ is the golden ratio $(1 + \sqrt{5})/2$, see Problem 8.9.

We’ll prove Theorem 8.2.2 directly by explaining how to find s and t . This job is tackled by a mathematical tool that dates back to sixth-century India, where it was called *kuttak*, which means “The Pulverizer.” Today, the Pulverizer is more commonly known as “the extended Euclidean GCD algorithm,” because it is so close to Euclid’s Algorithm.

For example, following Euclid’s Algorithm, we can compute the GCD of 259 and 70 as follows:

$$\begin{aligned}
 \gcd(259, 70) &= \gcd(70, 49) && \text{since } \text{rem}(259, 70) = 49 \\
 &= \gcd(49, 21) && \text{since } \text{rem}(70, 49) = 21 \\
 &= \gcd(21, 7) && \text{since } \text{rem}(49, 21) = 7 \\
 &= \gcd(7, 0) && \text{since } \text{rem}(21, 7) = 0 \\
 &= 7.
 \end{aligned}$$

The Pulverizer goes through the same steps, but requires some extra bookkeeping along the way: as we compute $\gcd(a, b)$, we keep track of how to write each of the remainders (49, 21, and 7, in the example) as a linear combination of a and b . This is worthwhile, because our objective is to write the last nonzero remainder, which is the GCD, as such a linear combination. For our example, here is this extra bookkeeping:

x	y	$(\text{rem}(x, y))$	$= x - q \cdot y$
259	70	49	$= 259 - 3 \cdot 70$
70	49	21	$= 70 - 1 \cdot 49$
			$= 70 - 1 \cdot (259 - 3 \cdot 70)$
			$= -1 \cdot 259 + 4 \cdot 70$
49	21	7	$= 49 - 2 \cdot 21$
			$= (259 - 3 \cdot 70) - 2 \cdot (-1 \cdot 259 + 4 \cdot 70)$
			$= \boxed{3 \cdot 259 - 11 \cdot 70}$
21	7	0	

We began by initializing two variables, $x = a$ and $y = b$. In the first two columns above, we carried out Euclid’s algorithm. At each step, we computed $\text{rem}(x, y)$, which can be written in the form $x - q \cdot y$. (Remember that the Division Algorithm says $x = q \cdot y + r$, where r is the remainder. We get $r = x - q \cdot y$ by rearranging terms.) Then we replaced x and y in this equation with equivalent linear combinations of a and b , which we already had computed. After simplifying, we were left with a linear combination of a and b that was equal to the remainder as desired. The final solution is boxed.

This should make it pretty clear how and why the Pulverizer works. Anyone who has doubts can work out Problem 8.8, where the Pulverizer is formalized as a state

machine and then verified using an invariant that is an extension of the one used for Euclid’s algorithm.

Since the Pulverizer requires only a little more computation than Euclid’s algorithm, you can “pulverize” very large numbers very quickly by using this algorithm. As we will soon see, its speed makes the Pulverizer a very useful tool in the field of cryptography.

Now we can restate the Water Jugs Lemma 8.1.6 in terms of the greatest common divisor:

Corollary 8.2.4. *Suppose that we have water jugs with capacities a and b . Then the amount of water in each jug is always a multiple of $\gcd(a, b)$.*

For example, there is no way to form 4 gallons using 3- and 6-gallon jugs, because 4 is not a multiple of $\gcd(3, 6) = 3$.

8.2.3 One Solution for All Water Jug Problems

Corollary 8.2.3 says that 3 can be written as a linear combination of 21 and 26, since 3 is a multiple of $\gcd(21, 26) = 1$. So the Pulverizer will give us integers s and t such that

$$3 = s \cdot 21 + t \cdot 26 \quad (8.5)$$

Now the coefficient s could be either positive or negative. However, we can readily transform this linear combination into an equivalent linear combination

$$3 = s' \cdot 21 + t' \cdot 26 \quad (8.6)$$

where the coefficient s' is positive. The trick is to notice that if in equation (8.5) we increase s by 26 and decrease t by 21, then the value of the expression $s \cdot 21 + t \cdot 26$ is unchanged overall. Thus, by repeatedly increasing the value of s (by 26 at a time) and decreasing the value of t (by 21 at a time), we get a linear combination $s' \cdot 21 + t' \cdot 26 = 3$ where the coefficient s' is positive. Of course t' must then be negative; otherwise, this expression would be much greater than 3.

Now we can form 3 gallons using jugs with capacities 21 and 26: We simply repeat the following steps s' times:

1. Fill the 21-gallon jug.
2. Pour all the water in the 21-gallon jug into the 26-gallon jug. If at any time the 26-gallon jug becomes full, empty it out, and continue pouring the 21-gallon jug into the 26-gallon jug.

At the end of this process, we must have emptied the 26-gallon jug exactly $-t'$ times. Here's why: we've taken $s' \cdot 21$ gallons of water from the fountain, and we've poured out some multiple of 26 gallons. If we emptied fewer than $-t'$ times, then by (8.6), the big jug would be left with at least $3 + 26$ gallons, which is more than it can hold; if we emptied it more times, the big jug would be left containing at most $3 - 26$ gallons, which is nonsense. But once we have emptied the 26-gallon jug exactly $-t'$ times, equation (8.6) implies that there are exactly 3 gallons left.

Remarkably, we don't even need to know the coefficients s' and t' in order to use this strategy! Instead of repeating the outer loop s' times, we could just repeat *until we obtain 3 gallons*, since that must happen eventually. Of course, we have to keep track of the amounts in the two jugs so we know when we're done. Here's the solution that approach gives:

$(0, 0) \xrightarrow{\text{fill 21}}$							
$(21, 0)$	$\xrightarrow{\text{pour 21 into 26}}$	$(0, 21)$					
$\xrightarrow{\text{fill 21}}$	$(21, 21)$	$\xrightarrow{\text{pour 21 to 26}}$	$(16, 26)$	$\xrightarrow{\text{empty 26}}$	$(16, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 16)$
$\xrightarrow{\text{fill 21}}$	$(21, 16)$	$\xrightarrow{\text{pour 21 to 26}}$	$(11, 26)$	$\xrightarrow{\text{empty 26}}$	$(11, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 11)$
$\xrightarrow{\text{fill 21}}$	$(21, 11)$	$\xrightarrow{\text{pour 21 to 26}}$	$(6, 26)$	$\xrightarrow{\text{empty 26}}$	$(6, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 6)$
$\xrightarrow{\text{fill 21}}$	$(21, 6)$	$\xrightarrow{\text{pour 21 to 26}}$	$(1, 26)$	$\xrightarrow{\text{empty 26}}$	$(1, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 1)$
$\xrightarrow{\text{fill 21}}$	$(21, 1)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 22)$				
$\xrightarrow{\text{fill 21}}$	$(21, 22)$	$\xrightarrow{\text{pour 21 to 26}}$	$(17, 26)$	$\xrightarrow{\text{empty 26}}$	$(17, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 17)$
$\xrightarrow{\text{fill 21}}$	$(21, 17)$	$\xrightarrow{\text{pour 21 to 26}}$	$(12, 26)$	$\xrightarrow{\text{empty 26}}$	$(12, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 12)$
$\xrightarrow{\text{fill 21}}$	$(21, 12)$	$\xrightarrow{\text{pour 21 to 26}}$	$(7, 26)$	$\xrightarrow{\text{empty 26}}$	$(7, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 7)$
$\xrightarrow{\text{fill 21}}$	$(21, 7)$	$\xrightarrow{\text{pour 21 to 26}}$	$(2, 26)$	$\xrightarrow{\text{empty 26}}$	$(2, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 2)$
$\xrightarrow{\text{fill 21}}$	$(21, 2)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 23)$				
$\xrightarrow{\text{fill 21}}$	$(21, 23)$	$\xrightarrow{\text{pour 21 to 26}}$	$(18, 26)$	$\xrightarrow{\text{empty 26}}$	$(18, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 18)$
$\xrightarrow{\text{fill 21}}$	$(21, 18)$	$\xrightarrow{\text{pour 21 to 26}}$	$(13, 26)$	$\xrightarrow{\text{empty 26}}$	$(13, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 13)$
$\xrightarrow{\text{fill 21}}$	$(21, 13)$	$\xrightarrow{\text{pour 21 to 26}}$	$(8, 26)$	$\xrightarrow{\text{empty 26}}$	$(8, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 8)$
$\xrightarrow{\text{fill 21}}$	$(21, 8)$	$\xrightarrow{\text{pour 21 to 26}}$	$(3, 26)$	$\xrightarrow{\text{empty 26}}$	$(3, 0)$	$\xrightarrow{\text{pour 21 to 26}}$	$(0, 3)$

The same approach works regardless of the jug capacities and even regardless the amount we're trying to produce! Simply repeat these two steps until the desired amount of water is obtained:

1. Fill the smaller jug.

2. Pour all the water in the smaller jug into the larger jug. If at any time the larger jug becomes full, empty it out, and continue pouring the smaller jug into the larger jug.

By the same reasoning as before, this method eventually generates every multiple —up to the size of the larger jug —of the greatest common divisor of the jug capacities, namely, all the quantities we can possibly produce. No ingenuity is needed at all!

8.3 The Fundamental Theorem of Arithmetic

We now have almost enough tools to prove something that you probably already know, namely, that every number has a unique prime factorization.

Let’s state this more carefully. A sequence of numbers is *weakly decreasing* when each number in the sequence is $\geq$ the numbers after it. Note that a sequence of just one number as well as a sequence of no numbers —the empty sequence —is weakly decreasing by this definition.

Theorem 8.3.1. [*Fundamental Theorem of Arithmetic*] Every integer greater than 1 is a product of a unique weakly decreasing sequence of primes.

The Fundamental Theorem is also called the *Unique Factorization Theorem*, which is both a more descriptive and less pretentious name —but hey, we really want to get your attention to the importance and non-obviousness of unique factorization.

Notice that the theorem would be false if 1 were considered a prime; for example, 15 could be written as $5 \cdot 3$, or $5 \cdot 3 \cdot 1$, or $5 \cdot 3 \cdot 1 \cdot 1$, . . .

What’s more, it’s a mistake to take unique factorization for granted. It can actually fail for other integer-like sets of numbers, such as the complex numbers of the form $n + m\sqrt{-5}$ for $m, n \in \mathbb{Z}$ (see Problem 8.13).

There is a certain wonder in the Fundamental Theorem, even if you’ve known it since you were in a crib. Primes show up erratically in the sequence of integers. In fact, their distribution seems almost random:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, . . .

Basic questions about this sequence have stumped humanity for centuries. And yet we know that every nonnegative integer can be built up from primes in *exactly one* way. These quirky numbers are the building blocks for the integers.

The Fundamental Theorem is not hard to prove, but we’ll need a couple of preliminary facts.

The Prime Number Theorem

Let $\pi(x)$ denote the number of primes less than or equal to x . For example, $\pi(10) = 4$ because 2, 3, 5, and 7 are the primes less than or equal to 10. Primes are very irregularly distributed, so the growth of π is similarly erratic. However, the Prime Number Theorem gives an approximate answer:

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x / \ln x} = 1$$

Thus, primes gradually taper off. As a rule of thumb, about 1 integer out of every $\ln x$ in the vicinity of x is a prime.

The Prime Number Theorem was conjectured by Legendre in 1798 and proved a century later by de la Vallee Poussin and Hadamard in 1896. However, after his death, a notebook of Gauss was found to contain the same conjecture, which he apparently made in 1791 at age 15. (You sort of have to feel sorry for all the otherwise “great” mathematicians who had the misfortune of being contemporaries of Gauss.)

In late 2004 a billboard appeared in various locations around the country:

$$\left\{ \begin{array}{l} \text{first 10-digit prime found} \\ \text{in consecutive digits of } e \end{array} \right\} . \text{com}$$

Substituting the correct number for the expression in curly-braces produced the URL for a Google employment page. The idea was that Google was interested in hiring the sort of people that could and would solve such a problem.

How hard is this problem? Would you have to look through thousands or millions or billions of digits of e to find a 10-digit prime? The rule of thumb derived from the Prime Number Theorem says that among 10-digit numbers, about 1 in

$$\ln 10^{10} \approx 23$$

is prime. This suggests that the problem isn’t really so hard! Sure enough, the first 10-digit prime in consecutive digits of e appears quite early:

$e = 2.718281828459045235360287471352662497757247093699959574966$
 $96762772407663035354759457138217852516642\mathbf{7427466391}9320030$
 $599218174135966290435729003342952605956307381323286279434 \dots$

Lemma 8.3.2. *If p is a prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.*

Proof. One case is if $\gcd(a, p) = p$. Then the claim holds, because a is a multiple of p .

Otherwise, $\gcd(a, p) \neq p$. In this case $\gcd(a, p)$ must be 1, since 1 and p are the only positive divisors of p . Since $\gcd(a, p)$ is a linear combination of a and p , we have $1 = sa + tp$ for some s, t . Then $b = s(ab) + (tb)p$, that is, b is a linear combination of ab and p . Since p divides both ab and p , it also divides their linear combination b . ■

A routine induction argument extends this statement to:

Lemma 8.3.3. *Let p be a prime. If $p \mid a_1 a_2 \cdots a_n$, then p divides some a_i .*

Now we’re ready to prove the Fundamental Theorem of Arithmetic.

Proof. Theorem 2.3.1 showed, using the Well Ordering Principle, that every positive integer can be expressed as a product of primes. So we just have to prove this expression is unique. We will use Well Ordering to prove this too.

The proof is by contradiction: assume, contrary to the claim, that there exist positive integers that can be written as products of primes in more than one way. By the Well Ordering Principle, there is a smallest integer with this property. Call this integer n , and let

$$\begin{aligned} n &= p_1 \cdot p_2 \cdots p_j, \\ &= q_1 \cdot q_2 \cdots q_k, \end{aligned}$$

where both products are in weakly decreasing order and $p_1 \leq q_1$.

If $q_1 = p_1$, then n/q_1 would also be the product of different weakly decreasing sequences of primes, namely,

$$\begin{aligned} p_2 \cdots p_j, \\ q_2 \cdots q_k. \end{aligned}$$

Since $n/q_1 < n$, this can’t be true, so we conclude that $p_1 < q_1$.

Since the p_i ’s are weakly decreasing, all the p_i ’s are less than q_1 . But $q_1 \mid n = p_1 \cdot p_2 \cdots p_j$, so Lemma 8.3.3 implies that q_1 divides one of the p_i ’s, which contradicts the fact that q_1 is bigger than all them. ■



Figure 8.1 Alan Turing

8.4 Alan Turing

The man pictured in Figure 8.1 is Alan Turing, the most important figure in the history of computer science. For decades, his fascinating life story was shrouded by government secrecy, societal taboo, and even his own deceptions.

At age 24, Turing wrote a paper entitled *On Computable Numbers, with an Application to the Entscheidungsproblem*. The crux of the paper was an elegant way to model a computer in mathematical terms. This was a breakthrough, because it allowed the tools of mathematics to be brought to bear on questions of computation. For example, with his model in hand, Turing immediately proved that there exist problems that no computer can solve—no matter how ingenious the programmer. Turing’s paper is all the more remarkable because he wrote it in 1936, a full decade before any electronic computer actually existed.

The word “Entscheidungsproblem” in the title refers to one of the 28 mathematical problems posed by David Hilbert in 1900 as challenges to mathematicians of the 20th century. Turing knocked that one off in the same paper. And perhaps you’ve heard of the “Church-Turing thesis”? Same paper. So Turing was obviously a brilliant guy who generated lots of amazing ideas. But this lecture is about one of Turing’s less-amazing ideas. It involved codes. It involved number theory. And it was sort of stupid.

Let’s look back to the fall of 1937. Nazi Germany was rearming under Adolf Hitler, world-shattering war looked imminent, and—like us—Alan Turing was pondering the usefulness of number theory. He foresaw that preserving military secrets would be vital in the coming conflict and proposed a way *to encrypt communications using number theory*. This is an idea that has ricocheted up to our own time. Today, number theory is the basis for numerous public-key cryptosystems, digital signature schemes, cryptographic hash functions, and electronic payment systems. Furthermore, military funding agencies are among the biggest investors in cryptographic research. Sorry Hardy!

Soon after devising his code, Turing disappeared from public view, and half a century would pass before the world learned the full story of where he’d gone and what he did there. We’ll come back to Turing’s life in a little while; for now, let’s investigate the code Turing left behind. The details are uncertain, since he never formally published the idea, so we’ll consider a couple of possibilities.

8.4.1 Turing’s Code (Version 1.0)

The first challenge is to translate a text message into an integer so we can perform mathematical operations on it. This step is not intended to make a message harder to read, so the details are not too important. Here is one approach: replace each letter of the message with two digits ($A = 01$, $B = 02$, $C = 03$, etc.) and string all the digits together to form one huge number. For example, the message “victory” could be translated this way:

	v	i	c	t	o	r	y
→	22	09	03	20	15	18	25

Turing’s code requires the message to be a prime number, so we may need to pad the result with a few more digits to make a prime. In this case, appending the digits 13 gives the number 2209032015182513, which is prime.

Here is how the encryption process works. In the description below, m is the unencoded message (which we want to keep secret), m^* is the encrypted message (which the Nazis may intercept), and k is the key.

Beforehand The sender and receiver agree on a secret key, which is a large prime k .

Encryption The sender encrypts the message m by computing:

$$m^* = m \cdot k$$

Decryption The receiver decrypts m^* by computing:

$$\frac{m^*}{k} = m.$$

For example, suppose that the secret key is the prime number $k = 22801763489$ and the message m is “victory.” Then the encrypted message is:

$$\begin{aligned} m^* &= m \cdot k \\ &= 2209032015182513 \cdot 22801763489 \\ &= 50369825549820718594667857 \end{aligned}$$

There are a couple of questions that one might naturally ask about Turing’s code.

1. How can the sender and receiver ensure that m and k are prime numbers, as required?

The general problem of determining whether a large number is prime or composite has been studied for centuries, and reasonably good primality tests were known even in Turing’s time. In 2002, Manindra Agrawal, Neeraj Kayal, and Nitin Saxena announced a primality test that is guaranteed to work on a number n in about $(\log n)^{12}$ steps, that is, a number of steps bounded by a twelfth degree polynomial in the length (in bits) of the input, n . This definitively places primality testing way below the problems of exponential difficulty. Amazingly, the description of their breakthrough algorithm was only thirteen lines long!

Of course, a twelfth degree polynomial grows pretty fast, so the Agrawal, *et al.* procedure is of no practical use. Still, good ideas have a way of breeding more good ideas, so there’s certainly hope that further improvements will lead to a procedure that is useful in practice. But the truth is, there’s no practical need to improve it, since very efficient *probabilistic* procedures for prime-testing have been known since the early 1970’s. These procedures have some probability of giving a wrong answer, but their probability of being wrong is so tiny that relying on their answers is the best bet you’ll ever make.

2. Is Turing’s code secure?

The Nazis see only the encrypted message $m^* = m \cdot k$, so recovering the original message m requires factoring m^* . Despite immense efforts, no really efficient factoring algorithm has ever been found. It appears to be a fundamentally difficult problem, though a breakthrough someday is not impossible. In effect, Turing’s code puts to practical use his discovery that there are limits to the power of computation. Thus, provided m and k are sufficiently large, the Nazis seem to be out of luck!

This all sounds promising, but there is a major flaw in Turing’s code.

8.4.2 Breaking Turing’s Code

Let’s consider what happens when the sender transmits a *second* message using Turing’s code and the same key. This gives the Nazis two encrypted messages to look at:

$$m_1^* = m_1 \cdot k \quad \text{and} \quad m_2^* = m_2 \cdot k$$

The greatest common divisor of the two encrypted messages, m_1^* and m_2^* , is the secret key k . And, as we’ve seen, the GCD of two numbers can be computed very efficiently. So after the second message is sent, the Nazis can recover the secret key and read *every* message!

A mathematician as brilliant as Turing is not likely to have overlooked such a glaring problem, and we can guess that he had a slightly different system in mind, one based on *modular* arithmetic.

8.5 Modular Arithmetic

On the first page of his masterpiece on number theory, *Disquisitiones Arithmeticae*, Gauss introduced the notion of “congruence.” Now, Gauss is another guy who managed to cough up a half-decent idea every now and then, so let’s take a look at this one. Gauss said that a is *congruent* to b *modulo* n iff $n \mid (a - b)$. This is written

$$a \equiv b \pmod{n}.$$

For example:

$$29 \equiv 15 \pmod{7} \quad \text{because } 7 \mid (29 - 15).$$

There is a close connection between congruences and remainders:

Lemma 8.5.1 (Remainder).

$$a \equiv b \pmod{n} \quad \text{iff} \quad \text{rem}(a, n) = \text{rem}(b, n).$$

Proof. By the Division Theorem 8.1.5, there exist unique pairs of integers q_1, r_1 and q_2, r_2 such that:

$$\begin{aligned} a &= q_1 n + r_1 \\ b &= q_2 n + r_2, \end{aligned}$$

where $r_1, r_2 \in [0, n)$. Subtracting the second equation from the first gives:

$$a - b = (q_1 - q_2)n + (r_1 - r_2),$$

where $r_1 - r_2$ is in the interval $(-n, n)$. Now $a \equiv b \pmod{n}$ if and only if n divides the left side of this equation. This is true if and only if n divides the right side, which holds if and only if $r_1 - r_2$ is a multiple of n . Given the bounds on $r_1 - r_2$, this happens precisely when $r_1 = r_2$, that is, when $\text{rem}(a, n) = \text{rem}(b, n)$. ■

So we can also see that

$$29 \equiv 15 \pmod{7} \quad \text{because } \text{rem}(29, 7) = 1 = \text{rem}(15, 7).$$

Notice that even though “ $\pmod{7}$ ” appears on the end, the $\equiv$ symbol isn’t any more strongly associated with the 15 than with the 29. It would really be clearer to write $29 \equiv_7 15$ for example, but the notation with the modulus at the end is firmly entrenched and we’ll stick to it.

The Remainder Lemma 8.5.1 explains why the congruence relation has properties like an equality relation. In particular, the following properties follow immediately:

Lemma 8.5.2.

$$\begin{aligned} a &\equiv a \pmod{n} && \text{(reflexivity)} \\ a &\equiv b \text{ iff } b \equiv a \pmod{n} && \text{(symmetry)} \\ (a &\equiv b \text{ and } b \equiv c) \text{ implies } a \equiv c \pmod{n} && \text{(transitivity)} \end{aligned}$$

We’ll make frequent use of another immediate Corollary of the Remainder Lemma 8.5.1:

Corollary 8.5.3.

$$a \equiv \text{rem}(a, n) \pmod{n}$$

Still another way to think about congruence modulo n is that it *defines a partition of the integers into n sets so that congruent numbers are all in the same set*. For example, suppose that we’re working modulo 3. Then we can partition the integers into 3 sets as follows:

$$\begin{aligned} &\{ \dots, -6, -3, 0, 3, 6, 9, \dots \} \\ &\{ \dots, -5, -2, 1, 4, 7, 10, \dots \} \\ &\{ \dots, -4, -1, 2, 5, 8, 11, \dots \} \end{aligned}$$

according to whether their remainders on division by 3 are 0, 1, or 2. The upshot is that when arithmetic is done modulo n there are really only n different kinds of numbers to worry about, because there are only n possible remainders. In this sense, modular arithmetic is a simplification of ordinary arithmetic.

The next most useful fact about congruences is that they are *preserved* by addition and multiplication:

Lemma 8.5.4. *For $n \geq 1$, if $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, then*

1. $a + c \equiv b + d \pmod{n}$,
2. $ac \equiv bd \pmod{n}$.

Proof. We have that n divides $(b - a)$ which is equal to $(b + c) - (a + c)$, so

$$a + c \equiv b + c \pmod{n}.$$

Also, n divides $(d - c)$, so by the same reasoning

$$b + c \equiv b + d \pmod{n}.$$

Combining these according to Lemma 8.5.2, we get

$$a + c \equiv b + d \pmod{n}.$$

The proof for multiplication is virtually identical, using the fact that if n divides $(b - a)$, then it obviously divides $(bc - ac)$ as well. ■

The overall theme is that *congruences work a lot like arithmetic equations*, though there are a couple of exceptions we’re about to examine.

8.5.1 Turing’s Code (Version 2.0)

In 1940, France had fallen before Hitler’s army, and Britain stood alone against the Nazis in western Europe. British resistance depended on a steady flow of supplies brought across the north Atlantic from the United States by convoys of ships. These convoys were engaged in a cat-and-mouse game with German “U-boats”—submarines—which prowled the Atlantic, trying to sink supply ships and starve Britain into submission. The outcome of this struggle pivoted on a balance of information: could the Germans locate convoys better than the Allies could locate U-boats or vice versa?

Germany lost.

But a critical reason behind Germany’s loss was made public only in 1974: Germany’s naval code, *Enigma*, had been broken by the [Polish Cipher Bureau](http://en.wikipedia.org/wiki/Polish_Cipher_Bureau)⁴ and the secret had been turned over to the British a few weeks before the Nazi invasion of Poland in 1939. Throughout much of the war, the Allies were able to route convoys around German submarines by listening in to German communications. The British government didn’t explain *how* Enigma was broken until 1996. When it was finally released (by the US), the story revealed that Alan Turing had joined the

⁴See http://en.wikipedia.org/wiki/Polish_Cipher_Bureau.

secret British codebreaking effort at Bletchley Park in 1939, where he became the lead developer of methods for rapid, bulk decryption of German Enigma messages. Turing’s Enigma deciphering was an invaluable contribution to the Allied victory over Hitler.

Governments are always tight-lipped about cryptography, but the half-century of official silence about Turing’s role in breaking Enigma and saving Britain may be related to some disturbing events after the war. More on that later. Let’s get back to number theory and consider an alternative interpretation of Turing’s code. Perhaps we had the basic idea right (multiply the message by the key), but erred in using *conventional* arithmetic instead of *modular* arithmetic. Maybe this is what Turing meant:

Beforehand The sender and receiver agree on a large prime p , which may be made public. (This will be the modulus for all our arithmetic.) They also agree on a secret key $k \in [1, p)$.

Encryption The message m can be any integer in $[0, p)$; in particular, the message is no longer required to be a prime. The sender encrypts the message m to produce m^* by computing:

$$m^* = \text{rem}(mk, p) \quad (8.7)$$

Decryption (Uh-oh.)

The decryption step is a problem. We might hope to decrypt in the same way as before: by dividing the encrypted message m^* by the key k . The difficulty is that m^* is the *remainder* when mk is divided by p . So dividing m^* by k might not even give us an integer!

This decoding difficulty can be overcome with a better understanding of arithmetic modulo a prime.

8.6 Arithmetic with a Prime Modulus

8.6.1 Multiplicative Inverses

The *multiplicative inverse* of a number x is another number x^{-1} such that:

$$x \cdot x^{-1} = 1$$

Generally, multiplicative inverses exist over the real numbers. For example, the multiplicative inverse of 3 is $1/3$ since:

$$3 \cdot \frac{1}{3} = 1$$

The sole exception is that 0 does not have an inverse. On the other hand, over the integers, only 1 and -1 have inverses.

Surprisingly, when we’re working *modulo a prime number*, every number that is not congruent to 0 has a multiplicative inverse. For example, if we’re working modulo 5, then 3 is a multiplicative inverse of 7, since:

$$7 \cdot 3 \equiv 1 \pmod{5}$$

(All numbers congruent to 3 modulo 5 are also multiplicative inverses of 7; for example, $7 \cdot 8 \equiv 1 \pmod{5}$ as well.) The only exception is that numbers congruent to 0 modulo 5 (that is, the multiples of 5) do not have inverses, much as 0 does not have an inverse over the real numbers. Let’s prove this.

Lemma 8.6.1. *If p is prime and k is not a multiple of p , then k has a multiplicative inverse modulo p .*

Proof. Since p is prime, it has only two divisors: 1 and p . And since k is not a multiple of p , we must have $\gcd(p, k) = 1$. Therefore, there is a linear combination of p and k equal to 1:

$$sp + tk = 1,$$

and therefore

$$sp + tk \equiv 1 \pmod{p}.$$

But $p \equiv 0 \pmod{p}$, so

$$tk \equiv 0 + tk \equiv sp + tk \equiv 1 \pmod{p}.$$

Thus, t is a multiplicative inverse of k . ■

Multiplicative inverses are the key to decryption in Turing’s code. Specifically, we can recover the original message by multiplying the encoded message by the *inverse* of the key:

$$\begin{aligned} m^* \cdot k^{-1} &= \text{rem}(mk, p) \cdot k^{-1} && \text{(the def. (8.7) of } m^*) \\ &\equiv (mk)k^{-1} \pmod{p} && \text{(by Cor. 8.5.3)} \\ &\equiv m \pmod{p}. \end{aligned}$$

This shows that $m \cdot k^{-1}$ is congruent to the original message m . Since m was in $[0, p)$, we can recover it exactly by taking a remainder:

$$m = \text{rem}(m \cdot k^{-1}, p).$$

So all we need to decrypt the message is to find a value of k^{-1} . From the proof of Lemma 8.6.1, we know that t is such a value, where $sp + tk = 1$. Finding t is easy using the Pulverizer.

8.6.2 Cancellation

Another sense in which real numbers are nice is that one can cancel multiplicative terms. In other words, if we know that $m_1k = m_2k$, then we can cancel the k 's and conclude that $m_1 = m_2$, provided $k \neq 0$. In general, cancellation is *not* valid in modular arithmetic. For example,

$$2 \cdot 3 \equiv 4 \cdot 3 \pmod{6},$$

but canceling the 3's leads to the *false* conclusion that $2 \equiv 4 \pmod{6}$. The fact that multiplicative terms cannot be canceled is the most significant sense in which congruences differ from ordinary equations. However, this difference goes away if we're working modulo a *prime*; then cancellation is valid.

Lemma 8.6.2. *Suppose p is a prime and k is not a multiple of p . Then*

$$ak \equiv bk \pmod{p} \quad \text{IMPLIES} \quad a \equiv b \pmod{p}.$$

Proof. Multiply both sides of the congruence by k^{-1} . ■

We can use this lemma to get a bit more insight into how Turing's code works. In particular, the encryption operation in Turing's code *permutes the set of possible messages*. This is stated more precisely in the following corollary.

Corollary 8.6.3. *Suppose p is a prime and k is not a multiple of p . Then the sequence of remainders on division by p of the sequence:*

$$1 \cdot k, \quad 2 \cdot k, \quad \dots, \quad (p-1) \cdot k$$

is a permutation⁵ of the sequence:

$$1, \quad 2, \quad \dots, \quad (p-1).$$

⁵A *permutation* of a sequence of elements is a reordering of the elements.

Proof. The sequence of remainders contains $p - 1$ numbers. Since $i \cdot k$ is not divisible by p for $i = 1, \dots, p-1$, all these remainders are in $[1, p)$ by the definition of remainder. Furthermore, the remainders are all different: no two numbers in $[1, p)$ are congruent modulo p , and by Lemma 8.6.2, $i \cdot k \equiv j \cdot k \pmod{p}$ if and only if $i \equiv j \pmod{p}$. Thus, the sequence of remainders must contain *all* of $[1, p)$ in some order. ■

For example, suppose $p = 5$ and $k = 3$. Then the sequence:

$$\underbrace{\text{rem}((1 \cdot 3), 5)}_{=3}, \quad \underbrace{\text{rem}((2 \cdot 3), 5)}_{=1}, \quad \underbrace{\text{rem}((3 \cdot 3), 5)}_{=4}, \quad \underbrace{\text{rem}((4 \cdot 3), 5)}_{=2}$$

is a permutation of 1, 2, 3, 4. As long as the Nazis don't know the secret key k , they don't know how the set of possible messages are permuted by the process of encryption and thus they can't read encoded messages.

8.6.3 Fermat's Little Theorem

An alternative approach to finding the inverse of the secret key k in Turing's code is to rely on Fermat's Little Theorem, which is much easier than his famous Last Theorem.

Theorem 8.6.4 (Fermat's Little Theorem). *Suppose p is a prime and k is not a multiple of p . Then:*

$$k^{p-1} \equiv 1 \pmod{p}$$

Proof. We reason as follows:

$$\begin{aligned} (p-1)! &::= 1 \cdot 2 \cdots (p-1) \\ &= \text{rem}(k, p) \cdot \text{rem}(2k, p) \cdots \text{rem}((p-1)k, p) && \text{(by Cor 8.6.3)} \\ &\equiv k \cdot 2k \cdots (p-1)k \pmod{p} && \text{(by Cor 8.5.3)} \\ &\equiv (p-1)! \cdot k^{p-1} \pmod{p} && \text{(rearranging terms)} \end{aligned}$$

Now $(p-1)!$ is not a multiple of p because the prime factorizations of $1, 2, \dots, (p-1)$ contain only primes smaller than p . So by Lemma 8.6.2, we can cancel $(p-1)!$ from the first and last expressions, which proves the claim. ■

Here is how we can find inverses using Fermat's Theorem. Suppose p is a prime and k is not a multiple of p . Then, by Fermat's Theorem, we know that:

$$k^{p-2} \cdot k \equiv 1 \pmod{p}$$

Therefore, k^{p-2} must be a multiplicative inverse of k . For example, suppose that we want the multiplicative inverse of 6 modulo 17. Then we need to compute $\text{rem}(6^{15}, 17)$, which we can do using the fast exponentiation procedure of Section 6.4.5, with all the arithmetic done modulo 17. Namely,

$$\begin{aligned} (6, 1, 15) &\longrightarrow (36, 6, 7) \equiv (2, 6, 7) \longrightarrow (4, 12, 3) \\ &\longrightarrow (16, 48, 1) \equiv (16, 14, 1) \longrightarrow (256, 224, 0) \equiv (1, 3, 0). \end{aligned}$$

where the $\equiv$'s are modulo 17. Therefore, $6^{15} \equiv 3 \pmod{17}$. Sure enough, 3 is the multiplicative inverse of 6 modulo 17 since

$$3 \cdot 6 = 18 \equiv 1 \pmod{17}.$$

In general, if we were working modulo a prime p , finding a multiplicative inverse by trying every value in $[1, p)$ would require about p operations. However, this approach, like the Pulverizer, requires only about $\log p$ transition, which is far better when p is large.

8.6.4 Breaking Turing's Code—Again

The Germans didn't bother to encrypt their weather reports with the highly-secure Enigma system. After all, so what if the Allies learned that there was rain off the south coast of Iceland? But, amazingly, this practice provided the British with a critical edge in the Atlantic naval battle during 1941.

The problem was that some of those weather reports had originally been transmitted using Enigma from U-boats out in the Atlantic. Thus, the British obtained both unencrypted reports and the same reports encrypted with Enigma. By comparing the two, the British were able to determine which key the Germans were using that day and could read all other Enigma-encoded traffic. Today, this would be called a *known-plaintext attack*.

Let's see how a known-plaintext attack would work against Turing's code. Suppose that the Nazis know both m and m^* where:

$$m^* \equiv mk \pmod{p}$$

Now they can compute:

$$\begin{aligned} m^{p-2} \cdot m^* &= m^{p-2} \cdot \text{rem}(mk, p) && \text{(def. (8.7) of } m^*) \\ &\equiv m^{p-2} \cdot mk \pmod{p} && \text{(by Cor 8.5.3)} \\ &\equiv m^{p-1} \cdot k \pmod{p} \\ &\equiv k \pmod{p} && \text{(Fermat's Theorem)} \end{aligned}$$

Now the Nazis have the secret key k and can decrypt any message!

This is a huge vulnerability, so Turing’s code has no practical value. Fortunately, Turing got better at cryptography after devising this code; his subsequent deciphering of Enigma messages surely saved thousands of lives, if not the whole of Britain.

8.6.5 Turing Postscript

A few years after the war, Turing’s home was robbed. Detectives soon determined that a former homosexual lover of Turing’s had conspired in the robbery. So they arrested him—that is, they arrested Alan Turing—because homosexuality was a British crime punishable by up to two years in prison at that time. Turing was sentenced to a hormonal “treatment” for his homosexuality: he was given estrogen injections. He began to develop breasts.

Three years later, Alan Turing, the founder of computer science, was dead. His mother explained what happened in a biography of her own son. Despite her repeated warnings, Turing carried out chemistry experiments in his own home. Apparently, her worst fear was realized: by working with potassium cyanide while eating an apple, he poisoned himself.

However, Turing remained a puzzle to the very end. His mother was a devoutly religious woman who considered suicide a sin. And, other biographers have pointed out, Turing had previously discussed committing suicide by eating a poisoned apple. Evidently, Alan Turing, who founded computer science and saved his country, took his own life in the end, and in just such a way that his mother could believe it was an accident.

Turing’s last project before he disappeared from public view in 1939 involved the construction of an elaborate mechanical device to test a mathematical conjecture called the Riemann Hypothesis. This conjecture first appeared in a sketchy paper by Bernhard Riemann in 1859 and is now one of the most famous unsolved problems in mathematics.

8.7 Arithmetic with an Arbitrary Modulus

Turing’s code did not work as he hoped. However, his essential idea—using number theory as the basis for cryptography—succeeded spectacularly in the decades after his death.

In 1977, Ronald Rivest, Adi Shamir, and Leonard Adleman at MIT proposed a highly secure cryptosystem (called **RSA**) based on number theory. Despite decades of attack, no significant weakness has been found. Moreover, RSA has a major ad-

The Riemann Hypothesis

The formula for the sum of an infinite geometric series says:

$$1 + x + x^2 + x^3 + \cdots = \frac{1}{1 - x}$$

Substituting $x = \frac{1}{2^s}$, $x = \frac{1}{3^s}$, $x = \frac{1}{5^s}$, and so on for each prime number gives a sequence of equations:

$$\begin{aligned} 1 + \frac{1}{2^s} + \frac{1}{2^{2s}} + \frac{1}{2^{3s}} + \cdots &= \frac{1}{1 - 1/2^s} \\ 1 + \frac{1}{3^s} + \frac{1}{3^{2s}} + \frac{1}{3^{3s}} + \cdots &= \frac{1}{1 - 1/3^s} \\ 1 + \frac{1}{5^s} + \frac{1}{5^{2s}} + \frac{1}{5^{3s}} + \cdots &= \frac{1}{1 - 1/5^s} \\ &\text{etc.} \end{aligned}$$

Multiplying together all the left sides and all the right sides gives:

$$\sum_{n=1}^{\infty} \frac{1}{n^s} = \prod_{p \in \text{primes}} \left(\frac{1}{1 - 1/p^s} \right)$$

The sum on the left is obtained by multiplying out all the infinite series and applying the Fundamental Theorem of Arithmetic. For example, the term $1/300^s$ in the sum is obtained by multiplying $1/2^{2s}$ from the first equation by $1/3^s$ in the second and $1/5^{2s}$ in the third. Riemann noted that every prime appears in the expression on the right. So he proposed to learn about the primes by studying the equivalent, but simpler expression on the left. In particular, he regarded s as a complex number and the left side as a function, $\zeta(s)$. Riemann found that the distribution of primes is related to values of s for which $\zeta(s) = 0$, which led to his famous conjecture:

Definition 8.6.5. The *Riemann Hypothesis*: Every nontrivial zero of the zeta function $\zeta(s)$ lies on the line $s = 1/2 + ci$ in the complex plane.

A proof would immediately imply, among other things, a strong form of the Prime Number Theorem.

Researchers continue to work intensely to settle this conjecture, as they have for over a century. It is another of the [Millennium Problems](#) whose solver will earn \$1,000,000 from the Clay Institute.

vantage over traditional codes: the sender and receiver of an encrypted message need not meet beforehand to agree on a secret key. Rather, the receiver has both a *private key*, which they guard closely, and a *public key*, which they distribute as widely as possible. A sender wishing to transmit a secret message to the receiver encrypts their message using the receiver’s widely-distributed public key. The receiver can then decrypt the received message using their closely-held private key. The use of such a *public key cryptography* system allows you and Amazon, for example, to engage in a secure transaction without meeting up beforehand in a dark alley to exchange a key.

Interestingly, RSA does not operate modulo a prime, as Turing’s scheme may have, but rather modulo the product of *two* large primes. Thus, we’ll need to know a bit about how arithmetic works modulo a composite number in order to understand RSA. Arithmetic modulo an arbitrary positive integer is really only a little more painful than working modulo a prime—though you may think this is like the doctor saying, “This is only going to hurt a little,” before he jams a big needle in your arm.

8.7.1 Relative Primality

Integers that have no prime factor in common are called *relatively prime*. This is the same as having no common divisor (prime or not) greater than 1. It is also equivalent to saying $\gcd(a, b) = 1$.

For example, 8 and 15 are relatively prime, since $\gcd(8, 15) = 1$. Note that, except for multiples of p , every integer is relatively prime to a prime number p .

Next we’ll need to generalize what we know about arithmetic modulo a prime to work modulo an arbitrary positive integer n . The basic theme is that arithmetic modulo n may be complicated, but the integers *relatively prime* to n keep the nice properties of having inverses and being cancellable. For example,

Lemma 8.7.1. *Let n be a positive integer. If k is relatively prime to n , then there exists an integer k^{-1} such that:*

$$k \cdot k^{-1} \equiv 1 \pmod{n}.$$

An inverse for any k relatively prime to n is simply the coefficient of k in the linear combination of k and n that equals 1, exactly as in the proof of Lemma 8.6.1.

As a consequence of this lemma, we can cancel a multiplicative term from both sides of a congruence if that term is relatively prime to the modulus:

Corollary 8.7.2. *Suppose n is a positive integer and k is relatively prime to n . Then*

$$ak \equiv bk \pmod{n} \text{ implies } a \equiv b \pmod{n}.$$

This holds because we can multiply both sides of the first congruence by k^{-1} and simplify to obtain the second.

The following lemma is a simple generalization of Corollary 8.6.3 with much the same proof.

Lemma 8.7.3. *Suppose n is a positive integer and k is relatively prime to n . Let $k_1, \dots, k_r$ be all the integers in the interval $[1, n)$ that are relatively prime to n . Then the sequence of remainders on division by n of:*

$$k_1 \cdot k, \quad k_2 \cdot k, \quad k_3 \cdot k, \dots, \quad k_r \cdot k$$

is a permutation of the sequence:

$$k_1, \quad k_2, \dots, \quad k_r.$$

Proof. We will show that the remainders in the first sequence are all distinct and are equal to some member of the sequence of k_j 's. Since the two sequences have the same length, the first must be a permutation of the second.

First, we show that the remainders in the first sequence are all distinct. Suppose that $\text{rem}(k_i k, n) = \text{rem}(k_j k, n)$. This is equivalent to $k_i k \equiv k_j k \pmod{n}$, which implies $k_i \equiv k_j \pmod{n}$ by Corollary 8.7.2. This, in turn, means that $k_i = k_j$ since both are in $[1, n)$. Thus, none of the remainder terms in the first sequence is equal to any other remainder term.

Next, we show that each remainder in the first sequence equals one of the k_i . By assumption, k_i and k are relatively prime to n , and therefore so is $k_i k$ by Unique Factorization. Hence,

$$\begin{aligned} \gcd(n, \text{rem}(k_i k, n)) &= \gcd(k_i k, n) && \text{(Lemma 8.2.1)} \\ &= 1. \end{aligned}$$

Since $\text{rem}(k_i k, n)$ is in $[0, n)$ by the definition of remainder, and since it is relatively prime to n , it must be equal to one of the k_i 's. ■

8.7.2 Euler's Theorem

RSA relies heavily on a generalization of Fermat's Theorem known as Euler's Theorem. For both theorems, the exponent of k needed to produce an inverse of k modulo n depends on the number, $\phi(n)$, of integers in $[0, n)$, that are relatively prime to n . This function ϕ is known as Euler's ϕ or *totient function*. For example, $\phi(7) = 6$ since 1, 2, 3, 4, 5, and 6 are all relatively prime to 7. Similarly, $\phi(12) = 4$ since 1, 5, 7, and 11 are the only numbers in $[0, 12)$ that are relatively prime to 12.

If n is prime, then $\phi(n) = n - 1$ since every positive number less than a prime number is relatively prime to that prime. When n is composite, however, the ϕ function gets a little complicated. We’ll get back to it in the next section.

We can now prove Euler’s Theorem:

Theorem 8.7.4 (Euler’s Theorem). *Suppose n is a positive integer and k is relatively prime to n . Then*

$$k^{\phi(n)} \equiv 1 \pmod{n}$$

Proof. Let $k_1, \dots, k_r$ denote all integers relatively prime to n where $k_i \in [0, n)$. Then $r = \phi(n)$, by the definition of the function ϕ . The remainder of the proof mirrors the proof of Fermat’s Theorem. In particular,

$$\begin{aligned} k_1 \cdot k_2 \cdots k_r &= \text{rem}(k_1 \cdot k, n) \cdot \text{rem}(k_2 \cdot k, n) \cdots \text{rem}(k_r \cdot k, n) && \text{(by Lemma 8.7.3)} \\ &\equiv (k_1 \cdot k) \cdot (k_2 \cdot k) \cdots (k_r \cdot k) \pmod{n} && \text{(by Cor 8.5.3)} \\ &\equiv (k_1 \cdot k_2 \cdots k_r) \cdot k^r \pmod{n} && \text{(rearranging terms)} \end{aligned}$$

By Lemma 8.7.2, each of the terms k_i can be cancelled, proving the claim. ■

We can find multiplicative inverses using Euler’s theorem as we did with Fermat’s theorem: if k is relatively prime to n , then $k^{\phi(n)-1}$ is a multiplicative inverse of k modulo n . However, this approach requires computing $\phi(n)$. In the next section, we’ll show that computing $\phi(n)$ is easy *if* we know the prime factorization of n . Unfortunately, finding the factors of n can be hard to do when n is large, and so the Pulverizer is generally the best approach to computing inverses modulo n .

8.7.3 Computing Euler’s ϕ Function

RSA works using arithmetic modulo the product of two large primes, so we begin with an elementary explanation of how to compute $\phi(pq)$ for primes p and q :

Lemma 8.7.5.

$$\phi(pq) = (p - 1)(q - 1)$$

for primes $p \neq q$.

Proof. Since p and q are prime, any number that is not relatively prime to pq must be a multiple of p or a multiple of q . Among the pq numbers in $[0, pq)$, there are precisely q multiples of p and p multiples of q . Since p and q are relatively prime,

the only number in $[0, pq)$ that is a multiple of both p and q is 0. Hence, there are $p + q - 1$ numbers in $[0, pq)$ that are *not* relatively prime to n . This means that

$$\begin{aligned}\phi(pq) &= pq - (p + q - 1) \\ &= (p - 1)(q - 1),\end{aligned}$$

as claimed.⁶ ■

The following theorem provides a way to calculate $\phi(n)$ for arbitrary n .

Theorem 8.7.6.

- (a) If p is a prime, then $\phi(p^k) = p^k - p^{k-1}$ for $k \geq 1$.
- (b) If a and b are relatively prime, then $\phi(ab) = \phi(a)\phi(b)$.

Here's an example of using Theorem 8.7.6 to compute $\phi(300)$:

$$\begin{aligned}\phi(300) &= \phi(2^2 \cdot 3 \cdot 5^2) \\ &= \phi(2^2) \cdot \phi(3) \cdot \phi(5^2) && \text{(by Theorem 8.7.6.(b))} \\ &= (2^2 - 2^1)(3^1 - 3^0)(5^2 - 5^1) && \text{(by Theorem 8.7.6.(a))} \\ &= 80.\end{aligned}$$

To prove Theorem 8.7.6.(a), notice that every p th number among the p^k numbers in $[0, p^k)$ is divisible by p , and only these are divisible by p . So $1/p$ of these numbers are divisible by p and the remaining ones are not. That is,

$$\phi(p^k) = p^k - (1/p)p^k = p^k - p^{k-1}.$$

We'll leave a proof of Theorem 8.7.6.(b) to Problem 8.30.

As a consequence of Theorem 8.7.6, we have

Corollary 8.7.7. *For any number n , if $p_1, p_2, \dots, p_j$ are the (distinct) prime factors of n , then*

$$\phi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_j}\right).$$

We'll give another proof of Corollary 8.7.7 in a few weeks based on rules for counting.

⁶This proof previews a kind of counting argument that we will explore more fully in Part III.

8.8 The RSA Algorithm

We are finally ready to see how the *RSA public key encryption scheme* works. The purpose of the RSA scheme is to transmit secret messages over public communication channels. The messages transmitted will actually be nonnegative integers of some fixed size—typically of hundreds of digits.

The details are in the box on the next page.

The RSA Cryptosystem

A **Receiver** who wants to be able to receive secret numerical messages creates a *private key*, which they keep secret, and a *public key* which they make publicly available. Anyone with the public key can then be a **Sender** who can publicly send secret messages to the **Receiver**—even if they have never communicated or shared any information besides the public key.

Here is how they do it:

Beforehand The **Receiver** creates a public key and a private key as follows.

1. Generate two distinct primes, p and q . These are used to generate the private key, and they must be kept hidden. (In current practice, p and q are chosen to be hundreds of digits long.)
2. Let $n ::= pq$.
3. Select an integer $e \in [1, n)$ such that $\gcd(e, (p-1)(q-1)) = 1$.
The *public key* is the pair (e, n) . This should be distributed widely.
4. Compute $d \in [1, n)$ such that $de \equiv 1 \pmod{(p-1)(q-1)}$. This can be done using the Pulverizer.
The *private key* is the pair (d, n) . This should be kept hidden!

Encoding To transmit a message $m \in [0, n)$ to **Receiver**, a **Sender** uses the public key to encrypt m into a numerical message

$$m^* ::= \text{rem}(m^e, n).$$

The **Sender** can then publicly transmit m^* to the **Receiver**.

Decoding The **Receiver** decrypts message m^* back to message m using the private key:

$$m = \text{rem}((m^*)^d, n).$$

If the message m is relatively prime to n , it is an almost immediate consequence of Euler’s Theorem that this way of decoding the encrypted message indeed reproduces the original unencrypted message. In fact, the decoding always works—even in (the highly unlikely) case that m is not relatively prime to n . The details are worked out in Problem 8.38.

Why is RSA thought to be secure? It would be easy to figure out the private key (d, n) if you knew p and q —you could do it the same way the **Receiver** does using the Pulverizer. Now no one knows for sure, but the mathematical, financial, and intelligence communities are convinced that if n is a very large number (say, with a thousand digits), then it would be hopelessly hard to factor n and find p and q , so that approach is not going to break RSA.

Could there be another approach to reverse engineering the private key from the public key that did not involve factoring n ? Not really. It turns out that given just the private and the public keys, it is easy to factor n (a proof of this is sketched in Problem 8.40). So if we are confident that factoring is hopelessly hard, then we can be equally confident that finding the private key just from the public key will be hopeless.⁷

You can hope that with more studying of number theory, you will be the first to figure out how to do factoring quickly. We should warn you that Gauss worked on it for years without a lot to show for his efforts. And if you do figure it out, you might wind up meeting some serious-looking fellows who work for a Federal agency....

8.9 What has SAT got to do with it?

So why does the world, or at least the world’s secret codes, fall apart if there is an efficient test for satisfiability? To explain this, remember that RSA can be managed computationally because multiplication of two primes is fast, but factoring a product of two primes seems to be overwhelmingly demanding.

Now designing digital multiplication circuits is completely routine. This means we can easily build a digital circuit out of AND, OR, and NOT gates that can take two input strings u, v of length n , and a third input string, z , of length $2n$, and “check” if z represents the product of the numbers represented by u and v . That is, it gives

⁷The possibility of decoding RSA messages *without finding the private key or factoring* has not been ruled out. It is an important unproven conjecture in cyptography that the ability to crack RSA would imply the ability to factor. This would be a much stronger theoretical assurance of RSA security than is presently known. But the real reason for confidence in RSA is that it has stood up against all the attacks by the world’s most sophisticated cryptographers for over 30 years.

output 1 if z represents the product of u and v , and gives output 0 otherwise.

Now here’s how to factor any number with a length $2n$ representation using a SAT solver. Fix the z input to be the representation of the number to be factored. Set the first digit of the u input to 1, and do a SAT test to see if there is a satisfying assignment of values for the remaining bits of u and v . That is, see if the remaining bits of u and v can be filled in to cause the circuit to give output 1. If there is such an assignment, fix the first bit of u to 1, otherwise fix the first bit of u to be 0. Now do the same thing to fix the second bit of u and then third, proceeding in this way through all the bits of u and then of v . The result is that after $2n$ SAT tests, we have found an assignment of values for u and v that makes the circuit give output 1. So u and v represent factors of the number represented by z . This means that if SAT could be done in time bounded by a degree d polynomial in n , then $2n$ digit numbers can be factored in time bounded by a polynomial in n of degree $d + 1$. In sum, if SAT was easy, then so is factoring, and so RSA would be easy to break.

Problems for Section 8.1

Practice Problems

Problem 8.1.

Prove that a linear combination of linear combinations of integers $a_0, \dots, a_n$ is a linear combination of $a_0, \dots, a_n$.

Problem 8.2. (a) Find integer coefficients, x, y , such that $25x + 32y = \text{GCD}(25, 32)$.

(b) What is the inverse (mod 25) of 32?

Class Problems

Problem 8.3.

A number is *perfect* if it is equal to the sum of its positive divisors, other than itself. For example, 6 is perfect, because $6 = 1 + 2 + 3$. Similarly, 28 is perfect, because $28 = 1 + 2 + 4 + 7 + 14$. Explain why $2^{k-1}(2^k - 1)$ is perfect when $2^k - 1$ is prime.⁸

⁸Euclid proved this 2300 years ago. About 250 years ago, Euler proved the converse: every even perfect number is of this form (for a simple proof see <http://primes.utm.edu/notes/proofs/EvenPerfect.html>). As is typical in number theory, apparently simple results lie at the brink of the unknown. For example, it is not known if there are an infinite number of even perfect numbers or any odd perfect numbers at all.

Problems for Section 8.2

Practice Problems

Problem 8.4.

Consider the two integers:

$$x = 21212121,$$

$$y = 12121212.$$

- (a) What is the GCD of x and y ? *Hint:* Looks scary, but it's not.
- (b) How many iterations of the Euclidean algorithm are needed to compute this GCD?

(An iteration of the Euclidean algorithm is defined as an application of the equation

$$\text{GCD}(a, b) = \text{GCD}(b, \text{rem}(a, b)).$$

The algorithm begins with $\text{GCD}(x, y)$ and ends with $\text{GCD}(d, 0)$ for some d .)

Problem 8.5.

Consider the following two numbers:

$$x = 17^{88} * 31^5 * 37^2 * 59^{1000}$$

$$y = 19^{(9^{22})} * 37^{12} * 53^{3678} * 59^{29}.$$

- (a) Compute their GCD.
- (b) Compute their LCM (least common multiple: $\text{lcm}(a, b) :=$ the smallest number that is a multiple of both a and b).

Class Problems

Problem 8.6. (a) Use the Pulverizer to find integers x, y such that

$$x \cdot 50 + y \cdot 21 = \text{gcd}(50, 21).$$

- (b) Now find integers x', y' with $y' > 0$ such that

$$x' \cdot 50 + y' \cdot 21 = \text{gcd}(50, 21)$$

Problem 8.7.

For nonzero integers, a, b , prove the following properties of divisibility and GCD’S. (You may use the fact that $\gcd(a, b)$ is an integer linear combination of a and b . You may *not* appeal to uniqueness of prime factorization because the properties below are needed to *prove* unique factorization.)

- (a) Every common divisor of a and b divides $\gcd(a, b)$.
- (b) If $a \mid bc$ and $\gcd(a, b) = 1$, then $a \mid c$.
- (c) If $p \mid ab$ for some prime, p , then $p \mid a$ or $p \mid b$.
- (d) Let m be the smallest integer linear combination of a and b that is positive. Show that $m = \gcd(a, b)$.

Homework Problems

Problem 8.8.

Define the Pulverizer State machine to have:

states $::= \mathbb{N}^6$
 start state $::= (a, b, 0, 1, 1, 0)$ (where $a \geq b > 0$)
 transitions $::= (x, y, s, t, u, v) \longrightarrow$
 $(y, \text{rem}(x, y), u - sq, v - tq, s, t)$ (for $q = \text{qcnt}(x, y), y > 0$).

(a) Show that the following properties are preserved invariants of the Pulverizer machine:

$$\gcd(x, y) = \gcd(a, b), \tag{8.8}$$

$$sa + tb = y, \text{ and} \tag{8.9}$$

$$ua + vb = x. \tag{8.10}$$

(b) Conclude that the Pulverizer machine is partially correct.

(c) Explain why the machine terminates after at most the same number of transitions as the Euclidean algorithm.

Problem 8.9.

Prove that the smallest positive integers $a \geq b$ for which, starting in state (a, b) , the Euclidean state machine will make n transitions are $F(n + 1)$ and $F(n)$, where $F(n)$ is the n th Fibonacci number.

Hint: Induction.

In a later chapter, we’ll show that $F(n) \leq \varphi^n$ where φ is the golden ratio $(1 + \sqrt{5})/2$. This implies that the Euclidean algorithm halts after at most $\log_\varphi(a)$ transitions. This is a somewhat smaller than the $2 \log_2 a$ bound derived from equation (8.4).

Problem 8.10.

Let’s extend the jug filling scenario of Section 8.1.3 to three jugs and a receptacle. Suppose the jugs can hold a , b , and c gallons of water, respectively.

The receptacle can be used to store an unlimited amount of water, but has no measurement markings. Excess water can be dumped into the drain. Among the possible moves are:

1. fill a bucket from the hose,
2. pour from the receptacle to a bucket until the bucket is full or the receptacle is empty, whichever happens first,
3. empty a bucket to the drain,
4. empty a bucket to the receptacle,
5. pour from one bucket to another until either the first is empty or the second is full,

(a) Model this scenario with a state machine. (What are the states? How does a state change in response to a move?)

(b) Prove that Bruce can get $k \in \mathbb{N}$ gallons of water into the receptacle using the above operations only if $\gcd(a, b, c) \mid k$.

(c) Prove conversely, that if $\gcd(a, b, c) \mid k$, then Bruce can get actually get k gallons of water into the receptacle.

Problem 8.11.

The binary-GCD state machine computes the GCD of a and b using only division by 2 and subtraction, which makes it run very efficiently on hardware that uses binary representation of numbers. In practice, it runs more quickly than the Euclidean algorithm state machine (8.3).

$\text{states} ::= \mathbb{N}^3$
 $\text{start state} ::= (a, b, 1)$ (where $a > b > 0$)
 $\text{transitions} ::=$ if $\min(x, y) > 0$, then $(x, y, e) \longrightarrow$
the first possible state according to the rules:
$$\left\{ \begin{array}{ll} (1, 0, ex) & (\text{if } x = y) \\ (1, 0, e) & (\text{if } y = 1), \\ (x/2, y/2, 2e) & (\text{if } 2 \mid x \text{ and } 2 \mid y), \\ (y, x, e) & (\text{if } y > x) \\ (x, y/2, e) & (\text{if } 2 \mid y) \\ (x/2, y, e) & (\text{if } 2 \mid x) \\ (x - y, y, e) & (\text{otherwise}). \end{array} \right.$$

(a) Prove that if this machine reaches a “final” state (x, y, e) in which no transition is possible, then $e = \gcd(a, b)$.

(b) Prove that the machine reaches a final state in at most $3 + 2 \log \max(a, b)$ transitions.

Hint: Strong induction on $\max(a, b)$.

Problems for Section 8.3

Class Problems

Problem 8.12. (a) Let $m = 2^9 5^{24} 11^7 17^{12}$ and $n = 2^3 7^{22} 11^{211} 13^1 17^9 19^2$. What is the $\gcd(m, n)$? What is the *least common multiple*, $\text{lcm}(m, n)$, of m and n ? Verify that

$$\gcd(m, n) \cdot \text{lcm}(m, n) = mn. \quad (8.11)$$

(b) Describe in general how to find the $\gcd(m, n)$ and $\text{lcm}(m, n)$ from the prime factorizations of m and n . Conclude that equation (8.11) holds for all positive integers m, n .

Homework Problems

Problem 8.13.

The set of complex numbers that are equal to $m + n\sqrt{-5}$ for some integers m, n is called $\mathbb{Z}[\sqrt{-5}]$. It will turn out that in $\mathbb{Z}[\sqrt{-5}]$, not all numbers have unique factorizations.

A sum or product of numbers in $\mathbb{Z}[\sqrt{-5}]$ is in $\mathbb{Z}[\sqrt{-5}]$, and since $\mathbb{Z}[\sqrt{-5}]$ is a subset of the complex numbers, all the usual rules for addition and multiplication are true for it. But some weird things do happen. For example, the prime 29 has factors:

(a) Find $x, y \in \mathbb{Z}[\sqrt{-5}]$ such that $xy = 29$ and $x \neq \pm 1 \neq y$.

On the other hand, the number 3 is still a “prime” even in $\mathbb{Z}[\sqrt{-5}]$. More precisely, a number $p \in \mathbb{Z}[\sqrt{-5}]$ is called *irreducible* over $\mathbb{Z}[\sqrt{-5}]$ iff when $xy = p$ for some $x, y \in \mathbb{Z}[\sqrt{-5}]$, either $x = \pm 1$ or $y = \pm 1$.

Claim. The numbers $3, 2 + \sqrt{-5}$, and $2 - \sqrt{-5}$ are irreducible over $\mathbb{Z}[\sqrt{-5}]$.

In particular, this Claim implies that the number 9 factors into irreducibles over $\mathbb{Z}[\sqrt{-5}]$ in two different ways:

$$3 \cdot 3 = 9 = (2 + \sqrt{-5})(2 - \sqrt{-5}). \quad (8.12)$$

So $\mathbb{Z}[\sqrt{-5}]$ is an example of what is called a *non-unique factorization* domain.

To verify the Claim, we’ll appeal (without proof) to a familiar technical property of complex numbers given in the following Lemma.

Definition 8.9.1. For a complex number $c = r + si$ where $r, s \in \mathbb{R}$ and i is $\sqrt{-1}$, the *norm*, $|c|$, of c is $\sqrt{r^2 + s^2}$.

Lemma. For $c, d \in \mathbb{C}$,

$$|cd| = |c| |d|.$$

(b) Prove that $|x|^2 \neq 3$ for all $x \in \mathbb{Z}[\sqrt{-5}]$.

(c) Prove that if $x \in \mathbb{Z}[\sqrt{-5}]$ and $|x| = 1$, then $x = \pm 1$.

(d) Prove that if $|xy| = 3$ for some $x, y \in \mathbb{Z}[\sqrt{-5}]$, then $x = \pm 1$ or $y = \pm 1$.

Hint: $|z^2| \in \mathbb{N}$ for $z \in \mathbb{Z}[\sqrt{-5}]$.

(e) Complete the proof of the Claim.

Problems for Section 8.5

Practice Problems

Problem 8.14.

A majority of the following statements are equivalent. List all statements in this majority. Assume that $n > 0$ and a and b are integers.

1. $a \equiv b \pmod{n}$

2. $a = b$
3. $\text{rem}(a, n) = \text{rem}(b, n)$
4. $n \mid (a - b)$
5. $\exists k \in \mathbb{Z}. a = b + nk$
6. $(a - b)$ is a multiple of n
7. $n \mid a$ OR $n \mid b$

(a)

Homework Problems

Problem 8.15.

Prove that congruence is preserved by arithmetic expressions. Namely, prove that

$$a \equiv b \pmod{n}, \quad (8.13)$$

then

$$\text{eval}(e, a) \equiv \text{eval}(e, b) \pmod{n}, \quad (8.14)$$

for all $e \in \text{Aexp}$ (see Section 7.4).

Class Problems

Problem 8.16.

The following properties of equivalence mod n follow directly from its definition and simple properties of divisibility. See if you can prove them without looking up the proofs in the text.

- (a) If $a \equiv b \pmod{n}$, then $ac \equiv bc \pmod{n}$.
- (b) If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$.
- (c) If $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, then $ac \equiv bd \pmod{n}$.
- (d) $\text{rem}(a, n) \equiv a \pmod{n}$.

Problem 8.17. (a) Why is a number written in decimal evenly divisible by 9 if and only if the sum of its digits is a multiple of 9? *Hint:* $10 \equiv 1 \pmod{9}$.

8.9. What has SAT got to do with it?

229

(b) Take a big number, such as 37273761261. Sum the digits, where every other one is negated:

$$3 + (-7) + 2 + (-7) + 3 + (-7) + 6 + (-1) + 2 + (-6) + 1 = -11$$

Explain why the original number is a multiple of 11 if and only if this sum is a multiple of 11.

Problem 8.18.

At one time, the Guinness Book of World Records reported that the “greatest human calculator” was a guy who could compute 13th roots of 100-digit numbers that were powers of 13. What a curious choice of tasks . . .

(a) Prove that

$$d^{13} \equiv d \pmod{10} \tag{8.15}$$

for $0 \leq d < 10$.

(b) Now prove that

$$n^{13} \equiv n \pmod{10} \tag{8.16}$$

for all n .

Exam Problems

Problem 8.19.

The sum of the digits of the base 10 representation of an integer is congruent modulo 9 to that integer. For example

$$763 \equiv 7 + 6 + 3 \pmod{9}.$$

This is not always true for the hexadecimal (base 16) representation however. For example,

$$(763)_{16} = 7 \cdot 16^2 + 6 \cdot 16 + 3 \equiv 1 \not\equiv 7 \equiv 7 + 6 + 3 \pmod{9}.$$

For exactly what integers $k > 1$ is it true that the sum of the digits of the base 16 representation of an integer is congruent modulo k to that integer? (No explanation is required, but no part credit without an explanation.)

Problems for Section 8.6

Practice Problems

Problem 8.20.

What is $\text{rem}(24^{78}, 79)$? *Hint:* : 79 is prime. You should not need to do any calculation!

Class Problems

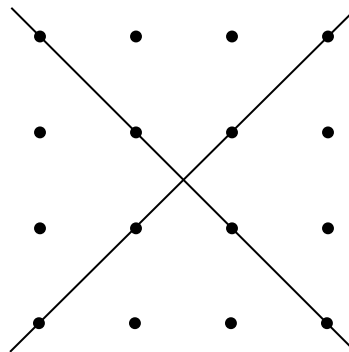
Problem 8.21.

Two nonparallel lines in the real plane intersect at a point. Algebraically, this means that the equations

$$y = m_1x + b_1$$

$$y = m_2x + b_2$$

have a unique solution (x, y) , provided $m_1 \neq m_2$. This statement would be false if we restricted x and y to the integers, since the two lines could cross at a noninteger point:



However, an analogous statement holds if we work over the integers *modulo a prime*, p . Find a solution to the congruences

$$y \equiv m_1x + b_1 \pmod{p}$$

$$y \equiv m_2x + b_2 \pmod{p}$$

when $m_1 \not\equiv m_2 \pmod{p}$. Express your solution in the form $x \equiv ? \pmod{p}$ and $y \equiv ? \pmod{p}$ where the ?'s denote expressions involving m_1 , m_2 , b_1 , and b_2 . You may find it helpful to solve the original equations over the reals first.

8.9. What has SAT got to do with it?

231

Problem 8.22.

Let $S_k = 1^k + 2^k + \dots + (p-1)^k$, where p is an odd prime and k is a positive multiple of $p-1$. Use Fermat's theorem to prove that $S_k \equiv -1 \pmod{p}$.

Homework Problems

Problem 8.23. (a) Use the Pulverizer to find the inverse of 13 modulo 23 in the interval $[1, 23)$.

(b) Use Fermat's theorem to find the inverse of 13 modulo 23 in $[1, 23)$.

Problems for Section 8.7

Practice Problems

Problem 8.24. (a) Prove that 22^{12001} has a multiplicative inverse modulo 175.

(b) What is the value of $\phi(175)$, where ϕ is Euler's function?

(c) What is the remainder of 22^{12001} divided by 175?

Problem 8.25. (a) Use the Pulverizer to find integers s, t such that

$$40s + 7t = \gcd(40, 7).$$

Show your work.

(b) Adjust your answer to part (a) to find an inverse modulo 40 of 7 in $[1, 40)$.

Problem 8.26.

How many numbers between 1 and 3780 (inclusive) are relatively prime to 3780?

Problem 8.27.

What is the multiplicative inverse (mod 7) of 2? *Reminder:* by definition, your answer must be an integer between 0 and 6.

Class Problems

Problem 8.28.

Let a and b be relatively prime positive integers.

- (a) How many integers in the interval $[0, ab)$ are divisible by a ?
- (b) How many integers in the interval $[0, ab)$ are divisible by both a and b ?
- (c) How many integers in the interval $[0, ab)$ are divisible by either a or b ?
- (d) Now suppose $p \neq q$ are both primes. How many integers in the interval $[0, pq)$ are *not* relatively prime to pq ? Observe that a different answer is required if p and q were merely relatively prime numbers a and b as in part (c).
- (e) Conclude that

$$\phi(pq) = (p-1)(q-1).$$

Problem 8.29.

Suppose a, b are relatively prime and greater than 1. In this problem you will prove the *Chinese Remainder Theorem*, which says that for all m, n , there is an x such that

$$x \equiv m \pmod{a}, \tag{8.17}$$

$$x \equiv n \pmod{b}. \tag{8.18}$$

Moreover, x is unique up to congruence modulo ab , namely, if x' also satisfies (8.17) and (8.18), then

$$x' \equiv x \pmod{ab}.$$

- (a) Prove that for any m, n , there is some x satisfying (8.17) and (8.18).

Hint: Let b^{-1} be an inverse of b modulo a and define $e_a ::= b^{-1}b$. Define e_b similarly. Let $x = me_a + ne_b$.

- (b) Prove that

$$[x \equiv 0 \pmod{a} \text{ AND } x \equiv 0 \pmod{b}] \text{ implies } x \equiv 0 \pmod{ab}.$$

- (c) Conclude that

$$[x \equiv x' \pmod{a} \text{ AND } x \equiv x' \pmod{b}] \text{ implies } x \equiv x' \pmod{ab}.$$

- (d) Conclude that the Chinese Remainder Theorem is true.

- (e) What about the converse of the implication in part (c)?

Homework Problems

Problem 8.30.

Suppose a, b are relatively prime integers greater than 1. In this problem you will prove that Euler’s function is *multiplicative*, namely, that

$$\phi(ab) = \phi(a)\phi(b).$$

The proof is an easy consequence of the Chinese Remainder Theorem (Problem 8.29).

(a) Conclude from the Chinese Remainder Theorem that the function $f : [0, ab) \rightarrow [0, a) \times [0, b)$ defined by

$$f(x) ::= (\text{rem}(x, a), \text{rem}(x, b))$$

is a bijection.

(b) For any positive integer, k , let k^* be the integers in $[1, k)$ that are relatively prime to k . Prove that the function f from part (a) also defines a bijection from $(ab)^*$ to $a^* \times b^*$.

(c) Conclude from the preceding parts of this problem that

$$\phi(ab) = \phi(a)\phi(b). \tag{8.19}$$

(d) Prove Corollary 8.7.7: for any number $n > 1$, if $p_1, p_2, \dots, p_j$ are the (distinct) prime factors of n , then

$$\phi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_j}\right).$$

Problem 8.31.

The general version of the Chinese Remainder theorem (Problem 8.29) extends to more than two relatively prime moduli. Namely,

Theorem (General Chinese Remainder). *Suppose $a_1, \dots, a_k$ are integers greater than 1 and each is relatively prime to the others. Let $n ::= a_1 \cdot a_2 \cdots a_k$. Then for any integers $m_1, m_2, \dots, m_k$, there is a unique $x \in [0, n)$ such that*

$$x \equiv m_i \pmod{a_i},$$

for $1 \leq i \leq k$.

The proof is a routine induction on k using a fact that follows immediately from unique factorization: if a number is relatively prime to some other numbers, then it is relatively prime to their product.

Now suppose an n -bit number, N , was a product of relatively prime k -bit numbers, where n was big, but k was small enough to be handled by cheap and available arithmetic hardware units. Suppose a calculation requiring a large number of additions and multiplications modulo N had to be performed starting with some small set of n -bit numbers. For example, suppose we wanted to compute

$$\text{rem}((x-3)^{110033}((y+7)^{27123} - z^{4328}), N)$$

which would require several dozen n -bit operations starting from the three numbers x, y, z .

Doing a multiplication or addition modulo N directly requires breaking up the n -bit numbers x, y, z and all the intermediate results of the mod N calculation into k -bit pieces, using the hardware to perform the additions and multiplications on the pieces, and then reassembling the k -bit results into an n -bit answer after each operation. Suppose N was a product of m relatively prime k -bit numbers.

Explain how the General Chinese Remainder Theorem offers a far more efficient approach to performing the required operations.

Exam Problems

Problem 8.32.

Circle true or false for the statements below, and provide counterexamples for those that are false. Variables, a, b, c, m, n range over the integers and $m, n > 1$.

- | | | |
|---|-------------|--------------|
| (a) $\gcd(1+a, 1+b) = 1 + \gcd(a, b)$. | true | false |
| (b) If $a \equiv b \pmod{n}$, then $p(a) \equiv p(b) \pmod{n}$ for any polynomial $p(x)$ with integer coefficients. | true | false |
| (c) If $a \mid bc$ and $\gcd(a, b) = 1$, then $a \mid c$. | true | false |
| (d) $\gcd(a^n, b^n) = (\gcd(a, b))^n$ | true | false |
| (e) If $\gcd(a, b) \neq 1$ and $\gcd(b, c) \neq 1$, then $\gcd(a, c) \neq 1$. | true | false |
| (f) If an integer linear combination of a and b equals 1, then so does some integer linear combination of a^2 and b^2 . | true | false |

8.9. What has SAT got to do with it?

235

- (g) If no integer linear combination of a and b equals 2, then neither does any integer linear combination of a^2 and b^2 . **true false**
- (h) If $ac \equiv bc \pmod{n}$ and n does not divide c , then $a \equiv b \pmod{n}$. **true false**
- (i) Assuming a, b have inverses modulo n , if $a^{-1} \equiv b^{-1} \pmod{n}$, then $a \equiv b \pmod{n}$. **true false**
- (j) If $ac \equiv bc \pmod{n}$ and n does not divide c , then $a \equiv b \pmod{n}$. **true false**
- (k) If $a \equiv b \pmod{\phi(n)}$ for $a, b > 0$, then $c^a \equiv c^b \pmod{n}$. **true false**
- (l) If $a \equiv b \pmod{nm}$, then $a \equiv b \pmod{n}$. **true false**
- (m) If $\gcd(m, n) = 1$, then $[a \equiv b \pmod{m} \text{ AND } a \equiv b \pmod{n}]$ iff $[a \equiv b \pmod{mn}]$ **true false**
- (n) If $\gcd(a, n) = 1$, then $a^{n-1} \equiv 1 \pmod{n}$ **true false**
- (o) If $a, b > 1$, then $[a \text{ has a multiplicative inverse mod } b \text{ iff } b \text{ has a multiplicative inverse mod } a]$. **true false**

Problem 8.33.

Find the remainder of $26^{1818181}$ divided by 297. *Hint:* $1818181 = (180 \cdot 10101) + 1$; Euler's theorem

Problem 8.34.

Find an integer $k > 1$ such that n and n^k agree in their last three digits whenever n is divisible by neither 2 nor 5. *Hint:* Euler's theorem.

Problem 8.35.

What is the remainder of 63^{9601} divided by 220?

Problem 8.36.

- (a) Explain why $(-12)^{526}$ has a multiplicative inverse modulo 175.
- (b) What is the value of $\phi(175)$, where ϕ is Euler’s function?
- (c) Call a number from 0 to 174 *powerful* iff some positive power of the number is congruent to 1 modulo 175. What is the probability that a random number from 0 to 174 is powerful?
- (d) What is the remainder of $(-12)^{482}$ divided by 175?

Problems for Section 8.9

Class Problems

Problem 8.37.

Let’s try out RSA!

- (a) As a team, go through the **beforehand** steps.
 - Choose primes p and q to be relatively small, say in the range 10-40. In practice, p and q might contain hundreds of digits, but small numbers are easier to handle with pencil and paper.
 - Try $e = 3, 5, 7, \dots$ until you find something that works. Use Euclid’s algorithm to compute the gcd.
 - Find d (using the Pulverizer or Euler’s Theorem).

When you’re done, put your public key on the board. This lets another team send you a message.

- (b) Now send an encrypted message to another team using their public key. Select your message m from the codebook below:

- 2 = Greetings and salutations!
- 3 = Yo, wassup?
- 4 = You guys are slow!

8.9. What has SAT got to do with it?

237

- 5 = All your base are belong to us.
- 6 = Someone on *our* team thinks someone on *your* team is kinda cute.
- 7 = You *are* the weakest link. Goodbye.

(c) Decrypt the message sent to you and verify that you received what the other team sent!

Problem 8.38.

A critical fact about RSA is, of course, that decrypting an encrypted message, m^* , always gives back the original message, m . Namely, if $n = pq$ where p and q are distinct primes, $m \in [0, pq)$, and

$$d \cdot e \equiv 1 \pmod{(p-1)(q-1)},$$

then

$$\text{rem}(\text{rem}(m^d, pq)^e, pq) = m. \quad (8.20)$$

We'll now prove this.

(a) Verify that for all $d, e \in \mathbb{N}$, if

$$(m^d)^e \equiv m \pmod{pq}, \quad (8.21)$$

then (8.20) is true.

(b) Prove that if p is prime, then $m^a \equiv m \pmod{p}$ for all $a \in \mathbb{N}$ congruent to 1 mod $p-1$.

(c) Prove that if $a \equiv b \pmod{p_i}$ for distinct primes $p_1, p_2, \dots, p_n$, then $a \equiv b \pmod{p_1 p_2 \cdots p_n}$.

(d) Prove

Lemma. If n is a product of distinct primes and $a \in \mathbb{N}$ is $\equiv 1 \pmod{\phi(n)}$, then $m^a \equiv m \pmod{n}$.

(e) Combine the previous parts to complete the proof of (8.20).

Problem 8.39.

Although RSA has successfully withstood cryptographic attacks for a more than a quarter century, it is not known that breaking RSA would imply that factoring is easy.

In this problem we will examine the *Rabin cryptosystem* that does have such a security certification. Namely, if someone has the ability to break the Rabin cryptosystem efficiently, then they also have the ability to factor numbers that are products of two primes.

Why should that convince us that it is hard to break the cryptosystem efficiently? Well, mathematicians have been trying to factor efficiently for centuries, and they still haven't figured out how to do it.

What is the Rabin cryptosystem? The public key will be a number N that is a product of two very large primes p, q such that $p \equiv q \equiv 3 \pmod{4}$. To send the message x , send $\text{rem}(x^2, N)$.⁹

The private key is the factorization of N , namely, the primes p, q . We need to show that if the person being sent the message knows p, q , then they can decode the message. On the other hand, if an eavesdropper who doesn't know p, q listens in, then we must show that they are very unlikely to figure out this message.

First some definitions. We know what it means for a number to be a square over the integers, that is s is a square if there is another integer x such that $s = x^2$. Over the numbers mod N , we say that s is a *square modulo N* if there is an x such that $s \equiv x^2 \pmod{N}$. If x is such that $0 \leq x < N$ and $s \equiv x^2 \pmod{N}$, then x is the *square root of s* .

(a) What are the squares modulo 5? For each nonzero square in the interval $[0, 5)$, how many square roots does it have?

(b) For each integer in $[1, 15)$ that is relatively prime to 15, how many square roots (modulo 15) does it have? Note that all the square roots are *also* relatively prime to 15. We won't go through why this is so here, but keep in mind that this is a general phenomenon!

(c) Suppose that p is a prime such that $p \equiv 3 \pmod{4}$. It turns out that squares modulo p have exactly 2 square roots. First show that $(p + 1)/4$ is an integer. Next figure out the two square roots of 1 modulo p . Then show that you can find a “square root mod a prime p ” of a number by raising the number to the $(p + 1)/4$ th power. That is, given s , to find x such that $s \equiv x^2 \pmod{p}$, you can compute $\text{rem}(s^{(p+1)/4}, p)$.

(d) The Chinese Remainder Theorem (Problem 8.29) implies that if p, q are distinct primes, then s is a square modulo pq if and only if s is a square modulo p and s is a square modulo q . In particular, if $s \equiv x^2 \pmod{p} \equiv (x')^2 \pmod{p}$ and

⁹We will see soon, that there are other numbers that would be encrypted by $\text{rem}(x^2, N)$, so we'll have to disallow those other numbers as possible messages in order to make it possible to decode this cryptosystem, but let's ignore that for now.

$s \equiv y^2 \pmod{p} \equiv (y')^2 \pmod{p}$ then s has exactly four square roots, namely,

$$s \equiv (xy)^2 \equiv (x'y)^2 \equiv (xy')^2 \equiv (x'y')^2 \pmod{pq}.$$

So, if you know p, q , then using the solution to part (c), you can efficiently find the square roots of s ! Thus, given the private key, decoding is easy.

But what if you don't know p, q ? Suppose $N ::= pq$, where p, q are two primes equivalent to 3 (mod 4). Let's assume that the evil message interceptor claims to have a program that can find all four square roots of any number modulo N . Show that he can actually use this program to efficiently find the factorization of N . Thus, unless this evil message interceptor is extremely smart and has figured out something that the rest of the scientific community has been working on for years, it is very unlikely that this efficient square root program exists!

Hint: Pick r arbitrarily from $[1, N)$. If $\gcd(N, r) > 1$, then you are done (why?) so you can halt. Otherwise, use the program to find all four square roots of r , call them $r, -r, r', -r'$. Note that $r^2 \equiv r'^2 \pmod{N}$. How can you use these roots to factor N ?

(e) If the evil message interceptor knows that the message is the encoding one of two possible candidate messages (that is, either “meet at dome at dusk” or “meet at dome at dawn”) and is just trying to figure out which of the two, then can he break this cryptosystem?

Problem 8.40.

You've seen how the RSA encryption scheme works, but why is it hard to break? In this problem, you will see that finding private keys is as hard as finding the prime factorizations of integers. Since there is a general consensus in the crypto community (enough to persuade many large financial institutions, for example) that factoring numbers with a few hundred digits requires astronomical computing resources, we can therefore be sure it will take the same kind of overwhelming effort to find RSA private keys of a few hundred digits. This means we can be confident the private RSA keys are not somehow revealed by the public keys ¹⁰

For this problem, assume that $n = p \cdot q$ where p, q are both *odd* primes and that e is the public key and d the private key of the RSA protocol.. Let $x ::= e \cdot d - 1$.

(a) Show that $\phi(n)$ divides x .

¹⁰This is a very weak kind of “security” property, because it doesn't even rule out the possibility of deciphering RSA encoded messages by some method that did not require knowing the private key. Nevertheless, over twenty years experience supports the security of RSA in practice.

(b) Conclude that 4 divides x .

(c) Show that if $\gcd(r, n) = 1$, then $r^x \equiv 1 \pmod{n}$.

A *square root* of m modulo n is a nonnegative integer $s < n$ such that $s^2 \equiv m \pmod{n}$. Here is a nice fact to know: when n is a product of two odd primes, then every number m such that $\gcd(m, n) = 1$ has 4 square roots modulo n .

In particular, the number 1 has four square roots modulo n . The two trivial ones are 1 and $n - 1$ (which is $\equiv -1 \pmod{n}$). The other two are called the *nontrivial* square roots of 1.

(d) Since you know x , then for any integer, r , you can also compute the remainder, y , of $r^{x/2}$ divided by n . So $y^2 \equiv r^x \pmod{n}$. Now if r is relatively prime to n , then y will be a square root of 1 modulo n by part (c).

Show that if y turns out to be a *nontrivial* root of 1 modulo n , then you can factor n . *Hint:* From the fact that $y^2 - 1 = (y + 1)(y - 1)$, show that $y + 1$ must be divisible by exactly one of q and p .

(e) It turns out that at least half the positive integers $r < n$ that are relatively prime to n will yield y 's in part (d) that are nontrivial roots of 1. Conclude that if, in addition to n and the public key, e , you also knew the private key d , then you can be sure of being able to factor n .

II Structures



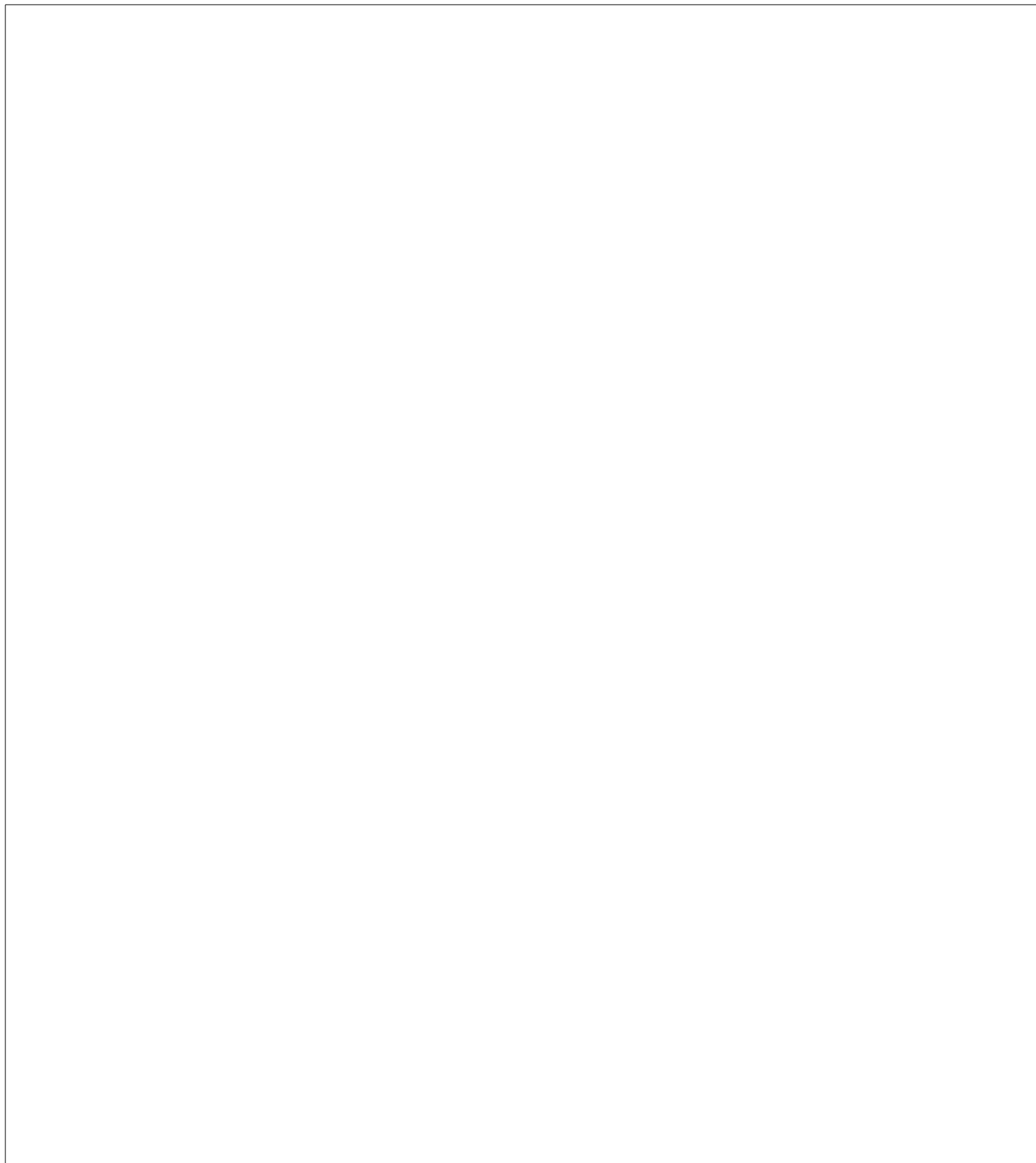
Introduction

Structure is fundamental in computer science. Whether you are writing code, solving an optimization problem, or designing a network, you will be dealing with structure. The better you can understand the structure, the better your results will be. And if you can reason about structure, then you will be in a good position to convince others (and yourself) that your results are worthy.

The most important structure in computer science is a *graph*, also known as a *network*). Graphs provide an excellent mechanism for modeling associations between pairs of objects; for example, two exams that cannot be given at the same time, two people that like each other, or two subroutines that can be run independently. In Chapter 9, we study *directed graphs* which model *one-way* relationships such as being bigger than, loving (sadly, it’s often not mutual), being a prerequisite for. A highlight is the special case of acyclic digraphs (*DAGs*) that correspond to a class of relations called *partial orders*. Partial orders arise frequently in the study of scheduling and concurrency. Digraphs as models for data communication and routing problems are the topic of Chapter 10.

In Chapter 11 we focus on *simple graphs* that represent mutual or *symmetric* relationships, such as being congruent modulo 17, being in conflict, being compatible, being independent, being capable of running in parallel. Simple graphs that can be drawn in the plane are examined in Chapter 12. The impossibility of placing 50 geocentric satellites in orbit so that they *uniformly* blanket the globe will be one of the conclusions reached in this chapter.

This part of the text concludes with Chapter 13 which elaborates the use of the *state machines* in program verification and modeling concurrent computation.



9 Directed graphs & Partial Orders

Directed graphs, called *digraphs* for short, provide a handy way to represent how things are connected together and how to get from one thing to another by following the connections. They are usually pictured as a bunch of dots or circles with arrows between some of the dots as in Figure 9.1. The dots are called *nodes* (or *vertices*) and the lines are called *directed edges* or *arrows*, so the digraph in Figure 9.1 has 4 nodes and 6 directed edges.

Digraphs appear everywhere in computer science. In Chapter 10, we’ll use digraphs to describe communication nets for routing data packets. The digraph in Figure 9.2 has three “in” nodes (pictured as little squares) representing locations where packets may arrive at the net, the three “out” nodes representing destination locations for packets, and the remaining six nodes (pictured with little circles) represent switches. The 16 edges indicate paths that packets can take through the router.

Another digraph example is the hyperlink structure of the World Wide Web. Letting the vertices $x_1, \dots, x_n$ correspond to web pages, and using arrows to indicate when one page has a hyperlink to another, yields a digraph like the one in Figure 9.3. In the graph of the real World Wide Web, n would be a number in the billions and probably even the trillions. At first glance, this graph wouldn’t seem to be very interesting. But in 1995, two students at Stanford, Larry Page and Sergey Brin, ultimately became multibillionaires from the realization of how useful the structure of this graph could be in building a search engine. So pay attention to graph theory, and who knows what might happen!

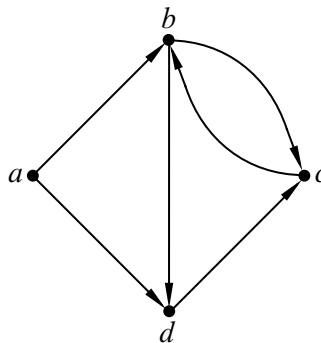


Figure 9.1 A 4-node directed graph with 6 edges.

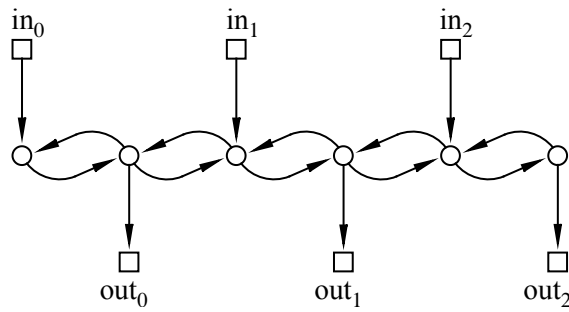


Figure 9.2 A 6-switch packet routing digraph.

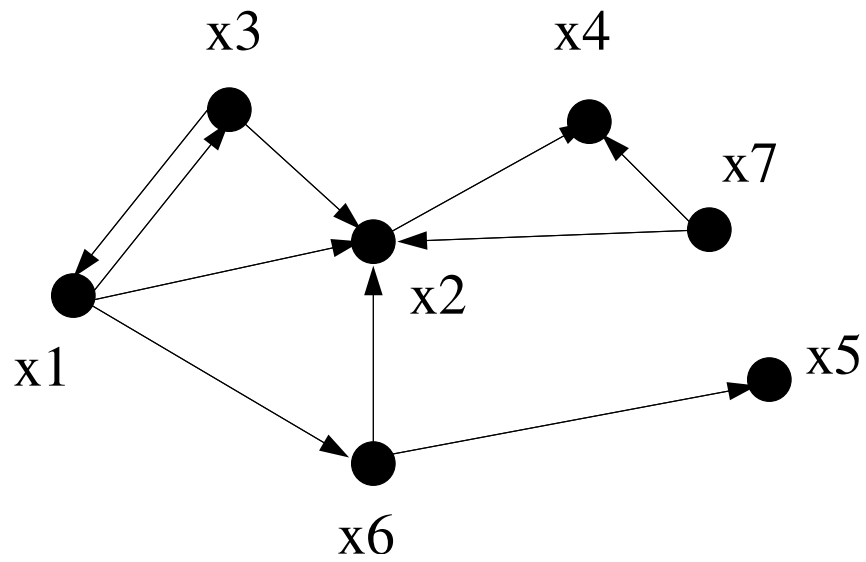


Figure 9.3 Links among Web Pages.

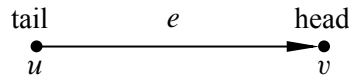


Figure 9.4 A directed edge $e = \langle u \rightarrow v \rangle$. The edge e starts at the tail vertex, u , and ends at the head vertex, v .

9.1 Digraphs & Vertex Degrees

Definition 9.1.1. A *directed graph*, G , consists of a nonempty set, $V(G)$, called the *vertices* of G , and a set, $E(G)$, called the *edges* of G . An element of $V(G)$ is called a *vertex*. A vertex is also called a *node*; the words “vertex” and “node” are used interchangeably. An element of $E(G)$ is called a *directed edge*. A directed edge is also called an “arrow” or simply an “edge.” A directed edge *starts* at some vertex, u , called the *tail* of the edge, and *ends* at some vertex, v , called the *head* of the edge, as in Figure 9.4. Such an edge can be represented by the ordered pair (u, v) . The notation $\langle u \rightarrow v \rangle$ denotes this edge.

There is nothing new in Definition 9.1.1 except for a lot of vocabulary. Formally, a digraph G is the same as a binary relation on the set, $V = V(G)$ —that is, a digraph is just a binary relation whose domain and codomain are the same set, V . In fact we’ve already referred to the arrows in a relation G as the “graph” of G . For example, the divisibility relation on the integers in the interval $[1, 12]$ could be pictured by the digraph in Figure 9.5.

The *in-degree* of a vertex in a digraph is the number of arrows coming into it and similarly its *out-degree* is the number of arrows out of it. More precisely,

Definition 9.1.2. If G is a digraph and $v \in V(G)$, then

$$\text{indeg}(v) ::= |\{e \in E(G) \mid \text{head}(e) = v\}|$$

$$\text{outdeg}(v) ::= |\{e \in E(G) \mid \text{tail}(e) = v\}|$$

An immediate consequence of this definition is

Lemma 9.1.3.

$$\sum_{v \in V(G)} \text{indeg}(v) = \sum_{v \in V(G)} \text{outdeg}(v).$$

Proof. Both sums are obviously equal to $|E(G)|$. ■

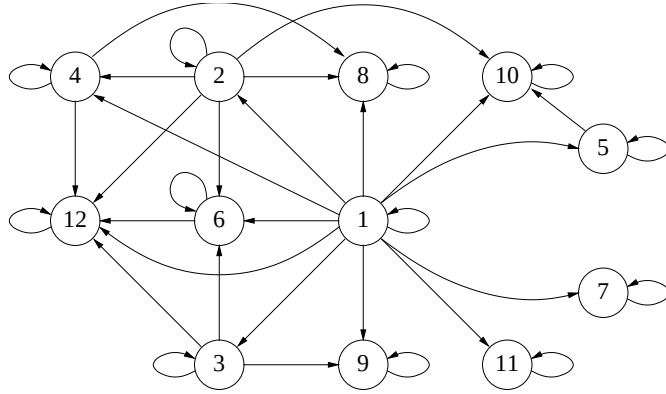


Figure 9.5 The Digraph for Divisibility on $\{1, 2, \dots, 12\}$.

9.2 Digraph Walks and Paths

Picturing digraphs with points and arrows makes it natural to talk about following successive edges through the graph. For example, in the digraph of Figure 9.5, you might start at vertex 1, successively follow the edges from vertex 1 to vertex 2, from 2 to 4, from 4 to 12, and then from 12 to 12 twice (or as many times as you like). The sequence of edges followed in this way is called a *walk* through the graph.

The obvious way to represent a walk is with the sequence of successive vertices it went through, in this case:

1 2 4 12 12 12.

However, it is conventional to represent a walk by an alternating sequence of successive vertices and edges, so this walk would formally be

$$1 \langle 1 \rightarrow 2 \rangle 2 \langle 2 \rightarrow 4 \rangle 4 \langle 4 \rightarrow 12 \rangle 12 \langle 12 \rightarrow 12 \rangle 12 \langle 12 \rightarrow 12 \rangle 12. \quad (9.1)$$

The redundancy of this definition is enough to make any computer scientist cringe, but it does make it easy to talk about how many times vertices and edges occur on the walk. Here is a formal definition:

Definition 9.2.1. A *walk in a digraph*, G , is an alternating sequence of vertices and edges that begins with a vertex, ends with a vertex, and such that for every edge $\langle u \rightarrow v \rangle$ in the walk, vertex u is the element just before the edge, and vertex v is the next element after the edge.

So a walk, $\mathbf{v}$, is a sequence of the form

$$\mathbf{v} ::= v_0 \langle v_0 \rightarrow v_1 \rangle v_1 \langle v_1 \rightarrow v_2 \rangle v_2 \dots \langle v_{k-1} \rightarrow v_k \rangle v_k$$

where $\langle v_i \rightarrow v_{i+1} \rangle \in E(G)$ for $i \in [0, k)$. The walk is said to *start* at v_0 , to *end* at v_k , and the *length*, $|v|$, of the walk is defined to be k . The walk is a *path* iff all the v_i 's are different, that is, if $i \neq j$, then $v_i \neq v_j$.

A *closed walk* is a walk that begins and ends at the same vertex. A *cycle* is a closed walk whose vertices are distinct except for the beginning and end vertices.

Note that a single vertex counts as a length zero path, and also a length zero cycle, that begins and ends at itself.

Although a walk is officially an alternating sequence of vertices and edges, it is completely determined just by the sequence of successive vertices on it, or by the sequence of edges on it, and we will describe walks that way whenever it's convenient. For example, for the graph in Figure 9.1,

- (a, b, d) , or simply abd , is (a vertex-sequence description of) a length-2 path,
- $(\langle a \rightarrow b \rangle, \langle b \rightarrow d \rangle)$, or simply $\langle a \rightarrow b \rangle \langle b \rightarrow d \rangle$, is (an edge-sequence description of) the same length-2 path,
- $abcdb$ is a length-4 walk,
- $dcbcbd$ is a length-5 closed walk,
- $bdc b$ is a length-3 cycle,
- $\langle b \rightarrow c \rangle \langle c \rightarrow b \rangle$ is a length-2 cycle, and
- $\langle c \rightarrow b \rangle \langle b \leftarrow a \rangle \langle a \rightarrow d \rangle$ is *not* a walk. A walk is not allowed to follow edges in the wrong direction.

Length-1 cycles are also possible. The graph in Figure 9.1 has none, but every vertex in the divisibility relation digraph of Figure 9.5 is in a length-1 cycle. Length-1 cycles are sometimes called *self-loops*.

If you walk for a while, stop for a rest at some vertex, and then continue walking, you have broken a walk into two parts. For example, stopping to rest after following two edges in the walk (9.1) through the divisibility graph breaks the walk into the first part of the walk

$$1 \langle 1 \rightarrow 2 \rangle 2 \langle 2 \rightarrow 4 \rangle 4 \tag{9.2}$$

from 1 to 4, and the rest of the walk

$$4 \langle 4 \rightarrow 12 \rangle 12 \langle 12 \rightarrow 12 \rangle 12 \langle 12 \rightarrow 12 \rangle 12. \tag{9.3}$$

from 4 to 12, and we'll say the whole walk (9.1) is the *merge* of the walks (9.2) and (9.3). In general, if a walk $\mathbf{f}$ ends with a vertex, v , and a walk $\mathbf{r}$ starts with the

same vertex, v , we'll say that their *merge*, $\mathbf{f} \hat{\vee} \mathbf{r}$, is the walk that starts with $\mathbf{f}$ and continues with $\mathbf{r}$.¹ Two walks can only be merged if the first ends with the same vertex, v , that the second one starts with. Sometimes it's useful to name the node v where the walks merge; we'll use the notation $\mathbf{f} \hat{\vee}_v \mathbf{r}$ to describe the merge of a walk $\mathbf{f}$ that ends at v with a walk $\mathbf{r}$ that begins at v .

A consequence of this definition is that

Lemma 9.2.2.

$$|\mathbf{f} \hat{\vee} \mathbf{r}| = |\mathbf{f}| + |\mathbf{r}|.$$

In the next section we'll get mileage out of walking this way.

9.2.1 Finding a Path

If you were trying to walk somewhere quickly, you'd know you were in trouble if you came to the same place twice. This is actually a basic theorem of graph theory.

Theorem 9.2.3. *The shortest walk from one vertex to another is a path.*

Proof. If there is a walk from vertex u to v , there must, by the Well Ordering Principle, be a minimum length walk $\mathbf{w}$ from u to v . We claim $\mathbf{w}$ is a path.

To prove the claim, suppose to the contrary that $\mathbf{w}$ is not a path, namely, some vertex x occurs twice on this walk. That is,

$$\mathbf{w} = \mathbf{e} \hat{\vee} \mathbf{f} \hat{\vee} \mathbf{g}$$

for some walks $\mathbf{e}, \mathbf{f}, \mathbf{g}$ where the length of $\mathbf{f}$ is positive. But then “deleting” $\mathbf{f}$ yields a strictly shorter walk

$$\mathbf{e} \hat{\vee} \mathbf{g}$$

from u to v , contradicting the minimality of $\mathbf{w}$. ■

Definition 9.2.4. The *distance* $\text{dist}(u, v)$, in a graph from vertex u to vertex v is the length of a shortest path from u to v .

As would be expected, this definition of distance satisfies:

Lemma 9.2.5. *[The Triangle Inequality]*

$$\text{dist}(u, v) \leq \text{dist}(u, x) + \text{dist}(x, v)$$

for all vertices u, v, x with equality holding iff x is on a shortest path from u to v .

¹It's tempting to say the *merge* is the concatenation of the two walks, but that wouldn't quite be right because if the walks were concatenated, the vertex v would appear twice in a row where the walks meet.

Of course you may expect this property to be true, but distance has a technical definition and its properties can’t be taken for granted. For example, unlike ordinary distance in space, the distance from u to v is typically different from the distance from v to u . So let’s prove the Triangle Inequality:

Proof. To prove the inequality, suppose $\mathbf{f}$ is a shortest path from u to x and $\mathbf{r}$ is a shortest path from x to v . Then by Lemma 9.2.2, $\mathbf{f} \hat{\times} \mathbf{r}$ is a walk of length $\text{dist}(u, x) + \text{dist}(x, v)$ from u to v , so this sum is an upper bound on the length of the shortest path from u to v by Theorem 9.2.3.

To prove the “iff” from left to right, suppose $\text{dist}(u, v) = \text{dist}(u, x) + \text{dist}(x, v)$. Then merging a shortest path from u to x with shortest path from x to v yields a walk whose length is $\text{dist}(u, x) + \text{dist}(x, v)$ which by assumption equals $\text{dist}(u, v)$. This walk must be a path or it could be shortened, giving a smaller distance from u to v . So this is a shortest path containing x .

To prove the “iff” from right to left, suppose vertex x is on a shortest path $\mathbf{w}$ from u to v , namely, $\mathbf{w}$ is a shortest path of the form $\mathbf{f} \hat{\times} \mathbf{r}$. The path $\mathbf{f}$ must be a shortest path from u to x ; otherwise replacing $\mathbf{f}$ by a shorter path from u to x would yield a shorter path from u to v than $\mathbf{w}$. Likewise $\mathbf{r}$ must be a shortest path from x to v . So $\text{dist}(u, v) = |\mathbf{w}| = |\mathbf{f}| + |\mathbf{r}| = \text{dist}(u, x) + \text{dist}(x, v)$. ■

9.3 Adjacency Matrices

If a graph, G , has n vertices, $v_0, v_1, \dots, v_{n-1}$, a useful way to represent it is with a $n \times n$ matrix of zeroes and ones called its *adjacency matrix*, A_G . The ij th entry, $(A_G)_{ij}$, of the adjacency matrix is 1 if there is an edge from vertex v_i to vertex v_j , and 0 otherwise. That is,

$$(A_G)_{ij} ::= \begin{cases} 1 & \text{if } \langle v_i \rightarrow v_j \rangle \in E(G), \\ 0 & \text{otherwise.} \end{cases}$$

For example, let H be the 4-node graph shown in Figure 9.1. Then its adjacency matrix A_H is the 4×4 matrix:

$$A_H = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 1 & 0 & 1 \\ b & 0 & 0 & 1 & 1 \\ c & 0 & 1 & 0 & 0 \\ d & 0 & 0 & 1 & 0 \end{array}$$

A payoff of this representation is that we can use matrix powers to count numbers of walks between vertices. For example, there are two length-2 walks between vertices a and c in the graph H , namely

$$\begin{array}{l} a \langle a \rightarrow b \rangle b \langle b \rightarrow c \rangle c \\ a \langle a \rightarrow d \rangle d \langle d \rightarrow c \rangle c \end{array}$$

and these are the only length-2 walks from a to c . Also, there is exactly one length-2 walk from b to c and exactly one length-2 walk from c to c and from d to b , and these are the only length-2 walks in H . It turns out we could have read these counts from the entries in the matrix $(A_H)^2$:

$$(A_H)^2 = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 0 & 2 & 1 \\ b & 0 & 1 & 1 & 0 \\ c & 0 & 0 & 1 & 1 \\ d & 0 & 1 & 0 & 0 \end{array}$$

More generally, the matrix $(A_G)^k$ provides a count of the number of length k walks between vertices in any digraph, G , as we'll now explain.

Definition 9.3.1. The length- k walk counting matrix for an n -vertex graph G is the $n \times n$ matrix C such that

$$C_{uv} ::= \text{the number of length-}k \text{ walks from } u \text{ to } v. \quad (9.4)$$

Notice that the adjacency matrix A_G is the length-1 walk counting matrix for G , and that $(A_G)^0$, which by convention is the identity matrix, is the length-0 walk counting matrix.

Theorem 9.3.2. If C is the length- k walk counting matrix for a graph G , and D is the length- m walk counting matrix, then CD is the length $k + m$ walk counting matrix for G .

According to this theorem, the square $(A_G)^2$ of the adjacency matrix is the length-2 walk counting matrix for G . Applying the theorem again to $(A_G)^2 A_G$, shows that the length-3 walk counting matrix is $(A_G)^3$. More generally, it follows by induction that

Corollary 9.3.3. The length- k counting matrix of a digraph, G , is $(A_G)^k$, for all $k \in \mathbb{N}$.

In other words, you can determine the number of length k walks between any pair of vertices simply by computing the k th power of the adjacency matrix!

That may seem amazing, but the proof uncovers this simple relationship between matrix multiplication and numbers of walks.

Proof of Theorem 9.3.2. Any length- $(k + m)$ walk between vertices u and v begins with a length- k walk starting at u and ending at some vertex, w , followed by a length- m walk starting at w and ending at v . So the number of length- $(k + m)$ walks from u to v that go through w at the k th step equals the number C_{uw} of length- k walks from u to w , times the number D_{wv} of length- m walks from w to v . We can get the total number of length- $(k + m)$ walks from u to v by summing, over all possible vertices w , the number of such walks that go through w at the k th step. In other words,

$$\text{\#length-}(k + m) \text{ walks from } u \text{ to } v = \sum_{w \in V(G)} C_{uw} \cdot D_{wv} \quad (9.5)$$

But the right hand side of (9.5) is precisely the definition of $(CD)_{uv}$. Thus, CD is indeed the length- $(k + m)$ walk counting matrix. ■

9.3.1 Shortest Paths

The relation between powers of the adjacency matrix and numbers of walks is cool (to us math nerds at least), but a much more important problem is finding shortest paths between pairs of nodes. For example, when you drive home for vacation, you generally want to take the shortest-time route.

One simple way to find the lengths of all the shortest paths in an n -vertex graph, G , is to compute the successive powers of A_G one by one up to the $n - 1$ st, watching for the first power at which each entry becomes positive. That’s because Theorem 9.3.2 implies that the length of the shortest path, if any, between u and v , that is, the distance from u to v , will be the smallest value k for which $(A_G)_{uv}^k$ is nonzero, and if there is a shortest path, its length will be $\leq n - 1$. Refinements of this idea lead to methods that find shortest paths in reasonably efficient ways. The methods apply as well to weighted graphs, where edges are labelled with weights or costs and the objective is to find least weight, cheapest paths. These refinements are typically covered in introductory algorithm courses, and we won’t go into them here any further.

9.4 Walk Relations

A basic question about a digraph is whether there is a path from one particular vertex to another. So for any digraph, G , we are interested in a binary relation, G^* , called the *walk relation* on $V(G)$ where

$$u G^* v ::= \text{there is a walk in } G \text{ from } u \text{ to } v. \quad (9.6)$$

Similarly, there is a *positive walk relation*

$$u G^+ v ::= \text{there is a positive length walk in } G \text{ from } u \text{ to } v. \quad (9.7)$$

Since merging a walk from u to v with a walk from v to w gives a walk from u to w , both walk relations have a relational property called *transitivity*:

Definition 9.4.1. A binary relation, R , on a set, A , is *transitive* iff

$$(a R b \text{ AND } b R c) \text{ IMPLIES } a R c$$

for every $a, b, c \in A$.

Since there is a length-0 walk from any vertex to itself, the walk relation has another relational property called *reflexivity*:

Definition 9.4.2. A binary relation, R , on a set, A , is *reflexive* iff $a R a$ for all $a \in A$.

9.4.1 Composition of Relations

There is a simple way to extend composition of functions to composition of relations, and this gives another way to talk about walks and paths in digraphs.

Definition 9.4.3. Let $R : B \rightarrow C$ and $S : A \rightarrow B$ be binary relations. Then the composition of R with S is the binary relation $(R \circ S) : A \rightarrow C$ defined by the rule

$$a (R \circ S) c ::= \exists b \in B. (a S b) \text{ AND } (b R c). \quad (9.8)$$

This agrees with the Definition 4.3.1 of composition in the special case when R and S are functions.²

²The reversal of the order of R and S in (9.8) is not a typo. This is so that relational composition generalizes function composition. The value of function f composed with function g at an argument, x , is $f(g(x))$. So in the composition, $f \circ g$, the function g is applied first.

Remembering that a digraph is a binary relation on its vertices, it makes sense to compose a digraph G with itself. Then if we let G^n denote the composition of G with itself n times, it's easy to check (see Problem 9.9) that G^n is the *length- n walk relation*:

$$a G^n b \quad \text{iff} \quad \text{there is a length-}n \text{ walk in } G \text{ from } a \text{ to } b.$$

This even works for $n = 0$, with the usual convention that G^0 is the identity relation $\text{Id}_{V(G)}$ on the set of vertices.³ Since there is a walk iff there is a path, and every path is of length at most $|V(G)| - 1$, we now have⁴

$$G^* = G^0 \cup G^1 \cup G^2 \cup \dots \cup G^{|V(G)|-1} = (G \cup G^0)^{|V(G)|-1}. \quad (9.9)$$

The final equality points to the use of repeated squaring as a way to compute G^* with $\log n$ rather than $n - 1$ compositions of relations.

9.5 Directed Acyclic Graphs & Partial Orders

Some of the prerequisites of MIT computer science subjects are shown in Figure 9.6. An edge going from subject s to subject t indicates that s is listed in the catalogue as a direct prerequisite of t . Of course, in order to take subject t , you not only have to take subject s first, but you also have to take all the prerequisites of s , as well as any prerequisites of these prerequisites, and so on. We can state this precisely in terms of the positive walk relation: if D is the direct prerequisite relation on subjects, then subject u has to be completed before taking subject v iff $u D^+ v$.

It would clearly have a dire effect on the time it takes to graduate if this direct prerequisite graph had a positive length cycle :-). So the direct prerequisite graph among subjects had better be *acyclic*:

Definition 9.5.1. A *directed acyclic graph (DAG)* is a directed graph with no positive length cycles.

³The identity relation, Id_A , on a set, A , is the equality relation:

$$a \text{Id}_A b \quad \text{iff} \quad a = b,$$

for $a, b \in A$.

⁴Equation (9.9) involves a harmless abuse of notation: we should have written

$$\text{graph}(G^*) = \text{graph}(G^0) \cup \text{graph}(G^1) \dots$$

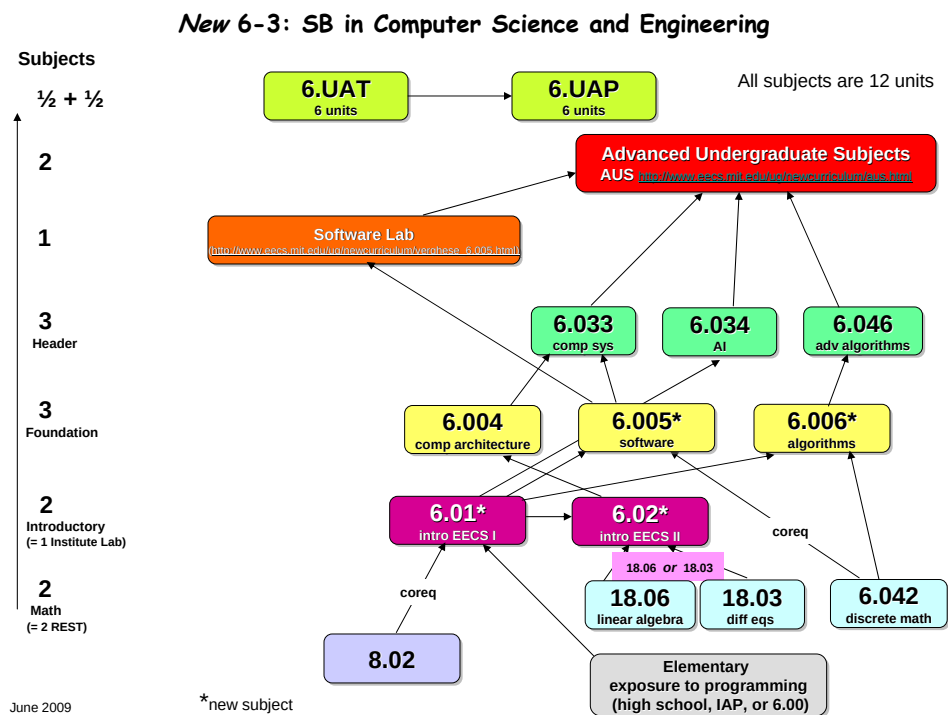


Figure 9.6 Subject prerequisites for MIT Computer Science (6-3) Majors.

DAG's come up constantly because, among other things, they model task scheduling problems, where nodes represent tasks to be completed and arrows indicate which tasks must be completed before others can begin. They have particular importance in computer science because, besides modeling task scheduling problems, they capture key concepts used, for example, in analyzing concurrency control; we'll expand on this in Section 9.10.

The relationship between walks and paths extends to closed walks and cycles.

Lemma 9.5.2. *The shortest positive length closed walk through a vertex is a positive length cycle through that vertex.*

The proof is essentially the same as for Theorem 9.2.3 (see Problem 9.7). This implies that a graph D is a DAG iff it has no positive length walk from any vertex to itself. This relational property of D^+ is called *irreflexivity*.

Definition 9.5.3. A binary relation, R , on a set, A , is *irreflexive* iff

$$\text{NOT}(a R a)$$

for all $a \in A$.

So we have

Lemma 9.5.4. *R is a DAG iff R^+ is irreflexive.*

Definition 9.5.5. A relation that is transitive and irreflexive is called a *strict partial order*.

Since we know that the positive walk relation is transitive, we have

Lemma 9.5.6. *If D is a DAG, then D^+ is a strict partial order.*

The transitivity property of a relation says that where there's a length two walk, there is an edge. This implies by induction that where there is a walk of any positive length, there is an edge (see Problem 9.8), namely:

Lemma 9.5.7. *If a binary relation R is transitive, then $R^+ = R$.*

Corollary 9.5.8. *If R is a strict partial order, then R is a DAG.*

Proof. If vertex a is on a positive length cycle in the graph of R , then $a R^+ a$ holds by definition, which in particular implies that R^+ is not irreflexive. This means that if R^+ is irreflexive, then R must be a DAG.

But if R is a strict partial order, then by definition it is irreflexive and by Lemma 9.5.7 $R^+ = R$, so R^+ is indeed irreflexive. ■

To summarize, we have

Theorem 9.5.9. *A relation is a strict partial order iff it is the positive walk relation of a DAG.*

Another consequence of Lemma 9.5.2 is that if a graph is a DAG, it cannot have two vertices with positive length walks in both directions between them. This relational property of a positive walk relation is called *asymmetry*.

Definition 9.5.10. A binary relation, R , on a set, A , is *asymmetric* iff

$$a R b \text{ IMPLIES NOT}(b R a)$$

for all $a, b \in A$.

That is, Lemma 9.5.2 implies

Corollary 9.5.11. *R is a DAG iff R^+ is asymmetric.*

And immediately from Corollary 9.5.11 and Theorem 9.5.9 we get

Corollary 9.5.12. *R is a strict partial order iff it is transitive and asymmetric.⁵*

A strict partial order may be the positive walk relation of different DAG's. This raises the question of finding a DAG with the *smallest* number of edges that determines a given strict partial order. For *finite* strict partial orders, the smallest such DAG turns out to be unique and easy to find (see Problem 9.3).

9.6 Weak Partial Orders

Partial orders come up in many situations which on the face of it have nothing to do with digraphs. For example, the less-than order, $<$, on numbers is a partial order:

- if $x < y$ and $y < z$ then $x < z$, so less-than is transitive, and
- if $x < y$ then $y \not< x$, so less-than is asymmetric.

The proper containment relation $\subset$ is also a partial order:

- if $A \subset B$ and $B \subset C$ then $A \subset C$, so containment is transitive, and
- $A \not\subset A$, so proper containment is irreflexive.

⁵Some texts use this Corollary to define strict partial orders.

The less-than-or-equal relation, $\leq$, is at least as familiar as the less-than strict partial order, and the ordinary containment relation, $\subseteq$, is even more common than the proper containment relation. These are examples of *weak partial orders*, which are just strict partial orders with the additional condition that every element is related to itself. To state this precisely, we have to relax the asymmetry property so it does not apply when a vertex is compared to itself; this relaxed property is called *antisymmetry*:

Definition 9.6.1. A binary relation, R , on a set A , is *antisymmetric* iff

$$a R b \text{ IMPLIES NOT}(b R a)$$

for all $a \neq b \in A$.

Now we can give an axiomatic definition of weak partial orders that parallels the definition of strict partial orders.⁶

Definition 9.6.2. A binary relation on a set is a *weak partial order* iff it is transitive, reflexive, and antisymmetric.

The following lemma gives another characterizations of weak partial orders that follows directly from this definition.

Lemma 9.6.3. A relation R on a set, A , is a *weak partial order* iff there is a *strict partial order*, S , on A such that

$$a R b \text{ iff } (a S b \text{ OR } a = b),$$

for all $a, b \in A$.

Since a length zero walk goes from a vertex to itself, this lemma combined with Theorem 9.5.9 yields:

Corollary 9.6.4. A relation is a *weak partial order* iff it is the walk relation of a DAG.

For weak partial orders in general, we often write an ordering-style symbol like $\leq$ or $\sqsubseteq$ instead of a letter symbol like R .⁷ Likewise, we generally use $<$ or $\sqsubset$ to indicate a strict partial order.

Two more examples of partial orders are worth mentioning:

⁶Some authors define partial orders to be what we call weak partial orders, but we’ll use the phrase “partial order” to mean either a weak or strict one.

⁷General relations are usually denoted by a letter like R instead of a cryptic squiggly symbol, so $\leq$ is kind of like the musical performer/composer Prince, who redefined the spelling of his name to be his own squiggly symbol. A few years ago he gave up and went back to the spelling “Prince.”

Example 9.6.5. Let A be some family of sets and define $a R b$ iff $a \supset b$. Then R is a strict partial order.

For integers, m, n we write $m \mid n$ to mean that m divides n , namely, there is an integer, k , such that $n = km$.

Example 9.6.6. The divides relation is a weak partial order on the nonnegative integers.

9.7 Representing Partial Orders by Set Containment

Axioms can be a great way to abstract and reason about important properties of objects, but it helps to have a clear picture of the things that satisfy the axioms. DAG's provide one way to picture partial orders, but it also can help to picture them in terms of other familiar mathematical objects. In this section we'll show that every partial order can be pictured as a collection of sets related by containment. That is, every partial order has the “same shape” as such a collection. The technical word for “same shape” is “isomorphic.”

Definition 9.7.1. A binary relation, R , on a set, A , is *isomorphic* to a relation, S , on a set B iff there is a relation-preserving bijection from A to B . That is, there is a bijection $f : A \rightarrow B$, such that for all $a, a' \in A$,

$$a R a' \quad \text{iff} \quad f(a) S f(a').$$

To picture a partial order, $\leq$, on a set, A , as a collection of sets, we simply represent each element A by the set of elements that are $\leq$ to that element, that is,

$$a \longleftrightarrow \{b \in A \mid b \leq a\}.$$

For example, if $\leq$ is the divisibility relation on the set of integers, $\{1, 3, 4, 6, 8, 12\}$, then we represent each of these integers by the set of integers in A that divides it. So

$$1 \longleftrightarrow \{1\}$$

$$3 \longleftrightarrow \{1, 3\}$$

$$4 \longleftrightarrow \{1, 4\}$$

$$6 \longleftrightarrow \{1, 3, 6\}$$

$$8 \longleftrightarrow \{1, 4, 8\}$$

$$12 \longleftrightarrow \{1, 3, 4, 6, 12\}$$

So, the fact that $3 \mid 12$ corresponds to the fact that $\{1, 3\} \subseteq \{1, 3, 4, 6, 12\}$.

In this way we have completely captured the weak partial order $\preceq$ by the subset relation on the corresponding sets. Formally, we have

Lemma 9.7.2. *Let $\preceq$ be a weak partial order on a set, A . Then $\preceq$ is isomorphic to the subset relation, $\subseteq$, on the collection of inverse images under the $\preceq$ relation of elements $a \in A$.*

We leave the proof to Problem 9.17. Essentially the same construction shows that strict partial orders can be represented by sets under the proper subset relation, $\subset$ (Problem 9.18). To summarize:

Theorem 9.7.3. *Every weak partial order, $\preceq$, is isomorphic to the subset relation, $\subseteq$, on a collection of sets.*

Every strict partial order, $\prec$, is isomorphic to the proper subset relation, $\subset$, on a collection of sets.

9.8 Path-Total Orders

The familiar order relations on numbers have an important additional property: given two different numbers, one will be bigger than the other. Partial orders with this property are said to be *path-total orders*.⁸

Definition 9.8.1. Let R be a binary relation on a set, A , and let a, b be elements of A . Then a and b are *comparable* with respect to R iff $[a R b \text{ OR } b R a]$. A partial order for which every two different elements are comparable is called a *path-total order*.

So $<$ and $\leq$ are path-total orders on $\mathbb{R}$. On the other hand, the subset relation is *not* path-total, since, for example, any two different finite sets of the same size will be incomparable under $\subseteq$. The prerequisite relation on Course 6 required subjects is also not path-total because, for example, neither 8.01 nor 6.042 is a prerequisite of the other.

The name path-total is based on the following

⁸Path-total partial orders are conventionally just called “total.” But this terminology conflicts with the definition of “total relation,” and it regularly confuses students. So we chose the terminology “path-total” to avoid the confusion. Some texts use *linear orders* as the name for path-total orders.

Being a path-total partial order is a much stronger condition than being a partial order that is a total relation. For example, any weak partial order such as $\subseteq$ is a total relation but generally won’t be path-total.

Lemma 9.8.2. *For any finite, nonempty set of vertices from a path-total digraph, there is a directed path going through exactly these vertices. In fact, if the digraph is a DAG, the directed path is unique.*

Lemma 9.8.2 is easy to prove by induction on the size of the set of vertices. The proof is given in Problem 9.4.

9.9 Product Orders

Taking the product of two relations is a useful way to construct new relations from old ones.

Definition 9.9.1. The product, $R_1 \times R_2$, of relations R_1 and R_2 is defined to be the relation with

$$\begin{aligned} \text{domain}(R_1 \times R_2) &::= \text{domain}(R_1) \times \text{domain}(R_2), \\ \text{codomain}(R_1 \times R_2) &::= \text{codomain}(R_1) \times \text{codomain}(R_2), \\ (a_1, a_2) (R_1 \times R_2) (b_1, b_2) &\text{ iff } [a_1 R_1 b_1 \text{ and } a_2 R_2 b_2]. \end{aligned}$$

Example 9.9.2. Define a relation, Y , on age-height pairs of being younger *and* shorter. This is the relation on the set of pairs (y, h) where y is a nonnegative integer ≤ 2400 which we interpret as an age in months, and h is a nonnegative integer ≤ 120 describing height in inches. We define Y by the rule

$$(y_1, h_1) Y (y_2, h_2) \text{ iff } y_1 \leq y_2 \text{ AND } h_1 \leq h_2.$$

That is, Y is the product of the $\leq$ -relation on ages and the $\leq$ -relation on heights.

It follows directly from the definitions that products preserve the properties of transitivity, reflexivity, irreflexivity, and antisymmetry, as shown in Problem 9.27. That is, if R_1 and R_2 both have one of these properties, then so does $R_1 \times R_2$. This implies that if R_1 and R_2 are both partial orders, then so is $R_1 \times R_2$.

On the other hand, the property of being a path-total order is not preserved. For example, the age-height relation Y is the product of two path-total orders, but it is not path-total: the age 240 months, height 68 inches pair, $(240, 68)$, and the pair $(228, 72)$ are incomparable under Y .

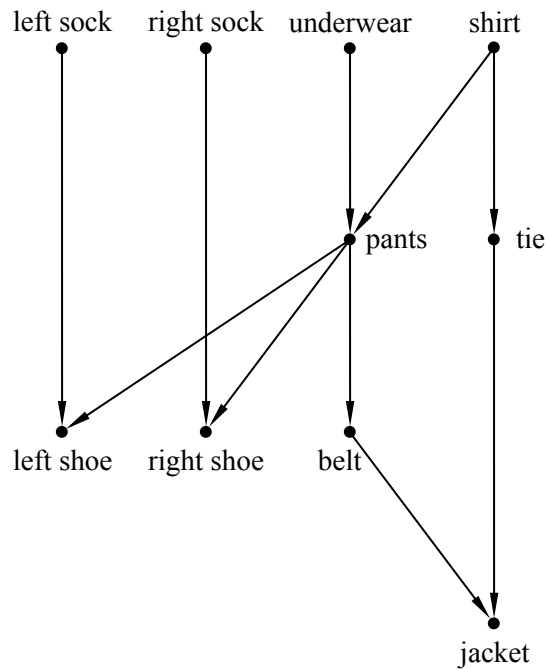


Figure 9.7 DAG describing which clothing items have to be put on before others.

9.10 Scheduling

Scheduling problems are a common source of partial orders: there is a set, A , of tasks and a set of constraints specifying that starting a certain task depends on other tasks being completed beforehand. We can picture the constraints by drawing labelled boxes corresponding to different tasks, with an arrow from one box to another if the first box corresponds to a task that must be completed before starting the second one.

For example, the DAG for in Figure 9.7 describes how a guy might get dressed for a formal occasion. The vertices correspond to garments and the edges specify which garments have to be put on before others are.

When we have a partial order like this on the order in which tasks can be performed, it can be useful to have an order in which to perform all the tasks, one at a time, while respecting the dependency constraints. This amounts to finding a path-total order that is consistent with the partial order. This task of finding a path-total ordering that is consistent with a partial order is known as *topological sorting*.

underwear	left sock
shirt	shirt
belt	tie
pants	underwear
tie	right sock
jacket	pants
left sock	right shoe
right sock	belt
left shoe	jacket
right shoe	left shoe
(a)	(b)

Figure 9.8 Two possible topological sorts of the partial order described in Figure 9.7. In each case, the elements are listed so that $x \preceq y$ iff x is above y in the list.

Definition 9.10.1. A *topological sort* of a partial order, $\prec$, on a set, A , is a path-total ordering, $\sqsubset$, on A such that

$$a \prec b \text{ IMPLIES } a \sqsubset b.$$

There are several path-total orders that are consistent with the partial order shown in Figure 9.7. We have shown two of them in list form in Figure 9.8. Each such list is a topological sort for the partial order in Figure 9.7. In what follows, we will prove that every *finite* partial order has a topological sort. You can think of this as a mathematical proof that you *can* get dressed in the morning (and then show up for math lecture).

Topological sorts for partial orders on finite sets are easy to construct by starting from *minimal* elements:

Definition 9.10.2. Let $\preceq$ be a partial order on a set, A . An element $a_0 \in A$ is *minimum* iff it is $\preceq$ every other element of A , that is, $a_0 \preceq b$ for all $b \neq a_0$.

The element a_0 is *minimal* iff no other element is $\preceq a_0$, that is, NOT($b \preceq a_0$) for all $b \neq a_0$.

There are corresponding definitions for *maximum* and *maximal*. Alternatively, a maximum(al) element for a relation, R , could be defined as a minimum(al) element for R^{-1} .

In a path-total order, minimum and minimal elements are the same thing. But a partial order may have no minimum element but lots of minimal elements. There

are four minimal elements in the clothes example: leftsock, rightsock, underwear, and shirt.

To construct a path-total ordering for getting dressed, we pick one of these minimal elements, say shirt. Next we pick a minimal element among the remaining ones. For example, once we have removed shirt, tie becomes minimal. We continue in this way removing successive minimal elements until all elements have been picked. The sequence of elements in the order they were picked will be a topological sort. This is how the topological sort above for getting dressed was constructed.

So our construction shows:

Theorem 9.10.3. *Every partial order on a finite set has a topological sort.*

There are many other ways of constructing topological sorts. For example, instead of starting “from the bottom” with minimal elements, we could build a path-total ordering starting *anywhere* and simply keep putting additional elements into the path-total order wherever they will fit. In fact, the domain of the partial order need not even be finite: we won’t prove it, but *all* partial orders, even infinite ones, have topological sorts.

9.10.1 Parallel Task Scheduling

For a partial order of task dependencies, topological sorting provides a way to execute tasks one after another while respecting the dependencies. But what if we have the ability to execute more than one task at the same time? For example, say tasks are programs, the partial order indicates data dependence, and we have a parallel machine with lots of processors instead of a sequential machine with only one. How should we schedule the tasks? Our goal should be to minimize the total *time* to complete all the tasks. For simplicity, let’s say all the tasks take the same amount of time and all the processors are identical.

So, given a finite partially ordered set of tasks, how long does it take to do them all, in an optimal parallel schedule? We can also use partial order concepts to analyze this problem.

In the first unit of time, we should do all minimal items, so we would put on our left sock, our right sock, our underwear, and our shirt.⁹ In the second unit of time, we should put on our pants and our tie. Note that we cannot put on our left or right shoe yet, since we have not yet put on our pants. In the third unit of time, we should

⁹Yes, we know that you can’t actually put on both socks at once, but imagine you are being dressed by a bunch of robot processors and you are in a big hurry. Still not working for you? Ok, forget about the clothes and imagine they are programs with the precedence constraints shown in Figure 9.7.

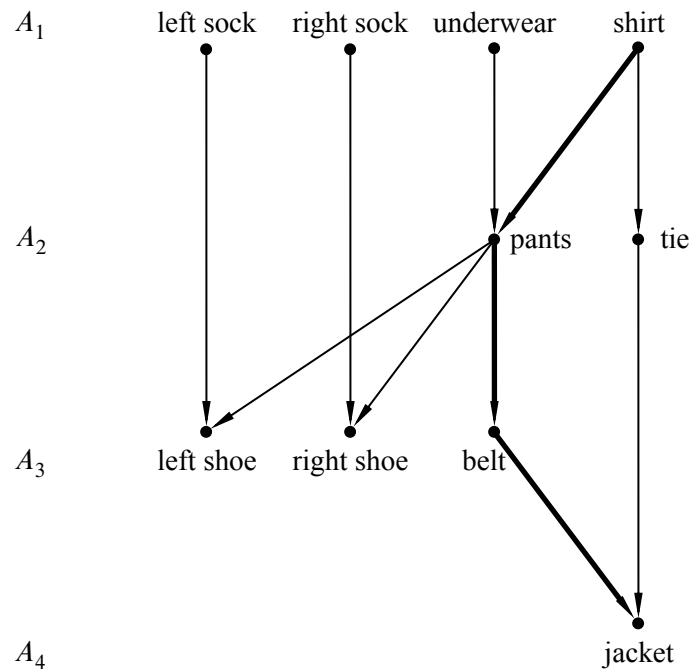


Figure 9.9 A parallel schedule for the tasks-in-getting-dressed partial order in Figure 9.7. The tasks in A_i can be performed in step i for $1 \leq i \leq 4$. A chain of length 4 (the critical path in this example) is shown with bold edges.

put on our left shoe, our right shoe, and our belt. Finally, in the last unit of time, we can put on our jacket. This schedule is illustrated in Figure 9.9.

The total time to do these tasks is 4 units. We cannot do better than 4 units of time because there is a sequence of 4 tasks, each needing to be done before the next, of length 4. For example, we must put on our shirt before our pants, our pants before our belt, and our belt before our jacket. Such a sequence of items is known as a *chain*.

Definition 9.10.4. A *chain* in a partial order is a set of elements such that any two different elements in the set are comparable. A chain is said to *end at* its maximum element.

Thus, the time it takes to schedule tasks, even with an unlimited number of processors, is at least the length of the longest chain. Indeed, if we used less time, then two items from a longest chain would have to be done at the same time, which contradicts the precedence constraints. For this reason, a longest chain is also known as a *critical path*. For example, Figure 9.9 shows the critical path for the getting-

dressed partial order.

In this example, we were in fact able to schedule all the tasks in t steps, where t is the length of the longest chain. The really nice thing about partial orders is that this is always possible! In other words, for any partial order, there is a legal parallel schedule that runs in t steps, where t is the length of the longest chain.

In general, a *schedule* for performing tasks specifies which tasks to do at successive steps. Every task, a , has to be scheduled at some step, and all the tasks that have to be completed before task a must be scheduled for an earlier step.

Definition 9.10.5. A *partition* of a set A is a set of nonempty subsets of A called the *blocks*¹⁰ of the partition, such that

- every element of A is in some block, and
- if B and B' are different blocks, then $B \cap B' = \emptyset$.

For example, one possible partition of the set $\{a, b, c, d, e\}$ into three blocks is

$$\{a, c\} \quad \{b, e\} \quad \{d\}.$$

Definition 9.10.6. A *parallel schedule* for a strict partial order, $<$, on a set, A , is a partition of A into blocks $A_0, A_1, \dots$, such that for all $a, b \in A, k \in \mathbb{N}$,

$$[a \in A_k \text{ AND } b < a] \quad \text{IMPLIES} \quad b \in A_j \text{ for some } j < k.$$

The block A_k is called the set of elements *scheduled at step k* , and the *length* of the schedule is the number of blocks in the partition. The maximum number of elements scheduled at any step is called the *number of processors* required by the schedule.

In general, the earliest step at which an element a can ever be scheduled must be at least as large as any chain that ends at a . A *largest* chain ending at a is called a *critical path* to a , and the size of the critical path is called the *depth* of a . So in any possible parallel schedule, it takes at least $\text{depth}(a)$ steps to complete task a .

There is a very simple schedule that completes every task in this minimum number of steps. Just use a “greedy” strategy of performing tasks as soon as possible. Namely, schedule all the elements of depth k at step k . That’s how we found the schedule for getting dressed given above.

Theorem 9.10.7. Let $<$ be a strict partial order on a set, A . A minimum length schedule for $<$ consists of the sets $A_0, A_1, \dots$, where

$$A_k ::= \{a \mid \text{depth}(a) = k\}.$$

¹⁰We think it would be nicer to call them the *parts* of the partition, but “blocks” is the standard terminology.

We’ll leave to Problem 9.35 the proof that the sets A_k are a parallel schedule according to Definition 9.10.6.

The minimum number of steps needed to schedule a partial order, $\prec$, is called the *parallel time* required by $\prec$, and a largest possible chain in $\prec$ is called a *critical path* for $\prec$. So we can summarize the story above by this way: with an unlimited number of processors, the parallel time to complete all tasks is simply the size of a critical path:

Corollary 9.10.8. *Parallel time = length of critical path.*

Things get a little more interesting when the number of processors is bounded (see Problem 9.37).

9.10.2 Dilworth’s Lemma

Definition 9.10.9. An *antichain* in a partial order is a set of elements such that any two elements in the set are incomparable.

Our conclusions about scheduling also tell us something about antichains.

Corollary 9.10.10. *If the largest chain in a partial order on a set, A , is of size t , then A can be partitioned into t antichains.*

Proof. Let the antichains be the sets $A_k ::= \{a \mid \text{depth}(a) = k\}$. It is an easy exercise to verify that each A_k is an antichain (Problem 9.35). ■

Corollary 9.10.10 implies a famous result¹¹ about partially ordered sets:

Lemma 9.10.11 (Dilworth). *For all $t > 0$, every partially ordered set with n elements must have either a chain of size greater than t or an antichain of size at least n/t .*

Proof. Assume there is no chain of size greater than t , that is, the largest chain is of size $\leq t$. Then by Corollary 9.10.10, the n elements can be partitioned into t or fewer antichains. Let ℓ be the size of the largest antichain. Since every element belongs to exactly one antichain, and there are at most t antichains, there can’t be more than ℓt elements, namely, $\ell t \geq n$. So there is an antichain with at least $\ell \geq n/t$ elements. ■

Corollary 9.10.12. *Every partially ordered set with n elements has a chain of size greater than $\sqrt{n}$ or an antichain of size at least $\sqrt{n}$.*

¹¹Lemma 9.10.11 also follows from a more general result known as Dilworth’s Theorem which we will not discuss.

Proof. Set $t = \sqrt{n}$ in Lemma 9.10.11. ■

Example 9.10.13. In the dressing partially ordered set, $n = 10$.

Try $t = 3$. There is a chain of size 4.

Try $t = 4$. There is no chain of size 5, but there is an antichain of size $4 \geq 10/4$.

Example 9.10.14. Suppose we have a class of 101 students. Then using the product partial order, Y , from Example 9.9.2, we can apply Dilworth’s Lemma to conclude that there is a chain of 11 students who get taller as they get older, or an antichain of 11 students who get taller as they get younger, which makes for an amusing in-class demo.

9.11 Equivalence Relations

A relation is an *equivalence relation* if it is reflexive, symmetric, and transitive. Congruence modulo n is an excellent example of an equivalence relation:

- It is reflexive because $x \equiv x \pmod{n}$.
- It is symmetric because $x \equiv y \pmod{n}$ implies $y \equiv x \pmod{n}$.
- It is transitive because $x \equiv y \pmod{n}$ and $y \equiv z \pmod{n}$ imply that $x \equiv z \pmod{n}$.

There is an even more well-known example of an equivalence relation: equality itself. Thus, an equivalence relation is a relation that shares some key properties with “=”.

9.11.1 Equivalence Classes

Equivalence relations are closely related to partitions because the images of elements under an equivalence relation form the blocks of a partition.

Definition 9.11.1. Given an equivalence relation $R : A \rightarrow A$, the *equivalence class*, $[a]_R$, of an element $a \in A$ is the set of all elements of A related to a by R . Namely,

$$[a]_R ::= \{x \in A \mid a R x\}.$$

For example, suppose that $A = \mathbb{Z}$ and $a R b$ means that $a \equiv b \pmod{5}$. Then

$$[7]_R = \{\dots, -3, 2, 7, 12, 22, \dots\}.$$

Notice that 7, 12, 17, etc., all have the same equivalence class; that is, $[7]_R = [12]_R = [17]_R = \dots$.

There is an exact correspondence between equivalence relations on A and partitions of A . Namely, given on one hand any partition of a set, then being in the same block is obviously an equivalence relation. On the other hand we have:

Theorem 9.11.2. *The equivalence classes of an equivalence relation on a set A form a partition of A .*

We’ll leave the proof of Theorem 9.11.2 as an easy exercise in axiomatic reasoning (see Problem 9.40), but let’s look at an example. The congruent-mod-5 relation partitions the integers into five equivalence classes:

$$\begin{aligned} &\{\dots, -5, 0, 5, 10, 15, 20, \dots\} \\ &\{\dots, -4, 1, 6, 11, 16, 21, \dots\} \\ &\{\dots, -3, 2, 7, 12, 17, 22, \dots\} \\ &\{\dots, -2, 3, 8, 13, 18, 23, \dots\} \\ &\{\dots, -1, 4, 9, 14, 19, 24, \dots\} \end{aligned}$$

In these terms, $x \equiv y \pmod{5}$ is equivalent to the assertion that x and y are both in the same block of this partition. For example, $6 \equiv 16 \pmod{5}$, because they’re both in the second block, but $2 \not\equiv 9 \pmod{5}$ because 2 is in the third block while 9 is in the last block.

In social terms, if “likes” were an equivalence relation, then everyone would be partitioned into cliques of friends who all like each other and no one else.

9.12 Summary of Relational Properties

A relation $R : A \rightarrow A$ is the same as a digraph with vertices A .

Reflexivity R is *reflexive* when

$$\forall x \in A. x R x.$$

Every vertex in R has a self-loop.

Irreflexivity R is *irreflexive* when

$$\text{NOT}[\exists x \in A. x R x].$$

There are no self-loops in R .

Symmetry R is *symmetric* when

$$\forall x, y \in A. x R y \text{ IMPLIES } y R x.$$

If there is an edge from x to y in R , then there is an edge back from y to x as well.

Asymmetry R is *asymmetric* when

$$\forall x, y \in A. x R y \text{ IMPLIES NOT}(y R x).$$

There is at most one directed edge between any two vertices in R , and there are no self-loops.

Antisymmetry R is *antisymmetric* when

$$\forall x \neq y \in A. x R y \text{ IMPLIES NOT}(y R x).$$

Equivalently,

$$\forall x, y \in A. (x R y \text{ AND } y R x) \text{ IMPLIES } x = y.$$

There is at most one directed edge between any two distinct vertices, but there may be self-loops.

Transitivity R is *transitive* if

$$\forall x, y, z \in A. (x R y \text{ AND } y R z) \text{ IMPLIES } x R z.$$

If there is a positive length path from u to v , then there is an edge from u to v .

Path-Total R is *path-total* when

$$\forall x \neq y \in A. (x R y \text{ OR } y R x)$$

Given any two vertices in R , there is an edge in one direction or the other between them.

For any finite, nonempty set of vertices of R , there is a directed path going through exactly these vertices.

Strict Partial Order R is a *strict partial order* iff R is transitive and irreflexive iff R is transitive and asymmetric iff it is the positive length walk relation of a DAG.

Weak Partial Order R is a *weak partial order* iff R is transitive and anti-symmetric and reflexive iff R is the walk relation of a DAG.

Equivalence Relation R is an *equivalence relation* iff R is reflexive, symmetric and transitive iff R equals the *in-the-same-block*-relation for some partition of $\text{domain}(R)$.

Problems for Section 9.5

Practice Problems

Problem 9.1.

In this DAG (Figure 9.10) for the divisibility relation on $\{1, \dots, 12\}$, there is an upward path from a to b iff $a|b$. If 24 was added as a vertex, what is the minimum number of edges that must be added to the DAG to represent divisibility on $\{1, \dots, 12, 24\}$? What are those edges?

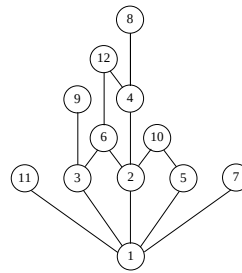


Figure 9.10

Problem 9.2. (a) Why is every strict partial order a DAG?

(b) Give an example of a DAG that is not a strict partial order.

(c) Why is the positive walk relation of a DAG a strict partial order?

Class Problems

Problem 9.3.

If a and b are distinct nodes of a digraph, then a is said to *cover* b if there is an edge from a to b and every path from a to b includes this edge. If a covers b , the edge from a to b is called a *covering edge*.

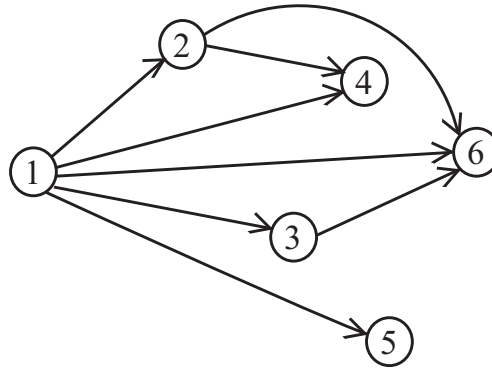


Figure 9.11 DAG with edges not needed in paths

(a) What are the covering edges in the DAG in Figure 9.11?

(b) Let $\text{covering}(D)$ be the subgraph of D consisting of only the covering edges. Suppose D is a finite DAG. Explain why $\text{covering}(D)$ has the same positive walk relation as D .

Hint: Consider *longest* paths between a pair of vertices.

(c) Show that if two DAG's have the same positive walk relation, then they have the same set of covering edges.

(d) Conclude that $\text{covering}(D)$ is the *unique* DAG with the smallest number of edges among all digraphs with the same positive walk relation as D .

The following examples show that the above results don't work in general for digraphs with cycles.

(e) Describe two graphs with vertices $\{1, 2\}$ which have the same set of covering edges, but not the same positive walk relation (*Hint:* Self-loops.)

(f) (i) The *complete digraph* without self-loops on vertices 1, 2, 3 has edges between every two distinct vertices. What are its covering edges?

(ii) What are the covering edges of the graph with vertices 1, 2, 3 and edges $\langle 1 \rightarrow 2 \rangle, \langle 2 \rightarrow 3 \rangle, \langle 3 \rightarrow 1 \rangle$?

(iii) What about their positive walk relations?

Problem 9.4.

In a round-robin tournament, every two distinct players play against each other just once. For a round-robin tournament with no tied games, a record of who beat whom can be described with a *tournament digraph*, where the vertices correspond to players and there is an edge $\langle x \rightarrow y \rangle$ iff x beat y in their game.

A *ranking* is a path that includes all the players. So in a ranking, each player won the game against the next lowest ranked player, but may very well have lost their games against must lower ranked players —whoever does the ranking can have a lot of room to play favorites.

- (a) Give an example of a tournament digraph with more than one ranking.
- (b) Prove that if a tournament digraph is a DAG, then it has at most one ranking.
- (c) Prove that every finite tournament digraph has a ranking.
- (d) Prove that the less-than relation, $>$, on the rational numbers, $\mathbb{Q}$ is a DAG and a tournament graph that has no ranking.

Problem 9.5.

In an n -player *round-robin tournament*, every pair of distinct players compete in a single game. Assume that every game has a winner —there are no ties. The results of such a tournament can then be represented with a *tournament digraph* where the vertices correspond to players and there is an edge $\langle x \rightarrow y \rangle$ iff x beat y in their game.

- (a) Explain why a tournament digraph cannot have cycles of length 1 or 2.
- (b) Is the “beats” relation for a tournament graph always/sometimes/never:
 - asymmetric?
 - reflexive?
 - irreflexive?
 - transitive?

Explain.

- (c) Show that a tournament graph represents a path-total order iff there are no cycles of length 3.

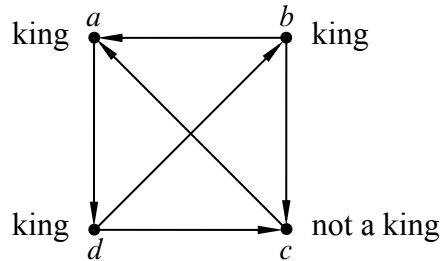


Figure 9.12 A 4-chicken tournament in which chickens a , b , and d are kings.

Problem 9.6.

Suppose that there are n chickens in a farmyard. Chickens are rather aggressive birds that tend to establish dominance in relationships by pecking. (Hence the term “pecking order.”) In particular, for each pair of distinct chickens, either the first pecks the second or the second pecks the first, but not both. We say that chicken u *virtually pecks* chicken v if either:

- Chicken u directly pecks chicken v , or
- Chicken u pecks some other chicken w who in turn pecks chicken v .

A chicken that virtually pecks every other chicken is called a *king chicken*.

We can model this situation with a *chicken digraph* whose vertices are chickens with an edge from chicken u to chicken v precisely when u pecks v . In the graph in Figure 9.12, three of the four chickens are kings. Chicken c is not a king in this example since it does not peck chicken b and it does not peck any chicken that pecks chicken b . Chicken a is a king since it pecks chicken d , who in turn pecks chickens b and c .

(a) Define a 10-chicken graph with a king chicken that has degree 1.

(b) Describe a 5-chicken graph in which every player is a king.

(c) Prove

Theorem (King Chicken Theorem). *The chicken with the largest outdegree in an n -chicken tournament is a king.*

Homework Problems

Problem 9.7. (a) Give an example of a digraph that has a closed walk including two vertices but has no cycle including those vertices.

(b) Prove Lemma 9.5.2:

Lemma. *The shortest positive length closed walk through a vertex is a cycle.*

Problem 9.8.

Prove that if R is a transitive binary relation on a set, A , then $R = R^+$.

Problem 9.9.

Let R be a binary relation on a set A and C^n be the composition of R with itself n times for $n \geq 0$. So $C^0 ::= \text{Id}_A$, and $C^{n+1} ::= R \circ C^n$. Regarding R as a digraph, let R^n denote the length- n walk relation in the digraph R , that is,

$$a R^n b ::= \text{there is a length-}n \text{ walk from } a \text{ to } b \text{ in } R.$$

Prove that

$$R^n = C^n \tag{9.10}$$

for all $n \in \mathbb{N}$.

Problem 9.10.

If R is a binary relation on a set, A , then R^k denotes the relational composition of R with itself k times.

(a) Prove that if R is a relation on a finite set, A , then

$$a (R \cup I_A)^n b \quad \text{iff} \quad \text{there is a path in } R \text{ of length } \leq n \text{ from } a \text{ to } b.$$

(b) Conclude that if A is a finite set, then

$$R^* = (R \cup I_A)^{|A|-1}. \tag{9.11}$$

Problem 9.11.

Prove that the shortest odd-length closed walk through a vertex is an odd-length cycle.

Problem 9.12.

An *Euler tour*¹² of a graph is a closed walk that includes every edge exactly once.

¹²In some other texts, this is called an *Euler circuit*.

Such walks are named after the famous 17th century mathematician Leonhard Euler. (Same Euler as for the constant $e \approx 2.718$ and the totient function ϕ —he did a lot of stuff.)

So how do you tell in general whether a graph has an Euler tour? At first glance this may seem like a daunting problem (the similar sounding problem of finding a cycle that touches every vertex exactly once is one of those million dollar NP-complete problems known as the *Traveling Salesman Problem*) —but it turns out to be easy.

(a) Show that if a graph has an Euler tour, then the in-degree of each vertex equals its out-degree.

A digraph is *weakly connected* if there is a “path” between any two vertices that may follow edges backwards or forwards.¹³ In the remaining parts, we’ll work out the converse: if a graph is weakly connected, and if the in-degree of every vertex equals its out-degree, then the graph has an Euler tour.

A *trail* is a walk in which each edge occurs *at most* once.

(b) Suppose that an trail in a connected graph does not include every edge. Explain why there must be an edge not on the trail that starts or ends at a vertex on the trail.

In the remaining parts, let w be the *longest* trail in the graph.

(c) Show that if w is closed, then it must be an Euler tour.

Hint: part (b)

(d) Explain why all the edges starting at the end of w must be on w .

(e) Show that if the end of w was not closed, then the in-degree of the end would be bigger than its out-degree.

Hint: part (d)

(f) Conclude that if in a finite, weakly connected digraph, the in-degree of every vertex equals its out-degree, then the digraph has an Euler tour.

¹³More precisely, a graph G is weakly connected iff there is a path from any vertex to any other vertex in the graph H with

$$V(H) = V(G), \text{ and}$$

$$E(H) = E(G) \cup \{ \langle v \rightarrow u \rangle \mid \langle u \rightarrow v \rangle \in E(G) \}.$$

In other words $H = G \cup G^{-1}$.

Problem 9.13.

A 3-bit string is a string made up of 3 characters, each a 0 or a 1. Suppose you’d like to write out, in one string, all eight of the 3-bit strings in any convenient order. For example, if you wrote out the 3-bit strings in the usual order starting with 000 001 010..., you could concatenate them together to get a length $3 \cdot 8 = 24$ string that started 000001010....

But you can get a shorter string containing all eight 3-bit strings by starting with 00010.... Now 000 is present as bits 1 through 3, and 001 is present as bits 2 through 4, and 010 is present as bits 3 through 5,

(a) Say a string is *3-good* if it contains every 3-bit string as 3 consecutive bits somewhere in it. Find a 3-good string of length 10, and explain why this is the minimum length for any string that is 3-good.

(b) Explain how any walk that includes every edge in the graph shown in Figure 9.13 determines a string that is 3-good. Find the walk in this graph that determines your good 3-good string from part (a).

(c) Explain why a walk in the graph of Figure 9.13 that includes every edge *exactly once* provides a minimum length 3-good string.

(d) The situation above generalizes to $k \geq 2$. Namely, there is a digraph, B_k , such that $V(B_k) ::= \{0, 1\}^k$, and any walk through B_k that contains every edge exactly once determines a minimum length $(k + 1)$ -good bit-string. What is this minimum length?

Define the transitions of B_k . Verify that the in-degree and out-degree of every vertex is even, and that there is a positive path from any vertex to any other vertex (including itself) of length at most k .¹⁴

Exam Problems

Problem 9.14.

For each of the relations below, indicate whether it is

a *weak partial order* (**WPO**), a *strict partial order* (**SPO**),

and if so, whether it is *path-total* (**Tot**)

If it is neither (**WPO**) nor (**SPO**), indicate whether it is

¹⁴Problem 9.12 shows that if the in-degree of every vertex of a digraph is equal to its out-degree, and there are paths between any two vertices, then there is a closed walk that includes every edge exactly once. So the graph B_k implies that there always is a length- $2^{k+1} + k$ bit-string in which every length- $(k + 1)$ bit-string appears as a substring. Such strings are known as *de Bruijn sequences*.

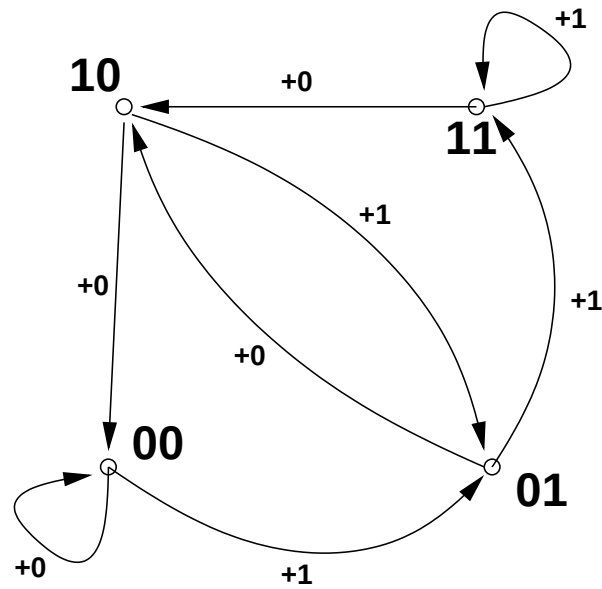


Figure 9.13 The 2-bit graph.

transitive (**Tr**), *symmetric* (**Sym**), *asymmetric* (**Asym**)

- (a) The relation $a = b + 1$ between integers, a, b ,
- (b) The superset relation, $\supseteq$ on the power set of the integers.
- (c) The relation $\text{Ex}[R] < \text{Ex}[S]$ between real-valued random variables R, S .
- (d) The empty relation on the set of rationals.
- (e) The divides relation on the positive powers of 4.

For the next three parts, let f, g be nonnegative functions from the integers to the real numbers.

- (f) The “Big Oh” relation, $f = O(g)$,
- (g) The “Little Oh” relation, $f = o(g)$,
- (h) The “asymptotically equal” relation, $f \sim g$.

Problems for Section 9.7

Class Problems

Problem 9.15.

Direct Prerequisites	Subject
18.01	6.042
18.01	18.02
18.01	18.03
8.01	8.02
8.01	6.01
6.042	6.046
18.02, 18.03, 8.02, 6.01	6.02
6.01, 6.042	6.006
6.01	6.034
6.02	6.004

(a) For the above table of MIT subject prerequisites, draw a diagram showing the subject numbers with a line going down to every subject from each of its (direct) prerequisites.

(b) Give an example of a collection of sets partially ordered by the proper subset relation, $\subset$, that is isomorphic to (“same shape as”) the prerequisite relation among MIT subjects from part (a).

(c) Explain why the empty relation is a strict partial order and describe a collection of sets partially ordered by the proper subset relation that is isomorphic to the empty relation on five elements—that is, the relation under which none of the five elements is related to anything.

(d) Describe a *simple* collection of sets partially ordered by the proper subset relation that is isomorphic to the “properly contains” relation, $\supset$, on $\mathcal{P}\{1, 2, 3, 4\}$.

Problem 9.16.

The proper subset relation, $\subset$, defines a strict partial order on the subsets of $[1, 6]$, that is $\mathcal{P}[1, 6]$.

(a) What is the size of a maximal chain in this partial order? Describe one.

(b) Describe the largest antichain you can find in this partial order.

(c) What are the maximal and minimal elements? Are they maximum and minimum?

(d) Answer the previous part for the $\subset$ partial order on the set $\mathcal{P}\{1, 2, \dots, 6\} - \emptyset$.

Problem 9.17.

This problem asks for a proof of Lemma 9.7.2 showing that every weak partial order can be represented by (is isomorphic to) a collection of sets partially ordered under set inclusion ($\subseteq$). Namely,

Lemma. *Let $\preceq$ be a weak partial order on a set, A . For any element $a \in A$, let*

$$L(a) ::= \{b \in A \mid b \preceq a\},$$

$$\mathcal{L} ::= \{L(a) \mid a \in A\}.$$

Then the function $L() : A \rightarrow \mathcal{L}$ is an isomorphism from the $\preceq$ relation on A , to the subset relation on $\mathcal{L}$.

(a) Prove that the function $L() : A \rightarrow \mathcal{L}$ is a bijection.

(b) Complete the proof by showing that

$$a \preceq b \quad \text{iff} \quad L(a) \subseteq L(b) \tag{9.12}$$

for all $a, b \in A$.

Homework Problems

Problem 9.18.

Every partial order is isomorphic to a collection of sets under the subset relation (see Section 9.7). In particular, if R is a *strict* partial order on a set, A , and $a \in A$, define

$$L(a) ::= \{a\} \cup \{x \in A \mid x R a\}. \tag{9.13}$$

Then

$$a R b \quad \text{iff} \quad L(a) \subset L(b) \tag{9.14}$$

holds for all $a, b \in A$.

(a) Carefully prove statement (9.14), starting from the definitions of strict partial order and the strict subset relation, $\subset$.

(b) Prove that if $L(a) = L(b)$ then $a = b$.

(c) Give an example showing that the conclusion of part (b) would not hold if the definition of $L(a)$ in equation (9.13) had omitted the expression “ $\{a\} \cup$.”

Problems for Section 9.8

Practice Problems

Problem 9.19.

For each of the binary relations below, state whether it is a strict partial order, a weak partial order, or neither. If it is not a partial order, indicate which of the axioms for partial order it violates.

- (a) The superset relation, $\supseteq$ on the power set $\mathcal{P}\{1, 2, 3, 4, 5\}$.
- (b) The relation between any two nonnegative integers, a, b that $a \equiv b \pmod{8}$.
- (c) The relation between propositional formulas, G, H , that $G \text{ IMPLIES } H$ is valid.
- (d) The relation 'beats' on Rock, Paper and Scissor (for those who don't know the game Rock, Paper, Scissors, Rock beats Scissors, Scissors beats Paper and Paper beats Rock).
- (e) The empty relation on the set of real numbers.
- (f) The identity relation on the set of integers.

Problem 9.20. (a) Verify that the divisibility relation on the set of nonnegative integers is a weak partial order.

- (b) What about the divisibility relation on the set of integers?

Problem 9.21.

Prove directly from the definitions (without appealing to DAG properties) that if a binary relation R on a set A is transitive and irreflexive, then it is asymmetric.

Class Problems

Problem 9.22.

Show that the set of nonnegative integers partially ordered under the divides relation. . .

- (a) . . . has a minimum element.
- (b) . . . has a maximum element.

(c) ... has an infinite chain.

(d) ... has an infinite antichain.

(e) What are the minimal elements of divisibility on the integers greater than 1? What are the maximal elements?

Problem 9.23.

How many binary relations are there on the set $\{0, 1\}$?

How many are there that are transitive?, ... asymmetric?, ... reflexive?, ... irreflexive?, ... strict partial orders?, ... weak partial orders?

Hint: There are easier ways to find these numbers than listing all the relations and checking which properties each one has.

Problem 9.24.

Prove that if R is a partial order, then so is R^{-1}

Homework Problems

Problem 9.25.

Let R and S be transitive binary relations on the same set, A . Which of the following new relations must also be transitive? For each part, justify your answer with a brief argument if the new relation is transitive and a counterexample if it is not.

(a) R^{-1}

(b) $R \cap S$

(c) $R \circ R$

(d) $R \circ S$

Exam Problems

Problem 9.26.

(a) For each row in the following table, indicate whether the binary relation, R , on the set, A , is a weak partial order or a path-total order by filling in the appropriate entries with either Y = YES or N = NO. In addition, list the minimal and maximal elements for each relation.

A	a R b	weak p. o.	path-total order	minimal(s)	maximal(s)
$\mathbb{R} - \mathbb{R}^+$	$a \mid b$				
$\mathcal{P}(\{1, 2, 3\})$	$a \subseteq b$				
$\mathbb{N} \cup \{i\}$	$a > b$				

(b) What is the longest *chain* on the subset relation, $\subseteq$, on $\mathcal{P}(\{1, 2, 3\})$? (If there is more than one, provide *one* of them.)

(c) What is the longest *antichain* on the subset relation, $\subseteq$, on $\mathcal{P}(\{1, 2, 3\})$? (If there is more than one, provide *one* of them.)

Problems for Section 9.9

Class Problems

Problem 9.27.

Let R_1, R_2 be binary relations on the same set, A . A relational property is preserved under product, if $R_1 \times R_2$ has the property whenever both R_1 and R_2 have the property.

(a) Verify that each of the following properties are preserved under product.

1. reflexivity,
2. antisymmetry,
3. transitivity.

(b) Verify that if *either* of R_1 or R_2 is irreflexive, then so is $R_1 \times R_2$.

Note that it now follows immediately that if R_1 and R_2 are partial orders and at least one of them is strict, then $R_1 \times R_2$ is a strict partial order.

Problems for Section 9.10

Practice Problems

Problem 9.28.

What is the size of the longest chain that is guaranteed to exist in any partially ordered set of n elements? What about the largest antichain?

Problem 9.29.

Describe a sequence consisting of the integers from 1 to 10,000 in some order so that there is no increasing or decreasing subsequence of size 101.

Problem 9.30.

What is the smallest number of partially ordered tasks for which there can be more than one minimum time schedule, if there are unlimited number of processors? Explain your answer.

Class Problems

Problem 9.31.

The table below lists some prerequisite information for some subjects in the MIT Computer Science program (in 2006). This defines an indirect prerequisite relation that is a DAG with these subjects as vertices.

18.01 $\rightarrow$ 6.042	18.01 $\rightarrow$ 18.02
18.01 $\rightarrow$ 18.03	6.046 $\rightarrow$ 6.840
8.01 $\rightarrow$ 8.02	6.001 $\rightarrow$ 6.034
6.042 $\rightarrow$ 6.046	18.03, 8.02 $\rightarrow$ 6.002
6.001, 6.002 $\rightarrow$ 6.003	6.001, 6.002 $\rightarrow$ 6.004
6.004 $\rightarrow$ 6.033	6.033 $\rightarrow$ 6.857

(a) Explain why exactly six terms are required to finish all these subjects, if you can take as many subjects as you want per term. Using a *greedy* subject selection strategy, you should take as many subjects as possible each term. Exhibit your complete class schedule each term using a greedy strategy.

(b) In the second term of the greedy schedule, you took five subjects including 18.03. Identify a set of five subjects not including 18.03 such that it would be possible to take them in any one term (using some nongreedy schedule). Can you figure out how many such sets there are?

(c) Exhibit a schedule for taking all the courses—but only one per term.

(d) Suppose that you want to take all of the subjects, but can handle only two per term. Exactly how many terms are required to graduate? Explain why.

(e) What if you could take three subjects per term?

Problem 9.32.

A pair of Math for Computer Science Teaching Assistants, Oshani and Oscar, have decided to devote some of their spare time this term to establishing dominion over the entire galaxy. Recognizing this as an ambitious project, they worked out the following table of tasks on the back of Oscar’s copy of the lecture notes.

1. **Devise a logo** and cool imperial theme music - 8 days.
2. **Build a fleet** of Hyperwarp Stardestroyers out of eating paraphernalia swiped from Lobdell - 18 days.
3. **Seize control** of the United Nations - 9 days, after task #1.
4. **Get shots** for Oshani’s cat, Tailspin - 11 days, after task #1.
5. **Open a Starbucks chain** for the army to get their caffeine - 10 days, after task #3.
6. **Train an army** of elite interstellar warriors by dragging people to see *The Phantom Menace* dozens of times - 4 days, after tasks #3, #4, and #5.
7. **Launch the fleet** of Stardestroyers, crush all sentient alien species, and establish a Galactic Empire - 6 days, after tasks #2 and #6.
8. **Defeat Microsoft** - 8 days, after tasks #2 and #6.

We picture this information in Figure 9.14 below by drawing a point for each task, and labelling it with the name and weight of the task. An edge between two points indicates that the task for the higher point must be completed before beginning the task for the lower one.

(a) Give some valid order in which the tasks might be completed.

Oshani and Oscar want to complete all these tasks in the shortest possible time. However, they have agreed on some constraining work rules.

- Only one person can be assigned to a particular task; they can not work together on a single task.
- Once a person is assigned to a task, that person must work exclusively on the assignment until it is completed. So, for example, Oshani cannot work on building a fleet for a few days, run to get shots for Tailspin, and then return to building the fleet.

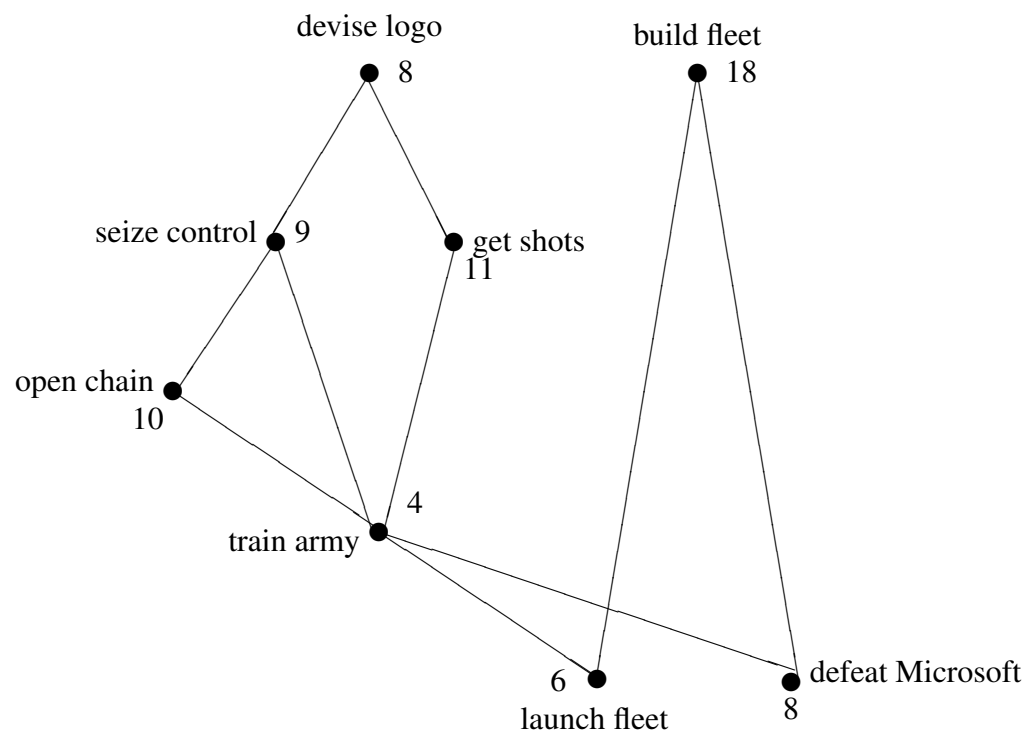


Figure 9.14 Graph representing the task precedence constraints.

(b) Oshani and Oscar want to know how long conquering the galaxy will take. Oscar suggests dividing the total number of days of work by the number of workers, which is two. What lower bound on the time to conquer the galaxy does this give, and why might the actual time required be greater?

(c) Oshani proposes a different method for determining the duration of their project. She suggests looking at the duration of the “critical path”, the most time-consuming sequence of tasks such that each depends on the one before. What lower bound does this give, and why might it also be too low?

(d) What is the minimum number of days that Oshani and Oscar need to conquer the galaxy? No proof is required.

Problem 9.33. (a) What are the maximal and minimal elements, if any, of the power set $\mathcal{P}(\{1, \dots, n\})$, where n is a positive integer, under the *empty relation*?

(b) What are the maximal and minimal elements, if any, of the set, $\mathbb{N}$, of all non-negative integers under divisibility? Is there a minimum or maximum element?

(c) What are the minimal and maximal elements, if any, of the set of integers greater than 1 under divisibility?

(d) Describe a partially ordered set that has no minimal or maximal elements.

(e) Describe a partially ordered set that has a *unique minimal* element, but no minimum element. *Hint:* It will have to be infinite.

Homework Problems

Problem 9.34.

The following procedure can be applied to any digraph, G :

1. Delete an edge that is in a cycle.
2. Delete edge $\langle u \rightarrow v \rangle$ if there is a path from vertex u to vertex v that does not include $\langle u \rightarrow v \rangle$.
3. Add edge $\langle u \rightarrow v \rangle$ if there is no path in either direction between vertex u and vertex v .

Repeat these operations until none of them are applicable.

This procedure can be modeled as a state machine. The start state is G , and the states are all possible digraphs with the same vertices as G .

(a) Let G be the graph with vertices $\{1, 2, 3, 4\}$ and edges

$$\{\langle 1 \rightarrow 2 \rangle, \langle 2 \rightarrow 3 \rangle, \langle 3 \rightarrow 4 \rangle, \langle 3 \rightarrow 2 \rangle, \langle 1 \rightarrow 4 \rangle\}$$

What are the possible final states reachable from G ?

A *line graph* is a graph whose edges are all on one path. All the final graphs in part (a) are line graphs.

(b) Prove that if the procedure terminates with a digraph, H , then H is a line graph with the same vertices as G .

Hint: Show that if H is *not* a line graph, then some operation must be applicable.

(c) Prove that being a DAG is a preserved invariant of the procedure.

(d) Prove that if G is a DAG and the procedure terminates, then the walk relation of the final line graph is a topological sort of G .

Hint: Verify that the predicate

$$P(u, v) ::= \text{there is a directed path from } u \text{ to } v$$

is a preserved invariant of the procedure, for any two vertices u, v of a DAG.

(e) Prove that if G is finite, then the procedure terminates.

Hint: Let s be the number of cycles, e be the number of edges, and p be the number of pairs of vertices with a directed path (in either direction) between them. Note that $p \leq n^2$ where n is the number of vertices of G . Find coefficients a, b, c such that $as + bp + e + c$ is nonnegative integer valued and decreases at each transition.

Problem 9.35.

Let $<$ be a partial order on a set, A , and let

$$A_k ::= \{a \mid \text{depth}(a) = k\}$$

where $k \in \mathbb{N}$.

(a) Prove that $A_0, A_1, \dots$ is a parallel schedule for $<$ according to Definition 9.10.6.

(b) Prove that A_k is an antichain.

Problem 9.36.

Let S be a sequence of n different numbers. A *subsequence* of S is a sequence that can be obtained by deleting elements of S .

For example, if

$$S = (6, 4, 7, 9, 1, 2, 5, 3, 8)$$

Then 647 and 7253 are both subsequences of S (for readability, we have dropped the parentheses and commas in sequences, so 647 abbreviates $(6, 4, 7)$, for example).

An *increasing subsequence* of S is a subsequence of whose successive elements get larger. For example, 1238 is an increasing subsequence of S . Decreasing subsequences are defined similarly; 641 is a decreasing subsequence of S .

(a) List all the maximum length increasing subsequences of S , and all the maximum length decreasing subsequences.

Now let A be the *set* of numbers in S . (So $A = \{1, 2, 3, \dots, 9\}$ for the example above.) There are two straightforward ways to partial-order A . The first is to order its elements numerically, that is, to order A with the $<$ relation. The second is to order the elements by which comes first in S ; call this order $<_S$. So for the example above, we would have

$$6 <_S 4 <_S 7 <_S 9 <_S 1 <_S 2 <_S 5 <_S 3 <_S 8$$

Next, define the partial order $<$ on A defined by the rule

$$a < a' ::= a < a' \text{ and } a <_S a'.$$

(It's not hard to prove that $<$ is strict partial order, but you may assume it.)

(b) Draw a diagram of the partial order, $<$, on A . What are the maximal elements, ... the minimal elements?

(c) Explain the connection between increasing and decreasing subsequences of S , and chains and anti-chains under $<$.

(d) Prove that every sequence, S , of length n has an increasing subsequence of length greater than $\sqrt{n}$ or a decreasing subsequence of length at least $\sqrt{n}$.

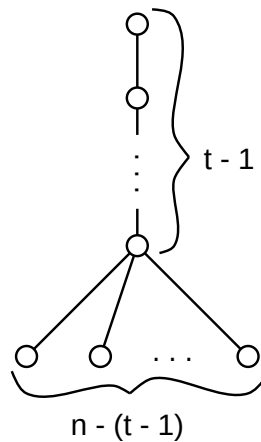
(e) (Optional, tricky) Devise an efficient procedure for finding the longest increasing and the longest decreasing subsequence in any given sequence of integers. (There is a nice one.)

Problem 9.37.

We want to schedule n tasks with prerequisite constraints among the tasks defined by a DAG.

(a) Explain why any schedule that requires only p processors must take time at least $\lceil n/p \rceil$.

(b) Let $D_{n,t}$ be the DAG with n elements that consists of a chain of $t - 1$ elements, with the bottom element in the chain being a prerequisite of all the remaining elements as in the following figure:



What is the minimum time schedule for $D_{n,t}$? Explain why it is unique. How many processors does it require?

(c) Write a simple formula, $M(n, t, p)$, for the minimum time of a p -processor schedule to complete $D_{n,t}$.

(d) Show that *every* partial order with n vertices and maximum chain size, t , has a p -processor schedule that runs in time $M(n, t, p)$.

Hint: Induction on t .

Problems for Section 9.11

Practice Problems

Problem 9.38.

For each of the following relations, decide whether it is reflexive, whether it is symmetric, whether it is transitive, and whether it is an equivalence relation.

- (a) $\{(a, b) \mid a \text{ and } b \text{ are the same age}\}$
- (b) $\{(a, b) \mid a \text{ and } b \text{ have the same parents}\}$
- (c) $\{(a, b) \mid a \text{ and } b \text{ speak a common language}\}$

Problem 9.39.

For each of the binary relations below, state whether it is a strict partial order, a weak partial order, an equivalence relation or none of these. If it is a partial order, state whether it is a path-total order. If it is none, indicate which of the axioms for partial order and equivalence relations it violates.

- (a) The superset relation, $\supseteq$ on the power set $\mathcal{P}\{1, 2, 3, 4, 5\}$.
- (b) The relation between any two nonnegative integers, a, b that $a \equiv b \pmod{8}$.
- (c) The relation between propositional formulas, G, H , that $G \text{ IMPLIES } H$ is valid.
- (d) The relation between propositional formulas, G, H , that $G \text{ IFF } H$ is valid.
- (e) The relation 'beats' on Rock, Paper and Scissor (for those who don't know the game Rock, Paper, Scissors, Rock beats Scissors, Scissors beats Paper and Paper beats Rock).
- (f) The empty relation on the set of real numbers.
- (g) The identity relation on the set of integers.
- (h) The divisibility relation on the integers, $\mathbb{Z}$.

Class Problems

Problem 9.40.

Prove Theorem 9.11.2: The equivalence classes of an equivalence relation form a partition of the domain.

Namely, let R be an equivalence relation on a set, A , and define the equivalence class of an element $a \in A$ to be

$$[a]_R ::= \{b \in A \mid a R b\}.$$

- (a) Prove that every block is nonempty and every element of A is in some block.
- (b) Prove that if $[a]_R \cap [b]_R \neq \emptyset$, then $a R b$.

(c) Suppose that $a R b$ is true. Prove that $[a]_R \subseteq [b]_R$, and conclude that $[a]_R = [b]_R$.

Problem 9.41.

For any total function $f : A \rightarrow B$ define a relation $\equiv_f$ by the rule:

$$a \equiv_f a' \quad \text{iff} \quad f(a) = f(a'). \quad (9.15)$$

(a) Prove that $\equiv_f$ is an equivalence relation on A .

(b) Prove that every equivalence relation, R , on a set, A , is equal to $\equiv_f$ for some total function $f : A \rightarrow B$.

Hint: Let $f(a) ::= [a]_R$ where $[a]_R$ is the equivalence class of R that contains a .

Homework Problems

Problem 9.42.

Let R_1 and R_2 be two equivalence relations on a set, A . Which of the following relations must also be equivalence relations? Prove it.

(a) $R_1 \cap R_2$.

(b) $R_1 \cup R_2$.



10 Communication Networks

Modeling communication networks is an important application of digraphs in computer science. In this such models, vertices represent computers, processors, and switches; edges will represent wires, fiber, or other transmission lines through which data flows. For some communication networks, like the internet, the corresponding graph is enormous and largely chaotic. Highly structured networks, by contrast, find application in telephone switching systems and the communication hardware inside parallel computers. In this chapter, we’ll look at some of the nicest and most commonly used structured networks.

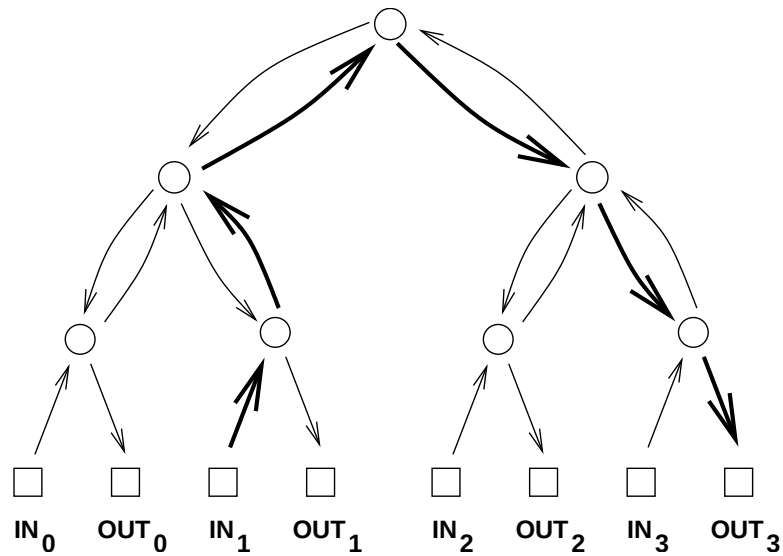
10.1 Complete Binary Tree

Let’s start with a *complete binary tree*. Here is an example with 4 inputs and 4 outputs. The kinds of communication networks we consider aim to transmit packets of data between computers, processors, telephones, or other devices. The term *packet* refers to some roughly fixed-size quantity of data— 256 bytes or 4096 bytes or whatever. In this diagram and many that follow, the squares represent *terminals*, sources and destinations for packets of data. The circles represent *switches*, which direct packets through the network. A switch receives packets on incoming edges and relays them forward along the outgoing edges. Thus, you can imagine a data packet hopping through the network from an input terminal, through a sequence of switches joined by directed edges, to an output terminal.

Recall that there is a unique path between every pair of vertices in a tree. So the natural way to route a packet of data from an input terminal to an output in the complete binary tree is along the corresponding directed path. For example, the route of a packet traveling from input 1 to output 3 is shown in bold.

10.2 Routing Problems

Communication networks are supposed to get packets from inputs to outputs, with each packet entering the network at its own input switch and arriving at its own output switch. We’re going to consider several different communication network designs, where each network has N inputs and N outputs; for convenience, we’ll



assume N is a power of two.

Which input is supposed to go where is specified by a permutation of $\{0, 1, \dots, N-1\}$. So a permutation, π , defines a *routing problem*: get a packet that starts at input i to output $\pi(i)$. A *routing*, P , that *solves* a routing problem, π , is a set of paths from each input to its specified output. That is, P is a set of n paths, P_i , for $i = 0 \dots, N-1$, where P_i goes from input i to output $\pi(i)$.

10.3 Network Diameter

The delay between the time that a packets arrives at an input and arrives at its designated output is a critical issue in communication networks. Generally this delay is proportional to the length of the path a packet follows. Assuming it takes one time unit to travel across a wire, the delay of a packet will be the number of wires it crosses going from input to output.

Generally packets are routed to go from input to output by the shortest path possible. With a shortest path routing, the worst case delay is the distance between the input and output that are farthest apart. This is called the *diameter* of the network. In other words, the diameter of a network¹ is the maximum length of any shortest

¹The usual definition of *diameter* for a general *graph* (simple or directed) is the largest distance between *any* two vertices, but in the context of a communication network we’re only interested in the distance between inputs and outputs, not between arbitrary pairs of vertices.

path between an input and an output. For example, in the complete binary tree above, the distance from input 1 to output 3 is six. No input and output are farther apart than this, so the diameter of this tree is also six.

More generally, the diameter of a complete binary tree with N inputs and outputs is $2 \log N + 2$. (All logarithms in this lecture— and in most of computer science — are base 2.) This is quite good, because the logarithm function grows very slowly. We could connect up $2^{10} = 1024$ inputs and outputs using a complete binary tree and the worst input-output delay for any packet would be this diameter, namely, $2 \log(2^{10}) + 2 = 22$.

10.3.1 Switch Size

One way to reduce the diameter of a network is to use larger switches. For example, in the complete binary tree, most of the switches have three incoming edges and three outgoing edges, which makes them 3×3 switches. If we had 4×4 switches, then we could construct a complete *ternary* tree with an even smaller diameter. In principle, we could even connect up all the inputs and outputs via a single monster $N \times N$ switch.

This isn't very productive, however, since we've just concealed the original network design problem inside this abstract switch. Eventually, we'll have to design the internals of the monster switch using simpler components, and then we're right back where we started. So the challenge in designing a communication network is figuring out how to get the functionality of an $N \times N$ switch using fixed size, elementary devices, like 3×3 switches.

10.4 Switch Count

Another goal in designing a communication network is to use as few switches as possible. The number of switches in a complete binary tree is $1 + 2 + 4 + 8 + \cdots + N$, since there is 1 switch at the top (the “root switch”), 2 below it, 4 below those, and so forth. By the formula for geometric sums from Problem 6.2,

$$\sum_{i=0}^n r^i = \frac{r^{n+1} - 1}{r - 1},$$

the total number of switches is $2N - 1$, which is nearly the best possible with 3×3 switches.

10.5 Network Latency

We’ll sometimes be choosing routings through a network that optimize some quantity besides delay. For example, in the next section we’ll be trying to minimize packet congestion. When we’re not minimizing delay, shortest routings are not always the best, and in general, the delay of a packet will depend on how it is routed. For any routing, the most delayed packet will be the one that follows the longest path in the routing. The length of the longest path in a routing is called its *latency*.

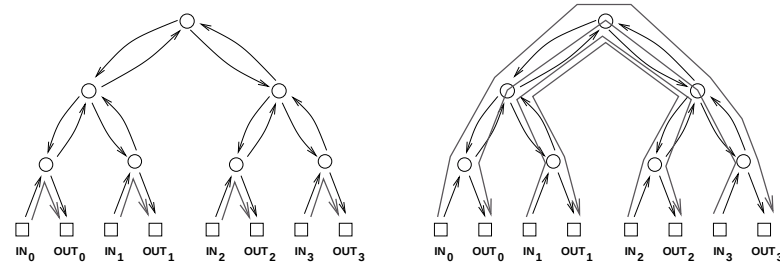
The latency of a *network* depends on what’s being optimized. It is measured by assuming that optimal routings are always chosen in getting inputs to their specified outputs. That is, for each routing problem, π , we choose an optimal routing that solves π . Then *network latency* is defined to be the largest routing latency among these optimal routings. Network latency will equal network diameter if routings are always chosen to optimize delay, but it may be significantly larger if routings are chosen to optimize something else.

For the networks we consider below, paths from input to output are uniquely determined (in the case of the tree) or all paths are the same length, so network latency will always equal network diameter.

10.6 Congestion

The complete binary tree has a fatal drawback: the root switch is a bottleneck. At best, this switch must handle an enormous amount of traffic: every packet traveling from the left side of the network to the right or vice-versa. Passing all these packets through a single switch could take a long time. At worst, if this switch fails, the network is broken into two equal-sized pieces.

For example, if the routing problem is given by the identity permutation, $\text{Id}(i) ::= i$, then there is an easy routing, P , that solves the problem: let P_i be the path from input i up through one switch and back down to output i . On the other hand, if the problem was given by $\pi(i) ::= (N - 1) - i$, then in *any* solution, Q , for π , each path Q_i beginning at input i must eventually loop all the way up through the root switch and then travel back down to output $(N - 1) - i$. These two situations are illustrated below. We can distinguish between a “good” set of paths and a “bad” set based on congestion. The *congestion* of a routing, P , is equal to the largest number of paths in P that pass through a single switch. For example, the congestion of the routing on the left is 1, since at most 1 path passes through each switch. However,



the congestion of the routing on the right is 4, since 4 paths pass through the root switch (and the two switches directly below the root). Generally, lower congestion is better since packets can be delayed at an overloaded switch.

By extending the notion of congestion to networks, we can also distinguish between “good” and “bad” networks with respect to bottleneck problems. For each routing problem, π , for the network, we assume a routing is chosen that optimizes congestion, that is, that has the minimum congestion among all routings that solve π . Then the largest congestion that will ever be suffered by a switch will be the maximum congestion among these optimal routings. This “maximin” congestion is called the *congestion of the network*.

So for the complete binary tree, the worst permutation would be $\pi(i) ::= (N - 1) - i$. Then in every possible solution for π , every packet, would have to follow a path passing through the root switch. Thus, the max congestion of the complete binary tree is N —which is horrible!

Let’s tally the results of our analysis so far:

network	diameter	switch size	# switches	congestion
complete binary tree	$2 \log N + 2$	3×3	$2N - 1$	N

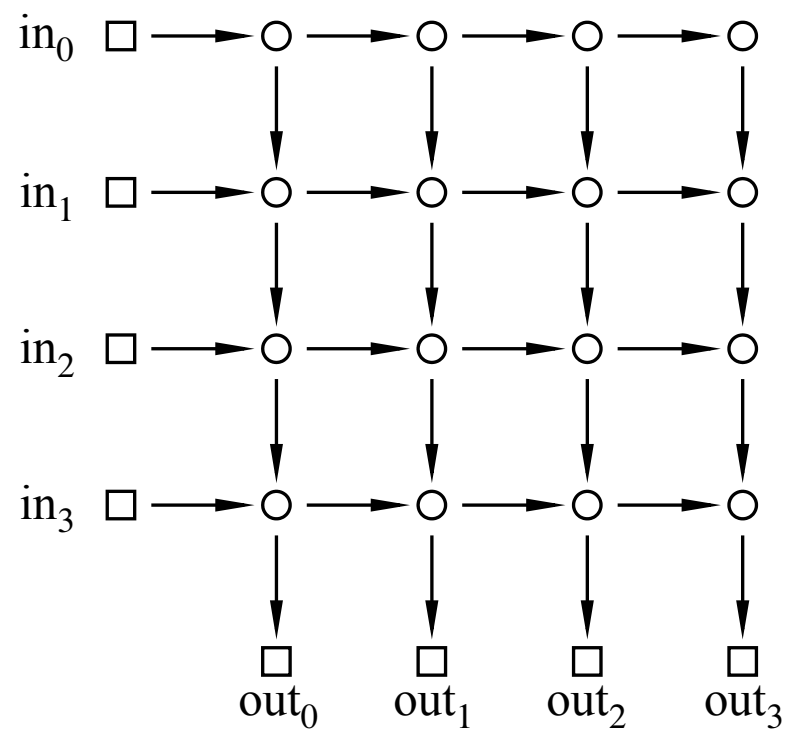
10.7 2-D Array

Let’s look at an another communication network. This one is called a *2-dimensional array* or *grid*.

Here there are four inputs and four outputs, so $N = 4$.

The diameter in this example is 8, which is the number of edges between input 0 and output 3. More generally, the diameter of an array with N inputs and outputs is $2N$, which is much worse than the diameter of $2 \log N + 2$ in the complete binary tree. On the other hand, replacing a complete binary tree with an array almost eliminates congestion.

Theorem 10.7.1. *The congestion of an N -input array is 2.*



Proof. First, we show that the congestion is at most 2. Let π be any permutation. Define a solution, P , for π to be the set of paths, P_i , where P_i goes to the right from input i to column $\pi(i)$ and then goes down to output $\pi(i)$. Thus, the switch in row i and column j transmits at most two packets: the packet originating at input i and the packet destined for output j .

Next, we show that the congestion is at least 2. This follows because in any routing problem, π , where $\pi(0) = 0$ and $\pi(N - 1) = N - 1$, two packets must pass through the lower left switch. ■

As with the tree, the network latency when minimizing congestion is the same as the diameter. That’s because all the paths between a given input and output are the same length.

Now we can record the characteristics of the 2-D array.

network	diameter	switch size	# switches	congestion
complete binary tree	$2 \log N + 2$	3×3	$2N - 1$	N
2-D array	$2N$	2×2	N^2	2

The crucial entry here is the number of switches, which is N^2 . This is a major defect of the 2-D array; a network of size $N = 1000$ would require a *million* 2×2 switches! Still, for applications where N is small, the simplicity and low congestion of the array make it an attractive choice.

10.8 Butterfly

The Holy Grail of switching networks would combine the best properties of the complete binary tree (low diameter, few switches) and of the array (low congestion). The *butterfly* is a widely-used compromise between the two.

A good way to understand butterfly networks is as a recursive data type. The recursive definition works better if we define just the switches and their connections, omitting the terminals. So we recursively define F_n to be the switches and connections of the butterfly net with $N ::= 2^n$ input and output switches.

The base case is F_1 with 2 input switches and 2 output switches connected as in Figure 10.1.

In the constructor step, we construct F_{n+1} with 2^{n+1} inputs and outputs out of two F_n nets connected to a new set of 2^{n+1} input switches, as shown in as in Figure 10.2. That is, the i th and $2^n + i$ th new input switches are each connected to the same two switches, namely, to the i th input switches of each of two F_n

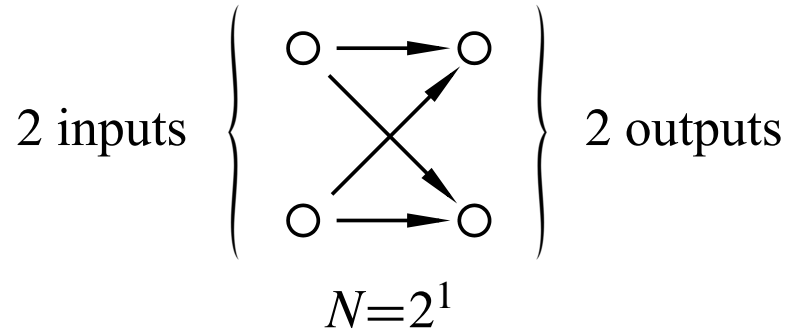


Figure 10.1 F_1 , the Butterfly Net switches with $N = 2^1$.

components for $i = 1, \dots, 2^n$. The output switches of F_{n+1} are simply the output switches of each of the F_n copies.

So F_{n+1} is laid out in columns of height 2^{n+1} by adding one more column of switches to the columns in F_n . Since the construction starts with two columns when $n = 1$, the F_{n+1} switches are arrayed in $n + 1$ columns. The total number of switches is the height of the columns times the number of columns, namely, $2^{n+1}(n + 1)$. Remembering that $n = \log N$, we conclude that the Butterfly Net with N inputs has $N(\log N + 1)$ switches.

Since every path in F_{n+1} from an input switch to an output is the same length, namely, $n + 1$, the diameter of the Butterfly net with 2^{n+1} inputs is this length plus two because of the two edges connecting to the terminals (square boxes) — one edge from input terminal to input switch (circle) and one from output switch to output terminal.

There is an easy recursive procedure to route a packet through the Butterfly Net. In the base case, there is obviously only one way to route a packet from one of the two inputs to one of the two outputs. Now suppose we want to route a packet from an input switch to an output switch in F_{n+1} . If the output switch is in the “top” copy of F_n , then the first step in the route must be from the input switch to the unique switch it is connected to in the top copy; the rest of the route is determined by recursively routing the rest of the way in the top copy of F_n . Likewise, if the output switch is in the “bottom” copy of F_n , then the first step in the route must be to the switch in the bottom copy, and the rest of the route is determined by recursively routing in the bottom copy of F_n . In fact, this argument shows that the routing is *unique*: there is exactly one path in the Butterfly Net from each input to each output, which implies that the network latency when minimizing congestion is the same as the diameter.

The congestion of the butterfly network is about $\sqrt{N}$, more precisely, the con-

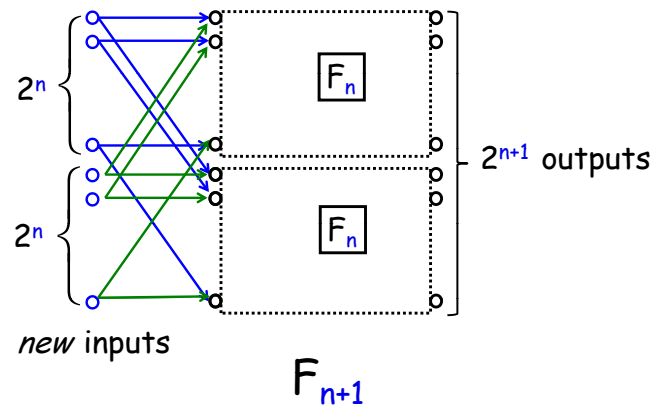


Figure 10.2 F_{n+1} , the Butterfly Net switches with 2^{n+1} inputs and outputs.

gestion is $\sqrt{N}$ if N is an even power of 2 and $\sqrt{N/2}$ if N is an odd power of 2. A simple proof of this appears in Problem 10.8.

Let’s add the butterfly data to our comparison table:

network	diameter	switch size	# switches	congestion
complete binary tree	$2 \log N + 2$	3×3	$2N - 1$	N
2-D array	$2N$	2×2	N^2	2
butterfly	$\log N + 2$	2×2	$N(\log(N) + 1)$	$\sqrt{N}$ or $\sqrt{N/2}$

The butterfly has lower congestion than the complete binary tree. And it uses fewer switches and has lower diameter than the array. However, the butterfly does not capture the best qualities of each network, but rather is a compromise somewhere between the two. So our quest for the Holy Grail of routing networks goes on.

10.9 Beneš Network

In the 1960’s, a researcher at Bell Labs named Beneš had a remarkable idea. He obtained a marvelous communication network with congestion 1 by placing *two* butterflies back-to-back. This amounts to recursively growing *Beneš nets* by adding

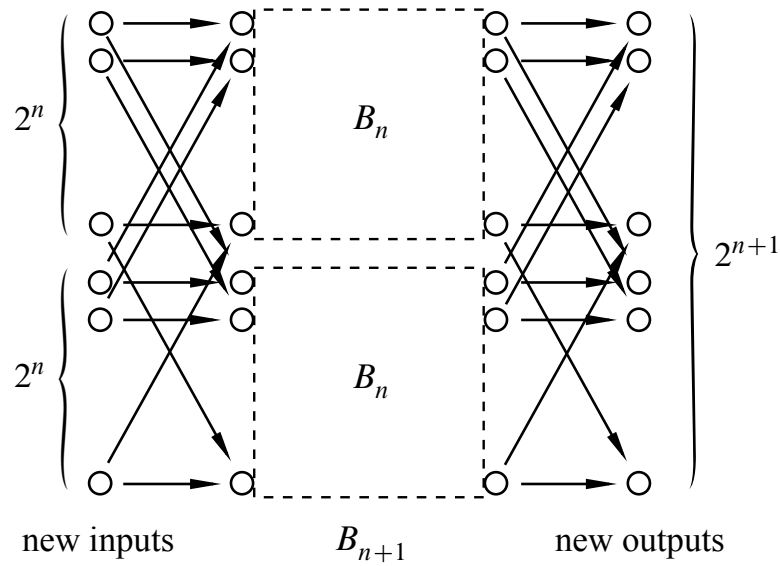


Figure 10.3 B_{n+1} , the Beneš Net switches with 2^{n+1} inputs and outputs.

both inputs and outputs at each stage. Now we recursively define B_n to be the switches and connections (without the terminals) of the Beneš net with $N ::= 2^n$ input and output switches.

The base case, B_1 , with 2 input switches and 2 output switches is exactly the same as F_1 in Figure 10.1.

In the constructor step, we construct B_{n+1} out of two B_n nets connected to a new set of 2^{n+1} input switches *and also* a new set of 2^{n+1} output switches. This is illustrated in Figure 10.3.

Namely, the i th and $2^n + i$ th new input switches are each connected to the same two switches, namely, to the i th input switches of each of two B_n components for $i = 1, \dots, 2^n$, exactly as in the Butterfly net. In addition, the i th and $2^n + i$ th new *output* switches are connected to the same two switches, namely, to the i th output switches of each of two B_n components.

Now B_{n+1} is laid out in columns of height 2^{n+1} by adding two more columns of switches to the columns in B_n . So the B_{n+1} switches are arrayed in $2(n + 1)$ columns. The total number of switches is the number of columns times the height of the columns, namely, $2(n + 1)2^{n+1}$.

All paths in B_{n+1} from an input switch to an output are the same length, namely, $2(n + 1) - 1$, and the diameter of the Beneš net with 2^{n+1} inputs is this length plus two because of the two edges connecting to the terminals.

So Beneš has doubled the number of switches and the diameter, of course, but

completely eliminates congestion problems! The proof of this fact relies on a clever induction argument that we’ll come to in a moment. Let’s first see how the Beneš network stacks up:

network	diameter	switch size	# switches	congestion
complete binary tree	$2 \log N + 2$	3×3	$2N - 1$	N
2-D array	$2N$	2×2	N^2	2
butterfly	$\log N + 2$	2×2	$N(\log(N) + 1)$	$\sqrt{N}$ or $\sqrt{N/2}$
Beneš	$2 \log N + 1$	2×2	$2N \log N$	1

The Beneš network has small size and diameter, and completely eliminates congestion. The Holy Grail of routing networks is in hand!

Theorem 10.9.1. *The congestion of the N -input Beneš network is 1.*

Proof. By induction on n where $N = 2^n$. So the induction hypothesis is

$$P(n) ::= \text{the congestion of } B_n \text{ is } 1.$$

Base case ($n = 1$): $B_1 = F_1$ is shown in Figure 10.1. The unique routings in F_1 have congestion 1.

Inductive step: We assume that the congestion of an $N = 2^n$ -input Beneš network is 1 and prove that the congestion of a $2N$ -input Beneš network is also 1.

Digression. Time out! Let’s work through an example, develop some intuition, and then complete the proof. In the Beneš network shown in Figure 10.4 with $N = 8$ inputs and outputs, the two 4-input/output subnetworks are in dashed boxes.

By the inductive assumption, the subnetworks can each route an arbitrary permutation with congestion 1. So if we can guide packets safely through just the first and last levels, then we can rely on induction for the rest! Let’s see how this works in an example. Consider the following permutation routing problem:

$$\begin{array}{ll} \pi(0) = 1 & \pi(4) = 3 \\ \pi(1) = 5 & \pi(5) = 6 \\ \pi(2) = 4 & \pi(6) = 0 \\ \pi(3) = 7 & \pi(7) = 2 \end{array}$$

We can route each packet to its destination through either the upper subnetwork or the lower subnetwork. However, the choice for one packet may constrain the choice for another. For example, we can not route both packet 0 *and* packet 4 through the same network since that would cause two packets to collide at a single

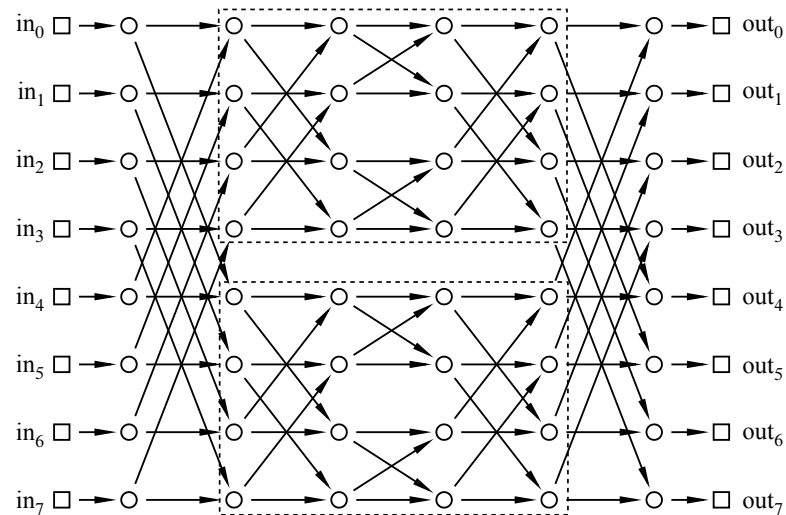
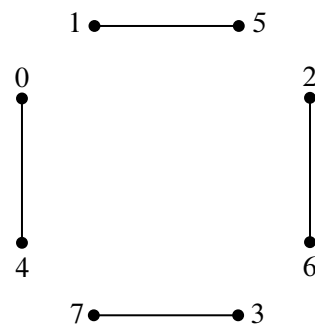


Figure 10.4 Beneš net B_3 .

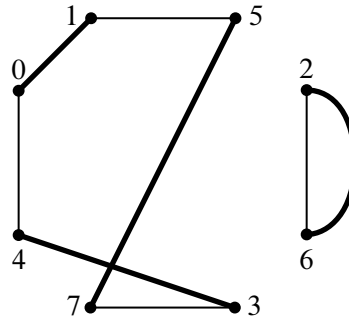
switch, resulting in congestion. So one packet must go through the upper network and the other through the lower network. Similarly, packets 1 and 5, 2 and 6, and 3 and 7 must be routed through different networks. Let's record these constraints in a graph. The vertices are the 8 packets. If two packets must pass through different networks, then there is an edge between them. Thus, our constraint graph looks like this:



Notice that at most one edge is incident to each vertex.

The output side of the network imposes some further constraints. For example, the packet destined for output 0 (which is packet 6) and the packet destined for output 4 (which is packet 2) can not both pass through the same network; that would require both packets to arrive from the same switch. Similarly, the packets destined for outputs 1 and 5, 2 and 6, and 3 and 7 must also pass through different

switches. We can record these additional constraints in our graph with gray edges:



Notice that at most one new edge is incident to each vertex. The two lines drawn between vertices 2 and 6 reflect the two different reasons why these packets must be routed through different networks. However, we intend this to be a simple graph; the two lines still signify a single edge.

Now here’s the key insight: suppose that we could color each vertex either red or blue so that adjacent vertices are colored differently. Then all constraints are satisfied if we send the red packets through the upper network and the blue packets through the lower network. Such a *2-coloring of the graph corresponds to a solution to the routing problem*. The only remaining question is whether the constraint graph is 2-colorable, which is easy to verify:

Lemma 10.9.2. *Prove that if the edges of a graph can be grouped into two sets such that every vertex has at most 1 edge from each set incident to it, then the graph is 2-colorable.*

Proof. It is not hard to show that a graph is 2-colorable iff every cycle in it has even length (see Theorem 11.10.1). We’ll take this for granted here.

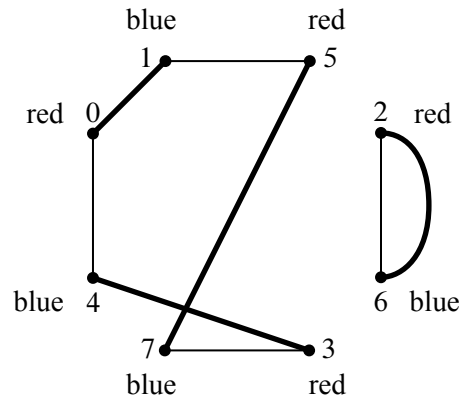
So all we have to do is show that every cycle has even length. Since the two sets of edges may overlap, let’s call an edge that is in both sets a *doubled edge*.

There are two cases:

Case 1: [The cycle contains a doubled edge.] No other edge can be incident to either of the endpoints of a doubled edge, since that endpoint would then be incident to two edges from the same set. So a cycle traversing a doubled edge has nowhere to go but back and forth along the edge an even number of times.

Case 2: [No edge on the cycle is doubled.] Since each vertex is incident to at most one edge from each set, any path with no doubled edges must traverse successive edges that alternate from one set to the other. In particular, a cycle must traverse a path of alternating edges that begins and ends with edges from different sets. This means the cycle has to be of even length. ■

For example, here is a 2-coloring of the constraint graph:



The solution to this graph-coloring problem provides a start on the packet routing problem:

We can complete the routing in the two smaller Beneš networks by induction! Back to the proof. **End of Digression.**

Let π be an arbitrary permutation of $\{0, 1, \dots, N-1\}$. Let G be the graph whose vertices are packet numbers $0, 1, \dots, N-1$ and whose edges come from the union of these two sets:

$$E_1 ::= \{\langle u-v \rangle \mid |u-v| = N/2\}, \text{ and}$$

$$E_2 ::= \{\langle u-w \rangle \mid |\pi(u) - \pi(w)| = N/2\}.$$

Now any vertex, u , is incident to at most two edges: a unique edge $\langle u-v \rangle \in E_1$ and a unique edge $\langle u-w \rangle \in E_2$. So according to Lemma 10.9.2, there is a 2-coloring for the vertices of G . Now route packets of one color through the upper subnetwork and packets of the other color through the lower subnetwork. Since for each edge in E_1 , one vertex goes to the upper subnetwork and the other to the lower subnetwork, there will not be any conflicts in the first level. Since for each edge in E_2 , one vertex comes from the upper subnetwork and the other from the lower subnetwork, there will not be any conflicts in the last level. We can complete the routing within each subnetwork by the induction hypothesis $P(n)$. ■

Problems for Section 10.9

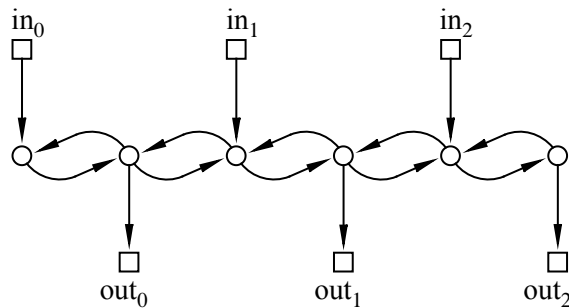
Exam Problems

Problem 10.1.

Consider the following communication network:

(a) What is the max congestion?

0.5in



(b) Give an input/output permutation, π_0 , that forces maximum congestion:

$$\pi_0(0) = \underline{\hspace{1cm}} \quad \pi_0(1) = \underline{\hspace{1cm}} \quad \pi_0(2) = \underline{\hspace{1cm}}$$

(c) Give an input/output permutation, π_1 , that allows *minimum* congestion:

$$\pi_1(0) = \underline{\hspace{1cm}} \quad \pi_1(1) = \underline{\hspace{1cm}} \quad \pi_1(2) = \underline{\hspace{1cm}}$$

(d) What is the latency for the permutation π_1 ? (If you could not find π_1 , just choose a permutation and find its latency.) 0.5in

Class Problems

Problem 10.2.

The Beneš network has a max congestion of 1; that is, every permutation can be routed in such a way that a single packet passes through each switch. Let’s work through an example. Within the Beneš network of size $N = 8$ shown in Figure 10.4, the two subnetworks of size $N = 4$ are marked. We’ll refer to these as the *upper* and *lower* subnetworks.

(a) Now consider the following permutation routing problem:

$$\begin{array}{ll} \pi(0) = 3 & \pi(4) = 2 \\ \pi(1) = 1 & \pi(5) = 0 \\ \pi(2) = 6 & \pi(6) = 7 \\ \pi(3) = 5 & \pi(7) = 4 \end{array}$$

Each packet must be routed through either the upper subnetwork or the lower subnetwork. Construct a graph with vertices $0, 1, \dots, 7$ and draw a *dashed* edge between each pair of packets that can not go through the same subnetwork because a collision would occur in the second column of switches.

(b) Add a *solid* edge in your graph between each pair of packets that can not go

through the same subnetwork because a collision would occur in the next-to-last column of switches.

(c) Color the vertices of your graph red and blue so that adjacent vertices get different colors. Why must this be possible, regardless of the permutation π ?

(d) Suppose that red vertices correspond to packets routed through the upper subnetwork and blue vertices correspond to packets routed through the lower subnetwork. On the attached copy of the Beneš network, highlight the first and last edge traversed by each packet.

(e) All that remains is to route packets through the upper and lower subnetworks. One way to do this is by applying the procedure described above recursively on each subnetwork. However, since the remaining problems are small, see if you can complete all the paths on your own.

Problem 10.3.

A *multiple binary-tree network* has n inputs and n outputs, where n is a power of 2. Each input is connected to the root of a binary tree with $n/2$ leaves and with edges pointing away from the root. Likewise, each output is connected to the root of a binary tree with $n/2$ leaves and with edges pointing toward the root.

Two edges point from each leaf of an input tree, and each of these edges points to a leaf of an output tree. The matching of leaf edges is arranged so that for every input and output tree, there is an edge from a leaf of the input tree to a leaf of the output tree, and every output tree leaf has exactly two edges pointing to it.

(a) Draw such a multiple binary-tree net for $n = 4$.

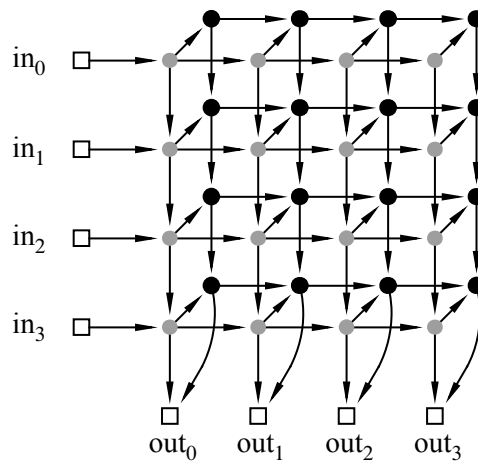
(b) Fill in the table, and explain your entries.

# switches	switch size	diameter	max congestion

Problem 10.4.

The n -input 2-D Array network was shown to have congestion 2. An n -input 2-Layer Array consisting of two n -input 2-D Arrays connected as pictured below for $n = 4$.

In general, an n -input 2-Layer Array has two layers of switches, with each layer connected like an n -input 2-D Array. There is also an edge from each switch in



the first layer to the corresponding switch in the second layer. The inputs of the 2-Layer Array enter the left side of the first layer, and the n outputs leave from the bottom row of either layer.

- (a) For any given input-output permutation, there is a way to route packets that achieves congestion 1. Describe how to route the packets in this way.
- (b) What is the latency of a routing designed to minimize latency?
- (c) Explain why the congestion of any minimum latency (CML) routing of packets through this network is greater than the network’s congestion.

Problem 10.5.

A 5-path communication network is shown below. From this, it’s easy to see what an n -path network would be. Fill in the table of properties below, and be prepared to justify your answers.

network	# switches	switch size	diameter	max congestion
5-path				
n -path				

Problem 10.6.

Tired of being a TA, Megumi has decided to become famous by coming up with a

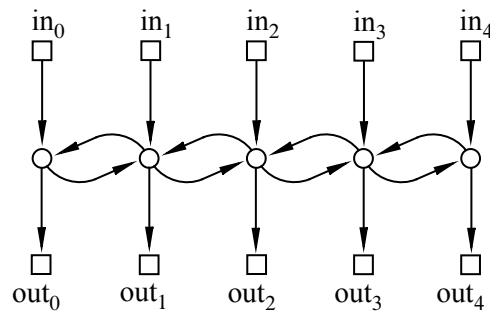
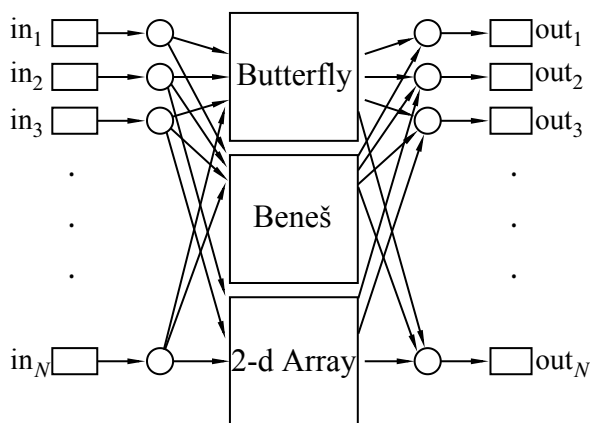


Figure 10.5 5-Path

new, better communication network design. Her network has the following specifications: every input node will be sent to a butterfly network, a Beneš network and a 2-d array network. At the end, the outputs of all three networks will converge on the new output.

In the Megumi-net a minimum latency routing does not have minimum congestion. The *latency for min-congestion (LMC)* of a net is the best bound on latency achievable using routings that minimize congestion. Likewise, the *congestion for min-latency (CML)* is the best bound on congestion achievable using routings that minimize latency.



Fill in the following chart for Megumi's new net and explain your answers.

network	diameter	# switches	congestion	LMC	CML
Megumi's net					

Homework Problems

Problem 10.7.

Louis Reasoner figures that, wonderful as the Beneš network may be, the butterfly network has a few advantages, namely: fewer switches, smaller diameter, and an easy way to route packets through it. So Louis designs an N -input/output network he modestly calls a *Reasoner-net* with the aim of combining the best features of both the butterfly and Beneš nets:

The i th input switch in a Reasoner-net connects to two switches, a_i and b_i , and likewise, the j th output switch has two switches, y_j and z_j , connected to it. Then the Reasoner-net has an N -input Beneš network connected using the a_i switches as input switches and the y_j switches as its output switches. The Reasoner-net also has an N -input butterfly net connected using the b_i switches as inputs and the z_j switches as outputs.

In the Reasoner-net a minimum latency routing does not have minimum congestion. The *latency for min-congestion (LMC)* of a net is the best bound on latency achievable using routings that minimize congestion. Likewise, the *congestion for min-latency (CML)* is the best bound on congestion achievable using routings that minimize latency.

Fill in the following chart for the Reasoner-net and briefly explain your answers.

diameter	switch size(s)	# switches	congestion	LMC	CML

Problem 10.8.

Show that the congestion of the butterfly net, F_n , is exactly $\sqrt{N}$ when n is even.

Hint:

- There is a unique path from each input to each output, so the congestion is the maximum number of messages passing through a vertex for any routing problem.
- If v is a vertex in column i of the butterfly network, there is a path from exactly 2^i input vertices to v and a path from v to exactly 2^{n-i} output vertices.
- At which column of the butterfly network must the congestion be worst? What is the congestion of the topmost switch in that column of the network?



11 Simple Graphs

Simple graphs model relationships that are *symmetric*, meaning that the relationship is mutual. Examples of such mutual relationships are being married, speaking the same language, not speaking the same language, occurring during overlapping time intervals, or being connected by a conducting wire. They come up in all sorts of applications, including scheduling, constraint satisfaction, computer graphics, and communications, but we’ll start with an application designed to get your attention: we are going to make a professional inquiry into sexual behavior. Namely, we’ll look at some data about who, on average, has more opposite-gender partners, men or women.

Sexual demographics have been the subject of many studies. In one of the largest studies, researchers from the University of Chicago interviewed a random sample of 2500 people over several years to try to get an answer to this question. Their study, published in 1994, and entitled *The Social Organization of Sexuality* found that on average men have 74% more opposite-gender partners than women.

Other studies have found that the disparity is even larger. In particular, ABC News claimed that the average man has 20 partners over his lifetime, and the average woman has 6, for a percentage disparity of 233%. The ABC News study, aired on Primetime Live in 2004, purported to be one of the most scientific ever done, with only a 2.5% margin of error. It was called “American Sex Survey: A peek between the sheets,” —which raises some questions about the seriousness of their reporting.

Yet again, in August, 2007, the N.Y. Times [reported](#) on a study by the National Center for Health Statistics of the U.S. government showing that men had seven partners while women had four. Anyway, whose numbers do you think are more accurate, the University of Chicago, ABC News, or the National Center? —don’t answer; this is a setup question like “When did you stop beating your wife?” Using a little graph theory, we’ll explain why none of these findings can be anywhere near the truth.

11.1 Vertex Adjacency and Degrees

Simple graphs are defined as digraphs in which edges are *undirected* —they just connect two vertices without pointing in either direction between the vertices. So instead of a directed edge $\langle v \rightarrow w \rangle$ which starts at vertex v and ends at vertex w , a

simple graph only has an undirected edge, $\langle v-w \rangle$, that connects v and w .

Definition 11.1.1. A *simple graph*, G , consists of a nonempty set, $V(G)$, called the *vertices* of G , and a set $E(G)$ called the *edges* of G . An element of $V(G)$ is called a *vertex*. A vertex is also called a *node*; the words “vertex” and “node” are used interchangeably. An element of $E(G)$ an *undirected edge* or simply an “edge.” An undirected edge has two vertices $u \neq v$ called its *endpoints*. Such an edge can be represented by the two element set $\{u, v\}$. The notation $\langle u-v \rangle$ denotes this edge.

Both $\langle u-v \rangle$ and $\langle v-u \rangle$ define the same undirected edge, namely the one whose endpoints are u and v .

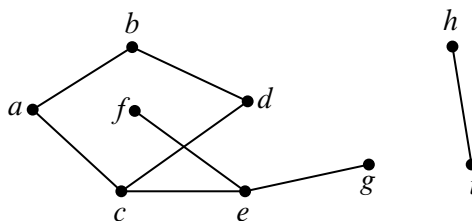


Figure 11.1 An example of a graph with 9 nodes and 8 edges.

For example, let H be the graph pictured in Figure 11.1. The vertices of H correspond to the nine dots in Figure 11.1, that is,

$$V(H) = \{a, b, c, d, e, f, g, h, i\}.$$

The edges correspond to the eight lines, that is,

$$E(H) = \{ \langle a-b \rangle, \langle a-c \rangle, \langle b-d \rangle, \langle c-d \rangle, \langle c-e \rangle, \langle e-f \rangle, \langle e-g \rangle, \langle h-i \rangle \}.$$

Mathematically, that’s all there is to the graph H .

Definition 11.1.2. Two vertices in a simple graph are said to be *adjacent* iff they are the endpoints of the same edge, and an edge is said to be *incident* to each of its endpoints. The number of edges incident to a vertex v is called the *degree* of the vertex and is denoted by $\deg(v)$. Equivalently, the degree of a vertex is the number of vertices adjacent to it.

For example, for the graph H of Figure 11.1, vertex a is adjacent to vertex b , and b is adjacent to d . The edge $\langle a-c \rangle$ is incident to its endpoints a and c . Vertex h has degree 1, d has degree 2, and $\deg(e) = 3$. It is possible for a vertex to have degree 0, in which case it is not adjacent to any other vertices. A simple graph, G , does not need to have any edges at all, namely $|E(G)|$ could be zero, which implies

that the degree of every vertex is also zero. But a simple graph must have at least one vertex, that is, $|V(G)|$ is required to be at least one.

An edge whose endpoints are the same is called a *self-loop*. Self-loops aren’t allowed in simple graphs.¹ In a more general class of graphs called *multigraphs* there can be more than one edge with the same two endpoints, but this doesn’t happen in simple graphs since every edge is uniquely determined by its two endpoints.

Sometimes graphs with no vertices, with self-loops, or with more than one edge between the same two vertices are convenient to have, but we don’t need them, and sticking with simple graphs is simpler. : –)

For the rest of this chapter we’ll use “graphs” as an abbreviation for “simple graphs.”

A synonym for “vertices” is “*nodes*,” and we’ll use these words interchangeably. Simple graphs are sometimes called *networks*, edges are sometimes called *arcs*. We mention this as a “heads up” in case you look at other graph theory literature; we won’t use these words.

11.2 Sexual Demographics in America

Let’s model the question of heterosexual partners in graph theoretic terms. To do this, we’ll let G be the graph whose vertices, V , are all the people in America. Then we split V into two separate subsets: M , which contains all the males, and F , which contains all the females.² We’ll put an edge between a male and a female iff they have been sexual partners. This graph is pictured in Figure 11.2 with males on the left and females on the right.

Actually, this is a pretty hard graph to figure out, let alone draw. The graph is *enormous*: the US population is about 300 million, so $|V| \approx 300M$. Of these, approximately 50.8% are female and 49.2% are male, so $|M| \approx 147.6M$, and $|F| \approx 152.4M$. And we don’t even have trustworthy estimates of how many edges there are, let alone exactly which couples are adjacent. But it turns out that we don’t need to know any of this —we just need to figure out the relationship between the average number of partners per male and partners per female. To do this, we note that every edge has exactly one endpoint in M vertex (remember, we’re only considering male-female relationships); so the sum of the degrees of the M vertices equals the number of edges. For the same reason, the sum of the degrees of the F

¹You might try to represent a self-loop going between a vertex v and itself as $\{v, v\}$, but this equals $\{v\}$, and it wouldn’t be an edge which is defined to be a set of *two* vertices.

²For simplicity, we’ll ignore the possibility of someone being *both* a man and a woman, or neither.

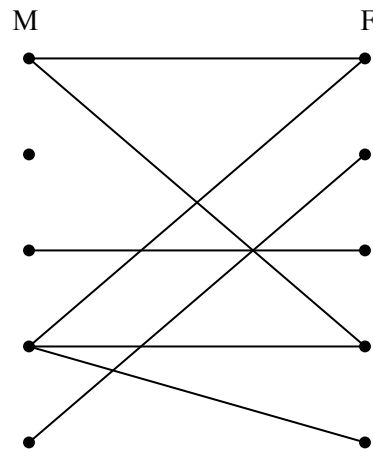


Figure 11.2 The sex partners graph.

vertices equals the number of edges. So these sums are equal:

$$\sum_{x \in M} \deg(x) = \sum_{y \in F} \deg(y).$$

Now suppose we divide both sides of this equation by the product of the sizes of the two sets, $|M| \cdot |F|$:

$$\left(\frac{\sum_{x \in M} \deg(x)}{|M|} \right) \cdot \frac{1}{|F|} = \left(\frac{\sum_{y \in F} \deg(y)}{|F|} \right) \cdot \frac{1}{|M|}$$

The terms above in parentheses are the *average degree of an M vertex* and the *average degree of a F vertex*. So we know:

$$\text{Avg. deg in } M = \frac{|F|}{|M|} \cdot \text{Avg. deg in } F \quad (11.1)$$

In other words, we’ve proved that the average number of female partners of males in the population compared to the average number of males per female is *determined solely by the relative number of males and females in the population*.

Now the Census Bureau reports that there are slightly more females than males in America; in particular $|F|/|M|$ is about 1.035. So we know that on average, males have 3.5% more opposite-gender partners than females, and this tells us nothing about any sex’s promiscuity or selectivity. Rather, it just has to do with the relative number of males and females. Collectively, males and females have the same number of opposite gender partners, since it takes one of each set for every partnership,

but there are fewer males, so they have a higher ratio. This means that the University of Chicago, ABC, and the Federal government studies are way off. After a huge effort, they gave a totally wrong answer.

There’s no definite explanation for why such surveys are consistently wrong. One hypothesis is that males exaggerate their number of partners —or maybe females downplay theirs —but these explanations are speculative. Interestingly, the principal author of the National Center for Health Statistics study reported that she knew the results had to be wrong, but that was the data collected, and her job was to report it.

The same underlying issue has led to serious misinterpretations of other survey data. For example, a couple of years ago, the Boston Globe ran a story on a survey of the study habits of students on Boston area campuses. Their survey showed that on average, minority students tended to study with non-minority students more than the other way around. They went on at great length to explain why this “remarkable phenomenon” might be true. But it’s not remarkable at all —using our graph theory formulation, we can see that all it says is that there are fewer minority students than non-minority students, which is, of course what “minority” means.

11.2.1 Handshaking Lemma

The previous argument hinged on the connection between a sum of degrees and the number edges. There is a simple connection between these in any graph:

Lemma 11.2.1. *The sum of the degrees of the vertices in a graph equals twice the number of edges.*

Proof. Every edge contributes two to the sum of the degrees, one for each of its endpoints. ■

Lemma 11.2.1 is sometimes called the *Handshake Lemma*: if we total up the number of people each person at a party shakes hands with, the total will be twice the number of handshakes that occurred.

11.3 Some Common Graphs

Some graphs come up so frequently that they have names. A *complete graph* K_n has n vertices and an edge between every two vertices, for a total of $n(n - 1)/2$ edges. For example, K_5 is shown in Figure 11.3.

The *empty graph* has no edges at all. For example, the empty graph with 5 nodes is shown in Figure 11.4.

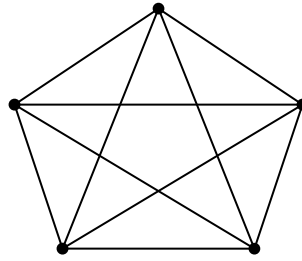


Figure 11.3 K_5 : the complete graph on 5 nodes.

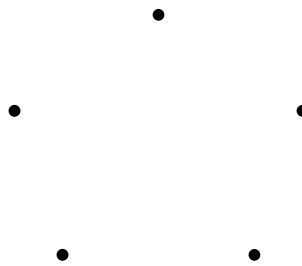


Figure 11.4 An empty graph with 5 nodes.

An n -node graph containing $n-1$ edges in sequence is known as a *line graph* L_n . More formally, L_n has

$$V(L_n) = \{v_1, v_2, \dots, v_n\}$$

and

$$E(L_n) = \{\langle v_1-v_2 \rangle, \langle v_2-v_3 \rangle, \dots, \langle v_{n-1}-v_n \rangle\}$$

For example, L_5 is pictured in Figure 11.5.

There is also a one-way infinite line graph L_∞ which can be defined by letting the nonnegative integers $\mathbb{N}$ be the vertices with edges $\langle k-(k+1) \rangle$ for all $k \in \mathbb{N}$.

If we add the edge $\langle v_n-v_1 \rangle$ to the line graph L_n , we get a graph called a *length- n cycle* C_n . Figure 11.6 shows a picture of length-5 cycle.

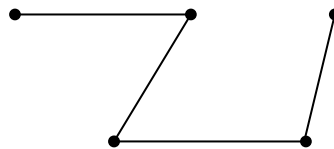


Figure 11.5 L_5 : a 5-node line graph.

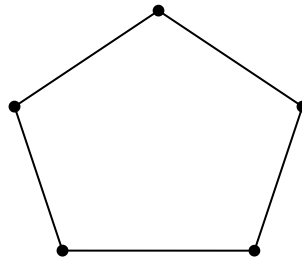


Figure 11.6 C_5 : a 5-node cycle graph.

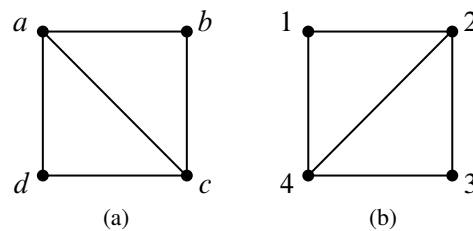


Figure 11.7 Two Isomorphic graphs.

11.4 Isomorphism

Two graphs that look the same might actually be different in a formal sense. For example, the two graphs in Figure 11.7 are both 4-vertex, 5-edge graphs and you get graph (b) by a 90° clockwise rotation of graph (a).

Strictly speaking, these graphs are different mathematical objects, but this difference doesn’t reflect the fact that the two graphs can be described by the same picture—except for the labels on the vertices. This idea of having the same picture “up to relabeling” can be captured neatly by adapting Definition 9.7.1 of isomorphism of digraphs to handle simple graphs. An isomorphism between two graphs is an edge-preserving bijection between their sets of vertices:

Definition 11.4.1. An isomorphism between graphs G and H is a bijection $f : V(G) \rightarrow V(H)$ such that

$$\langle u-v \rangle \in E(G) \quad \text{iff} \quad \langle f(u)-f(v) \rangle \in E(H)$$

for all $u, v \in V(G)$. Two graphs are isomorphic when there is an isomorphism between them.

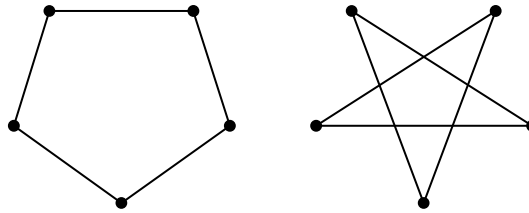


Figure 11.8 Isomorphic C_5 graphs.

Here is an isomorphism, f , between the two graphs in Figure 11.7:

$$\begin{array}{ll} f(a) ::= 2 & f(b) ::= 3 \\ f(c) ::= 4 & f(d) ::= 1. \end{array}$$

You can check that there is an edge between two vertices in the graph on the left if and only if there is an edge between the two corresponding vertices in the graph on the right.

Two isomorphic graphs may be drawn very differently. For example, Figure 11.8 shows two different ways of drawing C_5 :

Notice that if f is an isomorphism between G and H , then f^{-1} is an isomorphism between H and G . Isomorphism is also transitive because the composition of isomorphisms is an isomorphism. So isomorphism is in fact an equivalence relation.

Isomorphism preserves the connection properties of a graph, abstracting out what the vertices are called, what they are made out of, or where they appear in a drawing of the graph. More precisely, a property of a graph is said to be *preserved under isomorphism* if whenever G has that property, every graph isomorphic to G also has that property. For example, since an isomorphism is a bijection between sets of vertices, isomorphic graphs must have the same number of vertices. What’s more, if f is a graph isomorphism that maps a vertex, v , of one graph to the vertex, $f(v)$, of an isomorphic graph, then by definition of isomorphism, every vertex adjacent to v in the first graph will be mapped by f to a vertex adjacent to $f(v)$ in the isomorphic graph. That is, v and $f(v)$ will have the same degree. So if one graph has a vertex of degree 4 and another does not, then they can’t be isomorphic. In fact, they can’t be isomorphic if the number of degree 4 vertices in each of the graphs is not the same.

Looking for preserved properties can make it easy to determine that two graphs are not isomorphic, or to guide the search for an isomorphism when there is one. It’s generally easy in practice to decide whether two graphs are isomorphic. However, no one has yet found a procedure for determining whether two graphs are

isomorphic that is *guaranteed* to run in polynomial time on all pairs of graphs.³

Having such a procedure would be useful. For example, it would make it easy to search for a particular molecule in a database given the molecular bonds. On the other hand, knowing there is no such efficient procedure would also be valuable: secure protocols for encryption and remote authentication can be built on the hypothesis that graph isomorphism is computationally exhausting.

The definitions of bijection and isomorphism apply infinite graphs as well as finite graphs, as do most of the results in the rest of this chapter. But graph theory focuses mostly on finite graphs, and we will too. So

in the rest of this chapter we'll assume graphs are finite.

We've actually been taking isomorphism for granted ever since we wrote “ K_n has n vertices...” at the beginning of section 11.3.

Graph theory is all about properties preserved by isomorphism.

11.5 Bipartite Graphs & Matchings

There were two kinds of vertices in the “Sex in America” graph —males and females, and edges only went between the two kinds. Graphs like this come up so frequently that they have earned a special name —they are called *bipartite graphs*.

Definition 11.5.1. A *bipartite graph* is a graph whose vertices can be partitioned⁴ into two sets, $L(G)$ and $R(G)$, such that every edge has one endpoint in $L(G)$ and the other endpoint in $R(G)$.

So every bipartite graph looks something like the graph in Figure 11.2.

11.5.1 The Bipartite Matching Problem

The bipartite matching problem is related to the sex-in-America problem that we just studied; only now the goal is to get everyone happily married. As you might imagine, this is not possible for a variety of reasons, not the least of which is the fact that there are more women in America than men. So, it is simply not possible to marry every woman to a man so that every man is married at most once.

But what about getting a mate for every man so that every woman is married at most once? Is it possible to do this so that each man is paired with a woman that

³A procedure runs in *polynomial time* when it needs an amount of time of at most $p(n)$, where n is the total number of vertices and $p()$ is a fixed polynomial.

⁴Partitioning a set means cutting it up into *nonempty* pieces. In this case, it means that $L(G)$ and $R(G)$ are nonempty, $L(G) \cup R(G) = V(G)$, and $L(G) \cap R(G) = \emptyset$.

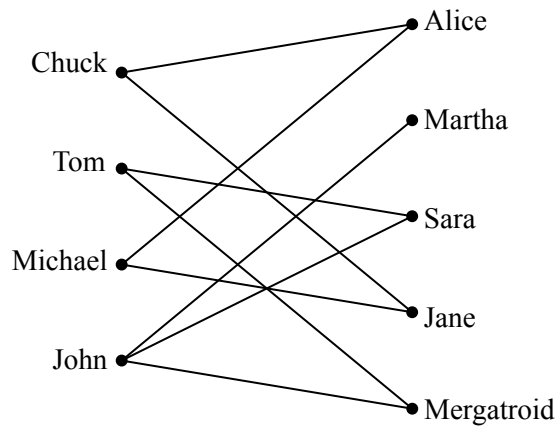


Figure 11.9 A graph where an edge between a man and woman denotes that the man likes the woman.

he likes? The answer, of course, depends on the bipartite graph that represents who likes who, but the good news is that it is possible to find natural properties of the who-likes-who graph that completely determine the answer to this question.

In general, suppose that we have a set of men and an equal-sized or larger set of women, and there is a graph with an edge between a man and a woman if the man likes the woman. In this scenario, the “likes” relationship need not be symmetric, since for the time being, we will only worry about finding a mate for each man that he likes.⁵ (Later, we will consider the “likes” relationship from the female perspective as well.) For example, we might obtain the graph in Figure 11.9.

A *matching* is defined to be an assignment of a woman to each man so that different men are assigned to different women, and a man is always assigned a woman that he likes. For example, one possible matching for the men is shown in Figure 11.10.

The Matching Condition

A famous result known as Hall’s Matching Theorem gives necessary and sufficient conditions for the existence of a matching in a bipartite graph. It turns out to be a remarkably useful mathematical tool.

We’ll state and prove Hall’s Theorem using man-likes-woman terminology. Define the *set of women liked by a given set of men* to consist of all women liked by

⁵By the way, we do not mean to imply that marriage should or should not be of a heterosexual nature. Nor do we mean to imply that men should get their choice instead of women. It’s just that with bipartite graphs, the edges only connected male nodes to female nodes and there are fewer men in America. So please don’t take offense.

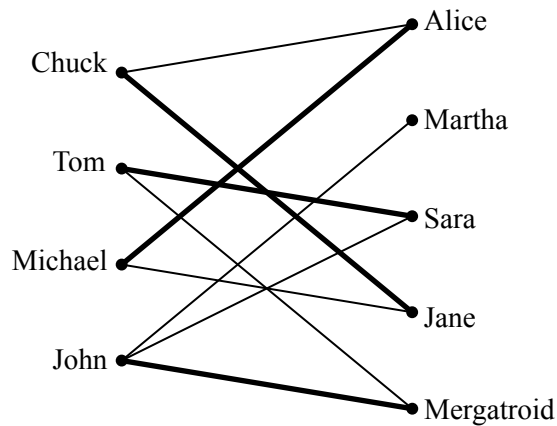


Figure 11.10 One possible matching for the men is shown with bold edges. For example, John is matched with Mergatroid.

at least one of those men. For example, the set of women liked by Tom and John in Figure 11.9 consists of Martha, Sara, and Mergatroid. For us to have any chance at all of matching up the men, the following *matching condition* must hold:

The Matching Condition: every subset of men likes at least as large a set of women.

For example, we can not find a matching if some set of 4 men like only 3 women. Hall’s Theorem says that this necessary condition is actually sufficient; if the matching condition holds, then a matching exists.

Theorem 11.5.2. *A matching for a set M of men with a set W of women can be found if and only if the matching condition holds.*

Proof. First, let’s suppose that a matching exists and show that the matching condition holds. For any subset of men, each man likes at least the woman he is matched with and a woman is matched with at most one man. Therefore, every subset of men likes at least as large a set of women. Thus, the matching condition holds.

Next, let’s suppose that the matching condition holds and show that a matching exists. We use strong induction on $|M|$, the number of men, on the predicate:

$$P(m) ::= \text{if the matching condition holds for a set, } M, \\ \text{of } m \text{ men, then there is a matching for } M.$$

Base case ($|M| = 1$): If $|M| = 1$, then the matching condition implies that the lone man likes at least one woman, and so a matching exists.

Inductive Step: Suppose that $|M| = m + 1 \geq 2$. To find a matching for M , there are two cases.

Case 1: Every nonempty subset of at most m men likes a *strictly larger* set of women. In this case, we have some latitude: we pair an arbitrary man with a woman he likes and send them both away. This leaves m men and one fewer women, and the matching condition will still hold. So the induction hypothesis $P(m)$ implies we can match the remaining m men.

Case 2: Some nonempty subset, X , of at most m men likes an *equal-size* set, Y , of women. The matching condition must hold within X , so the strong induction hypothesis implies we can match the men in X with the women in Y . This leaves the problem of matching the set $M - X$ of men to the set $W - Y$ of women.

But the problem of matching $M - X$ against $W - Y$ also satisfies the bottleneck condition, because any bottleneck for $M - X$ would imply a bottleneck within the original set of men, M . Namely, if a subset $M_0 \subseteq M - X$ liked only a strictly smaller subset of women $W_0 \subseteq W - Y$, then the set $M_0 \cup X$ of men would like only women in the strictly smaller set $W_0 \cup Y$. So again the strong induction hypothesis implies we can match the men in $M - X$ with the women in $W - Y$, which completes a matching for M .

So in both cases, there is a matching for the men, which completes the proof of the Inductive step. The theorem follows by induction. ■

The proof of Theorem 11.5.2 gives an algorithm for finding a matching in a bipartite graph, albeit not a very efficient one. However, efficient algorithms for finding a matching in a bipartite graph do exist. Thus, if a problem can be reduced to finding a matching, the problem is essentially solved from a computational perspective.

A Formal Statement

Let’s restate Theorem 11.5.2 in abstract terms so that you’ll not always be condemned to saying, “Now this group of men likes at least as many women...”

Definition 11.5.3. A *matching* in a graph G is a set M of edges of G such that no vertex is an endpoint of more than one edge in M . A matching is said to *cover* a set, S , of vertices iff each vertex in S is an endpoint of an edge of the matching. A matching is said to be *perfect* if it covers $V(G)$. In any graph, the set $N(S)$ of *neighbors* of some set S of vertices is the image of S under the edge-relation, that is,

$$N(S) ::= \{ r \mid \langle s-r \rangle \in E(G) \text{ for some } s \in S \}.$$

S is called a *bottleneck* if

$$|S| > |N(S)|.$$

Theorem 11.5.4 (Hall’s Theorem). *Let G be a bipartite graph. There is a matching in G that covers $L(G)$ iff no subset of $L(G)$ is a bottleneck.*

An Easy Matching Condition

The bipartite matching condition requires that *every* subset of men has a certain property. In general, verifying that every subset has some property, even if it’s easy to check any particular subset for the property, quickly becomes overwhelming because the number of subsets of even relatively small sets is enormous—over a billion subsets for a set of size 30. However, there is a simple property of vertex degrees in a bipartite graph that guarantees the existence of a matching. Namely, call a bipartite graph *degree-constrained* if vertex degrees on the left are at least as large as those on the right. More precisely,

Definition 11.5.5. A bipartite graph G is *degree-constrained* when $\deg(l) \geq \deg(r)$ for every $l \in L(G)$ and $r \in R(G)$.

For example, the graph in Figure 11.9 is degree-constrained since every node on the left is adjacent to at least two nodes on the right while every node on the right is adjacent to at most two nodes on the left.

Theorem 11.5.6. *If G is a degree-constrained bipartite graph, then there is a matching that covers $L(G)$.*

Proof. We will show that G satisfies Hall’s condition, namely, if S is an arbitrary subset of $L(G)$, then

$$|N(S)| \geq |S|. \quad (11.2)$$

Since G is degree-constrained, there is a $d > 0$ such that $\deg(l) \geq d \geq \deg(r)$ for every $l \in L$ and $r \in R$. Since every edge with an endpoint in S has its other endpoint in $N(S)$ by definition, and every node in $N(S)$ is incident to at most d edges, we know that

$$d|N(S)| \geq \text{\#edges with an endpoint in } S.$$

Also, since every node in S is the endpoint of at least d edges,

$$\text{\#edges incident to a vertex in } S \geq d|S|.$$

It follows that $d|N(S)| \geq d|S|$. Cancelling d completes the derivation of equation (11.2). ■

Regular graphs are a large class of degree-constrained graphs that often arise in practice. Hence, we can use Theorem 11.5.6 to prove that every regular bipartite graph has a perfect matching. This turns out to be a surprisingly useful result in computer science.

Definition 11.5.7. A graph is said to be *regular* if every node has the same degree.

Theorem 11.5.8. *Every regular bipartite graph has a perfect matching.*

Proof. Let G be a regular bipartite graph. Since regular graphs are degree-constrained, we know by Theorem 11.5.6 that there must be a matching in G that covers $L(G)$. Such a matching is only possible when $|L(G)| \leq |R(G)|$. But G is also degree-constrained if the roles of $L(G)$ and $R(G)$ are switched, which implies that $|R(G)| \leq |L(G)|$ also. That is, $L(G)$ and $R(G)$ are the same size, and any matching covering $L(G)$ will also cover $R(G)$. So every node in G is an endpoint of an edge in the matching, and thus G has a perfect matching. ■

11.6 The Stable Marriage Problem

We next consider a version of the bipartite matching problem where there are an equal number of men and women, and where each person has preferences about who they would like to marry. In fact, we assume that each man has a complete list of all the women ranked according to his preferences, with no ties. Likewise, each woman has a ranked list of all of the men.

The preferences don't have to be symmetric. That is, Jennifer might like Brad best, but Brad doesn't necessarily like Jennifer best. The goal is to marry everyone: every man must marry exactly one woman and vice-versa—no polygamy. Moreover, we would like to find a matching between men and women that is *stable* in the sense that there is no pair of people that prefer each other to their spouses.

For example, suppose *every* man likes Angelina best, and every woman likes Brad best, but Brad and Angelina are married to other people, say Jennifer and Billy Bob. Now *Brad and Angelina prefer each other to their spouses*, which puts their marriages at risk: pretty soon, they're likely to start spending late nights together working on problem sets!

This unfortunate situation is illustrated in Figure 11.11, where the digits “1” and “2” near a man shows which of the two women he ranks first and second, respectively, and similarly for the women.

More generally, in any matching, a man and woman who are not married to each other and who like each other better than their spouses, is called a *rogue couple*. In the situation shown in Figure 11.11, Brad and Angelina would be a rogue couple.

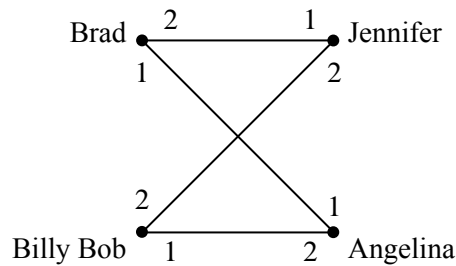


Figure 11.11 Preferences for four people. Both men like Angelina best and both women like Brad best.

Having a rogue couple is not a good thing, since it threatens the stability of the marriages. On the other hand, if there are no rogue couples, then for any man and woman who are not married to each other, at least one likes their spouse better than the other, and so they won’t be tempted to start an affair.

Definition 11.6.1. A *stable matching* is a matching with no rogue couples.

The question is, given everybody’s preferences, how do you find a stable set of marriages? In the example consisting solely of the four people in Figure 11.11, we could let Brad and Angelina both have their first choices by marrying each other. Now neither Brad nor Angelina prefers anybody else to their spouse, so neither will be in a rogue couple. This leaves Jen not-so-happily married to Billy Bob, but neither Jen nor Billy Bob can entice somebody else to marry them, and so there is a stable matching.

Surprisingly, there always is a stable matching among a group of men and women. The surprise springs in part from considering the apparently similar “buddy” matching problem. That is, if people can be paired off as buddies, regardless of gender, then a stable matching *may not* be possible. For example, Figure 11.12 shows a situation with a love triangle and a fourth person who is everyone’s last choice. In this figure Mergatroid’s preferences aren’t shown because they don’t even matter. Let’s see why there is no stable matching.

Lemma 11.6.2. *There is no stable buddy matching among the four people in Figure 11.12.*

Proof. We’ll prove this by contradiction.

Assume, for the purposes of contradiction, that there is a stable matching. Then there are two members of the love triangle that are matched. Since preferences in the triangle are symmetric, we may assume in particular, that Robin and Alex are matched. Then the other pair must be Bobby-Joe matched with Mergatroid.

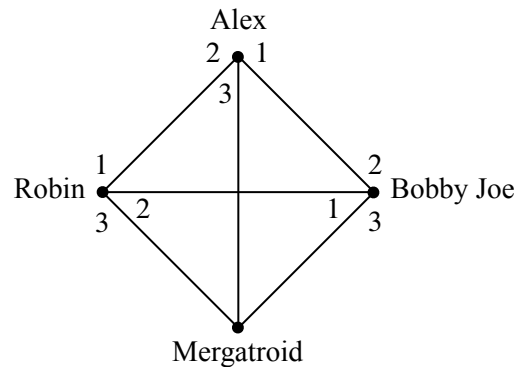


Figure 11.12 Some preferences with no stable buddy matching.

But then there is a rogue couple: Alex likes Bobby-Joe best, and Bobby-Joe prefers Alex to his buddy Mergatroid. That is, Alex and Bobby-Joe are a rogue couple, contradicting the assumed stability of the matching. ■

So getting a stable *buddy* matching may not only be hard, it may be impossible. But when men are only allowed to marry women, and vice versa, then it turns out that a stable matching can always be found.⁶

11.6.1 The Mating Ritual

The procedure for finding a stable matching involves a *Mating Ritual* that takes place over several days. The following events happen each day:

Morning: Each woman stands on her balcony. Each man stands under the balcony of his favorite among the women on his list, and he serenades her. If a man has no women left on his list, he stays home and does his math homework.

Afternoon: Each woman who has one or more suitors serenading her, says to her favorite among them, “We might get engaged. Come back tomorrow.” To the other suitors, she says, “No. I will never marry you! Take a hike!”

Evening: Any man who is told by a woman to take a hike, crosses that woman off his list.

Termination condition: When a day arrives in which every woman has at most one suitor, the ritual ends with each woman marrying her suitor, if she has one.

There are a number of facts about this Mating Ritual that we would like to prove:

- The Ritual eventually reaches the termination condition.

⁶Once again, we disclaim any political statement here—it’s just the way that the math works out.

- Everybody ends up married.
- The resulting marriages are stable.

11.6.2 There is a Marriage Day

It’s easy to see why the Mating Ritual has a terminal day when people finally get married. Every day on which the ritual hasn’t terminated, at least one man crosses a woman off his list. (If the ritual hasn’t terminated, there must be some woman serenaded by at least two men, and at least one of them will have to cross her off his list). If we start with n men and n women, then each of the n men’s lists initially has n women on it, for a total of n^2 list entries. Since no women ever gets added to a list, the total number of entries on the lists decreases every day that the Ritual continues, and so the Ritual can continue for at most n^2 days.

11.6.3 They All Live Happily Ever After...

We still have to prove that the Mating Ritual leaves everyone in a stable marriage. To do this, we note one very useful fact about the Ritual: if a woman has a favorite suitor on some morning of the Ritual, then that favorite suitor will still be serenading her the next morning—because his list won’t have changed. So she is sure to have today’s favorite man among her suitors tomorrow. That means she will be able to choose a favorite suitor tomorrow who is at least as desirable to her as today’s favorite. So day by day, her favorite suitor can stay the same or get better, never worse. This sounds like an invariant, and it is.

Definition 11.6.3. Let P be the predicate: For every woman, w , and every man, m , if w is crossed off m ’s list, then w has a suitor whom she prefers over m .

Lemma 11.6.4. P is an invariant for The Mating Ritual.

Proof. By induction on the number of days.

Base case: In the beginning—that is, at the end of day 0—every woman is on every list. So no one has been crossed off, and P is vacuously true.

Inductive Step: Assume P is true at the end of day d and let w be a woman that has been crossed off a man m ’s list by the end of day $d + 1$.

Case 1: w was crossed off m ’s list on day $d + 1$. Then, w must have a suitor she prefers on day $d + 1$.

Case 2: w was crossed off m ’s list prior to day $d + 1$. Since P is true at the end of day d , this means that w has a suitor she prefers to m on day d . She therefore has the same suitor or someone she prefers better at the end of day $d + 1$.

In both cases, P is true at the end of day $d + 1$ and so P must be an invariant. ■

With Lemma 11.6.4 in hand, we can now prove:

Theorem 11.6.5. *Everyone is married by the Mating Ritual.*

Proof. By contradiction. Assume that it is the last day of the Mating Ritual and someone does not get married. Since there are an equal number of men and women, and since bigamy is not allowed, this means that at least one man (call him Bob) and at least one woman do not get married.

Since Bob is not married, he can't be serenading anybody and so his list must be empty. This means that Bob has crossed every woman off his list and so, by invariant P , every woman has a suitor whom she prefers to Bob. Since it is the last day and every woman still has a suitor, this means that every woman gets married. This is a contradiction since we already argued that at least one woman is *not* married. Hence our assumption must be false and so everyone must be married. ■

Theorem 11.6.6. *The Mating Ritual produces a stable matching.*

Proof. Let Brad and Jen be any man and woman, respectively, that are *not* married to each other on the last day of the Mating Ritual. We will prove that Brad and Jen are not a rogue couple, and thus that all marriages on the last day are stable. There are two cases to consider.

Case 1: Jen is not on Brad's list by the end. Then by invariant P , we know that Jen has a suitor (and hence a husband) that she prefers to Brad. So she's not going to run off with Brad—Brad and Jen cannot be a rogue couple.

Case 2: Jen is on Brad's list. But since Brad is not married to Jen, he must be choosing to serenade his wife instead of Jen, so he must prefer his wife. So he's not going to run off with Jen—once again, Brad and Jen are not a rogue couple. ■

11.6.4 ...Especially the Men

Who is favored by the Mating Ritual, the men or the women? The women *seem* to have all the power: they stand on their balconies choosing the finest among their suitors and spurning the rest. What's more, we know their suitors can only change for the better as the Ritual progresses. Similarly, a man keeps serenading the woman he most prefers among those on his list until he must cross her off, at which point he serenades the next most preferred woman on his list. So from the man's perspective, the woman he is serenading can only change for the worse. Sounds like a good deal for the women.

But it's not! The fact is that from the beginning, the men are serenading their first choice woman, and the desirability of the woman being serenaded decreases only enough to ensure overall stability. The Mating Ritual actually does as well as possible for all the men and does the worst possible job for the women.

To explain all this we need some definitions. Let's begin by observing that while The Mating Ritual produces one stable matching, there may be other stable matchings among the same set of men and women. For example, reversing the roles of men and women will often yield a different stable matching among them.

But some spouses might be out of the question in all possible stable matchings. For example, given the preferences shown in Figure 11.11, Brad is just not in the realm of possibility for Jennifer, since if you ever pair them, Brad and Angelina will form a rogue couple.

Definition 11.6.7. Given a set of preference lists for all men and women, one person is in another person's *realm of possible spouses* if there is a stable matching in which the two people are married. A person's *optimal spouse* is their most preferred person within their realm of possibility. A person's *pessimal spouse* is their least preferred person in their realm of possibility.

Everybody has an optimal and a pessimal spouse, since we know there is at least one stable matching, namely, the one produced by the Mating Ritual. Now here is the shocking truth about the Mating Ritual:

Theorem 11.6.8. *The Mating Ritual marries every man to his optimal spouse.*

Proof. By contradiction. Assume for the purpose of contradiction that some man does not get his optimal spouse. Then there must have been a day when he crossed off his optimal spouse—otherwise he would still be serenading (and would ultimately marry) her or some even more desirable woman.

By the Well Ordering Principle, there must be a *first* day when a man (call him “Keith”) crosses off his optimal spouse (call her Nicole). According to the rules of the Ritual, Keith crosses off Nicole because Nicole has a preferred suitor (call him Tom), so

Nicole prefers Tom to Keith. (*)

Since this is the first day an optimal woman gets crossed off, we know that Tom had not previously crossed off his optimal spouse, and so

Tom ranks Nicole at least as high as his optimal spouse. (**)

By the definition of an optimal spouse, there must be some stable set of marriages in which Keith gets his optimal spouse, Nicole. But then the preferences given in (*) and (**) imply that Nicole and Tom are a rogue couple within this supposedly stable set of marriages (think about it). This is a contradiction. ■

Theorem 11.6.9. *The Mating Ritual marries every woman to her pessimal spouse.*

Proof. Assume for the sake of contradiction that the theorem is not true. Hence there must be a stable set of marriages $\mathcal{M}$ where some woman (call her Nicole) is married to a man (call him Tom) that she likes less than her spouse in The Mating Ritual (call him Keith). This means that

Nicole prefers Keith to Tom. (+)

By Theorem 11.6.8 and the fact that Nicole and Keith are married in the Mating Ritual, we know that

Keith prefers Nicole to his spouse in $\mathcal{M}$. (++)

This means that Keith and Nicole form a rogue couple in $\mathcal{M}$, which contradicts the stability of $\mathcal{M}$. ■

11.6.5 Applications

The Mating Ritual was first announced in a paper by D. Gale and L.S. Shapley in 1962, but ten years before the Gale-Shapley paper was published, and unknown by them, a similar algorithm was being used to assign residents to hospitals by the National Resident Matching Program (NRMP)⁷. The NRMP has, since the turn of the twentieth century, assigned each year’s pool of medical school graduates to hospital residencies (formerly called “internships”) with hospitals and graduates playing the roles of men and women. (In this case, there may be multiple women married to one man, a scenario we consider in the problem section at the end of the chapter.). Before the Ritual-like algorithm was adopted, there were chronic disruptions and awkward countermeasures taken to preserve assignments of graduates to residencies. The Ritual resolved these problems so successfully, that it was used essentially without change at least through 1989.⁸

The Internet infrastructure company, Akamai, also uses a variation of the Mating Ritual to assign web traffic to its servers. In the early days, Akamai used other combinatorial optimization algorithms that got to be too slow as the number of servers (over 65,000 in 2010) and requests (over 800 billion per day) increased. Akamai switched to a Ritual-like approach since it is fast and can be run in a distributed

⁷Of course, there is no serenading going on in the hospitals—the preferences are submitted to a program and the whole process is carried out by a computer.

⁸Much more about the Stable Marriage Problem can be found in the very readable mathematical monograph by Dan Gusfield and Robert W. Irving, [The Stable Marriage Problem: Structure and Algorithms](#), MIT Press, Cambridge, Massachusetts, 1989, 240 pp.

manner. In this case, web requests correspond to women and web servers correspond to men. The web requests have preferences based on latency and packet loss, and the web servers have preferences based on cost of bandwidth and colocation.

Not surprisingly, the Mating Ritual is also used by at least one large online dating agency. Even here, there is no serenading going on—everything is handled by computer.

11.7 Coloring

In Section 11.2, we used edges to indicate an affinity between a pair of nodes. But there are lots of situations where edges will correspond to *conflicts* between nodes. Exam scheduling is a typical example.

11.7.1 An Exam Scheduling Problem

Each term, the MIT Schedules Office must assign a time slot for each final exam. This is not easy, because some students are taking several classes with finals, and (even at MIT) a student can take only one test during a particular time slot. The Schedules Office wants to avoid all conflicts. Of course, you can make such a schedule by having every exam in a different slot, but then you would need hundreds of slots for the hundreds of courses, and the exam period would run all year! So, the Schedules Office would also like to keep exam period short.

The Schedules Office’s problem is easy to describe as a graph. There will be a vertex for each course with a final exam, and two vertices will be adjacent exactly when some student is taking both courses. For example, suppose we need to schedule exams for 6.041, 6.042, 6.002, 6.003 and 6.170. The scheduling graph might appear as in Figure 11.13.

6.002 and 6.042 cannot have an exam at the same time since there are students in both courses, so there is an edge between their nodes. On the other hand, 6.042 and 6.170 can have an exam at the same time if they’re taught at the same time (which they sometimes are), since no student can be enrolled in both (that is, no student *should* be enrolled in both when they have a timing conflict).

We next identify each time slot with a color. For example, Monday morning is red, Monday afternoon is blue, Tuesday morning is green, etc. Assigning an exam to a time slot is then equivalent to coloring the corresponding vertex. The main constraint is that *adjacent vertices must get different colors*—otherwise, some student has two exams at the same time. Furthermore, in order to keep the exam period short, we should try to color all the vertices using as *few different colors as*

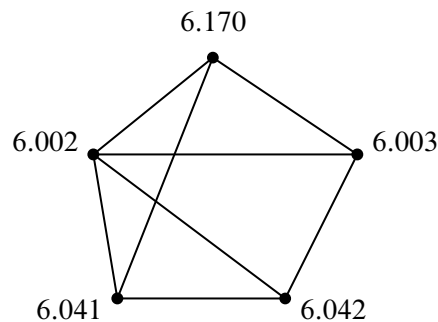


Figure 11.13 A scheduling graph for five exams. Exams connected by an edge cannot be given at the same time.

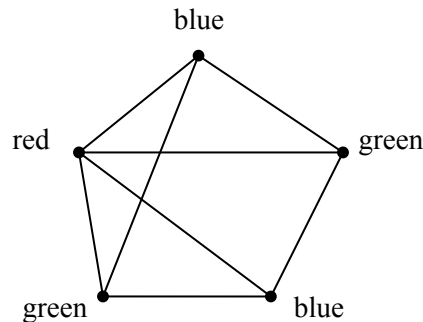


Figure 11.14 A 3-coloring of the exam graph from Figure 11.13.

possible. As shown in Figure 11.14, three colors suffice for our example.

The coloring in Figure 11.14 corresponds to giving one final on Monday morning (red), two Monday afternoon (blue), and two Tuesday morning (green). Can we use fewer than three colors? No! We can’t use only two colors since there is a triangle in the graph, and three vertices in a triangle must all have different colors.

This is an example of a *graph coloring* problem: given a graph G , assign colors to each node such that adjacent nodes have different colors. A color assignment with this property is called a *valid coloring* of the graph—a “*coloring*,” for short. A graph G is *k-colorable* if it has a coloring that uses at most k colors.

Definition 11.7.1. The minimum value of k for which a graph, G , has a valid coloring is called its *chromatic number*, $\chi(G)$.

So G is k -colorable iff $\chi(G) \leq k$.

In general, trying to figure out if you can color a graph with a fixed number of colors can take a long time. It’s a classic example of a problem for which no fast

algorithms are known. In fact, it is easy to check if a coloring works, but it seems really hard to find it. (If you figure out how, then you can get a \$1 million Clay prize.)

11.7.2 Some Coloring Bounds

There are some simple properties of graphs that give useful bounds on colorability. The simplest property is being a cycle: an even-length closed cycle is 2-colorable, and since by definition it must have some edges, it is not 1-colorable. So

$$\chi(C_{\text{even}}) = 2.$$

On the other hand, an odd-length cycle requires 3 colors, that is,

$$\chi(C_{\text{odd}}) = 3. \quad (11.3)$$

You should take a moment to think about why this equality holds. Another simple example is a complete graph K_n :

$$\chi(K_n) = n$$

since no two vertices can have the same color.

Being bipartite is another property closely related to colorability. If a graph is bipartite, then you can color it with 2 colors using one color for the nodes on the “left” and a second color for the nodes on the “right.” Conversely, graphs with chromatic number 2 are all bipartite with all the vertices of one color on the “left” and those with the other color on the right. Since only graphs with no edges—the *empty graphs*—have chromatic number 1, we have:

Lemma 11.7.2. *A graph, G , with at least one edge is bipartite iff $\chi(G) = 2$.*

The chromatic number of a graph can also be shown to be small if the vertex degrees of the graph are small. In particular, if we have an upper bound on the degrees of all the vertices in a graph, then we can easily find a coloring with only one more color than the degree bound.

Theorem 11.7.3. *A graph with maximum degree at most k is $(k + 1)$ -colorable.*

Since k is the only nonnegative integer valued variable mentioned in the theorem, you might be tempted to try to prove this theorem using induction on k . Unfortunately, this approach leads to disaster—we don’t know of any reasonable way to do this and expect it would ruin your week if you tried it on a problem set. When you encounter such a disaster using induction on graphs, it is usually best to change what you are inducting on. In graphs, typical good choices for the induction parameter are n , the number of nodes, or e , the number of edges.

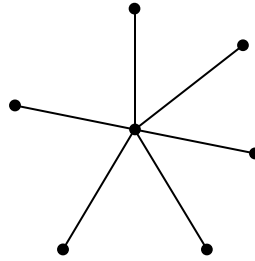


Figure 11.15 A 7-node star graph.

Proof of Theorem 11.7.3. We use induction on the number of vertices in the graph, which we denote by n . Let $P(n)$ be the proposition that an n -vertex graph with maximum degree at most k is $(k + 1)$ -colorable.

Base case ($n = 1$): A 1-vertex graph has maximum degree 0 and is 1-colorable, so $P(1)$ is true.

Inductive step: Now assume that $P(n)$ is true, and let G be an $(n + 1)$ -vertex graph with maximum degree at most k . Remove a vertex v (and all edges incident to it), leaving an n -vertex subgraph, H . The maximum degree of H is at most k , and so H is $(k + 1)$ -colorable by our assumption $P(n)$. Now add back vertex v . We can assign v a color (from the set of $k + 1$ colors) that is different from all its adjacent vertices, since there are at most k vertices adjacent to v and so at least one of the $k + 1$ colors is still available. Therefore, G is $(k + 1)$ -colorable. This completes the inductive step, and the theorem follows by induction. ■

Sometimes $k + 1$ colors is the best you can do. For example, $\chi(K_n) = n$ and every node in K_n has degree $k = n - 1$ and so this is an example where Theorem 11.7.3 gives the best possible bound. By a similar argument, we can show that Theorem 11.7.3 gives the best possible bound for *any* graph with degree bounded by k that has K_{k+1} as a subgraph.

But sometimes $k + 1$ colors is far from the best that you can do. For example, the n -node *star graph* shown in Figure 11.15 has maximum degree $n - 1$ but can be colored using just 2 colors.

11.7.3 Why coloring?

One reason coloring problems frequently arise in practice is because scheduling conflicts are so common. For example, at Akamai, a new version of software is deployed over each of 65,000 servers every few days. The updates cannot be done at the same time since the servers need to be taken down in order to deploy the

software. Also, the servers cannot be handled one at a time, since it would take forever to update them all (each one takes about an hour). Moreover, certain pairs of servers cannot be taken down at the same time since they have common critical functions. This problem was eventually solved by making a 65,000-node conflict graph and coloring it with 8 colors—so only 8 waves of install are needed!

Another example comes from the need to assign frequencies to radio stations. If two stations have an overlap in their broadcast area, they can’t be given the same frequency. Frequencies are precious and expensive, so you want to minimize the number handed out. This amounts to finding the minimum coloring for a graph whose vertices are the stations and whose edges connect stations with overlapping areas.

Coloring also comes up in allocating registers for program variables. While a variable is in use, its value needs to be saved in a register. Registers can be reused for different variables but two variables need different registers if they are referenced during overlapping intervals of program execution. So register allocation is the coloring problem for a graph whose vertices are the variables: vertices are adjacent if their intervals overlap, and the colors are registers. Once again, the goal is to minimize the number of colors needed to color the graph.

Finally, there’s the famous map coloring problem stated in Proposition 1.1.6. The question is how many colors are needed to color a map so that adjacent territories get different colors? This is the same as the number of colors needed to color a graph that can be drawn in the plane without edges crossing. A proof that four colors are enough for *planar* graphs was acclaimed when it was discovered about thirty years ago. Implicit in that proof was a 4-coloring procedure that takes time proportional to the number of vertices in the graph (countries in the map).

Surprisingly, it’s another of those million dollar prize questions to find an efficient procedure to tell if a planar graph really *needs* four colors, or if three will actually do the job. A proof that testing 3-colorability of graphs is as hard as the million dollar SAT problem is given in Problem 11.24; this turns out to be true even for planar graphs. (It is easy to tell if a graph is 2-colorable, as explained in Section 11.10.) In Chapter 12, we’ll develop enough planar graph theory to present an easy proof that all planar graphs are 5-colorable.

11.8 Getting from u to v in a Graph

Walks and paths in simple graphs are essentially the same as in digraphs. We just modify the digraph definitions using undirected edges instead of directed ones. For example, the formal definition of a walk in a simple graph is a virtually that same

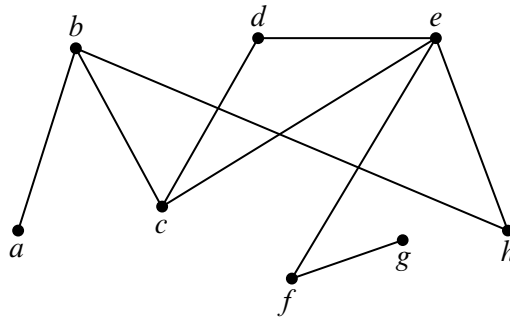


Figure 11.16 A graph with 3 cycles: $bhecb$, $cdec$, $bcdehb$.

as the Definition 9.2.1 of a walk in a digraph:

Definition 11.8.1. A walk in a simple graph, G , is an alternating sequence of vertices and edges that begins with a vertex, ends with a vertex, and such that for every edge $\langle u—v \rangle$ in the walk, one of the endpoints u, v is the element just before the edge, and the other endpoint is the next element after the edge. The *length of a walk* is the total number of occurrences of edges in it.

So a walk, $\mathbf{v}$, is a sequence of the form

$$\mathbf{v} ::= v_0 \langle v_0—v_1 \rangle v_1 \langle v_1—v_2 \rangle v_2 \dots \langle v_{k-1}—v_k \rangle v_k$$

where $\langle v_i—v_{i+1} \rangle \in E(G)$ for $i \in [0, k)$. The walk is said to *start* at v_0 , to *end* at v_k , and the *length*, $|\mathbf{v}|$, of the walk is k . The walk is a *path* iff all the v_i ’s are different, that is, if $i \neq j$, then $v_i \neq v_j$.

A *closed walk* is a walk that begins and ends at the same vertex. A *cycle* is a closed walk of length three or more whose vertices are distinct except for the beginning and end vertices.

Note that a single vertex counts as a length zero path and closed walk. But in contrast to digraphs, a single vertex is not considered to be a cycle.

As in digraphs, the length of a walk is *one less* than the number of occurrences of vertices in it. For example, the graph in Figure 11.16 has a length 6 path through the seven successive vertices $abcdefg$. This is the longest path in the graph. The graph in Figure 11.16 also has three cycles through successive vertices $bhecb$, $cdec$, and $bcdehb$.

11.8.1 Cycles as Subgraphs

A cycle does not really have a beginning or an end, and so can be described by *any* of the paths that go around it. For example, in the graph in Figure 11.16, the cycle

starting at b and going through vertices $bcdehb$ can also be described as starting at d and going through $decbcd$. Furthermore, cycles in simple graphs don't have a direction: $dcbed$ describes the same cycle as though it started and ended at d but went in the opposite direction.

A precise way to explain which closed walks describe the same cycle is to define cycle as a subgraph instead of as a closed walk. Namely, we could define a cycle in G to be a *subgraph* of G that looks like a length- n cycle for $n \geq 3$.

Definition 11.8.2. A graph G is said to be a *subgraph* of a graph H if $V(G) \subseteq V(H)$ and $E(G) \subseteq E(H)$.

For example, the one-edge graph G where

$$V(G) = \{g, h, i\} \quad \text{and} \quad E(G) = \{ \langle h-i \rangle \}$$

is a subgraph of the graph H in Figure 11.1. On the other hand, any graph containing an edge $\langle g-h \rangle$ will not be a subgraph of H because this edge is not in $E(H)$. Another example is an empty graph on n nodes, which will be a subgraph of an L_n with same set of nodes; similarly, L_n is a subgraph of C_n , and C_n is a subgraph of K_n .

Definition 11.8.3. For $n \geq 3$, let C_n be the graph with vertices $1, \dots, n$ and edges

$$\langle 1-2 \rangle, \langle 2-3 \rangle, \dots, \langle (n-1)-n \rangle, \langle n-1 \rangle.$$

A *cycle of a graph*, G , is a subgraph of G that is isomorphic to C_n for some $n \geq 3$.

This definition formally captures the idea that cycles don't have direction or beginnings or ends.

11.9 Connectivity

Definition 11.9.1. Two vertices are *connected* in a graph when there is a path that begins at one and ends at the other. By convention, every vertex is connected to itself by a path of length zero. A *graph is connected* when every pair of vertices are connected.

11.9.1 Connected Components

Being connected is usually a good property for a graph to have. For example, it could mean that it is possible to get from any node to any other node, or that it is possible to communicate between any pair of nodes, depending on the application.

But not all graphs are connected. For example, the graph where nodes represent cities and edges represent highways might be connected for North American cities, but would surely not be connected if you also included cities in Australia. The same is true for communication networks like the Internet—in order to be protected from viruses that spread on the Internet, some government networks are completely isolated from the Internet.

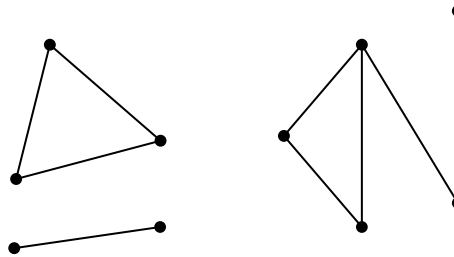


Figure 11.17 One graph with 3 connected components.

Another example, is shown in Figure 11.17, which looks like a picture of three graphs, but is intended to be a picture of *one* graph. This graph consists of three pieces (subgraphs). Each piece by itself is connected, but there are no paths between vertices in different pieces. These connected pieces of a graph are called its *connected components*.

Definition 11.9.2. A *connected component* of a graph is a subgraph consisting of some vertex and every node and edge that is connected to that vertex.

So a graph is connected iff it has exactly one connected component. At the other extreme, the empty graph on n vertices has n connected components.

11.9.2 k -Connected Graphs

If we think of a graph as modeling cables in a telephone network, or oil pipelines, or electrical power lines, then we not only want connectivity, but we want connectivity that survives component failure. So more generally we want to define how strongly two vertices are connected. One measure of connection strength is how many links must fail before connectedness fails. In particular, two vertices are *k -edge connected* when it takes at least k “edge-failures” to disconnect them. More precisely:

Definition 11.9.3. Two vertices in a graph are *k-edge connected* when they remain connected in every subgraph obtained by deleting up to $k - 1$ edges. A graph is *k-edge connected* when it has more than one vertex, and every subgraph obtained by deleting at most $k - 1$ edges is connected.

So two vertices are connected according to Definition 11.9.1 iff they are 1-edge connected according to Definition 11.9.3; likewise for any graph with more than one vertex.

There are other kinds of connectedness but edge-connectedness will be enough for us, so from now on we’ll drop the “edge” modifier and just say “connected.”⁹

For example, in the graph in Figure 11.16, vertices c and e are 3 connected, b and e are 2 connected, g and e are 1 connected, and no vertices are 4 connected. The graph as a whole is only 1 connected. A complete graph, K_n , is $(n - 1)$ connected. Every cycle is 2-connected.

The idea of a *cut edge* is a useful way to explain 2-connectivity.

Definition 11.9.4. If two vertices are connected in a graph G , but not connected when an edge e is removed, then e is called a *cut edge* of G .

So a graph with more than one vertex is 2-connected iff it is connected, and has no cut edges. The following Lemma is another immediate consequence of the definition:

Lemma 11.9.5. *An edge is a cut edge iff it is not on a cycle.*

More generally, if two vertices are connected by k edge-disjoint paths—that is, no edge occurs in two paths—then they must be k connected, since at least one edge will have to be removed from each of the paths before they could disconnect. A fundamental fact, whose ingenious proof we omit, is Menger’s theorem which confirms that the converse is also true: if two vertices are k -connected, then there are k edge-disjoint paths connecting them. It takes some ingenuity to prove this just for the case $k = 2$.

11.9.3 The Minimum Number of Edges in a Connected Graph

The following theorem says that a graph with few edges must have many connected components.

Theorem 11.9.6. *Every graph, G , has at least $|V(G)| - |E(G)|$ connected components.*

⁹There is an obvious definition of k -vertex connectedness based on deleting vertices rather than edges. Graph theory texts usually use “ k -connected” as shorthand for “ k -vertex connected.”

Of course for Theorem 11.9.6 to be of any use, there must be fewer edges than vertices.

Proof. We use induction on the number, k , of edges. Let $P(k)$ be the proposition that

every graph, G , with k edges has at least $|V(G)| - k$ connected components.

Base case ($k = 0$): In a graph with 0 edges, each vertex is itself a connected component, and so there are exactly $|V(G)| = |V(G)| - 0$ connected components. So $P(0)$ holds.

Inductive step:

Let G_e be the graph that results from removing an edge, $e \in E(G)$. So G_e has k edges, and by the induction hypothesis $P(k)$, we may assume that G_e has at least $|V(G)| - k$ connected components. Now add back the edge e to obtain the original graph G . If the endpoints of e were in the same connected component of G_e , then G has the same sets of connected vertices as G_e , so G has at least $|V(G)| - k > |V(G)| - (k + 1)$ components. Alternatively, if the endpoints of e were in different connected components of G_e , then these two components are merged into one component in G , while all other components remain unchanged, so that G has one fewer connected component than G_e . That is, G has at least $(|V(G)| - k) - 1 = |V(G)| - (k + 1)$ connected components. So in either case, G has at least $|V(G)| - (k + 1)$ components, as claimed.

This completes the inductive step and hence the entire proof by induction. ■

Corollary 11.9.7. *Every connected graph with n vertices has at least $n - 1$ edges.*

A couple of points about the proof of Theorem 11.9.6 are worth noticing. First, we used induction on the number of edges in the graph. This is very common in proofs involving graphs, as is induction on the number of vertices. When you’re presented with a graph problem, these two approaches should be among the first you consider.

The second point is more subtle. Notice that in the inductive step, we took an arbitrary $(k + 1)$ -edge graph, threw out an edge so that we could apply the induction assumption, and then put the edge back. You’ll see this shrink-down, grow-back process very often in the inductive steps of proofs related to graphs. This might seem like needless effort: why not start with an k -edge graph and add one more to get an $(k + 1)$ -edge graph? That would work fine in this case, but opens the door to a nasty logical error called *buildup error* illustrated in Problem 11.29.

11.10 Odd Cycles and 2-Colorability

We have already seen that determining the chromatic number of a graph is a challenging problem. There is one special case where this problem is very easy, namely, when the graph is 2-colorable.

Theorem 11.10.1. *The following graph properties are equivalent:*

1. *The graph contains an odd length cycle.*
2. *The graph is not 2-colorable.*
3. *The graph contains an odd length closed walk.*

In other words, if a graph has any one of the three properties above, then it has all of the properties.

We will show the following implications among these properties:

1. IMPLIES 2. IMPLIES 3. IMPLIES 1.

So each of these properties implies the other two, which means they all are equivalent.

1 IMPLIES 2 *Proof.* This follows from equation 11.3. ■

2 IMPLIES 3 If we prove this implication for connected graphs, then it will hold for an arbitrary graph because it will hold for each connected component. So we can assume that G is connected.

Proof. Pick an arbitrary vertex r of G . Since G is connected, for every node $u \in V(G)$, there will be a walk $\mathbf{w}_u$ starting at u and ending at r . Assign colors to vertices of G as follows:

$$\text{color}(u) = \begin{cases} \text{black,} & \text{if } |\mathbf{w}_u| \text{ is even,} \\ \text{white,} & \text{otherwise.} \end{cases}$$

Now since G is not colorable, this can't be a valid coloring. So there must be an edge between two nodes u and v with the same color. But in that case

$$\mathbf{w}_u \hat{\ } \text{reverse}(\mathbf{w}_v) \hat{\ } \langle v \text{---} u \rangle$$

is a closed walk starting and ending at u , and its length is

$$|\mathbf{w}_u| + |\mathbf{w}_v| + 1.$$

This length is odd, since $\mathbf{w}_u$ and $\mathbf{w}_v$ are both even length or are both odd length. ■

3 IMPLIES 1 *Proof.* Since there is an odd length closed walk, the WOP implies there is a odd length closed walk $\mathbf{w}$ of minimum length. We claim $\mathbf{w}$ must be a cycle. To show this, assume to the contrary that there is vertex x that appears twice on the walk, so $\mathbf{w}$ consists of a closed walk from x to x followed by another such walk. That is,

$$\mathbf{w} = \mathbf{f} \hat{x} \mathbf{r}$$

for some positive length walks $\mathbf{f}$ and $\mathbf{r}$ that begin and end at x . Since

$$|\mathbf{w}| = |\mathbf{f}| + |\mathbf{r}|$$

is odd, exactly one of $\mathbf{f}$ and $\mathbf{g}$ must have odd length, and that one will be an odd length closed walk shorter than $\mathbf{w}$, a contradiction. ■

This completes the proof of Theorem 11.10.1.

Theorem 11.10.1 turns out to be useful since bipartite graphs come up fairly often in practice. We'll see examples when we talk about planar graphs in Chapter 12.

11.11 Forests & Trees

We've already made good use of digraphs without cycles, but *simple* graphs without cycles are arguably the most important graphs of all in computer science.

11.11.1 Leaves, Parents & Children

Definition 11.11.1. An acyclic graph is called a *forest*. A connected acyclic graph is called a *tree*.

The graph shown in Figure 11.18 is a forest. Each of its connected components is by definition a tree.

One of the first things you will notice about trees is that they tend to have a lot of nodes with degree one. Such nodes are called *leaves*.

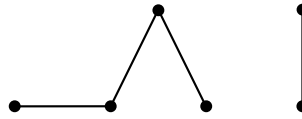


Figure 11.18 A 6-node forest consisting of 2 component trees.

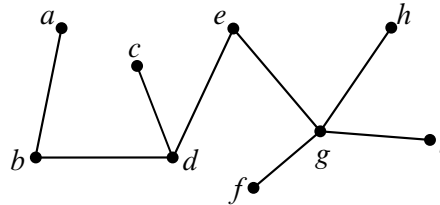


Figure 11.19 A 9-node tree with 5 leaves.

Definition 11.11.2. A degree 1 node in a forest is called a *leaf*.

The forest in Figure 11.18 has 4 leaves. The tree in Figure 11.19 has 5 leaves.

Trees are a fundamental data structure in computer science. For example, information is often stored in tree-like data structures and the execution of many recursive programs can be modeled as the traversal of a tree. In such cases, it is often useful to arrange the nodes in levels, where the node at the top level is identified as the *root* and where every edge joins a *parent* to a *child* one level below. Figure 11.20 shows the tree of Figure 11.19 redrawn in this way. Node *d* is a child of node *e* and the parent of nodes *b* and *c*.

11.11.2 Properties

Trees have many unique properties. We have listed some of them in the following theorem.

Theorem 11.11.3. *Every tree has the following properties:*

1. *Every connected subgraph is a tree.*
2. *There is a unique path between every pair of vertices.*
3. *Adding an edge between nonadjacent nodes in a tree creates a graph with a cycle.*
4. *Removing any edge disconnects the graph. That is, every edge is a cut edge.*
5. *If the tree has at least two vertices, then it has at least two leaves.*

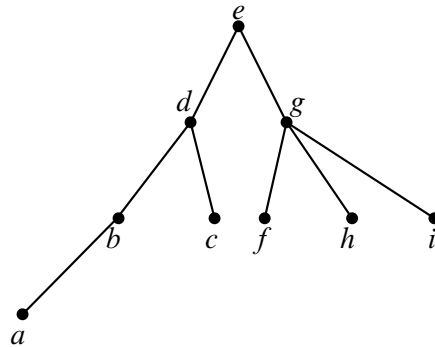


Figure 11.20 The tree from Figure 11.19 redrawn with node e as the root and the other nodes arranged in levels.

6. The number of vertices in a tree is one larger than the number of edges.

Proof. 1. A cycle in a subgraph is also a cycle in the whole graph, so any subgraph of an acyclic graph must also be acyclic. If the subgraph is also connected, then by definition, it is a tree.

2. Since a tree is connected, there is at least one path between every pair of vertices. Suppose for the purposes of contradiction, that there are two different paths between some pair of vertices. Then there are two distinct paths $\mathbf{p} \neq \mathbf{q}$ between the same two vertices with minimum total length $|\mathbf{p}| + |\mathbf{q}|$. If these paths shared a vertex, w , other than at the start and end of the paths, then the parts of $\mathbf{p}$ and $\mathbf{q}$ from start to w , or the parts of $\mathbf{p}$ and $\mathbf{q}$ from w to the end, must be distinct paths between the same vertices with total length less than $|\mathbf{p}| + |\mathbf{q}|$, contradicting the minimality of this sum. Therefore, $\mathbf{p}$ and $\mathbf{q}$ have no vertices in common besides their endpoints, and so $\mathbf{p} \hat{\text{reverse}}(\mathbf{q})$ is a cycle.

3. An additional edge $\langle u-v \rangle$ together with the unique path between u and v forms a cycle.

4. Suppose that we remove edge $\langle u-v \rangle$. Since the tree contained a unique path between u and v , that path must have been $\langle u-v \rangle$. Therefore, when that edge is removed, no path remains, and so the graph is not connected.

5. Since the tree has at least two vertices, the longest path in the tree will have different endpoints u and v . We claim u is a leaf. This follows because, since by definition of endpoint, u is incident to at most one edge on the path.

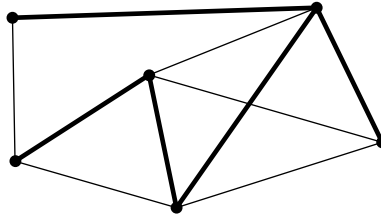


Figure 11.21 A graph where the edges of a spanning tree have been thickened.

Also, If u was incident to an edge not on the path, then the path could be lengthened by adding that edge, contradicting the fact that the path was as long as possible. It follows that u is incident only to a single edge, that is u is a leaf. The same hold for v .

6. We use induction on the proposition

$$P(n) ::= \text{there are } n - 1 \text{ edges in any } n\text{-vertex tree.}$$

Base case ($n = 1$): $P(1)$ is true since a tree with 1 node has 0 edges and $1 - 1 = 0$.

Inductive step: Now suppose that $P(n)$ is true and consider an $(n + 1)$ -vertex tree, T . Let v be a leaf of the tree. You can verify that deleting a vertex of degree 1 (and its incident edge) from any connected graph leaves a connected subgraph. So by Theorem 11.11.3.1, deleting v and its incident edge gives a smaller tree, and this smaller tree has $n - 1$ edges by induction. If we re-attach the vertex, v , and its incident edge, we find that T has $n = (n + 1) - 1$ edges. Hence, $P(n + 1)$ is true, and the induction proof is complete. ■

Various subsets of properties in Theorem 11.11.3 provide alternative characterizations of trees. For example,

Lemma 11.11.4. A graph G is a tree iff G is a forest and $|V(G)| = |E(G)| + 1$.

The proof is an easy consequence of Theorem 11.9.6.6.

11.11.3 Spanning Trees

Trees are everywhere. In fact, every connected graph contains a subgraph that is a tree with the same vertices as the graph. This is called a *spanning tree* for the graph. For example, Figure 11.21 is a connected graph with a spanning tree highlighted.

Definition 11.11.5. Define a *spanning subgraph* of a graph, G , to be a subgraph containing all the vertices of G .

Theorem 11.11.6. *Every connected graph contains a spanning tree.*

Proof. Suppose G is a connected graph, so the graph G itself is a connected, spanning subgraph. So by WOP, G must have a minimum-edge connected, spanning subgraph, T . We claim T is a spanning tree. Since T is a connected, spanning subgraph by definition, all we have to show is that T is acyclic.

But suppose to the contrary that T contained a cycle C . By Lemma 11.9.5, an edge e of C will not be a cut edge, so removing it would leave a connected, spanning subgraph that was smaller than T , contradicting the minimality to T . ■

11.11.4 Minimum Weight Spanning Trees

Spanning trees are interesting because they connect all the nodes of a graph using the smallest possible number of edges. For example the spanning tree for the 6-node graph shown in Figure 11.21 has 5 edges.

Spanning trees are very useful in practice, but in the real world, not all spanning trees are equally desirable. That’s because, in practice, there are often costs associated with the edges of the graph.

For example, suppose the nodes of a graph represent buildings or towns and edges represent connections between buildings or towns. The cost to actually make a connection may vary a lot from one pair of buildings or towns to another. The cost might depend on distance or topography. For example, the cost to connect LA to NY might be much higher than that to connect NY to Boston. Or the cost of a pipe through Manhattan might be more than the cost of a pipe through a cornfield.

In any case, we typically represent the cost to connect pairs of nodes with a weighted edge, where the weight of the edge is its cost. The weight of a spanning tree is then just the sum of the weights of the edges in the tree. For example, the weight of the spanning tree shown in Figure 11.22 is 19.

The goal, of course, is to find the spanning tree with minimum weight, called the minimum weight spanning tree (MST for short).

Definition 11.11.7. A *minimum weight spanning tree* (MST) of an edge-weighted graph G is a spanning tree of G with the smallest possible sum of edge weights.

Is the spanning tree shown in Figure 11.22(a) an MST of the weighted graph shown in Figure 11.22(b)? Actually, it is not, since the tree shown in Figure 11.23 is also a spanning tree of the graph shown in Figure 11.22(b), and this spanning tree has weight 17.

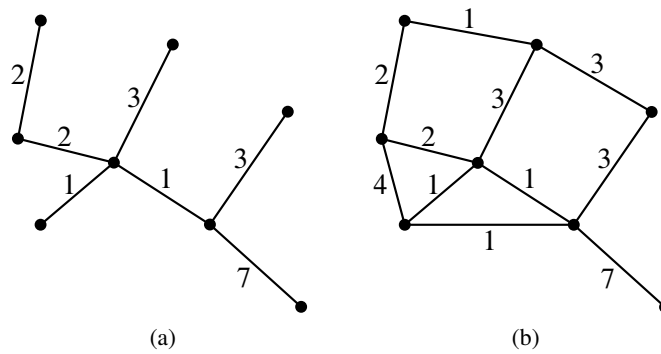


Figure 11.22 A spanning tree (a) with weight 19 for a graph (b).

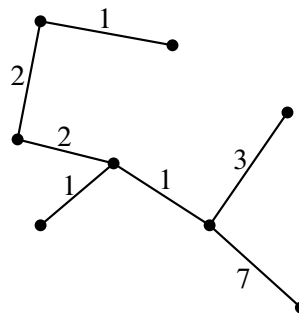


Figure 11.23 An MST with weight 17 for the graph in Figure 11.22(b).

What about the tree shown in Figure 11.23? Is it an MST? It seems to be, but how do we prove it? In general, how do we find an MST for a connected graph G ? We could try enumerating all subtrees of G , but that approach would be hopeless for large graphs.

There actually are many good ways to find MST's based on an invariance property of some subgraphs of G called pre-MST's.

Definition 11.11.8. A *pre-MST* for a graph G is a spanning subgraph of G that is also a subgraph of some MST of G .

So a pre-MST will necessarily be a forest.

For example, the empty graph with the same vertices as G is guaranteed to be a pre-MST of G , and so is any actual MST of G .

If e is an edge of G and S is a spanning subgraph, we'll write $S + e$ for the spanning subgraph with edges $E(S) \cup \{e\}$.

Definition 11.11.9. If F is a pre-MST and e is a new edge, that is $e \in E(G) - E(F)$, then e *extends* F when $F + e$ is also a pre-MST.

So being a pre-MST is by definition an invariant under addition of extending edges.

The standard methods for finding MST's all start with the empty spanning forest and build up to an MST by adding one extending edge after another. Since the empty spanning forest is a pre-MST, and being a pre-MST is invariant under extensions, every forest built in this way will be a pre-MST. But no spanning tree can be a subgraph of a different spanning tree. So when the pre-MST finally grows enough to become a tree, it will be an MST. By Lemma 11.11.4, this happens after exactly $|V(G)| - 1$ edge extensions.

So the problem of finding MST's reduces to the question of how to tell if an edge is an extending edge. Here's how:

Definition 11.11.10. Let F be a pre-MST, and color the vertices in each connected component of F either all black or all white. At least one component of each color is required. Call this a *solid coloring* of F . A *gray edge* of a solid coloring is an edge of G with different colored endpoints.

Any path in G from a white vertex to a black vertex obviously must include a gray edge, so for any solid coloring, there is guaranteed to be at least one gray edge. In fact, there will have to be at least as many gray edges as there are components with the same color. Here's the punchline:

Lemma 11.11.11. An edge extends a pre-MST F if it is a minimum weight gray edge in some solid coloring of F .

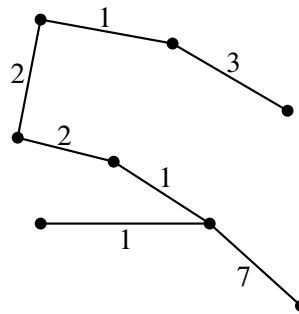


Figure 11.24 A spanning tree found by Algorithm 1.

So to extend a pre-MST, choose any solid coloring, find the gray edges, and among them choose one with minimum weight. Each of these steps is easy to do, so it is easy to keep extending and arrive at an MST. For example, here are three known algorithms that are explained by Lemma 11.11.11:

Algorithm 1. [Prim] Grow a tree one edge at a time by adding a minimum weight edge among the edges that have exactly one endpoint in the tree.

This is the algorithm that comes from coloring the growing tree white and all the vertices not in the tree black. Then the gray edges are the ones with exactly one endpoint in the tree.

Algorithm 2. [Kruskal] Grow a forest one edge at a time by adding a minimum weight edge among the edges with endpoints in different connected components.

The edges between different components are exactly the edges that are gray under some solid coloring, namely any coloring where the components it connects have different colors.

For example, in the weighted graph we have been considering, we might run Algorithm 1 as follows. We would start by choosing one of the weight 1 edges, since this is the smallest weight in the graph. Suppose we chose the weight 1 edge on the bottom of the triangle of weight 1 edges in our graph. This edge is incident to the same vertex as two weight 1 edges, a weight 4 edge, a weight 7 edge, and a weight 3 edge. We would then choose the incident edge of minimum weight. In this case, one of the two weight 1 edges. At this point, we cannot choose the third weight 1 edge: it won't be gray because its endpoints are both in the tree, and so are both colored white. But we can continue by choosing a weight 2 edge. We might end up with the spanning tree shown in Figure 11.24, which has weight 17, the smallest we've seen so far.

Now suppose we instead ran Algorithm 2 on our graph. We might again choose the weight 1 edge on the bottom of the triangle of weight 1 edges in our graph. Now, instead of choosing one of the weight 1 edges it touches, we might choose the weight 1 edge on the top of the graph. This edge still has minimum weight, and will be gray if we simply color its endpoints differently, so Algorithm 2 can choose it. We would then choose one of the remaining weight 1 edges. Note that neither causes us to form a cycle. Continuing the algorithm, we could end up with the same spanning tree in Figure 11.24, though this will depend on how the tie breaking rules used to choose among gray edges with the same minimum weight. For example, if the weight of every edge in G is one, then all spanning trees are MST's with weight $|V(G)| - 1$, and both of these algorithms can arrive at each of these spanning trees by suitable tie-breaking.

The coloring that explains Algorithm 1 also justifies a more flexible algorithm which has Algorithm 1 as a special case:

Algorithm 3. *Grow a forest one edge at a time by picking any component and adding a minimum weight edge among the edges leaving that component.*

This algorithm allows components that are not too close to grow in parallel and independently, which is great for “distributed” computation where separate processors share the work with limited communication between processors.

These are examples of greedy approaches to optimization. Sometimes greediness works and sometimes it doesn't. The good news is that it does work to find the MST. So we can be sure that the MST for our example graph has weight 17 since it was produced by Algorithm 2. And we have a fast algorithm for finding a minimum weight spanning tree for any graph.

Ok, to wrap up this story, all that's left is the proof that minimal gray edges are extending edges. This might sound like a chore, but it just uses the same reasoning we used to be sure there would be a gray edge when you need it.

Proof. (of Lemma 11.11.11)

Let F be a pre-MST that is a subgraph of some MST M of G , and suppose e is a minimum weight gray edge under some solid coloring of F . We want to show that $F + e$ is also a pre-MST.

If e happens to be an edge of M , then $F + e$ remains a subgraph of M , and so is a pre-MST.

The other case is when e is not an edge of M . In that case, $M + e$ will be a connected, spanning subgraph. Also M has a path $\mathbf{p}$ between the different colored endpoints of e , so $M + e$ has a cycle consisting of e together with $\mathbf{p}$. Now $\mathbf{p}$ has both a black endpoint and a white one, so it must contain some gray edge $g \neq e$. The trick is to remove g from $M + e$ to obtain a subgraph $M + e - g$. Since gray

edges by definition are not edges of F , the graph $M + e - g$ contains $F + e$. We claim that $M + e - g$ is an MST, which proves the claim that e extends F .

To prove this claim, note that $M + e$ is a connected, spanning subgraph, and g is on a cycle of $M + e$, so by Lemma 11.9.5, removing g won't disconnect anything. Therefore, $M + e - g$ is still a connected, spanning subgraph. Moreover, $M + e - g$ has the same number of edges as M , so Lemma 11.11.4 implies that it must be a spanning tree. Finally, since e is minimum weight among gray edges,

$$w(M + e - g) = w(M) + w(e) - w(g) \leq w(M).$$

This means that $M + e - g$ is a spanning tree whose weight is at most that of an MST, which implies that $M + e - g$ is also an MST. ■

Another interesting fact falls out of the proof of Lemma 11.11.11:

Corollary 11.11.12. *If all edges in a weighted graph have distinct weights, then the graph has a unique MST.*

The proof of Corollary 11.11.12 is left to Problem 11.42.

Problems for Section 11.2

Class Problems

Problem 11.1. (a) Prove that in every graph, there are an even number of vertices of odd degree.

Hint: The Handshaking Lemma 11.2.1.

(b) Conclude that at a party where some people shake hands, the number of people who shake hands an odd number of times is an even number.

(c) Call a sequence of two or more different people at the party a *handshake sequence* if, except for the last person, each person in the sequence has shaken hands with the next person in the sequence.

Suppose George was at the party and has shaken hands with an odd number of people. Explain why, starting with George, there must be a handshake sequence ending with a different person who has shaken an odd number of hands.

Hint: Just look at the people at the ends of handshake sequences that start with George.

Exam Problems

Problem 11.2.

A researcher analyzing data on heterosexual sexual behavior in a group of m males and f females found that within the group, the male average number of female partners was 10% larger than the female average number of male partners.

(a) Comment on the following claim. “Since we’re assuming that each encounter involves one man and one woman, the average numbers should be the same, so the males must be exaggerating.”

(b) For what constant c is $m = c \cdot f$?

(c) The data shows that approximately 20% of the females were virgins, while only 5% of the males were. The researcher wonders how excluding virgins from the population would change the averages. If he knew graph theory, the researcher would realize that the nonvirgin male average number of partners will be $x(f/m)$ times the nonvirgin female average number of partners. What is x ?

(d) For purposes of further research, it would be helpful to pair each female in the group with a unique male in the group. Explain why this is not possible.

Problems for Section 11.4

Class Problems

Problem 11.3.

For each of the following pairs of graphs, either define an isomorphism between them, or prove that there is none. (We write ab as shorthand for $\langle a-b \rangle$.)

(a)

$$G_1 \text{ with } V_1 = \{1, 2, 3, 4, 5, 6\}, E_1 = \{12, 23, 34, 14, 15, 35, 45\}$$

$$G_2 \text{ with } V_2 = \{1, 2, 3, 4, 5, 6\}, E_2 = \{12, 23, 34, 45, 51, 24, 25\}$$

(b)

$$G_3 \text{ with } V_3 = \{1, 2, 3, 4, 5, 6\}, E_3 = \{12, 23, 34, 14, 45, 56, 26\}$$

$$G_4 \text{ with } V_4 = \{a, b, c, d, e, f\}, E_4 = \{ab, bc, cd, de, ae, ef, cf\}$$

Homework Problems

Problem 11.4.

Determine which among the four graphs pictured in the Figure 11.25 are isomorphic. If two of these graphs are isomorphic, describe an isomorphism between

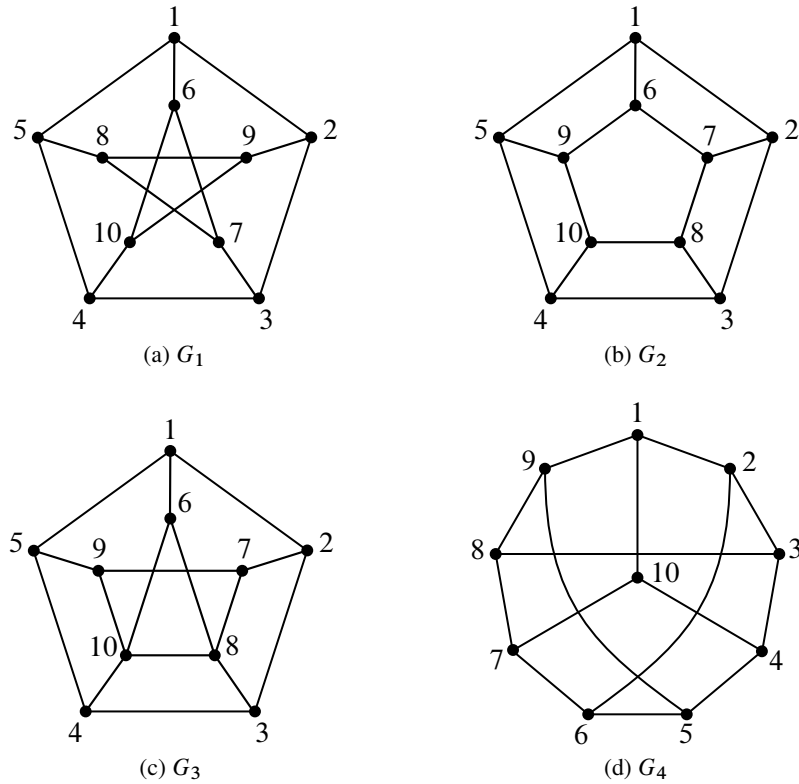


Figure 11.25 Which graphs are isomorphic?

them. If they are not, give a property that is preserved under isomorphism such that one graph has the property, but the other does not. For at least one of the properties you choose, *prove* that it is indeed preserved under isomorphism (you only need prove one of them).

Problem 11.5. (a) For any vertex, v , in a graph, let $N(v)$ be the set of *neighbors* of v , namely, the vertices adjacent to v :

$$N(v) ::= \{u \mid \langle u-v \rangle \text{ is an edge of the graph}\}.$$

Suppose f is an isomorphism from graph G to graph H . Prove that $f(N(v)) = N(f(v))$.

Your proof should follow by simple reasoning using the definitions of isomorphism and neighbors—no pictures or handwaving.

Hint: Prove by a chain of iff’s that

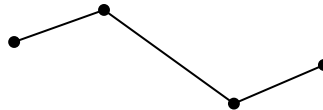
$$h \in N(f(v)) \quad \text{iff} \quad h \in f(N(v))$$

for every $h \in V_H$. Use the fact that $h = f(u)$ for some $u \in V_G$.

(b) Conclude that if G and H are isomorphic graphs, then for each $k \in \mathbb{N}$, they have the same number of degree k vertices.

Problem 11.6.

Let’s say that a graph has “two ends” if it has exactly two vertices of degree 1 and all its other vertices have degree 2. For example, here is one such graph:



(a) A *line graph* is a graph whose vertices can be listed in a sequence with edges between consecutive vertices only. So the two-ended graph above is also a line graph of length 4.

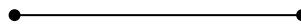
Prove that the following theorem is false by drawing a counterexample.

False Theorem. *Every two-ended graph is a line graph.*

(b) Point out the first erroneous statement in the following bogus proof of the false theorem and describe the error.

Bogus proof. We use induction. The induction hypothesis is that every two-ended graph with n edges is a path.

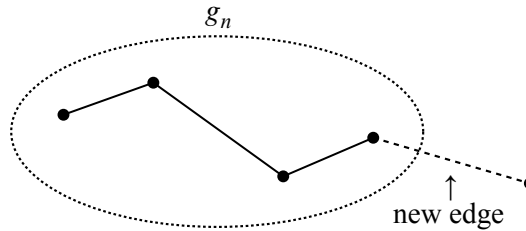
Base case ($n = 1$): The only two-ended graph with a single edge consists of two vertices joined by an edge:



Sure enough, this is a line graph.

Inductive case: We assume that the induction hypothesis holds for some $n \geq 1$ and prove that it holds for $n + 1$. Let G_n be any two-ended graph with n edges. By the induction assumption, G_n is a line graph. Now suppose that we create a

two-ended graph G_{n+1} by adding one more edge to G_n . This can be done in only one way: the new edge must join an endpoint of G_n to a new vertex; otherwise, G_{n+1} would not be two-ended.

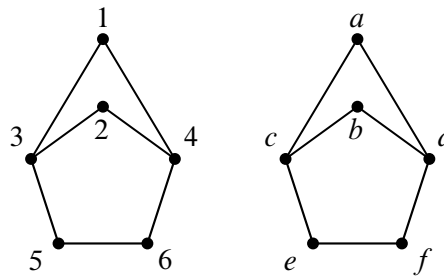


Clearly, G_{n+1} is also a line graph. Therefore, the induction hypothesis holds for all graphs with $n + 1$ edges, which completes the proof by induction. ■

Exam Problems

Problem 11.7.

There are four isomorphisms between these two graphs. List them.



Problems for Section 11.5

Class Problems

Problem 11.8.

A certain Institute of Technology has a lot of student clubs; these are loosely overseen by the Student Association. Each eligible club would like to delegate one of its members to appeal to the Dean for funding, but the Dean will not allow a student to be the delegate of more than one club. Fortunately, the Association VP took Math for Computer Science and recognizes a matching problem when she sees one.

(a) Explain how to model the delegate selection problem as a bipartite matching problem.

(b) The VP’s records show that no student is a member of more than 9 clubs. The VP also knows that to be eligible for support from the Dean’s office, a club must have at least 13 members. That’s enough for her to guarantee there is a proper delegate selection. Explain. (If only the VP had taken an *Algorithms*, she could even have found a delegate selection without much effort.)

Problem 11.9.

A *Latin square* is $n \times n$ array whose entries are the number $1, \dots, n$. These entries satisfy two constraints: every row contains all n integers in some order, and also every column contains all n integers in some order. Latin squares come up frequently in the design of scientific experiments for reasons illustrated by a little story in a footnote¹⁰

For example, here is a 4×4 Latin square:

1	2	3	4
3	4	2	1
2	1	4	3
4	3	1	2

(a) Here are three rows of what could be part of a 5×5 Latin square:

¹⁰At Guinness brewery in the early 1900’s, W. S. Gosset (a chemist) and E. S. Beavan (a “maltster”) were trying to improve the barley used to make the brew. The brewery used different varieties of barley according to price and availability, and their agricultural consultants suggested a different fertilizer mix and best planting month for each variety.

Somewhat sceptical about paying high prices for customized fertilizer, Gosset and Beavan planned a season long test of the influence of fertilizer and planting month on barley yields. For as many months as there were varieties of barley, they would plant one sample of each variety using a different one of the fertilizers. So every month, they would have all the barley varieties planted and all the fertilizers used, which would give them a way to judge the overall quality of that planting month. But they also wanted to judge the fertilizers, so they wanted each fertilizer to be used on each variety during the course of the season. Now they had a little mathematical problem, which we can abstract as follows.

Suppose there are n barley varieties and an equal number of recommended fertilizers. Form an $n \times n$ array with a column for each fertilizer and a row for each planting month. We want to fill in the entries of this array with the integers $1, \dots, n$ numbering the barley varieties, so that every row contains all n integers in some order (so every month each variety is planted and each fertilizer is used), and also every column contains all n integers (so each fertilizer is used on all the varieties over the course of the growing season).

2	4	5	3	1
4	1	3	2	5
3	2	1	5	4

Fill in the last two rows to extend this “Latin rectangle” to a complete Latin square.

(b) Show that filling in the next row of an $n \times n$ Latin rectangle is equivalent to finding a matching in some $2n$ -vertex bipartite graph.

(c) Prove that a matching must exist in this bipartite graph and, consequently, a Latin rectangle can always be extended to a Latin square.

Exam Problems

Problem 11.10.

Overworked and over-caffeinated, the Teaching Assistant’s (TA’s) decide to oust the lecturer and teach their own recitations. They will run a recitation session at 4 different times in the same room. There are exactly 20 chairs to which a student can be assigned in each recitation. Each student has provided the TA’s with a list of the recitation sessions her schedule allows and no student’s schedule conflicts with all 4 sessions. The TA’s must assign each student to a chair during recitation at a time she can attend, if such an assignment is possible.

Describe how to model this situation as a matching problem. Be sure to specify what the vertices/edges should be and briefly describe how a matching would determine seat assignments for each student in a recitation that does not conflict with his schedule. This is a *modeling problem* —you need not determine whether a match is always possible.

Problem 11.11.

Because of the incredible popularity of Math for Computer Science, Rajeev decides to give up on regular office hours. Instead, each student can join some study groups. Each group must choose a representative to talk to the staff, but there is a staff rule that a student can only represent one group. The problem is to find a representative from each group while obeying the staff rule.

(a) Explain how to model the delegate selection problem as a bipartite matching problem.

(b) The staff’s records show that no student is a member of more than 4 groups, and all the groups must have at least 4 members. That’s enough to guarantee there is a proper delegate selection. Explain.

Homework Problems

Problem 11.12.

Take a regular deck of 52 cards. Each card has a suit and a value. The suit is one of four possibilities: heart, diamond, club, spade. The value is one of 13 possibilities, $A, 2, 3, \dots, 10, J, Q, K$. There is exactly one card for each of the 4×13 possible combinations of suit and value.

Ask your friend to lay the cards out into a grid with 4 rows and 13 columns. They can fill the cards in any way they’d like. In this problem you will show that you can always pick out 13 cards, one from each column of the grid, so that you wind up with cards of all 13 possible values.

(a) Explain how to model this trick as a bipartite matching problem between the 13 column vertices and the 13 value vertices. Is the graph necessarily degree-constrained?

(b) Show that any n columns must contain at least n different values and prove that a matching must exist.

Problem 11.13.

Scholars through the ages have identified *twenty* fundamental human virtues: honesty, generosity, loyalty, prudence, completing the weekly course reading-response, etc. At the beginning of the term, every student in Math for Computer Science possessed exactly *eight* of these virtues. Furthermore, every student was unique; that is, no two students possessed exactly the same set of virtues. The Math for Computer Science course staff must select *one* additional virtue to impart to each student by the end of the term. Prove that there is a way to select an additional virtue for each student so that every student is unique at the end of the term as well.

Suggestion: Use Hall’s theorem. Try various interpretations for the vertices on the left and right sides of your bipartite graph.

Problems for Section 11.6

Practice Problems

Problem 11.14.

Four Students want separate assignments to four VI-A Companies. Here are their

preference rankings:

Student	Companies
Albert:	HP, Bellcore, AT&T, Draper
Nick:	AT&T, Bellcore, Draper, HP
Oshani:	HP, Draper, AT&T, Bellcore
Ali:	Draper, AT&T, Bellcore, HP

Company	Students
AT&T:	Ali, Albert, Oshani, Nick
Bellcore:	Oshani, Nick, Albert, Ali
HP:	Ali, Oshani, Albert, Nick
Draper:	Nick, Ali, Oshani, Albert

(a) Use the Mating Ritual to find *two* stable assignments of Students to Companies.

(b) Describe a simple procedure to determine whether any given stable marriage problem has a unique solution, that is, only one possible stable matching.

Problem 11.15.

We are interested in invariants of the Mating Ritual (Section 11.6) for finding stable marriages. Let Angelina and Jen be two of the girls, and Keith and Tom be two of the boys.

Which of the following predicates are invariants of the Mating Ritual no matter what the preferences are among the boys and girls? (Remember that a predicate that is always false is an invariant—check the definition of invariant to see why.)

- (a) Angelina is crossed off Tom’s list and she has a suitor that she prefers to Tom.
- (b) Tom is serenading Jen.
- (c) Tom is not serenading Jen.
- (d) Tom’s list of girls to serenade is empty.
- (e) All the boys have the same number of girls left uncrossed in their lists.
- (f) Jen is crossed off Keith’s list.
- (g) Jen is crossed off Keith’s list and Keith prefers Jen to anyone he is serenading.
- (h) Jen is the only girl on Keith’s list.

Class Problems

Problem 11.16.

Consider a stable marriage problem with 4 boys and 4 girls and the following partial information about their preferences:

B1:	G1	G2	–	–
B2:	G2	G1	–	–
B3:	–	–	G4	G3
B4:	–	–	G3	G4
G1:	B2	B1	–	–
G2:	B1	B2	–	–
G3:	–	–	B3	B4
G4:	–	–	B4	B3

(a) Verify that

$$(B1, G1), (B2, G2), (B3, G3), (B4, G4)$$

will be a stable matching whatever the unspecified preferences may be.

(b) Explain why the stable matching above is neither boy-optimal nor boy-pessimal and so will not be an outcome of the Mating Ritual.

(c) Describe how to define a set of marriage preferences among n boys and n girls which have at least $2^{n/2}$ stable assignments.

Hint: Arrange the boys into a list of $n/2$ pairs, and likewise arrange the girls into a list of $n/2$ pairs of girls. Choose preferences so that the k th pair of boys ranks the k th pair of girls just below the previous pairs of girls, and likewise for the k th pair of girls. Within the k th pairs, make sure each boy’s first choice girl in the pair prefers the other boy in the pair.

Problem 11.17.

Suppose there are more boys than girls.

(a) Define what a stable matching should mean in this case.

(b) Explain why applying the Mating Ritual in this case will yield a stable matching in which every girl is married.

Homework Problems

Problem 11.18.

The most famous application of stable matching was in assigning graduating medical students to hospital residencies. Each hospital has a preference ranking of students and each student has a preference order of hospitals, but unlike the setup in the notes where there are an equal number of boys and girls and monogamous marriages, hospitals generally have differing numbers of available residencies, and the total number of residencies may not equal the number of graduating students. Modify the definition of stable matching so it applies in this situation, and explain how to modify the Mating Ritual so it yields stable assignments of students to residencies.

Briefly indicate what, if any, modifications of the preserved invariant used to verify the original Mating are needed to verify this one for hospitals and students.

Problem 11.19.

Give an example of a stable matching between 3 boys and 3 girls where no person gets their first choice. Briefly explain why your matching is stable.

Problem 11.20.

In a stable matching between n boys and girls produced by the Mating Ritual, call a person *lucky* if they are matched up with one of their $\lceil n/2 \rceil$ top choices. We will prove:

Theorem. *There must be at least one lucky person.*

To prove this, define the following derived variables for the Mating Ritual:

$q(B) = j$, where j is the rank of the girl that boy B is courting. That is to say, boy B is always courting the j th girl on his list.

$r(G)$ is the number of boys that girl G has rejected.

(a) Let

$$S ::= \sum_{B \in \text{Boys}} q(B) - \sum_{G \in \text{Girls}} r(G). \quad (11.4)$$

Show that S remains the same from one day to the next in the Mating Ritual.

(b) Prove the Theorem above. (You may assume for simplicity that n is even.)

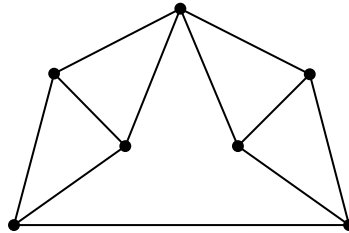
Hint: A girl is sure to be lucky if she has rejected half the boys.

Problems for Section 11.7

Class Problems

Problem 11.21.

Let G be the graph below¹¹. Carefully explain why $\chi(G) = 4$.



Homework Problems

Problem 11.22.

6.042 is often taught using recitations. Suppose it happened that 8 recitations were needed, with two or three staff members running each recitation. The assignment of staff to recitation sections, using their secret codenames, is as follows:

- R1: Maverick, Goose, Iceman
- R2: Maverick, Stinger, Viper
- R3: Goose, Merlin
- R4: Slider, Stinger, Cougar
- R5: Slider, Jester, Viper
- R6: Jester, Merlin
- R7: Jester, Stinger
- R8: Goose, Merlin, Viper

Two recitations can not be held in the same 90-minute time slot if some staff member is assigned to both recitations. The problem is to determine the minimum number of time slots required to complete all the recitations.

¹¹From *Discrete Mathematics*, Lovász, Pelikan, and Vesztergombi. Springer, 2003. Exercise 13.3.1

- (a) Recast this problem as a question about coloring the vertices of a particular graph. Draw the graph and explain what the vertices, edges, and colors represent.
- (b) Show a coloring of this graph using the fewest possible colors. What schedule of recitations does this imply?

Problem 11.23.

This problem generalizes the result proved Theorem 11.7.3 that any graph with maximum degree at most w is $(w + 1)$ -colorable.

A simple graph, G , is said to have *width*, w , iff its vertices can be arranged in a sequence such that each vertex is adjacent to at most w vertices that precede it in the sequence. If the degree of every vertex is at most w , then the graph obviously has width at most w —just list the vertices in any order.

- (a) Describe an example of a graph with 100 vertices, width 3, but *average* degree more than 5. *Hint:* Don’t get stuck on this; if you don’t see it after five minutes, ask for a hint.
- (b) Prove that every graph with width at most w is $(w + 1)$ -colorable.
- (c) Prove that the average degree of a graph of width w is at most $2w$.

Problem 11.24.

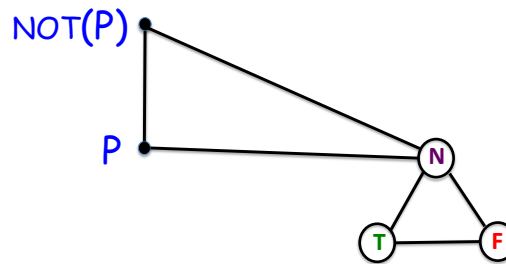
This problem will show that 3-coloring a graph is just as difficult as finding a satisfying truth assignment for a propositional formula. The graphs considered will all be taken to have three designated *color-vertices* connected in a triangle to force them to have different colors in any coloring of the graph. The colors assigned to the color-vertices will be called T , F and N .

Suppose f is an n -argument truth function. That is,

$$f : \{T, F\}^n \rightarrow \{T, F\}.$$

A graph G is called a *3-color- f -gate* iff G has n designated *input vertices* and a designated *output vertex*, such that

- G can be 3-colored *only* if its input vertices are colored with T ’s and F ’s.
- For every sequence $b_1, b_2, \dots, b_n \in \{T, F\}$, there is a 3-coloring of G in which the input vertices $v_1, v_2, \dots, v_n \in V(G)$ have the colors $b_1, b_2, \dots, b_n \in \{T, F\}$.



[h]

Figure 11.26 A 3-coloring NOT-gate

- In any 3-coloring of G where the input vertices $v_1, v_2, \dots, v_n \in V(G)$ have colors $b_1, b_2, \dots, b_n \in \{T, F\}$, the output vertex has color $f(b_1, b_2, \dots, b_n)$.

For example, a 3-color-NOT-gate consists simply of two adjacent vertices. One vertex is designated to be the input vertex, P , and the other is designated to be the output vertex. Both vertices have to be constrained so they can only be colored with T 's or F 's in any proper 3-coloring. This constraint can be imposed by making them adjacent to the color-vertex N , as shown in Figure 11.26.

(a) Verify that the graph in Figure 11.27 is a 3-color-OR-gate. (Color-vertex F and the edges among the coloring vertices are not shown. Also not shown are edges from each of the input vertices P and Q to color-vertex N ; these edges constrain P and Q to be colored T or F in any proper 3-coloring.)

(b) Let E be an n -variable propositional formula, and suppose E defines a truth function $f : \{T, F\}^n \rightarrow \{T, F\}$. Explain a simple way to construct a graph that is a 3-color- f -gate.

(c) Explain why an efficient procedure for determining if a graph was 3-colorable would lead to an efficient procedure to solve the satisfiability problem, SAT.

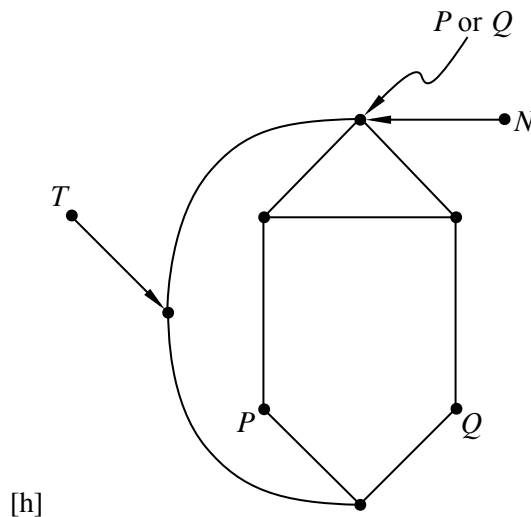


Figure 11.27 A 3-coloring OR-gate

Exam Problems

Problem 11.25.

False Claim. Let G be a graph whose vertex degrees are all $\leq k$. If G has a vertex of degree strictly less than k , then G is k -colorable.

- (a) Give a counterexample to the False Claim when $k = 2$.
- (b) Underline the exact sentence or part of a sentence that is the first unjustified step in the following bogus proof of the False Claim.

Bogus proof. Proof by induction on the number n of vertices:

Induction hypothesis:

$P(n) ::=$ “Let G be an n -vertex graph whose vertex degrees are all $\leq k$. If G also has a vertex of degree strictly less than k , then G is k -colorable.”

Base case: ($n = 1$) G has one vertex, the degree of which is 0. Since G is 1-colorable, $P(1)$ holds.

Inductive step:

We may assume $P(n)$. To prove $P(n + 1)$, let G_{n+1} be a graph with $n + 1$ vertices whose vertex degrees are all k or less. Also, suppose G_{n+1} has a vertex, v , of degree strictly less than k . Now we only need to prove that G_{n+1} is k -colorable.

To do this, first remove the vertex v to produce a graph, G_n , with n vertices. Let u be a vertex that is adjacent to v in G_{n+1} . Removing v reduces the degree of u by 1. So in G_n , vertex u has degree strictly less than k . Since no edges were added, the vertex degrees of G_n remain $\leq k$. So G_n satisfies the conditions of the induction hypothesis, $P(n)$, and so we conclude that G_n is k -colorable.

Now a k -coloring of G_n gives a coloring of all the vertices of G_{n+1} , except for v . Since v has degree less than k , there will be fewer than k colors assigned to the nodes adjacent to v . So among the k possible colors, there will be a color not used to color these adjacent nodes, and this color can be assigned to v to form a k -coloring of G_{n+1} . ■

(c) With a slightly strengthened condition, the preceding proof of the False Claim could be revised into a sound proof of the following Claim:

Claim. *Let G be a graph whose vertex degrees are all $\leq k$. If $\langle$ statement inserted from below $\rangle$ has a vertex of degree strictly less than k , then G is k -colorable.*

Circle each of the statements below that could be inserted to make the proof correct.

- G is connected and
- G has no vertex of degree zero and
- G does not contain a complete graph on k vertices and
- every connected component of G
- some connected component of G

Problems for Section 11.9

Class Problems

Problem 11.26.

The n -dimensional hypercube, H_n , is a graph whose vertices are the binary strings of length n . Two vertices are adjacent if and only if they differ in exactly 1 bit. For example, in H_3 , vertices 111 and 011 are adjacent because they differ only in the first bit, while vertices 101 and 011 are not adjacent because they differ at both the first and second bits.

(a) Prove that it is impossible to find two spanning trees of H_3 that do not share some edge.

(b) Verify that for any two vertices $x \neq y$ of H_3 , there are 3 paths from x to y in H_3 , such that, besides x and y , no two of those paths have a vertex in common.

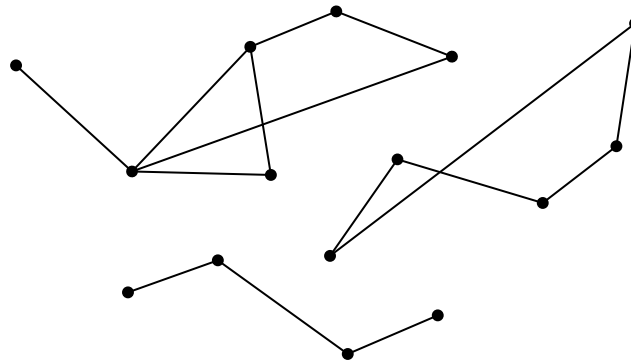
(c) Conclude that the connectivity of H_3 is 3.

(d) Try extending your reasoning to H_4 . (In fact, the connectivity of H_n is n for all $n \geq 1$. A proof appears in the problem solution.)

Problem 11.27.

A set, M , of vertices of a graph is a *maximal connected set* if every pair of vertices in the set are connected, and any set of vertices properly containing M will contain two vertices that are not connected.

(a) What are the maximal connected subsets of the following (unconnected) graph?



(b) Explain the connection between maximal connected sets and connected components. Prove it.

Problem 11.28. (a) Prove that K_n is $(n - 1)$ -edge connected for $n > 1$.

Let M_n be a graph defined as follows: begin by taking n graphs with non-overlapping sets of vertices, where each of the n graphs is $(n - 1)$ -edge connected (they could be disjoint copies of K_n , for example). These will be subgraphs of M_n . Then pick n vertices, one from each subgraph, and add enough edges between pairs of picked vertices that the subgraph of the n picked vertices is also $(n - 1)$ -edge connected.

(b) Draw a picture of M_4 .

(c) Explain why M_n is $(n - 1)$ -edge connected.

Problem 11.29.

False Claim. *If every vertex in a graph has positive degree, then the graph is connected.*

(a) Prove that this Claim is indeed false by providing a counterexample.

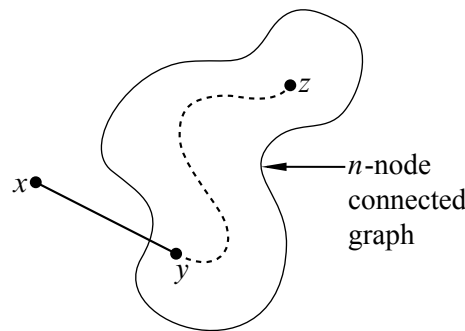
(b) Since the Claim is false, there must be a logical mistake in the following bogus proof. Pinpoint the *first* logical mistake (unjustified step) in the proof.

Bogus proof. We prove the Claim above by induction. Let $P(n)$ be the proposition that if every vertex in an n -vertex graph has positive degree, then the graph is connected.

Base cases: ($n \leq 2$). In a graph with 1 vertex, that vertex cannot have positive degree, so $P(1)$ holds vacuously.

$P(2)$ holds because there is only one graph with two vertices of positive degree, namely, the graph with an edge between the vertices, and this graph is connected.

Inductive step: We must show that $P(n)$ implies $P(n+1)$ for all $n \geq 2$. Consider an n -vertex graph in which every vertex has positive degree. By the assumption $P(n)$, this graph is connected; that is, there is a path between every pair of vertices. Now we add one more vertex x to obtain an $(n+1)$ -vertex graph:



All that remains is to check that there is a path from x to every other vertex z . Since x has positive degree, there is an edge from x to some other vertex, y . Thus, we can obtain a path from x to z by going from x to y and then following the path from y to z . This proves $P(n+1)$.

By the principle of induction, $P(n)$ is true for all $n \geq 0$, which proves the Claim. ■

Homework Problems

Problem 11.30. (a) Give an example of a simple graph that has two vertices $u \neq v$ and two distinct paths between u and v , but no cycle including either u or v .

(b) Prove that if there are different paths between two vertices in a simple graph, then the graph has a cycle.

Problem 11.31.

The entire field of graph theory began when Euler asked whether the seven bridges of Königsberg could all be crossed exactly once. Abstractly, we can represent the parts of the city separated by rivers as vertices and the bridges as edges between the vertices. Then Euler’s question asks whether there is a closed walk through the graph that includes every edge in a graph exactly once. In his honor, such a walk is called an *Euler tour*.

So how do you tell in general whether a graph has an Euler tour? At first glance this may seem like a daunting problem. The similar sounding problem of finding a cycle that touches every vertex exactly once is one of those Millenium Prize NP-complete problems known as the *Traveling Salesman Problem*). But it turns out to be easy to characterize which graphs have Euler tours.

Theorem. *A connected graph has an Euler tour if and only if every vertex has even degree.*

(a) Show that if a graph has an Euler tour, then the degree of each of its vertices is even.

In the remaining parts, we’ll work out the converse: if the degree of every vertex of a connected finite graph is even, then it has an Euler tour. To do this, let’s define an *Euler walk* to be a walk that includes each edge *at most* once.

(b) Suppose that an Euler walk in a connected graph does not include every edge. Explain why there must be an unincluded edge that is incident to a vertex on the walk.

In the remaining parts, let $\mathbf{w}$ be the *longest* Euler walk in some finite, connected graph.

(c) Show that if $\mathbf{w}$ is a closed walk, then it must be an Euler tour.

Hint: part (b)

(d) Explain why all the edges incident to the end of $\mathbf{w}$ must already be in $\mathbf{w}$.

(e) Show that if the end of w was not equal to the start of w , then the degree of the end would be odd.

Hint: part (d)

(f) Conclude that if every vertex of a finite, connected graph has even degree, then it has an Euler tour.

Homework Problems

Problem 11.32.

An edge is said to *leave* a set of vertices if one end of the edge is in the set and the other end is not.

(a) An n -node graph is said to be *mangled* if there is an edge leaving every set of $\lfloor n/2 \rfloor$ or fewer vertices. Prove the following:

Claim. *Every mangled graph is connected.*

An n -node graph is said to be *tangled* if there is an edge leaving every set of $\lceil n/3 \rceil$ or fewer vertices.

(b) Draw a tangled graph that is not connected.

(c) Find the error in the bogus proof of the following

False Claim. *Every tangled graph is connected.*

Bogus proof. The proof is by strong induction on the number of vertices in the graph. Let $P(n)$ be the proposition that if an n -node graph is tangled, then it is connected. In the base case, $P(1)$ is true because the graph consisting of a single node is trivially connected.

For the inductive case, assume $n \geq 1$ and $P(1), \dots, P(n)$ hold. We must prove $P(n+1)$, namely, that if an $(n+1)$ -node graph is tangled, then it is connected.

So let G be a tangled, $(n+1)$ -node graph. Choose $\lceil n/3 \rceil$ of the vertices and let G_1 be the tangled subgraph of G with these vertices and G_2 be the tangled subgraph with the rest of the vertices. Note that since $n \geq 1$, the graph G has at least two vertices, and so both G_1 and G_2 contain at least one vertex. Since G_1 and G_2 are tangled, we may assume by strong induction that both are connected. Also, since G is tangled, there is an edge leaving the vertices of G_1 which necessarily connects to a vertex of G_2 . This means there is a path between any two vertices of G : a path within one subgraph if both vertices are in the same subgraph, and a path traversing the connecting edge if the vertices are in separate subgraphs. Therefore, the entire graph, G , is connected. This completes the proof of the inductive case, and the Claim follows by strong induction.



Problem 11.33.

Let G be the graph formed from C_{2n} , the cycle of length $2n$, by connecting every pair of vertices at maximum distance from each other in C_{2n} by an edge in G .

- (a) Given two vertices of G find their distance in G .
- (b) What is the *diameter* of G , that is, the largest distance between two vertices?
- (c) Prove that the graph is not 4-connected.
- (d) Prove that the graph is 3-connected.

Problems for Section 11.11

Practice Problems

Problem 11.34. (a) Prove that the average degree of a tree is less than 2.

(b) Suppose every vertex in a graph has degree at least k . Explain why the graph has a path of length k .

Hint: Consider a longest path.

Exam Problems

Problem 11.35.

The n -dimensional hypercube, H_n , is a simple graph whose vertices are the binary strings of length n . Two vertices are adjacent if and only if they differ in exactly one bit. Consider for example H_3 , shown in Figure 11.28. (Here, vertices 111 and 011 are adjacent because they differ only in the first bit, while vertices 101 and 011 are not adjacent because they differ in both the first and second bits.)

Explain why it is impossible to find two spanning trees of H_3 that have no edges in common.

Class Problems

Problem 11.36.

Procedure *Mark* starts with a connected, simple graph with all edges unmarked and then marks some edges. At any point in the procedure a path that includes only marked edges is called a *fully marked* path, and an edge that has no fully marked path between its endpoints is called *eligible*.

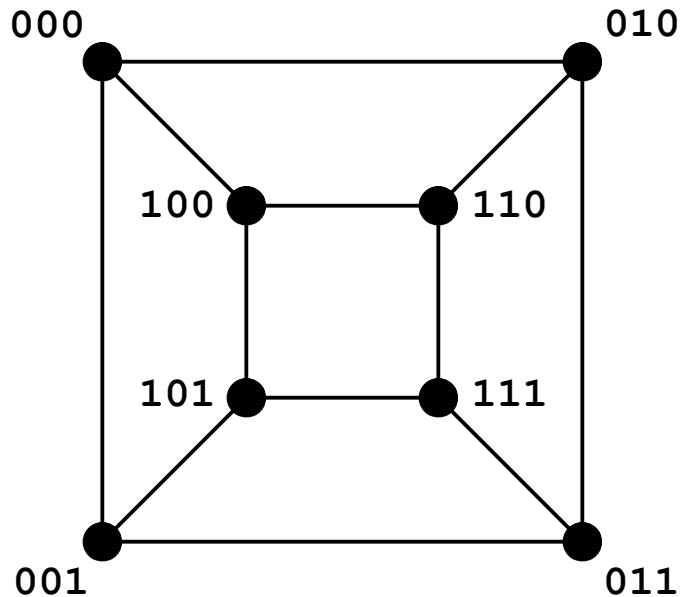


Figure 11.28 H_3 .

Procedure *Mark* simply keeps marking eligible edges, and terminates when there are none.

Prove that *Mark* terminates, and that when it does, the set of marked edges forms a spanning tree of the original graph.

Problem 11.37.

A procedure for connecting up a (possibly disconnected) simple graph and creating a spanning tree can be modelled as a state machine whose states are finite simple graphs. A state is *final* when no further transitions are possible. The transitions determined by the following rules:

Procedure create-spanning-tree

1. If there is an edge $\langle u-v \rangle$ on a cycle, then delete $\langle u-v \rangle$.
2. If vertices u and v are not connected, then add the edge $\langle u-v \rangle$.

(a) Draw all the possible final states reachable starting with the graph with vertices

$\{1, 2, 3, 4\}$ and edges

$$\{\langle 1-2 \rangle, \langle 3-4 \rangle\}.$$

(b) Prove that if the machine reaches a final state, then the final state will be a tree on the vertices graph on which it started.

(c) For any graph, G' , let e be the number of edges in G' , c be the number of connected components it has, and s be the number of cycles. For each of the quantities below, indicate the *strongest* of the properties that it is guaranteed to satisfy, no matter what the starting graph is.

The choices for properties are: *constant*, *strictly increasing*, *strictly decreasing*, *weakly increasing*, *weakly decreasing*, *none of these*.

- (i) e
- (ii) c
- (iii) s
- (iv) $e - s$
- (v) $c + e$
- (vi) $3c + 2e$
- (vii) $c + s$

(d) Prove that one of the quantities from part (c) strictly decreases at each transition. Conclude that for every starting state, the machine will reach a final state.

Problem 11.38.

Prove that a graph is a tree iff it has a unique path between every two vertices.

Problem 11.39.

Let G be a weighted graph and suppose there is a unique edge $e \in E(G)$ with smallest weight, that is, $w(e) < w(f)$ for all edges $f \in E(G) - \{e\}$. Prove that any minimum weight spanning tree (MST) of G must include e .

Problem 11.40.

Let G be a 4×4 grid with vertical and horizontal edges between neighboring vertices. Formally,

$$V(G) = [0, 3]^2 ::= \{(k, j) \mid 0 \leq k, j \leq 3\}.$$

Letting $h_{i,j}$ be the horizontal edge $\langle(i, j) — (i + 1, j)\rangle$ and $v_{j,i}$ be the vertical edge $\langle(j, i) — (j, i + 1)\rangle$ for $i \in [0, 2], j \in [0, 3]$. The weights of these edges are

$$w(h_{i,j}) ::= \frac{4i + j}{100},$$

$$w(v_{j,i}) ::= 1 + \frac{i + 4j}{100}.$$

(A picture of G would help; you might like to draw one.)

(a) Construct a minimum weight spanning tree (MST) for G by initially selecting the minimum weight edge, and then successively selecting the minimum weight edge that does not create a cycle with the previously selected edges. Stop when the selected edges form a spanning tree of G . (This is Kruskal’s MST algorithm.)

(b) Grow an MST for G starting with the tree consisting of the single vertex $(1, 2)$ and successively adding the minimum weight edge with exactly one endpoint in the tree. Stop when the tree spans G . (This is Prim’s MST algorithm.)

(c) Grow an MST for G by treating the vertices $(0, 0), (0, 3), (2, 3)$ as single vertex trees and then successively adding, for each tree in parallel, the minimum weight edge among the edges with one endpoint in the tree. Continue until the trees merge and form a spanning tree of G . (This is 6.042’s parallel MST algorithm.)

(d) Verify that you got the same MST each time.

Problem 11.41.

In this problem you will prove:

Theorem. *A graph G is 2-colorable iff it contains no odd length closed walk.*

As usual with “iff” assertions, the proof splits into two proofs: part (a) asks you to prove that the left side of the “iff” implies the right side. The other problem parts prove that the right side implies the left.

(a) Assume the left side and prove the right side. Three to five sentences should suffice.

(b) Now assume the right side. As a first step toward proving the left side, explain why we can focus on a single connected component H within G .

(c) As a second step, explain how to 2-color any tree.

(d) Choose any 2-coloring of a spanning tree, T , of H . Prove that H is 2-colorable by showing that any edge *not* in T must also connect different-colored vertices.

Homework Problems

Problem 11.42.

Prove Corollary 11.11.12: If all edges in a finite weighted graph have distinct weights, then the graph has a *unique* MST.

Hint: Suppose M and N were different MST's of the same graph. Let e be the smallest edge in one and not the other, say $e \in M - N$, and observe that $N + e$ must have a cycle.



12 Planar Graphs

12.1 Drawing Graphs in the Plane

Suppose there are three dog houses and three human houses, as shown in Figure 12.1. Can you find a route from each dog house to each human house such that no route crosses any other route?

A similar question comes up about a little-known animal called a *quadrapi* that looks like an octopus with four stretchy arms instead of eight. If five quadrapi are resting on the sea floor, as shown in Figure 12.2, can each quadrapi simultaneously shake hands with every other in such a way that no arms cross?

Both these puzzles can be understood as asking about drawing graphs in the plane. Replacing dogs and houses by nodes, the dog house puzzle can be rephrased as asking whether there is a planar drawing of the graph with six nodes and edges between each of the first three nodes and each of the second three nodes. This graph is called the *complete bipartite graph* $K_{3,3}$ and is shown in Figure 12.3.(a). The quadrapi puzzle asks whether there is a planar drawing of the complete graph K_5 shown in Figure 12.3.(b).

In each case, the answer is, “No —but almost!” In fact, if you remove an edge from either of these graphs, then the resulting graph *can* be redrawn in the plane so that no edges cross, as shown in Figure 12.4.

Planar drawings have applications in circuit layout and are helpful in displaying graphical data such as program flow charts, organizational charts, and scheduling conflicts. For these applications, the goal is to draw the graph in the plane with as few edge crossings as possible. (See the box on the following page for one such example.)

12.2 Definitions of Planar Graphs

We took the idea of a planar drawing for granted in the previous section, but if we’re going to *prove* things about planar graphs, we better have precise definitions.

Definition 12.2.1. A *drawing* of a graph assigns to each node a distinct point in the plane and assigns to each edge a smooth curve in the plane whose endpoints correspond to the nodes incident to the edge. The drawing is *planar* if none of the



Figure 12.1 Three dog houses and and three human houses. Is there a route from each dog house to each human house so that no pair of routes cross each other?

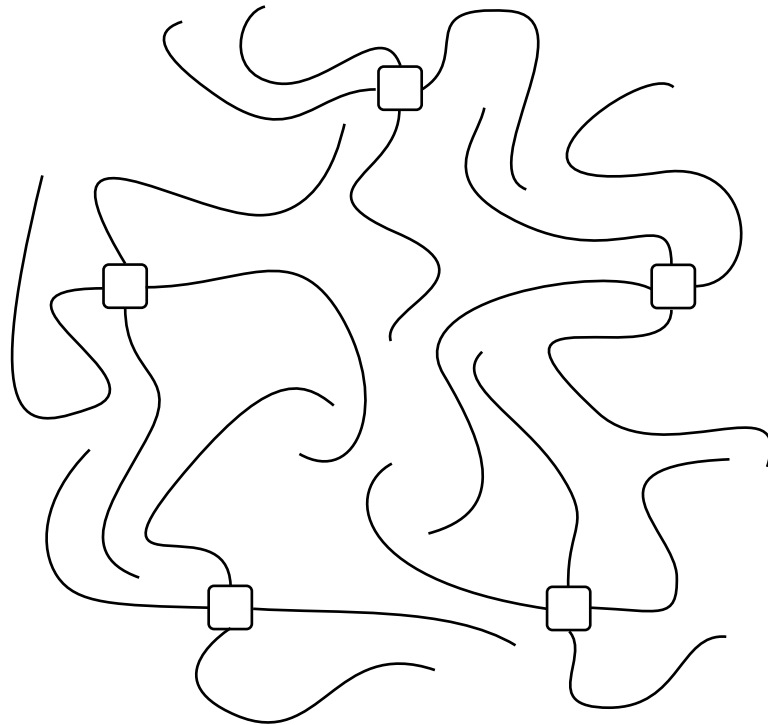


Figure 12.2 Five quadrapi (4-armed creatures).

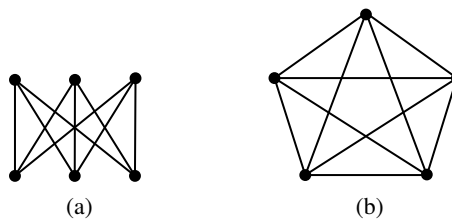


Figure 12.3 $K_{3,3}$ (a) and K_5 (b). Can you redraw these graphs so that no pairs of edges cross?

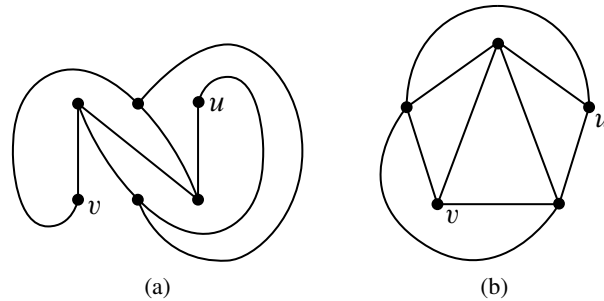


Figure 12.4 Planar drawings of (a) $K_{3,3}$ without $\{u-v\}$, and (b) K_5 without $\{u-v\}$.

Steve Wozniak and a Planar Circuit Design

When wires are arranged on a surface, like a circuit board or microchip, crossings require troublesome three-dimensional structures. When Steve Wozniak designed the disk drive for the early Apple II computer, he struggled mightily to achieve a nearly planar design according to the following excerpt from apple2history.org which in turn quotes *Fire in the Valley* by Freiburger and Swaine:

For two weeks, he worked late each night to make a satisfactory design. When he was finished, he found that if he moved a connector he could cut down on feedthroughs, making the board more reliable. To make that move, however, he had to start over in his design. This time it only took twenty hours. He then saw another feedthrough that could be eliminated, and again started over on his design. “The final design was generally recognized by computer engineers as brilliant and was by engineering aesthetics beautiful. Woz later said, ‘It’s something you can only do if you’re the engineer and the PC board layout person yourself. That was an artistic layout. The board has virtually no feedthroughs.’

curves cross themselves or other curves, namely, the only points that appear more than once on any of the curves are the node points. A graph is *planar* when it has a planar drawing.

Definition 12.2.1 is precise but depends on further concepts: “smooth planar curves” and “points appearing more than once” on them. We haven’t defined these concepts—we just showed the simple picture in Figure 12.4 and hoped you would get the idea.

Pictures can be a great way to get a new idea across, but it is generally not a good idea to use a picture to replace precise mathematics. Relying solely on pictures can sometimes lead to disaster—or to bogus proofs, anyway. There is a long history of bogus proofs about planar graphs based on misleading pictures.

The bad news is that to prove things about planar graphs using the planar drawings of Definition 12.2.1, we’d have to take a chapter-long excursion into continuous mathematics just to develop the needed concepts from plane geometry and point-set topology. The good news is that there is another way to define planar graphs that uses only discrete mathematics. In particular, we can define planar graphs as a recursive data type. In order to understand how it works, we first need to understand the concept of a *face* in a planar drawing.

12.2.1 Faces

The curves in a planar drawing divide up the plane into connected regions called the *continuous faces*¹ of the drawing. For example, the drawing in Figure 12.5 has four continuous faces. Face IV, which extends off to infinity in all directions, is called the *outside face*.

The vertices along the boundary of each continuous face in Figure 12.5 form a cycle. For example, labeling the vertices as in Figure 12.6, the cycles for each of the face boundaries can be described by the vertex sequences

$$abca \quad abda \quad bcdb \quad acda. \quad (12.1)$$

These four cycles correspond nicely to the four continuous faces in Figure 12.6—so nicely, in fact, that we can identify each of the faces in Figure 12.6 by its cycle. For example, the cycle *abca* identifies face III. The cycles in list 12.1 are called the *discrete faces* of the graph in Figure 12.6. We use the term “discrete” since cycles in a graph are a discrete data type—as opposed to a region in the plane, which is a continuous data type.

¹Most texts drop the adjective *continuous* from the definition of a face as a connected region. We need the adjective to distinguish continuous faces from the *discrete* faces we’re about to define.

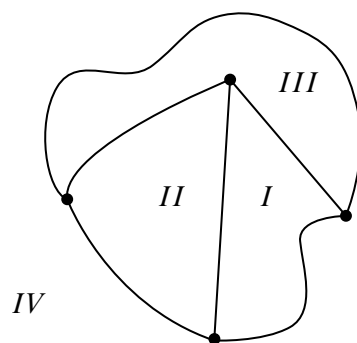


Figure 12.5 A planar drawing with four continuous faces.

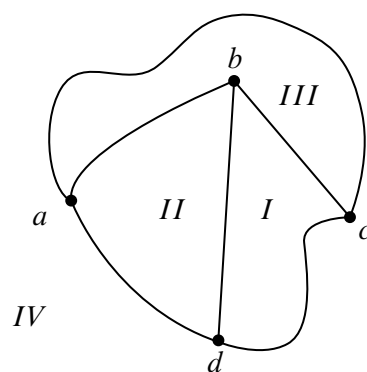


Figure 12.6 The drawing with labeled vertices.

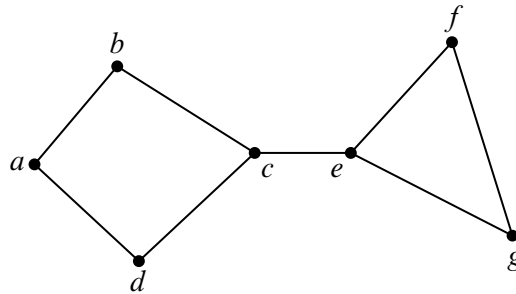


Figure 12.7 A planar drawing with a *bridge*.

Unfortunately, continuous faces in planar drawings are not always bounded by cycles in the graph —things can get a little more complicated. For example, the planar drawing in Figure 12.7 has what we will call a *bridge*, namely, a cut edge $\langle c—e \rangle$. The sequence of vertices along the boundary of the outer region of the drawing is

$$abcefgceda.$$

This sequence defines a closed walk, but does not define a cycle since the walk has two occurrences of the bridge $\langle c—e \rangle$ and each of its endpoints.

The planar drawing in Figure 12.8 illustrates another complication. This drawing has what we will call a *dongle*, namely, the nodes v , x , y , and w , and the edges incident to them. The sequence of vertices along the boundary of the inner region is

$$rstvxyxvwvtur.$$

This sequence defines a closed walk, but once again does not define a cycle because it has two occurrences of *every* edge of the dongle —once “coming” and once “going.”

It turns out that bridges and dongles are the only complications, at least for connected graphs. In particular, every continuous face in a planar drawing corresponds to a closed walk in the graph. These closed walks will be called the *discrete faces* of the drawing, and we’ll define them next.

12.2.2 A Recursive Definition for Planar Embeddings

The association between the continuous faces of a planar drawing and closed walks provides the discrete data type we can use instead of continuous drawings. We’ll define a *planar embedding* of *connected* graph to be the set of closed walks that are its face boundaries. Since all we care about in a graph are the connections between

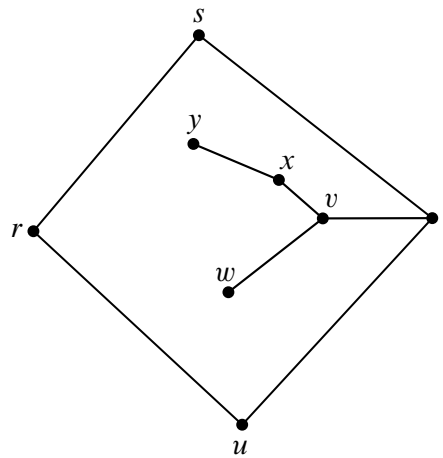


Figure 12.8 A planar drawing with a *dongle*.

vertices —not what a drawing of the graph actually looks like —planar embeddings are exactly what we need.

The question is how to define planar embeddings without appealing to continuous drawings. There is a simple way to do this based on the idea that any continuous drawing can be drawn step by step:

- either draw a new point somewhere in the plane to represent a vertex,
- or draw a curve between two vertex points that have already been laid down, making sure the new curve doesn’t cross any of the previously drawn curves.

A new curve won’t cross any other curves precisely when it stays within one of the continuous faces. Alternatively, a new curve won’t have to cross any other curves if it can go between the outer faces of two different drawings. So to be sure it’s ok to draw a new curve, we just need to check that its endpoints are on the boundary of the same face, or that its endpoints are on the outer faces of different drawings. Of course drawing the new curve changes the faces slightly, so the face boundaries will have to be updated once the new curve is drawn. This is the idea behind the following recursive definition.

Definition 12.2.2. A *planar embedding* of a *connected* graph consists of a nonempty set of closed walks of the graph called the *discrete faces* of the embedding. Planar embeddings are defined recursively as follows:

Base case: If G is a graph consisting of a single vertex, v , then a planar embedding of G has one discrete face, namely, the length zero closed walk, v .

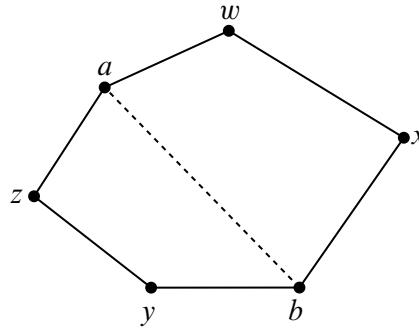


Figure 12.9 The “split a face” case: $awxbyza$ splits into $awxba$ and $abyza$.

Constructor case (split a face): Suppose G is a connected graph with a planar embedding, and suppose a and b are distinct, nonadjacent vertices of G that occur in some discrete face, γ , of the planar embedding. That is, γ is a closed walk of the form

$$\gamma = \alpha \hat{\ } \beta$$

where α is a walk from a to b and β is a walk from b to a . Then the graph obtained by adding the edge $\langle a-b \rangle$ to the edges of G has a planar embedding with the same discrete faces as G , except that face γ is replaced by the two discrete faces²

$$\alpha \hat{\ } \langle b-a \rangle \quad \text{and} \quad \langle a-b \rangle \hat{\ } \beta \tag{12.2}$$

as illustrated in Figure 12.9.³

Constructor case (add a bridge): Suppose G and H are connected graphs with planar embeddings and disjoint sets of vertices. Let γ be a discrete face of the embedding of G and suppose that γ begins and ends at vertex a .

Similarly, let δ be a discrete face of the embedding of H that begins and ends at vertex b .

² There is a minor exception to this definition of embedding in the special case when G is a line graph beginning with a and ending with b . In this case the cycles into which γ splits are actually the same. That’s because adding edge $\langle a-b \rangle$ creates a cycle that divides the plane into “inner” and “outer” continuous faces that are both bordered by this cycle. In order to maintain the correspondence between continuous faces and discrete faces in this case, we define the two discrete faces of the embedding to be two “copies” of this same cycle.

³Formally, merge is an operation on walks, not a walk and an edge, so in (12.2), we should have used a walk $(a \ \langle a-b \rangle \ b)$ instead of an edge $\langle a-b \rangle$ and written

$$\alpha \hat{\ } (b \ \langle b-a \rangle \ a) \quad \text{and} \quad (a \ \langle a-b \rangle \ b) \hat{\ } \beta$$

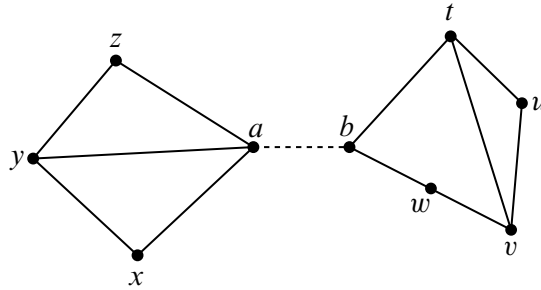


Figure 12.10 The “add a bridge” case.

Then the graph obtained by connecting G and H with a new edge, $\langle a-b \rangle$, has a planar embedding whose discrete faces are the union of the discrete faces of G and H , except that faces γ and δ are replaced by one new face

$$\gamma \wedge \langle a-b \rangle \wedge \delta \wedge \langle b-a \rangle.$$

This is illustrated in Figure 12.10, where the vertex sequences of the faces of G and H are:

$$G : \{axyza, axya, ayza\} \quad H : \{btuvwb, btvwb, tuvt\},$$

and after adding the bridge $\langle a-b \rangle$, there is a single connected graph whose faces have the vertex sequences

$$\{axyzbtuvwba, axya, ayza, btvwb, tuvt\}.$$

A bridge is simply a cut edge, but in the context of planar embeddings, the bridges are precisely the edges that occur *twice on the same discrete face* —as opposed to once on each of two faces. Dongles are trees made of bridges; we only use dongles in illustrations, so there’s no need to define them more precisely.

12.2.3 Does It Work?

Yes! In general, a graph is planar because it has a planar drawing according to Definition 12.2.1 if and only if each of its connected components has a planar embedding as specified in Definition 12.2.2. Of course we can’t prove this without an excursion into exactly the kind of continuous math that we’re trying to avoid. But now that the recursive definition of planar graphs is in place, we won’t ever need to fall back on the continuous stuff. That’s the good news.

The bad news is that Definition 12.2.2 is a lot more technical than the intuitively simple notion of a drawing whose edges don’t cross. In many cases it’s easier to

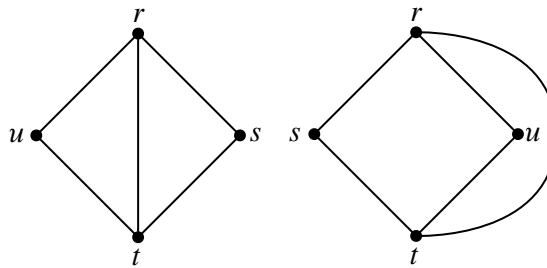


Figure 12.11 Two illustrations of the same embedding.

stick to the idea of planar drawings and give proofs in those terms. For example, it’s obvious that erasing edges from a planar drawing leaves a planar drawing. On the other hand, it’s not at all obvious, though of course it is true, that you can delete an edge from a planar embedding and still get a planar embedding (see Problem 12.9).

In the hands of experts, and perhaps in your hands too with a little more experience, proofs about planar graphs by appeal to drawings can be convincing and reliable. But given the long history of mistakes in such proofs, it’s safer to work from the precise definition of planar embedding. More generally, it’s also important to see how the abstract properties of curved drawings in the plane can be modelled successfully using a discrete data type.

12.2.4 Where Did the Outer Face Go?

Every planar drawing has an immediately-recognizable outer face —it’s the one that goes to infinity in all directions. But where is the outer face in a planar embedding?

There isn’t one! That’s because there really isn’t any need to distinguish one face from another. In fact, a planar embedding could be drawn with any given face on the outside. An intuitive explanation of this is to think of drawing the embedding on a *sphere* instead of the plane. Then any face can be made the outside face by “puncturing” that face of the sphere, stretching the puncture hole to a circle around the rest of the faces, and flattening the circular drawing onto the plane.

So pictures that show different “outside” boundaries may actually be illustrations of the same planar embedding. For example, the two embeddings shown in Figure 12.11 are really the same —check it: they have the same boundary cycles.

This is what justifies the “add bridge” case in Definition 12.2.2: whatever face is chosen in the embeddings of each of the disjoint planar graphs, we can draw a bridge between them without needing to cross any other edges in the drawing, because we can assume the bridge connects two “outer” faces.

12.3 Euler’s Formula

The value of the recursive definition is that it provides a powerful technique for proving properties of planar graphs, namely, structural induction. For example, we will now use Definition 12.2.2 and structural induction to establish one of the most basic properties of a connected planar graph, namely, that the number of vertices and edges completely determines the number of faces in every possible planar embedding of the graph.

Theorem 12.3.1 (Euler’s Formula). *If a connected graph has a planar embedding, then*

$$v - e + f = 2$$

where v is the number of vertices, e is the number of edges, and f is the number of faces.

For example, in Figure 12.5, $v = 4$, $e = 6$, and $f = 4$. Sure enough, $4 - 6 + 4 = 2$, as Euler’s Formula claims.

Proof. The proof is by structural induction on the definition of planar embeddings. Let $P(\mathcal{E})$ be the proposition that $v - e + f = 2$ for an embedding, $\mathcal{E}$.

Base case ($\mathcal{E}$ is the one-vertex planar embedding): By definition, $v = 1$, $e = 0$, and $f = 1$, and $1 - 0 + 1 = 2$, so $P(\mathcal{E})$ indeed holds.

Constructor case (split a face): Suppose G is a connected graph with a planar embedding, and suppose a and b are distinct, nonadjacent vertices of G that appear on some discrete face, $\gamma = a \dots b \dots a$, of the planar embedding.

Then the graph obtained by adding the edge $\langle a-b \rangle$ to the edges of G has a planar embedding with one more face and one more edge than G . So the quantity $v - e + f$ will remain the same for both graphs, and since by structural induction this quantity is 2 for G ’s embedding, it’s also 2 for the embedding of G with the added edge. So P holds for the constructed embedding.

Constructor case (add bridge): Suppose G and H are connected graphs with planar embeddings and disjoint sets of vertices. Then connecting these two graphs with a bridge merges the two bridged faces into a single face, and leaves all other faces unchanged. So the bridge operation yields a planar embedding of a connected

graph with $v_G + v_H$ vertices, $e_G + e_H + 1$ edges, and $f_G + f_H - 1$ faces. Since

$$\begin{aligned} & (v_G + v_H) - (e_G + e_H + 1) + (f_G + f_H - 1) \\ &= (v_G - e_G + f_G) + (v_H - e_H + f_H) - 2 \\ &= (2) + (2) - 2 \quad (\text{by structural induction hypothesis}) \\ &= 2, \end{aligned}$$

$v - e + f$ remains equal to 2 for the constructed embedding. That is, $P(\mathcal{E})$ also holds in this case.

This completes the proof of the constructor cases, and the theorem follows by structural induction. ■

12.4 Bounding the Number of Edges in a Planar Graph

Like Euler’s formula, the following lemmas follow by structural induction directly from Definition 12.2.2.

Lemma 12.4.1. *In a planar embedding of a connected graph, each edge occurs once in each of two different faces, or occurs exactly twice in one face.*

Lemma 12.4.2. *In a planar embedding of a connected graph with at least three vertices, each face is of length at least three.*

Combining Lemmas 12.4.1 and 12.4.2 with Euler’s Formula, we can now prove that planar graphs have a limited number of edges:

Theorem 12.4.3. *Suppose a connected planar graph has $v \geq 3$ vertices and e edges. Then*

$$e \leq 3v - 6. \tag{12.3}$$

Proof. By definition, a connected graph is planar iff it has a planar embedding. So suppose a connected graph with v vertices and e edges has a planar embedding with f faces. By Lemma 12.4.1, every edge has exactly two occurrences in the face boundaries. So the sum of the lengths of the face boundaries is exactly $2e$. Also by Lemma 12.4.2, when $v \geq 3$, each face boundary is of length at least three, so this sum is at least $3f$. This implies that

$$3f \leq 2e. \tag{12.4}$$

But $f = e - v + 2$ by Euler’s formula, and substituting into (12.4) gives

$$\begin{aligned} 3(e - v + 2) &\leq 2e \\ e - 3v + 6 &\leq 0 \\ e &\leq 3v - 6 \end{aligned}$$

■

12.5 Returning to K_5 and $K_{3,3}$

Finally we have a simple way to answer the quadrapi question at the beginning of this chapter: the five quadrapi can’t all shake hands without crossing. The reason is that we know the quadrapi question is the same as asking whether a complete graph K_5 is planar, and Theorem 12.4.3 has the immediate:

Corollary 12.5.1. K_5 is not planar.

Proof. K_5 is connected and has 5 vertices and 10 edges. But since $10 > 3 \cdot 5 - 6$, K_5 does not satisfy the inequality (12.3) that holds in all planar graphs. ■

We can also use Euler’s Formula to show that $K_{3,3}$ is not planar. The proof is similar to that of Theorem 12.3 except that we use the additional fact that $K_{3,3}$ is a bipartite graph.

Lemma 12.5.2. In a planar embedding of a connected bipartite graph with at least 3 vertices, each face has length at least 4.

Proof. By Lemma 12.4.2, every face of a planar embedding of the graph has length at least 3. But by Lemma 11.7.2 and Theorem 11.10.1.3, a bipartite graph can’t have odd length closed walks. Since the faces of a planar embedding are closed walks, there can’t be any faces of length 3 in a bipartite embedding. So every face must have length at least 4. ■

Theorem 12.5.3. Suppose a connected bipartite graph with $v \geq 3$ vertices and e edges is planar. Then

$$e \leq 2v - 4. \tag{12.5}$$

Proof. Lemma 12.5.2 implies that all the faces of an embedding of the graph have length at least 4. Now arguing as in the proof of Theorem 12.4.3, we find that the sum of the lengths of the face boundaries is exactly $2e$ and at least $4f$. Hence,

$$4f \leq 2e \tag{12.6}$$

for any embedding of a planar bipartite graph. By Euler’s theorem, $f = 2 - v + e$. Substituting $2 - v + e$ for f in (12.6), we have

$$4(2 - v + e) \leq 2e,$$

which simplifies to (12.5). ■

Corollary 12.5.4. $K_{3,3}$ is not planar.

Proof. $K_{3,3}$ is connected, bipartite and has 6 vertices and 9 edges. But since $9 > 2 \cdot 6 - 4$, $K_{3,3}$ does not satisfy the inequality (12.3) that holds in all bipartite planar graphs. ■

12.6 Coloring Planar Graphs

We’ve covered a lot of ground with planar graphs, but not nearly enough to prove the famous 4-color theorem. But we can get awfully close. Indeed, we have done almost enough work to prove that every planar graph can be colored using only 5 colors.

There are two familiar facts about planarity that we will need.

Lemma 12.6.1. *Any subgraph of a planar graph is planar.*

Lemma 12.6.2. *Merging two adjacent vertices of a planar graph leaves another planar graph.*

Merging two adjacent vertices, n_1 and n_2 of a graph means deleting the two vertices and then replacing them by a new “merged” vertex, m , adjacent to all the vertices that were adjacent to either of n_1 or n_2 , as illustrated in Figure 12.12.

Many authors take Lemmas 12.6.1 and 12.6.2 for granted for continuous drawings of planar graphs described by Definition 12.2.1. With the recursive Definition 12.2.2 both Lemmas can actually be proved using structural induction (see Problem 12.9).

We need only one more lemma:

Lemma 12.6.3. *Every planar graph has a vertex of degree at most five.*

Proof. Assuming to the contrary that every vertex of some planar graph had degree at least 6, then the sum of the vertex degrees is at least $6v$. But the sum of the vertex degrees equals $2e$ by the Handshake Lemma 11.2.1, so we have $e \geq 3v$ contradicting the fact that $e \leq 3v - 6 < 3v$ by Theorem 12.4.3. ■

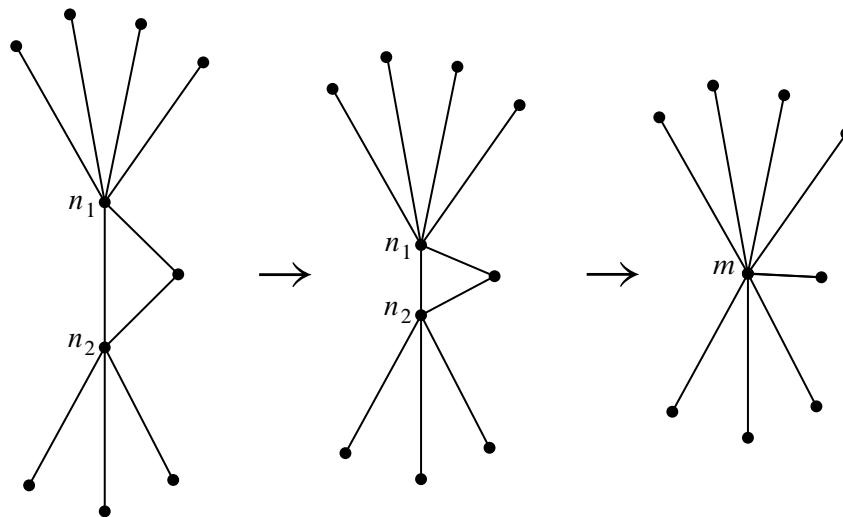


Figure 12.12 Merging adjacent vertices n_1 and n_2 into new vertex, m .

Theorem 12.6.4. *Every planar graph is five-colorable.*

Proof. The proof will be by strong induction on the number, v , of vertices, with induction hypothesis:

Every planar graph with v vertices is five-colorable.

Base cases ($v \leq 5$): immediate.

Inductive case: Suppose G is a planar graph with $v + 1$ vertices. We will describe a five-coloring of G .

First, choose a vertex, g , of G with degree at most 5; Lemma 12.6.3 guarantees there will be such a vertex.

Case 1: ($\deg(g) < 5$): Deleting g from G leaves a graph, H , that is planar by Lemma 12.6.1, and, since H has v vertices, it is five-colorable by induction hypothesis. Now define a five coloring of G as follows: use the five-coloring of H for all the vertices besides g , and assign one of the five colors to g that is not the same as the color assigned to any of its neighbors. Since there are fewer than 5 neighbors, there will always be such a color available for g .

Case 2: ($\deg(g) = 5$): If the five neighbors of g in G were all adjacent to each other, then these five vertices would form a nonplanar subgraph isomorphic to K_5 , contradicting Lemma 12.6.1 (since K_5 is not planar). So there must

be two neighbors, n_1 and n_2 , of g that are not adjacent. Now merge n_1 and g into a new vertex, m . In this new graph, n_2 is adjacent to m , and the graph is planar by Lemma 12.6.2. So we can then merge m and n_2 into a another new vertex, m' , resulting in a new graph, G' , which by Lemma 12.6.2 is also planar. Since G' has $v - 1$ vertices, it is five-colorable by the induction hypothesis.

Now define a five coloring of G as follows: use the five-coloring of G' for all the vertices besides g , n_1 and n_2 . Next assign the color of m' in G' to be the color of the neighbors n_1 and n_2 . Since n_1 and n_2 are not adjacent in G , this defines a proper five-coloring of G except for vertex g . But since these two neighbors of g have the same color, the neighbors of g have been colored using fewer than five colors altogether. So complete the five-coloring of G by assigning one of the five colors to g that is not the same as any of the colors assigned to its neighbors.

■

12.7 Classifying Polyhedra

The Pythagoreans had two great mathematical secrets, the irrationality of $\sqrt{2}$ and a geometric construct that we're about to rediscover!

A *polyhedron* is a convex, three-dimensional region bounded by a finite number of polygonal faces. If the faces are identical regular polygons and an equal number of polygons meet at each corner, then the polyhedron is *regular*. Three examples of regular polyhedra are shown in Figure 12.13: the tetrahedron, the cube, and the octahedron.

We can determine how many more regular polyhedra there are by thinking about planarity. Suppose we took *any* polyhedron and placed a sphere inside it. Then we could project the polyhedron face boundaries onto the sphere, which would give an image that was a planar graph embedded on the sphere, with the images of the corners of the polyhedron corresponding to vertices of the graph. We've already observed that embeddings on a sphere are the same as embeddings on the plane, so Euler's formula for planar graphs can help guide our search for regular polyhedra.

For example, planar embeddings of the three polyhedra in Figure 12.1 are shown in Figure 12.14.

Let m be the number of faces that meet at each corner of a polyhedron, and let n be the number of edges on each face. In the corresponding planar graph, there are m edges incident to each of the v vertices. By the Handshake Lemma 11.2.1, we

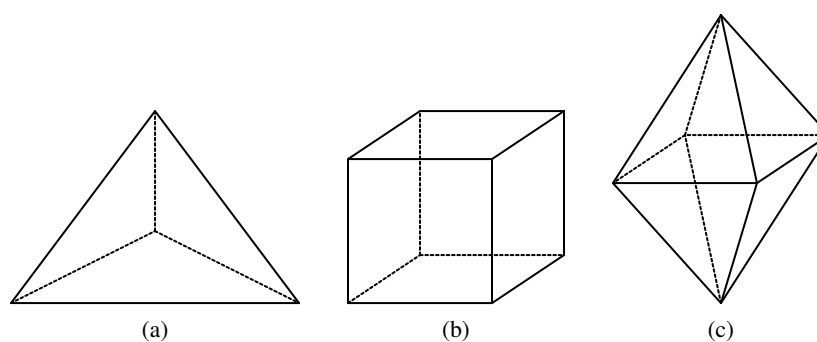


Figure 12.13 The tetrahedron (a), cube (b), and octahedron (c).

v

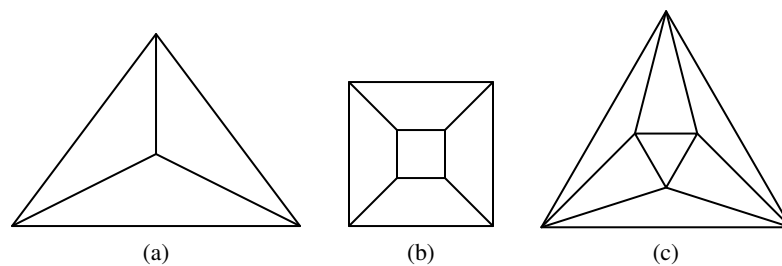


Figure 12.14 Planar embeddings of the tetrahedron (a), cube (b), and octahedron (c).

n	m	v	e	f	polyhedron
3	3	4	6	4	tetrahedron
4	3	8	12	6	cube
3	4	6	12	8	octahedron
3	5	12	30	20	icosahedron
5	3	20	30	12	dodecahedron

Figure 12.15 The only possible regular polyhedra.

know:

$$mv = 2e.$$

Also, each face is bounded by n edges. Since each edge is on the boundary of two faces, we have:

$$nf = 2e$$

Solving for v and f in these equations and then substituting into Euler’s formula gives:

$$\frac{2e}{m} - e + \frac{2e}{n} = 2$$

which simplifies to

$$\frac{1}{m} + \frac{1}{n} = \frac{1}{e} + \frac{1}{2} \quad (12.7)$$

Equation 12.7 places strong restrictions on the structure of a polyhedron. Every nondegenerate polygon has at least 3 sides, so $n \geq 3$. And at least 3 polygons must meet to form a corner, so $m \geq 3$. On the other hand, if either n or m were 6 or more, then the left side of the equation could be at most $1/3 + 1/6 = 1/2$, which is less than the right side. Checking the finitely-many cases that remain turns up only five solutions, as shown in Figure 12.15. For each valid combination of n and m , we can compute the associated number of vertices v , edges e , and faces f . And polyhedra with these properties do actually exist. The largest polyhedron, the dodecahedron, was the other great mathematical secret of the Pythagorean sect.

The 5 polyhedra in Figure 12.15 are the only possible regular polyhedra. So if you want to put more than 20 geocentric satellites in orbit so that they *uniformly* blanket the globe—tough luck!

12.8 Another Characterization for Planar Graphs

We did not pick K_5 and $K_{3,3}$ as examples because of their application to dog houses or quadrapi shaking hands. We really picked them because they provide another, famous, discrete characterization of planar graphs:

Theorem 12.8.1 (Kuratowski). *A graph is not planar if and only if it contains K_5 or $K_{3,3}$ as a minor.*

Definition 12.8.2. A *minor* of a graph G is a graph that can be obtained by repeatedly⁴ deleting vertices, deleting edges, and merging *adjacent* vertices of G .

For example, Figure 12.16 illustrates why C_3 is a minor of the graph in Figure 12.16(a). In fact C_3 is a minor of a connected graph G if and only if G is not a tree.

The known proofs of Kuratowski’s Theorem 12.8.1 are a little too long to include in an introductory text, so we won’t give one.

Problems for Section 12.2

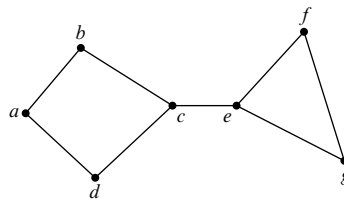
Practice Problems

Problem 12.1.

What are the discrete faces of the following two graphs?

Write each cycle as a sequence of letters without spaces, starting with the alphabetically earliest letter in the clockwise direction, for example “adbfa.” Separate the sequences with spaces.

(a)



(b)

⁴The three operations can each be performed any number of times in any order.

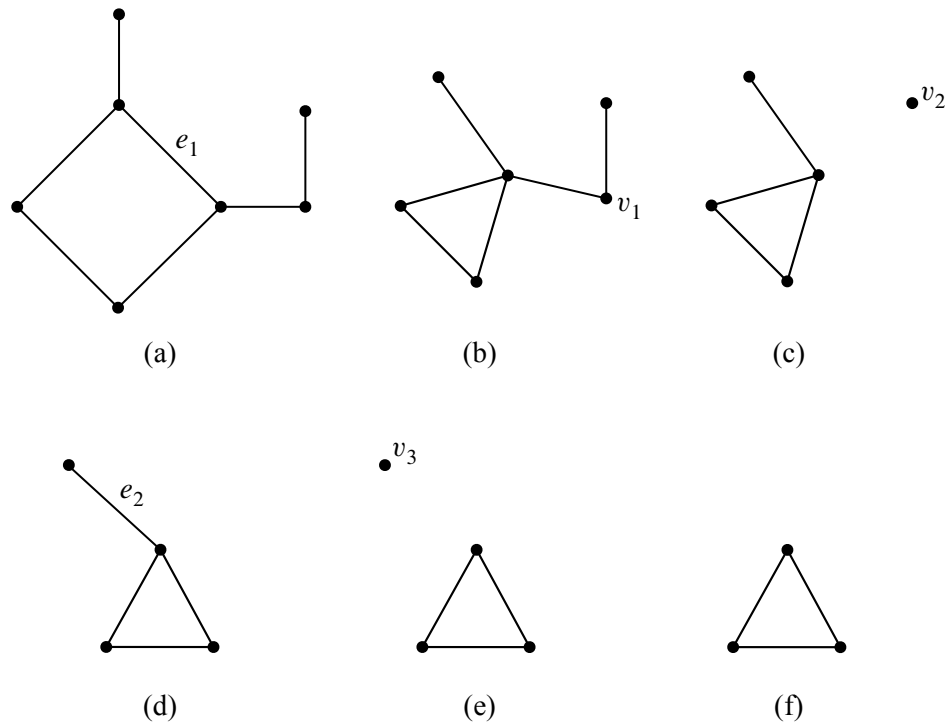
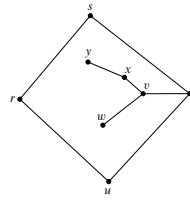


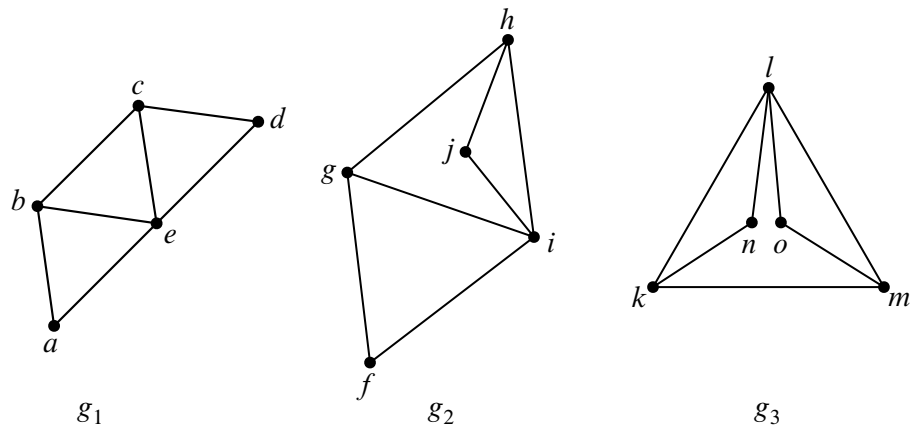
Figure 12.16 One method by which the graph in (a) can be reduced to C_3 (f), thereby showing that C_3 is a minor of the graph. The steps are: merging the nodes incident to e_1 (b), deleting v_1 and all edges incident to it (c), deleting v_2 (d), deleting e_2 , and deleting v_3 (f).



Problems for Section 12.8

Exam Problems

Problem 12.2.



(a) Describe an isomorphism between graphs G_1 and G_2 , and another isomorphism between G_2 and G_3 .

(b) Why does part (a) imply that there is an isomorphism between graphs G_1 and G_3 ?

Let G and H be planar graphs. An embedding E_G of G is isomorphic to an embedding E_H of H iff there is an isomorphism from G to H that also maps each face of E_G to a face of E_H .

(c) One of the embeddings pictured above is not isomorphic to either of the others. Which one? Briefly explain why.

(d) Explain why all embeddings of two isomorphic planar graphs must have the same number of faces.

Problem 12.3. (a) Give an example of a planar graph with two planar embeddings, where the first embedding has a face whose length is not equal to the length of any face in the second embedding. Draw the two embeddings to demonstrate this.

(b) Define the length of a planar embedding, $\mathcal{E}$, to be the sum of the lengths of the faces of $\mathcal{E}$. Prove that all embeddings of the same planar graph have the same length.

Problem 12.4.

Definition 12.2.2 of planar graph embeddings applied only to connected planar graphs. The definition can be extended to planar graphs that are not necessarily connected by adding the following additional constructor case to the definition:

- **Constructor Case:** (collect disjoint graphs) Suppose $\mathcal{E}_1$ and $\mathcal{E}_2$ are planar embeddings with no vertices in common. Then $\mathcal{E}_1 \cup \mathcal{E}_2$ is a planar embedding.

Euler’s Planar Graph Theorem now generalizes to unconnected graphs as follows: if a planar embedding, $\mathcal{E}$, has v vertices, e edges, f faces, and c connected components, then

$$v - e + f - 2c = 0. \quad (12.8)$$

This can be proved by structural induction on the definition of planar embedding.

- (a)** State and prove the base case of the structural induction.
- (b)** Let v_i, e_i, f_i , and c_i be the number of vertices, edges, faces, and connected components in embedding $\mathcal{E}_i$ and let v, e, f, c be the numbers for the embedding from the (collect disjoint graphs) constructor case. Express v, e, f, c in terms of v_i, e_i, f_i, c_i .
- (c)** Prove the (collect disjoint graphs) case of the structural induction.

Problem 12.5. (a) A simple graph has 8 vertices and 24 edges. What is the average degree per vertex?

(b) A connected planar simple graph has 5 more edges than it has vertices. How many faces does it have?

(c) A connected simple graph has one more vertex than it has edges. Explain why it is a planar graph.

- (d) How many faces does a planar graph from part c have?
- (e) How many distinct isomorphisms from exist between the graph given in Figure 12.17 and itself? (Include the identity isomorphism.)

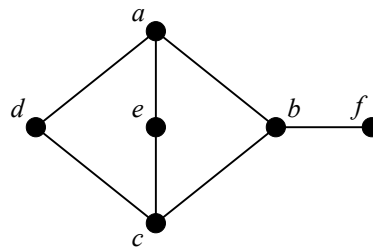


Figure 12.17

Class Problems

Problem 12.6.

Figure 12.18 shows four different pictures of planar graphs.

- (a) For each picture, describe its discrete faces (closed walks that define the region borders).
- (b) Which of the pictured graphs are isomorphic? Which pictures represent the same planar embedding? —that is, they have the same discrete faces.
- (c) Describe a way to construct the embedding in Figure 4 according to the recursive Definition 12.2.2 of planar embedding. For each application of a constructor rule, be sure to indicate the faces (cycles) to which the rule was applied and the cycles which result from the application.

Problem 12.7.

Prove the following assertions by structural induction on the definition of planar embedding.

- (a) In a planar embedding of a graph, each edge occurs exactly twice in the faces of the embedding.
- (b) In a planar embedding of a connected graph with at least three vertices, each face is of length at least three.

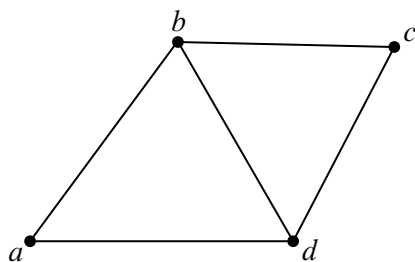


figure 1

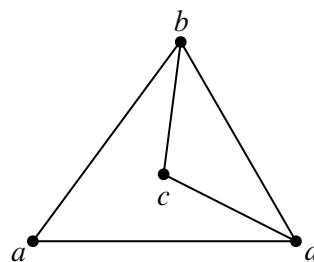


figure 2

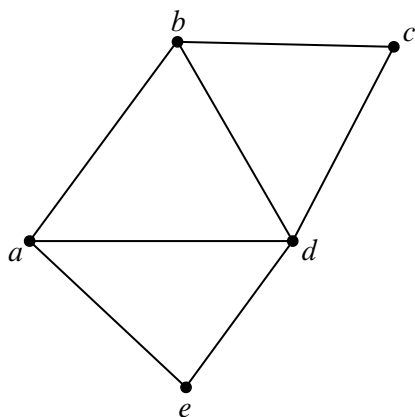


figure 3

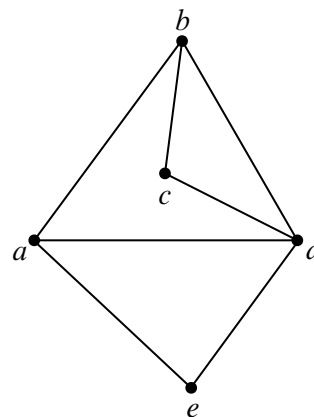


figure 4

Figure 12.18

Homework Problems

Problem 12.8.

A simple graph is *triangle-free* when it has no cycle of length three.

(a) Prove for any connected triangle-free planar graph with $v > 2$ vertices and e edges,

$$e \leq 2v - 4. \quad (12.9)$$

(b) Show that any connected triangle-free planar graph has at least one vertex of degree three or less.

(c) Prove that any connected triangle-free planar graph is 4-colorable.

Problem 12.9. (a) Prove

Lemma (Switch Edges). *Suppose that, starting from some embeddings of planar graphs with disjoint sets of vertices, it is possible by two successive applications of constructor operations to add edges e and then f to obtain a planar embedding, $\mathcal{F}$. Then starting from the same embeddings, it is also possible to obtain $\mathcal{F}$ by adding f and then e with two successive applications of constructor operations.*

Hint: There are four cases to analyze, depending on which two constructor operations are applied to add e and then f . Structural induction is not needed.

(b) Prove

Corollary (Permute Edges). *Suppose that, starting from some embeddings of planar graphs with disjoint sets of vertices, it is possible to add a sequence of edges $e_0, e_1, \dots, e_n$ by successive applications of constructor operations to obtain a planar embedding, $\mathcal{F}$. Then starting from the same embeddings, it is also possible to obtain $\mathcal{F}$ by applications of constructor operations that successively add any permutation⁵ of the edges $e_0, e_1, \dots, e_n$.*

Hint: By induction on the number of switches of adjacent elements needed to convert the sequence $0, 1, \dots, n$ into a permutation $\pi(0), \pi(1), \dots, \pi(n)$.

(c) Prove

Corollary (Delete Edge). *Deleting an edge from a planar graph leaves a planar graph.*

(d) Conclude that any subgraph of a planar graph is planar.

⁵If $\pi : \{0, 1, \dots, n\} \rightarrow \{0, 1, \dots, n\}$ is a bijection, then the sequence $e_{\pi(0)}, e_{\pi(1)}, \dots, e_{\pi(n)}$ is called a *permutation* of the sequence $e_0, e_1, \dots, e_n$.

13 State Machines

DRAFT

We’ve already demonstrated the use of state machines as abstract models of step-by-step processes. In this chapter we examine two further applications of state machines, first as models of concurrent computational processes, and second as the basis for reasoning about programs.

13.1 The Alternating Bit Protocol

The Alternating Bit Protocol is a well-known two-process communication protocol that achieves reliable FIFO communication over unreliable channels that operate concurrently. The unreliable channels may lose or duplicate messages, but are assumed not to reorder them. We’ll use the Invariant Method to verify the Protocol.

The Protocol allows a **Sender** process to send a sequence of messages from a message alphabet, M , to a **Receiver** process. It works as follows.

Sender repeatedly sends the rightmost message in its **outgoing-queue** of messages, tagged with a **tagbit** that is initially 1. When **Receiver** receives this tagged message, it sets its **ackbit** to be the message tag 1, and adds the message to the left-hand end of its **received-msgs** list. Then as an acknowledgement, **Receiver** sends back **ackbit** 1 repeatedly. When **Sender** gets this acknowledgement bit, it deletes the rightmost outgoing message in its queue, sets its **tagbit** to 0, and begins sending the new rightmost outgoing message, tagged with **tagbit**.

Receiver, having already accepted the message tagged with **ackbit** 1, ignores subsequent messages with tag 1, and waits until it sees the first message with tag 0; it adds this message to the left-hand side of its **received-msgs** list, sets **ackbit** to 0 and acknowledges repeatedly with **ackbit** 0. **Sender** now waits till it gets acknowledgement bit 0, then goes on to send the next outgoing message with tag 1. In this way, it alternates use of the tags 1 and 0 for successive messages.

We claim that this causes **Sender** to receive *suffix* original **outgoing-msgs** queue. That is, at any stage in the process when the **outgoing-msgs**

(The fact that **Sender** actually outputs the entire outgoing queue is a *liveness* claim—liveness properties are a generalization of termination properties. We’ll ignore this issue for now.)

We formalize the description above as a state whose states consist of:

outgoing-msgs, a finite sequence of M , whose initial value is called **all-msgs**
tagbit $\in \{0, 1\}$, initially 1

received-msgs, a finite sequence of M , initially empty
ackbit $\in \{0, 1\}$, initially 0

msg-channel, a finite sequence of $M \times \{0, 1\}$, initially empty,
ack-channel, a finite sequence of $\{0, 1\}$, initially empty

The transitions are:

- SEND:** (a) **action:** **send-msg**(m, b)
precondition: $m = \text{rightend}(\text{outgoing-msgs})$ AND $b = \text{tagbit}$
effect: add (m, b) to the lefthand end of **msg-channel**, any number ≥ 0 of times
- (b) **action:** **send-ack**(b)
precondition: $b = \text{ackbit}$
effect: add b to the righthand end of **ack-channel**, any number ≥ 0 of times
- RECEIVE:** (a) **action:** **receive-msg**(m, b)
precondition: $(m, b) = \text{rightend}(\text{msg-channel})$
effect: remove **rightend** of **msg-channel**;
if $b \neq \text{ackbit}$, then [add m to the lefthand end of **receive-msgs**; **ackbit** := b .]
- (b) **action:** **receive-ack**(b)
precondition: $b = \text{leftend}(\text{ack-channel})$
effect: remove **leftend** of **ack-channel**.
if $b = \text{tagbit}$, then [remove **rightend** of **outgoing-msgs** (if nonempty);
tagbit := **tagbit**]

Our goal is to show that when **tagbit** $\neq$ **ackbit**, then

$$\text{outgoing-queue} \cdot \text{received-msgs} = \text{all-msgs}. \quad (13.1)$$

This requires three auxiliary invariants. For the first of these, we need a definition.

Let **tag-sequence** be the sequence consisting of bits in **ack-channel**, in right-to-left order, followed by **tagbit**, followed by the tag components of the elements of **msg-channel**, in left-to-right order, followed by **ackbit**.

Property 2: **tag-sequence** consists of one of the following:

1. All 0's.
2. All 1's.
3. A positive number of 0's followed by a positive number of 1's.
4. A positive number of 1's followed by a positive number of 0's.

What is being ruled out by these four cases is the situation where the sequence contains more than one switch of tag value.

The fact that Property 2 is an invariant can be proved easily by induction. We also need:

Property 3: If (m, tag) is in **msg-channel** then $m = \text{rightend}(\text{outgoing-queue})$.

Proof. (That Property 3 is an invariant)

By induction, using Property 2.

Base: Obvious, since no message is in the channel initially.

Inductive step: It is easy to see that the property is preserved by **send**_{*m,b*}, which adds new messages to **channel**_{1,2}. The only other case that could cause a problem is **receive**(*b*)_{2,1}, which could cause **tag**₁ to change when there is another message already in **channel**_{1,2} with the same tag. But this can't happen, by Property 2 applied before the step—since the incoming tag *g* must be equal to **tag**₁ in this case, all the tags in **tag-sequence** must be the same. ■

Finally, we need that the following counterpart to (13.1): when **tagbit** = **ackbit**, then

$$\text{lefttail}(\text{outgoing-queue}) \cdot \text{received-msgs} = \text{all-msgs}, \quad (13.2)$$

where **lefttail**(**outgoing-queue**) all but the rightmost message, if any, in **outgoing-queue**.

Property 4, part 2, easily implies the goal Property 1. It also implies that **work-buf**₂ is always nonempty when **receive**(*b*)_{2,1} occurs with equal tags; therefore, the parenthetical check in the code always works out to be true.

Proof. (That Property 4 is an invariant)

By induction. Base: In an initial state, the tags are unequal, **work-buf**₁ = **buf**₁ and **buf**₂ is empty. This suffices to show part 1. part 2 is vacuous.

Inductive step: When a **send** occurs, the tags and buffers are unchanged, so the truth of the invariants must be preserved. It remains to consider **receive** events.

receive(*m, b*)_{1,2}:

If $b = \text{tag}_2$, nothing happens, so the invariants are preserved. So suppose that $b \neq \text{tag}_2$. Then Property 2 implies that $b = \text{tag}_1$, and then Property 3 implies that

m is the first message on **work-buf**₁. The effect of the transition is to change **tag**₂ to make it equal to **tag**₁, and to replicate the first element of **work-buf**₂ at the end of **buf**₂.

The inductive hypothesis implies that, before the step, **buf**₂ · **work-buf**₁ = **buf**₁. The changes caused by the step imply that, after the step, **tag**₁ = **tag**₂, **work-buf**₁ and **buf**₂ are nonempty, **head**(**work-buf**₁) = **last**(**buf**₂), and **buf**₂ · **tail**(**work-buf**₁) = **buf**₁. This is as needed.

receive(b)_{2,1}:

The argument is similar to the one for **receive**(m, b)_{1,2}. If $b \neq \mathbf{tag}_1$, nothing happens so the invariants are preserved. So suppose that $b = \mathbf{tag}_1$. Then Property 2 implies that $b = \mathbf{tag}_2$, and the step changes **tag**₁ to make it unequal to **tag**₂. The step also removes the first element of **work-buf**₁. The inductive hypothesis implies that, before the step, **work-buf**₁ and **buf**₂ are nonempty, **head**(**work-buf**₁) = **last**(**buf**₂), and **buf**₂ · **tail**(**work-buf**₁) = **buf**₁. The changes caused by the step imply that, after the step, **tag**₁ $\neq$ **tag**₂ and **buf**₂ · **work-buf**₁ = **buf**₁. This is as needed. ■

13.2 Reasoning About While Programs

Real programs and programming languages are often huge and complicated, making them hard to model and even harder to reason about. Still, making programs “reasonable” is a crucial aspect of software engineering. In this section we’ll illustrate what it means to have a clean mathematical model of a simple programming language and reasoning principles that go with it—if only real programming languages allowed for such simple, accurate modeling.

13.2.1 While Programs

The programs we’ll study are called “*while programs*.” We can define them as a recursive data type:

Definition 13.2.1.

base cases:

- $x := e$ is a **while** program, called an *assignment statement*, where x is a variable and e is an expression.
- *Done* is a **while** program.

constructor cases: If C and D are **while** programs, and T is a test, then the following are also **while** programs:

- $C;D$ —called the *sequencing* of C and D ,
- **if** T **then** C **else** D —called a *conditional* with *test*, T , and *branches*, C and D ,
- **while** T **do** C **od**—called a *while loop* with *test*, T , and *body*, C .

For simplicity we’ll stick to **while** programs operating on integers. So by expressions we’ll mean any of the familiar integer valued expressions involving integer constants and operations such as addition, multiplication, exponentiation, quotient or remainder. As *tests*, we’ll allow propositional formulas built from basic formulas of the form $e \leq f$ where e and f are expressions. For example, here is the Euclidean algorithm for $\text{gcd}(a, b)$ expressed as a **while** program.

```
x := a;
y := b;
while y  $\neq$  0 do
  t := y;
  y := rem(x, y);
  x := t od
```

13.2.2 The While Program State Machine

A **while** program acts as a pure command: it is run solely for its side effects on stored data and it doesn’t return a value. The data consists of integers stored as the values of variables, namely environments:

Definition 13.2.2. An *environment* is a total function from variables to integers. Let Env be set of all environments.

So if ρ is an environment and x is a variable, then $\rho(x)$ is an integer. More generally, the environment determines the integer value of each expression, e , and the truth value of each test, T . We can think of an expression, e as defining a function $\llbracket e \rrbracket : \text{Env} \rightarrow \mathbb{Z}$, and refer to this function, $\llbracket e \rrbracket$ as the *meaning* of e , and likewise for tests.

It’s standard in programming language theory to write $\llbracket e \rrbracket \rho$ as shorthand for $\llbracket e \rrbracket(\rho)$, that is, applying the *meaning*, $\llbracket e \rrbracket$, of e to ρ . For example, if $\rho(x) = 4$, and $\rho(y) = -2$, then

$$\llbracket x^2 + y - 3 \rrbracket \rho = \rho(x)^2 + \rho(y) - 3 = 11. \quad (13.3)$$

Executing a program causes a succession of changes to the environment¹ which may continue until the program halts. Actually the only command which immediately alters the environment is an assignment command. Namely, the effect of the command

$$x := e$$

on an environment is that the value assigned to the variable x is changed to the value of e in the original environment. We can say this precisely and concisely using the following notation: $f[a \leftarrow b]$ is a function that is the same as the function, f , except that when applied to element a its value is b . Namely,

Definition 13.2.3. If $f : A \rightarrow B$ is a function and a, b are arbitrary elements, define

$$f[a \leftarrow b]$$

to be the function g such that

$$g(u) = \begin{cases} b & \text{if } u = a. \\ f(u) & \text{otherwise.} \end{cases}$$

Now we can specify the step-by-step execution of a **while** program as a state machine, where the states of the machine consist of a **while** program paired with an environment. The transitions of this state machine are defined recursively on the definition of **while** programs.

Definition 13.2.4. The transitions $\langle C, \rho \rangle \longrightarrow \langle D, \rho' \rangle$ of the **while** program state machine are defined as follows:

base cases:

$$\langle x := e, \rho \rangle \longrightarrow \langle \mathbf{Done}, \rho[x \leftarrow \llbracket e \rrbracket \rho] \rangle$$

constructor cases: If C and D are **while** programs, and T is a test, then:

- if $\langle C, \rho \rangle \longrightarrow \langle C', \rho' \rangle$, then

$$\langle C; D, \rho \rangle \longrightarrow \langle C'; D, \rho' \rangle.$$

Also,

$$\langle \mathbf{Done}; D, \rho \rangle \longrightarrow \langle D, \rho \rangle.$$

¹More sophisticated programming models distinguish the environment from a *store* which is affected by commands, but this distinction is unnecessary for our purposes.

- if $\llbracket T \rrbracket \rho = \mathbf{T}$, then

$$\langle \text{if } T \text{ then } C \text{ else } D, \rho \rangle \longrightarrow \langle C, \rho \rangle,$$

- or if $\llbracket T \rrbracket \rho = \mathbf{F}$, then

$$\langle \text{if } T \text{ then } C \text{ else } D, \rho \rangle \longrightarrow \langle D, \rho \rangle.$$

- if $\llbracket T \rrbracket \rho = \mathbf{T}$, then

$$\langle \text{while } T \text{ do } C \text{ od}, \rho \rangle \longrightarrow \langle C; \text{while } T \text{ do } C \text{ od}, \rho \rangle$$

- or if $\llbracket T \rrbracket \rho = \mathbf{F}$, then

$$\langle \text{while } T \text{ do } C \text{ od}, \rho \rangle \longrightarrow \langle \text{Done}, \rho \rangle.$$

Now **while** programs are probably going to be the simplest kind of programs you will ever see, but being condescending about them would be a mistake. It turns that *every function on nonnegative integers that can be computed by any program* on any machine whatsoever can also be computed by **while** programs (maybe more slowly). We can't take the time to explain how such a sweeping claim can be justified, but you can find out by taking a course in computability theory such as 6.045 or 6.840.

13.2.3 Denotational Semantics

The net effect of starting a **while** program in some environment is reflected in the final environment when the program halts. So we can think of a **while** program, C , as defining a function, $\llbracket C \rrbracket : \text{Env} \rightarrow \text{Env}$, from initial environments to environments at halting. The function $\llbracket C \rrbracket$ is called the *meaning* of C .

$\llbracket C \rrbracket$ of a **while** program, C to be a partial function from Env to Env mapping an initial environment to the final halting environment.

We'll need one bit of notation first. For any function $f : S \rightarrow S$, let $f^{(n)}$ be the composition of f with itself n times where $n \in \mathbb{N}$. Namely,

$$\begin{aligned} f^{(0)} &::= \text{Id}_S \\ f^{(n+1)} &::= f \circ f^{(n)}, \end{aligned}$$

where “ $\circ$ ” denotes functional composition.

The recursive definition of the meaning of a program follows the definition of the **while** program recursive data type.

Definition 13.2.5. base cases:

- $\llbracket x := e \rrbracket$ is the function from Env to Env defined by the rule:

$$\llbracket x := e \rrbracket \rho ::= \rho[x \leftarrow \llbracket e \rrbracket \rho].$$

-

$$\llbracket \mathbf{Done} \rrbracket ::= \text{Id}_{\text{Env}}$$

where Id_{Env} is the identity function on Env. In other words, $\llbracket \mathbf{Done} \rrbracket \rho ::= \rho$.

constructor cases: If C and D are **while** programs, and T is a test, then:

-

$$\llbracket C; D \rrbracket ::= \llbracket D \rrbracket \circ \llbracket C \rrbracket$$

That is,

$$\llbracket C; D \rrbracket \rho ::= \llbracket D \rrbracket (\llbracket C \rrbracket \rho).$$

-

$$\llbracket \text{if } T \text{ then } C \text{ else } D \rrbracket \rho ::= \begin{cases} \llbracket C \rrbracket \rho & \text{if } \llbracket T \rrbracket \rho = \mathbf{T} \\ \llbracket D \rrbracket \rho & \text{if } \llbracket T \rrbracket \rho = \mathbf{F}. \end{cases}$$

-

$$\llbracket \text{while } T \text{ do } C \text{ od} \rrbracket \rho ::= \llbracket C \rrbracket^{(n)} \rho$$

where n is the least nonnegative integer such that $\llbracket T \rrbracket (\llbracket C \rrbracket^{(n)} \rho) = \mathbf{F}$. (If there is no such n , then $\llbracket \text{while } T \text{ do } C \text{ od} \rrbracket \rho$ is undefined.)

We can use the denotational semantics of **while** programs to reason about **while** programs using structural induction on programs, and this is often much simpler than reasoning about them using induction on the number of steps in an execution. This is OK as long as the denotational semantics accurately captures the state machine behavior. In particular, using the notation $\longrightarrow^*$ for the transitive closure of the transition relation:

Theorem 13.2.6.

$$\langle C, \rho \rangle \longrightarrow^* \langle \mathbf{Done}, \rho' \rangle \quad \text{iff} \quad \llbracket C \rrbracket \rho = \rho'$$

Theorem 13.2.6 can be proved easily by induction; it appear in Problem 13.1.

13.2.4 Logic of Programs

A typical program specification describes the kind of inputs and environments the program should handle, and then describes what should result from an execution. The specification of the inputs or initial environment is called the *precondition* for program execution, and the prescription of what the result of execution should be is called the *postcondition*. So if P is a logical formula expressing the precondition for a program, C , and likewise Q expresses the postcondition, the specification requires that

If P holds when C is started, then Q will hold if and when C halts.

We’ll express this requirement as a formula

$$P \{C\} Q$$

called a *partial correctness assertion*.

For example, if E is the **while** program above for the Euclidean algorithm, then the partial correctness of E can be expressed as

$$(a, b \in \mathbb{N} \text{ AND } a \neq 0) \{E\} (x = \text{gcd}(a, b)). \quad (13.4)$$

More precisely, notice that just as the value of an expression in an environment is an integer, the value of a logical formula in an environment is a truth value. For example, if $\rho(x) = 4$, and $\rho(y) = -2$, then by (13.3), $\llbracket x^2 + y - 3 \rrbracket \rho = 11$, so

$$\begin{aligned} \llbracket \exists z. z > 4 \text{ AND } x^2 + y - 3 = z \rrbracket \rho &= \mathbf{T}, \\ \llbracket \exists z. z > 13 \text{ AND } x^2 + y - 3 = z \rrbracket \rho &= \mathbf{F}. \end{aligned}$$

Definition 13.2.7. For logical formulas P and Q , and **while** program, C , the partial correctness assertion

$$P \{C\} Q$$

is true proving that for all environments, *rho*, if $\llbracket P \rrbracket \rho$ is true, and $\langle C, \rho \rangle \longrightarrow^* \langle \mathbf{Done}, \rho' \rangle$ for some ρ' , then $\llbracket Q \rrbracket \rho'$ is true.

In the 1970’s, Prof. Tony Hoare (now Sir Anthony) at Univ. Dublin formulated a set of inference rules for proving partial correctness formulas. These rules are known as *Hoare Logic*.

The first rule captures the fact that strengthening the preconditions and weakening the postconditions makes a partial correctness specification easier to satisfy:

$$\frac{P \text{ IMPLIES } R, \quad R \{C\} S, \quad S \text{ IMPLIES } Q}{P \{C\} Q}$$

The rest of the logical rules follow the recursive definition of **while** programs. There are axioms for the base case commands:

$$\frac{P(x) \{x := e\} P(e)}{P \{\mathbf{Done}\} P},$$

and proof rules for the constructor cases:

- $$\frac{P \{C\} Q \text{ AND } Q \{D\} R}{P \{C;D\} R}$$
- $$\frac{P \text{ AND } T \{C\} Q}{P \text{ AND } T \{\mathbf{if } T \text{ then } C \text{ else } D\} Q \text{ AND } T}$$
- $$\frac{P \text{ AND } T \{C\} P}{P \{\mathbf{while } T \text{ do } C \text{ od}\} P \text{ AND NOT}(T)}$$

Example 13.2.8. **TBA - Formal correctness proof of (13.4) for the Euclidean algorithm.**

Problems for Section 13.2

Homework Problems

Problem 13.1.

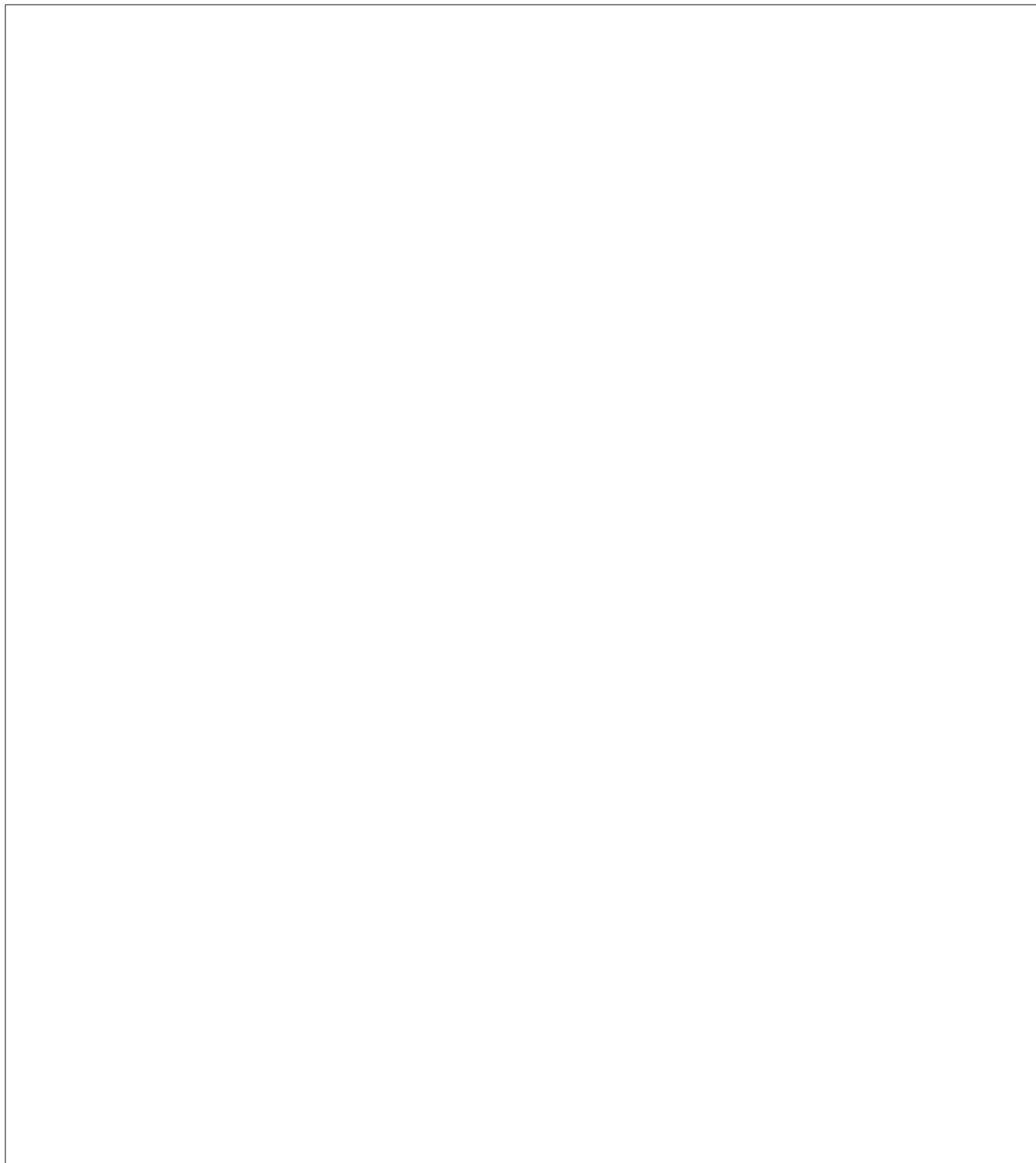
Prove Theorem 13.2.6:

Theorem.

$$\langle C, \rho \rangle \longrightarrow^* \langle \mathbf{Done}, \rho' \rangle \quad \text{iff} \quad \llbracket C \rrbracket \rho = \rho'$$

Hint: Prove the left to right direction by induction on the number of steps C needs to halt starting in environment ρ . Prove the right to left direction by structural induction on the definition of **while** programs. Both proofs follow almost mechanically from the definitions.

III Counting



Introduction

Counting is useful in computer science for several reasons:

- Determining the time and storage required to solve a computational problem —a central objective in computer science —often comes down to solving a counting problem.
- Counting is the basis of probability theory, which plays a central role in all sciences, including computer science.
- Two remarkable proof techniques, the “pigeonhole principle” and “combinatorial proof,” rely on counting.

Counting seems easy enough: 1, 2, 3, 4, etc. This direct approach works well for counting simple things —like your toes —and may be the only approach for extremely complicated things with no identifiable structure. However, subtler methods can help you count many things in the vast middle ground, such as:

- The number of different ways to select a dozen doughnuts when there are five varieties available.
- The number of 16-bit numbers with exactly 4 ones.

Perhaps surprisingly, but certainly not coincidentally, these two numbers are the same: 1820.

We begin our study of counting in Chapter 14 with a collection of rules and methods for finding closed-form expressions for commonly-occurring sums and products such as $\sum_{i=1}^n x^i$ and $n! = \prod_{i=1}^n i$. We also introduce asymptotic notations such as $\sim$, O , and Θ that are commonly used in computer science to express

the how a quantity such as the running time of a program grows with the size of the input.

Chapter 15 describes the most basic rules for determining the cardinality of a set. These rules are actually theorems, but our focus won’t be on their proofs *per se*—our objective is to teach you simple counting as a practical skill, like integration.

But counting can be tricky, and people make counting mistakes all the time, so a crucial part of counting skill is being able to verify a counting argument. Sometimes this can be done simply by finding an alternative way to count and then comparing answers—they better agree. But most elementary counting arguments reduce to finding a bijection between objects to be counted and easy-to-count sequences. The chapter shows how explicitly defining these bijections—and verifying that they are bijections—is another useful way to verify counting arguments. The material in Chapter 15 is simple yet powerful, and it provides a great tool set for use in your future career.

14 Sums and Asymptotics

Sums and products arise regularly in the analysis of algorithms, financial applications, physical problems, and probabilistic systems. For example, according to Theorem 2.2.1,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}. \quad (14.1)$$

Of course the lefthand sum could be expressed concisely as a subscripted summation

$$\sum_{i=1}^n i,$$

but the right hand expression $n(n+1)/2$ is not only concise, it is also easier to evaluate, and it more clearly reveals properties such as the growth rate of the sum. Expressions like $n(n+1)/2$ that do not make use of subscripted summations or products —or those handy but sometimes troublesome dots—are called *closed forms*.

Another example is the closed form for a *geometric sum*

$$1 + x + x^2 + x^3 + \cdots + x^n = \frac{1 - x^{n+1}}{1 - x} \quad (14.2)$$

given in Problem 6.2. The sum as described on the left hand side of (14.2) involves n additions and $1 + 2 + \cdots + (n-1) = (n-1)n/2$ multiplications, but its closed form on the right hand side can be evaluated using fast exponentiation with at most $2 \log n$ multiplications and a couple of subtractions. Also, the closed form makes the growth and limiting behavior of the sum much more apparent.

Equations (14.1) and (14.2) were easy to verify by induction, but, as is often the case, the proofs by induction gave no hint about how these formulas were found in the first place. Finding them is part math and part art, which we’ll start examining in this chapter.

A first motivating example will be figuring out the value of the annuity. The value will be a large and nasty-looking sum. We will then describe several methods for finding closed forms for several sorts of sums, including the annuity sums. In some cases, a closed form for a sum may not exist and so we will provide a general method for finding closed forms for good upper and lower bounds on the sum.

The methods we develop for sums will also work for products since any product can be converted into a sum by taking a logarithm of the product. As an example,

we will use this approach to find a good closed-form approximation to the *factorial function*

$$n! ::= 1 \cdot 2 \cdot 3 \cdots n.$$

We conclude the chapter with a discussion of asymptotic notation. Asymptotic notation is often used to bound the error terms when there is no exact closed form expression for a sum or product. It also provides a convenient way to express the growth rate or order of magnitude of a sum or product.

14.1 The Value of an Annuity

Would you prefer a million dollars today or \$50,000 a year for the rest of your life? On the one hand, instant gratification is nice. On the other hand, the *total dollars* received at \$50K per year is much larger if you live long enough.

Formally, this is a question about the value of an annuity. An *annuity* is a financial instrument that pays out a fixed amount of money at the beginning of every year for some specified number of years. In particular, an n -year, m -payment annuity pays m dollars at the start of each year for n years. In some cases, n is finite, but not always. Examples include lottery payouts, student loans, and home mortgages. There are even Wall Street people who specialize in trading annuities.¹

A key question is, “What is an annuity worth?” For example, lotteries often pay out jackpots over many years. Intuitively, \$50,000 a year for 20 years ought to be worth less than a million dollars right now. If you had all the cash right away, you could invest it and begin collecting interest. But what if the choice were between \$50,000 a year for 20 years and a *half* million dollars today? Now it is not clear which option is better.

14.1.1 The Future Value of Money

In order to answer such questions, we need to know what a dollar paid out in the future is worth today. To model this, let’s assume that money can be invested at a fixed annual interest rate p . We’ll assume an 8% rate² for the rest of the discussion.

Here is why the interest rate p matters. Ten dollars invested today at interest rate p will become $(1 + p) \cdot 10 = 10.80$ dollars in a year, $(1 + p)^2 \cdot 10 \approx 11.66$ dollars

¹Such trading ultimately led to the subprime mortgage disaster in 2008–2009. We’ll talk more about that in a later chapter.

²U.S. interest rates have dropped steadily for several years, and ordinary bank deposits now earn around 1.0%. But just a few years ago the rate was 8%; this rate makes some of our examples a little more dramatic. The rate has been as high as 17% in the past thirty years.

in two years, and so forth. Looked at another way, ten dollars paid out a year from now is only really worth $1/(1+p) \cdot 10 \approx 9.26$ dollars today. The reason is that if we had the \$9.26 today, we could invest it and would have \$10.00 in a year anyway. Therefore, p determines the value of money paid out in the future.

So for an n -year, m -payment annuity, the first payment of m dollars is truly worth m dollars. But the second payment a year later is worth only $m/(1+p)$ dollars. Similarly, the third payment is worth $m/(1+p)^2$, and the n -th payment is worth only $m/(1+p)^{n-1}$. The total value, V , of the annuity is equal to the sum of the payment values. This gives:

$$\begin{aligned} V &= \sum_{i=1}^n \frac{m}{(1+p)^{i-1}} \\ &= m \cdot \sum_{j=0}^{n-1} \left(\frac{1}{1+p} \right)^j && \text{(substitute } j = i - 1) \\ &= m \cdot \sum_{j=0}^{n-1} x^j && \text{(substitute } x = 1/(1+p)). \end{aligned} \quad (14.3)$$

The goal of the preceding substitutions was to get the summation into the form of a simple geometric sum. This leads us to an explanation of a way you could have discovered the closed form (14.2) in the first place using the *Perturbation Method*.

14.1.2 The Perturbation Method

Given a sum that has a nice structure, it is often useful to “perturb” the sum so that we can somehow combine the sum with the perturbation to get something much simpler. For example, suppose

$$S = 1 + x + x^2 + \cdots + x^n.$$

An example of a perturbation would be

$$xS = x + x^2 + \cdots + x^{n+1}.$$

The difference between S and xS is not so great, and so if we were to subtract xS from S , there would be massive cancellation:

$$\begin{aligned} S &= 1 + x + x^2 + x^3 + \cdots + x^n \\ -xS &= -x - x^2 - x^3 - \cdots - x^n - x^{n+1}. \end{aligned}$$

The result of the subtraction is

$$S - xS = 1 - x^{n+1}.$$

Solving for S gives the desired closed-form expression in equation 14.2, namely,

$$S = \frac{1 - x^{n+1}}{1 - x}.$$

We’ll see more examples of this method when we introduce *generating functions* in a later chapter.

14.1.3 A Closed Form for the Annuity Value

Using equation 14.2, we can derive a simple formula for V , the value of an annuity that pays m dollars at the start of each year for n years.

$$V = m \left(\frac{1 - x^n}{1 - x} \right) \quad (\text{by equations 14.3 and 14.2}) \quad (14.4)$$

$$= m \left(\frac{1 + p - (1/(1 + p))^{n-1}}{p} \right) \quad (\text{substituting } x = 1/(1 + p)). \quad (14.5)$$

Equation 14.5 is much easier to use than a summation with dozens of terms. For example, what is the real value of a winning lottery ticket that pays \$50,000 per year for 20 years? Plugging in $m = \$50,000$, $n = 20$, and $p = 0.08$ gives $V \approx \$530,180$. So because payments are deferred, the million dollar lottery is really only worth about a half million dollars! This is a good trick for the lottery advertisers.

14.1.4 Infinite Geometric Series

The question we began with was whether you would prefer a million dollars today or \$50,000 a year for the rest of your life. Of course, this depends on how long you live, so optimistically assume that the second option is to receive \$50,000 a year *forever*. This sounds like infinite money! But we can compute the value of an annuity with an infinite number of payments by taking the limit of our geometric sum in equation 14.2 as n tends to infinity.

Theorem 14.1.1. *If $|x| < 1$, then*

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1 - x}.$$

Proof.

$$\begin{aligned}\sum_{i=0}^{\infty} x^i &::= \lim_{n \rightarrow \infty} \sum_{i=0}^n x^i \\ &= \lim_{n \rightarrow \infty} \frac{1 - x^{n+1}}{1 - x} && \text{(by equation 14.2)} \\ &= \frac{1}{1 - x}.\end{aligned}$$

The final line follows from that fact that $\lim_{n \rightarrow \infty} x^{n+1} = 0$ when $|x| < 1$. ■

In our annuity problem, $x = 1/(1 + p) < 1$, so Theorem 14.1.1 applies, and we get

$$\begin{aligned}V &= m \cdot \sum_{j=0}^{\infty} x^j && \text{(by equation 14.3)} \\ &= m \cdot \frac{1}{1 - x} && \text{(by Theorem 14.1.1)} \\ &= m \cdot \frac{1 + p}{p} && (x = 1/(1 + p)).\end{aligned}$$

Plugging in $m = \$50,000$ and $p = 0.08$, we see that the value V is only \$675,000. Amazingly, a million dollars today is worth much more than \$50,000 paid every year forever! Then again, if we had a million dollars today in the bank earning 8% interest, we could take out and spend \$80,000 a year forever. So on second thought, this answer really isn’t so amazing.

14.1.5 Examples

Equation 14.2 and Theorem 14.1.1 are incredibly useful in computer science.

Here are some other common sums that can be put into closed form using equa-

tion 14.2 and Theorem 14.1.1:

$$1 + 1/2 + 1/4 + \cdots = \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = \frac{1}{1 - (1/2)} = 2 \quad (14.6)$$

$$0.99999\cdots = 0.9 \sum_{i=0}^{\infty} \left(\frac{1}{10}\right)^i = 0.9 \left(\frac{1}{1 - 1/10}\right) = 0.9 \left(\frac{10}{9}\right) = 1 \quad (14.7)$$

$$1 - 1/2 + 1/4 - \cdots = \sum_{i=0}^{\infty} \left(\frac{-1}{2}\right)^i = \frac{1}{1 - (-1/2)} = \frac{2}{3} \quad (14.8)$$

$$1 + 2 + 4 + \cdots + 2^{n-1} = \sum_{i=0}^{n-1} 2^i = \frac{1 - 2^n}{1 - 2} = 2^n - 1 \quad (14.9)$$

$$1 + 3 + 9 + \cdots + 3^{n-1} = \sum_{i=0}^{n-1} 3^i = \frac{1 - 3^n}{1 - 3} = \frac{3^n - 1}{2} \quad (14.10)$$

If the terms in a geometric sum grow smaller, as in equation 14.6, then the sum is said to be *geometrically decreasing*. If the terms in a geometric sum grow progressively larger, as in equations 14.9 and 14.10, then the sum is said to be *geometrically increasing*. In either case, the sum is usually approximately equal to the term in the sum with the greatest absolute value. For example, in equations 14.6 and 14.8, the largest term is equal to 1 and the sums are 2 and 2/3, both relatively close to 1. In equation 14.9, the sum is about twice the largest term. In equation 14.10, the largest term is 3^{n-1} and the sum is $(3^n - 1)/2$, which is only about a factor of 1.5 greater. You can see why this rule of thumb works by looking carefully at equation 14.2 and Theorem 14.1.1.

14.1.6 Variations of Geometric Sums

We now know all about geometric sums —if you have one, life is easy. But in practice one often encounters sums that cannot be transformed by simple variable substitutions to the form $\sum x^i$.

A non-obvious, but useful way to obtain new summation formulas from old ones is by differentiating or integrating with respect to x . As an example, consider the following sum:

$$\sum_{i=1}^{n-1} i x^i = x + 2x^2 + 3x^3 + \cdots + (n-1)x^{n-1}$$

This is not a geometric sum, since the ratio between successive terms is not fixed, and so our formula for the sum of a geometric sum cannot be directly applied. But

differentiating equation 14.2 leads to:

$$\frac{d}{dx} \left(\sum_{i=0}^{n-1} x^i \right) = \frac{d}{dx} \left(\frac{1-x^n}{1-x} \right). \quad (14.11)$$

The left-hand side of equation 14.11 is simply

$$\sum_{i=0}^{n-1} \frac{d}{dx} (x^i) = \sum_{i=0}^{n-1} i x^{i-1}.$$

The right-hand side of equation 14.11 is

$$\begin{aligned} \frac{-nx^{n-1}(1-x) - (-1)(1-x^n)}{(1-x)^2} &= \frac{-nx^{n-1} + nx^n + 1 - x^n}{(1-x)^2} \\ &= \frac{1 - nx^{n-1} + (n-1)x^n}{(1-x)^2}. \end{aligned}$$

Hence, equation 14.11 means that

$$\sum_{i=0}^{n-1} i x^{i-1} = \frac{1 - nx^{n-1} + (n-1)x^n}{(1-x)^2}.$$

Incidentally, Problem 14.2 shows how the perturbation method could also be applied to derive this formula.

Often, differentiating or integrating messes up the exponent of x in every term. In this case, we now have a formula for a sum of the form $\sum i x^{i-1}$, but we want a formula for the series $\sum i x^i$. The solution is simple: multiply by x . This gives:

$$\sum_{i=1}^{n-1} i x^i = \frac{x - nx^n + (n-1)x^{n+1}}{(1-x)^2} \quad (14.12)$$

and we have the desired closed-form expression for our sum³. It's a little complicated looking, but it's easier to work with than the sum.

Notice that if $|x| < 1$, then this series converges to a finite value even if there are infinitely many terms. Taking the limit of equation 14.12 as n tends infinity gives the following theorem:

³Since we could easily have made a mistake in the calculation, it is always a good idea to go back and validate a formula obtained this way with a proof by induction.

Theorem 14.1.2. *If $|x| < 1$, then*

$$\sum_{i=1}^{\infty} ix^i = \frac{x}{(1-x)^2}. \quad (14.13)$$

As a consequence, suppose that there is an annuity that pays im dollars at the end of each year i forever. For example, if $m = \$50,000$, then the payouts are \$50,000 and then \$100,000 and then \$150,000 and so on. It is hard to believe that the value of this annuity is finite! But we can use Theorem 14.1.2 to compute the value:

$$\begin{aligned} V &= \sum_{i=1}^{\infty} \frac{im}{(1+p)^i} \\ &= m \cdot \frac{1/(1+p)}{(1 - \frac{1}{1+p})^2} \\ &= m \cdot \frac{1+p}{p^2}. \end{aligned}$$

The second line follows by an application of Theorem 14.1.2. The third line is obtained by multiplying the numerator and denominator by $(1+p)^2$.

For example, if $m = \$50,000$, and $p = 0.08$ as usual, then the value of the annuity is $V = \$8,437,500$. Even though the payments increase every year, the increase is only additive with time; by contrast, dollars paid out in the future decrease in value exponentially with time. The geometric decrease swamps out the additive increase. Payments in the distant future are almost worthless, so the value of the annuity is finite.

The important thing to remember is the trick of taking the derivative (or integral) of a summation formula. Of course, this technique requires one to compute nasty derivatives correctly, but this is at least theoretically possible!

14.2 Sums of Powers

In Chapter 6, we verified the formula (14.1), but the source of this formula is still a mystery. Sure, we can prove it is true using well ordering or induction, but where did the expression on the right come from in the first place? Even more inexplicable is the closed form expression for the sum of consecutive squares:

$$\sum_{i=1}^n i^2 = \frac{(2n+1)(n+1)n}{6}. \quad (14.14)$$

It turns out that there is a way to derive these expressions, but before we explain it, we thought it would be fun⁴ to show you how Gauss is supposed to have proved equation 14.1 when he was a young boy.

Gauss’s idea is related to the perturbation method we used in Section 14.1.2. Let

$$S = \sum_{i=1}^n i.$$

Then we can write the sum in two orders:

$$\begin{aligned} S &= 1 + 2 + \dots + (n-1) + n, \\ S &= n + (n-1) + \dots + 2 + 1. \end{aligned}$$

Adding these two equations gives

$$\begin{aligned} 2S &= (n+1) + (n+1) + \dots + (n+1) + (n+1) \\ &= n(n+1). \end{aligned}$$

Hence,

$$S = \frac{n(n+1)}{2}.$$

Not bad for a young child —Gauss showed some potential. . .

Unfortunately, the same trick does not work for summing consecutive squares. However, we can observe that the result might be a third-degree polynomial in n , since the sum contains n terms that average out to a value that grows quadratically in n . So we might guess that

$$\sum_{i=1}^n i^2 = an^3 + bn^2 + cn + d.$$

If the guess is correct, then we can determine the parameters a , b , c , and d by plugging in a few values for n . Each such value gives a linear equation in a , b , c , and d . If we plug in enough values, we may get a linear system with a unique solution. Applying this method to our example gives:

$$\begin{aligned} n = 0 & \text{ implies } 0 = d \\ n = 1 & \text{ implies } 1 = a + b + c + d \\ n = 2 & \text{ implies } 5 = 8a + 4b + 2c + d \\ n = 3 & \text{ implies } 14 = 27a + 9b + 3c + d. \end{aligned}$$

⁴OK, our definition of “fun” may be different than yours.

Solving this system gives the solution $a = 1/3$, $b = 1/2$, $c = 1/6$, $d = 0$. Therefore, *if* our initial guess at the form of the solution was correct, then the summation is equal to $n^3/3 + n^2/2 + n/6$, which matches equation 14.14.

The point is that if the desired formula turns out to be a polynomial, then once you get an estimate of the *degree* of the polynomial, all the coefficients of the polynomial can be found automatically.

Be careful! This method lets you discover formulas, but it doesn’t guarantee they are right! After obtaining a formula by this method, it’s important to go back and *prove* it using induction or some other method, because if the initial guess at the solution was not of the right form, then the resulting formula will be completely wrong! A later chapter will describe a method based on generating functions that does not require any guessing at all.

14.3 Approximating Sums

Unfortunately, it is not always possible to find a closed-form expression for a sum. For example, consider the sum

$$S = \sum_{i=1}^n \sqrt{i}.$$

No closed form expression is known for S .

In such cases, we need to resort to approximations for S if we want to have a closed form. The good news is that there is a general method to find closed-form upper and lower bounds that work for most any sum. Even better, the method is simple and easy to remember. It works by replacing the sum by an integral and then adding either the first or last term in the sum.

Definition 14.3.1. A function $f : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ is *strictly increasing* when

$$x < y \text{ IMPLIES } f(x) < f(y),$$

and it is *weakly increasing*⁵ when

$$x < y \text{ IMPLIES } f(x) \leq f(y).$$

⁵Weakly increasing functions are usually called *nondecreasing* functions. We will avoid this terminology to prevent confusion between being a nondecreasing function and the much weaker property of *not* being a decreasing function.

Similarly, f is *strictly decreasing* when

$$x < y \text{ IMPLIES } f(x) > f(y),$$

and it is *weakly decreasing*⁶ when

$$x < y \text{ IMPLIES } f(x) \geq f(y).$$

For example, 2^x and $\sqrt{x}$ are strictly increasing functions, while $\max x, 2$ and $\lceil x \rceil$ are weakly increasing functions. The functions $1/x$ and 2^{-x} are strictly decreasing, while $\min 1/x, 1/2$ and $\lfloor 1/x \rfloor$ are weakly decreasing.

Theorem 14.3.2. *Let $f : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ be a weakly increasing function. Define*

$$S ::= \sum_{i=1}^n f(i) \tag{14.15}$$

and

$$I ::= \int_1^n f(x) dx.$$

Then

$$I + f(1) \leq S \leq I + f(n). \tag{14.16}$$

Similarly, if f is weakly decreasing, then

$$I + f(n) \leq S \leq I + f(1).$$

Proof. Suppose $f : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ is weakly increasing. The value of the sum S in (14.15) is the sum of the areas of n unit-width rectangles of heights $f(1), f(2), \dots, f(n)$. This area of these rectangles is shown shaded in Figure 14.1.

The value of

$$I = \int_1^n f(x) dx$$

is the shaded area under the curve of $f(x)$ from 1 to n shown in Figure 14.2.

Comparing the shaded regions in Figures 14.1 and 14.2 shows that S is at least I plus the area of the leftmost rectangle. Hence,

$$S \geq I + f(1) \tag{14.17}$$

This is the lower bound for S given in (14.16).

To derive the upper bound for S given in (14.16), we shift the curve of $f(x)$ from 1 to n one unit to the left as shown in Figure 14.3.

⁶Weakly decreasing functions are usually called *nonincreasing*.

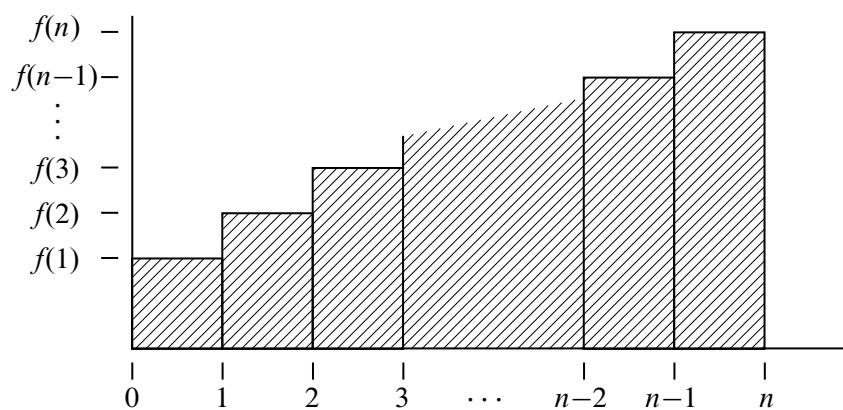


Figure 14.1 The area of the i th rectangle is $f(i)$. The shaded region has area $\sum_{i=1}^n f(i)$.

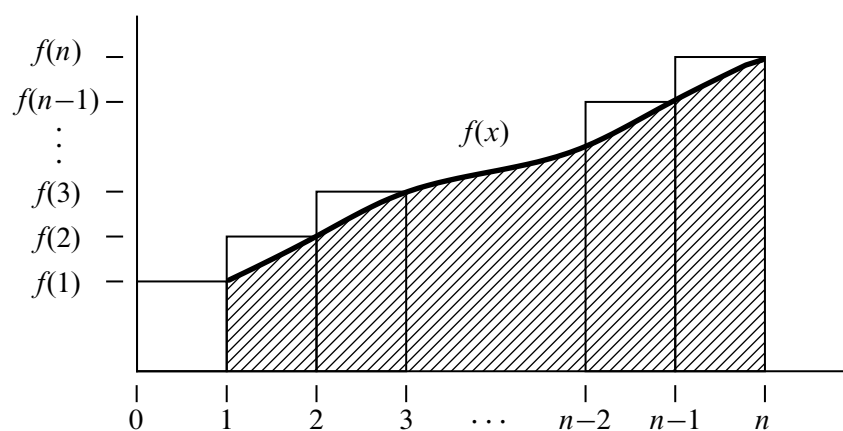


Figure 14.2 The shaded area under the curve of $f(x)$ from 1 to n (shown in bold) is $I = \int_1^n f(x) dx$.

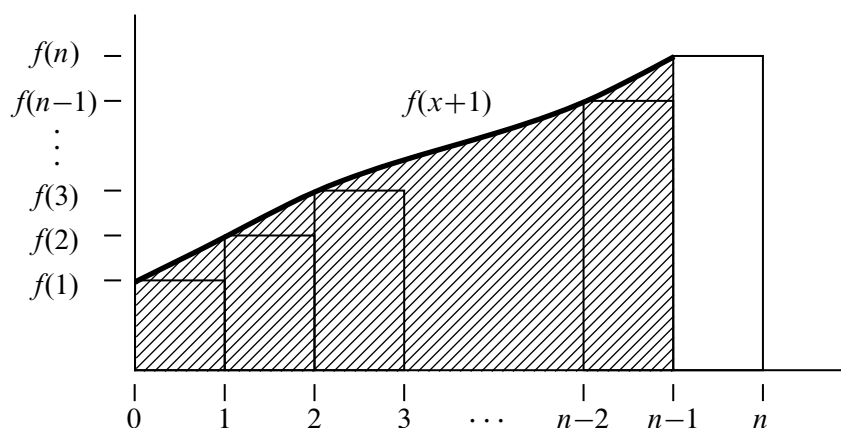


Figure 14.3 This curve is the same as the curve in Figure 14.2 shifted left by 1.

Comparing the shaded regions in Figures 14.1 and 14.3 shows that S is at most I plus the area of the rightmost rectangle. That is,

$$S \leq I + f(n),$$

which is the upper bound for S given in (14.16).

The very similar argument for the weakly decreasing case is left to Problem 14.7. ■

Theorem 14.3.2 provides good bounds for most sums. At worst, the bounds will be off by the largest term in the sum. For example, we can use Theorem 14.3.2 to bound the sum

$$S = \sum_{i=1}^n \sqrt{i}$$

as follows.

We begin by computing

$$\begin{aligned} I &= \int_1^n \sqrt{x} \, dx \\ &= \frac{x^{3/2}}{3/2} \Big|_1^n \\ &= \frac{2}{3}(n^{3/2} - 1). \end{aligned}$$

We then apply Theorem 14.3.2 to conclude that

$$\frac{2}{3}(n^{3/2} - 1) + 1 \leq S \leq \frac{2}{3}(n^{3/2} - 1) + \sqrt{n}$$

and thus that

$$\frac{2}{3}n^{3/2} + \frac{1}{3} \leq S \leq \frac{2}{3}n^{3/2} + \sqrt{n} - \frac{2}{3}.$$

In other words, the sum is very close to $\frac{2}{3}n^{3/2}$.

We’ll be using Theorem 14.3.2 extensively going forward. At the end of this chapter, we will also introduce some notation that expresses phrases like “the sum is very close to” in a more precise mathematical manner. But first, we’ll see how Theorem 14.3.2 can be used to resolve a classic paradox in structural engineering.

14.4 Hanging Out Over the Edge

Suppose we have n identical unit length rectangular blocks that are uniformly weighted. We want to stack them one on top of the next on a table as shown in Figure 14.4. Is there some value of n for which it is possible to arrange the stack so that one of the blocks hangs out completely over the edge of the table without having the stack fall over? (You are not allowed to use glue or otherwise hold the stack in position.)

Most people’s first response to this question —sometimes also their second and third responses —is “No. No block will ever get completely past the edge of the table.” But in fact, if n is large enough, you can get the top block to stick out as far as you want: one block-length, two block-lengths, any number of block-lengths!

14.4.1 Stability

A stack of blocks is said to be *stable* if it will not fall over of its own accord. For example, the stack illustrated in Figure 14.4 is not stable because the top block is sure to fall over. This is because the center or mass of the top block is hanging out over air.

In general, a stack of n blocks will be stable if and only if the center of mass of the top i blocks sits over the $(i + 1)$ st block for $i = 1, 2, \dots, n - 1$, and over the table for $i = n$.

We define the *overhang* of a stable stack to be the distance between the edge of the table and the rightmost end of the rightmost block in the stack. Our goal is thus to maximize the overhang of a stable stack.

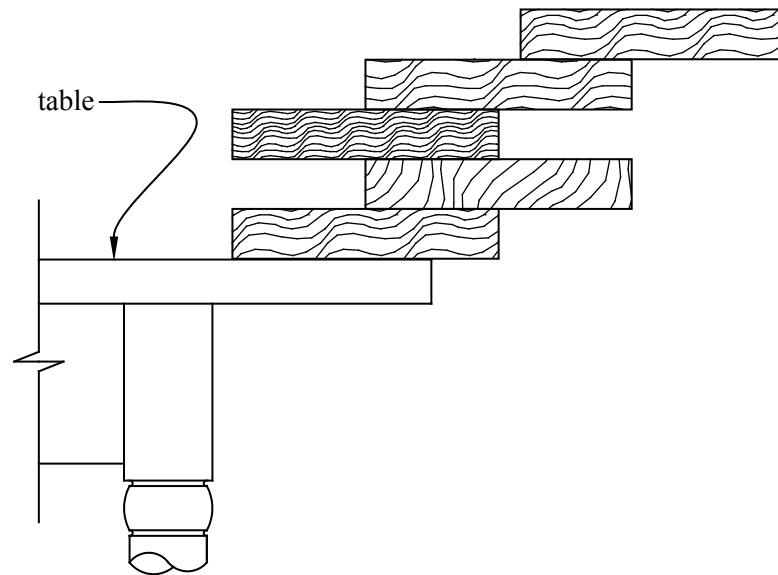


Figure 14.4 A stack of 5 identical blocks on a table. The top block is hanging out over the edge of the table, but if you try stacking the blocks this way, the stack will fall over.

For example, the maximum possible overhang for a single block is $1/2$. That is because the center of mass of a single block is in the middle of the block (which is distance $1/2$ from the right edge of the block). If we were to place the block so that its right edge is more than $1/2$ from the edge of the table, the center of mass would be over air and the block would tip over. But we can place the block so the center of mass is at the edge of the table, thereby achieving overhang $1/2$. This position is illustrated in Figure 14.5.

In general, the overhang of a stack of blocks is maximized by sliding the entire stack rightward until its center of mass is at the edge of the table. The overhang will then be equal to the distance between the center of mass of the stack and the rightmost edge of the rightmost block. We call this distance the *spread* of the stack. Note that the spread does not depend on the location of the stack on the table—it is purely a property of the blocks in the stack. Of course, as we just observed, the maximum possible overhang is equal to the maximum possible spread. This relationship is illustrated in Figure 14.6.

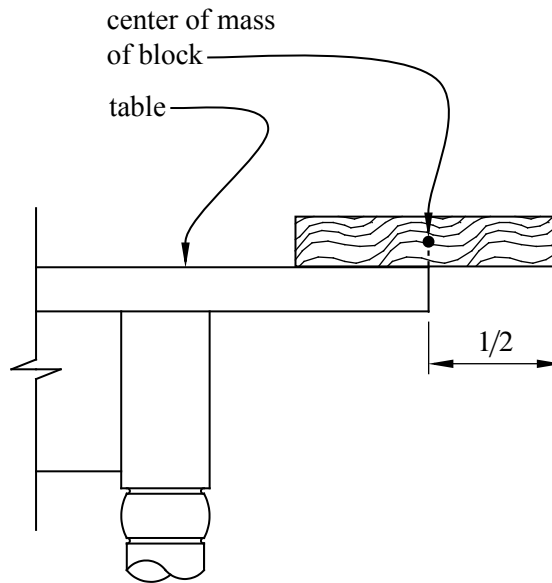


Figure 14.5 One block can overhang half a block length.

14.4.2 A Recursive Solution

Our goal is to find a formula for the maximum possible spread S_n that is achievable with a stable stack of n blocks.

We already know that $S_1 = 1/2$ since the right edge of a single block with length 1 is always distance $1/2$ from its center of mass. Let's see if we can use a recursive approach to determine S_n for all n . This means that we need to find a formula for S_n in terms of S_i where $i < n$.

Suppose we have a stable stack S of n blocks with maximum possible spread S_n . There are two cases to consider depending on where the rightmost block is in the stack.

Case 1: *The rightmost block in S is the bottom block.* Since the center of mass of the top $n - 1$ blocks must be over the bottom block for stability, the spread is maximized by having the center of mass of the top $n - 1$ blocks be directly over the left edge of the bottom block. In this case the center of mass of S is⁷

$$\frac{(n-1) \cdot 1 + (1) \cdot \frac{1}{2}}{n} = 1 - \frac{1}{2n}$$

⁷The center of mass of a stack of blocks is the average of the centers of mass of the individual blocks.

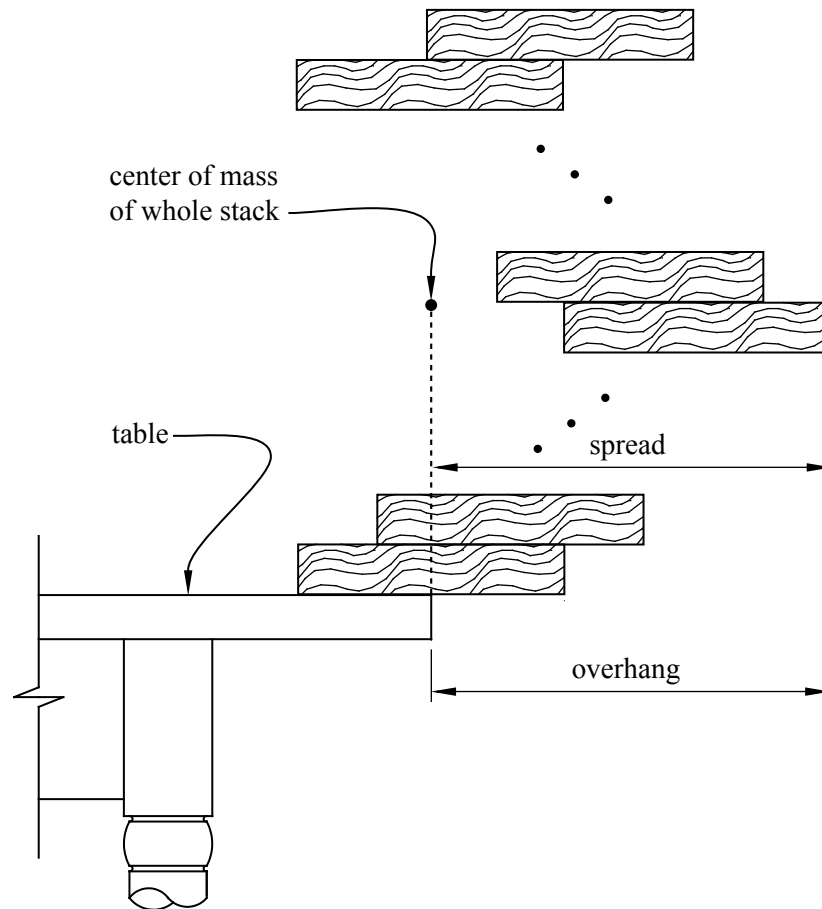


Figure 14.6 The overhang is maximized by maximizing the spread and then placing the stack so that the center of mass is at the edge of the table.

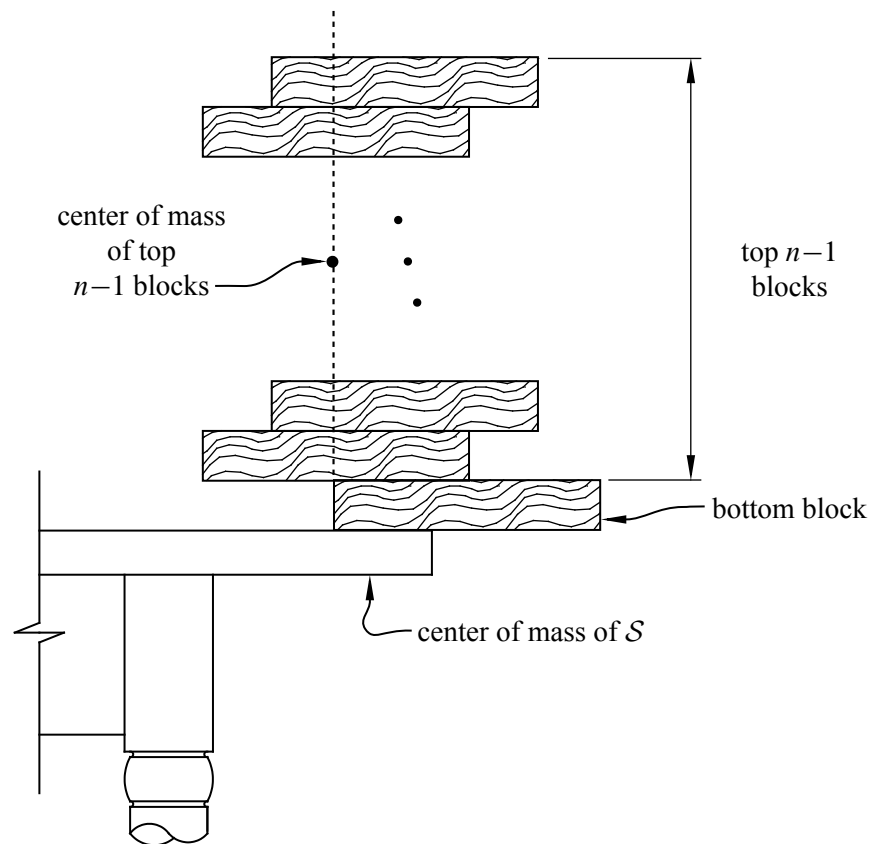


Figure 14.7 The scenario where the bottom block is the rightmost block. In this case, the spread is maximized by having the center of mass of the top $n - 1$ blocks be directly over the left edge of the bottom block.

to the left of the right edge of the bottom block and so the spread for $\mathcal{S}$ is

$$1 - \frac{1}{2n}. \quad (14.18)$$

For example, see Figure 14.7.

In fact, the scenario just described is easily achieved by arranging the blocks as shown in Figure 14.8, in which case we have the spread given by equation 14.18. For example, the spread is $3/4$ for 2 blocks, $5/6$ for 3 blocks, $7/8$ for 4 blocks, etc.

Can we do any better? The best spread in Case 1 is always less than 1, which means that we cannot get a block fully out over the edge of the table in this scenario. Maybe our intuition was right that we can't do better. Before we jump to any false conclusions, however, let's see what happens in the other case.

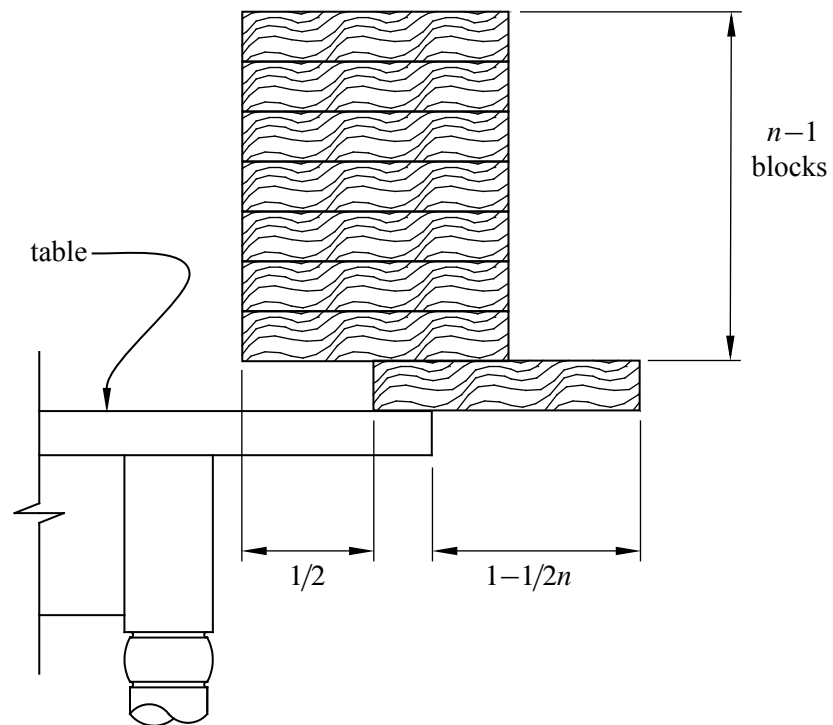


Figure 14.8 A method for achieving spread (and hence overhang) $1 - 1/2n$ with n blocks, where the bottom block is the rightmost block.

Case 2: *The rightmost block in $\mathcal{S}$ is among the top $n - 1$ blocks.* In this case, the spread is maximized by placing the top $n - 1$ blocks so that their center of mass is directly over the *right* end of the bottom block. This means that the center of mass for $\mathcal{S}$ is at location

$$\frac{(n-1) \cdot C + 1 \cdot (C - \frac{1}{2})}{n} = C - \frac{1}{2n}$$

where C is the location of the center of mass of the top $n - 1$ blocks. In other words, the center of mass of $\mathcal{S}$ is $1/2n$ to the left of the center of mass of the top $n - 1$ blocks. (The difference is due to the effect of the bottom block, whose center of mass is $1/2$ unit to the left of C .) This means that the spread of $\mathcal{S}$ is $1/2n$ greater than the spread of the top $n - 1$ blocks (because we are in the case where the rightmost block is among the top $n - 1$ blocks.)

Since the rightmost block is among the top $n - 1$ blocks, the spread for $\mathcal{S}$ is maximized by maximizing the spread for the top $n - 1$ blocks. Hence the maximum spread for $\mathcal{S}$ in this case is

$$S_{n-1} + \frac{1}{2n} \tag{14.19}$$

where S_{n-1} is the maximum possible spread for $n - 1$ blocks (using any strategy).

We are now almost done. There are only two cases to consider when designing a stack with maximum spread and we have analyzed both of them. This means that we can combine equation 14.18 from Case 1 with equation 14.19 from Case 2 to conclude that

$$S_n = \max \left\{ 1 - \frac{1}{2n}, S_{n-1} + \frac{1}{2n} \right\} \tag{14.20}$$

for any $n > 1$.

Uh-oh. This looks complicated. Maybe we are not almost done after all!

Equation 14.20 is an example of a *recurrence*. We will describe numerous techniques for solving recurrences in a later chapter, but, fortunately, equation 14.20 is simple enough that we can solve it directly.

One of the first things to do when you have a recurrence is to get a feel for it by computing the first few terms. This often gives clues about a way to solve the recurrence, as it will in this case.

We already know that $S_1 = 1/2$. What about S_2 ? From equation 14.20, we find that

$$\begin{aligned} S_2 &= \max \left\{ 1 - \frac{1}{4}, \frac{1}{2} + \frac{1}{4} \right\} \\ &= 3/4. \end{aligned}$$

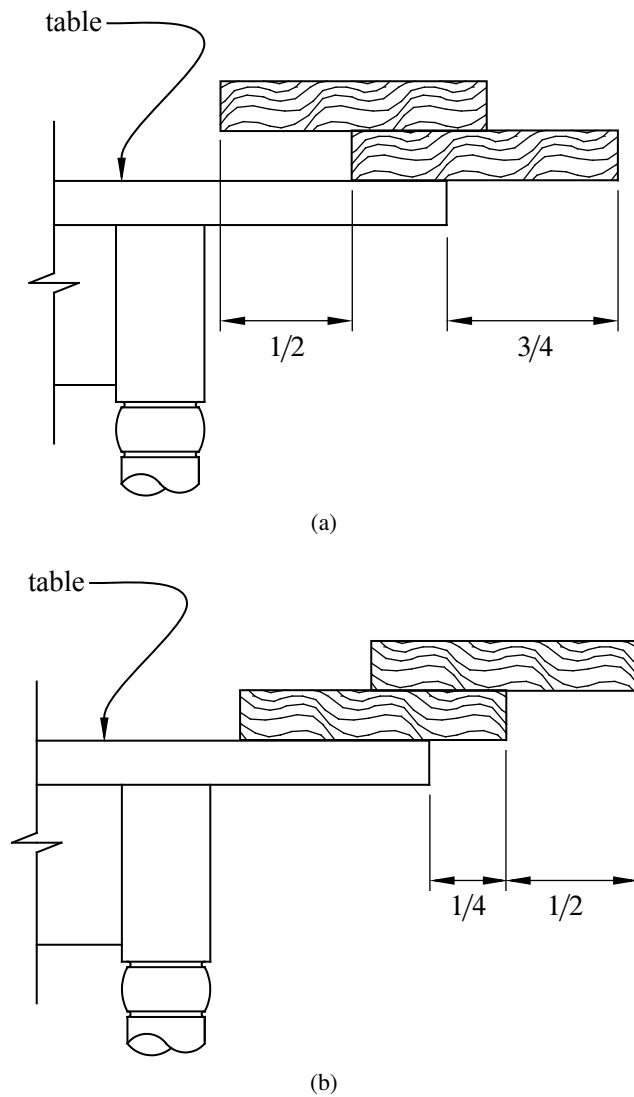


Figure 14.9 Two ways to achieve spread (and hence overhang) $3/4$ with $n = 2$ blocks. The first way (a) is from Case 1 and the second (b) is from Case 2.

Both cases give the same spread, albeit by different approaches. For example, see Figure 14.9.

That was easy enough. What about S_3 ?

$$\begin{aligned} S_3 &= \max \left\{ 1 - \frac{1}{6}, \frac{3}{4} + \frac{1}{6} \right\} \\ &= \max \left\{ \frac{5}{6}, \frac{11}{12} \right\} \\ &= \frac{11}{12}. \end{aligned}$$

As we can see, the method provided by Case 2 is the best. Let's check $n = 4$.

$$\begin{aligned} S_4 &= \max \left\{ 1 - \frac{1}{8}, \frac{11}{12} + \frac{1}{8} \right\} \\ &= \frac{25}{24}. \end{aligned} \tag{14.21}$$

Wow! This is a breakthrough—for two reasons. First, equation 14.21 tells us that by using only 4 blocks, we can make a stack so that one of the blocks is hanging out completely over the edge of the table. The two ways to do this are shown in Figure 14.10.

The second reason that equation 14.21 is important is that we now know that $S_4 > 1$, which means that we no longer have to worry about Case 1 for $n > 4$ since Case 1 never achieves spread greater than 1. Moreover, even for $n \leq 4$, we have now seen that the spread achieved by Case 1 never exceeds the spread achieved by Case 2, and they can be equal only for $n = 1$ and $n = 2$. This means that

$$S_n = S_{n-1} + \frac{1}{2n} \tag{14.22}$$

for all $n > 1$ since we have shown that the best spread can always be achieved using Case 2.

The recurrence in equation 14.22 is much easier to solve than the one we started with in equation 14.20. We can solve it by expanding the equation as follows:

$$\begin{aligned} S_n &= S_{n-1} + \frac{1}{2n} \\ &= S_{n-2} + \frac{1}{2(n-1)} + \frac{1}{2n} \\ &= S_{n-3} + \frac{1}{2(n-2)} + \frac{1}{2(n-1)} + \frac{1}{2n} \end{aligned}$$

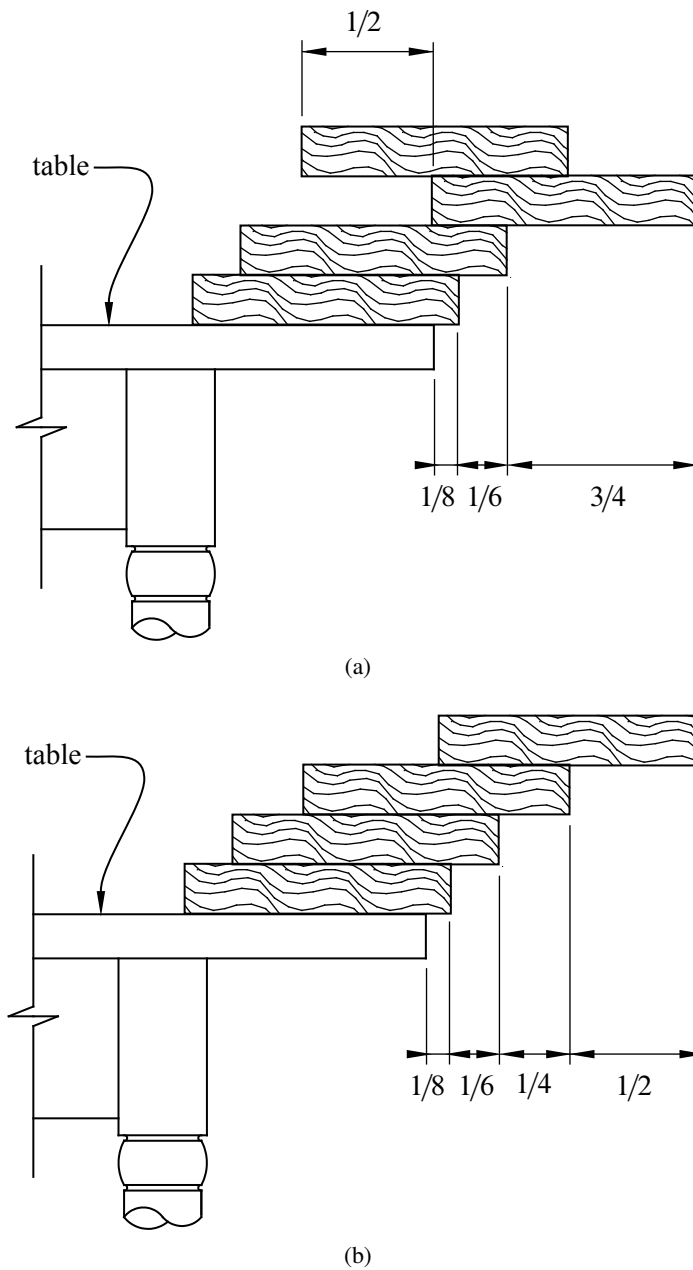


Figure 14.10 The two ways to achieve spread (and overhang) $25/24$. The method in (a) uses Case 1 for the top 2 blocks and Case 2 for the others. The method in (b) uses Case 2 for every block that is added to the stack.

and so on. This suggests that

$$S_n = \sum_{i=1}^n \frac{1}{2^i}, \quad (14.23)$$

which is, indeed, the case.

Equation 14.23 can be verified by induction. The base case when $n = 1$ is true since we know that $S_1 = 1/2$. The inductive step follows from equation 14.22.

So we now know the maximum possible spread and hence the maximum possible overhang for any stable stack of books. Are we done? Not quite. Although we know that $S_4 > 1$, we still don't know how big the sum $\sum_{i=1}^n \frac{1}{2^i}$ can get.

It turns out that S_n is very close to a famous sum known as the n th Harmonic number H_n .

14.4.3 Harmonic Numbers

Definition 14.4.1. The n th Harmonic number is

$$H_n ::= \sum_{i=1}^n \frac{1}{i}.$$

So equation 14.23 means that

$$S_n = \frac{H_n}{2}. \quad (14.24)$$

The first few Harmonic numbers are easy to compute. For example,

$$H_4 = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = \frac{25}{12}.$$

There is good news and bad news about Harmonic numbers. The bad news is that there is no closed-form expression known for the Harmonic numbers. The good news is that we can use Theorem 14.3.2 to get close upper and lower bounds on H_n . In particular, since

$$\int_1^n \frac{1}{x} dx = \ln(x) \Big|_1^n = \ln(n),$$

Theorem 14.3.2 means that

$$\ln(n) + \frac{1}{n} \leq H_n \leq \ln(n) + 1. \quad (14.25)$$

In other words, the n th Harmonic number is very close to $\ln(n)$.

Because the Harmonic numbers frequently arise in practice, mathematicians have worked hard to get even better approximations for them. In fact, it is now known that

$$H_n = \ln(n) + \gamma + \frac{1}{2n} + \frac{1}{12n^2} + \frac{\epsilon(n)}{120n^4} \quad (14.26)$$

Here γ is a value $0.577215664\dots$ called *Euler’s constant*, and $\epsilon(n)$ is between 0 and 1 for all n . We will not prove this formula.

We are now finally done with our analysis of the block stacking problem. Plugging the value of H_n into equation 14.24, we find that the maximum overhang for n blocks is very close to $\frac{1}{2} \ln(n)$. Since $\ln(n)$ grows to infinity as n increases, this means that if we are given enough blocks (in theory anyway), we can get a block to hang out arbitrarily far over the edge of the table. Of course, the number of blocks we need will grow as an exponential function of the overhang, so it will probably take you a long time to achieve an overhang of 2 or 3, never mind an overhang of 100.

14.4.4 Asymptotic Equality

For cases like equation 14.26 where we understand the growth of a function like H_n up to some (unimportant) error terms, we use a special notation, $\sim$, to denote the leading term of the function. For example, we say that $H_n \sim \ln(n)$ to indicate that the leading term of H_n is $\ln(n)$. More precisely:

Definition 14.4.2. For functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, we say f is *asymptotically equal* to g , in symbols,

$$f(x) \sim g(x)$$

iff

$$\lim_{x \rightarrow \infty} f(x)/g(x) = 1.$$

Although it is tempting to write $H_n \sim \ln(n) + \gamma$ to indicate the two leading terms, this is not really right. According to Definition 14.4.2, $H_n \sim \ln(n) + c$ where c is *any constant*. The correct way to indicate that γ is the second-largest term is $H_n - \ln(n) \sim \gamma$.

The reason that the $\sim$ notation is useful is that often we do not care about lower order terms. For example, if $n = 100$, then we can compute $H(n)$ to great precision using only the two leading terms:

$$|H_n - \ln(n) - \gamma| \leq \left| \frac{1}{200} - \frac{1}{120000} + \frac{1}{120 \cdot 100^4} \right| < \frac{1}{200}.$$

We will spend a lot more time talking about asymptotic notation at the end of the chapter. But for now, let’s get back to using sums.

14.5 Products

We’ve covered several techniques for finding closed forms for sums but no methods for dealing with products. Fortunately, we do not need to develop an entirely new set of tools when we encounter a product such as

$$n! ::= \prod_{i=1}^n i. \quad (14.27)$$

That’s because we can convert any product into a sum by taking a logarithm. For example, if

$$P = \prod_{i=1}^n f(i),$$

then

$$\ln(P) = \sum_{i=1}^n \ln(f(i)).$$

We can then apply our summing tools to find a closed form (or approximate closed form) for $\ln(P)$ and then exponentiate at the end to undo the logarithm.

For example, let’s see how this works for the factorial function $n!$. We start by taking the logarithm:

$$\begin{aligned} \ln(n!) &= \ln(1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n) \\ &= \ln(1) + \ln(2) + \ln(3) + \cdots + \ln(n-1) + \ln(n) \\ &= \sum_{i=1}^n \ln(i). \end{aligned}$$

Unfortunately, no closed form for this sum is known. However, we can apply Theorem 14.3.2 to find good closed-form bounds on the sum. To do this, we first compute

$$\begin{aligned} \int_1^n \ln(x) dx &= x \ln(x) - x \Big|_1^n \\ &= n \ln(n) - n + 1. \end{aligned}$$

Plugging into Theorem 14.3.2, this means that

$$n \ln(n) - n + 1 \leq \sum_{i=1}^n \ln(i) \leq n \ln(n) - n + 1 + \ln(n).$$

Exponentiating then gives

$$\frac{n^n}{e^{n-1}} \leq n! \leq \frac{n^{n+1}}{e^{n-1}}. \quad (14.28)$$

This means that $n!$ is within a factor of n of n^n/e^{n-1} .

14.5.1 Stirling’s Formula

$n!$ is probably the most commonly used product in discrete mathematics, and so mathematicians have put in the effort to find much better closed-form bounds on its value. The most useful bounds are given in Theorem 14.5.1.

Theorem 14.5.1 (Stirling’s Formula). *For all $n \geq 1$,*

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\epsilon(n)}$$

where

$$\frac{1}{12n+1} \leq \epsilon(n) \leq \frac{1}{12n}.$$

Theorem 14.5.1 can be proved by induction on n , but the details are a bit painful (even for us) and so we will not go through them here.

There are several important things to notice about Stirling’s Formula. First, $\epsilon(n)$ is always positive. This means that

$$n! > \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (14.29)$$

for all $n \in \mathbb{N}^+$.

Second, $\epsilon(n)$ tends to zero as n gets large. This means that

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (14.30)$$

which is rather surprising. After all, who would expect both π and e to show up in a closed-form expression that is asymptotically equal to $n!$?

Third, $\epsilon(n)$ is small even for small values of n . This means that Stirling’s Formula provides good approximations for $n!$ for most all values of n . For example, if we use

$$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

as the approximation for $n!$, as many people do, we are guaranteed to be within a factor of

$$e^{\epsilon(n)} \leq e^{\frac{1}{12n}}$$

Approximation	$n \geq 1$	$n \geq 10$	$n \geq 100$	$n \geq 1000$
$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n$	$< 10\%$	$< 1\%$	$< 0.1\%$	$< 0.01\%$
$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{1/12n}$	$< 1\%$	$< 0.01\%$	$< 0.0001\%$	$< 0.000001\%$

Table 14.1 Error bounds on common approximations for $n!$ from Theorem 14.5.1. For example, if $n \geq 100$, then $\sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ approximates $n!$ to within 0.1%.

of the correct value. For $n \geq 10$, this means we will be within 1% of the correct value. For $n \geq 100$, the error will be less than 0.1%.

If we need an even closer approximation for $n!$, then we could use either

$$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{1/12n}$$

or

$$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{1/(12n+1)}$$

depending on whether we want an upper bound or a lower bound, respectively. By Theorem 14.5.1, we know that both bounds will be within a factor of

$$e^{\frac{1}{12n} - \frac{1}{12n+1}} = e^{\frac{1}{144n^2 + 12n}}$$

of the correct value. For $n \geq 10$, this means that either bound will be within 0.01% of the correct value. For $n \geq 100$, the error will be less than 0.0001%.

For quick future reference, these facts are summarized in Corollary 14.5.2 and Table 14.1.

Corollary 14.5.2.

$$n! < \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \cdot \begin{cases} 1.09 & \text{for } n \geq 1, \\ 1.009 & \text{for } n \geq 10, \\ 1.0009 & \text{for } n \geq 100. \end{cases}$$

14.6 Double Trouble

Sometimes we have to evaluate sums of sums, otherwise known as *double summations*. This sounds hairy, and sometimes it is. But usually, it is straightforward—you just evaluate the inner sum, replace it with a closed form, and then evaluate the

outer sum (which no longer has a summation inside it). For example,⁸

$$\begin{aligned}
 \sum_{n=0}^{\infty} \left(y^n \sum_{i=0}^n x^i \right) &= \sum_{n=0}^{\infty} \left(y^n \frac{1-x^{n+1}}{1-x} \right) && \text{equation 14.2} \\
 &= \left(\frac{1}{1-x} \right) \sum_{n=0}^{\infty} y^n - \left(\frac{1}{1-x} \right) \sum_{n=0}^{\infty} y^n x^{n+1} \\
 &= \frac{1}{(1-x)(1-y)} - \left(\frac{x}{1-x} \right) \sum_{n=0}^{\infty} (xy)^n && \text{Theorem 14.1.1} \\
 &= \frac{1}{(1-x)(1-y)} - \frac{x}{(1-x)(1-xy)} && \text{Theorem 14.1.1} \\
 &= \frac{(1-xy) - x(1-y)}{(1-x)(1-y)(1-xy)} \\
 &= \frac{1-x}{(1-x)(1-y)(1-xy)} \\
 &= \frac{1}{(1-y)(1-xy)}.
 \end{aligned}$$

When there’s no obvious closed form for the inner sum, a special trick that is often useful is to try *exchanging the order of summation*. For example, suppose we want to compute the sum of the first n Harmonic numbers

$$\sum_{k=1}^n H_k = \sum_{k=1}^n \sum_{j=1}^k \frac{1}{j} \tag{14.31}$$

For intuition about this sum, we can apply Theorem 14.3.2 to equation 14.25 to conclude that the sum is close to

$$\int_1^n \ln(x) dx = x \ln(x) - x \Big|_1^n = n \ln(n) - n + 1.$$

Now let’s look for an exact answer. If we think about the pairs (k, j) over which

⁸Ok, so maybe this one is a little hairy, but it is also fairly straightforward. Wait till you see the next one!

we are summing, they form a triangle:

		j						
		1	2	3	4	5	...	n
k	1	1						
	2	1	1/2					
	3	1	1/2	1/3				
	4	1	1/2	1/3	1/4			
	...	...						
n		1	1/2		...			1/n

The summation in equation 14.31 is summing each row and then adding the row sums. Instead, we can sum the columns and then add the column sums. Inspecting the table we see that this double sum can be written as

$$\begin{aligned}
 \sum_{k=1}^n H_k &= \sum_{k=1}^n \sum_{j=1}^k \frac{1}{j} \\
 &= \sum_{j=1}^n \sum_{k=j}^n \frac{1}{j} \\
 &= \sum_{j=1}^n \frac{1}{j} \sum_{k=j}^n 1 \\
 &= \sum_{j=1}^n \frac{1}{j} (n - j + 1) \\
 &= \sum_{j=1}^n \frac{n+1}{j} - \sum_{j=1}^n \frac{j}{j} \\
 &= (n+1) \sum_{j=1}^n \frac{1}{j} - \sum_{j=1}^n 1 \\
 &= (n+1)H_n - n.
 \end{aligned} \tag{14.32}$$

14.7 Asymptotic Notation

Asymptotic notation is a shorthand used to give a quick measure of the behavior of a function $f(n)$ as n grows large. For example, the asymptotic notation $\sim$ of Definition 14.4.2 is a binary relation indicating that two functions grow at the *same* rate. There is also a binary relation indicating that one function grows at a significantly *slower* rate than another.

14.7.1 Little Oh

Definition 14.7.1. For functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, with g nonnegative, we say f is *asymptotically smaller* than g , in symbols,

$$f(x) = o(g(x)),$$

iff

$$\lim_{x \rightarrow \infty} f(x)/g(x) = 0.$$

For example, $1000x^{1.9} = o(x^2)$, because $1000x^{1.9}/x^2 = 1000/x^{0.1}$ and since $x^{0.1}$ goes to infinity with x and 1000 is constant, we have $\lim_{x \rightarrow \infty} 1000x^{1.9}/x^2 = 0$. This argument generalizes directly to yield

Lemma 14.7.2. $x^a = o(x^b)$ for all nonnegative constants $a < b$.

Using the familiar fact that $\log x < x$ for all $x > 1$, we can prove

Lemma 14.7.3. $\log x = o(x^\epsilon)$ for all $\epsilon > 0$.

Proof. Choose $\epsilon > \delta > 0$ and let $x = z^\delta$ in the inequality $\log x < x$. This implies

$$\log z < z^\delta / \delta = o(z^\epsilon) \quad \text{by Lemma 14.7.2.} \quad (14.33)$$

■

Corollary 14.7.4. $x^b = o(a^x)$ for any $a, b \in \mathbb{R}$ with $a > 1$.

Lemma 14.7.3 and Corollary 14.7.4 can also be proved using l’Hôpital’s Rule or the McLaurin Series for $\log x$ and e^x . Proofs can be found in most calculus texts.

14.7.2 Big Oh

Big Oh is the most frequently used asymptotic notation. It is used to give an upper bound on the growth of a function, such as the running time of an algorithm.

Definition 14.7.5. Given nonnegative functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, we say that

$$f = O(g)$$

iff

$$\limsup_{x \rightarrow \infty} f(x)/g(x) < \infty.$$

This definition⁹ makes it clear that

Lemma 14.7.6. *If $f = o(g)$ or $f \sim g$, then $f = O(g)$.*

Proof. $\lim f/g = 0$ or $\lim f/g = 1$ implies $\lim f/g < \infty$. ■

It is easy to see that the converse of Lemma 14.7.6 is not true. For example, $2x = O(x)$, but $2x \not\sim x$ and $2x \neq o(x)$.

The usual formulation of Big Oh spells out the definition of $\limsup$ without mentioning it. Namely, here is an equivalent definition:

Definition 14.7.7. Given functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, we say that

$$f = O(g)$$

iff there exists a constant $c \geq 0$ and an x_0 such that for all $x \geq x_0$, $|f(x)| \leq cg(x)$.

This definition is rather complicated, but the idea is simple: $f(x) = O(g(x))$ means $f(x)$ is less than or equal to $g(x)$, except that we’re willing to ignore a constant factor, namely, c , and to allow exceptions for small x , namely, $x < x_0$.

We observe,

Lemma 14.7.8. *If $f = o(g)$, then it is not true that $g = O(f)$.*

⁹We can’t simply use the limit as $x \rightarrow \infty$ in the definition of $O()$, because if $f(x)/g(x)$ oscillates between, say, 3 and 5 as x grows, then $f = O(g)$ because $f \leq 5g$, but $\lim_{x \rightarrow \infty} f(x)/g(x)$ does not exist. So instead of limit, we use the technical notion of $\limsup$. In this oscillating case, $\limsup_{x \rightarrow \infty} f(x)/g(x) = 5$.

The precise definition of $\limsup$ is

$$\limsup_{x \rightarrow \infty} h(x) ::= \lim_{x \rightarrow \infty} \text{lub}_{y \geq x} h(y),$$

where “lub” abbreviates “least upper bound.”

Proof.

$$\lim_{x \rightarrow \infty} \frac{g(x)}{f(x)} = \frac{1}{\lim_{x \rightarrow \infty} f(x)/g(x)} = \frac{1}{0} = \infty,$$

so $g \neq O(f)$. ■

Proposition 14.7.9. $100x^2 = O(x^2)$.

Proof. Choose $c = 100$ and $x_0 = 1$. Then the proposition holds, since for all $x \geq 1$, $|100x^2| \leq 100x^2$. ■

Proposition 14.7.10. $x^2 + 100x + 10 = O(x^2)$.

Proof. $(x^2 + 100x + 10)/x^2 = 1 + 100/x + 10/x^2$ and so its limit as x approaches infinity is $1 + 0 + 0 = 1$. So in fact, $x^2 + 100x + 10 \sim x^2$, and therefore $x^2 + 100x + 10 = O(x^2)$. Indeed, it's conversely true that $x^2 = O(x^2 + 100x + 10)$. ■

Proposition 14.7.10 generalizes to an arbitrary polynomial:

Proposition 14.7.11. $a_k x^k + a_{k-1} x^{k-1} + \cdots + a_1 x + a_0 = O(x^k)$.

We'll omit the routine proof.

Big Oh notation is especially useful when describing the running time of an algorithm. For example, the usual algorithm for multiplying $n \times n$ matrices uses a number of operations proportional to n^3 in the worst case. This fact can be expressed concisely by saying that the running time is $O(n^3)$. So this asymptotic notation allows the speed of the algorithm to be discussed without reference to constant factors or lower-order terms that might be machine specific. It turns out that there is another, ingenious matrix multiplication procedure that uses $O(n^{2.55})$ operations. This procedure will therefore be much more efficient on large enough matrices. Unfortunately, the $O(n^{2.55})$ -operation multiplication procedure is almost never used in practice because it happens to be less efficient than the usual $O(n^3)$ procedure on matrices of practical size.¹⁰

14.7.3 Theta

Sometimes we want to specify that a running time $T(n)$ is precisely quadratic up to constant factors (both upper bound *and* lower bound). We could do this by saying that $T(n) = O(n^2)$ and $n^2 = O(T(n))$, but rather than say both, mathematicians have devised yet another symbol, Θ , to do the job.

¹⁰It is even conceivable that there is an $O(n^2)$ matrix multiplication procedure, but none is known.

Definition 14.7.12.

$$f = \Theta(g) \text{ iff } f = O(g) \text{ and } g = O(f).$$

The statement $f = \Theta(g)$ can be paraphrased intuitively as “ f and g are equal to within a constant factor.”

The Theta notation allows us to highlight growth rates and allow suppression of distracting factors and low-order terms. For example, if the running time of an algorithm is

$$T(n) = 10n^3 - 20n^2 + 1,$$

then we can more simply write

$$T(n) = \Theta(n^3).$$

In this case, we would say that T is of order n^3 or that $T(n)$ grows cubically, which is probably what we really want to know. Another such example is

$$\pi^2 3^{x-7} + \frac{(2.7x^{113} + x^9 - 86)^4}{\sqrt{x}} - 1.08^{3x} = \Theta(3^x).$$

Just knowing that the running time of an algorithm is $\Theta(n^3)$, for example, is useful, because if n doubles we can predict that the running time will *by and large*¹¹ increase by a factor of at most 8 for large n . In this way, Theta notation preserves information about the scalability of an algorithm or system. Scalability is, of course, a big issue in the design of algorithms and systems.

14.7.4 Pitfalls with Asymptotic Notation

There is a long list of ways to make mistakes with asymptotic notation. This section presents some of the ways that Big Oh notation can lead to ruin and despair. With minimal effort, you can cause just as much chaos with the other symbols.

The Exponential Fiasco

Sometimes relationships involving Big Oh are not so obvious. For example, one might guess that $4^x = O(2^x)$ since 4 is only a constant factor larger than 2. This reasoning is incorrect, however; 4^x actually grows as the square of 2^x .

¹¹Since $\Theta(n^3)$ only implies that the running time, $T(n)$, is between cn^3 and dn^3 for constants $0 < c < d$, the time $T(2n)$ could regularly exceed $T(n)$ by a factor as large as $8d/c$. The factor is sure to be close to 8 for all large n only if $T(n) \sim n^3$.

Constant Confusion

Every constant is $O(1)$. For example, $17 = O(1)$. This is true because if we let $f(x) = 17$ and $g(x) = 1$, then there exists a $c > 0$ and an x_0 such that $|f(x)| \leq cg(x)$. In particular, we could choose $c = 17$ and $x_0 = 1$, since $|17| \leq 17 \cdot 1$ for all $x \geq 1$. We can construct a false theorem that exploits this fact.

False Theorem 14.7.13.

$$\sum_{i=1}^n i = O(n)$$

Bogus proof. Define $f(n) = \sum_{i=1}^n i = 1 + 2 + 3 + \cdots + n$. Since we have shown that every constant i is $O(1)$, $f(n) = O(1) + O(1) + \cdots + O(1) = O(n)$. ■

Of course in reality $\sum_{i=1}^n i = n(n+1)/2 \neq O(n)$.

The error stems from confusion over what is meant in the statement $i = O(1)$. For any *constant* $i \in \mathbb{N}$ it is true that $i = O(1)$. More precisely, if f is any constant function, then $f = O(1)$. But in this False Theorem, i is not constant—it ranges over a set of values $0, 1, \dots, n$ that depends on n .

And anyway, we should not be adding $O(1)$ ’s as though they were numbers. We never even defined what $O(g)$ means by itself; it should only be used in the context “ $f = O(g)$ ” to describe a relation between functions f and g .

Lower Bound Blunder

Sometimes people incorrectly use Big Oh in the context of a lower bound. For example, they might say, “The running time, $T(n)$, is at least $O(n^2)$,” when they probably mean “ $n^2 = O(T(n))$.”¹²

Equality Blunder

The notation $f = O(g)$ is too firmly entrenched to avoid, but the use of “=” is really regrettable. For example, if $f = O(g)$, it seems quite reasonable to write $O(g) = f$. But doing so might tempt us to the following blunder: because $2n = O(n)$, we can say $O(n) = 2n$. But $n = O(n)$, so we conclude that $n = O(n) = 2n$, and therefore $n = 2n$. To avoid such nonsense, we will never write “ $O(f) = g$.”

Similarly, you will often see statements like

$$H_n = \ln(n) + \gamma + O\left(\frac{1}{n}\right)$$

¹²This would more usually be expressed as “ $T(n) = \Omega(n^2)$.”

or

$$n! = (1 + o(1))\sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

In such cases, the true meaning is

$$H_n = \ln(n) + \gamma + f(n)$$

for some $f(n)$ where $f(n) = O(1/n)$, and

$$n! = (1 + g(n))\sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

where $g(n) = o(1)$. These last transgressions are OK as long as you (and your reader) know what you mean.

14.7.5 Omega

Suppose you want to make a statement of the form “the running time of the algorithm is a least...” Can you say it is “at least $O(n^2)$ ”? No! This statement is meaningless since big-oh can only be used for *upper* bounds. For lower bounds, we use a different symbol, called “big-Omega.”

Definition 14.7.14. Given functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, define

$$f = \Omega(g)$$

to mean

$$g = O(f).$$

For example, $x^2 = \Omega(x)$, $2^x = \Omega(x^2)$, and $x/100 = \Omega(100x + \sqrt{x})$.

So if the running time of your algorithm on inputs of size n is $T(n)$, and you want to say it is at least quadratic, say

$$T(n) = \Omega(n^2).$$

Likewise, there is also a symbol called little-omega, analogous to little-oh, to denote that one function grows strictly faster than another function.

Definition 14.7.15. For functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$ with f nonnegative, define

$$f = \omega(g)$$

to mean

$$g = o(f).$$

For example, $x^{1.5} = \omega(x)$ and $\sqrt{x} = \omega(\ln^2(x))$.

The little-omega symbol is not as widely used as the other asymptotic symbols we defined.

Problems for Section 14.1

Class Problems

Problem 14.1.

We begin with two large glasses. The first glass contains a pint of water, and the second contains a pint of wine. We pour $1/3$ of a pint from the first glass into the second, stir up the wine/water mixture in the second glass, and then pour $1/3$ of a pint of the mix back into the first glass and repeat this pouring back-and-forth process a total of n times.

(a) Describe a closed form formula for the amount of wine in the first glass after n back-and-forth pourings.

(b) What is the limit of the amount of wine in each glass as n approaches infinity?

Problem 14.2.

You’ve seen this neat trick for evaluating a geometric sum:

$$\begin{aligned} S &= 1 + z + z^2 + \dots + z^n \\ zS &= z + z^2 + \dots + z^n + z^{n+1} \\ S - zS &= 1 - z^{n+1} \\ S &= \frac{1 - z^{n+1}}{1 - z} \end{aligned}$$

Use the same approach to find a closed-form expression for this sum:

$$T = 1z + 2z^2 + 3z^3 + \dots + nz^n$$

Homework Problems

Problem 14.3.

Is a Harvard degree really worth more than an MIT degree?! Let us say that a person with a Harvard degree starts with \$40,000 and gets a \$20,000 raise every year after graduation, whereas a person with an MIT degree starts with \$30,000, but gets a 20% raise every year. Assume inflation is a fixed 8% every year. That is, \$1.08 a year from now is worth \$1.00 today.

(a) How much is a Harvard degree worth today if the holder will work for n years following graduation?

(b) How much is an MIT degree worth in this case?

(c) If you plan to retire after twenty years, which degree would be worth more?

Problem 14.4.

Suppose you deposit \$100 into your MIT Credit Union account today, \$99 in one month from now, \$98 in two months from now, and so on. Given that the interest rate is constantly 0.3% per month, how long will it take to save \$5,000?

Problems for Section 14.3

Practice Problems

Problem 14.5.

Let

$$S ::= \sum_{n=1}^5 n^{\frac{1}{3}}$$

Using the **Integral Method**, we can find integers, a , b , c , d , and a real number, e , such that

$$\int_a^b x^e dx \leq S \leq \int_c^d x^e dx$$

What are appropriate values for a – e ?

Exam Problems

Problem 14.6.

Assume n is an integer larger than 1. Circle all the correct inequalities below.

Explanations are not required, but partial credit for wrong answers will not be given without them. *Hint:* You may find the graphs in Figure 14.11 helpful.

$$\bullet \sum_{i=1}^n \ln(i+1) \leq \ln 2 + \int_1^n \ln(x+1) dx$$

$$\bullet \sum_{i=1}^n \ln(i+1) \leq \int_0^n \ln(x+2) dx$$

$$\bullet \sum_{i=1}^n \frac{1}{i} \geq \int_0^n \frac{1}{x+1} dx$$

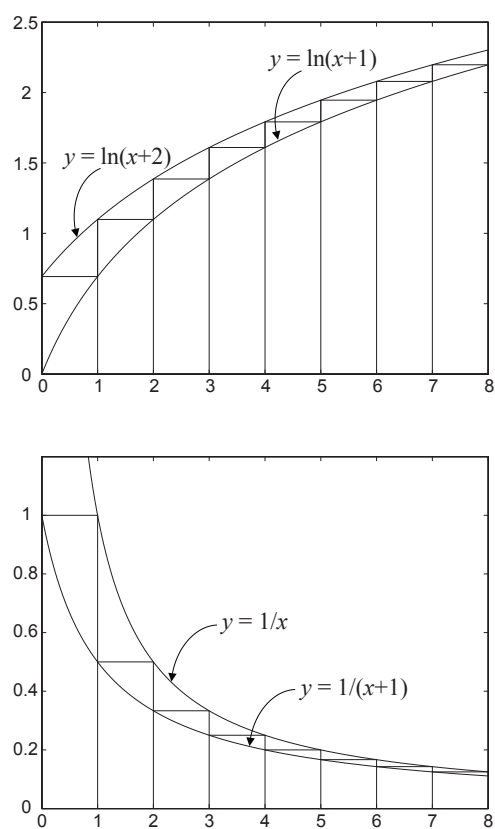


Figure 14.11 Integral bounds for two sums

Homework Problems

Problem 14.7.

Let $f : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ be a weakly decreasing function. Define

$$S ::= \sum_{i=1}^n f(i)$$

and

$$I ::= \int_1^n f(x) dx.$$

Prove that

$$I + f(n) \leq S \leq I + f(1).$$

Problem 14.8.

Use integration to find upper and lower bounds that differ by at most 0.1 for the following sum. (You may need to add the first few terms explicitly and then use integrals to bound the sum of the remaining terms.)

$$\sum_{i=1}^{\infty} \frac{1}{(2i+1)^2}$$

Problems for Section 14.4

Class Problems

Problem 14.9.

An explorer is trying to reach the Holy Grail, which she believes is located in a desert shrine d days walk from the nearest oasis. In the desert heat, the explorer must drink continuously. She can carry at most 1 gallon of water, which is enough for 1 day. However, she is free to make multiple trips carrying up to a gallon each time to create water caches out in the desert.

For example, if the shrine were $2/3$ of a day's walk into the desert, then she could recover the Holy Grail after two days using the following strategy. She leaves the oasis with 1 gallon of water, travels $1/3$ day into the desert, caches $1/3$ gallon, and then walks back to the oasis—arriving just as her water supply runs out. Then she picks up another gallon of water at the oasis, walks $1/3$ day into the desert, tops off her water supply by taking the $1/3$ gallon in her cache, walks the remaining $1/3$ day to the shrine, grabs the Holy Grail, and then walks for $2/3$ of a day back to the oasis—again arriving with no water to spare.

But what if the shrine were located farther away?

(a) What is the most distant point that the explorer can reach and then return to the oasis if she takes a total of only 1 gallon from the oasis?

(b) What is the most distant point the explorer can reach and still return to the oasis if she takes a total of only 2 gallons from the oasis? No proof is required; just do the best you can.

(c) The explorer will travel using a recursive strategy to go far into the desert and back drawing a total of n gallons of water from the oasis. Her strategy is to build up a cache of $n - 1$ gallons, plus enough to get home, a certain fraction of a day's distance into the desert. On the last delivery to the cache, instead of returning home, she proceeds recursively with her $n - 1$ gallon strategy to go farther into the desert and return to the cache. At this point, the cache has just enough water left to get her home.

Prove that with n gallons of water, this strategy will get her $H_n/2$ days into the desert and back, where H_n is the n th Harmonic number:

$$H_n ::= \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

Conclude that she can reach the shrine, however far it is from the oasis.

(d) Suppose that the shrine is $d = 10$ days walk into the desert. Use the asymptotic approximation $H_n \sim \ln n$ to show that it will take more than a million years for the explorer to recover the Holy Grail.

Problem 14.10.

There is a number a such that $\sum_{i=1}^{\infty} i^p$ converges iff $p < a$. What is the value of a ?

Hint: Find a value for a you think that works, then apply the integral bound.

Homework Problems

Problem 14.11.

There is a bug on the edge of a 1-meter rug. The bug wants to cross to the other side of the rug. It crawls at 1 cm per second. However, at the end of each second, a malicious first-grader named Mildred Anderson *stretches* the rug by 1 meter. Assume that her action is instantaneous and the rug stretches uniformly. Thus, here's what happens in the first few seconds:

- The bug walks 1 cm in the first second, so 99 cm remain ahead.

- Mildred stretches the rug by 1 meter, which doubles its length. So now there are 2 cm behind the bug and 198 cm ahead.
- The bug walks another 1 cm in the next second, leaving 3 cm behind and 197 cm ahead.
- Then Mildred strikes, stretching the rug from 2 meters to 3 meters. So there are now $3 \cdot (3/2) = 4.5$ cm behind the bug and $197 \cdot (3/2) = 295.5$ cm ahead.
- The bug walks another 1 cm in the third second, and so on.

Your job is to determine this poor bug's fate.

(a) During second i , what *fraction* of the rug does the bug cross?

(b) Over the first n seconds, what fraction of the rug does the bug cross altogether? Express your answer in terms of the Harmonic number H_n .

(c) The known universe is thought to be about $3 \cdot 10^{10}$ light years in diameter. How many universe diameters must the bug travel to get to the end of the rug? (This distance is NOT the inflated distance caused by the stretching but only the actual walking done by the bug).

Problems for Section 14.7

Practice Problems

Problem 14.12.

Find the least nonnegative integer, n , such that $f(x)$ is $O(x^n)$ when f is defined by each of the expressions below.

- (a) $2x^3 + (\log x)x^2$
- (b) $2x^2 + (\log x)x^3$
- (c) $(1.1)^x$
- (d) $(0.1)^x$
- (e) $(x^4 + x^2 + 1)/(x^3 + 1)$
- (f) $(x^4 + 5 \log x)/(x^4 + 1)$
- (g) $2^{(3 \log_2 x^2)}$

Problem 14.13.

Let $f(n) = n^3$. For each function $g(n)$ in the table below, indicate which of the indicated asymptotic relations hold.

$g(n)$	$f = O(g)$	$f = o(g)$	$g = O(f)$	$g = o(f)$
$6 - 5n - 4n^2 + 3n^3$				
$n^3 \log n$				
$(\sin(\pi n/2) + 2)n^3$				
$n^{\sin(\pi n/2)+2}$				
$\log n!$				
$e^{0.2n} - 100n^3$				

Problem 14.14.

Circle each of the true statements below.

Explanations are not required, but partial credit for wrong answers will not be given without them.

- $n^2 \sim n^2 + n$
- $3^n = O(2^n)$
- $n^{\sin(n\pi/2)+1} = o(n^2)$
- $n = \Theta\left(\frac{3n^3}{(n+1)(n-1)}\right)$

Problem 14.15.

Show that

$$\ln(n^2!) = \Theta(n^2 \ln n)$$

Problem 14.16.

The quantity

$$\frac{(2n)!}{2^{2n}(n!)^2} \tag{14.34}$$

will come up later in the course (it is the probability that in 2^{2n} flips of a fair coin, exactly n will be Heads). Show that it is asymptotically equal to $\frac{1}{\sqrt{\pi n}}$.

Homework Problems

Problem 14.17. (a) Prove that $\log x < x$ for all $x > 1$ (requires elementary calculus).

(b) Prove that the relation, R , on functions such that $f R g$ iff $f = o(g)$ is a strict partial order.

(c) Prove that $f \sim g$ iff $f = g + h$ for some function $h = o(g)$.

Problem 14.18.

Indicate which of the following holds for each pair of functions $(f(n), g(n))$ in the table below. Assume $k \geq 1$, $\epsilon > 0$, and $c > 1$ are constants. Pick the four table entries you consider to be the most challenging or interesting and justify your answers to these.

$f(n)$	$g(n)$	$f = O(g)$	$f = o(g)$	$g = O(f)$	$g = o(f)$	$f = \Theta(g)$	$f \sim g$
2^n	$2^{n/2}$						
$\sqrt{n}$	$n^{\sin(n\pi/2)}$						
$\log(n!)$	$\log(n^n)$						
n^k	c^n						
$\log^k n$	n^ϵ						

Problem 14.19.

Let f, g be nonnegative real-valued functions such that $\lim_{x \rightarrow \infty} f(x) = \infty$ and $f \sim g$.

(a) Give an example of f, g such that $\text{NOT}(2^f \sim 2^g)$.

(b) Prove that $\log f \sim \log g$.

(c) Use Stirling’s formula to prove that in fact

$$\log(n!) \sim n \log n$$

Problem 14.20.

Determine which of these choices

$\Theta(n)$, $\Theta(n^2 \log n)$, $\Theta(n^2)$, $\Theta(1)$, $\Theta(2^n)$, $\Theta(2^{n \ln n})$, none of these

describes each function’s asymptotic behavior. Full proofs are not required, but briefly explain your answers.

(a)

$$n + \ln n + (\ln n)^2$$

(b)

$$\frac{n^2 + 2n - 3}{n^2 - 7}$$

(c)

$$\sum_{i=0}^n 2^{2i+1}$$

(d)

$$\ln(n^2!)$$

(e)

$$\sum_{k=1}^n k \left(1 - \frac{1}{2^k}\right)$$

Problem 14.21. (a) Either prove or disprove each of the following statements.

- $n! = O((n+1)!)$
- $(n+1)! = O(n!)$
- $n! = \Theta((n+1)!)$
- $n! = o((n+1)!)$
- $(n+1)! = o(n!)$

(b) Show that $\left(\frac{n}{3}\right)^{n+e} = o(n!)$.

Problem 14.22.

Prove that $\sum_{k=1}^n k^6 = \Theta(n^7)$.

Class Problems

Problem 14.23.

Give an elementary proof (without appealing to Stirling’s formula) that $\log(n!) = \Theta(n \log n)$.

Problem 14.24.

Suppose $f, g : \mathbb{N}^+ \rightarrow \mathbb{N}^+$ and $f \sim g$.

- (a) Prove that $2f \sim 2g$.
- (b) Prove that $f^2 \sim g^2$.
- (c) Give examples of f and g such that $2^f \not\sim 2^g$.

Problem 14.25.

Recall that for functions f, g on $\mathbb{N}$, $f = O(g)$ iff

$$\exists c \in \mathbb{N} \exists n_0 \in \mathbb{N} \forall n \geq n_0 \quad c \cdot g(n) \geq |f(n)|. \quad (14.35)$$

For each pair of functions below, determine whether $f = O(g)$ and whether $g = O(f)$. In cases where one function is $O()$ of the other, indicate the *smallest nonnegative integer*, c , and for that smallest c , the *smallest corresponding nonnegative integer* n_0 ensuring that condition (14.35) applies.

(a) $f(n) = n^2, g(n) = 3n$.

$f = O(g)$	YES	NO	If YES, $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$
$g = O(f)$	YES	NO	If YES, $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$

(b) $f(n) = (3n - 7)/(n + 4), g(n) = 4$

$f = O(g)$	YES	NO	If YES, $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$
$g = O(f)$	YES	NO	If YES, $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$

(c) $f(n) = 1 + (n \sin(n\pi/2))^2, g(n) = 3n$

$f = O(g)$	YES	NO	If yes, $c = \underline{\hspace{2cm}} n_0 = \underline{\hspace{2cm}}$
$g = O(f)$	YES	NO	If yes, $c = \underline{\hspace{2cm}} n_0 = \underline{\hspace{2cm}}$

Problem 14.26.

False Claim.

$$2^n = O(1). \quad (14.36)$$

Explain why the claim is false. Then identify and explain the mistake in the following bogus proof.

Bogus proof. The proof by induction on n where the induction hypothesis, $P(n)$, is the assertion (14.36).

base case: $P(0)$ holds trivially.

inductive step: We may assume $P(n)$, so there is a constant $c > 0$ such that $2^n \leq c \cdot 1$. Therefore,

$$2^{n+1} = 2 \cdot 2^n \leq (2c) \cdot 1,$$

which implies that $2^{n+1} = O(1)$. That is, $P(n+1)$ holds, which completes the proof of the inductive step.

We conclude by induction that $2^n = O(1)$ for all n . That is, the exponential function is bounded by a constant.

■

Problem 14.27. (a) Prove that the relation, R , on functions such that $f R g$ iff $f = o(g)$ is a strict partial order.

(b) Describe two functions f, g that are incomparable under big Oh:

$$f \neq O(g) \text{ AND } g \neq O(f).$$

Conclude that R is not a path-total order.

Exam Problems

Problem 14.28. (a) Show that

$$(an)^{b/n} \sim 1.$$

where a, b are positive constants and $\sim$ denotes asymptotic equality. *Hint:* $an = a2^{\log_2 n}$.

(b) You may assume that if $f(n) \geq 1$ and $g(n) \geq 1$ for all n , then $f \sim g \longrightarrow f^{\frac{1}{n}} \sim g^{\frac{1}{n}}$. Show that

$$\sqrt[n]{n!} = \Theta(n).$$

Problem 14.29.

- (a) Define a function $f(n)$ such that $f = \Theta(n^2)$ and $\text{NOT}(f \sim n^2)$.
 (b) Define a function $g(n)$ such that $g = O(n^2)$, $g \neq \Theta(n^2)$ and $g \neq o(n^2)$.

Problem 14.30. (a) Show that

$$(an)^{b/n} \sim 1.$$

where a, b are positive constants and $\sim$ denotes asymptotic equality. *Hint:* $an = a2^{\log_2 n}$.

- (b) Show that

$$\sqrt[n]{n!} = \Theta(n).$$

Problem 14.31.

(a) Indicate which of the following asymptotic relations below on the set of non-negative real-valued functions are *equivalence relations*, (**E**), strict partial orders (**S**), weak partial orders (**W**), or *none* of the above (**N**).

- $f \sim g$, the “asymptotically Equal” relation.
- $f = o(g)$, the “little Oh” relation.
- $f = O(g)$, the “big Oh” relation.
- $f = \Theta(g)$, the “Theta” relation.
- $f = O(g)$ AND $\text{NOT}(g = O(f))$.

- (b) Define two functions f, g that are incomparable under big Oh:

$$f \neq O(g) \text{ AND } g \neq O(f).$$

Problem 14.32.

Recall that if f and g are nonnegative real-valued functions on $\mathbb{Z}^+$, then $f = O(g)$ iff there exist $c, n_0 \in \mathbb{Z}^+$ such that

$$\forall n \geq n_0. f(n) \leq cg(n).$$

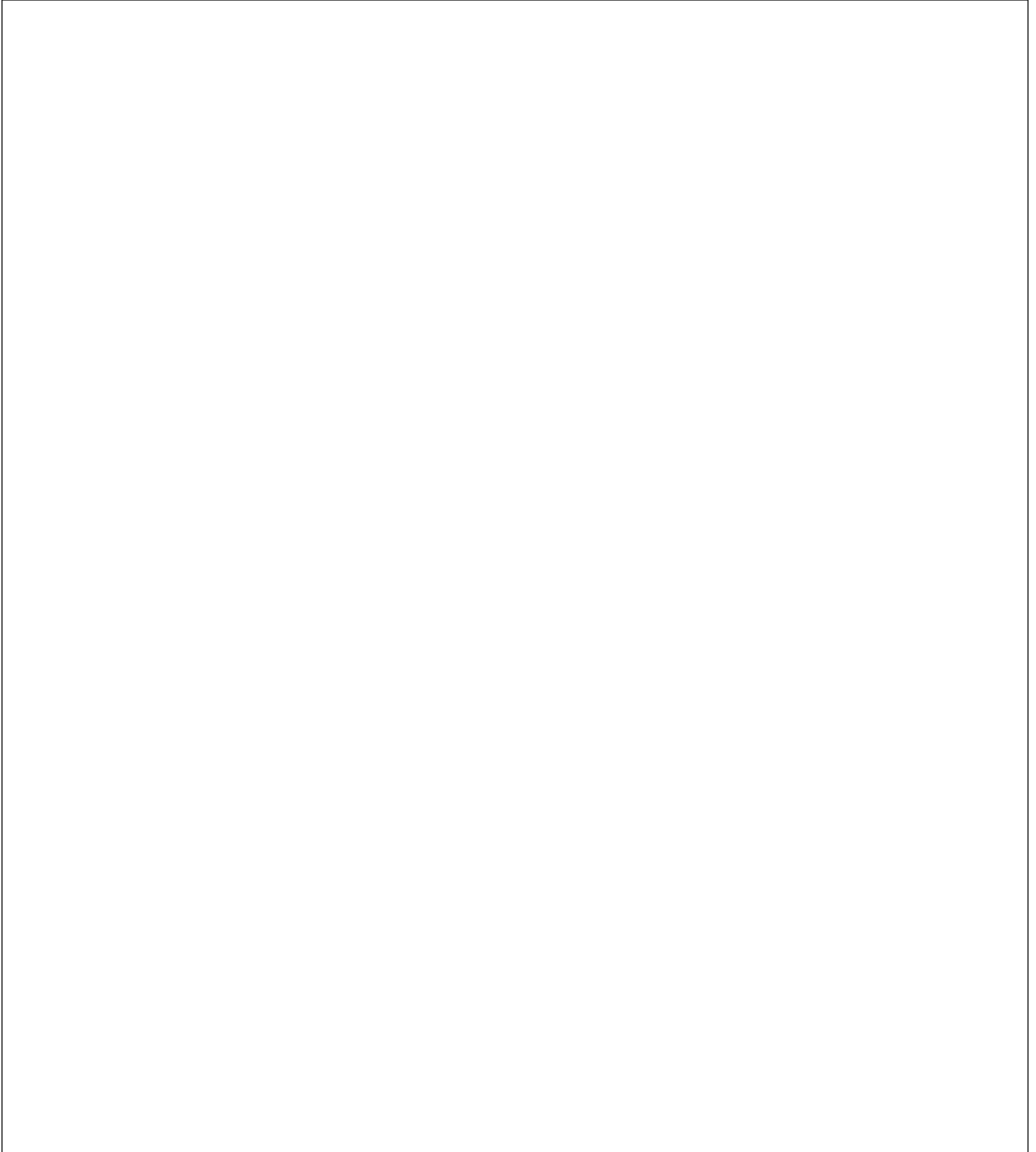
For each pair of functions f and g below, indicate the **smallest** $c \in \mathbb{Z}^+$, and for that smallest c , the **smallest corresponding** $n_0 \in \mathbb{Z}^+$, that would establish $f = O(g)$ by the definition given above. If there is no such c , write ∞ .

(a) $f(n) = \frac{1}{2} \ln n^2, g(n) = n.$ $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$

(b) $f(n) = n, g(n) = n \ln n.$ $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$

(c) $f(n) = 2^n, g(n) = n^4 \ln n$ $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$

(d) $f(n) = 3 \sin\left(\frac{\pi(n-1)}{100}\right) + 2, g(n) = 0.2.$ $c = \underline{\hspace{2cm}}, n_0 = \underline{\hspace{2cm}}$



15 Cardinality Rules

15.1 Counting One Thing by Counting Another

How do you count the number of people in a crowded room? You could count heads, since for each person there is exactly one head. Alternatively, you could count ears and divide by two. Of course, you might have to adjust the calculation if someone lost an ear in a pirate raid or someone was born with three ears. The point here is that you can often *count one thing by counting another*, though some fudge factors may be required. This is a central theme of counting, from the easiest problems to the hardest. In fact, we’ve already seen this technique used in Theorem 5.1.5 where the number of subsets of an n -element set was proved to be the same as the number of length- n bit-strings by describing a bijection between the subsets and the bit-strings.

The most direct way to count one thing by counting another is to find a bijection between them, since if there is a bijection between two sets, then the sets have the same size. This important fact is commonly known as the *Bijection Rule*. We’ve already seen it as the Mapping Rules bijective case (5.3).

15.1.1 The Bijection Rule

The Bijection Rule acts as a magnifier of counting ability; if you figure out the size of one set, then you can immediately determine the sizes of many other sets via bijections. For example, let’s look at the two sets mentioned at the beginning of Part III:

A = all ways to select a dozen doughnuts when five varieties are available

B = all 16-bit sequences with exactly 4 ones

An example of an element of set A is:

0 0		0 0 0 0 0 0	0 0	0 0
⏟	⏟	⏟	⏟	⏟
chocolate	lemon-filled	sugar	glazed	plain

Here, we’ve depicted each doughnut with a 0 and left a gap between the different varieties. Thus, the selection above contains two chocolate doughnuts, no lemon-filled, six sugar, two glazed, and two plain. Now let’s put a 1 into each of the four

gaps:

$$\underbrace{00}_{\text{chocolate}} \quad 1 \quad \underbrace{}_{\text{lemon-filled}} \quad 1 \quad \underbrace{000000}_{\text{sugar}} \quad 1 \quad \underbrace{00}_{\text{glazed}} \quad 1 \quad \underbrace{00}_{\text{plain}}$$

and close up the gaps:

0011000000100100.

We’ve just formed a 16-bit number with exactly 4 ones—an element of B !

This example suggests a bijection from set A to set B : map a dozen doughnuts consisting of:

c chocolate, l lemon-filled, s sugar, g glazed, and p plain

to the sequence:

$$\underbrace{0\dots0}_c \quad 1 \quad \underbrace{0\dots0}_l \quad 1 \quad \underbrace{0\dots0}_s \quad 1 \quad \underbrace{0\dots0}_g \quad 1 \quad \underbrace{0\dots0}_p$$

The resulting sequence always has 16 bits and exactly 4 ones, and thus is an element of B . Moreover, the mapping is a bijection; every such bit sequence comes from exactly one order of a dozen doughnuts. Therefore, $|A| = |B|$ by the Bijection Rule!

This example demonstrates the magnifying power of the bijection rule. We managed to prove that two very different sets are actually the same size—even though we don’t know exactly how big either one is. But as soon as we figure out the size of one set, we’ll immediately know the size of the other.

This particular bijection might seem frighteningly ingenious if you’ve not seen it before. But you’ll use essentially this same argument over and over, and soon you’ll consider it routine.

15.2 Counting Sequences

The Bijection Rule lets us count one thing by counting another. This suggests a general strategy: get really good at counting just a *few* things and then use bijections to count *everything else*. This is the strategy we’ll follow. In particular, we’ll get really good at counting *sequences*. When we want to determine the size of some other set T , we’ll find a bijection from T to a set of sequences S . Then we’ll use our super-ninja sequence-counting skills to determine $|S|$, which immediately gives us $|T|$. We’ll need to hone this idea somewhat as we go along, but that’s pretty much the plan!

15.2.1 The Product Rule

The *Product Rule* gives the size of a product of sets. Recall that if $P_1, P_2, \dots, P_n$ are sets, then

$$P_1 \times P_2 \times \dots \times P_n$$

is the set of all sequences whose first term is drawn from P_1 , second term is drawn from P_2 and so forth.

Rule 15.2.1 (Product Rule). *If $P_1, P_2, \dots, P_n$ are finite sets, then:*

$$|P_1 \times P_2 \times \dots \times P_n| = |P_1| \cdot |P_2| \cdots |P_n|$$

For example, suppose a *daily diet* consists of a breakfast selected from set B , a lunch from set L , and a dinner from set D where:

$$B = \{\text{pancakes, bacon and eggs, bagel, Doritos}\}$$

$$L = \{\text{burger and fries, garden salad, Doritos}\}$$

$$D = \{\text{macaroni, pizza, frozen burrito, pasta, Doritos}\}$$

Then $B \times L \times D$ is the set of all possible daily diets. Here are some sample elements:

(pancakes, burger and fries, pizza)

(bacon and eggs, garden salad, pasta)

(Doritos, Doritos, frozen burrito)

The Product Rule tells us how many different daily diets are possible:

$$\begin{aligned} |B \times L \times D| &= |B| \cdot |L| \cdot |D| \\ &= 4 \cdot 3 \cdot 5 \\ &= 60. \end{aligned}$$

15.2.2 Subsets of an n -element Set

The fact that there are 2^n subsets of an n -element set was proved in Theorem 5.1.5 by setting up a bijection between the subsets and the length- n bit-strings. So the original problem about subsets was transformed into a question about sequences — *exactly according to plan!*. Now we can fill in the missing explanation of why there are 2^n length- n bit-strings: we can write the set of all n -bit sequences as a product of sets:

$$\{0, 1\}^n ::= \underbrace{\{0, 1\} \times \{0, 1\} \times \dots \times \{0, 1\}}_{n \text{ terms}}.$$

Then Product Rule gives the answer:

$$|\{0, 1\}^n| = |\{0, 1\}|^n = 2^n.$$

15.2.3 The Sum Rule

Bart allocates his little sister Lisa a quota of 20 crabby days, 40 irritable days, and 60 generally surly days. On how many days can Lisa be out-of-sorts one way or another? Let set C be her crabby days, I be her irritable days, and S be the generally surly. In these terms, the answer to the question is $|C \cup I \cup S|$. Now assuming that she is permitted at most one bad quality each day, the size of this union of sets is given by the *Sum Rule*:

Rule 15.2.2 (Sum Rule). *If $A_1, A_2, \dots, A_n$ are disjoint sets, then:*

$$|A_1 \cup A_2 \cup \dots \cup A_n| = |A_1| + |A_2| + \dots + |A_n|$$

Thus, according to Bart’s budget, Lisa can be out-of-sorts for:

$$\begin{aligned} |C \cup I \cup S| &= |C| + |I| + |S| \\ &= 20 + 40 + 60 \\ &= 120 \text{ days} \end{aligned}$$

Notice that the Sum Rule holds only for a union of *disjoint* sets. Finding the size of a union of overlapping sets is a more complicated problem that we’ll take up in Section 15.12.

15.2.4 Counting Passwords

Few counting problems can be solved with a single rule. More often, a solution is a flurry of sums, products, bijections, and other methods.

For solving problems involving passwords, telephone numbers, and license plates, the sum and product rules are useful together. For example, on a certain computer system, a valid password is a sequence of between six and eight symbols. The first symbol must be a letter (which can be lowercase or uppercase), and the remaining symbols must be either letters or digits. How many different passwords are possible?

Let’s define two sets, corresponding to valid symbols in the first and subsequent positions in the password.

$$\begin{aligned} F &= \{a, b, \dots, z, A, B, \dots, Z\} \\ S &= \{a, b, \dots, z, A, B, \dots, Z, 0, 1, \dots, 9\} \end{aligned}$$

In these terms, the set of all possible passwords is:¹

$$(F \times S^5) \cup (F \times S^6) \cup (F \times S^7)$$

¹The notation S^5 means $S \times S \times S \times S \times S$.

Thus, the length-six passwords are in the set $F \times S^5$, the length-seven passwords are in $F \times S^6$, and the length-eight passwords are in $F \times S^7$. Since these sets are disjoint, we can apply the Sum Rule and count the total number of possible passwords as follows:

$$\begin{aligned}
 & |(F \times S^5) \cup (F \times S^6) \cup (F \times S^7)| \\
 &= |F \times S^5| + |F \times S^6| + |F \times S^7| && \text{Sum Rule} \\
 &= |F| \cdot |S|^5 + |F| \cdot |S|^6 + |F| \cdot |S|^7 && \text{Product Rule} \\
 &= 52 \cdot 62^5 + 52 \cdot 62^6 + 52 \cdot 62^7 \\
 &\approx 1.8 \cdot 10^{14} \text{ different passwords.}
 \end{aligned}$$

15.3 The Generalized Product Rule

In how many ways can, say, a Nobel prize, a Japan prize, and a Pulitzer prize be awarded to n people? This is easy to answer using our strategy of translating the problem about awards into a problem about sequences. Let P be the set of n people taking the course. Then there is a bijection from ways of awarding the three prizes to the set $P^3 ::= P \times P \times P$. In particular, the assignment:

“Barak wins a Nobel, George wins a Japan, and Bill wins a Pulitzer prize”

maps to the sequence (Barak, George, Bill). By the Product Rule, we have $|P^3| = |P|^3 = n^3$, so there are n^3 ways to award the prizes to a class of n people. Notice that P^3 includes triples like (Barak, Bill, Barak) where one person wins more than one prize.

But what if the three prizes must be awarded to *different* students? As before, we could map first assignment to the triple (Bill, George, Barak) $\in P^3$. But this function is *no longer a bijection*. For example, no valid assignment maps to the triple (Barak, Bill, Barak) because now we’re not allowing Barak to receive two prize. However, there *is* a bijection from prize assignments to the set:

$$S = \{(x, y, z) \in P^3 \mid x, y, \text{ and } z \text{ are different people}\}$$

This reduces the original problem to a problem of counting sequences. Unfortunately, the Product Rule does not apply directly to counting sequences of this type because the entries depend on one another; in particular, they must all be different. However, a slightly sharper tool does the trick.

Prizes for *truly exceptional* Coursework

Given everyone’s hard work on this material, the instructors considered awarding some prizes for truly exceptional coursework. Here are three possible prize categories:

Best Administrative Critique We asserted that the quiz was closed-book. On the cover page, one strong candidate for this award wrote, “There is no book.”

Awkward Question Award “Okay, the left sock, right sock, and pants are in an antichain, but how —even with assistance —could I put on all three at once?”

Best Collaboration Statement Inspired by a student who wrote “I worked alone” on Quiz 1.

Rule 15.3.1 (Generalized Product Rule). *Let S be a set of length- k sequences. If there are:*

- n_1 possible first entries,
- n_2 possible second entries for each first entry,
- $\vdots$
- n_k possible k th entries for each sequence of first $k - 1$ entries,

then:

$$|S| = n_1 \cdot n_2 \cdot n_3 \cdots n_k$$

In the awards example, S consists of sequences (x, y, z) . There are n ways to choose x , the recipient of prize #1. For each of these, there are $n - 1$ ways to choose y , the recipient of prize #2, since everyone except for person x is eligible. For each combination of x and y , there are $n - 2$ ways to choose z , the recipient of prize #3, because everyone except for x and y is eligible. Thus, according to the Generalized Product Rule, there are

$$|S| = n \cdot (n - 1) \cdot (n - 2)$$

ways to award the 3 prizes to different people.

15.3.1 Defective Dollar Bills

A dollar bill is *defective* if some digit appears more than once in the 8-digit serial number. If you check your wallet, you’ll be sad to discover that defective bills are all-too-common. In fact, how common are *nondefective* bills? Assuming that the digit portions of serial numbers all occur equally often, we could answer this question by computing

$$\text{fraction of nondefective bills} = \frac{|\{\text{serial \#’s with all digits different}\}|}{|\{\text{serial numbers}\}|}. \quad (15.1)$$

Let’s first consider the denominator. Here there are no restrictions; there are 10 possible first digits, 10 possible second digits, 10 third digits, and so on. Thus, the total number of 8-digit serial numbers is 10^8 by the Product Rule.

Next, let’s turn to the numerator. Now we’re not permitted to use any digit twice. So there are still 10 possible first digits, but only 9 possible second digits, 8 possible third digits, and so forth. Thus, by the Generalized Product Rule, there are

$$10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 = \frac{10!}{2} = 1,814,400$$

serial numbers with all digits different. Plugging these results into Equation 15.1, we find:

$$\text{fraction of nondefective bills} = \frac{1,814,400}{100,000,000} = 1.8144\%$$

15.3.2 A Chess Problem

In how many different ways can we place a pawn (P), a knight (N), and a bishop (B) on a chessboard so that no two pieces share a row or a column? A valid configuration is shown in Figure 15.1(a), and an invalid configuration is shown in Figure 15.1(b).

First, we map this problem about chess pieces to a question about sequences. There is a bijection from configurations to sequences

$$(r_P, c_P, r_N, c_N, r_B, c_B)$$

where r_P , r_N , and r_B are distinct rows and c_P , c_N , and c_B are distinct columns. In particular, r_P is the pawn’s row, c_P is the pawn’s column, r_N is the knight’s row, etc. Now we can count the number of such sequences using the Generalized Product Rule:

- r_P is one of 8 rows

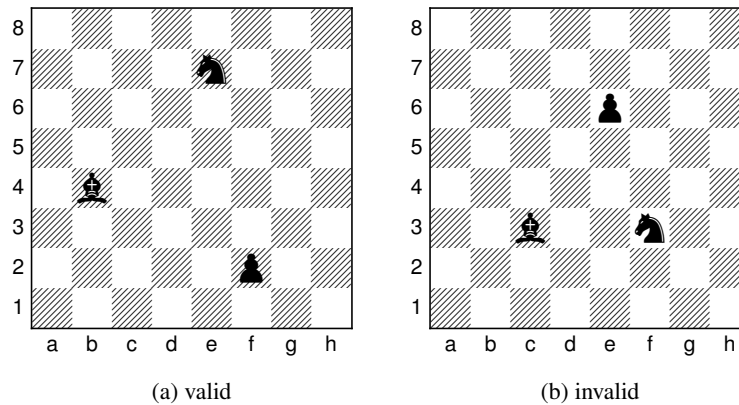


Figure 15.1 Two ways of placing a pawn (♟), a knight (♞), and a bishop (♝) on a chessboard. The configuration shown in (b) is invalid because the bishop and the knight are in the same row.

- c_P is one of 8 columns
- r_N is one of 7 rows (any one but r_P)
- c_N is one of 7 columns (any one but c_P)
- r_B is one of 6 rows (any one but r_P or r_N)
- c_B is one of 6 columns (any one but c_P or c_N)

Thus, the total number of configurations is $(8 \cdot 7 \cdot 6)^2$.

15.3.3 Permutations

A *permutation* of a set S is a sequence that contains every element of S exactly once. For example, here are all the permutations of the set $\{a, b, c\}$:

$$\begin{array}{lll} (a, b, c) & (a, c, b) & (b, a, c) \\ (b, c, a) & (c, a, b) & (c, b, a) \end{array}$$

How many permutations of an n -element set are there? Well, there are n choices for the first element. For each of these, there are $n - 1$ remaining choices for the second element. For every combination of the first two elements, there are $n - 2$ ways to choose the third element, and so forth. Thus, there are a total of

$$n \cdot (n - 1) \cdot (n - 2) \cdots 3 \cdot 2 \cdot 1 = n!$$

permutations of an n -element set. In particular, this formula says that there are

$3! = 6$ permutations of the 3-element set $\{a, b, c\}$, which is the number we found above.

Permutations will come up again in this course approximately 1.6 bazillion times. In fact, permutations are the reason why factorial comes up so often and why we taught you Stirling’s approximation:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

15.4 The Division Rule

Counting ears and dividing by two is a silly way to count the number of people in a room, but this approach is representative of a powerful counting principle.

A *k-to-1 function* maps exactly k elements of the domain to every element of the codomain. For example, the function mapping each ear to its owner is 2-to-1. Similarly, the function mapping each finger to its owner is 10-to-1, and the function mapping each finger and toe to its owner is 20-to-1. The general rule is:

Rule 15.4.1 (Division Rule). *If $f : A \rightarrow B$ is k -to-1, then $|A| = k \cdot |B|$.*

For example, suppose A is the set of ears in the room and B is the set of people. There is a 2-to-1 mapping from ears to people, so by the Division Rule, $|A| = 2 \cdot |B|$. Equivalently, $|B| = |A|/2$, expressing what we knew all along: the number of people is half the number of ears. Unlikely as it may seem, many counting problems are made much easier by initially counting every item multiple times and then correcting the answer using the Division Rule. Let’s look at some examples.

15.4.1 Another Chess Problem

In how many different ways can you place two identical rooks on a chessboard so that they do not share a row or column? A valid configuration is shown in Figure 15.2(a), and an invalid configuration is shown in Figure 15.2(b).

Let A be the set of all sequences

$$(r_1, c_1, r_2, c_2)$$

where r_1 and r_2 are distinct rows and c_1 and c_2 are distinct columns. Let B be the set of all valid rook configurations. There is a natural function f from set A to set B ; in particular, f maps the sequence (r_1, c_1, r_2, c_2) to a configuration with one rook in row r_1 , column c_1 and the other rook in row r_2 , column c_2 .

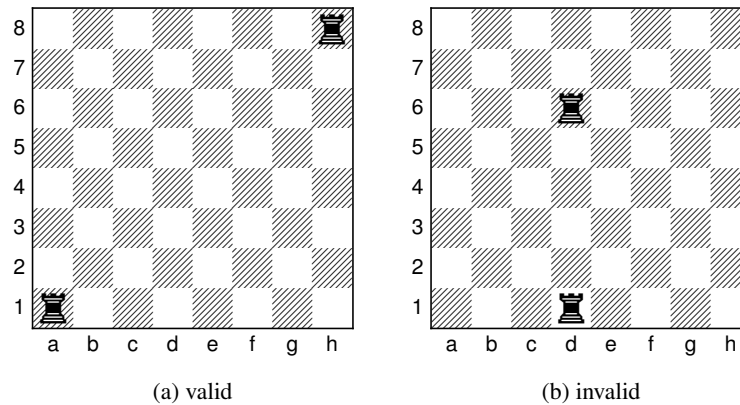


Figure 15.2 Two ways to place 2 rooks (♖) on a chessboard. The configuration in (b) is invalid because the rooks are in the same column.

But now there’s a snag. Consider the sequences:

$$(1, 1, 8, 8) \quad \text{and} \quad (8, 8, 1, 1)$$

The first sequence maps to a configuration with a rook in the lower-left corner and a rook in the upper-right corner. The second sequence maps to a configuration with a rook in the upper-right corner and a rook in the lower-left corner. The problem is that those are two different ways of describing the *same* configuration! In fact, this arrangement is shown in Figure 15.2(a).

More generally, the function f maps exactly two sequences to *every* board configuration; that is f is a 2-to-1 function. Thus, by the quotient rule, $|A| = 2 \cdot |B|$. Rearranging terms gives:

$$|B| = \frac{|A|}{2} = \frac{(8 \cdot 7)^2}{2}.$$

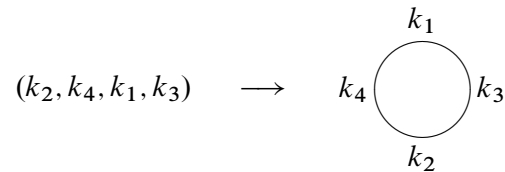
On the second line, we’ve computed the size of A using the General Product Rule just as in the earlier chess problem.

15.4.2 Knights of the Round Table

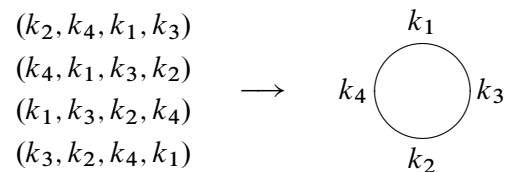
In how many ways can King Arthur arrange to seat his n different knights at his round table? Two seatings are considered to be the same *arrangement* if they yield the same sequence of knights starting at knight number 1 and going clockwise around the table. For example, the following two seatings determine the same arrangement:



So a seating is determined by the sequence of knights going clockwise around the table starting at the top seat. This means seatings are formally the same as the set, A , of all permutations of the knights. An arrangement is determined by the sequence of knights going clockwise around the table starting after knight number 1, so it is formally the same as the set, B , of all permutations of knights 2 through n . We can map each permutation in A to an arrangement in set B by seating the first knight in the permutation at the top of the table, putting the second knight to his left, the third knight to the left of the second, and so forth all the way around the table. For example:



This mapping is actually an n -to-1 function from A to B , since all n cyclic shifts of the original sequence map to the same seating arrangement. In the example, $n = 4$ different sequences map to the same seating arrangement:



Therefore, by the division rule, the number of circular seating arrangements is:

$$|B| = \frac{|A|}{n} = \frac{n!}{n} = (n-1)!$$

Note that $|A| = n!$ since there are $n!$ permutations of n knights.

15.5 Counting Subsets

How many k -element subsets of an n -element set are there? This question arises all the time in various guises:

- In how many ways can I select 5 books from my collection of 100 to bring on vacation?
- How many different 13-card Bridge hands can be dealt from a 52-card deck?
- In how many ways can I select 5 toppings for my pizza if there are 14 available toppings?

This number comes up so often that there is a special notation for it:

$$\binom{n}{k} ::= \text{the number of } k\text{-element subsets of an } n\text{-element set.}$$

The expression $\binom{n}{k}$ is read “ n choose k .” Now we can immediately express the answers to all three questions above:

- I can select 5 books from 100 in $\binom{100}{5}$ ways.
- There are $\binom{52}{13}$ different Bridge hands.
- There are $\binom{14}{5}$ different 5-topping pizzas, if 14 toppings are available.

15.5.1 The Subset Rule

We can derive a simple formula for the n choose k number using the Division Rule. We do this by mapping any permutation of an n -element set $\{a_1, \dots, a_n\}$ into a k -element subset simply by taking the first k elements of the permutation. That is, the permutation $a_1 a_2 \dots a_n$ will map to the set $\{a_1, a_2, \dots, a_k\}$.

Notice that any other permutation with the same first k elements $a_1, \dots, a_k$ in *any order* and the same remaining elements $n - k$ elements in *any order* will also map to this set. What’s more, a permutation can only map to $\{a_1, a_2, \dots, a_k\}$ if its first k elements are the elements $a_1, \dots, a_k$ in some order. Since there are $k!$ possible permutations of the first k elements and $(n - k)!$ permutations of the remaining elements, we conclude from the Product Rule that exactly $k!(n - k)!$ permutations of the n -element set map to the particular subset, S . In other words, the mapping from permutations to k -element subsets is $k!(n - k)!$ -to-1.

But we know there are $n!$ permutations of an n -element set, so by the Division Rule, we conclude that

$$n! = k!(n - k)! \binom{n}{k}$$

which proves:

Rule 15.5.1 (Subset Rule). *The number of k -element subsets of an n -element set is*

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}.$$

Notice that this works even for 0-element subsets: $n!/0!n! = 1$. Here we use the fact that $0!$ is a *product* of 0 terms, which by convention² equals 1.

15.5.2 Bit Sequences

How many n -bit sequences contain exactly k ones? We’ve already seen the straightforward bijection between subsets of an n -element set and n -bit sequences. For example, here is a 3-element subset of $\{x_1, x_2, \dots, x_8\}$ and the associated 8-bit sequence:

$$\begin{array}{cccccccc} \{ & x_1, & & x_4, & x_5 & & & \} \\ (& 1, & 0, & 0, & 1, & 1, & 0, & 0 \end{array}$$

Notice that this sequence has exactly 3 ones, each corresponding to an element of the 3-element subset. More generally, the n -bit sequences corresponding to a k -element subset will have exactly k ones. So by the Bijection Rule,

Corollary. *The number of n -bit sequences with exactly k ones is $\binom{n}{k}$.*

15.6 Sequences with Repetitions

15.6.1 Sequences of Subsets

Choosing a k -element subset of an n -element set is the same as splitting the set into a pair of subsets: the first subset of size k and the second subset consisting of the remaining $n - k$ elements. So the Subset Rule can be understood as a rule for counting the number of such splits into pairs of subsets.

²We don’t use it here, but a *sum* of zero terms equals 0.

We can generalize this to splits into more than two subsets. Namely, let A be an n -element set and $k_1, k_2, \dots, k_m$ be nonnegative integers whose sum is n . A $(k_1, k_2, \dots, k_m)$ -split of A is a sequence

$$(A_1, A_2, \dots, A_m)$$

where the A_i are disjoint subsets of A and $|A_i| = k_i$ for $i = 1, \dots, m$.

To count the number of splits we take the same approach as for the Subset Rule. Namely, we map any permutation $a_1 a_2 \dots a_n$ of an n -element set A into a $(k_1, k_2, \dots, k_m)$ -split by letting the 1st subset in the split be the first k_1 elements of the permutation, the 2nd subset of the split be the next k_2 elements, $\dots$, and the m th subset of the split be the final k_m elements of the permutation. This map is a $k_1! k_2! \dots k_m!$ -to-1 function from the $n!$ permutations to the $(k_1, k_2, \dots, k_m)$ -splits of A , so from the Division Rule we conclude the Subset Split Rule:

Definition 15.6.1. For $n, k_1, \dots, k_m \in \mathbb{N}$, such that $k_1 + k_2 + \dots + k_m = n$, define the *multinomial coefficient*

$$\binom{n}{k_1, k_2, \dots, k_m} ::= \frac{n!}{k_1! k_2! \dots k_m!}.$$

Rule 15.6.2 (Subset Split Rule). *The number of $(k_1, k_2, \dots, k_m)$ -splits of an n -element set is*

$$\binom{n}{k_1, \dots, k_m}.$$

15.6.2 The Bookkeeper Rule

We can also generalize our count of n -bit sequences with k ones to counting sequences of n letters over an alphabet with more than two letters. For example, how many sequences can be formed by permuting the letters in the 10-letter word BOOKKEEPER?

Notice that there are 1 B, 2 O's, 2 K's, 3 E's, 1 P, and 1 R in BOOKKEEPER. This leads to a straightforward bijection between permutations of BOOKKEEPER and $(1, 2, 2, 3, 1, 1)$ -splits of $\{1, 2, \dots, 10\}$. Namely, map a permutation to the sequence of sets of positions where each of the different letters occur.

For example, in the permutation BOOKKEEPER itself, the B is in the 1st position, the O's occur in the 2nd and 3rd positions, K's in 4th and 5th, the E's in the 6th, 7th and 9th, P in the 8th, and R is in the 10th position. So BOOKKEEPER maps to

$$(\{1\}, \{2, 3\}, \{4, 5\}, \{6, 7, 9\}, \{8\}, \{10\}).$$

From this bijection and the Subset Split Rule, we conclude that the number of ways to rearrange the letters in the word BOOKKEEPER is:

$$\frac{\overbrace{10!}^{\text{total letters}}}{\underbrace{1!}_{\text{B's}} \underbrace{2!}_{\text{O's}} \underbrace{2!}_{\text{K's}} \underbrace{3!}_{\text{E's}} \underbrace{1!}_{\text{P's}} \underbrace{1!}_{\text{R's}}}$$

This example generalizes directly to an exceptionally useful counting principle which we will call the

Rule 15.6.3 (Bookkeeper Rule). *Let $l_1, \dots, l_m$ be distinct elements. The number of sequences with k_1 occurrences of l_1 , and k_2 occurrences of l_2 , ..., and k_m occurrences of l_m is*

$$\binom{k_1 + k_2 + \dots + k_m}{k_1, \dots, k_m}.$$

For example, suppose you are planning a 20-mile walk, which should include 5 northward miles, 5 eastward miles, 5 southward miles, and 5 westward miles. How many different walks are possible?

There is a bijection between such walks and sequences with 5 N's, 5 E's, 5 S's, and 5 W's. By the Bookkeeper Rule, the number of such sequences is:

$$\frac{20!}{(5!)^4}.$$

15.7 The Binomial Theorem

Counting gives insight into one of the basic theorems of algebra. A *binomial* is a sum of two terms, such as $a + b$. Now consider its 4th power, $(a + b)^4$.

If we multiply out this 4th power expression completely, we get

$$\begin{aligned} (a + b)^4 = & \quad aaaa + aaab + aaba + aabb \\ & + abaa + abab + abba + abbb \\ & + baaa + baab + baba + babb \\ & + bbaa + bbab + bbba + bbbb \end{aligned}$$

Notice that there is one term for every sequence of a 's and b 's. So there are 2^4 terms, and the number of terms with k copies of b and $n - k$ copies of a is:

$$\frac{n!}{k! (n - k)!} = \binom{n}{k}$$

by the Bookkeeper Rule. Hence, the coefficient of $a^{n-k}b^k$ is $\binom{n}{k}$. So for $n = 4$, this means:

$$(a + b)^4 = \binom{4}{0} \cdot a^4 b^0 + \binom{4}{1} \cdot a^3 b^1 + \binom{4}{2} \cdot a^2 b^2 + \binom{4}{3} \cdot a^1 b^3 + \binom{4}{4} \cdot a^0 b^4$$

In general, this reasoning gives the Binomial Theorem:

Theorem 15.7.1 (Binomial Theorem). *For all $n \in \mathbb{N}$ and $a, b \in \mathbb{R}$:*

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

The Binomial Theorem explains why the n choose k number is called a *binomial coefficient*.

This reasoning about binomials extends nicely to *multinomials*, which are sums of two or more terms. For example, suppose we wanted the coefficient of

$$bo^2k^2e^3pr$$

in the expansion of $(b + o + k + e + p + r)^{10}$. Each term in this expansion is a product of 10 variables where each variable is one of b, o, k, e, p , or r . Now, the coefficient of $bo^2k^2e^3pr$ is the number of those terms with exactly 1 b , 2 o 's, 2 k 's, 3 e 's, 1 p , and 1 r . And the number of such terms is precisely the number of rearrangements of the word BOOKKEEPER:

$$\binom{10}{1, 2, 2, 3, 1, 1} = \frac{10!}{1! 2! 2! 3! 1! 1!}.$$

This reasoning extends to a general theorem.

Theorem 15.7.2 (Multinomial Theorem). *For all $n \in \mathbb{N}$,*

$$(z_1 + z_2 + \cdots + z_m)^n = \sum_{\substack{k_1, \dots, k_m \in \mathbb{N} \\ k_1 + \cdots + k_m = n}} \binom{n}{k_1, k_2, \dots, k_m} z_1^{k_1} z_2^{k_2} \cdots z_m^{k_m}.$$

You'll be better off remembering the reasoning behind the Multinomial Theorem rather than this cumbersome formal statement.

15.8 A Word about Words

Someday you might refer to the Subset Split Rule or the Bookkeeper Rule in front of a roomful of colleagues and discover that they’re all staring back at you blankly. This is not because they’re dumb, but rather because we made up the name “Bookkeeper Rule.” However, the rule is excellent and the name is apt, so we suggest that you play through: “You know? The Bookkeeper Rule? Don’t you guys know *anything???*”

The Bookkeeper Rule is sometimes called the “formula for permutations with indistinguishable objects.” The size k subsets of an n -element set are sometimes called k -combinations. Other similar-sounding descriptions are “combinations with repetition, permutations with repetition, r -permutations, permutations with indistinguishable objects,” and so on. However, the counting rules we’ve taught you are sufficient to solve all these sorts of problems without knowing this jargon, so we won’t burden you with it.

15.9 Counting Practice: Poker Hands

Five-Card Draw is a card game in which each player is initially dealt a *hand* consisting of 5 cards from a deck of 52 cards.³ (Then the game gets complicated, but let’s not worry about that.) The number of different hands in Five-Card Draw is the number of 5-element subsets of a 52-element set, which is

$$\binom{52}{5} = 2,598,960.$$

Let’s get some counting practice by working out the number of hands with various special properties.

³There are 52 cards in a standard deck. Each card has a *suit* and a *rank*. There are four suits:

♠ (spades) ♥ (hearts) ♣ (clubs) ♦ (diamonds)

And there are 13 ranks, listed here from lowest to highest:

Ace Jack Queen King
A, 2, 3, 4, 5, 6, 7, 8, 9, J, Q, K.

Thus, for example, $8♥$ is the 8 of hearts and $A♠$ is the ace of spades.

15.9.1 Hands with a Four-of-a-Kind

A *Four-of-a-Kind* is a set of four cards with the same rank. How many different hands contain a Four-of-a-Kind? Here are a couple examples:

$$\{8\spadesuit, 8\diamond, Q\heartsuit, 8\clubsuit\}$$

$$\{A\clubsuit, 2\clubsuit, 2\heartsuit, 2\diamond, 2\spadesuit\}$$

As usual, the first step is to map this question to a sequence-counting problem. A hand with a Four-of-a-Kind is completely described by a sequence specifying:

1. The rank of the four cards.
2. The rank of the extra card.
3. The suit of the extra card.

Thus, there is a bijection between hands with a Four-of-a-Kind and sequences consisting of two distinct ranks followed by a suit. For example, the three hands above are associated with the following sequences:

$$(8, Q, \heartsuit) \leftrightarrow \{8\spadesuit, 8\diamond, 8\heartsuit, 8\clubsuit, Q\heartsuit\}$$

$$(2, A, \clubsuit) \leftrightarrow \{2\clubsuit, 2\heartsuit, 2\diamond, 2\spadesuit, A\clubsuit\}$$

Now we need only count the sequences. There are 13 ways to choose the first rank, 12 ways to choose the second rank, and 4 ways to choose the suit. Thus, by the Generalized Product Rule, there are $13 \cdot 12 \cdot 4 = 624$ hands with a Four-of-a-Kind. This means that only 1 hand in about 4165 has a Four-of-a-Kind. Not surprisingly, Four-of-a-Kind is considered to be a very good poker hand!

15.9.2 Hands with a Full House

A *Full House* is a hand with three cards of one rank and two cards of another rank. Here are some examples:

$$\{2\spadesuit, 2\clubsuit, 2\diamond, J\clubsuit, J\diamond\}$$

$$\{5\diamond, 5\clubsuit, 5\heartsuit, 7\heartsuit, 7\clubsuit\}$$

Again, we shift to a problem about sequences. There is a bijection between Full Houses and sequences specifying:

1. The rank of the triple, which can be chosen in 13 ways.
2. The suits of the triple, which can be selected in $\binom{4}{3}$ ways.
3. The rank of the pair, which can be chosen in 12 ways.
4. The suits of the pair, which can be selected in $\binom{4}{2}$ ways.

The example hands correspond to sequences as shown below:

$$(2, \{\spadesuit, \clubsuit, \diamondsuit\}, J, \{\clubsuit, \diamondsuit\}) \leftrightarrow \{2\spadesuit, 2\clubsuit, 2\diamondsuit, J\clubsuit, J\diamondsuit\}$$

$$(5, \{\diamondsuit, \clubsuit, \heartsuit\}, 7, \{\heartsuit, \clubsuit\}) \leftrightarrow \{5\diamondsuit, 5\clubsuit, 5\heartsuit, 7\heartsuit, 7\clubsuit\}$$

By the Generalized Product Rule, the number of Full Houses is:

$$13 \cdot \binom{4}{3} \cdot 12 \cdot \binom{4}{2}.$$

We’re on a roll —but we’re about to hit a speed bump.

15.9.3 Hands with Two Pairs

How many hands have *Two Pairs*; that is, two cards of one rank, two cards of another rank, and one card of a third rank? Here are examples:

$$\{3\diamondsuit, 3\spadesuit, Q\diamondsuit, Q\heartsuit, A\clubsuit\}$$

$$\{9\heartsuit, 9\diamondsuit, 5\heartsuit, 5\clubsuit, K\spadesuit\}$$

Each hand with Two Pairs is described by a sequence consisting of:

1. The rank of the first pair, which can be chosen in 13 ways.
2. The suits of the first pair, which can be selected $\binom{4}{2}$ ways.
3. The rank of the second pair, which can be chosen in 12 ways.
4. The suits of the second pair, which can be selected in $\binom{4}{2}$ ways.
5. The rank of the extra card, which can be chosen in 11 ways.
6. The suit of the extra card, which can be selected in $\binom{4}{1} = 4$ ways.

Thus, it might appear that the number of hands with Two Pairs is:

$$13 \cdot \binom{4}{2} \cdot 12 \cdot \binom{4}{2} \cdot 11 \cdot 4.$$

Wrong answer! The problem is that there is *not* a bijection from such sequences to hands with Two Pairs. This is actually a 2-to-1 mapping. For example, here are the pairs of sequences that map to the hands given above:

$$\begin{array}{ll} (3, \{\diamond, \spadesuit\}, Q, \{\diamond, \heartsuit\}, A, \clubsuit) & \searrow \\ (Q, \{\diamond, \heartsuit\}, 3, \{\diamond, \spadesuit\}, A, \clubsuit) & \nearrow \\ & \{3\diamond, 3\spadesuit, Q\diamond, Q\heartsuit, A\clubsuit\} \\ (9, \{\heartsuit, \diamond\}, 5, \{\heartsuit, \clubsuit\}, K, \spadesuit) & \searrow \\ (5, \{\heartsuit, \clubsuit\}, 9, \{\heartsuit, \diamond\}, K, \spadesuit) & \nearrow \\ & \{9\heartsuit, 9\diamond, 5\heartsuit, 5\clubsuit, K\spadesuit\} \end{array}$$

The problem is that nothing distinguishes the first pair from the second. A pair of 5's and a pair of 9's is the same as a pair of 9's and a pair of 5's. We avoided this difficulty in counting Full Houses because, for example, a pair of 6's and a triple of kings is different from a pair of kings and a triple of 6's.

We ran into precisely this difficulty last time, when we went from counting arrangements of *different* pieces on a chessboard to counting arrangements of two *identical* rooks. The solution then was to apply the Division Rule, and we can do the same here. In this case, the Division rule says there are twice as many sequences as hands, so the number of hands with Two Pairs is actually:

$$\frac{13 \cdot \binom{4}{2} \cdot 12 \cdot \binom{4}{2} \cdot 11 \cdot 4}{2}.$$

Another Approach

The preceding example was disturbing! One could easily overlook the fact that the mapping was 2-to-1 on an exam, fail the course, and turn to a life of crime. You can make the world a safer place in two ways:

1. Whenever you use a mapping $f : A \rightarrow B$ to translate one counting problem to another, check that the same number elements in A are mapped to each element in B . If k elements of A map to each of element of B , then apply the Division Rule using the constant k .
2. As an extra check, try solving the same problem in a different way. Multiple approaches are often available—and all had better give the same answer!

(Sometimes different approaches give answers that *look* different, but turn out to be the same after some algebra.)

We already used the first method; let’s try the second. There is a bijection between hands with two pairs and sequences that specify:

1. The ranks of the two pairs, which can be chosen in $\binom{13}{2}$ ways.
2. The suits of the lower-rank pair, which can be selected in $\binom{4}{2}$ ways.
3. The suits of the higher-rank pair, which can be selected in $\binom{4}{2}$ ways.
4. The rank of the extra card, which can be chosen in 11 ways.
5. The suit of the extra card, which can be selected in $\binom{4}{1} = 4$ ways.

For example, the following sequences and hands correspond:

$$\begin{aligned} (\{3, Q\}, \{\diamond, \spadesuit\}, \{\diamond, \heartsuit\}, A, \clubsuit) &\leftrightarrow \{3\diamond, 3\spadesuit, Q\diamond, Q\heartsuit, A\clubsuit\} \\ (\{9, 5\}, \{\heartsuit, \clubsuit\}, \{\heartsuit, \diamond\}, K, \spadesuit) &\leftrightarrow \{9\heartsuit, 9\diamond, 5\heartsuit, 5\clubsuit, K\spadesuit\} \end{aligned}$$

Thus, the number of hands with two pairs is:

$$\binom{13}{2} \cdot \binom{4}{2} \cdot \binom{4}{2} \cdot 11 \cdot 4.$$

This is the same answer we got before, though in a slightly different form.

15.9.4 Hands with Every Suit

How many hands contain at least one card from every suit? Here is an example of such a hand:

$$\{7\diamond, K\clubsuit, 3\diamond, A\heartsuit, 2\spadesuit\}$$

Each such hand is described by a sequence that specifies:

1. The ranks of the diamond, the club, the heart, and the spade, which can be selected in $13 \cdot 13 \cdot 13 \cdot 13 = 13^4$ ways.
2. The suit of the extra card, which can be selected in 4 ways.
3. The rank of the extra card, which can be selected in 12 ways.

For example, the hand above is described by the sequence:

$$(7, K, A, 2, \diamond, 3) \leftrightarrow \{7\diamond, K\clubsuit, A\heartsuit, 2\spadesuit, 3\diamond\}.$$

Are there other sequences that correspond to the same hand? There is one more! We could equally well regard either the $3\diamond$ or the $7\diamond$ as the extra card, so this is actually a 2-to-1 mapping. Here are the two sequences corresponding to the example hand:

$$\begin{array}{ccc} (7, K, A, 2, \diamond, 3) & \searrow & \{7\diamond, K\clubsuit, A\heartsuit, 2\spadesuit, 3\diamond\} \\ (3, K, A, 2, \diamond, 7) & \nearrow & \end{array}$$

Therefore, the number of hands with every suit is:

$$\frac{13^4 \cdot 4 \cdot 12}{2}.$$

15.10 The Pigeonhole Principle

Here is an old puzzle:

A drawer in a dark room contains red socks, green socks, and blue socks. How many socks must you withdraw to be sure that you have a matching pair?

For example, picking out three socks is not enough; you might end up with one red, one green, and one blue. The solution relies on the

Pigeonhole Principle

If there are more pigeons than holes they occupy, then at least two pigeons must be in the same hole.

What pigeons have to do with selecting footwear under poor lighting conditions may not be immediately obvious, but if we let socks be pigeons and the colors be three pigeonholes, then as soon as you pick four socks, there are bound to be two in the same hole, that is, with the same color. So four socks are enough to ensure a matched pair. For example, one possible mapping of four socks to three colors is shown in Figure 15.3.

A rigorous statement of the Principle goes this way:

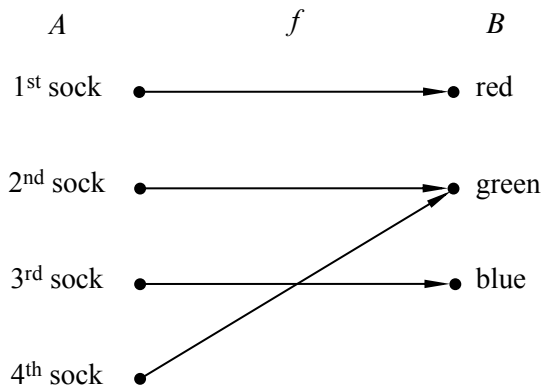


Figure 15.3 One possible mapping of four socks to three colors.

Rule 15.10.1 (Pigeonhole Principle). *If $|A| > |B|$, then for every total function $f : A \rightarrow B$, there exist two different elements of A that are mapped by f to the same element of B .*

Stating the Principle this way may be less intuitive, but it should now sound familiar: it is simply the contrapositive of the Mapping Rules injective case (5.2). Here, the pigeons form set A , the pigeonholes are the set B , and f describes which hole each pigeon occupies.

Mathematicians have come up with many ingenious applications for the pigeonhole principle. If there were a cookbook procedure for generating such arguments, we’d give it to you. Unfortunately, there isn’t one. One helpful tip, though: when you try to solve a problem with the pigeonhole principle, the key is to clearly identify three things:

1. The set A (the pigeons).
2. The set B (the pigeonholes).
3. The function f (the rule for assigning pigeons to pigeonholes).

15.10.1 Hairs on Heads

There are a number of generalizations of the pigeonhole principle. For example:

Rule 15.10.2 (Generalized Pigeonhole Principle). *If $|A| > k \cdot |B|$, then every total function $f : A \rightarrow B$ maps at least $k + 1$ different elements of A to the same element of B .*

For example, if you pick two people at random, surely they are extremely unlikely to have *exactly* the same number of hairs on their heads. However, in the remarkable city of Boston, Massachusetts there are actually *three* people who have exactly the same number of hairs! Of course, there are many bald people in Boston, and they all have zero hairs. But we’re talking about non-bald people; say a person is non-bald if they have at least ten thousand hairs on their head.

Boston has about 500,000 non-bald people, and the number of hairs on a person’s head is at most 200,000. Let A be the set of non-bald people in Boston, let $B = \{10,000, 10,001, \dots, 200,000\}$, and let f map a person to the number of hairs on his or her head. Since $|A| > 2|B|$, the Generalized Pigeonhole Principle implies that at least three people have exactly the same number of hairs. We don’t know who they are, but we know they exist!

15.10.2 Subsets with the Same Sum

For your reading pleasure, we have displayed ninety 25-digit numbers in Figure 15.4. Are there two different subsets of these 25-digit numbers that have the same sum? For example, maybe the sum of the last ten numbers in the first column is equal to the sum of the first eleven numbers in the second column?

Finding two subsets with the same sum may seem like a silly puzzle, but solving these sorts of problems turns out to be useful in diverse applications such as finding good ways to fit packages into shipping containers and decoding secret messages.

It turns out that it is hard to find different subsets with the same sum, which is why this problem arises in cryptography. But it is easy to prove that two such subsets *exist*. That’s where the Pigeonhole Principle comes in.

Let A be the collection of all subsets of the 90 numbers in the list. Now the sum of any subset of numbers is at most $90 \cdot 10^{25}$, since there are only 90 numbers and every 25-digit number is less than 10^{25} . So let B be the set of integers $\{0, 1, \dots, 90 \cdot 10^{25}\}$, and let f map each subset of numbers (in A) to its sum (in B).

We proved that an n -element set has 2^n different subsets in Section 15.2. Therefore:

$$|A| = 2^{90} \geq 1.237 \times 10^{27}$$

On the other hand:

$$|B| = 90 \cdot 10^{25} + 1 \leq 0.901 \times 10^{27}.$$

Both quantities are enormous, but $|A|$ is a bit greater than $|B|$. This means that f maps at least two elements of A to the same element of B . In other words, by the Pigeonhole Principle, two different subsets must have the same sum!

Notice that this proof gives no indication *which* two sets of numbers have the same sum. This frustrating variety of argument is called a *nonconstructive proof*.

15.10. The Pigeonhole Principle

495

0020480135385502964448038	3171004832173501394113017
5763257331083479647409398	8247331000042995311646021
0489445991866915676240992	3208234421597368647019265
5800949123548989122628663	8496243997123475922766310
1082662032430379651370981	3437254656355157864869113
6042900801199280218026001	8518399140676002660747477
1178480894769706178994993	3574883393058653923711365
6116171789137737896701405	8543691283470191452333763
1253127351683239693851327	3644909946040480189969149
6144868973001582369723512	8675309258374137092461352
1301505129234077811069011	3790044132737084094417246
6247314593851169234746152	8694321112363996867296665
1311567111143866433882194	3870332127437971355322815
6814428944266874963488274	8772321203608477245851154
1470029452721203587686214	4080505804577801451363100
6870852945543886849147881	8791422161722582546341091
1578271047286257499433886	4167283461025702348124920
6914955508120950093732397	9062628024592126283973285
1638243921852176243192354	4235996831123777788211249
6949632451365987152423541	9137845566925526349897794
1763580219131985963102365	4670939445749439042111220
7128211143613619828415650	9153762966803189291934419
1826227795601842231029694	4815379351865384279613427
7173920083651862307925394	9270880194077636406984249
1843971862675102037201420	4837052948212922604442190
7215654874211755676220587	9324301480722103490379204
2396951193722134526177237	5106389423855018550671530
7256932847164391040233050	9436090832146695147140581
2781394568268599801096354	5142368192004769218069910
7332822657075235431620317	9475308159734538249013238
2796605196713610405408019	5181234096130144084041856
7426441829541573444964139	9492376623917486974923202
2931016394761975263190347	5198267398125617994391348
7632198126531809327186321	9511972558779880288252979
2933458058294405155197296	5317592940316231219758372
7712154432211912882310511	9602413424619187112552264
3075514410490975920315348	5384358126771794128356947
7858918664240262356610010	9631217114906129219461111
8149436716871371161932035	3157693105325111284321993
3111474985252793452860017	5439211712248901995423441
7898156786763212963178679	9908189853102753335981319
3145621587936120118438701	5610379826092838192760458
8147591017037573337848616	9913237476341764299813987
3148901255628881103198549	5632317555465228677676044
5692168374637019617423712	8176063831682536571306791

Figure 15.4 Ninety 25-digit numbers. Can you find two different subsets of these numbers that have the same sum?

The \$100 prize for two same-sum subsets

To see if it was possible to actually *find* two different subsets of the ninety 25-digit numbers with the same sum, we offered a \$100 prize to the first student who did it. We didn’t expect to have to pay off this bet, but we underestimated the ingenuity and initiative of the students. One computer science major wrote a program that cleverly searched only among a reasonably small set of “plausible” sets, sorted them by their sums, and actually found a couple with the same sum. He won the prize. A few days later, a math major figured out how to reformulate the sum problem as a “lattice basis reduction” problem; then he found a software package implementing an efficient basis reduction procedure, and using it, he very quickly found lots of pairs of subsets with the same sum. He didn’t win the prize, but he got a standing ovation from the class —staff included.

The \$500 Prize for Sets with Distinct Subset Sums

How can we construct a set of n positive integers such that all its subsets have *distinct* sums? One way is to use powers of two:

$$\{1, 2, 4, 8, 16\}$$

This approach is so natural that one suspects all other such sets must involve larger numbers. (For example, we could safely replace 16 by 17, but not by 15.) Remarkably, there are examples involving *smaller* numbers. Here is one:

$$\{6, 9, 11, 12, 13\}$$

One of the top mathematicians of the Twentieth Century, Paul Erdős, conjectured in 1931 that there are no such sets involving *significantly* smaller numbers. More precisely, he conjectured that the largest number in such a set must be greater than $c2^n$ for some constant $c > 0$. He offered \$500 to anyone who could prove or disprove his conjecture, but the problem remains unsolved.

15.11 A Magic Trick

A Magician sends an Assistant into the audience with a deck of 52 cards while the Magician looks away.

Five audience members each select one card from the deck. The Assistant then gathers up the five cards and holds up four of them so the Magician can see them. The Magician concentrates for a short time and then correctly names the secret, fifth card!

Since we don’t really believe the Magician can read minds, we know the Assistant has somehow communicated the secret card to the Magician. Real Magicians and Assistants are not to be trusted, so we expect that the Assistant would secretly signal the Magician with coded phrases or body language, but for this trick they don’t have to cheat. In fact, the Magician and Assistant could be kept out of sight of each other while some audience member holds up the 4 cards designated by the Assistant for the Magician to see.

Of course, without cheating, there is still an obvious way the Assistant can communicate to the Magician: he can choose any of the $4! = 24$ permutations of the 4 cards as the order in which to hold up the cards. However, this alone won’t quite work: there are 48 cards remaining in the deck, so the Assistant doesn’t have enough choices of orders to indicate exactly what the secret card is (though he could narrow it down to two cards).

15.11.1 The Secret

The method the Assistant can use to communicate the fifth card exactly is a nice application of what we know about counting and matching.

The Assistant has a second legitimate way to communicate: he can choose *which of the five cards to keep hidden*. Of course, it’s not clear how the Magician could determine which of these five possibilities the Assistant selected by looking at the four visible cards, but there is a way, as we’ll now explain.

The problem facing the Magician and Assistant is actually a bipartite matching problem. Each vertex on left will correspond to the information available to the Assistant, namely, a *set* of 5 cards. So the set X of left hand vertices will have $\binom{52}{5}$ elements.

Each vertex on right will correspond to the information available to the Magician, namely, a *sequence* of 4 distinct cards. So the set Y of right hand vertices will have $52 \cdot 51 \cdot 50 \cdot 49$ elements. When the audience selects a set of 5 cards, then the Assistant must reveal a sequence of 4 cards from that hand. This constraint is represented by having an edge between a set of 5 cards on the left and a sequence of 4 cards on the right precisely when every card in the sequence is also in the set. This specifies the bipartite graph. Some edges are shown in the diagram in Figure 15.5.

For example,

$$\{8\heartsuit, K\spadesuit, Q\spadesuit, 2\diamondsuit, 6\diamondsuit\} \tag{15.2}$$

is an element of X on the left. If the audience selects this set of 5 cards, then

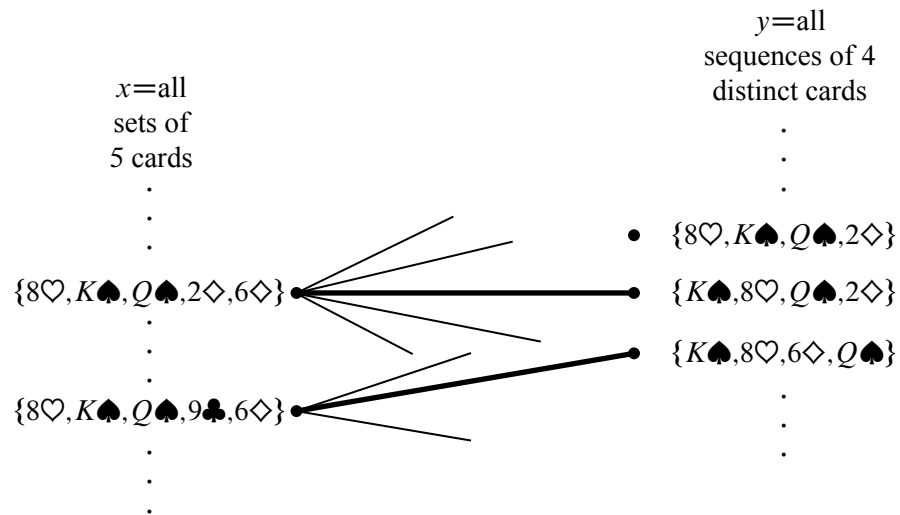


Figure 15.5 The bipartite graph where the nodes on the left correspond to *sets* of 5 cards and the nodes on the right correspond to *sequences* of 4 cards. There is an edge between a set and a sequence whenever all the cards in the sequence are contained in the set.

there are many different 4-card sequences on the right in set Y that the Assistant could choose to reveal, including $(8\heartsuit, K\spadesuit, Q\spadesuit, 2\diamond)$, $(K\spadesuit, 8\heartsuit, Q\spadesuit, 2\diamond)$, and $(K\spadesuit, 8\heartsuit, 6\diamond, Q\spadesuit)$.

What the Magician and his Assistant need to perform the trick is a *matching* for the X vertices. If they agree in advance on some matching, then when the audience selects a set of 5 cards, the Assistant reveals the matching sequence of 4 cards. The Magician uses the matching to find the audience’s chosen set of 5 cards, and so he can name the one not already revealed.

For example, suppose the Assistant and Magician agree on a matching containing the two bold edges in Figure 15.5. If the audience selects the set

$$\{8\heartsuit, K\spadesuit, Q\spadesuit, 9\clubsuit, 6\diamond\}, \quad (15.3)$$

then the Assistant reveals the corresponding sequence

$$(K\spadesuit, 8\heartsuit, 6\diamond, Q\spadesuit). \quad (15.4)$$

Using the matching, the Magician sees that the hand (15.3) is matched to the sequence (15.4), so he can name the one card in the corresponding set not already revealed, namely, the $9\clubsuit$. Notice that the fact that the sets are *matched*, that is, that different sets are paired with *distinct* sequences, is essential. For example, if

the audience picked the previous hand (15.2), it would be possible for the Assistant to reveal the same sequence (15.4), but he better not do that; if he did, then the Magician would have no way to tell if the remaining card was the $9\clubsuit$ or the $2\diamondsuit$.

So how can we be sure the needed matching can be found? The answer is that each vertex on the left has degree $5 \cdot 4! = 120$, since there are five ways to select the card kept secret and there are $4!$ permutations of the remaining 4 cards. In addition, each vertex on the right has degree 48, since there are 48 possibilities for the fifth card. So this graph is *degree-constrained* according to Definition 11.5.5, and so has a matching by Theorem 11.5.6.

In fact, this reasoning shows that the Magician could still pull off the trick if 120 cards were left instead of 48, that is, the trick would work with a deck as large as 124 different cards —without any magic!

15.11.2 The Real Secret

But wait a minute! It’s all very well in principle to have the Magician and his Assistant agree on a matching, but how are they supposed to remember a matching with $\binom{52}{5} = 2,598,960$ edges? For the trick to work in practice, there has to be a way to match hands and card sequences mentally and on the fly.

We’ll describe one approach. As a running example, suppose that the audience selects:

$10\heartsuit \quad 9\diamondsuit \quad 3\heartsuit \quad Q\spadesuit \quad J\diamondsuit.$

- The Assistant picks out two cards of the same suit. In the example, the assistant might choose the $3\heartsuit$ and $10\heartsuit$. This is always possible because of the Pigeonhole Principle —there are five cards and 4 suits so two cards must be in the same suit.
- The Assistant locates the ranks of these two cards on the cycle shown in Figure 15.6. For any two distinct ranks on this cycle, one is always between 1 and 6 hops clockwise from the other. For example, the $3\heartsuit$ is 6 hops clockwise from the $10\heartsuit$.
- The more counterclockwise of these two cards is revealed first, and the other becomes the secret card. Thus, in our example, the $10\heartsuit$ would be revealed, and the $3\heartsuit$ would be the secret card. Therefore:
 - The suit of the secret card is the same as the suit of the first card revealed.
 - The rank of the secret card is between 1 and 6 hops clockwise from the rank of the first card revealed.

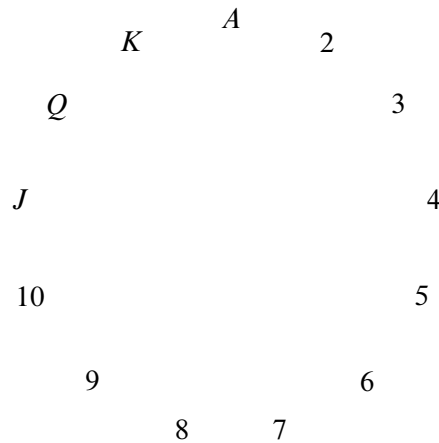


Figure 15.6 The 13 card ranks arranged in cyclic order.

- All that remains is to communicate a number between 1 and 6. The Magician and Assistant agree beforehand on an ordering of all the cards in the deck from smallest to largest such as:

$A\clubsuit A\heartsuit A\spadesuit 2\clubsuit 2\heartsuit 2\spadesuit \dots K\heartsuit K\spadesuit$

The order in which the last three cards are revealed communicates the number according to the following scheme:

(small, medium, large)	= 1
(small, large, medium)	= 2
(medium, small, large)	= 3
(medium, large, small)	= 4
(large, small, medium)	= 5
(large, medium, small)	= 6

In the example, the Assistant wants to send 6 and so reveals the remaining three cards in large, medium, small order. Here is the complete sequence that the Magician sees:

$10\heartsuit Q\spadesuit J\heartsuit 9\heartsuit$

- The Magician starts with the first card, $10\heartsuit$, and hops 6 ranks clockwise to reach $3\heartsuit$, which is the secret card!

So that’s how the trick can work with a standard deck of 52 cards. On the other hand, Hall’s Theorem implies that the Magician and Assistant can *in principle* perform the trick with a deck of up to 124 cards. It turns out that there is a method

which they could actually learn to use with a reasonable amount of practice for a 124-card deck, but we won't explain it here.⁴

15.11.3 The Same Trick with Four Cards?

Suppose that the audience selects only *four* cards and the Assistant reveals a sequence of *three* to the Magician. Can the Magician determine the fourth card?

Let X be all the sets of four cards that the audience might select, and let Y be all the sequences of three cards that the Assistant might reveal. Now, on one hand, we have

$$|X| = \binom{52}{4} = 270,725$$

by the Subset Rule. On the other hand, we have

$$|Y| = 52 \cdot 51 \cdot 50 = 132,600$$

by the Generalized Product Rule. Thus, by the Pigeonhole Principle, the Assistant must reveal the *same* sequence of three cards for at least

$$\left\lceil \frac{270,725}{132,600} \right\rceil = 3$$

different four-card hands. This is bad news for the Magician: if he sees that sequence of three, then there are at least three possibilities for the fourth card which he cannot distinguish. So there is no legitimate way for the Assistant to communicate exactly what the fourth card is!

15.12 Inclusion-Exclusion

How big is a union of sets? For example, suppose there are 60 math majors, 200 EECS majors, and 40 physics majors. How many students are there in these three departments? Let M be the set of math majors, E be the set of EECS majors, and P be the set of physics majors. In these terms, we're asking for $|M \cup E \cup P|$.

The Sum Rule says that if M , E , and P are disjoint, then the sum of their sizes is

$$|M \cup E \cup P| = |M| + |E| + |P|.$$

However, the sets M , E , and P might *not* be disjoint. For example, there might be a student majoring in both math and physics. Such a student would be counted

⁴See [The Best Card Trick](#) by Michael Kleber for more information.

twice on the right side of this equation, once as an element of M and once as an element of P . Worse, there might be a triple-major⁵ counted *three* times on the right side!

Our most-complicated counting rule determines the size of a union of sets that are not necessarily disjoint. Before we state the rule, let’s build some intuition by considering some easier special cases: unions of just two or three sets.

15.12.1 Union of Two Sets

For two sets, S_1 and S_2 , the *Inclusion-Exclusion Rule* is that the size of their union is:

$$|S_1 \cup S_2| = |S_1| + |S_2| - |S_1 \cap S_2| \quad (15.5)$$

Intuitively, each element of S_1 is accounted for in the first term, and each element of S_2 is accounted for in the second term. Elements in *both* S_1 and S_2 are counted *twice* —once in the first term and once in the second. This double-counting is corrected by the final term.

15.12.2 Union of Three Sets

So how many students are there in the math, EECS, and physics departments? In other words, what is $|M \cup E \cup P|$ if:

$$|M| = 60$$

$$|E| = 200$$

$$|P| = 40.$$

The size of a union of three sets is given by a more complicated Inclusion-Exclusion formula:

$$\begin{aligned} |S_1 \cup S_2 \cup S_3| &= |S_1| + |S_2| + |S_3| \\ &\quad - |S_1 \cap S_2| - |S_1 \cap S_3| - |S_2 \cap S_3| \\ &\quad + |S_1 \cap S_2 \cap S_3|. \end{aligned}$$

Remarkably, the expression on the right accounts for each element in the union of S_1 , S_2 , and S_3 exactly once. For example, suppose that x is an element of all three sets. Then x is counted three times (by the $|S_1|$, $|S_2|$, and $|S_3|$ terms), subtracted off three times (by the $|S_1 \cap S_2|$, $|S_1 \cap S_3|$, and $|S_2 \cap S_3|$ terms), and then counted once more (by the $|S_1 \cap S_2 \cap S_3|$ term). The net effect is that x is counted just once.

⁵... though not at MIT anymore.

If x is in two sets (say, S_1 and S_2), then x is counted twice (by the $|S_1|$ and $|S_2|$ terms) and subtracted once (by the $|S_1 \cap S_2|$ term). In this case, x does not contribute to any of the other terms, since $x \notin S_3$.

So we can't answer the original question without knowing the sizes of the various intersections. Let's suppose that there are:

- 4 math - EECS double majors
- 3 math - physics double majors
- 11 EECS - physics double majors
- 2 triple majors

Then $|M \cap E| = 4 + 2$, $|M \cap P| = 3 + 2$, $|E \cap P| = 11 + 2$, and $|M \cap E \cap P| = 2$. Plugging all this into the formula gives:

$$\begin{aligned} |M \cup E \cup P| &= |M| + |E| + |P| - |M \cap E| - |M \cap P| - |E \cap P| + |M \cap E \cap P| \\ &= 60 + 200 + 40 - 6 - 5 - 13 + 2 \\ &= 278 \end{aligned}$$

15.12.3 Sequences with 42, 04, or 60

In how many permutations of the set $\{0, 1, 2, \dots, 9\}$ do either 4 and 2, 0 and 4, or 6 and 0 appear consecutively? For example, none of these pairs appears in:

$$(7, 2, 9, 5, 4, 1, 3, 8, 0, 6).$$

The 06 at the end doesn't count; we need 60. On the other hand, both 04 and 60 appear consecutively in this permutation:

$$(7, 2, 5, \underline{6}, \underline{0}, \underline{4}, 3, 8, 1, 9).$$

Let P_{42} be the set of all permutations in which 42 appears. Define P_{60} and P_{04} similarly. Thus, for example, the permutation above is contained in both P_{60} and P_{04} , but not P_{42} . In these terms, we're looking for the size of the set $P_{42} \cup P_{04} \cup P_{60}$.

First, we must determine the sizes of the individual sets, such as P_{60} . We can use a trick: group the 6 and 0 together as a single symbol. Then there is an immediate bijection between permutations of $\{0, 1, 2, \dots, 9\}$ containing 6 and 0 consecutively and permutations of:

$$\{60, 1, 2, 3, 4, 5, 7, 8, 9\}.$$

For example, the following two sequences correspond:

$$(7, 2, 5, \underline{6}, \underline{0}, 4, 3, 8, 1, 9) \longleftrightarrow (7, 2, 5, \underline{60}, 4, 3, 8, 1, 9).$$

There are $9!$ permutations of the set containing 60, so $|P_{60}| = 9!$ by the Bijection Rule. Similarly, $|P_{04}| = |P_{42}| = 9!$ as well.

Next, we must determine the sizes of the two-way intersections, such as $P_{42} \cap P_{60}$. Using the grouping trick again, there is a bijection with permutations of the set:

$$\{42, 60, 1, 3, 5, 7, 8, 9\}.$$

Thus, $|P_{42} \cap P_{60}| = 8!$. Similarly, $|P_{60} \cap P_{04}| = 8!$ by a bijection with the set:

$$\{604, 1, 2, 3, 5, 7, 8, 9\}.$$

And $|P_{42} \cap P_{04}| = 8!$ as well by a similar argument. Finally, note that $|P_{60} \cap P_{04} \cap P_{42}| = 7!$ by a bijection with the set:

$$\{6042, 1, 3, 5, 7, 8, 9\}.$$

Plugging all this into the formula gives:

$$|P_{42} \cup P_{04} \cup P_{60}| = 9! + 9! + 9! - 8! - 8! - 8! + 7!.$$

15.12.4 Union of n Sets

The size of a union of n sets is given by the following rule.

Rule 15.12.1 (Inclusion-Exclusion).

$$|S_1 \cup S_2 \cup \cdots \cup S_n| =$$

$\begin{array}{ll} & \text{the sum of the sizes of the individual sets} \\ \text{minus} & \text{the sizes of all two-way intersections} \\ & \text{plus the sizes of all three-way intersections} \\ \text{minus} & \text{the sizes of all four-way intersections} \\ & \text{plus the sizes of all five-way intersections, etc.} \end{array}$

The formulas for unions of two and three sets are special cases of this general rule.

This way of expressing Inclusion-Exclusion is easy to understand and nearly as precise as expressing it in mathematical symbols, but we'll need the symbolic version below, so let's work on deciphering it now.

We already have a concise notation for the sum of sizes of the individual sets, namely,

$$\sum_{i=1}^n |S_i|.$$

A “two-way intersection” is a set of the form $S_i \cap S_j$ for $i \neq j$. We regard $S_j \cap S_i$ as the same two-way intersection as $S_i \cap S_j$, so we can assume that $i < j$. Now we can express the sum of the sizes of the two-way intersections as

$$\sum_{1 \leq i < j \leq n} |S_i \cap S_j|.$$

Similarly, the sum of the sizes of the three-way intersections is

$$\sum_{1 \leq i < j < k \leq n} |S_i \cap S_j \cap S_k|.$$

These sums have alternating signs in the Inclusion-Exclusion formula, with the sum of the k -way intersections getting the sign $(-1)^{k-1}$. This finally leads to a symbolic version of the rule:

Rule (Inclusion-Exclusion).

$$\begin{aligned} \left| \bigcup_{i=1}^n S_i \right| &= \sum_{i=1}^n |S_i| \\ &\quad - \sum_{1 \leq i < j \leq n} |S_i \cap S_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |S_i \cap S_j \cap S_k| + \cdots \\ &\quad + (-1)^{n-1} \left| \bigcap_{i=1}^n S_i \right|. \end{aligned}$$

While it’s often handy express the rule in this way as a sum of sums, it is not necessary to group the terms by how many sets are in the intersections. So another way to state the rule is:

Rule (Inclusion-Exclusion-II).

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{\emptyset \neq I \subseteq \{1, \dots, n\}} (-1)^{|I|+1} \left| \bigcap_{i \in I} S_i \right|$$

A proof of these rules using just highschool algebra is given in Problem [15.47](#).

15.12.5 Computing Euler’s Function

As an example, let’s use Inclusion-Exclusion to derive an explicit formula (15.6) for Euler’s function, $\phi(n)$. By definition, $\phi(n)$ is the number of nonnegative integers less than a positive integer n that are relatively prime to n . But the set S of nonnegative integers less than n that are *not* relatively prime to n will be easier to count.

Suppose the prime factorization of n is $p_1^{e_1} \cdots p_m^{e_m}$ for distinct primes p_i . This means that the integers in S are precisely the nonnegative integers less than n that are divisible by at least one of the p_i ’s. Letting C_a be the set of nonnegative integers less than n that are divisible by a , we have

$$S = \bigcup_{i=1}^m C_{p_i}.$$

We’ll be able to find the size of this union using Inclusion-Exclusion because the intersections of the C_p ’s are easy to count. For example, $C_p \cap C_q \cap C_r$ is the set of nonnegative integers less than n that are divisible by each of p , q and r . But since the p, q, r are distinct primes, being divisible by each of them is the same as being divisible by their product. Now observe that if k is a positive divisor of n , then exactly n/k nonnegative integers less than n are divisible by k , namely, $0, k, 2k, \dots, ((n/k) - 1)k$. So exactly n/pqr nonnegative integers less than n are divisible by all three primes p, q, r . In other words,

$$|C_p \cap C_q \cap C_r| = \frac{n}{pqr}.$$

Reasoning this way about all the intersections among the C_p ’s and applying Inclusion-Exclusion, we get

$$\begin{aligned}
 |S| &= \left| \bigcup_{i=1}^m C_{p_i} \right| \\
 &= \sum_{i=1}^m |C_{p_i}| - \sum_{1 \leq i < j \leq m} |C_{p_i} \cap C_{p_j}| \\
 &\quad + \sum_{1 \leq i < j < k \leq m} |C_{p_i} \cap C_{p_j} \cap C_{p_k}| - \cdots + (-1)^{m-1} \left| \bigcap_{i=1}^m C_{p_i} \right| \\
 &= \sum_{i=1}^m \frac{n}{p_i} - \sum_{1 \leq i < j \leq m} \frac{n}{p_i p_j} \\
 &\quad + \sum_{1 \leq i < j < k \leq m} \frac{n}{p_i p_j p_k} - \cdots + (-1)^{m-1} \frac{n}{p_1 p_2 \cdots p_m} \\
 &= n \left(\sum_{i=1}^m \frac{1}{p_i} - \sum_{1 \leq i < j \leq m} \frac{1}{p_i p_j} + \sum_{1 \leq i < j < k \leq m} \frac{1}{p_i p_j p_k} - \cdots + (-1)^{m-1} \frac{1}{p_1 p_2 \cdots p_m} \right)
 \end{aligned}$$

But $\phi(n) = n - |S|$ by definition, so

$$\begin{aligned}
 \phi(n) &= n \left(1 - \sum_{i=1}^m \frac{1}{p_i} + \sum_{1 \leq i < j \leq m} \frac{1}{p_i p_j} - \sum_{1 \leq i < j < k \leq m} \frac{1}{p_i p_j p_k} + \cdots + (-1)^m \frac{1}{p_1 p_2 \cdots p_m} \right) \\
 &= n \prod_{i=1}^m \left(1 - \frac{1}{p_i} \right). \tag{15.6}
 \end{aligned}$$

Yikes! That was pretty hairy. Are you getting tired of all that nasty algebra? If so, then good news is on the way. In the next section, we will show you how to prove some heavy-duty formulas without using any algebra at all. Just a few words and you are done. No kidding.

15.13 Combinatorial Proofs

Suppose you have n different T-shirts, but only want to keep k . You could equally well select the k shirts you want to keep or select the complementary set of $n - k$ shirts you want to throw out. Thus, the number of ways to select k shirts from

among n must be equal to the number of ways to select $n - k$ shirts from among n . Therefore:

$$\binom{n}{k} = \binom{n}{n-k}.$$

This is easy to prove algebraically, since both sides are equal to:

$$\frac{n!}{k! (n-k)!}.$$

But we didn’t really have to resort to algebra; we just used counting principles. Hmmm...

15.13.1 Pascal’s Identity

Bob, famed Math for Computer Science Teaching Assistant, has decided to try out for the US Olympic boxing team. After all, he’s watched all of the *Rocky* movies and spent hours in front of a mirror sneering, “Yo, you wanna piece a’ *me*?!” Bob figures that n people (including himself) are competing for spots on the team and only k will be selected. As part of maneuvering for a spot on the team, he needs to work out how many different teams are possible. There are two cases to consider:

- Bob *is* selected for the team, and his $k - 1$ teammates are selected from among the other $n - 1$ competitors. The number of different teams that can be formed in this way is:

$$\binom{n-1}{k-1}.$$

- Bob is *not* selected for the team, and all k team members are selected from among the other $n - 1$ competitors. The number of teams that can be formed this way is:

$$\binom{n-1}{k}.$$

All teams of the first type contain Bob, and no team of the second type does; therefore, the two sets of teams are disjoint. Thus, by the Sum Rule, the total number of possible Olympic boxing teams is:

$$\binom{n-1}{k-1} + \binom{n-1}{k}.$$

Ted, equally-famed Teaching Assistant, thinks Bob isn't so tough and so he might as well also try out. He reasons that n people (including himself) are trying out for k spots. Thus, the number of ways to select the team is simply:

$$\binom{n}{k}.$$

Ted and Bob each correctly counted the number of possible boxing teams. Thus, their answers must be equal. So we know:

Lemma 15.13.1 (Pascal's Identity).

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}. \quad (15.7)$$

This is called *Pascal's Identity*. And we proved it *without any algebra!* Instead, we relied purely on counting techniques.

15.13.2 Giving a Combinatorial Proof

A *combinatorial proof* is an argument that establishes an algebraic fact by relying on counting principles. Many such proofs follow the same basic outline:

1. Define a set S .
2. Show that $|S| = n$ by counting one way.
3. Show that $|S| = m$ by counting another way.
4. Conclude that $n = m$.

In the preceding example, S was the set of all possible Olympic boxing teams. Bob computed

$$|S| = \binom{n-1}{k-1} + \binom{n-1}{k}$$

by counting one way, and Ted computed

$$|S| = \binom{n}{k}$$

by counting another way. Equating these two expressions gave Pascal's Identity.

Checking a Combinatorial Proof

Combinatorial proofs are based on counting the same thing in different ways. This is fine when you’ve become practiced at different counting methods, but when in doubt, you can fall back on bijections and sequence counting to check such proofs.

For example, let’s take a closer look at the combinatorial proof of Pascal’s Identity (15.7). In this case, the set S of things to be counted is the collection of all size- k subsets of integers in the interval $[1, n]$.

Now we’ve already counted S one way, via the Bookkeeper Rule, and found $|S| = \binom{n}{k}$. The other “way” corresponds to defining a bijection between S and the disjoint union of two sets A and B where,

$$\begin{aligned} A &::= \{(1, X) \mid X \subseteq [2, n] \text{ AND } |X| = k - 1\} \\ B &::= \{(0, Y) \mid Y \subseteq [2, n] \text{ AND } |Y| = k\}. \end{aligned}$$

Clearly A and B are disjoint since the pairs in the two sets have different first coordinates, so $|A \cup B| = |A| + |B|$. Also,

$$\begin{aligned} |A| &= \# \text{ specified sets } X = \binom{n-1}{k-1}, \\ |B| &= \# \text{ specified sets } Y = \binom{n-1}{k}. \end{aligned}$$

Now finding a bijection $f : (A \cup B) \rightarrow S$ will prove the identity (15.7). In particular, we can define

$$f(c) ::= \begin{cases} X \cup \{1\} & \text{if } c = (1, X), \\ Y & \text{if } c = (0, Y). \end{cases}$$

It should be obvious that f is a bijection.

15.13.3 A Colorful Combinatorial Proof

The set that gets counted in a combinatorial proof in different ways is usually defined in terms of simple sequences or sets rather than an elaborate story about Teaching Assistants. Here is another colorful example of a combinatorial argument.

Theorem 15.13.2.

$$\sum_{r=0}^n \binom{n}{r} \binom{2n}{n-r} = \binom{3n}{n}$$

Proof. We give a combinatorial proof. Let S be all n -card hands that can be dealt from a deck containing n different red cards and $2n$ different black cards. First, note that every $3n$ -element set has

$$|S| = \binom{3n}{n}$$

n -element subsets.

From another perspective, the number of hands with exactly r red cards is

$$\binom{n}{r} \binom{2n}{n-r}$$

since there are $\binom{n}{r}$ ways to choose the r red cards and $\binom{2n}{n-r}$ ways to choose the $n-r$ black cards. Since the number of red cards can be anywhere from 0 to n , the total number of n -card hands is:

$$|S| = \sum_{r=0}^n \binom{n}{r} \binom{2n}{n-r}.$$

Equating these two expressions for $|S|$ proves the theorem. ■

Finding a Combinatorial Proof

Combinatorial proofs are almost magical. Theorem 15.13.2 looks pretty scary, but we proved it without any algebraic manipulations at all. The key to constructing a combinatorial proof is choosing the set S properly, which can be tricky. Generally, the simpler side of the equation should provide some guidance. For example, the right side of Theorem 15.13.2 is $\binom{3n}{n}$, which suggests that it will be helpful to choose S to be all n -element subsets of some $3n$ -element set.

Problems for Section 15.2

Practice Problems

Problem 15.1.

Alice is thinking of a number between 1 and 1000.

What is the least number of yes/no questions you could ask her and be guaranteed to discover what it is? (Alice always answers truthfully.)

(a)

Problem 15.2.

In how many different ways is it possible to answer the next chapter’s practice problems if:

- the first problem has four *true/false* questions,
- the second problem requires choosing one of four alternatives, and
- the answer to the third problem is an integer ≥ 15 and ≤ 20 ?

Problem 15.3.

How many total functions are there from set A to set B if $|A| = 3$ and $|B| = 7$?

Problem 15.4.

Consider a 6 element set X with elements $\{x_1, x_2, x_3, x_4, x_5, x_6\}$.

- (a) How many subsets of X contain x_1 ?
- (b) How many subsets of X contain x_2 and x_3 but do not contain x_6 ?

Class Problems

Problem 15.5.

A license plate consists of either:

- 3 letters followed by 3 digits (standard plate)
- 5 letters (vanity plate)
- 2 characters—letters or numbers (big shot plate)

Let L be the set of all possible license plates.

- (a) Express L in terms of

$$\mathcal{A} = \{A, B, C, \dots, Z\}$$

$$\mathcal{D} = \{0, 1, 2, \dots, 9\}$$

using unions ($\cup$) and set products ($\times$).

- (b) Compute $|L|$, the number of different license plates, using the sum and product rules.

Problem 15.6. (a) How many of the billion numbers in the range from 1 to 10^9 contain the digit 1? (*Hint:* How many don’t?)

(b) There are 20 books arranged in a row on a shelf. Describe a bijection between ways of choosing 6 of these books so that no two adjacent books are selected and 15-bit strings with exactly 6 ones.

Problem 15.7.

(a) Let $\mathcal{S}_{n,k}$ be the possible nonnegative integer solutions to the inequality

$$x_1 + x_2 + \cdots + x_k \leq n. \quad (15.8)$$

That is

$$\mathcal{S}_{n,k} ::= \{(x_1, x_2, \dots, x_k) \in \mathbb{N}^k \mid (15.8) \text{ is true}\}.$$

Describe a bijection between $\mathcal{S}_{n,k}$ and the set of binary strings with n zeroes and k ones.

(b) Let $\mathcal{L}_{n,k}$ be the length k weakly increasing sequences of nonnegative integers $\leq n$. That is

$$\mathcal{L}_{n,k} ::= \{(y_1, y_2, \dots, y_k) \in \mathbb{N}^k \mid y_1 \leq y_2 \leq \cdots \leq y_k \leq n\}.$$

Describe a bijection between $\mathcal{L}_{n,k}$ and $\mathcal{S}_{n,k}$.

Problem 15.8.

An n -vertex *numbered tree* is a tree whose vertex set is $\{1, 2, \dots, n\}$ for some $n > 2$. We define the *code* of the numbered tree to be a sequence of $n - 2$ integers from 1 to n obtained by the following recursive process:⁶

If there are more than two vertices left, write down the *father* of the largest leaf, delete this *leaf*, and continue this process on the resulting smaller tree. If there are only two vertices left, then stop—the code is complete.

For example, the codes of a couple of numbered trees are shown in the Figure 15.7.

⁶The necessarily unique node adjacent to a leaf is called its *father*.

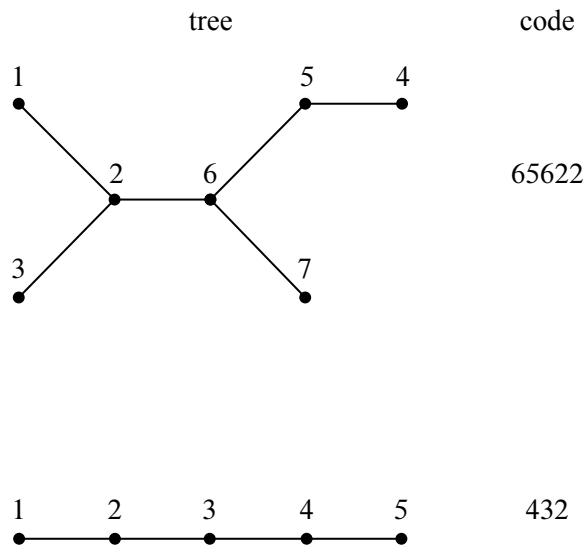


Figure 15.7

- (a) Describe a procedure for reconstructing a numbered tree from its code.
- (b) Conclude there is a bijection between the n -vertex numbered trees and $\{1, \dots, n\}^{n-2}$, and state how many n -vertex numbered trees there are.

Problem 15.9.

Let X and Y be finite sets.

- (a) How many binary relations from X to Y are there?
- (b) Define a bijection between the set $[X \rightarrow Y]$ of all total functions from X to Y and the set $Y^{|X|}$. (Recall Y^n is the cartesian product of Y with itself n times.) Based on that, what is $|[X \rightarrow Y]|$?
- (c) Using the previous part how many *functions*, not necessarily total, are there from X to Y ? How does the fraction of functions vs. total functions grow as the size of X grows? Is it $O(1)$, $O(|X|)$, $O(2^{|X|})$, ...?
- (d) Show a bijection between the powerset, $\mathcal{P}(X)$, and the set $[X \rightarrow \{0, 1\}]$ of 0-1-valued total functions on X .
- (e) Let $X ::= \{1, 2, \dots, n\}$. In this problem we count how many bijections there are from X to itself. Consider the set $B_{X,X}$ of all *bijections* from set X to set X .

Show a bijection from $B_{X,X}$ to the set of all permutations of X (as defined in the notes). Using that, count $B_{X,X}$.

Problems for Section 15.4

Homework Problems

Problem 15.10.

Here is a purely combinatorial proof of Fermat’s Little Theorem [8.6.4](#).

(a) Suppose there are beads available in a different colors for some integer $a > 1$, and let p be a prime number. How many different colored length p sequences of beads can be strung together? How many of them contain beads of at least two different colors?

(b) Make each string of p beads with at least two colors into a bracelet by tying the two ends of the string together. Two bracelets are the same if one can be rotated to yield the other. (Note, however, that you **cannot** “flip” a bracelet over or reflect it.) Show that for every bracelet, there are exactly p strings of beads that yield it.

Hint: Both the fact that p is prime and that the bracelet consists of at least two colors are needed for this to be true.

(c) Conclude that $p \mid (a^p - a)$ and from this conclude Fermat’s Little Theorem.

Problems for Section 15.5

Practice Problems

Problem 15.11.

8 students—Anna, Brian, Caine,...—are to be seated around a circular table in a circular room. Two seatings are regarded as defining the same *arrangement* if each student has the same student on his or her right in both seatings: it does not matter which way they face. We’ll be interested in counting how many arrangements there are of these 8 students, given some restrictions.

(a) As a start, how many different arrangements of these 8 students around the table are there without any restrictions?

(b) How many arrangements of these 8 students are there with Anna sitting next to Brian?

(c) How many arrangements are there with if Brian sitting next to both Anna AND Caine?

(d) How many arrangements are there with Brian sitting next to Anna OR Caine?

Problem 15.12.

How many different ways are there to select three dozen colored roses if red, yellow, pink, white, purple and orange roses are available?

Problem 15.13.

Suppose you want to select k out of n books on a shelf so that there are always at least 3 unselected books between selected books. Describe a bijection between book selection and bit-strings of length L containing exactly M 1's, so that counting the number of all such bit-strings gives us the number of book selections. Find L and M and briefly explain why it works.

(Assume n is large enough for this to be possible.)

Class Problems

Problem 15.14.

Your class tutorial has 12 students, who are supposed to break up into 4 groups of 3 students each. Your Teaching Assistant (TA) has observed that the students waste too much time trying to form balanced groups, so he decided to pre-assign students to groups and email the group assignments to his students.

(a) Your TA has a list of the 12 students in front of him, so he divides the list into consecutive groups of 3. For example, if the list is ABCDEFGHIJKL, the TA would define a sequence of four groups to be $(\{A, B, C\}, \{D, E, F\}, \{G, H, I\}, \{J, K, L\})$. This way of forming groups defines a mapping from a list of twelve students to a sequence of four groups. This is a k -to-1 mapping for what k ?

(b) A group assignment specifies which students are in the same group, but not any order in which the groups should be listed. If we map a sequence of 4 groups,

$$(\{A, B, C\}, \{D, E, F\}, \{G, H, I\}, \{J, K, L\}),$$

into a group assignment

$$\{\{A, B, C\}, \{D, E, F\}, \{G, H, I\}, \{J, K, L\}\},$$

this mapping is j -to-1 for what j ?

(c) How many group assignments are possible?

(d) In how many ways can $3n$ students be broken up into n groups of 3?

Problem 15.15.

A pizza house is having a promotional sale. Their commercial reads:

We offer 9 different toppings for your pizza! Buy 3 large pizzas at the regular price, and you can get each one with as many different toppings as you wish, absolutely free. That's 22,369,621 different ways to choose your pizzas!

The ad writer was a former Harvard student who had evaluated the formula $(2^9)^3/3!$ on his calculator and gotten close to 22,369,621. Unfortunately, $(2^9)^3/3!$ is obviously not an integer, so clearly something is wrong. What mistaken reasoning might have led the ad writer to this formula? Explain how to fix the mistake and get a correct formula.

Problem 15.16.

Answer the following questions using the Generalized Product Rule.

(a) Next week, I'm going to get really fit! On day 1, I'll exercise for 5 minutes. On each subsequent day, I'll exercise 0, 1, 2, or 3 minutes more than the previous day. For example, the number of minutes that I exercise on the seven days of next week might be 5, 6, 9, 9, 9, 11, 12. How many such sequences are possible?

(b) An r -permutation of a set is a sequence of r distinct elements of that set. For example, here are all the 2-permutations of $\{a, b, c, d\}$:

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

How many r -permutations of an n -element set are there? Express your answer using factorial notation.

(c) How many $n \times n$ matrices are there with *distinct* entries drawn from $\{1, \dots, p\}$, where $p \geq n^2$?

Problem 15.17. (a) There are 30 books arranged in a row on a shelf. In how many ways can eight of these books be selected so that there are at least two unselected books between any two selected books?

(b) How many nonnegative integer solutions are there for the following equality?

$$x_1 + x_2 + \cdots + x_m = k. \quad (15.9)$$

(c) How many nonnegative integer solutions are there for the following inequality?

$$x_1 + x_2 + \cdots + x_m \leq k. \quad (15.10)$$

(d) How many length- m weakly increasing sequences of nonnegative integers $\leq k$ are there?

Homework Problems

Problem 15.18.

This problem is about binary relations on the set of integers in the interval $[1, n]$, and digraphs and simple graphs whose vertex set is $[1, n]$.

- (a) How many digraphs are there?
- (b) How many simple graphs are there?
- (c) How many asymmetric binary relations are there?
- (d) How many path-total strict partial orders are there?

Problem 15.19.

Answer the following questions with a number or a simple formula involving factorials and binomial coefficients. Briefly explain your answers.

(a) How many ways are there to order the 26 letters of the alphabet so that no two of the vowels a, e, i, o, u appear consecutively and the last letter in the ordering is not a vowel?

Hint: Every vowel appears to the left of a consonant.

(b) How many ways are there to order the 26 letters of the alphabet so that there are *at least two* consonants immediately following each vowel?

(c) In how many different ways can $2n$ students be paired up?

(d) Two n -digit sequences of digits $0, 1, \dots, 9$ are said to be of the *same type* if the digits of one are a permutation of the digits of the other. For $n = 8$, for example, the sequences 03088929 and 00238899 are the same type. How many types of n -digit integers are there?

Problem 15.20.

In a standard 52-card deck, each card has one of thirteen *ranks* in the set, R , and one of four *suits* in the set, S , where

$$R ::= \{A, 2, \dots, 10, J, Q, K\},$$

$$S ::= \{\clubsuit, \diamondsuit, \heartsuit, \spadesuit\}.$$

A 5-card *hand* is a set of five distinct cards from the deck.

For each part describe a bijection between a set that can easily be counted using the Product and Sum Rules of Ch. 15.1, and the set of hands matching the specification. *Give bijections, not numerical answers.*

For instance, consider the set of 5-card hands containing all 4 suits. Each such hand must have 2 cards of one suit. We can describe a bijection between such hands and the set $S \times R_2 \times R^3$ where R_2 is the set of two-element subsets of R . Namely, an element

$$(s, \{r_1, r_2\}, (r_3, r_4, r_5)) \in S \times R_2 \times R^3$$

indicates

1. the repeated suit, $s \in S$,
2. the set, $\{r_1, r_2\} \in R_2$, of ranks of the cards of suit, s , and
3. the ranks (r_3, r_4, r_5) of remaining three cards, listed in increasing suit order where $\clubsuit < \diamondsuit < \heartsuit < \spadesuit$.

For example,

$$(\clubsuit, \{10, A\}, (J, J, 2)) \longleftrightarrow \{A\clubsuit, 10\clubsuit, J\diamondsuit, J\heartsuit, 2\spadesuit\}.$$

- (a) A single pair of the same rank (no 3-of-a-kind, 4-of-a-kind, or second pair).
- (b) Three or more aces.

Problem 15.21.

Suppose you have seven dice —each a different color of the rainbow; otherwise the dice are standard, with faces numbered 1 to 6. A *roll* is a sequence specifying a value for each die in rainbow (ROYGBIV) order. For example, one roll is $(3, 1, 6, 1, 4, 5, 2)$ indicating that the red die showed a 3, the orange die showed 1, the yellow 6,...

For the problems below, describe a bijection between the specified set of rolls and another set that is easily counted using the Product, Generalized Product, and

similar rules. Then write a simple arithmetic formula, possibly involving factorials and binomial coefficients, for the size of the set of rolls. You do not need to prove that the correspondence between sets you describe is a bijection, and you do not need to simplify the expression you come up with.

For example, let A be the set of rolls where 4 dice come up showing the same number, and the other 3 dice also come up the same, but with a different number. Let R be the set of seven rainbow colors and $S ::= [1, 6]$ be the set of dice values.

Define $B ::= P_{S,2} \times R_3$, where $P_{S,2}$ is the set of 2-permutations of S and R_3 is the set of size-3 subsets of R . Then define a bijection from A to B by mapping a roll in A to the sequence in B whose first element is an ordered pair consisting of the number that came up three times followed by the number that came up four times, and whose second element is the set of colors of the three matching dice.

For example, the roll

$$(4, 4, 2, 2, 4, 2, 4) \in A$$

maps to

$$((2, 4), \{\text{yellow, green, indigo}\}) \in B.$$

Now by the Bijection rule $|A| = |B|$, and by the Generalized Product and Subset rules,

$$|B| = 6 \cdot 5 \cdot \binom{7}{3}.$$

(a) For how many rolls do *exactly* two dice have the value 6 and the remaining five dice all have different values?

Example: $(6, 2, 6, 1, 3, 4, 5)$ is a roll of this type, but $(1, 1, 2, 6, 3, 4, 5)$ and $(6, 6, 1, 2, 4, 3, 4)$ are not.

(b) For how many rolls do two dice have the same value and the remaining five dice all have different values?

Example: $(4, 2, 4, 1, 3, 6, 5)$ is a roll of this type, but $(1, 1, 2, 6, 1, 4, 5)$ and $(6, 6, 1, 2, 4, 3, 4)$ are not.

(c) For how many rolls do two dice have one value, two different dice have a second value, and the remaining three dice a third value?

Example: $(6, 1, 2, 1, 2, 6, 6)$ is a roll of this type, but $(4, 4, 4, 4, 1, 3, 5)$ and $(5, 5, 5, 6, 6, 1, 2)$ are not.

Exam Problems

Problem 15.22.

Suppose that two identical 52-card decks are mixed together. Write a simple formula for the number of distinct permutations of the 104 cards.

Problems for Section 15.6

Practice Problems

Problem 15.23.

How many different permutations are there of the sequence of letters in “MISSISSIPPI”?

Exam Problems

Problem 15.24.

There is a robot that steps between integer positions in 3-dimensional space. Each step of the robot increments one coordinate and leaves the other two unchanged.

- (a) How many paths can the robot follow going from the origin $(0, 0, 0)$ to $(3, 4, 5)$?
- (b) How many paths can the robot follow going from the origin (i, j, k) to (m, n, p) ?

Problems for Section 15.7

Practice Problems

Problem 15.25.

Find the coefficients of $x^{10}y^5$ in $(19x + 4y)^{15}$

Problem 15.26.

Find the coefficient of x^4 in the following expressions.

- (a) $(x + 1)^9$?
- (b) $(3x + 2)^6$?

Class Problems

Problem 15.27.

Find the coefficients of

- (a) x^5 in $(1 + x)^{11}$
- (b) x^8y^9 in $(3x + 2y)^{17}$

(c) $a^6 b^6$ in $(a^2 + b^3)^5$

Problem 15.28. (a) Use the Multinomial Theorem 15.7.2 to prove that

$$(x_1 + x_2 + \cdots + x_n)^p \equiv x_1^p + x_2^p + \cdots + x_n^p \pmod{p} \quad (15.11)$$

for all primes p . (Do not prove it using Fermat’s “little” Theorem. The point of this problem is to offer an independent proof of Fermat’s theorem.)

Hint: Explain why $\binom{p}{k_1, k_2, \dots, k_n}$ is divisible by p if all the k_i ’s are positive integers less than p .

(b) Explain how (15.11) immediately proves Fermat’s Little Theorem 8.6.4: $n^{p-1} \equiv 1 \pmod{p}$ when n is not a multiple of p .

Homework Problems

Problem 15.29.

The *degree sequence* of a simple graph is the weakly decreasing sequence of degrees of its vertices. For example, the degree sequence for the 5-vertex numbered tree pictured in the Figure 15.7 in Problem 15.8 is $(2, 2, 2, 1, 1)$ and for the 7-vertex tree it is $(3, 3, 2, 1, 1, 1, 1)$.

We’re interested in counting how many numbered trees there are with a given degree sequence. We’ll do this using the bijection defined in Problem 15.8 between n -vertex numbered trees and length $n - 2$ code words whose characters are integers between 1 and n .

The *occurrence number* for a character in a word is the number of times that the character occurs in the word. For example, in the word 65622, the occurrence number for 6 is two, and the occurrence number for 5 is one. The *occurrence sequence* of a word is the weakly decreasing sequence of occurrence numbers of characters in the word. The occurrence sequence for this word is $(2, 2, 1)$ because it has two occurrences of each of the characters 6 and 2, and one occurrence of 5.

(a) There is simple relationship between the degree sequence of an n -vertex numbered tree and the occurrence sequence of its code. Describe this relationship and explain why it holds. Conclude that counting n -vertex numbered trees with a given degree sequence is the same as counting the number of length $n - 2$ code words with a given occurrence sequence.

Hint: How many times does a vertex of degree, d , occur in the code?

For simplicity, let’s focus on counting 9-vertex numbered trees with a given degree sequence. By part (a), this is the same as counting the number of length 7 code words with a given occurrence sequence.

Any length 7 code word has a *pattern*, which is another length 7 word over the alphabet a, b, c, d, e, f, g that has the same occurrence sequence.

(b) How many length 7 patterns are there with three occurrences of a , two occurrences of b , and one occurrence of c and d ?

(c) How many ways are there to assign occurrence numbers to integers $1, 2, \dots, 9$ so that a code word with those occurrence numbers would have the occurrence sequence $3, 2, 1, 1, 0, 0, 0, 0, 0$?

In general, to find the pattern of a code word, list its characters in decreasing order by *number of occurrences*, and list characters with the same number of occurrences in decreasing order. Then replace successive characters in the list by successive letters a, b, c, d, e, f, g . The code word 2468751, for example, has the pattern fecabdg, which is obtained by replacing its characters $8, 7, 6, 5, 4, 2, 1$ by a, b, c, d, e, f, g , respectively. The code word 2449249 has pattern caabcab, which is obtained by replacing its characters $4, 9, 2$ by a, b, c , respectively.

(d) What length 7 code word has three occurrences of 7, two occurrences of 8, one occurrence each of 2 and 9, and pattern abacbad?

(e) Explain why the number of 9-vertex numbered trees with degree sequence $(4, 3, 2, 2, 1, 1, 1, 1, 1)$ is the product of the answers to parts (b) and (c).

Problems for Section 15.8

Class Problems

Problem 15.30.

The Tao of BOOKKEEPER: we seek enlightenment through contemplation of the word *BOOKKEEPER*.

(a) In how many ways can you arrange the letters in the word *POKE*?

(b) In how many ways can you arrange the letters in the word BO_1O_2K ? Observe that we have subscripted the O 's to make them distinct symbols.

(c) Suppose we map arrangements of the letters in BO_1O_2K to arrangements of the letters in *BOOK* by erasing the subscripts. Indicate with arrows how the arrangements on the left are mapped to the arrangements on the right.

O_2BO_1K	
KO_2BO_1	
O_1BO_2K	$BOOK$
KO_1BO_2	$OBOK$
BO_1O_2K	$KOBO$
BO_2O_1K	$\dots$
$\dots$	

(d) What kind of mapping is this, young grasshopper?

(e) In light of the Division Rule, how many arrangements are there of $BOOK$?

(f) Very good, young master! How many arrangements are there of the letters in $KE_1E_2PE_3R$?

(g) Suppose we map each arrangement of $KE_1E_2PE_3R$ to an arrangement of $KEEPER$ by erasing subscripts. List all the different arrangements of $KE_1E_2PE_3R$ that are mapped to $REEPEK$ in this way.

(h) What kind of mapping is this?

(i) So how many arrangements are there of the letters in $KEEPER$?

Now you are ready to face the BOOKKEEPER!

(j) How many arrangements of $BO_1O_2K_1K_2E_1E_2PE_3R$ are there?

(k) How many arrangements of $BOOK_1K_2E_1E_2PE_3R$ are there?

(l) How many arrangements of $BOOKKE_1E_2PE_3R$ are there?

(m) How many arrangements of $BOOKKEEPER$ are there?

*Remember well what you have learned: subscripts on, subscripts off.
This is the Tao of Bookkeeper.*

(n) How many arrangements of $VOODOODOLL$ are there?

(o) How many length 52 sequences of digits contain exactly 17 two's, 23 fives, and 12 nines?

Problems for Section 15.9

Practice Problems

Problem 15.31.

Indicate how many 5-card hands there are of each of the following kinds.

(a) A **Sequence** is a hand consisting of five consecutive cards of any suit, such as

$$5\heartsuit - 6\heartsuit - 7\spadesuit - 8\diamondsuit - 9\clubsuit.$$

Note that an Ace may either be high (as in 10-J-Q-K-A), or low (as in A-2-3-4-5), but can't go “around the corner” (that is, Q-K-A-2-3 is *not* a sequence).

How many different **Sequence** hands are possible?

(b) A **Matching Suit** is a hand consisting of cards that are all of the same suit in any order.

How many different **Matching Suit** hands are possible?

(c) A **Straight Flush** is a hand that is both a *Sequence* and a *Matching Suit*.

How many different **Straight Flush** hands are possible?

(d) A **Straight** is a hand that is a *Sequence* but not a *Matching Suit*.

How many possible **Straights** are there?

(e) A **Flush** is a hand that is a *Matching Suit* but not a *Sequence*.

How many possible **Flushes** are there?

Class Problems

Problem 15.32.

Solve the following counting problems. Define an appropriate mapping (bijective or k -to-1) between a set whose size you know and the set in question.

(a) An independent living group is hosting nine new candidates for membership. Each candidate must be assigned a task: 1 must wash pots, 2 must clean the kitchen, 3 must clean the bathrooms, 1 must clean the common area, and 2 must serve dinner. Write a multinomial coefficient for the number of ways this can be done.

(b) How many nonnegative integers less than 1,000,000 have exactly one digit equal to 9 and have a sum of digits equal to 17?

Exam Problems

Problem 15.33.

Here are the solutions to the next 10 short answer questions, in no particular order. Enter the solution number after each question.

1. $\frac{n!}{(n-m)!}$
2. $\binom{n+m}{m}$
3. $(n-m)!$
4. m^n
5. $\binom{n-1+m}{m}$
6. $\binom{n-1+m}{n}$
7. 2^{mn}
8. n^m

(a) How many solutions over the nonnegative integers are there to the inequality

$$x_1 + x_2 + \cdots + x_n \leq m?$$

- (b) How many length m words can be formed from an n -letter alphabet, if no letter is used more than once?
- (c) How many length m words can be formed from an n -letter alphabet, if letters can be reused?
- (d) How many binary relations are there from set A to set B when $|A| = m$ and $|B| = n$?
- (e) How many total injective functions are there from set A to set B , where $|A| = m$ and $|B| = n \geq m$?
- (f) How many ways are there to place a total of m distinguishable balls into n distinguishable urns, with some urns possibly empty or with several balls?
- (g) How many ways are there to place a total of m indistinguishable balls into n distinguishable urns, with some urns possibly empty or with several balls?
- (h) How many ways are there to put a total of m distinguishable balls into n distinguishable urns with at most one ball in each urn?

Problem 15.34. (a) How many solutions over the *positive* integers are there to the inequality:

$$x_1 + x_2 + \dots + x_{10} \leq 100$$

(b) In how many ways can Mr. and Mrs. Grumperson distribute 13 identical pieces of coal to their three children for Christmas so that each child gets at least one piece?

Problems for Section 15.10

Practice Problems

Problem 15.35.

Below is a list of properties that a group of people might possess.

For each property, either give the minimum number of people that must be in a group to ensure that the property holds, or else indicate that the property need not hold even for arbitrarily large groups of people.

(Assume that every year has exactly 365 days; ignore leap years.)

- (a) At least 2 people were born on the same day of the year (ignore year of birth).
- (b) At least 2 people were born on January 1.
- (c) At least 3 people were born on the same day of the week.
- (d) At least 4 people were born in the same month.
- (e) At least 2 people were born exactly one week apart.

Class Problems

Problem 15.36.

Solve the following problems using the pigeonhole principle. For each problem, try to identify the *pigeons*, the *pigeonholes*, and a *rule* assigning each pigeon to a pigeonhole.

- (a) In a certain Institute of Technology, Every ID number starts with a 9. Suppose that each of the 75 students in a class sums the nine digits of their ID number. Explain why two people must arrive at the same sum.
- (b) In every set of 100 integers, there exist two whose difference is a multiple of 37.
- (c) For any five points inside a unit square (not on the boundary), there are two points at distance *less than* $1/\sqrt{2}$.

(d) Show that if $n + 1$ numbers are selected from $\{1, 2, 3, \dots, 2n\}$, two must be consecutive, that is, equal to k and $k + 1$ for some k .

Homework Problems

Problem 15.37.

Pigeon Huntin’

(a) Show that any odd integer x in the range $10^9 < x < 2 \cdot 10^9$ containing all ten digits $0, 1, \dots, 9$ must have consecutive even digits. *Hint:* What can you conclude about the parities of the first and last digit?

(b) Show that there are 2 vertices of equal degree in any finite undirected graph with $n \geq 2$ vertices. *Hint:* Cases conditioned upon the existence of a degree zero vertex.

Problem 15.38.

Show that for any set of 201 positive integers less than 300, there must be two whose quotient is a power of three (with no remainder).

Problem 15.39. (a) Color each point in the plane with integer coordinates either red, white or blue. Let R be a 4×82 rectangular grid of these points. Explain why at least two of the 82 rows in R must have the same sequence colors.

(b) Conclude that R contains four points with the same color that form the corners of a rectangle.

(c) Generalize the above argument to a coloring using the rainbow colors Red, Orange, Yellow, Green, Blue, Indigo, Violet as well as White and Black.

Problems for Section 15.11

Class Problems

Problem 15.40. (a) Show that the Magician could not pull off the trick with a deck larger than 124 cards.

Hint: Compare the number of 5-card hands in an n -card deck with the number of 4-card sequences.

(b) Show that, in principle, the Magician could pull off the Card Trick with a deck of 124 cards.

Hint: Hall’s Theorem and degree-constrained (11.5.5) graphs.

Problem 15.41.

The Magician can determine the 5th card in a poker hand when his Assisant reveals the other 4 cards. Describe a similar method for determining 2 hidden cards in a hand of 9 cards when your Assisant reveals the other 7 cards.

Homework Problems

Problem 15.42.

Section 15.11.3 explained why it is not possible to perform a four-card variant of the hidden-card magic trick with one card hidden. But the Magician and her Assistant are determined to find a way to make a trick like this work. They decide to change the rules slightly: instead of the Assistant lining up the three unhidden cards for the Magician to see, he will line up all four cards with one card face down and the other three visible. We’ll call this the *face-down four-card trick*.

For example, suppose the audience members had selected the cards $9\heartsuit$, $10\diamondsuit$, $A\clubsuit$, $5\clubsuit$. Then the Assistant could choose to arrange the 4 cards in any order so long as one is face down and the others are visible. Two possibilities are:

$A\clubsuit$	?	$10\diamondsuit$	$5\clubsuit$
?	$5\clubsuit$	$9\heartsuit$	$10\diamondsuit$

(a) Explain how to model this the face-down four-card trick as a matching problem, and show that there must be a bipartite matching which theoretically will allow the Magician and Assistant to perform the trick.

(b) There is actually a simple way to perform the face-down four-card trick.⁷

⁷This elegant method was devised in Fall '09 by student Katie E Everett.

Case 1. *there are two cards with the same suit:* Say there are two ♠ cards. The Assistant proceeds as in the original card trick: he puts one of the ♠ cards *face up as the first card*. He will place the second ♠ card *face down*. He then uses a permutation of the face down card and the remaining two face up cards to code the offset of the face down card from the first card.

Case 2. *all four cards have different suits:* Assign numbers 0, 1, 2, 3 to the four suits in some agreed upon way. The Assistant computes, s , the sum modulo 4 of the ranks of the four cards, and chooses the card with suit s to be placed *face down as the first card*. He then uses a permutation of the remaining three face-up cards to code the rank of the face down card.

Explain how in Case 2. the Magician can determine the face down card from the cards the Assistant shows her.

(c) Explain how any method for performing the face-down four-card trick can be adapted to perform the regular (5-card hand, show 4 cards) with a 52-card deck consisting of the usual 52 cards along with a 53rd card call the *joker*.

Problems for Section 15.12

Practice Problems

Problem 15.43.

Let A_1, A_2, A_3 be sets with $|A_1| = 100$, $|A_2| = 1,000$, and $|A_3| = 10,000$.

Determine $|A_1 \cup A_2 \cup A_3|$ in each of the following cases:

- (a) $A_1 \subset A_2 \subset A_3$.
- (b) The sets are pairwise disjoint.
- (c) For any two of the sets, there is exactly one element in both.
- (d) There are two elements common to each pair of sets and one element in all three sets.

Problem 15.44.

The working days in the next year can be numbered 1, 2, 3, ..., 300. I'd like to avoid as many as possible.

- On even-numbered days, I'll say I'm sick.
- On days that are a multiple of 3, I'll say I was stuck in traffic.

- On days that are a multiple of 5, I'll refuse to come out from under the blankets.

In total, how many work days will I *avoid* in the coming year?

Class Problems

Problem 15.45.

A certain company wants to have security for their computer systems. So they have given everyone a password. A length 10 word containing each of the characters:

a, d, e, f, i, l, o, p, r, s,

is called a *cword*. A password will be a cword which does not contain any of the subwords "fails", "failed", or "drop".

For example, the following two words are passwords: adefiloprs, srpolifeda,
but the following three cwords are not: **adrop**eflis, **failedrops**, **drope**fails.

- How many cwords contain the subword "drop"?
- How many cwords contain both "drop" and "fails"?
- Use the Inclusion-Exclusion Principle to find a simple arithmetic formula involving factorials for the number of passwords.

Problem 15.46.

We want to count step-by-step paths between points in the plane with integer coordinates. Only two kinds of step are allowed: a right-step which increments the x coordinate, and an up-step which increments the y coordinate.

- How many paths are there from $(0, 0)$ to $(20, 30)$?
- How many paths are there from $(0, 0)$ to $(20, 30)$ that go through the point $(10, 10)$?
- How many paths are there from $(0, 0)$ to $(20, 30)$ that do *not* go through either of the points $(10, 10)$ and $(15, 20)$?

Hint: Let P be the set of paths from $(0, 0)$ to $(20, 30)$, N_1 be the paths in P that go through $(10, 10)$ and N_2 be the paths in P that go through $(15, 20)$.

Problem 15.47.

Let’s develop a proof of the Inclusion-Exclusion formula using high school algebra.

(a) Most high school students will get freaked by the following formula, even though they actually know the rule it expresses. How would you explain it to them?

$$\prod_{i=1}^n (1 - x_i) = \sum_{I \subseteq \{1, \dots, n\}} (-1)^{|I|} \prod_{j \in I} x_j. \quad (15.12)$$

Hint: Show them an example.

For any set, S , let M_S be the *membership* function of S :

$$M_S(x) = \begin{cases} 1 & \text{if } x \in S, \\ 0 & \text{if } x \notin S. \end{cases}$$

Let $S_1, \dots, S_n$ be a sequence of finite sets, and abbreviate M_{S_i} as M_i . Let the domain of discourse, D , be the union of the S_i ’s. That is, we let

$$D ::= \bigcup_{i=1}^n S_i,$$

and take complements with respect to D , that is,

$$\overline{T} ::= D - T,$$

for $T \subseteq D$.

(b) Verify that for $T \subseteq D$ and $I \subseteq \{1, \dots, n\}$,

$$M_{\overline{T}} = 1 - M_T, \quad (15.13)$$

$$M_{(\cap_{i \in I} S_i)} = \prod_{i \in I} M_{S_i}, \quad (15.14)$$

$$M_{(\cup_{i \in I} S_i)} = 1 - \prod_{i \in I} (1 - M_i). \quad (15.15)$$

(Note that (15.14) holds when I is empty because, by convention, an empty product equals 1, and an empty intersection equals the domain of discourse, D .)

(c) Use (15.12) and (15.15) to prove

$$M_D = \sum_{\emptyset \neq I \subseteq \{1, \dots, n\}} (-1)^{|I|+1} \prod_{j \in I} M_j. \quad (15.16)$$

(d) Prove that

$$|T| = \sum_{u \in D} M_T(u). \quad (15.17)$$

(e) Now use the previous parts to prove

$$|D| = \sum_{\emptyset \neq I \subseteq \{1, \dots, n\}} (-1)^{|I|+1} \left| \bigcap_{i \in I} S_i \right| \quad (15.18)$$

(f) Finally, explain why (15.18) immediately implies the usual form of the Inclusion-Exclusion Principle:

$$|D| = \sum_{i=1}^n (-1)^{i+1} \sum_{\substack{I \subseteq \{1, \dots, n\} \\ |I|=i}} \left| \bigcap_{j \in I} S_j \right|. \quad (15.19)$$

Homework Problems

Problem 15.48.

How many paths are there from point $(0, 0)$ to $(50, 50)$ if every step increments one coordinate and leaves the other unchanged? How many are there when there are impassable boulders sitting at points $(10, 11)$ and $(21, 20)$? (You do not have to calculate the number explicitly; your answer may be an expression involving binomial coefficients.)

Hint: Count the number of paths going through $(10, 11)$, the number through $(21, 20)$, and use Inclusion-Exclusion.

Problem 15.49.

A *derangement* is a permutation $(x_1, x_2, \dots, x_n)$ of the set $\{1, 2, \dots, n\}$ such that $x_i \neq i$ for all i . For example, $(2, 3, 4, 5, 1)$ is a derangement, but $(2, 1, 3, 5, 4)$ is not because 3 appears in the third position. The objective of this problem is to count derangements.

It turns out to be easier to start by counting the permutations that are *not* derangements. Let S_i be the set of all permutations $(x_1, x_2, \dots, x_n)$ that are not derangements because $x_i = i$. So the set of non-derangements is

$$\bigcup_{i=1}^n S_i.$$

- (a) What is $|S_i|$?
- (b) What is $|S_i \cap S_j|$ where $i \neq j$?
- (c) What is $|S_{i_1} \cap S_{i_2} \cap \cdots \cap S_{i_k}|$ where $i_1, i_2, \dots, i_k$ are all distinct?
- (d) Use the inclusion-exclusion formula to express the number of non-derangements in terms of sizes of possible intersections of the sets $S_1, \dots, S_n$.
- (e) How many terms in the expression in part (d) have the form $|S_{i_1} \cap S_{i_2} \cap \cdots \cap S_{i_k}|$?
- (f) Combine your answers to the preceding parts to prove the number of non-derangements is:

$$n! \left(\frac{1}{1!} - \frac{1}{2!} + \frac{1}{3!} - \cdots \pm \frac{1}{n!} \right).$$

Conclude that the number of derangements is

$$n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots \pm \frac{1}{n!} \right).$$

- (g) As n goes to infinity, the number of derangements approaches a constant fraction of all permutations. What is that constant? *Hint:*

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

Problem 15.50.

How many of the numbers $2, \dots, n$ are prime? The Inclusion-Exclusion Principle offers a useful way to calculate the answer when n is large. Actually, we will use Inclusion-Exclusion to count the number of *composite* (nonprime) integers from 2 to n . Subtracting this from $n - 1$ gives the number of primes.

Let C_n be the set of composites from 2 to n , and let A_m be the set of numbers in the range $m + 1, \dots, n$ that are divisible by m . Notice that by definition, $A_m = \emptyset$ for $m \geq n$. So

$$C_n = \bigcup_{i=2}^{n-1} A_i. \quad (15.20)$$

- (a) Verify that if $m \mid k$, then $A_m \supseteq A_k$.
- (b) Explain why the right hand side of (15.20) equals

$$\bigcup_{\text{primes } p \leq \sqrt{n}} A_p. \quad (15.21)$$

- (c) Explain why $|A_m| = \lfloor n/m \rfloor - 1$ for $m \geq 2$.
- (d) Consider any two relatively prime numbers $p, q \leq n$. What is the one number in $(A_p \cap A_q) - A_{p \cdot q}$?
- (e) Let $\mathcal{P}$ be a finite set of at least two primes. Give a simple formula for

$$\left| \bigcap_{p \in \mathcal{P}} A_p \right|.$$

- (f) Use the Inclusion-Exclusion principle to obtain a formula for $|C_{150}|$ in terms the sizes of intersections among the sets $A_2, A_3, A_5, A_7, A_{11}$. (Omit the intersections that are empty; for example, any intersection of more than three of these sets must be empty.)
- (g) Use this formula to find the number of primes up to 150.

Exam Problems

Problem 15.51. (a) How many length n binary strings are there in which 011 occurs starting at the 4th position?

(b) Let A_i be the set of length n binary strings in which 011 occurs starting at the i th position. (So A_i is empty for $i > n - 2$.) For $i < j$, the intersections $A_i \cap A_j$ that are nonempty are all the same size. What is $|A_i \cap A_j|$ in this case?

(c) Let t be the number of intersections $A_i \cap A_j$ that are nonempty, where $i < j$. Express t as a binomial coefficient.

(d) How many length 9 binary strings are there that contain the substring 011? You should express your answer as an integer or as a simple expression which may include the constant, t , of part (c).

Hint: Inclusion-exclusion for $|\bigcup_1^7 A_i|$.

Problem 15.52.

There are 10 students $A, B, \dots, J$ who will be lined up left to right according to the some rules below.

Rule I: Student A must not be rightmost.

Rule II: Student B must be adjacent to C (directly to the left or right of C).

Rule III: Student D is always second.

You may answer the following questions with a numerical formula that may involve factorials.

(a) How many possible lineups are there that satisfy all three of these rules?

(b) How many possible lineups are there that satisfy at least one of these rules?

Problems for Section 15.13

Class Problems

Problem 15.53.

According to the Multinomial theorem, $(w + x + y + z)^n$ can be expressed as a sum of terms of the form

$$\binom{n}{r_1, r_2, r_3, r_4} w^{r_1} x^{r_2} y^{r_3} z^{r_4}.$$

(a) How many terms are there in the sum?

(b) The sum of these multinomial coefficients has an easily expressed value. What is it?

$$\sum_{\substack{r_1 + r_2 + r_3 + r_4 = n, \\ r_i \in \mathbb{N}}} \binom{n}{r_1, r_2, r_3, r_4} = ? \quad (15.22)$$

Hint: How many terms are there when $(w + x + y + z)^n$ is expressed as a sum of monomials in w, x, y, z *before* terms with like powers of these variables are collected together under a single coefficient?

Problem 15.54.

(a) Give a combinatorial proof of the following identity by letting S be the set of all length- n sequences of letters a, b and a single c and counting $|S|$ in two different ways.

$$n2^{n-1} = \sum_{k=1}^n k \binom{n}{k} \quad (15.23)$$

(b) Now prove (15.23) algebraically by applying the Binomial Theorem to $(1+x)^n$ and taking derivatives.

Problem 15.55.

What do the following expressions equal? Give both algebraic and combinatorial proofs for your answers.

(a)

$$\sum_{i=0}^n \binom{n}{i}$$

(b)

$$\sum_{i=0}^n \binom{n}{i} (-1)^i$$

Hint: Consider the bit strings with an even number of ones and an odd number of ones.

Homework Problems

Problem 15.56.

Prove the following identity by algebraic manipulation and by giving a combinatorial argument:

$$\binom{n}{r} \binom{r}{k} = \binom{n}{k} \binom{n-k}{r-k}$$

Problem 15.57. (a) Find a combinatorial (*not* algebraic) proof that

$$\sum_{i=0}^n \binom{n}{i} = 2^n.$$

(b) Below is a combinatorial proof of an equation. What is the equation?

Proof. Stinky Peterson owns n newts, t toads, and s slugs. Conveniently, he lives in a dorm with $n + t + s$ other students. (The students are distinguishable, but creatures of the same variety are not distinguishable.) Stinky wants to put one creature in each neighbor’s bed. Let W be the set of all ways in which this can be done.

On one hand, he could first determine who gets the slugs. Then, he could decide who among his remaining neighbors has earned a toad. Therefore, $|W|$ is equal to the expression on the left.

On the other hand, Stinky could first decide which people deserve newts and slugs and then, from among those, determine who truly merits a newt. This shows that $|W|$ is equal to the expression on the right.

Since both expressions are equal to $|W|$, they must be equal to each other. ■

(Combinatorial proofs are real proofs. They are not only rigorous, but also convey an intuitive understanding that a purely algebraic argument might not reveal. However, combinatorial proofs are usually less colorful than this one.)

Problem 15.58.

According to the Multinomial Theorem 15.7.2, $(x_1 + x_2 + \cdots + x_k)^n$ can be expressed as a sum of terms of the form

$$\binom{n}{r_1, r_2, \dots, r_k} x_1^{r_1} x_2^{r_2} \cdots x_k^{r_k}.$$

(a) How many terms are there in the sum?

(b) The sum of these multinomial coefficients has an easily expressed value:

$$\sum_{\substack{r_1 + r_2 + \cdots + r_k = n, \\ r_i \in \mathbb{N}}} \binom{n}{r_1, r_2, \dots, r_k} = k^n \quad (15.24)$$

Give a combinatorial proof of this identity.

Hint: How many terms are there when $(x_1 + x_2 + \cdots + x_k)^n$ is expressed as a sum of monomials in x_i before terms with like powers of these variables are collected together under a single coefficient?

Problem 15.59.

You want to choose a team of m people for your startup company from a pool of n applicants, and from these m people you want to choose k to be the team managers. You took a Math for Computer Science subject, so you know you can do this in

$$\binom{n}{m} \binom{m}{k}$$

ways. But your CFO, who went to Harvard Business School, comes up with the formula

$$\binom{n}{k} \binom{n-k}{m-k}.$$

Before doing the reasonable thing—dump on your CFO or Harvard Business School—you decide to check his answer against yours.

- (a) Give a *combinatorial proof* that your CFO’s formula agrees with yours.
- (b) Verify this combinatorial proof by giving an *algebraic* proof of this same fact.



16 Generating Functions

Generating Functions are one of the most surprising and useful inventions in Discrete Math. Roughly speaking, generating functions transform problems about *sequences* into problems about *functions*. This is great because we’ve got piles of mathematical machinery for manipulating functions. Thanks to generating functions, we can apply all that machinery to problems about sequences. In this way, we can use generating functions to solve all sorts of counting problems. There is a huge chunk of mathematics concerning generating functions, so we will only get a taste of the subject.

In this chapter, we’ll put sequences in angle brackets to more clearly distinguish them from the many other mathematical expressions floating around.

The *ordinary generating function* for $\langle g_0, g_1, g_2, g_3 \dots \rangle$ is the power series:

$$G(x) = g_0 + g_1x + g_2x^2 + g_3x^3 + \dots$$

There are a few other kinds of generating functions in common use, but ordinary generating functions are enough to illustrate the power of the idea, so we’ll stick to them. So from now on *generating function* will mean the ordinary kind.

A generating function is a “formal” power series in the sense that we usually regard x as a placeholder rather than a number. Only in rare cases will we actually evaluate a generating function by letting x take a real number value, so we generally ignore the issue of convergence.

Throughout this chapter, we’ll indicate the correspondence between a sequence and its generating function with a double-sided arrow as follows:

$$\langle g_0, g_1, g_2, g_3, \dots \rangle \longleftrightarrow g_0 + g_1x + g_2x^2 + g_3x^3 + \dots$$

For example, here are some sequences and their generating functions:

$$\langle 0, 0, 0, 0, \dots \rangle \longleftrightarrow 0 + 0x + 0x^2 + 0x^3 + \dots = 0$$

$$\langle 1, 0, 0, 0, \dots \rangle \longleftrightarrow 1 + 0x + 0x^2 + 0x^3 + \dots = 1$$

$$\langle 3, 2, 1, 0, \dots \rangle \longleftrightarrow 3 + 2x + 1x^2 + 0x^3 + \dots = 3 + 2x + x^2$$

The pattern here is simple: the i th term in the sequence (indexing from 0) is the coefficient of x^i in the generating function.

Recall that the sum of an infinite geometric series is:

$$1 + z + z^2 + z^3 + \dots = \frac{1}{1 - z}$$

This equation does not hold when $|z| \geq 1$, but as remarked, we don't worry about convergence issues. This formula gives closed form generating functions for a whole range of sequences. For example:

$$\langle 1, 1, 1, 1, \dots \rangle \longleftrightarrow 1 + x + x^2 + x^3 + \dots = \frac{1}{1-x}$$

$$\langle 1, -1, 1, -1, \dots \rangle \longleftrightarrow 1 - x + x^2 - x^3 + x^4 - \dots = \frac{1}{1+x}$$

$$\langle 1, a, a^2, a^3, \dots \rangle \longleftrightarrow 1 + ax + a^2x^2 + a^3x^3 + \dots = \frac{1}{1-ax}$$

$$\langle 1, 0, 1, 0, 1, 0, \dots \rangle \longleftrightarrow 1 + x^2 + x^4 + x^6 + \dots = \frac{1}{1-x^2}$$

16.1 Operations on Generating Functions

The magic of generating functions is that we can carry out all sorts of manipulations on sequences by performing mathematical operations on their associated generating functions. Let's experiment with various operations and characterize their effects in terms of sequences.

16.1.1 Scaling

Multiplying a generating function by a constant scales every term in the associated sequence by the same constant. For example, we noted above that:

$$\langle 1, 0, 1, 0, 1, 0, \dots \rangle \longleftrightarrow 1 + x^2 + x^4 + x^6 + \dots = \frac{1}{1-x^2}$$

Multiplying the generating function by 2 gives

$$\frac{2}{1-x^2} = 2 + 2x^2 + 2x^4 + 2x^6 + \dots$$

which generates the sequence:

$$\langle 2, 0, 2, 0, 2, 0, \dots \rangle$$

Rule 1 (Scaling Rule). *If*

$$\langle f_0, f_1, f_2, \dots \rangle \longleftrightarrow F(x),$$

then

$$\langle cf_0, cf_1, cf_2, \dots \rangle \longleftrightarrow c \cdot F(x).$$

The idea behind this rule is that:

$$\begin{aligned}\langle cf_0, cf_1, cf_2, \dots \rangle &\longleftrightarrow cf_0 + cf_1x + cf_2x^2 + \dots \\ &= c \cdot (f_0 + f_1x + f_2x^2 + \dots) \\ &= cF(x)\end{aligned}$$

16.1.2 Addition

Adding generating functions corresponds to adding the two sequences term by term. For example, adding two of our earlier examples gives:

$$\begin{aligned}\langle 1, 1, 1, 1, 1, 1, \dots \rangle &\longleftrightarrow \frac{1}{1-x} \\ + \langle 1, -1, 1, -1, 1, -1, \dots \rangle &\longleftrightarrow \frac{1}{1+x} \\ \hline \langle 2, 0, 2, 0, 2, 0, \dots \rangle &\longleftrightarrow \frac{1}{1-x} + \frac{1}{1+x}\end{aligned}$$

We’ve now derived two different expressions that both generate the sequence $\langle 2, 0, 2, 0, \dots \rangle$. They are, of course, equal:

$$\frac{1}{1-x} + \frac{1}{1+x} = \frac{(1+x) + (1-x)}{(1-x)(1+x)} = \frac{2}{1-x^2}$$

Rule 2 (Addition Rule). *If*

$$\begin{aligned}\langle f_0, f_1, f_2, \dots \rangle &\longleftrightarrow F(x), & \text{and} \\ \langle g_0, g_1, g_2, \dots \rangle &\longleftrightarrow G(x),\end{aligned}$$

then

$$\langle f_0 + g_0, f_1 + g_1, f_2 + g_2, \dots \rangle \longleftrightarrow F(x) + G(x).$$

The idea behind this rule is that:

$$\begin{aligned}\langle f_0 + g_0, f_1 + g_1, f_2 + g_2, \dots \rangle &\longleftrightarrow \sum_{n=0}^{\infty} (f_n + g_n)x^n \\ &= \left(\sum_{n=0}^{\infty} f_n x^n \right) + \left(\sum_{n=0}^{\infty} g_n x^n \right) \\ &= F(x) + G(x)\end{aligned}$$

16.1.3 Right Shifting

Let’s start over again with a simple sequence and its generating function:

$$\langle 1, 1, 1, 1, \dots \rangle \longleftrightarrow \frac{1}{1-x}$$

Now let’s *right-shift* the sequence by adding k leading zeros:

$$\begin{aligned} \underbrace{\langle 0, 0, \dots, 0, 1, 1, 1, \dots \rangle}_{k \text{ zeroes}} &\longleftrightarrow x^k + x^{k+1} + x^{k+2} + x^{k+3} + \dots \\ &= x^k \cdot (1 + x + x^2 + x^3 + \dots) \\ &= \frac{x^k}{1-x} \end{aligned}$$

Evidently, adding k leading zeros to the sequence corresponds to multiplying the generating function by x^k . This holds true in general.

Rule 3 (Right-Shift Rule). *If $\langle f_0, f_1, f_2, \dots \rangle \longleftrightarrow F(x)$, then:*

$$\underbrace{\langle 0, 0, \dots, 0, f_0, f_1, f_2, \dots \rangle}_{k \text{ zeroes}} \longleftrightarrow x^k \cdot F(x)$$

The idea behind this rule is that:

$$\begin{aligned} \underbrace{\langle 0, 0, \dots, 0, f_0, f_1, f_2, \dots \rangle}_{k \text{ zeroes}} &\longleftrightarrow f_0 x^k + f_1 x^{k+1} + f_2 x^{k+2} + \dots \\ &= x^k \cdot (f_0 + f_1 x + f_2 x^2 + f_3 x^3 + \dots) \\ &= x^k \cdot F(x) \end{aligned}$$

16.1.4 Differentiation

What happens if we take the *derivative* of a generating function? As an example, let’s differentiate the now-familiar generating function for an infinite sequence of 1’s.

$$\begin{aligned} \frac{d}{dx} (1 + x + x^2 + x^3 + x^4 + \dots) &= \frac{d}{dx} \left(\frac{1}{1-x} \right) \\ 1 + 2x + 3x^2 + 4x^3 + \dots &= \frac{1}{(1-x)^2} \\ \langle 1, 2, 3, 4, \dots \rangle &\longleftrightarrow \frac{1}{(1-x)^2} \end{aligned} \tag{16.1}$$

We found a generating function for the sequence $\langle 1, 2, 3, 4, \dots \rangle$ of positive integers!

In general, differentiating a generating function has two effects on the corresponding sequence: each term is multiplied by its index and the entire sequence is shifted left one place.

Rule 4 (Derivative Rule). *If*

$$\langle f_0, f_1, f_2, f_3, \dots \rangle \longleftrightarrow F(x),$$

then

$$\langle f_1, 2f_2, 3f_3, \dots \rangle \longleftrightarrow F'(x).$$

The idea behind this rule is that:

$$\begin{aligned} \langle f_1, 2f_2, 3f_3, \dots \rangle &\longleftrightarrow f_1 + 2f_2x + 3f_3x^2 + \dots \\ &= \frac{d}{dx} (f_0 + f_1x + f_2x^2 + f_3x^3 + \dots) \\ &= \frac{d}{dx} F(x) \end{aligned}$$

The Derivative Rule is very useful. In fact, there is frequent, independent need for each of differentiation’s two effects, multiplying terms by their index and left-shifting one place. Typically, we want just one effect and must somehow cancel out the other. For example, let’s try to find the generating function for the sequence of squares, $\langle 0, 1, 4, 9, 16, \dots \rangle$. If we could start with the sequence $\langle 1, 1, 1, 1, \dots \rangle$ and multiply each term by its index two times, then we’d have the desired result:

$$\langle 0 \cdot 0, 1 \cdot 1, 2 \cdot 2, 3 \cdot 3, \dots \rangle = \langle 0, 1, 4, 9, \dots \rangle$$

A challenge is that differentiation not only multiplies each term by its index, but also shifts the whole sequence left one place. However, the Right-Shift Rule 3 tells how to cancel out this unwanted left-shift: multiply the generating function by x .

Our procedure, therefore, is to begin with the generating function for $\langle 1, 1, 1, 1, \dots \rangle$,

differentiate, multiply by x , and then differentiate and multiply by x once more.

$$\begin{aligned}\langle 1, 1, 1, 1, \dots \rangle &\longleftrightarrow \frac{1}{1-x} \\ \langle 1, 2, 3, 4, \dots \rangle &\longleftrightarrow \frac{d}{dx} \frac{1}{1-x} = \frac{1}{(1-x)^2} \\ \langle 0, 1, 2, 3, \dots \rangle &\longleftrightarrow x \cdot \frac{1}{(1-x)^2} = \frac{x}{(1-x)^2} \\ \langle 1, 4, 9, 16, \dots \rangle &\longleftrightarrow \frac{d}{dx} \frac{x}{(1-x)^2} = \frac{1+x}{(1-x)^3} \\ \langle 0, 1, 4, 9, \dots \rangle &\longleftrightarrow x \cdot \frac{1+x}{(1-x)^3} = \frac{x(1+x)}{(1-x)^3}\end{aligned}$$

Thus, the generating function for squares is:

$$\frac{x(1+x)}{(1-x)^3} \tag{16.2}$$

16.1.5 Products

Rule 5 (Product Rule). *If*

$$\langle a_0, a_1, a_2, \dots \rangle \longleftrightarrow A(x), \quad \text{and} \quad \langle b_0, b_1, b_2, \dots \rangle \longleftrightarrow B(x),$$

then

$$\langle c_0, c_1, c_2, \dots \rangle \longleftrightarrow A(x) \cdot B(x),$$

where

$$c_n ::= a_0 b_n + a_1 b_{n-1} + a_2 b_{n-2} + \dots + a_n b_0.$$

To understand this rule, let

$$C(x) ::= A(x) \cdot B(x) = \sum_{n=0}^{\infty} c_n x^n.$$

We can evaluate the product $A(x) \cdot B(x)$ by using a table to identify all the

cross-terms from the product of the sums:

	b_0x^0	b_1x^1	b_2x^2	b_3x^3	...
a_0x^0	$a_0b_0x^0$	$a_0b_1x^1$	$a_0b_2x^2$	$a_0b_3x^3$	...
a_1x^1	$a_1b_0x^1$	$a_1b_1x^2$	$a_1b_2x^3$	...	
a_2x^2	$a_2b_0x^2$	$a_2b_1x^3$	...		
a_3x^3	$a_3b_0x^3$	...			
$\vdots$	...				

Notice that all terms involving the same power of x lie on a $/$ -sloped diagonal. Collecting these terms together, we find that the coefficient of x^n in the product is the sum of all the terms on the $(n + 1)$ st diagonal, namely,

$$a_0b_n + a_1b_{n-1} + a_2b_{n-2} + \cdots + a_nb_0. \quad (16.3)$$

This expression (16.3) may be familiar from a signal processing course; the sequence $\langle c_0, c_1, c_2, \dots \rangle$ is called the *convolution* of sequences $\langle a_0, a_1, a_2, \dots \rangle$ and $\langle b_0, b_1, b_2, \dots \rangle$.

16.2 The Fibonacci Sequence

Sometimes we can find nice generating functions for more complicated sequences. For example, here is a generating function for the Fibonacci numbers:

$$\langle 0, 1, 1, 2, 3, 5, 8, 13, 21, \dots \rangle \longleftrightarrow \frac{x}{1 - x - x^2}$$

The Fibonacci numbers may seem like a fairly nasty bunch, but the generating function is simple!

We’re going to derive this generating function and then use it to find a closed form for the n th Fibonacci number. The techniques we’ll use are applicable to a large class of recurrence equations.

16.2.1 Finding a Generating Function

Let’s begin by recalling the definition of the Fibonacci numbers:

$$\begin{aligned} f_0 &= 0 \\ f_1 &= 1 \\ f_n &= f_{n-1} + f_{n-2} \quad (\text{for } n \geq 2) \end{aligned}$$

We can expand the final clause into an infinite sequence of equations. Thus, the Fibonacci numbers are defined by:

$$\begin{aligned} f_0 &= 0 \\ f_1 &= 1 \\ f_2 &= f_1 + f_0 \\ f_3 &= f_2 + f_1 \\ f_4 &= f_3 + f_2 \\ &\vdots \end{aligned}$$

Now the overall plan is to *define* a function $F(x)$ that generates the sequence on the left side of the equality symbols, which are the Fibonacci numbers. Then we *derive* a function that generates the sequence on the right side. Finally, we equate the two and solve for $F(x)$. Let’s try this. First, we define:

$$F(x) = f_0 + f_1x + f_2x^2 + f_3x^3 + f_4x^4 + \dots$$

Now we need to derive a generating function for the sequence:

$$\langle 0, 1, f_1 + f_0, f_2 + f_1, f_3 + f_2, \dots \rangle$$

One approach is to break this into a sum of three sequences for which we know generating functions and then apply the Addition Rule:

$$\begin{array}{rcll} \langle 0, & 1, & 0, & 0, & 0, & \dots \rangle & \longleftrightarrow & x \\ \langle 0, & f_0, & f_1, & f_2, & f_3, & \dots \rangle & \longleftrightarrow & xF(x) \\ + \langle 0, & 0, & f_0, & f_1, & f_2, & \dots \rangle & \longleftrightarrow & x^2F(x) \\ \hline \langle 0, & 1 + f_0, & f_1 + f_0, & f_2 + f_1, & f_3 + f_2, & \dots \rangle & \longleftrightarrow & x + xF(x) + x^2F(x) \end{array}$$

This sequence is almost identical to the right sides of the Fibonacci equations. The one blemish is that the second term is $1 + f_0$ instead of simply 1. However, this amounts to nothing, since $f_0 = 0$ anyway.

Now if we equate $F(x)$ with the new function $x + xF(x) + x^2F(x)$, then we’re implicitly writing down *all* of the equations that define the Fibonacci numbers in one fell swoop:

$$\begin{array}{ccccccc} F(x) & = & f_0 & + & f_1 x & + & f_2 x^2 + f_3 x^3 + \dots \\ \parallel & & \parallel & & \parallel & & \parallel \\ x + xF(x) + x^2F(x) & = & 0 & + & (1 + f_0)x & + & (f_1 + f_0)x^2 + (f_2 + f_1)x^3 + \dots \end{array}$$

Solving for $F(x)$ gives the generating function for the Fibonacci sequence:

$$F(x) = x + xF(x) + x^2F(x)$$

so

$$F(x) = \frac{x}{1 - x - x^2}.$$

Sure enough, this is the simple generating function we claimed at the outset.

16.2.2 Finding a Closed Form

Why should one care about the generating function for a sequence? There are several answers, but here is one: if we can find a generating function for a sequence, then we can often find a closed form for the n th coefficient—which can be pretty useful! For example, a closed form for the coefficient of x^n in the power series for $x/(1 - x - x^2)$ would be an explicit formula for the n th Fibonacci number.

So our next task is to extract coefficients from a generating function. There are several approaches. For a generating function that is a ratio of polynomials, we can use the method of *partial fractions*, which you learned in calculus. Just as the terms in a partial fraction expansion are easier to integrate, the coefficients of those terms are easy to compute.

Let’s try this approach with the generating function for Fibonacci numbers. First, we factor the denominator:

$$1 - x - x^2 = (1 - \alpha_1 x)(1 - \alpha_2 x)$$

where $\alpha_1 = \frac{1}{2}(1 + \sqrt{5})$ and $\alpha_2 = \frac{1}{2}(1 - \sqrt{5})$. Next, we find A_1 and A_2 which satisfy:

$$\frac{x}{1 - x - x^2} = \frac{A_1}{1 - \alpha_1 x} + \frac{A_2}{1 - \alpha_2 x}$$

We do this by plugging in various values of x to generate linear equations in A_1 and A_2 . We can then find A_1 and A_2 by solving a linear system. This gives:

$$\begin{aligned} A_1 &= \frac{1}{\alpha_1 - \alpha_2} = \frac{1}{\sqrt{5}} \\ A_2 &= \frac{-1}{\alpha_1 - \alpha_2} = -\frac{1}{\sqrt{5}} \end{aligned}$$

Substituting into the equation above gives the partial fractions expansion of $F(x)$:

$$\frac{x}{1-x-x^2} = \frac{1}{\sqrt{5}} \left(\frac{1}{1-\alpha_1 x} - \frac{1}{1-\alpha_2 x} \right)$$

Each term in the partial fractions expansion has a simple power series given by the geometric sum formula:

$$\begin{aligned} \frac{1}{1-\alpha_1 x} &= 1 + \alpha_1 x + \alpha_1^2 x^2 + \cdots \\ \frac{1}{1-\alpha_2 x} &= 1 + \alpha_2 x + \alpha_2^2 x^2 + \cdots \end{aligned}$$

Substituting in these series gives a power series for the generating function:

$$\begin{aligned} F(x) &= \frac{1}{\sqrt{5}} \left(\frac{1}{1-\alpha_1 x} - \frac{1}{1-\alpha_2 x} \right) \\ &= \frac{1}{\sqrt{5}} ((1 + \alpha_1 x + \alpha_1^2 x^2 + \cdots) - (1 + \alpha_2 x + \alpha_2^2 x^2 + \cdots)), \end{aligned}$$

so

$$\begin{aligned} f_n &= \frac{\alpha_1^n - \alpha_2^n}{\sqrt{5}} \\ &= \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right) \end{aligned}$$

This formula may be scary and astonishing—it’s not even obvious that its value is an integer—but it’s very useful. For example, it provides (via the repeated squaring method) a much more efficient way to compute Fibonacci numbers than crunching through the recurrence, and it also clearly reveals the exponential growth of these numbers.

16.3 Counting with Generating Functions

Generating functions are particularly useful for solving counting problems. In particular, problems involving choosing items from a set often lead to nice generating functions by letting the coefficient of x^n be the number of ways to choose n items.

16.3.1 Choosing Distinct Items from a Set

The generating function for binomial coefficients follows directly from the Binomial Theorem:

$$\left\langle \binom{k}{0}, \binom{k}{1}, \binom{k}{2}, \dots, \binom{k}{k}, 0, 0, 0, \dots \right\rangle \longleftrightarrow \binom{k}{0} + \binom{k}{1}x + \binom{k}{2}x^2 + \dots + \binom{k}{k}x^k = (1+x)^k$$

Thus, the coefficient of x^n in $(1+x)^k$ is $\binom{k}{n}$, the number of ways to choose n distinct items from a set of size k . For example, the coefficient of x^2 is $\binom{k}{2}$, the number of ways to choose 2 items from a set with k elements. Similarly, the coefficient of x^{k+1} is the number of ways to choose $k+1$ items from a size k set, which is zero. (Watch out for this reversal of the roles that k and n played in earlier examples; we’re led to this reversal because we’ve been using n to refer to the power of x in a power series.)

16.3.2 Building Generating Functions that Count

Often we can translate the description of a counting problem directly into a generating function for the solution. For example, we could figure out that $(1+x)^k$ generates the number of ways to select n distinct items from a k -element set without resorting to the Binomial Theorem or even fussing with binomial coefficients!

Here is how. First, consider a single-element set $\{a_1\}$. The generating function for the number of ways to select n elements from this set is simply $1+x$: we have 1 way to select zero elements, 1 way to select one element, and 0 ways to select more than one element. Similarly, the number of ways to select n elements from the set $\{a_2\}$ is also given by the generating function $1+x$. The fact that the elements differ in the two cases is irrelevant.

Now here is the main trick: *the generating function for choosing elements from a union of disjoint sets is the product of the generating functions for choosing from each set.* We’ll justify this in a moment, but let’s first look at an example. According to this principle, the generating function for the number of ways to select n elements from the $\{a_1, a_2\}$ is:

$$\underbrace{(1+x)}_{\text{gen func for selecting an } a_1} \cdot \underbrace{(1+x)}_{\text{gen func for selecting an } a_2} = \underbrace{(1+x)^2}_{\text{gen func for selecting from } \{a_1, a_2\}} = 1 + 2x + x^2$$

Sure enough, for the set $\{a_1, a_2\}$, we have 1 way to select zero elements, 2 ways to

select one element, 1 way to select two elements, and 0 ways to select more than two elements.

Repeated application of this rule gives the generating function for selecting n items from a k -element set $\{a_1, a_2, \dots, a_k\}$:

$$\underbrace{(1+x)}_{\text{gen func for selecting an } a_1} \cdot \underbrace{(1+x)}_{\text{gen func for selecting an } a_2} \cdots \underbrace{(1+x)}_{\text{gen func for selecting an } a_k} = \underbrace{(1+x)^k}_{\text{gen func for selecting from } \{a_1, a_2, \dots, a_k\}}$$

This is the same generating function that we obtained by using the Binomial Theorem. But this time around we translated directly from the counting problem to the generating function.

We can extend these ideas to a general principle:

Rule 6 (Convolution Rule). *Let $A(x)$ be the generating function for selecting items from set $\mathcal{A}$, and let $B(x)$ be the generating function for selecting items from set $\mathcal{B}$. If $\mathcal{A}$ and $\mathcal{B}$ are disjoint, then the generating function for selecting items from the union $\mathcal{A} \cup \mathcal{B}$ is the product $A(x) \cdot B(x)$.*

This rule is rather ambiguous: what exactly are the rules governing the selection of items from a set? Remarkably, the Convolution Rule remains valid under *many* interpretations of selection. For example, we could insist that distinct items be selected or we might allow the same item to be picked a limited number of times or any number of times. Informally, the only restrictions are that (1) the order in which items are selected is disregarded and (2) restrictions on the selection of items from sets $\mathcal{A}$ and $\mathcal{B}$ also apply in selecting items from $\mathcal{A} \cup \mathcal{B}$. (Formally, there must be a bijection between n -element selections from $\mathcal{A} \cup \mathcal{B}$ and ordered pairs of selections from $\mathcal{A}$ and $\mathcal{B}$ containing a total of n elements.)

To count the number of ways to select n items from $\mathcal{A} \cup \mathcal{B}$, we observe that we can select n items by choosing j items from $\mathcal{A}$ and $n - j$ items from $\mathcal{B}$, where j is any number from 0 to n . This can be done in $a_j b_{n-j}$ ways. Summing over all the possible values of j gives a total of

$$a_0 b_n + a_1 b_{n-1} + a_2 b_{n-2} + \cdots + a_n b_0$$

ways to select n items from $\mathcal{A} \cup \mathcal{B}$. By the Product Rule, this is precisely the coefficient of x^n in the series for $A(x)B(x)$.

16.3.3 Choosing Items with Repetition

The first counting problem we considered was the number of ways to select a dozen doughnuts when five flavors were available. We can generalize this question as

follows: in how many ways can we select n items from a k -element set if we're allowed to pick the same item multiple times? In these terms, the doughnut problem asks in how many ways we can select $n = 12$ doughnuts from the set of $k = 5$ flavors

{chocolate, lemon-filled, sugar, glazed, plain}

where, of course, we're allowed to pick several doughnuts of the same flavor. Let's approach this question from a generating functions perspective.

Suppose we make n choices (with repetition allowed) of items from a set containing a single item. Then there is one way to choose zero items, one way to choose one item, one way to choose two items, etc. Thus, the generating function for choosing n elements with repetition from a 1-element set is:

$$\begin{aligned} \langle 1, 1, 1, 1, \dots \rangle &\longleftrightarrow 1 + x + x^2 + x^3 + \dots \\ &= \frac{1}{1-x} \end{aligned}$$

The Convolution Rule says that the generating function for selecting items from a union of disjoint sets is the product of the generating functions for selecting items from each set:

$$\underbrace{\frac{1}{1-x}}_{\text{gen func for choosing } a_1\text{'s}} \cdot \underbrace{\frac{1}{1-x}}_{\text{gen func for choosing } a_2\text{'s}} \cdots \underbrace{\frac{1}{1-x}}_{\text{gen func for choosing } a_k\text{'s}} = \underbrace{\frac{1}{(1-x)^k}}_{\text{gen func for repeated choice from } \{a_1, a_2, \dots, a_k\}}$$

Therefore, the generating function for choosing items from a k -element set with repetition allowed is $1/(1-x)^k$.

Now the Bookkeeper Rule tells us that the number of ways to choose n items with repetition from an k element set is

$$\binom{n+k-1}{n},$$

so this is the coefficient of x^n in the series expansion of $1/(1-x)^k$.

On the other hand, it's instructive to derive this coefficient algebraically, which we can do using Taylor's Theorem:

Theorem 16.3.1 (Taylor's Theorem).

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots + \frac{f^{(n)}(0)}{n!}x^n + \dots$$

This theorem says that the n th coefficient of $1/(1-x)^k$ is equal to its n th derivative evaluated at 0 and divided by $n!$. Computing the n th derivative turns out not to be very difficult (Problem 16.9).

16.4 An “Impossible” Counting Problem

So far everything we’ve done with generating functions we could have done another way. But here is an absurd counting problem—really over the top! In how many ways can we fill a bag with n fruits subject to the following constraints?

- The number of apples must be even.
- The number of bananas must be a multiple of 5.
- There can be at most four oranges.
- There can be at most one pear.

For example, there are 7 ways to form a bag with 6 fruits:

Apples	6	4	4	2	2	0	0
Bananas	0	0	0	0	0	5	5
Oranges	0	2	1	4	3	1	0
Pears	0	0	1	0	1	0	1

These constraints are so complicated that the problem may seem hopeless. But let’s see what generating functions reveal.

Let’s first construct a generating function for choosing apples. We can choose a set of 0 apples in one way, a set of 1 apple in zero ways (since the number of apples must be even), a set of 2 apples in one way, a set of 3 apples in zero ways, and so forth. So we have:

$$A(x) = 1 + x^2 + x^4 + x^6 + \cdots = \frac{1}{1-x^2}$$

Similarly, the generating function for choosing bananas is:

$$B(x) = 1 + x^5 + x^{10} + x^{15} + \cdots = \frac{1}{1-x^5}$$

Now, we can choose a set of 0 oranges in one way, a set of 1 orange in one way, and so on. However, we can not choose more than four oranges, so we have the

generating function:

$$O(x) = 1 + x + x^2 + x^3 + x^4 = \frac{1 - x^5}{1 - x}$$

Here we’re using the geometric sum formula. Finally, we can choose only zero or one pear, so we have:

$$P(x) = 1 + x$$

The Convolution Rule says that the generating function for choosing from among all four kinds of fruit is:

$$\begin{aligned} A(x)B(x)O(x)P(x) &= \frac{1}{1 - x^2} \frac{1}{1 - x^5} \frac{1 - x^5}{1 - x} (1 + x) \\ &= \frac{1}{(1 - x)^2} \\ &= 1 + 2x + 3x^2 + 4x^3 + \dots \end{aligned}$$

Almost everything cancels! We’re left with $1/(1 - x)^2$, which we found a power series for earlier: the coefficient of x^n is simply $n + 1$. Thus, the number of ways to form a bag of n fruits is just $n + 1$. This is consistent with the example we worked out, since there were 7 different fruit bags containing 6 fruits. *Amazing!*

Problems for Section 16.2

Class Problems

Problem 16.1.

The famous mathematician, Fibonacci, has decided to start a rabbit farm to fill up his time while he’s not making new sequences to torment future college students. Fibonacci starts his farm on month zero (being a mathematician), and at the start of month one he receives his first pair of rabbits. Each pair of rabbits takes a month to mature, and after that breeds to produce one new pair of rabbits each month. Fibonacci decides that in order never to run out of rabbits or money, every time a batch of new rabbits is born, he’ll sell a number of newborn pairs equal to the total number of pairs he had three months earlier. Fibonacci is convinced that this way he’ll never run out of stock.

(a) Define the number, r_n , of pairs of rabbits Fibonacci has in month n , using a recurrence relation. That is, define r_n in terms of various r_i where $i < n$.

(b) Let $R(x)$ be the generating function for rabbit pairs,

$$R(x) ::= r_0 + r_1x + r_2x^2 + \dots$$

Express $R(x)$ as a quotient of polynomials.

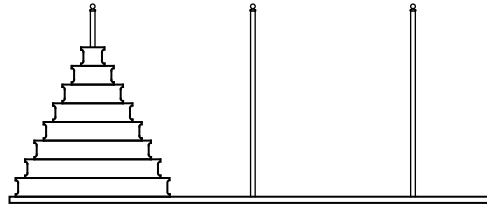


Figure 16.1 The initial configuration of the disks in the Towers of Sheboygan problem.

- (c) Find a partial fraction decomposition of the generating function $R(x)$.
- (d) Finally, use the partial fraction decomposition to come up with a closed form expression for the number of pairs of rabbits Fibonacci has on his farm on month n .

Problem 16.2.

Less well-known than the Towers of Hanoi —but no less fascinating —are the Towers of Sheboygan. As in Hanoi, the puzzle in Sheboygan involves 3 posts and n disks of different sizes. Initially, all the disks are on post #1 as in Figure 16.1.

The objective is to transfer all n disks to post #2 via a sequence of moves. A move consists of removing the top disk from one post and dropping it onto another post with the restriction that a larger disk can never lie above a smaller disk. Furthermore, a local ordinance requires that *a disk can be moved only from a post to the next post on its right —or from post #3 to post #1*. Thus, for example, moving a disk directly from post #1 to post #3 is not permitted.

(a) One procedure that solves the Sheboygan puzzle is defined recursively: to move an initial stack of n disks to the next post, move the top stack of $n - 1$ disks to the furthest post by moving it to the next post two times, then move the big, n th disk to the next post, and finally move the top stack another two times to land on top of the big disk. Let s_n be the number of moves that this procedure uses. Write a simple linear recurrence for s_n .

(b) Let $S(x)$ be the generating function for the sequence $\langle s_0, s_1, s_2, \dots \rangle$. Carefully Show that

$$S(x) = \frac{x}{(1-x)(1-4x)}.$$

- (c) Give a simple formula for s_n .

(d) A better (indeed optimal, but we won’t prove this) procedure to solve the Towers of Sheboygan puzzle can be defined in terms of two mutually recursive procedures, procedure $P_1(n)$ for moving a stack of n disks 1 pole forward, and $P_2(n)$ for moving a stack of n disks 2 poles forward. This is trivial for $n = 0$. For $n > 0$, define:

$P_1(n)$: Apply $P_2(n - 1)$ to move the top $n - 1$ disks two poles forward to the third pole. Then move the remaining big disk once to land on the second pole. Then apply $P_2(n - 1)$ again to move the stack of $n - 1$ disks two poles forward from the third pole to land on top of the big disk.

$P_2(n)$: Apply $P_2(n - 1)$ to move the top $n - 1$ disks two poles forward to land on the third pole. Then move the remaining big disk to the second pole. Then apply $P_1(n - 1)$ to move the stack of $n - 1$ disks one pole forward to land on the first pole. Now move the big disk 1 pole forward again to land on the third pole. Finally, apply $P_2(n - 1)$ again to move the stack of $n - 1$ disks two poles forward to land on the big disk.

Let t_n be the number of moves needed to solve the Sheboygan puzzle using procedure $P_1(n)$. Show that

$$t_n = 2t_{n-1} + 2t_{n-2} + 3, \quad (16.4)$$

for $n > 1$.

Hint: Let u_n be the number of moves used by procedure $P_2(n)$. Express each of t_n and u_n as linear combinations of t_{n-1} and u_{n-1} and solve for t_n .

(e) Derive values a, b, c, α, β such that

$$t_n = a\alpha^n + b\beta^n + c.$$

Conclude that $t_n = o(s_n)$.

Homework Problems

Problem 16.3.

Taking derivatives of generating functions is another useful operation. This is done termwise, that is, if

$$F(x) = f_0 + f_1x + f_2x^2 + f_3x^3 + \cdots,$$

then

$$F'(x) ::= f_1 + 2f_2x + 3f_3x^2 + \cdots.$$

For example,

$$\frac{1}{(1-x)^2} = \left(\frac{1}{(1-x)} \right)' = 1 + 2x + 3x^2 + \dots$$

so

$$H(x) ::= \frac{x}{(1-x)^2} = 0 + 1x + 2x^2 + 3x^3 + \dots$$

is the generating function for the sequence of nonnegative integers. Therefore

$$\frac{1+x}{(1-x)^3} = H'(x) = 1 + 2^2x + 3^2x^2 + 4^2x^3 + \dots,$$

so

$$\frac{x^2+x}{(1-x)^3} = xH'(x) = 0 + 1x + 2^2x^2 + 3^2x^3 + \dots + n^2x^n + \dots$$

is the generating function for the nonnegative integer squares.

(a) Prove that for all $k \in \mathbb{N}$, the generating function for the nonnegative integer k th powers is a quotient of polynomials in x . That is, for all $k \in \mathbb{N}$ there are polynomials $R_k(x)$ and $S_k(x)$ such that

$$[x^n] \left(\frac{R_k(x)}{S_k(x)} \right) = n^k. \quad (16.5)$$

Hint: Observe that the derivative of a quotient of polynomials is also a quotient of polynomials. It is not necessary work out explicit formulas for R_k and S_k to prove this part.

(b) Conclude that if $f(n)$ is a function on the nonnegative integers defined recursively in the form

$$f(n) = af(n-1) + bf(n-2) + cf(n-3) + p(n)\alpha^n$$

where the $a, b, c, \alpha \in \mathbb{C}$ and p is a polynomial with complex coefficients, then the generating function for the sequence $f(0), f(1), f(2), \dots$ will be a quotient of polynomials in x , and hence there is a closed form expression for $f(n)$.

Hint: Consider

$$\frac{R_k(\alpha x)}{S_k(\alpha x)}$$

Problem 16.4.

Generating functions provide an interesting way to count the number of strings of matched brackets. To do this, we’ll use a description of these strings as the set, GoodCount, of strings of brackets with a good count.

Namely, one precise way to determine if a string is matched is to start with 0 and read the string from left to right, adding 1 to the count for each left bracket and subtracting 1 from the count for each right bracket. For example, here are the counts for the two strings above

	[	]		]		[	[	[	[	[	]	]	]	]
0	1	0	-1	0	1	2	3	4	3	2	1	0		

	[	[		]	]		[	]	]		[	]
0	1	2	3	2	1	2	1	0	1	0		

A string has a *good count* if its running count never goes negative and ends with 0. So the second string above has a good count, but the first one does not because its count went negative at the third step.

Definition 16.4.1. Let

$$\text{GoodCount} ::= \{s \in \{[, \}\}^* \mid s \text{ has a good count}\}.$$

The matched strings can now be characterized precisely as this set of strings with good counts.

Let c_n be the number of strings in GoodCount with exactly n left brackets, and let $C(x)$ be the generating function for these numbers:

$$C(x) ::= c_0 + c_1x + c_2x^2 + \cdots.$$

(a) The *wrap* of a string, s , is the string, $[s]$, that starts with a left bracket followed by the characters of s , and then ends with a right bracket. Explain why the generating function for the wraps of strings with a good count is $xC(x)$.

Hint: The wrap of a string with good count also has a good count that starts and ends with 0 and remains *positive* everywhere else.

(b) Explain why, for every string, s , with a good count, there is a unique sequence of strings $s_1, \dots, s_k$ that are wraps of strings with good counts and $s = s_1 \cdots s_k$. For example, the string $r ::= [[[]][[]][[]]] \in \text{GoodCount}$ equals $s_1 s_2 s_3$ where $s_1 ::= [[[]]$, $s_2 ::= [[]]$, $s_3 ::= [[]][[]]$, and this is the only way to express r as a sequence of wraps of strings with good counts.

(c) Conclude that

$$C = 1 + xC + (xC)^2 + \cdots + (xC)^n + \cdots, \quad (16.6)$$

so

$$C = \frac{1}{1 - xC}, \quad (16.7)$$

and hence

$$C = \frac{1 \pm \sqrt{1 - 4x}}{2x}. \quad (16.8)$$

Let $D(x) ::= 2xC(x)$. Expressing D as a power series

$$D(x) = d_0 + d_1x + d_2x^2 + \cdots,$$

we have

$$c_n = \frac{d_{n+1}}{2}. \quad (16.9)$$

(d) Use (16.8), (16.9), and the value of c_0 to conclude that

$$D(x) = 1 - \sqrt{1 - 4x}.$$

(e) Prove that

$$d_n = \frac{(2n-3) \cdot (2n-5) \cdots 5 \cdot 3 \cdot 1 \cdot 2^n}{n!}.$$

Hint: $d_n = D^{(n)}(0)/n!$

(f) Conclude that

$$c_n = \frac{1}{n+1} \binom{2n}{n}.$$

Exam Problems

Problem 16.5.

Define the sequence $r_0, r_1, r_2, \dots$ recursively by the rule that $r_0 = r_1 = 0$ and

$$r_n = 7r_{n-1} + 4r_{n-2} + (n+1),$$

for $n \geq 2$. Express the generating function of this sequence as a quotient of polynomials or products of polynomials. You do *not* have to find a closed form for r_n .

Problems for Section 16.3

Practice Problems

Problem 16.6.

You would like to buy a bouquet of flowers. You find an online service that will make bouquets of **lilies**, **roses** and **tulips**, subject to the following constraints:

- there must be at most 3 lilies,
- there must be an odd number of tulips,
- there can be any number of roses.

Example: A bouquet of 3 tulips, 5 roses and no lilies satisfies the constraints.

Let f_n be the number of possible bouquets with n flowers that fit the service’s constraints. Express $F(x)$, the generating function corresponding to $\langle f_0, f_1, f_2, \dots \rangle$, as a quotient of polynomials (or products of polynomials). You do not need to simplify this expression.

Problem 16.7.

Let $b, c, a_0, a_1, a_2, \dots$ be real numbers such that

$$a_n = b(a_{n-1}) + c$$

for $n \geq 1$.

Let $G(x)$ be the generating function for this sequence.

- (a) The coefficient of x^n in the series expansion of $G(x)$ is
- (b) The coefficient of x^n for $n \geq 1$ in the series expansion of $bxG(x)$ is
- (c) The coefficient of x^n for $n \geq 1$ in the series expansion of $cx/(1-x)$ is
- (d) Therefore, $G(x) - bxG(x) - cx/(1-x) =$
- (e) Using the method of partial fractions, we can find real numbers d and e such that

$$G(x) = d/L(x) + e/M(x).$$

What are $L(x)$ and $M(x)$?

Problem 16.8.

Write a formula for the generating function of each of the following sequences.

- (a) 0, 0, 1, 1, 1, ...
- (b) 1, 1, 0, 0, 0, ...
- (c) 1, 0, 1, 0, 1, 0, 1, ...
- (d) 1, 4, 6, 4, 1, 0, 0, 0, ...
- (e) 1, 1, 1/2, 1/6, 1/24, 1/120, ...
- (f) 1, 2, 3, 4, 5, ...
- (g) 1, 4, 9, 16, 25, ...

Class Problems

Problem 16.9.

Let $A(x) = \sum_{n=0}^{\infty} a_n x^n$. Then it's easy to check that

$$a_n = \frac{A^{(n)}(0)}{n!},$$

where $A^{(n)}$ is the n th derivative of A . Use this fact (which you may assume) instead of the Convolution Counting Principle, to prove that

$$\frac{1}{(1-x)^k} = \sum_{n=0}^{\infty} \binom{n+k-1}{k-1} x^n.$$

So if we didn't already know the Bookkeeper Rule, we could have proved it from this calculation and the Convolution Rule for generating functions.

Problem 16.10.

We are interested in generating functions for the number of different ways to compose a bag of n donuts subject to various restrictions. For each of the restrictions in (a)-(e) below, find a closed form for the corresponding generating function.

- (a) All the donuts are chocolate and there are at least 3.
- (b) All the donuts are glazed and there are at most 2.
- (c) All the donuts are coconut and there are exactly 2 or there are none.
- (d) All the donuts are plain and their number is a multiple of 4.

(e) The donuts must be chocolate, glazed, coconut, or plain and:

- there must be at least 3 chocolate donuts, and
- there must be at most 2 glazed, and
- there must be exactly 0 or 2 coconut, and
- there must be a multiple of 4 plain.

(f) Find a closed form for the number of ways to select n donuts subject to the constraints of the previous part.

Problem 16.11. (a) Let

$$S(x) ::= \frac{x^2 + x}{(1 - x)^3}.$$

What is the coefficient of x^n in the generating function series for $S(x)$?

(b) Explain why $S(x)/(1 - x)$ is the generating function for the sums of squares. That is, the coefficient of x^n in the series for $S(x)/(1 - x)$ is $\sum_{k=1}^n k^2$.

(c) Use the previous parts to prove that

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Homework Problems

Problem 16.12.

We will use generating functions to determine how many ways there are to use pennies, nickels, dimes, quarters, and half-dollars to give n cents change.

(a) Write the sequence P_n for the number of ways to use only pennies to change n cents. Write the generating function for that sequence.

(b) Write the sequence N_n for the number of ways to use only nickels to change n cents. Write the generating function for that sequence.

(c) Write the generating function for the number of ways to use only nickels and pennies to change n cents.

(d) Write the generating function for the number of ways to use pennies, nickels, dimes, quarters, and half-dollars to give n cents change.

(e) Explain how to use this function to find out how many ways are there to change 50 cents; you do *not* have to provide the answer or actually carry out the process.

Exam Problems

Problem 16.13.

Find the coefficients of $x^{10}y^5$ in $(19x + 4y)^{15}$

Problems for Section 16.4

Homework Problems

Problem 16.14.

Miss McGillicuddy never goes outside without a collection of pets. In particular:

- She brings a positive number of songbirds, which always come in pairs.
- She may or may not bring her alligator, Freddy.
- She brings at least 2 cats.
- She brings two or more chihuahuas and labradors leashed together in a line.

Let P_n denote the number of different collections of n pets that can accompany her, where we regard chihuahuas and labradors leashed up in different orders as different collections, even if there are the same number chihuahuas and labradors leashed in the line.

For example, $P_6 = 4$ since there are 4 possible collections of 6 pets:

- 2 songbirds, 2 cats, 2 chihuahuas leashed in line
- 2 songbirds, 2 cats, 2 labradors leashed in line
- 2 songbirds, 2 cats, a labrador leashed behind a chihuahua
- 2 songbirds, 2 cats, a chihuahua leashed behind a labrador

And $P_7 = 16$ since there are 16 possible collections of 7 pets:

- 2 songbirds, 3 cats, 2 chihuahuas leashed in line
- 2 songbirds, 3 cats, 2 labradors leashed in line
- 2 songbirds, 3 cats, a labrador leashed behind a chihuahua
- 2 songbirds, 3 cats, a chihuahua leashed behind a labrador

- 4 collections consisting of 2 songbirds, 2 cats, 1 alligator, and a line of 2 dogs
- 8 collections consisting of 2 songbirds, 2 cats, and a line of 3 dogs.

(a) Let

$$P(x) ::= P_0 + P_1x + P_2x^2 + P_3x^3 + \cdots$$

be the generating function for the number of Miss McGillicuddy’s pet collections. Verify that

$$P(x) = \frac{4x^6}{(1-x)^2(1-2x)}.$$

(b) Find a simple formula for P_n .

Exam Problems

Problem 16.15.

T-Pain is planning an epic boat trip and he needs to decide what to bring with him.

- He must bring some burgers, but they only come in packs of 6.
- He and his two friends can’t decide whether they want to dress formally or casually. He’ll either bring 0 pairs of flip flops or 3 pairs.
- He doesn’t have very much room in his suitcase for towels, so he can bring at most 2.
- In order for the boat trip to be truly epic, he has to bring at least 1 nautical-themed pashmina afghan.

(a) Let $B(x)$ be the generating function for the number of ways to bring n burgers, $F(x)$ for the number of ways to bring n pairs of flip flops, $T(x)$ for towels, and $A(x)$ for Afghans. Write simple formulas for each of these.

$$B(x) : \quad F(x) :$$

$$T(x) : \quad A(x) :$$

(b) Let g_n be the the number of different ways for T-Pain to bring n items (burgers, pairs of flip flops, towels, and/or afghans) on his boat trip. Let $G(x)$ be the generating function $\sum_{n=0}^{\infty} g_n x^n$. Verify that

$$G(x) = \frac{x^7}{(1-x)^2}.$$

(c) Find a simple formula for g_n .

IV Probability



Introduction

Probability is one of the most important disciplines in all of the sciences. It is also one of the least well understood.

Probability is especially important in computer science—it arises in virtually every branch of the field. In algorithm design and game theory, for example, *randomized* algorithms and strategies (those that use a random number generator as a key input for decision making) frequently outperform deterministic algorithms and strategies. In information theory and signal processing, an understanding of randomness is critical for filtering out noise and compressing data. In cryptography and digital rights management, probability is crucial for achieving security. The list of examples is long.

Given the impact that probability has on computer science, it seems strange that probability should be so misunderstood by so many. Perhaps the trouble is that basic human intuition is wrong as often as it is right when it comes to problems involving random events. As a consequence, many students develop a fear of probability. Indeed, we have witnessed many graduate oral exams where a student will solve the most horrendous calculation, only to then be tripped up by the simplest probability question. Indeed, even some faculty will start squirming if you ask them a question that starts “What is the probability that...?”

Our goal in the remaining chapters is to equip you with the tools that will enable you to solve basic problems involving probability easily and confidently.

Chapter 17 introduces the basic definitions and an elementary 4-step process that can be used to determine the probability that a specified event occurs. We illustrate the method on two famous problems where your intuition will probably fail you. The key concepts of Conditional probability and independence are introduced, along with examples of their use, and regrettable misuse, in practice: the probability you have a disease given that a diagnostic test says you do, and the probability

that a suspect is guilty given that his blood type matches the blood found at the scene of the crime.

Random variables provide a more quantitative way to measure random events and We study them in Chapter 18. For example, instead of determining the probability that it will rain, we may want to determine *how much* or *how long* it is likely to rain. The fundamental concept of the *expected value* of a random variable is introduced and some of its key properties are developed.

Chapter 19 examines the probability that a random variable deviates significantly from its expected value. Probability of deviation provides the theoretical basis for estimation by sampling which is fundamental in science, engineering, and human affairs. It is also especially important in engineering practice, where things are generally fine if they are going as expected, and you would like to be assured that the probability of an unexpected event is very low.

A final chapter applies the previously probabilistic tools to solve problems involving more complex random processes. You will see why you will probably never get very far ahead at the casino and how two Stanford graduate students became billionaires by combining graph theory and probability theory to design a better search engine for the web.

17 Events and Probability Spaces

17.1 Let's Make a Deal

In the September 9, 1990 issue of *Parade* magazine, columnist Marilyn vos Savant responded to this letter:

Suppose you're on a game show, and you're given the choice of three doors. Behind one door is a car, behind the others, goats. You pick a door, say number 1, and the host, who knows what's behind the doors, opens another door, say number 3, which has a goat. He says to you, "Do you want to pick door number 2?" Is it to your advantage to switch your choice of doors?

Craig. F. Whitaker
Columbia, MD

The letter describes a situation like one faced by contestants in the 1970's game show *Let's Make a Deal*, hosted by Monty Hall and Carol Merrill. Marilyn replied that the contestant should indeed switch. She explained that if the car was behind either of the two unpicked doors—which is twice as likely as the the car being behind the picked door—the contestant wins by switching. But she soon received a torrent of letters, many from mathematicians, telling her that she was wrong. The problem became known as the *Monty Hall Problem* and it generated thousands of hours of heated debate.

This incident highlights a fact about probability: the subject uncovers lots of examples where ordinary intuition leads to completely wrong conclusions. So until you've studied probabilities enough to have refined your intuition, a way to avoid errors is to fall back on a rigorous, systematic approach such as the Four Step Method that we will describe shortly. First, let's make sure we really understand the setup for this problem. This is always a good thing to do when you are dealing with probability.

17.1.1 Clarifying the Problem

Craig's original letter to Marilyn vos Savant is a bit vague, so we must make some assumptions in order to have any hope of modeling the game formally. For example, we will assume that:

1. The car is equally likely to be hidden behind each of the three doors.
2. The player is equally likely to pick each of the three doors, regardless of the car’s location.
3. After the player picks a door, the host *must* open a different door with a goat behind it and offer the player the choice of staying with the original door or switching.
4. If the host has a choice of which door to open, then he is equally likely to select each of them.

In making these assumptions, we’re reading a lot into Craig Whitaker’s letter. There are other plausible interpretations that lead to different answers. But let’s accept these assumptions for now and address the question, “What is the probability that a player who switches wins the car?”

17.2 The Four Step Method

Every probability problem involves some sort of randomized experiment, process, or game. And each such problem involves two distinct challenges:

1. How do we model the situation mathematically?
2. How do we solve the resulting mathematical problem?

In this section, we introduce a four step approach to questions of the form, “What is the probability that...?” In this approach, we build a probabilistic model step-by-step, formalizing the original question in terms of that model. Remarkably, the structured thinking that this approach imposes provides simple solutions to many famously-confusing problems. For example, as you’ll see, the four step method cuts through the confusion surrounding the Monty Hall problem like a Ginsu knife.

17.2.1 Step 1: Find the Sample Space

Our first objective is to identify all the possible outcomes of the experiment. A typical experiment involves several randomly-determined quantities. For example, the Monty Hall game involves three such quantities:

1. The door concealing the car.
2. The door initially chosen by the player.

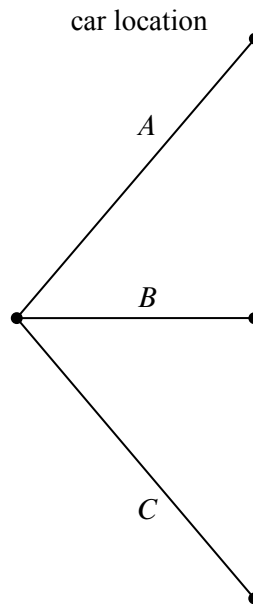


Figure 17.1 The first level in a tree diagram for the Monty Hall Problem. The branches correspond to the door behind which the car is located.

3. The door that the host opens to reveal a goat.

Every possible combination of these randomly-determined quantities is called an *outcome*. The set of all possible outcomes is called the *sample space* for the experiment.

A *tree diagram* is a graphical tool that can help us work through the four step approach when the number of outcomes is not too large or the problem is nicely structured. In particular, we can use a tree diagram to help understand the sample space of an experiment. The first randomly-determined quantity in our experiment is the door concealing the prize. We represent this as a tree with three branches, as shown in Figure 17.1. In this diagram, the doors are called *A*, *B*, and *C* instead of 1, 2, and 3, because we’ll be adding a lot of other numbers to the picture later.

For each possible location of the prize, the player could initially choose any of the three doors. We represent this in a second layer added to the tree. Then a third layer represents the possibilities of the final step when the host opens a door to reveal a goat, as shown in Figure 17.2.

Notice that the third layer reflects the fact that the host has either one choice or two, depending on the position of the car and the door initially selected by the player. For example, if the prize is behind door *A* and the player picks door *B*, then

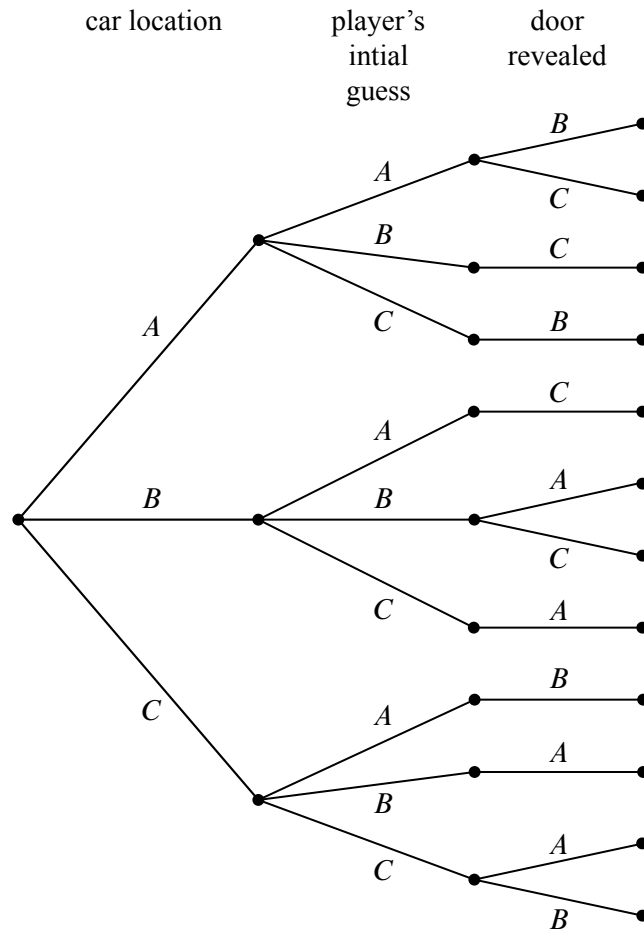


Figure 17.2 The full tree diagram for the Monty Hall Problem. The second level indicates the door initially chosen by the player. The third level indicates the door revealed by Monty Hall.

the host must open door C. However, if the prize is behind door A and the player picks door A, then the host could open either door B or door C.

Now let’s relate this picture to the terms we introduced earlier: the leaves of the tree represent *outcomes* of the experiment, and the set of all leaves represents the *sample space*. Thus, for this experiment, the sample space consists of 12 outcomes. For reference, we’ve labeled each outcome in Figure 17.3 with a triple of doors indicating:

(door concealing prize, door initially chosen, door opened to reveal a goat).

In these terms, the sample space is the set

$$\mathcal{S} = \left\{ \begin{array}{l} (A, A, B), (A, A, C), (A, B, C), (A, C, B), (B, A, C), (B, B, A), \\ (B, B, C), (B, C, A), (C, A, B), (C, B, A), (C, C, A), (C, C, B) \end{array} \right\}$$

The tree diagram has a broader interpretation as well: we can regard the whole experiment as following a path from the root to a leaf, where the branch taken at each stage is “randomly” determined. Keep this interpretation in mind; we’ll use it again later.

17.2.2 Step 2: Define Events of Interest

Our objective is to answer questions of the form “What is the probability that . . . ?”, where, for example, the missing phrase might be “the player wins by switching”, “the player initially picked the door concealing the prize”, or “the prize is behind door C.” Each of these phrases characterizes a set of outcomes. For example, the outcomes specified by “the prize is behind door C” is:

$$\{(C, A, B), (C, B, A), (C, C, A), (C, C, B)\}.$$

A set of outcomes is called an *event* and it is a subset of the sample space. So the event that the player initially picked the door concealing the prize is the set:

$$\{(A, A, B), (A, A, C), (B, B, A), (B, B, C), (C, C, A), (C, C, B)\}.$$

And what we’re really after, the event that the player wins by switching, is the set of outcomes:

$$\begin{aligned} &[\text{switching-wins}] \\ &::= \{(A, B, C), (A, C, B), (B, A, C), (B, C, A), (C, A, B), (C, B, A)\}. \end{aligned} \quad (17.1)$$

These outcomes have check marks in Figure 17.4.

Notice that exactly half of the outcomes are checked, meaning that the player wins by switching in half of all outcomes. You might be tempted to conclude that a player who switches wins with probability $1/2$. *This is wrong*. The reason is that these outcomes are not all equally likely, as we’ll see shortly.

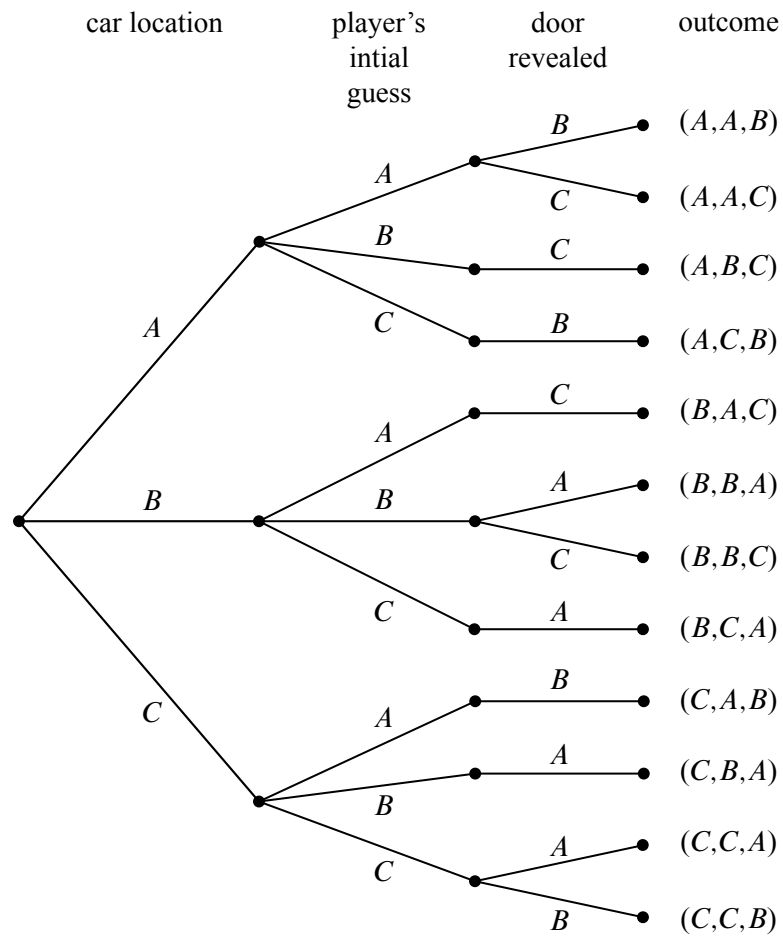


Figure 17.3 The tree diagram for the Monty Hal Problem with the outcomes labeled for each path from root to leaf. For example, outcome (A, A, B) corresponds to the car being behind door A , the player initially choosing door A , and Monty Hall revealing the goat behind door B .

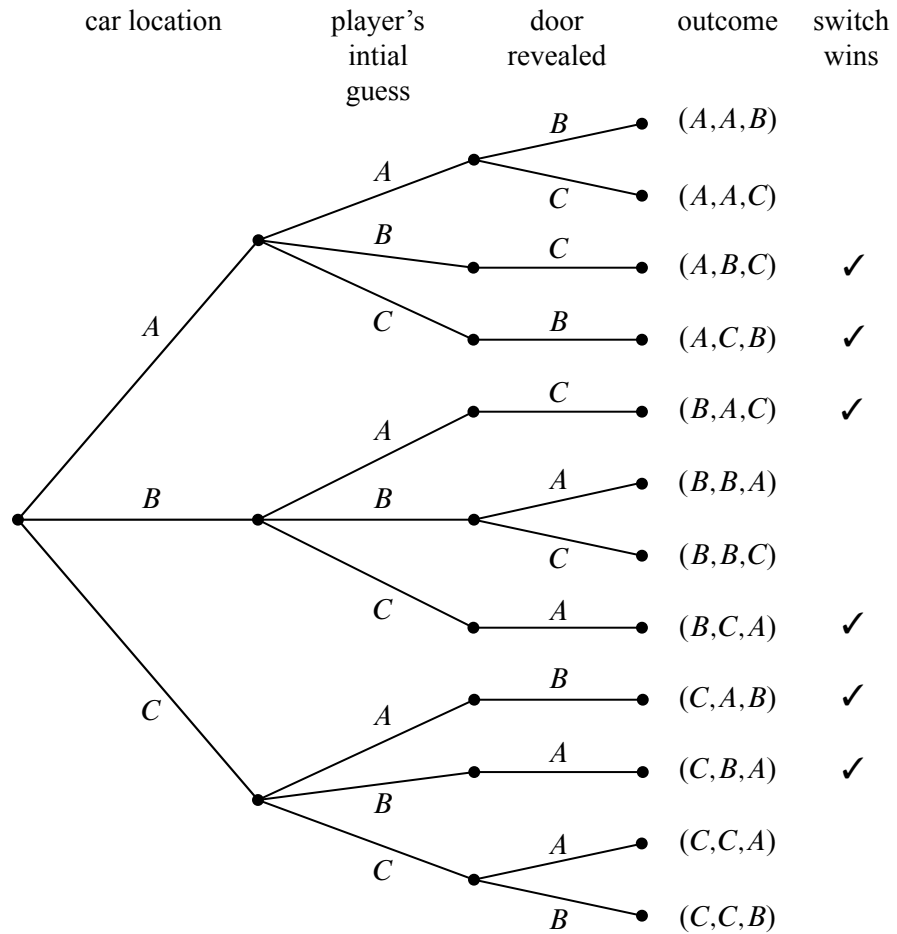


Figure 17.4 The tree diagram for the Monty Hall Problem where the outcomes in the event where the player wins by switching are denoted with a check mark.

17.2.3 Step 3: Determine Outcome Probabilities

So far we’ve enumerated all the possible outcomes of the experiment. Now we must start assessing the likelihood of those outcomes. In particular, the goal of this step is to assign each outcome a probability, indicating the fraction of the time this outcome is expected to occur. The sum of all outcome probabilities must be one, reflecting the fact that there always is an outcome.

Ultimately, outcome probabilities are determined by the phenomenon we’re modeling and thus are not quantities that we can derive mathematically. However, mathematics can help us compute the probability of every outcome *based on fewer and more elementary modeling decisions*. In particular, we’ll break the task of determining outcome probabilities into two stages.

Step 3a: Assign Edge Probabilities

First, we record a probability on each *edge* of the tree diagram. These edge-probabilities are determined by the assumptions we made at the outset: that the prize is equally likely to be behind each door, that the player is equally likely to pick each door, and that the host is equally likely to reveal each goat, if he has a choice. Notice that when the host has no choice regarding which door to open, the single branch is assigned probability 1. For example, see Figure 17.5.

Step 3b: Compute Outcome Probabilities

Our next job is to convert edge probabilities into outcome probabilities. This is a purely mechanical process:

the probability of an outcome is equal to the product of the edge-probabilities on the path from the root to that outcome.

For example, the probability of the topmost outcome in Figure 17.5, (A, A, B) , is

$$\frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{18}.$$

There’s an easy, intuitive justification for this rule. As the steps in an experiment progress randomly along a path from the root of the tree to a leaf, the probabilities on the edges indicate how likely the path is to proceed along each branch. For example, a path starting at the root in our example is equally likely to go down each of the three top-level branches.

How likely is such a path to arrive at the topmost outcome, (A, A, B) ? Well, there is a 1-in-3 chance that a path would follow the A -branch at the top level, a 1-in-3 chance it would continue along the A -branch at the second level, and 1-in-2 chance it would follow the B -branch at the third level. Thus, it seems that

1 path in 18 should arrive at the (A, A, B) leaf, which is precisely the probability we assign it.

We have illustrated all of the outcome probabilities in Figure 17.5.

Specifying the probability of each outcome amounts to defining a function that maps each outcome to a probability. This function is usually called $\Pr[\cdot]$. In these terms, we’ve just determined that:

$$\begin{aligned}\Pr[(A, A, B)] &= \frac{1}{18}, \\ \Pr[(A, A, C)] &= \frac{1}{18}, \\ \Pr[(A, B, C)] &= \frac{1}{9}, \\ &\text{etc.}\end{aligned}$$

17.2.4 Step 4: Compute Event Probabilities

We now have a probability for each *outcome*, but we want to determine the probability of an *event*. The probability of an event E is denoted by $\Pr[E]$ and it is the sum of the probabilities of the outcomes in E . For example, the probability of the [switching wins] event (17.1) is

$$\begin{aligned}\Pr[\text{switching wins}] &= \Pr[(A, B, C)] + \Pr[(A, C, B)] + \Pr[(B, A, C)] + \\ &\quad \Pr[(B, C, A)] + \Pr[(C, A, B)] + \Pr[(C, B, A)] \\ &= \frac{1}{9} + \frac{1}{9} + \frac{1}{9} + \frac{1}{9} + \frac{1}{9} + \frac{1}{9} \\ &= \frac{2}{3}.\end{aligned}$$

It seems Marilyn’s answer is correct! A player who switches doors wins the car with probability $2/3$. In contrast, a player who stays with his or her original door wins with probability $1/3$, since staying wins if and only if switching loses.

We’re done with the problem! We didn’t need any appeals to intuition or ingenious analogies. In fact, no mathematics more difficult than adding and multiplying fractions was required. The only hard part was resisting the temptation to leap to an “intuitively obvious” answer.

17.2.5 An Alternative Interpretation of the Monty Hall Problem

Was Marilyn really right? Our analysis indicates that she was. But a more accurate conclusion is that her answer is correct *provided we accept her interpretation of the*

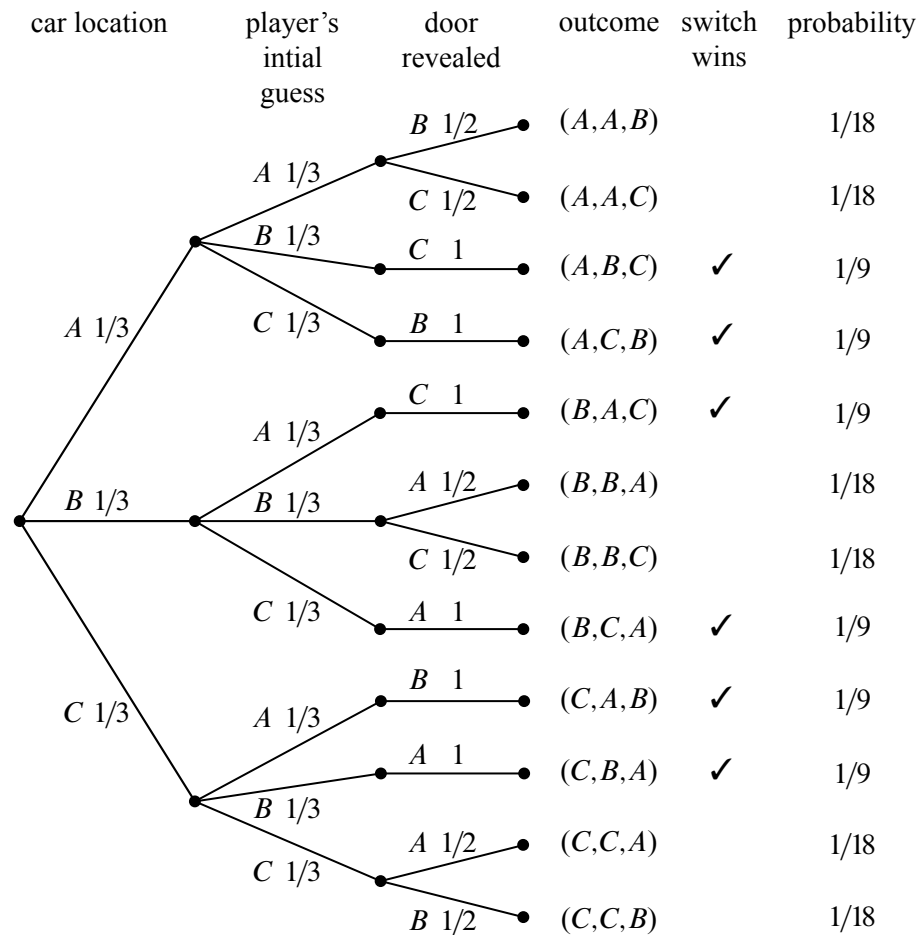


Figure 17.5 The tree diagram for the Monty Hall Problem where edge weights denote the probability of that branch being taken given that we are at the parent of that branch. For example, if the car is behind door A , then there is a $1/3$ chance that the player's initial selection is door B . The rightmost column shows the outcome probabilities for the Monty Hall Problem. Each outcome probability is simply the product of the probabilities on the path from the root to the outcome leaf.

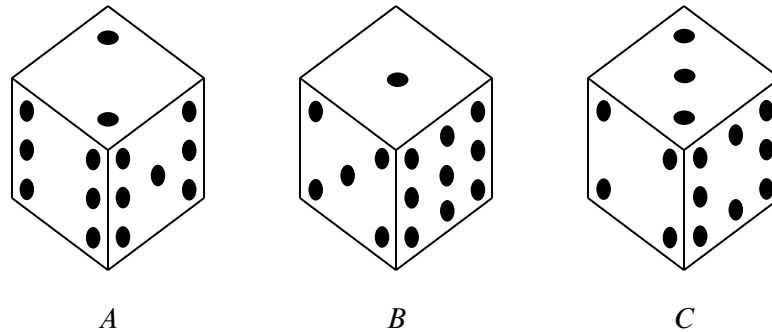


Figure 17.6 The strange dice. The number of pips on each concealed face is the same as the number on the opposite face. For example, when you roll die *A*, the probabilities of getting a 2, 6, or 7 are each $1/3$.

question. There is an equally plausible interpretation in which Marilyn’s answer is wrong. Notice that Craig Whitaker’s original letter does not say that the host is *required* to reveal a goat and offer the player the option to switch, merely that he *did* these things. In fact, on the *Let’s Make a Deal* show, Monty Hall sometimes simply opened the door that the contestant picked initially. Therefore, if he wanted to, Monty could give the option of switching only to contestants who picked the correct door initially. In this case, switching never works!

17.3 Strange Dice

The four-step method is surprisingly powerful. Let’s get some more practice with it. Imagine, if you will, the following scenario.

It’s a typical Saturday night. You’re at your favorite pub, contemplating the true meaning of infinite cardinalities, when a burly-looking biker plops down on the stool next to you. Just as you are about to get your mind around $\mathcal{P}(\mathcal{P}(\mathbb{R}))$, biker dude slaps three strange-looking dice on the bar and challenges you to a \$100 wager. His rules are simple. Each player selects one die and rolls it once. The player with the lower value pays the other player \$100.

Naturally, you are skeptical, especially after you see that these are not ordinary dice. Each die has the usual six sides, but opposite sides have the same number on them, and the numbers on the dice are different, as shown in Figure 17.6.

Biker dude notices your hesitation, so he sweetens his offer: he will pay you \$105 if you roll the higher number, but you only need pay him \$100 if he rolls

higher, *and* he will let you pick a die first, after which he will pick one of the other two. The sweetened deal sounds persuasive since it gives you a chance to pick what you think is the best die, so you decide you will play. But which of the dice should you choose? Die *B* is appealing because it has a 9, which is a sure winner if it comes up. Then again, die *A* has two fairly large numbers and die *C* has an 8 and no really small values.

In the end, you choose die *B* because it has a 9, and then biker dude selects die *A*. Let’s see what the probability is that you will win. (Of course, you probably should have done this before picking die *B* in the first place.) Not surprisingly, we will use the four-step method to compute this probability.

17.3.1 Die *A* versus Die *B*

Step 1: Find the sample space.

The tree diagram for this scenario is shown in Figure 17.7. In particular, the sample space for this experiment are the nine pairs of values that might be rolled with Die *A* and Die *B*:

For this experiment, the sample space is a set of nine outcomes:

$$\mathcal{S} = \{ (2, 1), (2, 5), (2, 9), (6, 1), (6, 5), (6, 9), (7, 1), (7, 5), (7, 9) \}.$$

Step 2: Define events of interest.

We are interested in the event that the number on die *A* is greater than the number on die *B*. This event is a set of five outcomes:

$$\{ (2, 1), (6, 1), (6, 5), (7, 1), (7, 5) \}.$$

These outcomes are marked *A* in the tree diagram in Figure 17.7.

Step 3: Determine outcome probabilities.

To find outcome probabilities, we first assign probabilities to edges in the tree diagram. Each number on each die comes up with probability $1/3$, regardless of the value of the other die. Therefore, we assign all edges probability $1/3$. The probability of an outcome is the product of the probabilities on the corresponding root-to-leaf path, which means that every outcome has probability $1/9$. These probabilities are recorded on the right side of the tree diagram in Figure 17.7.

Step 4: Compute event probabilities.

The probability of an event is the sum of the probabilities of the outcomes in that event. In this case, all the outcome probabilities are the same, so we say that the sample space is *uniform*. Computing event probabilities for uniform sample spaces

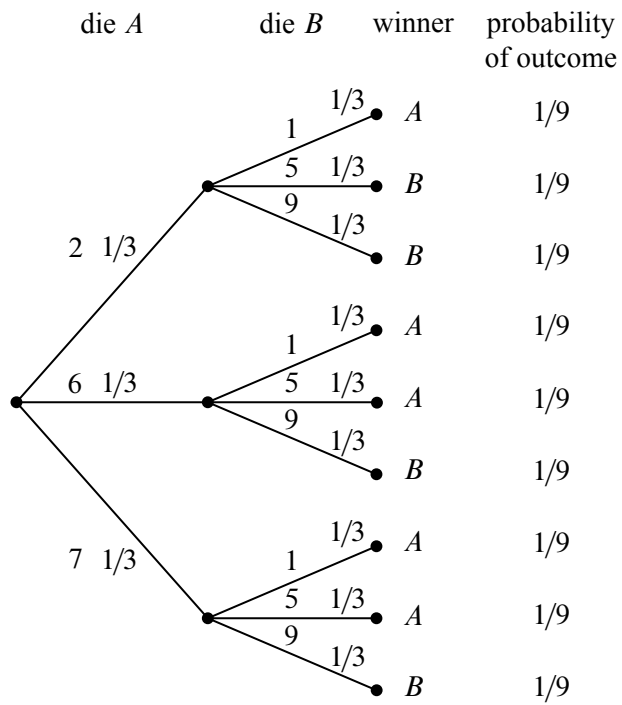


Figure 17.7 The tree diagram for one roll of die *A* versus die *B*. Die *A* wins with probability $\frac{5}{9}$.

is particularly easy since you just have to compute the number of outcomes in the event. In particular, for any event E in a uniform sample space S ,

$$\Pr[E] = \frac{|E|}{|S|}. \quad (17.2)$$

In this case, E is the event that die A beats die B , so $|E| = 5$, $|S| = 9$, and

$$\Pr[E] = 5/9.$$

This is bad news for you. Die A beats die B more than half the time and, not surprisingly, you just lost \$100.

Biker dude consoles you on your “bad luck” and, given that he’s a sensitive guy beneath all that leather, he offers to go double or nothing.¹ Given that your wallet only has \$25 in it, this sounds like a good plan. Plus, you figure that choosing die A will give *you* the advantage.

So you choose A , and then biker dude chooses C . Can you guess who is more likely to win? (Hint: it is generally not a good idea to gamble with someone you don’t know in a bar, especially when you are gambling with strange dice.)

17.3.2 Die A versus Die C

We can construct the tree diagram and outcome probabilities as before. The result is shown in Figure 17.8 and there is bad news again. Die C will beat die A with probability $5/9$, and you lose once again.

You now owe the biker dude \$200 and he asks for his money. You reply that you need to go to the bathroom.

17.3.3 Die B versus Die C

Being a sensitive guy, biker dude nods understandingly and offers yet another wager. This time, he’ll let you have die C . He’ll even let you raise the wager to \$200 so you can win your money back.

This is too good a deal to pass up. You know that die C is likely to beat die A and that die A is likely to beat die B , and so die C is *surely* the best. Whether biker dude picks A or B , the odds would be in your favor this time. Biker dude must really be a nice guy.

So you pick C , and then biker dude picks B . Wait, how come you haven’t caught on yet and worked out the tree diagram before you took this bet :-)? If

¹*Double or nothing* is slang for doing another wager after you have lost the first. If you lose again, you will owe biker dude *double* what you owed him before. If you win, you will owe him *nothing*; in fact, since he should pay you \$210 if he loses, you would come out \$10 ahead.

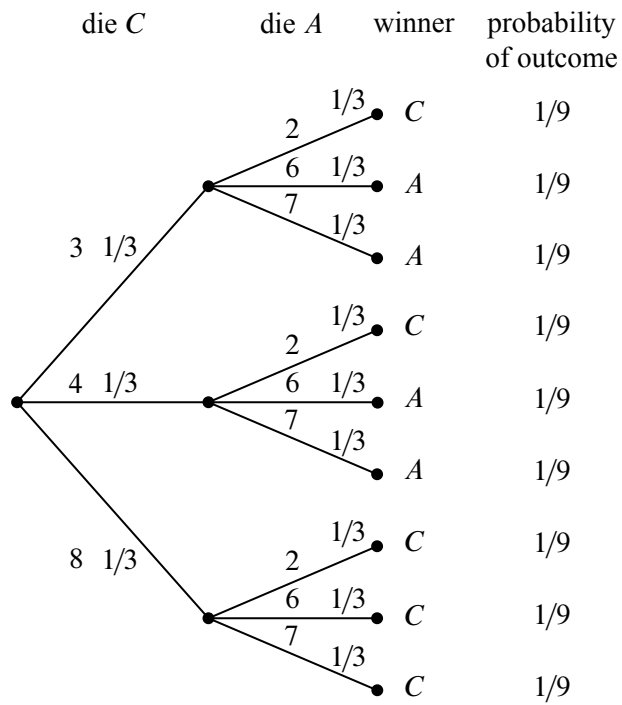


Figure 17.8 The tree diagram for one roll of die *C* versus die *A*. Die *C* wins with probability $5/9$.

you do it now, you’ll see by the same reasoning as before that B beats C with probability $5/9$. But surely there is a mistake! How is it possible that

C beats A with probability $5/9$,
 A beats B with probability $5/9$,
 B beats C with probability $5/9$?

The problem is not with the math, but with your intuition. Since A will beat B more often than not, and B will beat C more often than not, it *seems* like A ought to beat C more often than not, that is, the “beats more often” relation ought to be *transitive*. But this intuitive idea is simply false: whatever die you pick, biker dude can pick one of the others and be likely to win. So picking first is actually a big disadvantage, and as a result, you now owe biker dude \$400.

Just when you think matters can’t get worse, biker dude offers you one final wager for \$1,000. This time, instead of rolling each die once, you will each roll your die twice, and your score is the sum of your rolls, and he will even let you pick your die second, that is, after he picks his. Biker dude chooses die B . Now you know that die A will beat die B with probability $5/9$ on one roll, so, jumping at this chance to get ahead, you agree to play, and you pick die A . After all, you figure that since a roll of die A beats a roll of die B more often than not, two rolls of die A are even more likely to beat two rolls of die B , right?

Wrong! (Did we mention that playing strange gambling games with strangers in a bar is a bad idea?)

17.3.4 Rolling Twice

If each player rolls twice, the tree diagram will have four levels and $3^4 = 81$ outcomes. This means that it will take a while to write down the entire tree diagram. But it’s easy to write down the first two levels as in Figure 17.9(a) and then notice that the remaining two levels consist of nine identical copies of the tree in Figure 17.9(b).

The probability of each outcome is $(1/3)^4 = 1/81$ and so, once again, we have a uniform probability space. By equation (17.2), this means that the probability that A wins is the number of outcomes where A beats B divided by 81.

To compute the number of outcomes where A beats B , we observe that the sum of the two rolls of die A is equally likely to be any element of the following multiset:

$$\mathcal{S}_A = \{4, 8, 8, 9, 9, 12, 13, 13, 14\}.$$

The sum of two rolls of die B is equally likely to be any element of the following multiset:

$$\mathcal{S}_B = \{2, 6, 6, 10, 10, 10, 14, 14, 18\}.$$

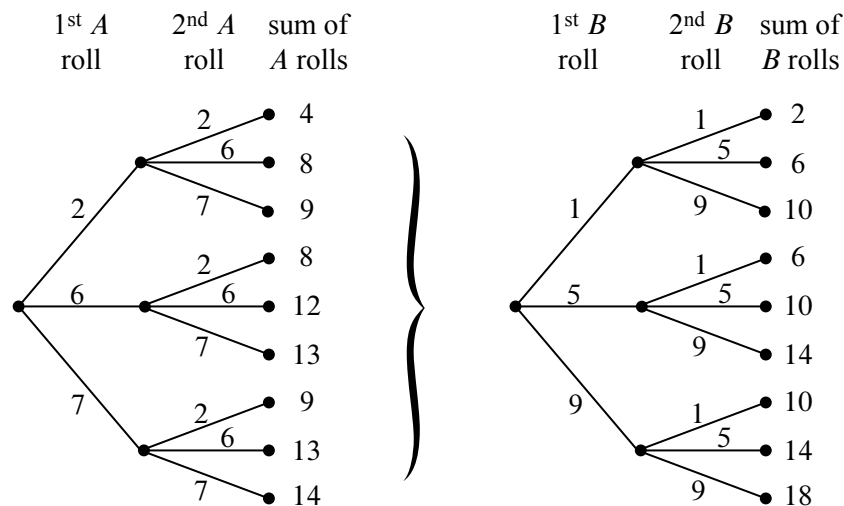


Figure 17.9 Parts of the tree diagram for die B versus die A where each die is rolled twice. The first two levels are shown in (a). The last two levels consist of nine copies of the tree in (b).

We can treat each outcome as a pair $(x, y) \in \mathcal{S}_A \times \mathcal{S}_B$, where A wins iff $x > y$. If $x = 4$, there is only one y (namely $y = 2$) for which $x > y$. If $x = 8$, there are three values of y for which $x > y$. Continuing the count in this way, the number of pairs for which $x > y$ is

$$1 + 3 + 3 + 3 + 3 + 6 + 6 + 6 + 6 = 37.$$

A similar count shows that there are 42 pairs for which $x < y$, and there are two pairs $((14, 14), (14, 14))$ which result in ties. This means that A loses to B with probability $42/81 > 1/2$ and ties with probability $2/81$. Die A wins with probability only $37/81$.

How can it be that A is more likely than B to win with one roll, but B is more likely to win with two rolls? Well, why not? The only reason we’d think otherwise is our unreliable, untrained intuition. (Even the authors were surprised when they first learned about this, but at least we didn’t lose \$1400 to biker dude. :-)) In fact, the die strength reverses no matter which two die we picked. So for 1 roll,

$$A \succ B \succ C \succ A,$$

but for two rolls,

$$A \prec B \prec C \prec A,$$

where we have used the symbols $\succ$ and $\prec$ to denote which die is more likely to result in the larger value.

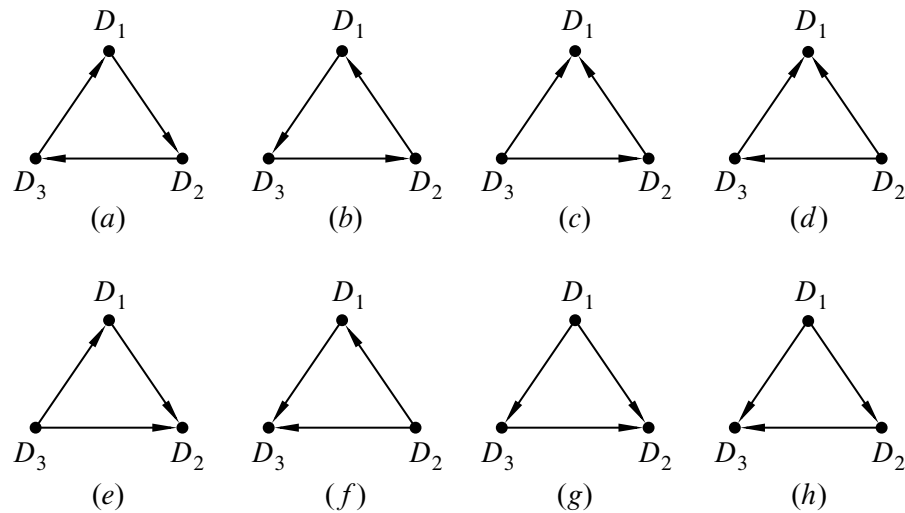


Figure 17.10 All possible relative strengths for three dice D_1 , D_2 , and D_3 . The edge $\langle D_i \rightarrow D_j \rangle$ denotes that the sum of rolls for D_i is likely to be greater than the sum of rolls for D_j .

Even Stranger Dice

The weird behavior of the three strange dice above generalizes in a remarkable way.² The idea is that you can find arbitrarily large sets of dice which will beat each other in any desired pattern according to how many times the dice are rolled. The precise statement of this result involves several alternations of universal and existential quantifiers, so it may take a few readings to understand what it is saying:

Theorem 17.3.1. *For any $n \geq 2$, there is a set of n dice with the following property: for any n -node digraph with exactly one directed edge between every two distinct nodes,³ there is a number of rolls k such that the sum of k rolls of the i th die is bigger than the sum for the j th die with probability greater than $1/2$ iff there is an edge from the i th to the j th node in the graph.*

For example, the eight possible relative strengths for $n = 3$ dice are shown in Figure 17.10.

Our analysis for the dice in Figure 17.6 showed that for 1 roll, we have the relative strengths shown in Figure 17.10(a), and for two rolls, we have the (reverse) relative strengths shown in Figure 17.10(b). If you are prone to gambling with

²Reference Ron Graham paper.

³In other words, for every pair of nodes $u \neq v$, either $\langle u \rightarrow v \rangle$ or $\langle v \rightarrow u \rangle$, but not both, are edges of the graph. Such graphs are called *tournament graphs*, see Problem 9.5.

strangers in bars, it would be a good idea to try figuring out what other relative strengths are possible for the dice in Figure 17.6 when using more rolls.

17.4 Set Theory and Probability

Let’s abstract what we’ve just done with the Monty Hall and strange dice examples into a general mathematical definition of sample spaces and probability.

17.4.1 Probability Spaces

Definition 17.4.1. A countable *sample space* $\mathcal{S}$ is a nonempty countable set.⁴ An element $\omega \in \mathcal{S}$ is called an *outcome*. A subset of $\mathcal{S}$ is called an *event*.

Definition 17.4.2. A *probability function* on a sample space $\mathcal{S}$ is a total function $\Pr : \mathcal{S} \rightarrow \mathbb{R}$ such that

- $\Pr[\omega] \geq 0$ for all $\omega \in \mathcal{S}$, and
- $\sum_{\omega \in \mathcal{S}} \Pr[\omega] = 1$.

A sample space together with a probability function is called a *probability space*. For any event $E \subseteq \mathcal{S}$, the *probability of E* is defined to be the sum of the probabilities of the outcomes in E :

$$\Pr[E] ::= \sum_{\omega \in E} \Pr[\omega].$$

In the previous examples there were only finitely many possible outcomes, but we’ll quickly come to examples that have a countably infinite number of outcomes.

The study of probability is closely tied to set theory because any set can be a sample space and any subset can be an event. General probability theory deals with uncountable sets like the set of real numbers, but we won’t need these, and sticking to countable sets lets us define the probability of events using sums instead of integrals. It also lets us avoid some distracting technical problems in set theory like the Banach-Tarski “paradox” mentioned in Chapter 5.

17.4.2 Probability Rules from Set Theory

Most of the rules and identities that we have developed for finite sets extend very naturally to probability.

⁴Yes, sample spaces can be infinite. If you did not read Chapter 5, don’t worry —*countable* just means that you can list the elements of the sample space as $\omega_0, \omega_1, \omega_2, \dots$

An immediate consequence of the definition of event probability is that for *disjoint* events E and F ,

$$\Pr[E \cup F] = \Pr[E] + \Pr[F].$$

This generalizes to a countable number of events, as follows.

Rule 17.4.3 (Sum Rule). *If $\{E_0, E_1, \dots\}$ is collection of disjoint events, then*

$$\Pr\left[\bigcup_{n \in \mathbb{N}} E_n\right] = \sum_{n \in \mathbb{N}} \Pr[E_n].$$

The Sum Rule lets us analyze a complicated event by breaking it down into simpler cases. For example, if the probability that a randomly chosen MIT student is native to the United States is 60%, to Canada is 5%, and to Mexico is 5%, then the probability that a random MIT student is native to North America is 70%.

Another consequence of the Sum Rule is that $\Pr[A] + \Pr[\bar{A}] = 1$, which follows because $\Pr[S] = 1$ and S is the union of the disjoint sets A and $\bar{A}$. This equation often comes up in the form:

$$\Pr[\bar{A}] = 1 - \Pr[A]. \quad (\text{Complement Rule})$$

Sometimes the easiest way to compute the probability of an event is to compute the probability of its complement and then apply this formula.

Some further basic facts about probability parallel facts about cardinalities of finite sets. In particular:

$$\Pr[B - A] = \Pr[B] - \Pr[A \cap B], \quad (\text{Difference Rule})$$

$$\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B], \quad (\text{Inclusion-Exclusion})$$

$$\Pr[A \cup B] \leq \Pr[A] + \Pr[B], \quad (\text{Boole's Inequality})$$

$$\text{If } A \subseteq B, \text{ then } \Pr[A] \leq \Pr[B]. \quad (\text{Monotonicity Rule})$$

The Difference Rule follows from the Sum Rule because B is the union of the disjoint sets $B - A$ and $A \cap B$. Inclusion-Exclusion then follows from the Sum and Difference Rules, because $A \cup B$ is the union of the disjoint sets A and $B - A$. Boole's inequality is an immediate consequence of Inclusion-Exclusion since probabilities are nonnegative. Monotonicity follows from the definition of event probability and the fact that outcome probabilities are nonnegative.

The two-event Inclusion-Exclusion equation above generalizes to n events in the same way as the corresponding Inclusion-Exclusion rule for n sets. Boole's inequality also generalizes to

Rule 17.4.4 (Union Bound).

$$\Pr[E_1 \cup \cdots \cup E_n] \leq \Pr[E_1] + \cdots + \Pr[E_n]. \quad (17.3)$$

This simple Union Bound is useful in many calculations. For example, suppose that E_i is the event that the i -th critical component in a spacecraft fails. Then $E_1 \cup \cdots \cup E_n$ is the event that *some* critical component fails. If $\sum_{i=1}^n \Pr[E_i]$ is small, then the Union Bound can give an adequate upper bound on this vital probability.

17.4.3 Uniform Probability Spaces

Definition 17.4.5. A finite probability space, $\mathcal{S}$, is said to be *uniform* if $\Pr[\omega]$ is the same for every outcome $\omega \in \mathcal{S}$.

As we saw in the strange dice problem, uniform sample spaces are particularly easy to work with. That’s because for any event $E \subseteq \mathcal{S}$,

$$\Pr[E] = \frac{|E|}{|\mathcal{S}|}. \quad (17.4)$$

This means that once we know the cardinality of E and $\mathcal{S}$, we can immediately obtain $\Pr[E]$. That’s great news because we developed lots of tools for computing the cardinality of a set in Part III.

For example, suppose that you select five cards at random from a standard deck of 52 cards. What is the probability of having a full house? Normally, this question would take some effort to answer. But from the analysis in Section 15.9.2, we know that

$$|\mathcal{S}| = \binom{52}{5}$$

and

$$|E| = 13 \cdot \binom{4}{3} \cdot 12 \cdot \binom{4}{2}$$

where E is the event that we have a full house. Since every five-card hand is equally likely, we can apply equation (17.4) to find that

$$\begin{aligned} \Pr[E] &= \frac{13 \cdot 12 \cdot \binom{4}{3} \cdot \binom{4}{2}}{\binom{52}{5}} \\ &= \frac{13 \cdot 12 \cdot 4 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2}{52 \cdot 51 \cdot 50 \cdot 49 \cdot 48} = \frac{18}{12495} \\ &\approx \frac{1}{694}. \end{aligned}$$

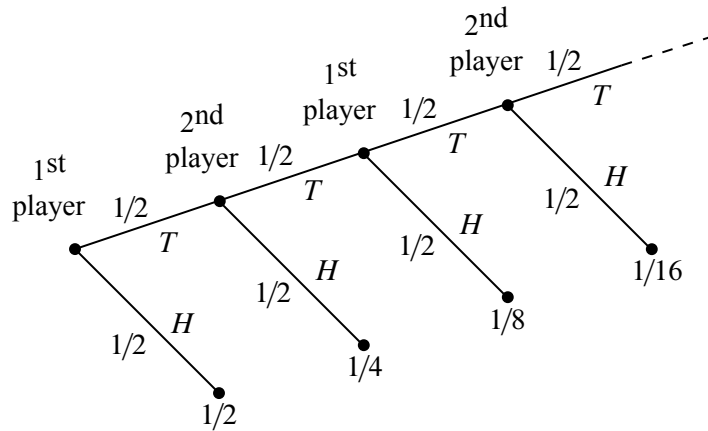


Figure 17.11 The tree diagram for the game where players take turns flipping a fair coin. The first player to flip heads wins.

17.4.4 Infinite Probability Spaces

Infinite probability spaces are fairly common. For example, two players take turns flipping a fair coin. Whoever flips heads first is declared the winner. What is the probability that the first player wins? A tree diagram for this problem is shown in Figure 17.11.

The event that the first player wins contains an infinite number of outcomes, but we can still sum their probabilities:

$$\begin{aligned} \Pr[\text{first player wins}] &= \frac{1}{2} + \frac{1}{8} + \frac{1}{32} + \frac{1}{128} + \cdots \\ &= \frac{1}{2} \sum_{n=0}^{\infty} \left(\frac{1}{4}\right)^n \\ &= \frac{1}{2} \left(\frac{1}{1 - 1/4}\right) = \frac{2}{3}. \end{aligned}$$

Similarly, we can compute the probability that the second player wins:

$$\Pr[\text{second player wins}] = \frac{1}{4} + \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \cdots = \frac{1}{3}.$$

In this case, the sample space is the infinite set

$$\mathcal{S} ::= \{T^n H \mid n \in \mathbb{N}\},$$

where T^n stands for a length n string of T's. The probability function is

$$\Pr[T^n H] ::= \frac{1}{2^{n+1}}.$$

To verify that this is a probability space, we just have to check that all the probabilities are nonnegative and that they sum to 1. Nonnegativity is obvious, and applying the formula for the sum of a geometric series, we find that

$$\sum_{n \in \mathbb{N}} \Pr[T^n H] = \sum_{n \in \mathbb{N}} \frac{1}{2^{n+1}} = 1.$$

Notice that this model does not have an outcome corresponding to the possibility that both players keep flipping tails forever—in the diagram, flipping forever corresponds to following the infinite path in the tree without ever reaching a leaf/outcome. If leaving this possibility out of the model bothers you, you're welcome to fix it by adding another outcome, ω_{forever} , to indicate that that's what happened. Of course since the probabilities of the other outcomes already sum to 1, you have to define the probability of ω_{forever} to be 0. Now outcomes with probability zero will have no impact on our calculations, so there's no harm in adding it in if it makes you happier. On the other hand, in countable probability spaces it isn't necessary to have outcomes with probability zero, and we will generally ignore them.

17.5 Conditional Probability

Suppose that we pick a random person in the world. Everyone has an equal chance of being selected. Let A be the event that the person is an MIT student, and let B be the event that the person lives in Cambridge. What are the probabilities of these events? Intuitively, we're picking a random point in the big ellipse shown in Figure 17.12 and asking how likely that point is to fall into region A or B .

The vast majority of people in the world neither live in Cambridge nor are MIT students, so events A and B both have low probability. But what about the probability that a person is an MIT student, *given* that the person lives in Cambridge? This should be much greater—but what is it exactly?

What we're asking for is called a *conditional probability*; that is, the probability that one event happens, given that some other event definitely happens. Questions about conditional probabilities come up all the time:

- What is the probability that it will rain this afternoon, given that it is cloudy this morning?

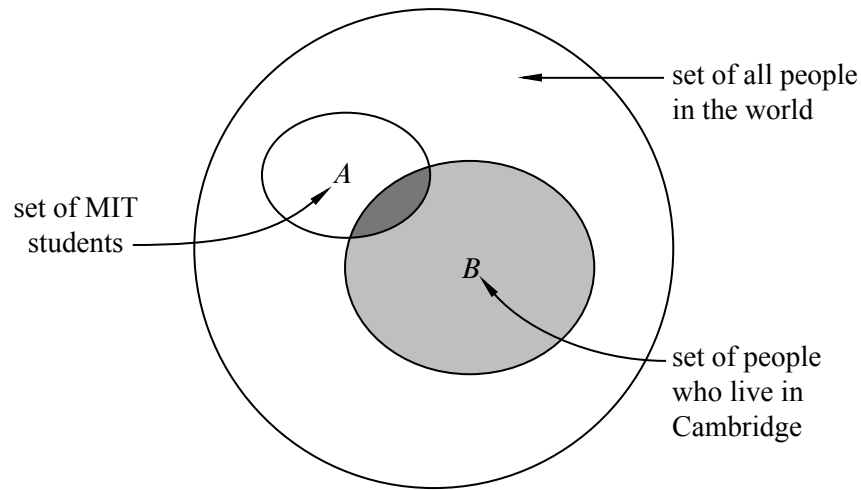


Figure 17.12 Selecting a random person. A is the event that the person is an MIT student. B is the event that the person lives in Cambridge.

- What is the probability that two rolled dice sum to 10, given that both are odd?
- What is the probability that I’ll get four-of-a-kind in Texas No Limit Hold ‘Em Poker, given that I’m initially dealt two queens?

There is a special notation for conditional probabilities. In general, $\Pr[A \mid B]$ denotes the probability of event A , given that event B happens. So, in our example, $\Pr[A \mid B]$ is the probability that a random person is an MIT student, given that he or she is a Cambridge resident.

How do we compute $\Pr[A \mid B]$? Since we are *given* that the person lives in Cambridge, we can forget about everyone in the world who does not. Thus, all outcomes outside event B are irrelevant. So, intuitively, $\Pr[A \mid B]$ should be the fraction of Cambridge residents that are also MIT students; that is, the answer should be the probability that the person is in set $A \cap B$ (the darkly shaded region in Figure 17.12) divided by the probability that the person is in set B (the lightly shaded region). This motivates the definition of conditional probability:

Definition 17.5.1.

$$\Pr[A \mid B] ::= \frac{\Pr[A \cap B]}{\Pr[B]}$$

If $\Pr[B] = 0$, then the conditional probability $\Pr[A \mid B]$ is undefined.

Pure probability is often counterintuitive, but conditional probability is even worse! Conditioning can subtly alter probabilities and produce unexpected results in randomized algorithms and computer systems as well as in betting games. Yet, the mathematical definition of conditional probability given above is very simple and should give you no trouble—provided that you rely on mathematical reasoning and not intuition. The four-step method will also be very helpful as we will see in the next examples.

17.5.1 The Four-Step Method for Conditional Probability: The “Halting Problem”

The *Halting Problem* was the first example of a property that could not be tested by any program. It was introduced by Alan Turing in his seminal 1936 paper. The problem is to determine whether a Turing machine halts on a given . . . yadda yadda yadda . . . more importantly, it was *the name of the MIT EECS department’s famed C-league hockey team*.

In a best-of-three tournament, the Halting Problem wins the first game with probability $1/2$. In subsequent games, their probability of winning is determined by the outcome of the previous game. If the Halting Problem won the previous game, then they are invigorated by victory and win the current game with probability $2/3$. If they lost the previous game, then they are demoralized by defeat and win the current game with probability only $1/3$. What is the probability that the Halting Problem wins the tournament, given that they win the first game?

This is a question about a conditional probability. Let A be the event that the Halting Problem wins the tournament, and let B be the event that they win the first game. Our goal is then to determine the conditional probability $\Pr[A \mid B]$.

We can tackle conditional probability questions just like ordinary probability problems: using a tree diagram and the four step method. A complete tree diagram is shown in Figure 17.13.

Step 1: Find the Sample Space

Each internal vertex in the tree diagram has two children, one corresponding to a win for the Halting Problem (labeled W) and one corresponding to a loss (labeled L). The complete sample space is:

$$\mathcal{S} = \{WW, WLW, WLL, LWW, LWL, LL\}.$$

Step 2: Define Events of Interest

The event that the Halting Problem wins the whole tournament is:

$$T = \{WW, WLW, LWW\}.$$

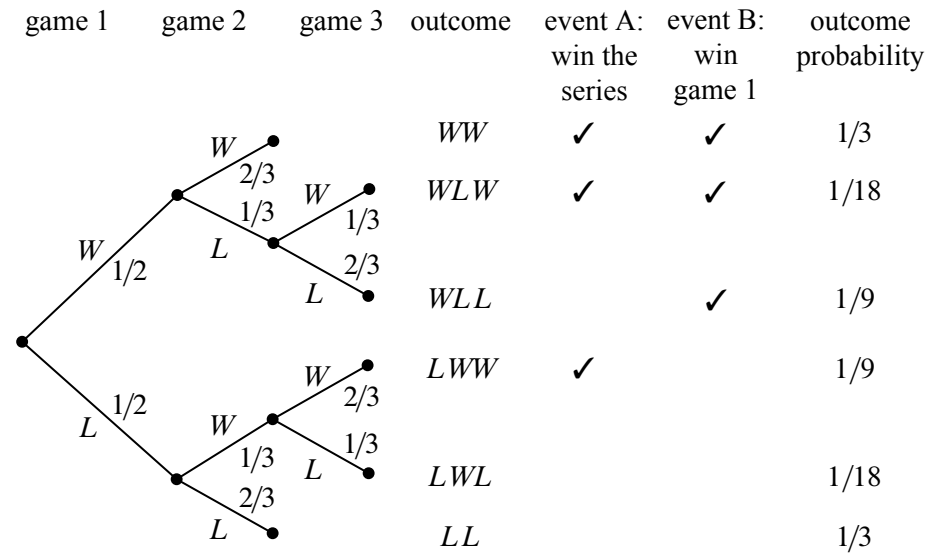


Figure 17.13 The tree diagram for computing the probability that the “Halting Problem” wins two out of three games given that they won the first game.

And the event that the Halting Problem wins the first game is:

$$F = \{WW, WLW, WLL\}.$$

The outcomes in these events are indicated with check marks in the tree diagram in Figure 17.13.

Step 3: Determine Outcome Probabilities

Next, we must assign a probability to each outcome. We begin by labeling edges as specified in the problem statement. Specifically, The Halting Problem has a $1/2$ chance of winning the first game, so the two edges leaving the root are each assigned probability $1/2$. Other edges are labeled $1/3$ or $2/3$ based on the outcome of the preceding game. We then find the probability of each outcome by multiplying all probabilities along the corresponding root-to-leaf path. For example, the probability of outcome WLL is:

$$\frac{1}{2} \cdot \frac{1}{3} \cdot \frac{2}{3} = \frac{1}{9}.$$

Step 4: Compute Event Probabilities

We can now compute the probability that The Halting Problem wins the tournament, given that they win the first game:

$$\begin{aligned}\Pr[A \mid B] &= \frac{\Pr[A \cap B]}{\Pr[B]} \\ &= \frac{\Pr[\{WW, WLW\}]}{\Pr[\{WW, WLW, WLL\}]} \\ &= \frac{1/3 + 1/18}{1/3 + 1/18 + 1/9} \\ &= \frac{7}{9}.\end{aligned}$$

We’re done! If the Halting Problem wins the first game, then they win the whole tournament with probability 7/9.

17.5.2 Why Tree Diagrams Work

We’ve now settled into a routine of solving probability problems using tree diagrams. But we’ve left a big question unaddressed: what is the mathematical justification behind those funny little pictures? Why do they work?

The answer involves conditional probabilities. In fact, the probabilities that we’ve been recording on the edges of tree diagrams *are* conditional probabilities. For example, consider the uppermost path in the tree diagram for the Halting Problem, which corresponds to the outcome WW . The first edge is labeled $1/2$, which is the probability that the Halting Problem wins the first game. The second edge is labeled $2/3$, which is the probability that the Halting Problem wins the second game, *given* that they won the first—that’s a conditional probability! More generally, on each edge of a tree diagram, we record the probability that the experiment proceeds along that path, given that it reaches the parent vertex.

So we’ve been using conditional probabilities all along. But why can we multiply edge probabilities to get outcome probabilities? For example, we concluded that:

$$\Pr[WW] = \frac{1}{2} \cdot \frac{2}{3} = \frac{1}{3}.$$

Why is this correct?

The answer goes back to Definition 17.5.1 of conditional probability which could be written in a form called the *Product Rule* for probabilities:

Rule (Product Rule: 2 Events). *If $\Pr[E_1] \neq 0$, then:*

$$\Pr[E_1 \cap E_2] = \Pr[E_1] \cdot \Pr[E_2 \mid E_1].$$

Multiplying edge probabilities in a tree diagram amounts to evaluating the right side of this equation. For example:

$$\begin{aligned} & \Pr[\text{win first game} \cap \text{win second game}] \\ &= \Pr[\text{win first game}] \cdot \Pr[\text{win second game} \mid \text{win first game}] \\ &= \frac{1}{2} \cdot \frac{2}{3}. \end{aligned}$$

So the Product Rule is the formal justification for multiplying edge probabilities to get outcome probabilities! Of course to justify multiplying edge probabilities along longer paths, we need a Product Rule for n events.

Rule (Product Rule: n Events).

$$\begin{aligned} \Pr[E_1 \cap E_2 \cap \dots \cap E_n] &= \Pr[E_1] \cdot \Pr[E_2 \mid E_1] \cdot \Pr[E_3 \mid E_1 \cap E_2] \cdots \\ &\quad \cdot \Pr[E_n \mid E_1 \cap E_2 \cap \dots \cap E_{n-1}] \end{aligned}$$

provided that

$$\Pr[E_1 \cap E_2 \cap \dots \cap E_{n-1}] \neq 0.$$

This rule follows by routine induction from the definition of conditional probability.

17.5.3 Medical Testing

There is an unpleasant condition called *BO* suffered by 10% of the population. There are no prior symptoms; victims just suddenly start to stink. Fortunately, there is a test for latent *BO* before things start to smell. The test is not perfect, however:

- If you have the condition, there is a 10% chance that the test will say you do not have it. These are called “false negatives.”
- If you do not have the condition, there is a 30% chance that the test will say you do. These are “false positives.”

Suppose a random person is tested for latent *BO*. If the test is positive, then what is the probability that the person has the condition?

Step 1: Find the Sample Space

The sample space is found with the tree diagram in Figure 17.14.

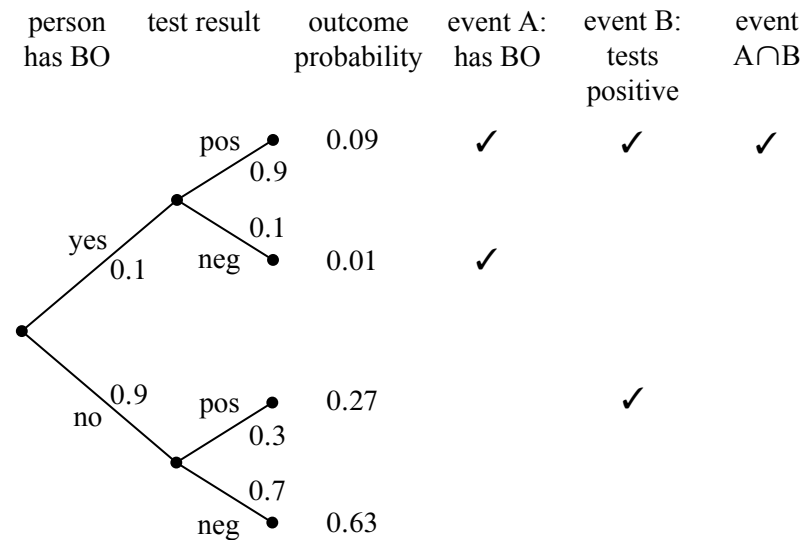


Figure 17.14 The tree diagram for the BO problem.

Step 2: Define Events of Interest

Let A be the event that the person has BO . Let B be the event that the test was positive. The outcomes in each event are marked in the tree diagram. We want to find $\Pr[A \mid B]$, the probability that a person has BO , given that the test was positive.

Step 3: Find Outcome Probabilities

First, we assign probabilities to edges. These probabilities are drawn directly from the problem statement. By the Product Rule, the probability of an outcome is the product of the probabilities on the corresponding root-to-leaf path. All probabilities are shown in Figure 17.14.

Step 4: Compute Event Probabilities

From Definition 17.5.1, we have

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]} = \frac{0.09}{0.09 + 0.27} = \frac{1}{4}.$$

So, if you test positive, then there is only a 25% chance that you have the condition!

This answer is initially surprising, but makes sense on reflection. There are two ways you could test positive. First, it could be that you have the condition and the test is correct. Second, it could be that you are healthy and the test is incorrect. The

problem is that almost everyone is healthy; therefore, most of the positive results arise from incorrect tests of healthy people!

We can also compute the probability that the test is correct for a random person. This event consists of two outcomes. The person could have the condition and test positive (probability 0.09), or the person could be healthy and test negative (probability 0.63). Therefore, the test is correct with probability $0.09 + 0.63 = 0.72$. This is a relief; the test is correct almost three-quarters of the time.

But wait! There is a simple way to make the test correct 90% of the time: always return a negative result! This “test” gives the right answer for all healthy people and the wrong answer only for the 10% that actually have the condition. So a better strategy by this measure is to completely ignore the test result!

There is a similar paradox in weather forecasting. During winter, almost all days in Boston are wet and overcast. Predicting miserable weather every day may be more accurate than really trying to get it right!

17.5.4 *A Posteriori Probabilities*

If you think about it too much, the medical testing problem we just considered could start to trouble you. The concern would be that by the time you take the test, you either have the BO condition or you don’t—you just don’t know which it is. So you may wonder if a statement like “If you tested positive, then you have the condition with probability 25%” makes sense.

In fact, such a statement does make sense. It means that 25% of the people who test positive actually have the condition. It is true that any particular person has it or they don’t, but a *randomly selected* person among those who test positive will have the condition with probability 25%.

Anyway, if the medical testing example bothers you, you will definitely be worried by the following examples, which go even further down this path.

17.5.5 The “Halting Problem,” in Reverse

Suppose that we turn the hockey question around: what is the probability that the Halting Problem won their first game, given that they won the series?

This seems like an absurd question! After all, if the Halting Problem won the series, then the winner of the first game has already been determined. Therefore, who won the first game is a question of fact, not a question of probability. However, our mathematical theory of probability contains no notion of one event preceding another—there is no notion of time at all. Therefore, from a mathematical perspective, this is a perfectly valid question. And this is also a meaningful question from a practical perspective. Suppose that you’re told that the Halting Problem won the series, but not told the results of individual games. Then, from your perspective, it

makes perfect sense to wonder how likely it is that The Halting Problem won the first game.

A conditional probability $\Pr[B \mid A]$ is called *a posteriori* if event B precedes event A in time. Here are some other examples of a posteriori probabilities:

- The probability it was cloudy this morning, given that it rained in the afternoon.
- The probability that I was initially dealt two queens in Texas No Limit Hold 'Em poker, given that I eventually got four-of-a-kind.

Mathematically, a posteriori probabilities are *no different* from ordinary probabilities; the distinction is only at a higher, philosophical level. Our only reason for drawing attention to them is to say, “Don’t let them rattle you.”

Let’s return to the original problem. The probability that the Halting Problem won their first game, given that they won the series is $\Pr[B \mid A]$. We can compute this using the definition of conditional probability and the tree diagram in Figure 17.13:

$$\Pr[B \mid A] = \frac{\Pr[B \cap A]}{\Pr[A]} = \frac{1/3 + 1/18}{1/3 + 1/18 + 1/9} = \frac{7}{9}.$$

This answer is suspicious! In the preceding section, we showed that $\Pr[A \mid B]$ was also $7/9$. Could it be true that $\Pr[A \mid B] = \Pr[B \mid A]$ in general? Some reflection suggests this is unlikely. For example, the probability that I feel uneasy, given that I was abducted by aliens, is pretty large. But the probability that I was abducted by aliens, given that I feel uneasy, is rather small.

Let’s work out the general conditions under which $\Pr[A \mid B] = \Pr[B \mid A]$. By the definition of conditional probability, this equation holds if and only if:

$$\frac{\Pr[A \cap B]}{\Pr[B]} = \frac{\Pr[A \cap B]}{\Pr[A]}$$

This equation, in turn, holds only if the denominators are equal or the numerator is 0; namely if

$$\Pr[B] = \Pr[A] \quad \text{or} \quad \Pr[A \cap B] = 0.$$

The former condition holds in the hockey example; the probability that the Halting Problem wins the series (event A) is equal to the probability that it wins the first game (event B) since both probabilities are $1/2$.

In general, such pairs of probabilities are related by Bayes’ Rule:

Theorem 17.5.2 (Bayes’ Rule). *If $\Pr[A]$ and $\Pr[B]$ are nonzero, then:*

$$\Pr[B | A] = \frac{\Pr[A | B] \cdot \Pr[B]}{\Pr[A]} \quad (17.5)$$

Proof. When $\Pr[A]$ and $\Pr[B]$ are nonzero, we have

$$\Pr[A | B] \cdot \Pr[B] = \Pr[A \cap B] = \Pr[B | A] \cdot \Pr[A]$$

by definition of conditional probability. Dividing by $\Pr[A]$ gives (17.5). ■

17.5.6 The Law of Total Probability

Breaking a probability calculation into cases simplifies many problems. The idea is to calculate the probability of an event A by splitting into two cases based on whether or not another event E occurs. That is, calculate the probability of $A \cap E$ and $A \cap \bar{E}$. By the Sum Rule, the sum of these probabilities equals $\Pr[A]$. Expressing the intersection probabilities as conditional probabilities yields:

Rule 17.5.3 (Law of Total Probability, single event). *If $\Pr[E]$ and $\Pr[\bar{E}]$ are nonzero, then*

$$\Pr[A] = \Pr[A | E] \cdot \Pr[E] + \Pr[A | \bar{E}] \cdot \Pr[\bar{E}].$$

For example, suppose we conduct the following experiment. First, we flip a fair coin. If heads comes up, then we roll one die and take the result. If tails comes up, then we roll two dice and take the sum of the two results. What is the probability that this process yields a 2? Let E be the event that the coin comes up heads, and let A be the event that we get a 2 overall. Assuming that the coin is fair, $\Pr[E] = \Pr[\bar{E}] = 1/2$. There are now two cases. If we flip heads, then we roll a 2 on a single die with probability $\Pr[A | E] = 1/6$. On the other hand, if we flip tails, then we get a sum of 2 on two dice with probability $\Pr[A | \bar{E}] = 1/36$. Therefore, the probability that the whole process yields a 2 is

$$\Pr[A] = \frac{1}{2} \cdot \frac{1}{6} + \frac{1}{2} \cdot \frac{1}{36} = \frac{7}{72}.$$

There is also a form of the rule to handle more than two cases.

Rule 17.5.4 (Law of Total Probability). *If $E_1, \dots, E_n$ are disjoint events whose union is the whole sample space, then:*

$$\Pr[A] = \sum_{i=1}^n \Pr[A | E_i] \cdot \Pr[E_i].$$

17.5.7 Conditioning on a Single Event

The probability rules that we derived in Section 17.4.2 extend to probabilities conditioned on the same event. For example, the Inclusion-Exclusion formula for two sets holds when all probabilities are conditioned on an event C :

$$\Pr[A \cup B \mid C] = \Pr[A \mid C] + \Pr[B \mid C] - \Pr[A \cap B \mid C].$$

This is easy to verify by plugging in the Definition 17.5.1 of conditional probability.⁵

It is important not to mix up events before and after the conditioning bar. For example, the following is *not* a valid identity:

False Claim.

$$\Pr[A \mid B \cup C] = \Pr[A \mid B] + \Pr[A \mid C] - \Pr[A \mid B \cap C]. \quad (17.6)$$

A simple counter-example is to let B and C be events over a uniform space with most of their outcomes in A , but not overlapping. This ensures that $\Pr[A \mid B]$ and $\Pr[A \mid C]$ are both close to 1. For example,

$$\begin{aligned} B &::= [0, 9], \\ C &::= [10, 18] \cup \{0\}, \\ A &::= [1, 18], \end{aligned}$$

so

$$\Pr[A \mid B] = \frac{9}{10} = \Pr[A \mid C].$$

Also, since 0 is the only outcome in $B \cap C$ and $0 \notin A$, we have

$$\Pr[A \mid B \cap C] = 0$$

So the right hand side of (17.6) is 1.8, while the left hand side is a probability which can be at most 1 —actually, it is 18/19.

17.5.8 Discrimination Lawsuit

Several years ago there was a sex discrimination lawsuit against a famous university. A woman math professor was denied tenure, allegedly because she was a woman. She argued that in every one of the university’s 22 departments, the percentage of men candidates granted tenure was greater than the percentage of women candidates granted tenure. This sounds very suspicious!

⁵Problem 17.11 explains why this and similar conditional identities follow on general principles from the corresponding unconditional identities.

However, the university’s lawyers argued that across the university as a whole, the percentage of male candidates granted tenure was actually *lower* than the percentage for women candidates. This suggests that if there was any sex discrimination, then it was against men! Surely, at least one party in the dispute must be lying.

Let’s clarify the problem by expressing both arguments in terms of conditional probabilities. To simplify matters, suppose that there are only two departments, EE and CS, and consider the experiment where we pick a random candidate. Define the following events:

- $A::=$ the candidate is granted tenure,
- $F_{EE}::=$ the candidate is a woman in the EE department,
- $F_{CS}::=$ the candidate is a woman in the CS department,
- $M_{EE}::=$ the candidate is a man in the EE department,
- $M_{CS}::=$ the candidate is a man in the CS department.

Assume that all candidates are either men or women, and that no candidate belongs to both departments. That is, the events F_{EE} , F_{CS} , M_{EE} , and M_{CS} are all disjoint.

In these terms, the plaintiff is making the following argument:

$$\begin{aligned} \Pr[A \mid F_{EE}] &< \Pr[A \mid M_{EE}] \quad \text{and} \\ \Pr[A \mid F_{CS}] &< \Pr[A \mid M_{CS}]. \end{aligned}$$

That is, in both departments, the probability that a woman candidate is granted tenure is less than the probability for a man.

The university retorts that *overall*, a woman candidate is *more* likely to be granted tenure than a man; namely that

$$\Pr[A \mid F_{EE} \cup F_{CS}] > \Pr[A \mid M_{EE} \cup M_{CS}].$$

It is easy to believe that these two positions are contradictory, and the phenomenon illustrated here is widely referred to as “Simpson’s Paradox.” But there is no contradiction or paradox, and in fact, Table 17.1 shows a set of candidate statistics for which the assertions of both the plaintiff and the university hold. In this case, a higher percentage of men candidates were granted tenure in each department, but overall a higher percentage of women candidates were granted tenure! How do we make sense of this?

CS	0 women granted tenure, 1 candidate	0%
	50 men granted tenure, 100 candidate	50%
EE	70 women granted tenure, 100 candidate	70%
	1 man granted tenure, 1 candidate	100%
Overall	70 women granted tenure, 101 candidate	$\approx 70\%$
	51 men granted tenure, 101 candidate	$\approx 51\%$

Table 17.1 A scenario where women are less likely to be granted tenure than men in each department, but more likely to be granted tenure overall.

With data like this showing that at the department level, women candidates were less likely to be granted tenure than men, university administrators would likely see an indication of bias against women, and the departments would be directed to reexamine their admission procedures.

But suppose we replaced “the candidate is a man/woman in the EE department,” by “the candidate is a man/woman for whom a tenure decision was made during an odd-numbered day of the month,” and likewise with CS and an even-numbered day of the month. Since we don’t think the parity of a date is a cause for the outcome of a tenure decision, we would ignore the “coincidence” that on both odd and even dates, men are more frequently granted tenure. Instead, we would judge, based on the overall data showing women more likely to be granted tenure, that gender bias against women was *not* an issue in the university.

The point is that it’s the *same data* that we interpret differently based on our implicit causal beliefs. It would be circular to claim that the gender correlation observed in the data corroborates our belief that there is discrimination, since our interpretation of the data correlation *depends* on our beliefs about the causes of tenure decisions.⁶ This illustrates a basic principle in statistics which people constantly ignore: *never assume that correlation implies causation*.

17.6 Independence

Suppose that we flip two fair coins simultaneously on opposite sides of a room. Intuitively, the way one coin lands does not affect the way the other coin lands. The mathematical concept that captures this intuition is called *independence*.

⁶These issues are thoughtfully examined in *Causality: Models, Reasoning and Inference*, Judea Pearl, Cambridge U. Press, 2001

Definition 17.6.1. An event with probability 0 is defined to be independent of every event (including itself). If $\Pr[B] \neq 0$, then event A is independent of event B iff

$$\Pr[A \mid B] = \Pr[A]. \quad (17.7)$$

In other words, A and B are independent if knowing that B happens does not alter the probability that A happens, as is the case with flipping two coins on opposite sides of a room.

Potential Pitfall

Students sometimes get the idea that disjoint events are independent. The *opposite* is true: if $A \cap B = \emptyset$, then knowing that A happens means you know that B does not happen. So disjoint events are *never* independent—unless one of them has probability zero.

17.6.1 Alternative Formulation

Sometimes it is useful to express independence in an alternate form which follows immediately from Definition 17.6.1:

Theorem 17.6.2. A is independent of B if and only if

$$\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]. \quad (17.8)$$

Notice that Theorem 17.6.2 makes apparent the symmetry between A being independent of B and B being independent of A :

Corollary 17.6.3. A is independent of B iff B is independent of A .

17.6.2 Independence Is an Assumption

Generally, independence is something that you *assume* in modeling a phenomenon. For example, consider the experiment of flipping two fair coins. Let A be the event that the first coin comes up heads, and let B be the event that the second coin is heads. If we assume that A and B are independent, then the probability that both coins come up heads is:

$$\Pr[A \cap B] = \Pr[A] \cdot \Pr[B] = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}.$$

In this example, the assumption of independence is reasonable. The result of one coin toss should have negligible impact on the outcome of the other coin toss. And if we were to repeat the experiment many times, we would be likely to have $A \cap B$ about 1/4 of the time.

There are, of course, many examples of events where assuming independence is *not* justified. For example, let C be the event that tomorrow is cloudy and R be the event that tomorrow is rainy. Perhaps $\Pr[C] = 1/5$ and $\Pr[R] = 1/10$ in Boston. If these events were independent, then we could conclude that the probability of a rainy, cloudy day was quite small:

$$\Pr[R \cap C] = \Pr[R] \cdot \Pr[C] = \frac{1}{5} \cdot \frac{1}{10} = \frac{1}{50}.$$

Unfortunately, these events are definitely not independent; in particular, every rainy day is cloudy. Thus, the probability of a rainy, cloudy day is actually $1/10$.

Deciding when to *assume* that events are independent is a tricky business. In practice, there are strong motivations to assume independence since many useful formulas (such as equation (17.8)) only hold if the events are independent. But you need to be careful: we’ll describe several famous examples where (false) assumptions of independence led to trouble. This problem gets even trickier when there are more than two events in play.

17.6.3 Mutual Independence

We have defined what it means for two events to be independent. What if there are more than two events? For example, how can we say that the flips of n coins are all independent of one another? A set of events is said to be *mutually independent* if the probability of each event in the set is the same no matter which of the other events has occurred. We could formalize this with conditional probabilities as in Definition 17.6.1, but we’ll jump directly to the cleaner definition based on products of probabilities as in Theorem 17.6.2:

Definition 17.6.4. A set of events $E_1, E_2, \dots, E_n$ is mutually independent iff for all subsets $S \subseteq [1, n]$,

$$\Pr \left[\bigcap_{j \in S} E_j \right] = \prod_{j \in S} \Pr[E_j].$$

Definition 17.6.4 says that $E_1, E_2, \dots, E_n$ are mutually independent if and only if all of the following equations hold for all distinct i, j, k , and l :

$$\begin{aligned} \Pr[E_i \cap E_j] &= \Pr[E_i] \cdot \Pr[E_j] \\ \Pr[E_i \cap E_j \cap E_k] &= \Pr[E_i] \cdot \Pr[E_j] \cdot \Pr[E_k] \\ \Pr[E_i \cap E_j \cap E_k \cap E_l] &= \Pr[E_i] \cdot \Pr[E_j] \cdot \Pr[E_k] \cdot \Pr[E_l] \\ &\vdots \\ \Pr[E_1 \cap \dots \cap E_n] &= \Pr[E_1] \cdot \dots \cdot \Pr[E_n]. \end{aligned}$$

For example, if we toss n fair coins, the tosses are mutually independent iff for every subset of m coins, the probability that every coin in the subset comes up heads is 2^{-m} .

17.6.4 DNA Testing

Assumptions about independence are routinely made in practice. Frequently, such assumptions are quite reasonable. Sometimes, however, the reasonableness of an independence assumption is not so clear, and the consequences of a faulty assumption can be severe.

For example, consider the following testimony from the O. J. Simpson murder trial on May 15, 1995:

Mr. Clarke: When you make these estimations of frequency—and I believe you touched a little bit on a concept called independence?

Dr. Cotton: Yes, I did.

Mr. Clarke: And what is that again?

Dr. Cotton: It means whether or not you inherit one allele that you have is not—does not affect the second allele that you might get. That is, if you inherit a band at 5,000 base pairs, that doesn’t mean you’ll automatically or with some probability inherit one at 6,000. What you inherit from one parent is what you inherit from the other.

Mr. Clarke: Why is that important?

Dr. Cotton: Mathematically that’s important because if that were not the case, it would be improper to multiply the frequencies between the different genetic locations.

Mr. Clarke: How do you—well, first of all, are these markers independent that you’ve described in your testing in this case?

Presumably, this dialogue was as confusing to you as it was for the jury. Essentially, the jury was told that genetic markers in blood found at the crime scene matched Simpson’s. Furthermore, they were told that the probability that the markers would be found in a randomly-selected person was at most 1 in 170 million. This astronomical figure was derived from statistics such as:

- 1 person in 100 has marker A .
- 1 person in 50 marker B .

- 1 person in 40 has marker C .
- 1 person in 5 has marker D .
- 1 person in 170 has marker E .

Then these numbers were multiplied to give the probability that a randomly-selected person would have all five markers:

$$\begin{aligned}\Pr[A \cap B \cap C \cap D \cap E] &= \Pr[A] \cdot \Pr[B] \cdot \Pr[C] \cdot \Pr[D] \cdot \Pr[E] \\ &= \frac{1}{100} \cdot \frac{1}{50} \cdot \frac{1}{40} \cdot \frac{1}{5} \cdot \frac{1}{170} = \frac{1}{170,000,000}.\end{aligned}$$

The defense pointed out that this assumes that the markers appear mutually independently. Furthermore, all the statistics were based on just a few hundred blood samples.

After the trial, the jury was widely mocked for failing to “understand” the DNA evidence. If you were a juror, would *you* accept the 1 in 170 million calculation?

17.6.5 Pairwise Independence

The definition of mutual independence seems awfully complicated—there are so many subsets of events to consider! Here’s an example that illustrates the subtlety of independence when more than two events are involved. Suppose that we flip three fair, mutually-independent coins. Define the following events:

- A_1 is the event that coin 1 matches coin 2.
- A_2 is the event that coin 2 matches coin 3.
- A_3 is the event that coin 3 matches coin 1.

Are A_1 , A_2 , A_3 mutually independent?

The sample space for this experiment is:

$$\{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}.$$

Every outcome has probability $(1/2)^3 = 1/8$ by our assumption that the coins are mutually independent.

To see if events A_1 , A_2 , and A_3 are mutually independent, we must check a sequence of equalities. It will be helpful first to compute the probability of each event A_i :

$$\begin{aligned}\Pr[A_1] &= \Pr[HHH] + \Pr[HHT] + \Pr[TTH] + \Pr[TTT] \\ &= \frac{1}{8} + \frac{1}{8} + \frac{1}{8} + \frac{1}{8} = \frac{1}{2}.\end{aligned}$$

By symmetry, $\Pr[A_2] = \Pr[A_3] = 1/2$ as well. Now we can begin checking all the equalities required for mutual independence in Definition 17.6.4:

$$\begin{aligned}\Pr[A_1 \cap A_2] &= \Pr[HHH] + \Pr[TTT] = \frac{1}{8} + \frac{1}{8} = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} \\ &= \Pr[A_1] \Pr[A_2].\end{aligned}$$

By symmetry, $\Pr[A_1 \cap A_3] = \Pr[A_1] \cdot \Pr[A_3]$ and $\Pr[A_2 \cap A_3] = \Pr[A_2] \cdot \Pr[A_3]$ must hold also. Finally, we must check one last condition:

$$\begin{aligned}\Pr[A_1 \cap A_2 \cap A_3] &= \Pr[HHH] + \Pr[TTT] = \frac{1}{8} + \frac{1}{8} = \frac{1}{4} \\ &\neq \frac{1}{8} = \Pr[A_1] \Pr[A_2] \Pr[A_3].\end{aligned}$$

The three events A_1 , A_2 , and A_3 are not mutually independent even though any two of them are independent! This not-quite mutual independence seems weird at first, but it happens. It even generalizes:

Definition 17.6.5. A set $A_1, A_2, \dots$, of events is *k-way independent* iff every set of k of these events is mutually independent. The set is *pairwise independent* iff it is 2-way independent.

So the sets A_1, A_2, A_3 above are pairwise independent, but not mutually independent. Pairwise independence is a much weaker property than mutual independence.

For example, suppose that the prosecutors in the O. J. Simpson trial were wrong and markers A, B, C, D , and E appear only *pairwise* independently. Then the probability that a randomly-selected person has all five markers is no more than:

$$\begin{aligned}\Pr[A \cap B \cap C \cap D \cap E] &\leq \Pr[A \cap E] = \Pr[A] \cdot \Pr[E] \\ &= \frac{1}{100} \cdot \frac{1}{170} = \frac{1}{17,000}.\end{aligned}$$

The first line uses the fact that $A \cap B \cap C \cap D \cap E$ is a subset of $A \cap E$. (We picked out the A and E markers because they’re the rarest.) We use pairwise independence on the second line. Now the probability of a random match is 1 in 17,000—a far cry from 1 in 170 million! And this is the strongest conclusion we can reach assuming only pairwise independence.

On the other hand, the 1 in 17,000 bound that we get by assuming pairwise independence is a lot better than the bound that we would have if there were no independence at all. For example, if the markers are dependent, then it is possible that

everyone with marker E has marker A ,
 everyone with marker A has marker B ,
 everyone with marker B has marker C , and
 everyone with marker C has marker D .

In such a scenario, the probability of a match is

$$\Pr[E] = \frac{1}{170}.$$

So a stronger independence assumption leads to a smaller bound on the probability of a match. The trick is to figure out what independence assumption is reasonable. Assuming that the markers are *mutually* independent may well *not* be reasonable unless you have examined hundreds of millions of blood samples. Otherwise, how would you know that marker D does not show up more frequently whenever the other four markers are simultaneously present?

We will conclude our discussion of independence with a useful, and somewhat famous, example known as the Birthday Principle.

17.6.6 The Birthday Principle

There are 95 students in a class. What is the probability that some birthday is shared by two people? Comparing 95 students to the 365 possible birthdays, you might guess the probability lies somewhere around $1/4$ —but you’d be wrong: the probability that there will be two people in the class with matching birthdays is actually more than 0.9999.

To work this out, we’ll assume that the probability that a randomly chosen student has a given birthday is $1/d$, where $d = 365$ in this case. We’ll also assume that a class is composed of n randomly and independently selected students, with $n = 95$ in this case. These randomness assumptions are not really true, since more babies are born at certain times of year, and students’ class selections are typically not independent of each other, but simplifying in this way gives us a start on analyzing the problem. More importantly, these assumptions are justifiable in important computer science applications of birthday matching. For example, the birthday matching is a good model for collisions between items randomly inserted into a hash table. So we won’t worry about things like Spring procreation preferences that make January birthdays more common, or about twins’ preferences to take classes together (or not).

Selecting a sequence of n students for a class yields a sequence of n birthdays. Under the assumptions above, the d^n possible birthday sequences are equally likely outcomes. Let’s examine the consequences of this probability model by focussing

on the i th and j th elements in a birthday sequence, where $1 \leq i \neq j \leq n$. It makes for a better story if we refer to the i th birthday as “Alice’s” and the j th as “Bob’s.”

Now if Alice, Bob, Carol, and Don are four different people, then whether Alice and Bob have matching birthdays is independent of whether Carol and Don do. What’s more interesting is that whether Alice and Carol have the same birthday is independent of whether Alice and Bob do. This follows because Carol is as likely to have the same birthday as Alice, independently of whatever birthdays Alice and Bob happen to have; a formal proof of this claim appears in Problem 18.2. In short, the set of all events that a couple has matching birthdays is *pairwise* independent, even for overlapping couples. This will be important Chapter 19 because pairwise independence will be enough to justify some conclusions about the expected number of matches. However, these matching birthday events are obviously *not* even 3-way independent: if Alice and Bob match, and also Alice and Carol match, then Bob and Carol will match.

It turns out that as long as the number of students is noticeably smaller than the number of possible birthdays, we can get a pretty good estimate of the birthday matching probabilities by *pretending* that the matching events are mutually independent. (An intuitive justification for this is that with only a small number of matching pairs, it’s likely that none of the pairs overlap.) Then the probability of *no* matching birthdays would be the same as r th power of the probability that a couple does *not* have matching birthdays, where $r ::= \binom{n}{2}$ is the number of couples. That is, the probability of no matching birthdays would be

$$(1 - 1/d)^{\binom{n}{2}}. \quad (17.9)$$

Using the fact that $1 + x < e^x$ for all x ,⁷ we would conclude that the probability of no matching birthdays is at most

$$e^{-\binom{n}{2}/d}. \quad (17.10)$$

The matching birthday problem fits in here so far as a nice example illustrating pairwise and mutual independence, but it’s actually not hard to justify the bound (17.10) without any pretence of independence. Namely, there are $d(d-1)(d-2)\cdots(d-(n-1))$ length n sequences of distinct birthdays. So the proba-

⁷This approximation is obtained by truncating the Taylor series $e^{-x} = 1 - x + x^2/2! - x^3/3! + \cdots$. The approximation $e^{-x} \approx 1 - x$ is pretty accurate when x is small.

bility that everyone has a different birthday is:

$$\begin{aligned}
 & \frac{d(d-1)(d-2)\cdots(d-(n-1))}{d^n} \\
 &= \frac{d}{d} \cdot \frac{d-1}{d} \cdot \frac{d-2}{d} \cdots \frac{d-(n-1)}{d} \\
 &= \left(1 - \frac{0}{d}\right) \left(1 - \frac{1}{d}\right) \left(1 - \frac{2}{d}\right) \cdots \left(1 - \frac{n-1}{d}\right) \\
 &< e^0 \cdot e^{-1/d} \cdot e^{-2/d} \cdots e^{-(n-1)/d} \quad (\text{since } 1+x < e^x) \\
 &= e^{-\left(\sum_{i=1}^{n-1} i/d\right)} \\
 &= e^{-(n(n-1)/2d)} \\
 &= \text{the bound (17.10).}
 \end{aligned}$$

For $n = 85$ and $d = 365$, the value of (17.10) is less than $1/17,000$, which means the probability of having some pair of matching birthdays actually is more than $1 - 1/17,000 > 0.9999$. So it would be pretty astonishing if there were no pair of students in the class with matching birthdays.

For $d \leq n^2/2$, the probability of no match turns out to be asymptotically equal to the upper bound (17.10). For $d = n^2/2$ in particular, the probability of no match is asymptotically equal to $1/e$. This leads to a rule of thumb which is useful in many contexts in computer science:

The Birthday Principle

If there are d days in a year and $\sqrt{2d}$ people in a room, then the probability that two share a birthday is about $1 - 1/e \approx 0.632$.

For example, the Birthday Principle says that if you have $\sqrt{2 \cdot 365} \approx 27$ people in a room, then the probability that two share a birthday is about 0.632. The actual probability is about 0.626, so the approximation is quite good.

Among other applications, it implies that to use a hash function that maps n items into a hash table of size d , you can expect many collisions unless n^2 is a small fraction of d . The Birthday Principle also famously comes into play as the basis of “birthday attacks” that crack certain cryptographic systems.

Problems for Section 17.2

Practice Problems

Problem 17.1.

Let B be the number of heads that come up on $2n$ independent tosses of a fair coin.

(a) $\Pr[B = n]$ is asymptotically equal to one of the expressions given below. Explain which one.

1. $\frac{1}{\sqrt{2\pi n}}$
2. $\frac{2}{\sqrt{\pi n}}$
3. $\frac{1}{\sqrt{\pi n}}$
4. $\sqrt{\frac{2}{\pi n}}$

Problem 17.2.

Suppose you flip a fair coin 100 times. The coin flips are all mutually independent.

- (a) What is the expected number of heads?
- (b) What upper bound on the probability that the number of heads is at least 70 can we derive using Markov's Theorem?
- (c) What is the variance of the number of heads?
- (d) What upper bound does Chebyshev's Theorem give us on the probability that the number of heads is either less than 30 or greater than 70?

Exam Problems

Problem 17.3. (a) What's the probability that 0 doesn't appear among k digits chosen independently and uniformly at random?

(b) A box contains 90 good and 10 defective screws. What's the probability that if we pick 10 screws from the box, none will be defective?

(c) First one digit is chosen uniformly at random from $\{1, 2, 3, 4, 5\}$ and is removed from the set; then a second digit is chosen uniformly at random from the remaining digits. What is the probability that an odd digit is picked the second time?

(d) Suppose that you *randomly* permute the digits $1, 2, \dots, n$, that is, you select a permutation uniformly at random. What is the probability the digit k ends up in the i th position after the permutation?

(e) A fair coin is flipped n times. What’s the probability that all the heads occur at the end of the sequence? (If no heads occur, then “all the heads are at the end of the sequence” is vacuously true.)

Class Problems

Problem 17.4.

In the alternate universe where the Red Sox don’t regularly collapse at the end of their season, the New York Yankees and the Boston Red Sox are playing a two-out-of-three series. (In other words, they play until one team has won two games. Then that team is declared the overall winner and the series ends. Again, a fantasy.) Assume that the Red Sox win each game with probability $3/5$, regardless of the outcomes of previous games.

Answer the questions below using the four step method. You can use the same tree diagram for all three problems.

- (a) What is the probability that a total of 3 games are played?
- (b) What is the probability that the winner of the series loses the first game?
- (c) What is the probability that the *correct* team wins the series?

Problem 17.5.

To determine which of two people gets a prize, a coin is flipped twice. If the flips are a Head and then a Tail, the first player wins. If the flips are a Tail and then a Head, the second player wins. However, if both coins land the same way, the flips don’t count and whole the process starts over.

Assume that on each flip, a Head comes up with probability p , regardless of what happened on other flips. Use the four step method to find a simple formula for the probability that the first player wins. What is the probability that neither player wins?

Suggestions: The tree diagram and sample space are infinite, so you’re not going to finish drawing the tree. Try drawing only enough to see a pattern. Summing all the winning outcome probabilities directly is difficult. However, a neat trick solves this problem and many others. Let s be the sum of all winning outcome probabilities in the whole tree. Notice that *you can write the sum of all the winning*

probabilities in certain subtrees as a function of s . Use this observation to write an equation in s and then solve.

Problem 17.6.

Suppose you need a fair coin to decide which door to choose in the 6.042 Monty Hall game. After making everyone in your group empty their pockets, all you managed to turn up is some old collaboration statements, a few used tissues, and one penny. However, the penny was from Prof. Meyer’s pocket, so it is **not** safe to assume that it is a fair coin.

How can we use a coin of unknown bias to get the same effect as a fair coin of bias $1/2$? Draw the tree diagram for your solution, but since it is infinite, draw only enough to see a pattern.

Suggestion: A neat trick allows you to sum all the outcome probabilities that cause you to say “Heads”: Let s be the sum of all “Heads” outcome probabilities in the whole tree. Notice that *you can write the sum of all the “Heads” outcome probabilities in certain subtrees as a function of s .* Use this observation to write an equation in s and then solve.

Homework Problems

Problem 17.7.

Let’s see what happens when *Let’s Make a Deal* is played with **four** doors. A prize is hidden behind one of the four doors. Then the contestant picks a door. Next, the host opens an unpicked door that has no prize behind it. The contestant is allowed to stick with their original door or to switch to one of the two unopened, unpicked doors. The contestant wins if their final choice is the door hiding the prize.

Let’s make the same assumptions as in the original problem:

1. The prize is equally likely to be behind each door.
2. The contestant is equally likely to pick each door initially, regardless of the prize’s location.
3. The host is equally likely to reveal each door that does not conceal the prize and was not selected by the player.

Use The Four Step Method to find the following probabilities. The tree diagram may become awkwardly large, in which case just draw enough of it to make its structure clear.

(a) Contestant Stu, a sanitation engineer from Trenton, New Jersey, stays with his original door. What is the probability that Stu wins the prize?

(b) Contestant Zelda, an alien abduction researcher from Helena, Montana, switches to one of the remaining two doors with equal probability. What is the probability that Zelda wins the prize?

Now let’s revise our assumptions about how contestants choose doors. Say the doors are labeled A, B, C, and D. Suppose that Carol always opens the *earliest* door possible (the door whose label is earliest in the alphabet) with the restriction that she can neither reveal the prize nor open the door that the player picked.

This gives contestant Mergatroid—an engineering student from Cambridge, MA—just a little more information about the location of the prize. Suppose that Mergatroid always switches to the earliest door, excluding his initial pick and the one Carol opened.

(c) What is the probability that Mergatroid wins the prize?

Problem 17.8.

I have a deck of 52 regular playing cards, 26 red, 26 black, randomly shuffled. They all lie face down in the deck so that you can’t see them. I will draw a card off the top of the deck and turn it face up so that you can see it and then put it aside. I will continue to turn up cards like this but at some point while there are still cards left in the deck, you have to declare that you want the next card in the deck to be turned up. If that next card turns up black you win and otherwise you lose. Either way, the game is then over.

(a) Show that if you take the first card before you have seen any cards, you then have probability $1/2$ of winning the game.

(b) Suppose you don’t take the first card and it turns up red. Show that you have then have a probability of winning the game that is greater than $1/2$.

(c) If there are r red cards left in the deck and b black cards, show that the probability of winning if you take the next card is $b/(r + b)$.

(d) Either,

1. come up with a strategy for this game that gives you a probability of winning strictly greater than $1/2$ and prove that the strategy works, or,
2. come up with a proof that no such strategy can exist.

Problems for Section 17.4

Class Problems

Problem 17.9.

Suppose there is a system, built by Caltech graduates, with n components. We know from past experience that any particular component will fail in a given year with probability p . That is, letting F_i be the event that the i th component fails within one year, we have

$$\Pr[F_i] = p$$

for $1 \leq i \leq n$. The system will fail if *any one* of its components fails. What can we say about the probability that the system will fail within one year?

Let F be the event that the system fails within one year. Without any additional assumptions, we can't get an exact answer for $\Pr[F]$. However, we can give useful upper and lower bounds, namely,

$$p \leq \Pr[F] \leq np. \quad (17.11)$$

We may as well assume $p < 1/n$, since the upper bound is trivial otherwise. For example, if $n = 100$ and $p = 10^{-5}$, we conclude that there is at most one chance in 1000 of system failure within a year and at least one chance in 100,000.

Let's model this situation with the sample space $\mathcal{S} ::= \mathcal{P}([1, n])$ whose outcomes are subsets of positive integers $\leq n$, where $s \in \mathcal{S}$ corresponds to the indices of exactly those components that fail within one year. For example, $\{2, 5\}$ is the outcome that the second and fifth components failed within a year and none of the other components failed. So the outcome that the system did not fail corresponds to the emptyset, $\emptyset$.

(a) Show that the probability that the system fails could be as small as p by describing appropriate probabilities for the outcomes. Make sure to verify that the sum of your outcome probabilities is 1.

(b) Show that the probability that the system fails could actually be as large as np by describing appropriate probabilities for the outcomes. Make sure to verify that the sum of your outcome probabilities is 1.

(c) Prove inequality (17.11).

Problem 17.10.

Here are some handy rules for reasoning about probabilities that all follow directly from the Disjoint Sum Rule. Prove them.

$$\Pr[A - B] = \Pr[A] - \Pr[A \cap B] \quad (\text{Difference Rule})$$

$$\Pr[\overline{A}] = 1 - \Pr[A] \quad (\text{Complement Rule})$$

$$\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B] \quad (\text{Inclusion-Exclusion})$$

$$\Pr[A \cup B] \leq \Pr[A] + \Pr[B] \quad (\text{2-event Union Bound})$$

$$\text{If } A \subseteq B, \text{ then } \Pr[A] \leq \Pr[B] \quad (\text{Monotonicity})$$

Problem 17.11.

Suppose $\Pr[\cdot] : \mathcal{S} \rightarrow [0, 1]$ is a probability function on a sample space, $\mathcal{S}$, and let B be an event such that $\Pr[B] > 0$. Define a function $\Pr_B[\cdot]$ on outcomes $w \in \mathcal{S}$ by the rule:

$$\Pr_B[\omega] ::= \begin{cases} \Pr[\omega] / \Pr[B] & \text{if } \omega \in B, \\ 0 & \text{if } \omega \notin B. \end{cases} \quad (17.12)$$

(a) Prove that $\Pr_B[\cdot]$ is also a probability function on $\mathcal{S}$ according to Definition 17.4.2.

(b) Prove that

$$\Pr_B[A] = \frac{\Pr[A \cap B]}{\Pr[B]}$$

for all $A \subseteq \mathcal{S}$.

Homework Problems

Problem 17.12.

Prove the following probabilistic identity, referred to as the **Union Bound**. You may assume the theorem that the probability of a union of *disjoint* sets is the sum of their probabilities.

Let $A_1, \dots, A_n$ be a collection of events. Then

$$\Pr[A_1 \cup A_2 \cup \dots \cup A_n] \leq \sum_{i=1}^n \Pr[A_i].$$

Hint: Induction.

Problems for Section 17.5

Practice Problems

Problem 17.13.

Dirty Harry places two bullets in the six-shell cylinder of his revolver. He gives the cylinder a random spin and says “Feeling lucky?” as he holds the gun against your heart.

- (a) What is the probability that you will get shot if he pulls the trigger?
- (b) Suppose he pulls the trigger and you don’t get shot. What is the probability that you will get shot if he pulls the trigger a second time?
- (c) Suppose you noticed that he placed the two shells next to each other in the cylinder. How does this change the answers to the previous two questions?

Class Problems

Problem 17.14.

There are two decks of cards. One is complete, but the other is missing the Ace of spades. Suppose you pick one of the two decks with equal probability and then select a card from that deck uniformly at random. What is the probability that you picked the complete deck, given that you selected the eight of hearts? Use the four-step method and a tree diagram.

Problem 17.15.

Suppose you have three cards: $A\heartsuit$, $A\spadesuit$, and a Jack. From these, you choose a random hand (that is, each card is equally likely to be chosen) of two cards, and let K be the number of Aces in your hand. You then randomly pick one of the cards in the hand and reveal it.

(a) Describe a simple probability space (that is, outcomes and their probabilities) for this scenario, and list the outcomes in each of the following events:

1. $[K \geq 1]$, (that is, your hand has an Ace in it),
2. $A\heartsuit$ is in your hand,
3. the revealed card is an $A\heartsuit$,
4. the revealed card is an Ace.

(b) Then calculate $\Pr[K = 2 \mid E]$ for E equal to each of the four events in part (a). Notice that most, but *not all*, of these probabilities are equal.

Now suppose you have a deck with d distinct cards, a different kinds of Aces (including an $A\heartsuit$), you draw a random hand with h cards, and then reveal a random card from your hand.

(c) Prove that $\Pr[A\heartsuit \text{ is in your hand}] = h/d$.

(d) Prove that

$$\Pr[K = 2 \mid A\heartsuit \text{ is in your hand}] = \Pr[K = 2] \cdot \frac{2d}{ah}. \quad (17.13)$$

(e) Conclude that

$$\Pr[K = 2 \mid \text{the revealed card is an Ace}] = \Pr[K = 2 \mid A\heartsuit \text{ is in your hand}].$$

Problem 17.16.

There are three prisoners in a maximum-security prison for fictional villains: the Evil Wizard Voldemort, the Dark Lord Sauron, and Little Bunny Foo-Foo. The parole board has declared that it will release two of the three, chosen uniformly at random, but has not yet released their names. Naturally, Sauron figures that he will be released to his home in Mordor, where the shadows lie, with probability $2/3$.

A guard offers to tell Sauron the name of one of the other prisoners who will be released (either Voldemort or Foo-Foo). If the guard has a choice of naming either Voldemort or Foo-Foo (because both are to be released), he names one of the two with equal probability.

Sauron knows the guard to be a truthful fellow. However, Sauron declines this offer. He reasons that if the guard says, for example, “Little Bunny Foo-Foo will be released”, then his own probability of release will drop to $1/2$. This is because he will then know that either he or Voldemort will also be released, and these two events are equally likely.

Dark Lord Sauron has made a typical mistake when reasoning about conditional probability. Using a tree diagram and the four-step method, explain his mistake. What is the probability that Sauron is released given that the guard says Foo-Foo is released?

Hint: Define the events S , F , and “ F ” as follows:

“ F ” = Guard says Foo-Foo is released

F = Foo-Foo is released

S = Sauron is released

Homework Problems

Problem 17.17.

Outside of their hum-drum duties as Math for Computer Science Teaching Assistants, Oscar is trying to learn to levitate using only intense concentration and Liz is trying to become the world champion flaming torch juggler. Suppose that Oscar’s probability of success is $1/6$, Liz’s chance of success is $1/4$, and these two events are independent.

- (a) If at least one of them succeeds, what is the probability that Oscar learns to levitate?
- (b) If at most one of them succeeds, what is the probability that Liz becomes the world flaming torch juggler champion?
- (c) If exactly one of them succeeds, what is the probability that it is Oscar?

Problem 17.18.

There is a course—not 6.042, naturally—in which 10% of the assigned problems contain errors. If you ask a Teaching Assistant (TA) whether a problem has an error, then they will answer correctly 80% of the time. This 80% accuracy holds regardless of whether or not a problem has an error. Likewise when you ask a lecturer, but with only 75% accuracy.

We formulate this as an experiment of choosing one problem randomly and asking a particular TA and Lecturer about it. Define the following events:

$E ::=$ “the problem has an error,”

$T ::=$ “the TA says the problem has an error,”

$L ::=$ “the lecturer says the problem has an error.”

- (a) Translate the description above into a precise set of equations involving conditional probabilities among the events E , T , and L .
- (b) Suppose you have doubts about a problem and ask a TA about it, and they tell you that the problem is correct. To double-check, you ask a lecturer, who says that the problem has an error. Assuming that *the correctness of the lecturers’ answer and the TA’s answer are independent of each other, regardless of whether there is an error*⁸, what is the probability that there is an error in the problem?

⁸This assumption is questionable: by and large, we would expect the lecturer and the TA’s to spot the same glaring errors and to be fooled by the same subtle ones.

(c) Is the event that “the TA says that there is an error”, independent of the event that “the lecturer says that there is an error”?

Problem 17.19. (a) Suppose you repeatedly flip a fair coin until you see the sequence HHT or the sequence TTH. What is the probability you will see HHT first? *Hint:* Symmetry between Heads and Tails.

(b) What is the probability you see the sequence HTT before you see the sequence HHT? *Hint:* Try to find the probability that HHT comes before HTT conditioning on whether you first toss an H or a T. The answer is not $1/2$.

Problem 17.20.

A 52-card deck is thoroughly shuffled and you are dealt a hand of 13 cards.

(a) If you have one ace, what is the probability that you have a second ace?

(b) If you have the ace of spades, what is the probability that you have a second ace? Remarkably, the answer is different from part (a).

Problem 17.21.

You are organizing a neighborhood census and instruct your census takers to knock on doors and note the sex of any child that answers the knock. Assume that there are two children in a household and that girls and boys are equally likely to be children and to open the door.

A sample space for this experiment has outcomes that are triples whose first element is either B or G for the sex of the elder child, likewise for the second element and the sex of the younger child, and whose third coordinate is E or Y indicating whether the elder child or younger child opened the door. For example, (B, G, Y) is the outcome that the elder child is a boy, the younger child is a girl, and the girl opened the door.

(a) Let T be the event that the household has two girls, and O be the event that a girl opened the door. List the outcomes in T and O .

(b) What is the probability $\Pr[T \mid O]$, that both children are girls, given that a girl opened the door?

(c) Where is the mistake in the following argument?

If a girl opens the door, then we know that there is at least one girl in the household. The probability that there is at least one girl is

$$1 - \Pr[\text{both children are boys}] = 1 - (1/2 \times 1/2) = 3/4. \quad (17.14)$$

So,

$$\Pr[T \mid \text{there is at least one girl in the household}] \quad (17.15)$$

$$= \frac{\Pr[T \cap \text{there is at least one girl in the household}]}{\Pr[\text{there is at least one girl in the household}]} \quad (17.16)$$

$$= \frac{\Pr[T]}{\Pr[\text{there is at least one girl in the household}]} \quad (17.17)$$

$$= (1/4)/(3/4) = 1/3. \quad (17.18)$$

Therefore, given that a girl opened the door, the probability that there are two girls in the household is 1/3.

Exam Problems

Problem 17.22.

Here’s a variation of Monty Hall’s game: the contestant still picks one of three doors, with a prize randomly placed behind one door and goats behind the other two. But now, instead of always opening a door to reveal a goat, Monty instructs Carol to *randomly* open one of the two doors that the contestant hasn’t picked. This means she may reveal a goat, or she may reveal the prize. If she reveals the prize, then the entire game is *restarted*, that is, the prize is again randomly placed behind some door, the contestant again picks a door, and so on until Carol finally picks a door with a goat behind it. Then the contestant can choose to *stick* with his original choice of door or *switch* to the other unopened door. He wins if the prize is behind the door he finally chooses.

To analyze this setup, we define two events:

GP: The event that the contestant guesses the door with the prize behind it on his first guess.

OP: The event that the game is restarted at least once. Another way to describe this is as the event that the door Carol first opens has a prize behind it.

(a) What is $\Pr[GP]$? ... $\Pr[OP \mid \overline{GP}]$?

(b) What is $\Pr[OP]$?

(c) Let R be the number of times the game is restarted before Carol picks a goat. What is $\text{Ex}[R]$? You may express the answer as a simple closed form in terms of $p ::= \text{Pr}[OP]$.

(d) What is the probability the game will continue forever?

(e) When Carol finally picks the goat, the contestant has the choice of sticking or switching. Let's say that the contestant adopts the strategy of sticking. Let W be the event that the contestant wins with this strategy, and let $w ::= \text{Pr}[W]$. Express the following conditional probabilities as simple closed forms in terms of w .

i) $\text{Pr}[W \mid GP] =$

ii) $\text{Pr}[W \mid \overline{GP} \cap OP] =$

iii) $\text{Pr}[W \mid \overline{GP} \cap \overline{OP}] =$

(f) What is $\text{Pr}[W]$?

(g) For any final outcome where the contestant wins with a “stick” strategy, he would lose if he had used a “switch” strategy, and vice versa. In the original Monty Hall game, we concluded immediately that the probability that he would win with a “switch” strategy was $1 - \text{Pr}[W]$. Why isn't this conclusion quite as obvious for this new, restartable game? Is this conclusion still sound? Briefly explain.

Problem 17.23.

There are two decks of cards, the red deck and the blue deck. They differ slightly in a way that makes drawing the eight of hearts slightly more likely from the red deck than from the blue deck.

One of the decks is randomly chosen and hidden in a box. You reach in the box and randomly pick a card that turns out to be the eight of hearts. You believe intuitively that this makes the red deck more likely to be in the box than the blue deck.

Your intuitive judgment about the red deck can be formalized and verified using some inequalities between probabilities and conditional probabilities involving the events

$R ::=$ Red deck is in the box,

$B ::=$ Blue deck is in the box,

$E ::=$ Eight of hearts is picked from the deck in the box.

(a) State an inequality between probabilities and/or conditional probabilities that formalizes the assertion, “picking the eight of hearts from the red deck is more likely than from the blue deck.”

(b) State a similar inequality that formalizes the assertion “picking the eight of hearts from the deck in the box makes the red deck more likely to be in the box than the blue deck.”

(c) Assuming the each deck is equally likely to be the one in the box, prove that the inequality of part (a) implies the inequality of part (b).

(d) Suppose you couldn’t be sure that the red deck and blue deck were equally likely to be in the box. Could you still conclude that picking the eight of hearts from the deck in the box makes the red deck more likely to be in the box than the blue deck? Briefly explain.

Problem 17.24.

There is a rare and serious disease called Beaver Fever which afflicts about 1 person in 1000. Victims of this disease start telling math jokes in social settings, believing other people will think they’re funny.

Doctor Meyer has some fairly reliable tests for this disease. In particular:

- If a person has Beaver Fever, the probability that Meyer diagnoses the person as having the disease is 0.99.
- If a person doesn’t have it, the probability that Meyer diagnoses that person as not having Beaver Fever is 0.97.

Let B be the event that a randomly chosen person has Beaver Fever, and Y be the event that Meyer’s diagnosis is “Yes, that person has Beaver Fever,” with $\overline{B}$ and $\overline{Y}$ the complements of these events.

(a) The description above explicitly gives the values of the following quantities. What are their values?

$$\Pr[B] \quad \Pr[Y \mid B] \quad \Pr[\overline{Y} \mid \overline{B}]$$

(b) Write formulas for $\Pr[\overline{B}]$ and $\Pr[Y \mid \overline{B}]$ solely in terms of the explicitly given expressions. Literally use the expressions, not their numeric values.

(c) Write a formula for the probability that Doctor Meyer says a person has the disease solely in terms of $\Pr[B]$, $\Pr[\overline{B}]$, $\Pr[Y \mid B]$ and $\Pr[Y \mid \overline{B}]$.

(d) Write a formula solely in terms of the expressions given in part (a) for the probability that a person has Beaver Fever given that Doctor Meyer says the person has it.

Problem 17.25.

Suppose that *Let's Make a Deal* is played according to slightly different rules and with a red goat and a blue goat. There are three doors, with a prize hidden behind one of them and the goats behind the others. No doors are opened until the contestant makes a final choice to stick or switch. The contestant is allowed to pick a door and ask a certain question that the host then answers honestly. The contestant may then stick with their chosen door, or switch to either of the other doors.

(a) If the contestant asks “is there is a goat behind one of the unchosen doors?” and the host answers “yes,” is the contestant more likely to win the prize if they stick, switch, or does it not matter? Clearly identify the probability space of outcomes and their probabilities you use to model this situation. What is the contestant’s probability of winning if he uses the best strategy?

(b) If the contestant asks “is the *red* goat behind one of the unchosen doors?” and the host answers “yes,” is the contestant more likely to win the prize if they stick, switch, or does it not matter? Clearly identify the probability space of outcomes and their probabilities you use to model this situation. What is the contestant’s probability of winning if he uses the best strategy?

Problems for Section 17.6

Practice Problems

Problem 17.26.

Bruce Lee, on a movie that didn’t go public, is practicing by breaking 5 boards with his fists. He is able to break a board with probability 0.8—he is practicing with his left fist, that’s why it’s not 1—and he breaks each board independently.

- (a) What is the probability that Bruce breaks exactly 2 out of the 5 boards that are placed before him?
- (b) What is the probability that Bruce breaks at most 3 out of the 5 boards that are placed before him?
- (c) What is the expected number of boards Bruce will break?

Problem 17.27.

Suppose 120 students take a final exam and the mean of their scores is 90. You have no other information about the students and the exam, *e.g.* you should not assume that the highest possible score is 100. You may, however, assume that exam scores are nonnegative.

- (a) State the best possible upper bound on the number of students who scored at least 180.
- (b) Now suppose somebody tells you that the lowest score on the exam is 30. Compute the new best possible upper bound on the number of students who scored at least 180.

Problem 17.28.

You want to estimate the fraction p of voters in the nation who will vote to re-elect the current president in the upcoming election. You do this by random sampling (with replacement). Specifically, you select n voters independently and randomly, ask them who they are going to vote for, and use the fraction P of those that say they will vote for the current president as an estimate for p .

(a) Our theorems about sampling and distributions allow us to calculate how confident we can be that the random variable P takes a value near the constant p . This calculation uses some facts about voters and the way they are chosen. Which of the following facts are true?

1. Given a particular voter, the probability of that voter preferring the president is p .
2. Given a particular voter, the probability of that voter preferring the President is 1 or 0.
3. The probability that some voter is chosen more than once in the sequence goes to zero as n increases.

4. All voters are equally likely to be selected as the third in our sequence of n choices of voters (assuming $n \geq 3$).
5. The probability that the second voter chosen will favor the President, given that the first voter chosen prefers the President, is greater than p .
6. The probability that the second voter chosen will favor the President, given that the second voter chosen is from the same state as the first, may not equal p .

(b) Suppose that, according to your calculations the following is true about your polling:

$$\Pr[|P - p| \leq 0.04] \geq 0.95$$

(c) You do the asking, you count how many said they will vote for the President, you divide by n , and find that $P = 0.53$. You call the President to give him your results. Which of the following are true?

1. Mr. President, $p = 0.53$!
2. Mr. President, with probability at least 95%, p is within 0.04 of 0.53.
3. Mr. President, either p is within 0.04 of 0.53 or something very strange (5-in-100) has happened.
4. Mr. President, we can be 95% confident that p is within 0.04 of 0.53.

Exam Problems

Problem 17.29.

Sally Smart just graduated from high school. She was accepted to three top colleges.

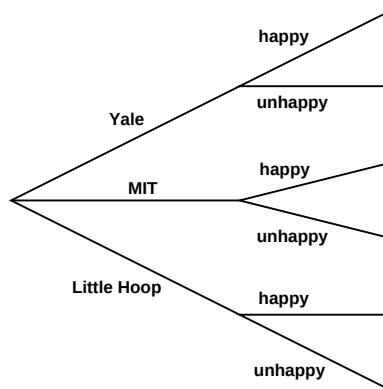
- With probability $4/12$, she attends Yale.
- With probability $5/12$, she attends MIT.
- With probability $3/12$, she attends Little Hoop Community College.

Sally will either be happy or unhappy in college.

- If she attends Yale, she is happy with probability $4/12$.
- If she attends MIT, she is happy with probability $7/12$.

- If she attends Little Hoop, she is happy with probability $11/12$.

(a) A tree diagram for Sally’s situation is shown below. On the diagram, fill in the edge probabilities and at each leaf write the probability of that outcome.



- (b) What is the probability that Sally is happy in college?
- (c) What is the probability that Sally Smart attends Yale, given that she is happy in college?
- (d) Show that the event that Sally attends Yale is **not** independent of the event that she is happy.
- (e) Show that the event that Sally Smart attends MIT **is** independent of the event that she is happy.

Problem 17.30.

Construct a probability space $\mathcal{S}$ such that $\mathcal{S}$ contains three events A , B , and C with the following properties:

- The three events satisfy the “product rule.” That is,

$$\Pr[A \cap B \cap C] = \Pr[A] \cdot \Pr[B] \cdot \Pr[C].$$

- The events are *not* mutually independent.

Hint: It may be helpful to draw a Venn diagram for $\mathcal{S}$ containing the three events, and then incrementally fill in the probabilities of the disjoint regions.

Class Problems

Problem 17.31.

Let A, B, C be events. For each of the following statements, prove it or give a counterexample.

- (a) If A is independent of B , and A is independent of C , then A is independent of $B \cap C$.
- (b) If A is independent of B , and A is independent of C , then A is independent of $B \cup C$.
- (c) If A is independent of B , and A is independent of C , and A is independent of $B \cap C$, then A is independent of $B \cup C$.

Problem 17.32.

Suppose that you flip three fair, mutually independent coins. Define the following events:

- Let A be the event that *the first* coin is heads.
 - Let B be the event that *the second* coin is heads.
 - Let C be the event that *the third* coin is heads.
 - Let D be the event that *an even number of* coins are heads.
- (a) Use the four step method to determine the probability space for this experiment and the probability of each of A, B, C, D .
 - (b) Show that these events are not mutually independent.
 - (c) Show that they are 3-way independent.

Homework Problems

Problem 17.33.

Define the events $A, F_{EE}, F_{CS}, M_{EE}$, and M_{CS} as in Section 17.5.8.

In these terms, the plaintiff in a discrimination suit against a university makes the argument that in both departments, the probability that a woman is granted tenure is less than the probability for a man. That is,

$$\Pr[A \mid F_{EE}] < \Pr[A \mid M_{EE}] \quad \text{and} \quad (17.19)$$

$$\Pr[A \mid F_{CS}] < \Pr[A \mid M_{CS}]. \quad (17.20)$$

The university’s defence attorneys retort that *overall*, a woman applicant is *more* likely to be granted tenure than a man, namely, that

$$\Pr[A \mid F_{EE} \cup F_{CS}] > \Pr[A \mid M_{EE} \cup M_{CS}]. \quad (17.21)$$

The judge then interrupts the trial and calls the plaintiff and defence attorneys to a conference in his office to resolve what he thinks are contradictory statements of facts about the tenure data. The judge points out that:

$$\begin{aligned} & \Pr[A \mid F_{EE} \cup F_{CS}] \\ &= \Pr[A \mid F_{EE}] + \Pr[A \mid F_{CS}] \quad (\text{because } F_{EE} \text{ and } F_{CS} \text{ are disjoint}) \\ &< \Pr[A \mid M_{EE}] + \Pr[A \mid M_{CS}] \quad (\text{by (17.19) and (17.20)}) \\ &= \Pr[A \mid M_{EE} \cup M_{CS}] \quad (\text{because } F_{EE} \text{ and } F_{CS} \text{ are disjoint}) \end{aligned}$$

so

$$\Pr[A \mid F_{EE} \cup F_{CS}] < \Pr[A \mid M_{EE} \cup M_{CS}],$$

which directly contradicts the university’s position (17.21)!

But the judge is mistaken; an example where the plaintiff and defence assertions are all true appears in Section 17.5.8. What is the mistake in the judge’s proof?

Problem 17.34.

Graphs, Logic & Probability

Let G be an undirected simple graph with $n > 3$ vertices. Let $E(x, y)$ mean that G has an edge between vertices x and y , and let $P(x, y)$ mean that there is a length 2 path in G between x and y .

- (a) Explain why $E(x, y)$ implies $P(x, x)$.
- (b) Circle the mathematical formula that best expresses the definition of $P(x, y)$.
 - $P(x, y) ::= \exists z. E(x, z) \text{ AND } E(y, z)$
 - $P(x, y) ::= x \neq y \text{ AND } \exists z. E(x, z) \text{ AND } E(y, z)$
 - $P(x, y) ::= \forall z. E(x, z) \text{ OR } E(y, z)$
 - $P(x, y) ::= \forall z. x \neq y \text{ IMPLIES } [E(x, z) \text{ OR } E(y, z)]$

For the following parts (c)–(e), let V be a fixed set of $n > 3$ vertices, and let G be a graph with these vertices constructed randomly as follows: for all distinct vertices

$x, y \in V$, independently include edge $\langle x-y \rangle$ as an edge of G with probability p . In particular, $\Pr[E(x, y)] = p$ for all $x \neq y$.

(c) For distinct vertices w, x, y and z in V , circle the event pairs that are independent.

1. $E(w, x)$ versus $E(x, y)$
2. $[E(w, x) \text{ AND } E(w, y)]$ versus $[E(z, x) \text{ AND } E(z, y)]$
3. $E(x, y)$ versus $P(x, y)$
4. $P(w, x)$ versus $P(x, y)$
5. $P(w, x)$ versus $P(y, z)$

(d) Write a simple formula in terms of n and p for $\Pr[\text{NOT } P(x, y)]$, for distinct vertices x and y in V .

Hint: Use part (c), item 2.

(e) What is the probability that two distinct vertices x and y lie on a three-cycle in G ? Answer with a simple expression in terms of p and r , where $r ::= \Pr[\text{NOT } P(x, y)]$ is the correct answer to part (d).

Hint: Express x and y being on a three-cycle as a simple formula involving $E(x, y)$ and $P(x, y)$.



18 Random Variables

Thus far, we have focused on probabilities of events. For example, we computed the probability that you win the Monty Hall game or that you have a rare medical condition given that you tested positive. But, in many cases we would like to know more. For example, *how many* contestants must play the Monty Hall game until one of them finally wins? *How long* will this condition last? *How much* will I lose gambling with strange dice all night? To answer such questions, we need to work with random variables.

18.1 Random Variable Examples

Definition 18.1.1. A random variable R on a probability space is a total function whose domain is the sample space.

The codomain of R can be anything, but will usually be a subset of the real numbers. Notice that the name “random variable” is a misnomer; random variables are actually functions!

For example, suppose we toss three independent, unbiased coins. Let C be the number of heads that appear. Let $M = 1$ if the three coins come up all heads or all tails, and let $M = 0$ otherwise. Now every outcome of the three coin flips uniquely determines the values of C and M . For example, if we flip heads, tails, heads, then $C = 2$ and $M = 0$. If we flip tails, tails, tails, then $C = 0$ and $M = 1$. In effect, C counts the number of heads, and M indicates whether all the coins match.

Since each outcome uniquely determines C and M , we can regard them as functions mapping outcomes to numbers. For this experiment, the sample space is:

$$\mathcal{S} = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}.$$

Now C is a function that maps each outcome in the sample space to a number as follows:

$$\begin{array}{ll} C(HHH) = 3 & C(THH) = 2 \\ C(HHT) = 2 & C(THT) = 1 \\ C(HTH) = 2 & C(TTH) = 1 \\ C(HTT) = 1 & C(TTT) = 0. \end{array}$$

Similarly, M is a function mapping each outcome another way:

$$\begin{array}{ll} M(HHH) = 1 & M(THH) = 0 \\ M(HHT) = 0 & M(THT) = 0 \\ M(HTH) = 0 & M(TTH) = 0 \\ M(HTT) = 0 & M(TTT) = 1. \end{array}$$

So C and M are random variables.

18.1.1 Indicator Random Variables

An *indicator random variable* is a random variable that maps every outcome to either 0 or 1. Indicator random variables are also called *Bernoulli variables*. The random variable M is an example. If all three coins match, then $M = 1$; otherwise, $M = 0$.

Indicator random variables are closely related to events. In particular, an indicator random variable partitions the sample space into those outcomes mapped to 1 and those outcomes mapped to 0. For example, the indicator M partitions the sample space into two blocks as follows:

$$\underbrace{HHH \quad TTT}_{M=1} \quad \underbrace{HHT \quad HTH \quad HTT \quad THH \quad THT \quad TTH}_{M=0}.$$

In the same way, an event E partitions the sample space into those outcomes in E and those not in E . So E is naturally associated with an indicator random variable, I_E , where $I_E(\omega) = 1$ for outcomes $\omega \in E$ and $I_E(\omega) = 0$ for outcomes $\omega \notin E$. Thus, $M = I_E$ where E is the event that all three coins match.

18.1.2 Random Variables and Events

There is a strong relationship between events and more general random variables as well. A random variable that takes on several values partitions the sample space into several blocks. For example, C partitions the sample space as follows:

$$\underbrace{TTT}_{C=0} \quad \underbrace{TTH \quad THT \quad HTT}_{C=1} \quad \underbrace{THH \quad HTH \quad HHT}_{C=2} \quad \underbrace{HHH}_{C=3}.$$

Each block is a subset of the sample space and is therefore an event. So the assertion that $C = 2$ defines the event

$$[C = 2] = \{THH, HTH, HHT\},$$

and this event has probability

$$\Pr[C = 2] = \Pr[THH] + \Pr[HTH] + \Pr[HHT] = \frac{1}{8} + \frac{1}{8} + \frac{1}{8} = 3/8.$$

Likewise $[M = 1]$ is the event $\{TTT, HHH\}$ and has probability $1/4$.

More generally, any assertion about the values of random variables defines an event. For example, the assertion that $C \leq 1$ defines

$$[C \leq 1] = \{TTT, TTH, THT, HTT\},$$

and so $\Pr[C \leq 1] = 1/2$.

Another example is the assertion that $C \cdot M$ is an odd number. This is an obscure way of saying that all three coins came up heads, namely,

$$[C \cdot M \text{ is odd}] = \{HHH\}.$$

Think about it!

18.2 Independence

The notion of independence carries over from events to random variables as well. Random variables R_1 and R_2 are *independent* iff for all x_1, x_2 , the two events

$$[R_1 = x_1] \quad \text{and} \quad [R_2 = x_2]$$

are independent.

For example, are C and M independent? Intuitively, the answer should be “no.” The number of heads, C , completely determines whether all three coins match; that is, whether $M = 1$. But, to verify this intuition, we must find some $x_1, x_2 \in \mathbb{R}$ such that:

$$\Pr[C = x_1 \text{ AND } M = x_2] \neq \Pr[C = x_1] \cdot \Pr[M = x_2].$$

One appropriate choice of values is $x_1 = 2$ and $x_2 = 1$. In this case, we have:

$$\Pr[C = 2 \text{ AND } M = 1] = 0 \neq \frac{1}{4} \cdot \frac{3}{8} = \Pr[M = 1] \cdot \Pr[C = 2].$$

The first probability is zero because we never have exactly two heads ($C = 2$) when all three coins match ($M = 1$). The other two probabilities were computed earlier.

On the other hand, let H_1 be the indicator variable for event that the first flip is a Head, so

$$[H_1 = 1] = \{HHH, HTH, HHT, HTT\}.$$

Then H_1 is independent of M , since

$$\begin{aligned}\Pr[M = 1] &= 1/4 = \Pr[M = 1 \mid H_1 = 1] = \Pr[M = 1 \mid H_1 = 0] \\ \Pr[M = 0] &= 3/4 = \Pr[M = 0 \mid H_1 = 1] = \Pr[M = 0 \mid H_1 = 0]\end{aligned}$$

This example is an instance of:

Lemma 18.2.1. *Two events are independent iff their indicator variables are independent.*

The simple proof is left to Problem 18.1.

As with events, the notion of independence generalizes to more than two random variables.

Definition 18.2.2. Random variables $R_1, R_2, \dots, R_n$ are *mutually independent* iff for all $x_1, x_2, \dots, x_n$, the n events

$$[R_1 = x_1], [R_2 = x_2], \dots, [R_n = x_n]$$

are mutually independent.

18.3 Distribution Functions

A random variable maps outcomes to values. The probability density function, $\text{PDF}_R(x)$, of a random variable, R , measures the probability that R takes the value x , and the closely related cumulative distribution function, $\text{CDF}_R(x)$, measures the probability that $R \leq x$. Random variables that show up for different spaces of outcomes often wind up behaving in much the same way because they have the same probability of taking different values, that is, because they have same pdf/cdf.

Definition 18.3.1. Let R be a random variable with codomain V . The *probability density function* of R is a function $\text{PDF}_R : V \rightarrow [0, 1]$ defined by:

$$\text{PDF}_R(x) ::= \begin{cases} \Pr[R = x] & \text{if } x \in \text{range}(R), \\ 0 & \text{if } x \notin \text{range}(R). \end{cases}$$

If the codomain is a subset of the real numbers, then the *cumulative distribution function* is the function $\text{CDF}_R : \mathbb{R} \rightarrow [0, 1]$ defined by:

$$\text{CDF}_R(x) ::= \Pr[R \leq x].$$

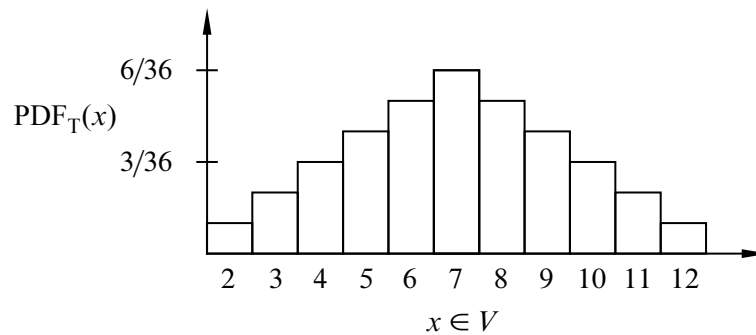


Figure 18.1 The probability density function for the sum of two 6-sided dice.

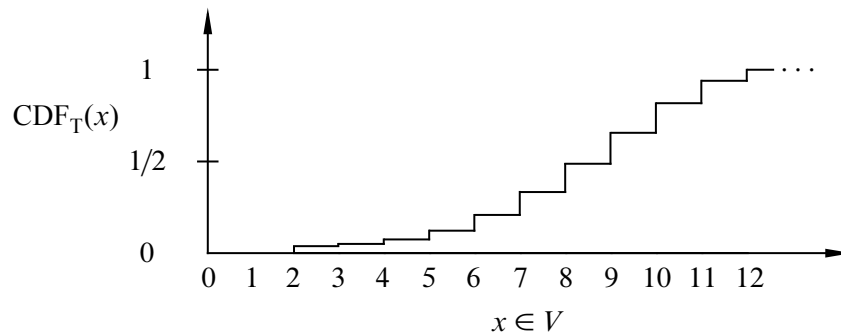


Figure 18.2 The cumulative distribution function for the sum of two 6-sided dice.

A consequence of this definition is that

$$\sum_{x \in \text{range}(R)} \text{PDF}_R(x) = 1.$$

This is because R has a value for each outcome, so summing the probabilities over all outcomes is the same as summing over the probabilities of each value in the range of R .

As an example, suppose that you roll two unbiased, independent, 6-sided dice. Let T be the random variable that equals the sum of the two rolls. This random variable takes on values in the set $V = \{2, 3, \dots, 12\}$. A plot of the probability density function for T is shown in Figure 18.1. The lump in the middle indicates that sums close to 7 are the most likely. The total area of all the rectangles is 1 since the dice must take on exactly one of the sums in $V = \{2, 3, \dots, 12\}$.

The cumulative distribution function for T is shown in Figure 18.2: The height of the i th bar in the cumulative distribution function is equal to the *sum* of the

heights of the leftmost i bars in the probability density function. This follows from the definitions of pdf and cdf:

$$\text{CDF}_R(x) = \Pr[R \leq x] = \sum_{y \leq x} \Pr[R = y] = \sum_{y \leq x} \text{PDF}_R(y).$$

It also follows from the definition that

$$\lim_{x \rightarrow \infty} \text{CDF}_R(x) = 1 \text{ and } \lim_{x \rightarrow -\infty} \text{CDF}_R(x) = 0.$$

Both PDF_R and CDF_R capture the same information R —obviously each one determines the other—but sometimes one is more convenient. The key point here is that neither the probability density function nor the cumulative distribution function involves the sample space of an experiment.

One of the really interesting things about density functions and distribution functions is that many random variables turn out to have the *same* pdf and cdf. In other words, even though R and S are different random variables on different probability spaces, it is often the case that

$$\text{PDF}_R = \text{PDF}_S.$$

In fact, some pdf’s are so common that they are given special names. For example, the three most important distributions in computer science are the *Bernoulli distribution*, the *uniform distribution*, and the *binomial distribution*. We look more closely at these common distributions in the next several sections.

18.3.1 Bernoulli Distributions

The Bernoulli distribution is the simplest and most common distribution function. That’s because it is the distribution function for an indicator random variable. Specifically, the *Bernoulli distribution* has a probability density function of the form $f_p : \{0, 1\} \rightarrow [0, 1]$ where

$$\begin{aligned} f_p(0) &= p, \quad \text{and} \\ f_p(1) &= 1 - p, \end{aligned}$$

for some $p \in [0, 1]$. The corresponding cumulative distribution function is $F_p : \mathbb{R} \rightarrow [0, 1]$ where

$$F_p(x) ::= \begin{cases} 0 & \text{if } x < 0 \\ p & \text{if } 0 \leq x < 1 \\ 1 & \text{if } 1 \leq x. \end{cases}$$

18.3.2 Uniform Distributions

A random variable that takes on each possible value in its codomain with the same probability is said to be *uniform*. If the codomain V has n elements, then the *uniform distribution* has a pdf of the form

$$f : V \rightarrow [0, 1]$$

where

$$f(v) = \frac{1}{n}$$

for all $v \in V$.

Uniform distributions come up all the time. For example, the number rolled on a fair die is uniform on the set $\{1, 2, \dots, 6\}$. An indicator variable is uniform when its pdf is $f_{1/2}$.

18.3.3 The Numbers Game

Enough definitions —let’s play a game! We have two envelopes. Each contains an integer in the range $0, 1, \dots, 100$, and the numbers are distinct. To win the game, you must determine which envelope contains the larger number. To give you a fighting chance, we’ll let you peek at the number in one envelope selected at random. Can you devise a strategy that gives you a better than 50% chance of winning?

For example, you could just pick an envelope at random and guess that it contains the larger number. But this strategy wins only 50% of the time. Your challenge is to do better.

So you might try to be more clever. Suppose you peek in one envelope and see the number 12. Since 12 is a small number, you might guess that the number in the other envelope is larger. But perhaps we’ve been tricky and put small numbers in *both* envelopes. Then your guess might not be so good!

An important point here is that the numbers in the envelopes may *not* be random. We’re picking the numbers and we’re choosing them in a way that we think will defeat your guessing strategy. We’ll only use randomization to choose the numbers if that serves our purpose, which is making you lose!

Intuition Behind the Winning Strategy

Amazingly, there is a strategy that wins more than 50% of the time, regardless of what numbers we put in the envelopes!

Suppose that you somehow knew a number x that was in between the numbers in the envelopes. Now you peek in one envelope and see a number. If it is bigger

than x , then you know you’re peeking at the higher number. If it is smaller than x , then you’re peeking at the lower number. In other words, if you know a number x between the numbers in the envelopes, then you are certain to win the game.

The only flaw with this brilliant strategy is that you do *not* know such an x . Oh well.

But what if you try to *guess* x ? There is some probability that you guess correctly. In this case, you win 100% of the time. On the other hand, if you guess incorrectly, then you’re no worse off than before; your chance of winning is still 50%. Combining these two cases, your overall chance of winning is better than 50%!

Informal arguments about probability, like this one, often sound plausible, but do not hold up under close scrutiny. In contrast, this argument sounds completely implausible—but is actually correct!

Analysis of the Winning Strategy

For generality, suppose that we can choose numbers from the set $\{0, 1, \dots, n\}$. Call the lower number L and the higher number H .

Your goal is to guess a number x between L and H . To avoid confusing equality cases, you select x at random from among the half-integers:

$$\left\{ \frac{1}{2}, 1\frac{1}{2}, 2\frac{1}{2}, \dots, n - \frac{1}{2} \right\}$$

But what probability distribution should you use?

The uniform distribution turns out to be your best bet. An informal justification is that if we figured out that you were unlikely to pick some number—say $50\frac{1}{2}$ —then we’d always put 50 and 51 in the envelopes. Then you’d be unlikely to pick an x between L and H and would have less chance of winning.

After you’ve selected the number x , you peek into an envelope and see some number T . If $T > x$, then you guess that you’re looking at the larger number. If $T < x$, then you guess that the other number is larger.

All that remains is to determine the probability that this strategy succeeds. We can do this with the usual four step method and a tree diagram.

Step 1: Find the sample space.

You either choose x too low ($< L$), too high ($> H$), or just right ($L < x < H$). Then you either peek at the lower number ($T = L$) or the higher number ($T = H$). This gives a total of six possible outcomes, as show in Figure 18.3.

Step 2: Define events of interest.

The four outcomes in the event that you win are marked in the tree diagram.

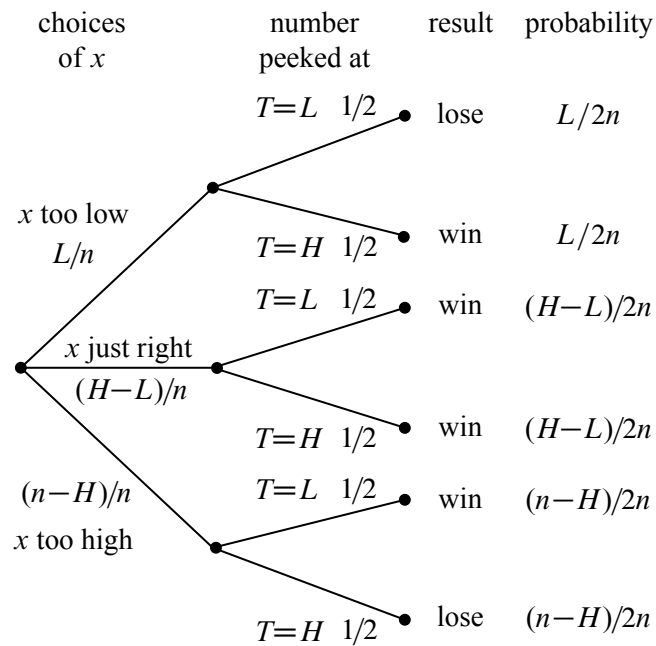


Figure 18.3 The tree diagram for the numbers game.

Step 3: Assign outcome probabilities.

First, we assign edge probabilities. Your guess x is too low with probability L/n , too high with probability $(n - H)/n$, and just right with probability $(H - L)/n$. Next, you peek at either the lower or higher number with equal probability. Multiplying along root-to-leaf paths gives the outcome probabilities.

Step 4: Compute event probabilities.

The probability of the event that you win is the sum of the probabilities of the four outcomes in that event:

$$\begin{aligned}
 \Pr[\text{win}] &= \frac{L}{2n} + \frac{H-L}{2n} + \frac{H-L}{2n} + \frac{n-H}{2n} \\
 &= \frac{1}{2} + \frac{H-L}{2n} \\
 &\geq \frac{1}{2} + \frac{1}{2n}
 \end{aligned}$$

The final inequality relies on the fact that the higher number H is at least 1 greater than the lower number L since they are required to be distinct.

Sure enough, you win with this strategy more than half the time, regardless of the numbers in the envelopes! For example, if I choose numbers in the range

0, 1, . . . , 100, then you win with probability at least $1/2 + 1/200 = 50.5\%$. Even better, if I’m allowed only numbers in the range 0, . . . , 10, then your probability of winning rises to 55%! By Las Vegas standards, those are great odds!

Randomized Algorithms

The best strategy to win the numbers game is an example of a *randomized algorithm*—it uses random numbers to influence decisions. Protocols and algorithms that make use of random numbers are very important in computer science. There are many problems for which the best known solutions are based on a random number generator.

For example, the most commonly-used protocol for deciding when to send a broadcast on a shared bus or Ethernet is a randomized algorithm known as *exponential backoff*. One of the most commonly-used sorting algorithms used in practice, called *quicksort*, uses random numbers. You’ll see many more examples if you take an algorithms course. In each case, randomness is used to improve the probability that the algorithm runs quickly or otherwise performs well.

18.3.4 Binomial Distributions

The third commonly-used distribution in computer science is the *binomial distribution*. The standard example of a random variable with a binomial distribution is the number of heads that come up in n independent flips of a coin. If the coin is fair, then the number of heads has an *unbiased binomial distribution*, specified by the pdf $f_n : \{0, 1, \dots, n\} \rightarrow [0, 1]$:

$$f_n(k) ::= \binom{n}{k} 2^{-n}.$$

This is because there are $\binom{n}{k}$ sequences of n coin tosses with exactly k heads, and each such sequence has probability 2^{-n} .

A plot of $f_{20}(k)$ is shown in Figure 18.4. The most likely outcome is $k = 10$ heads, and the probability falls off rapidly for larger and smaller values of k . The falloff regions to the left and right of the main hump are called the *tails of the distribution*.

In many fields, including Computer Science, probability analyses come down to getting small bounds on the tails of the binomial distribution. In the context of a problem, this typically means that there is very small probability that something *bad* happens, which could be a server or communication link overloading or a randomized algorithm running for an exceptionally long time or producing the wrong result.

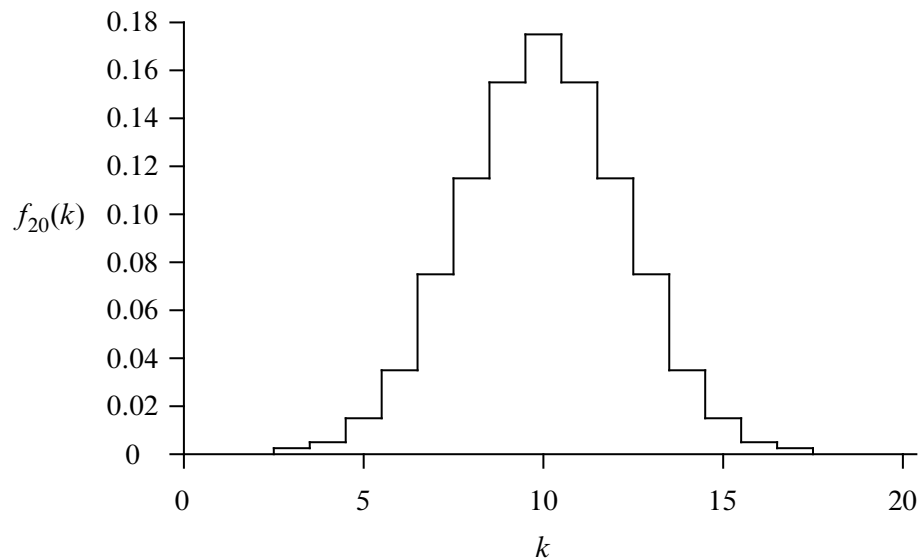


Figure 18.4 The pdf for the unbiased binomial distribution for $n = 20$, $f_{20}(k)$.

As an example, we can calculate the probability of flipping at most 25 heads in 100 tosses of a fair coin and see that it is very small, namely, less than 1 in 3,000,000.

In fact, the tail of the distribution falls off so rapidly that the probability of flipping exactly 25 heads is nearly twice the probability of flipping fewer than 25 heads! That is, the probability of flipping exactly 25 heads —small as it is— is still nearly twice as large as the probability of flipping exactly 24 heads *plus* the probability of flipping exactly 23 heads *plus* ... the probability of flipping no heads.

The General Binomial Distribution

If the coins are biased so that each coin is heads with probability p , then the number of heads has a *general binomial density function* specified by the pdf

$$f_{n,p} : \{0, 1, \dots, n\} \rightarrow [0, 1]$$

where

$$f_{n,p}(k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

for some $n \in \mathbb{N}^+$ and $p \in [0, 1]$. This is because there are $\binom{n}{k}$ sequences with k heads and $n - k$ tails, but now the probability of each such sequence is $p^k (1 - p)^{n-k}$.

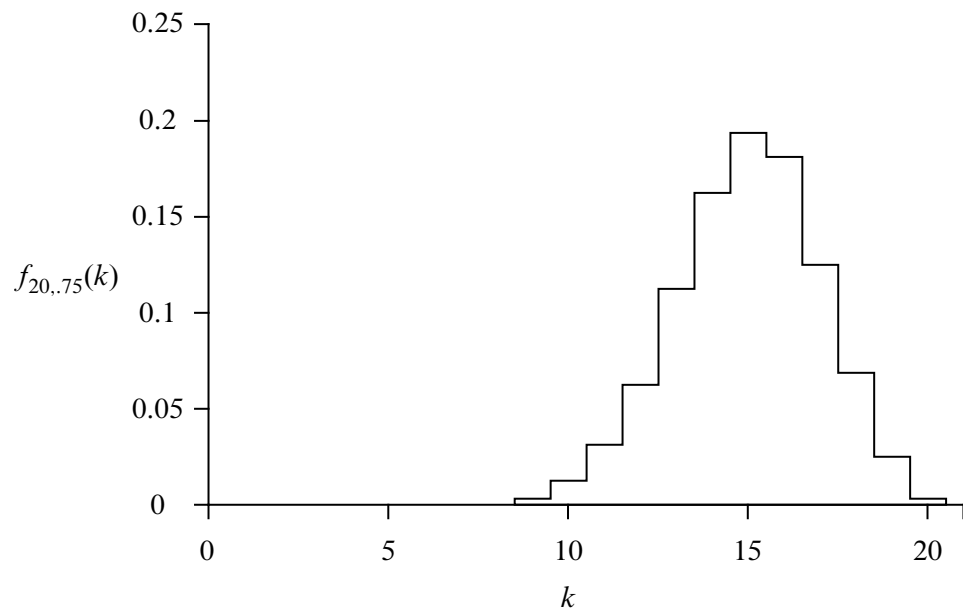


Figure 18.5 The pdf for the general binomial distribution $f_{n,p}(k)$ for $n = 20$ and $p = .75$.

For example, the plot in Figure 18.5 shows the probability density function $f_{n,p}(k)$ corresponding to flipping $n = 20$ independent coins that are heads with probability $p = 0.75$. The graph shows that we are most likely to get $k = 15$ heads, as you might expect. Once again, the probability falls off quickly for larger and smaller values of k .

18.4 Great Expectations

The *expectation* or *expected value* of a random variable is a single number that reveals a lot about the behavior of the variable. The expectation of a random variable is also known as its *mean* or *average*. It is the average value of the variable where each value is weighted according to its probability.

For example, suppose we select a student uniformly at random from the class, and let R be the student’s quiz score. Then $\text{Ex}[R]$ is just the class average—the first thing everyone wants to know after getting their test back! For similar reasons, the first thing you usually want to know about a random variable is its expected value.

Formally, the expected value of a random variable is defined as follows:

Definition 18.4.1. If R is a random variable defined on a sample space $\mathcal{S}$, then the expectation of R is

$$\text{Ex}[R] ::= \sum_{\omega \in \mathcal{S}} R(\omega) \Pr[\omega]. \quad (18.1)$$

Let’s work through some examples.

18.4.1 The Expected Value of a Uniform Random Variable

Rolling a 6-sided die provides an example of a uniform random variable. Let R be the value that comes up when you roll a fair 6-sided die. Then by (18.1), the expected value of R is

$$\text{Ex}[R] = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} = \frac{7}{2}.$$

This calculation shows that the name “expected” value is a little misleading; the random variable might *never* actually take on that value. You don’t ever expect to roll a $3\frac{1}{2}$ on an ordinary die!

In general, if R_n is a random variable with a uniform distribution on $\{1, 2, \dots, n\}$, then

$$\text{Ex}[R_n] = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{n(n+1)}{2n} = \frac{n+1}{2}.$$

18.4.2 The Expected Value of a Reciprocal Random Variable

Define a random variable S to be the reciprocal of the value that comes up when you roll a fair 6-sided die. That is, $S = 1/R$ where R is the value that you roll. Now,

$$\text{Ex}[S] = \text{Ex}\left[\frac{1}{R}\right] = \frac{1}{1} \cdot \frac{1}{6} + \frac{1}{2} \cdot \frac{1}{6} + \frac{1}{3} \cdot \frac{1}{6} + \frac{1}{4} \cdot \frac{1}{6} + \frac{1}{5} \cdot \frac{1}{6} + \frac{1}{6} \cdot \frac{1}{6} = \frac{49}{120}.$$

Notice that

$$\text{Ex}\left[\frac{1}{R}\right] \neq 1/\text{Ex}[R].$$

Assuming that these two quantities are equal is a common mistake.

18.4.3 The Expected Value of an Indicator Random Variable

The expected value of an indicator random variable for an event is just the probability of that event.

Lemma 18.4.2. *If I_A is the indicator random variable for event A , then*

$$\text{Ex}[I_A] = \Pr[A].$$

Proof.

$$\begin{aligned} \text{Ex}[I_A] &= 1 \cdot \Pr[I_A = 1] + 0 \cdot \Pr[I_A = 0] = \Pr[I_A = 1] \\ &= \Pr[A]. \end{aligned} \quad (\text{def of } I_A)$$

For example, if A is the event that a coin with bias p comes up heads, then $\text{Ex}[I_A] = \Pr[I_A = 1] = p$.

18.4.4 Alternate Definition of Expectation

There is another standard way to define expectation.

Theorem 18.4.3. *For any random variable R ,*

$$\text{Ex}[R] = \sum_{x \in \text{range}(R)} x \cdot \Pr[R = x]. \quad (18.2)$$

The proof of Theorem 18.4.3, like many of the elementary proofs about expectation in this chapter, follows by judicious regrouping of terms in equation (18.1):

Proof. Suppose R is defined on a sample space $\mathcal{S}$. Then,

$$\begin{aligned} \text{Ex}[R] &::= \sum_{\omega \in \mathcal{S}} R(\omega) \Pr[\omega] \\ &= \sum_{x \in \text{range}(R)} \sum_{\omega \in [R=x]} R(\omega) \Pr[\omega] \\ &= \sum_{x \in \text{range}(R)} \sum_{\omega \in [R=x]} x \Pr[\omega] \quad (\text{def of the event } [R = x]) \\ &= \sum_{x \in \text{range}(R)} x \left(\sum_{\omega \in [R=x]} \Pr[\omega] \right) \quad (\text{factoring } x \text{ from the inner sum}) \\ &= \sum_{x \in \text{range}(R)} x \cdot \Pr[R = x]. \quad (\text{def of } \Pr[R = x]) \end{aligned}$$

The first equality follows because the events $[R = x]$ for $x \in \text{range}(R)$ partition the sample space $\mathcal{S}$, so summing over the outcomes in $[R = x]$ for $x \in \text{range}(R)$ is the same as summing over $\mathcal{S}$. ■

In general, equation (18.2) is more useful than the defining equation (18.1) for calculating expected values. It also has the advantage that it does not depend on the sample space, but only on the density function of the random variable. On the other hand, summing over all outcomes as in equation (18.1) sometimes yields easier proofs about general properties of expectation.

Medians

The mean of a random variable is not the same as the *median*. The median is the *midpoint* of a distribution.

Definition 18.4.4. The *median* of a random variable R is the value $x \in \text{range}(R)$ such that

$$\begin{aligned} \Pr[R \leq x] &\leq \frac{1}{2} & \text{and} \\ \Pr[R > x] &< \frac{1}{2}. \end{aligned}$$

We won't devote much attention to the median. The expected value is more useful and has much more interesting properties.

18.4.5 Conditional Expectation

Just like event probabilities, expectations can be conditioned on some event. Given a random variable R , the expected value of R conditioned on an event A is the probability-weighted average value of R over outcomes in A . More formally:

Definition 18.4.5. The *conditional expectation* $\text{Ex}[R \mid A]$ of a random variable R given event A is:

$$\text{Ex}[R \mid A] ::= \sum_{r \in \text{range}(R)} r \cdot \Pr[R = r \mid A]. \quad (18.3)$$

For example, we can compute the expected value of a roll of a fair die, given that the number rolled is at least 4. We do this by letting R be the outcome of a roll of the die. Then by equation (18.3),

$$\text{Ex}[R \mid R \geq 4] = \sum_{i=1}^6 i \cdot \Pr[R = i \mid R \geq 4] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 + 4 \cdot \frac{1}{3} + 5 \cdot \frac{1}{3} + 6 \cdot \frac{1}{3} = 5.$$

Conditional expectation is useful in dividing complicated expectation calculations into simpler cases. We can find a desired expectation by calculating the conditional expectation in each simple case and averaging them, weighing each case by its probability.

For example, suppose that 49.8% of the people in the world are male and the rest female—which is more or less true. Also suppose the expected height of a randomly chosen male is 5' 11", while the expected height of a randomly chosen female is 5' 5." What is the expected height of a randomly chosen person? We can calculate this by averaging the heights of men and women. Namely, let H be the height (in feet) of a randomly chosen person, and let M be the event that the person is male and F the event that the person is female. Then

$$\begin{aligned}\text{Ex}[H] &= \text{Ex}[H \mid M] \Pr[M] + \text{Ex}[H \mid F] \Pr[F] \\ &= (5 + 11/12) \cdot 0.498 + (5 + 5/12) \cdot 0.502 \\ &= 5.665\end{aligned}$$

which is a little less than 5' 8."

This method is justified by:

Theorem 18.4.6 (Law of Total Expectation). *Let R be a random variable on a sample space $\mathcal{S}$, and suppose that $A_1, A_2, \dots$, is a partition of $\mathcal{S}$. Then*

$$\text{Ex}[R] = \sum_i \text{Ex}[R \mid A_i] \Pr[A_i].$$

Proof.

$$\begin{aligned}\text{Ex}[R] &= \sum_{r \in \text{range}(R)} r \cdot \Pr[R = r] && \text{(by 18.2)} \\ &= \sum_r r \cdot \sum_i \Pr[R = r \mid A_i] \Pr[A_i] && \text{(Law of Total Probability)} \\ &= \sum_r \sum_i r \cdot \Pr[R = r \mid A_i] \Pr[A_i] && \text{(distribute constant } r) \\ &= \sum_i \sum_r r \cdot \Pr[R = r \mid A_i] \Pr[A_i] && \text{(exchange order of summation)} \\ &= \sum_i \Pr[A_i] \sum_r r \cdot \Pr[R = r \mid A_i] && \text{(factor constant } \Pr[A_i]) \\ &= \sum_i \Pr[A_i] \text{Ex}[R \mid A_i]. && \text{(Def 18.4.5 of cond. expectation)}\end{aligned}$$

■

18.4.6 Mean Time to Failure

A computer program crashes at the end of each hour of use with probability p , if it has not crashed already. What is the expected time until the program crashes?

This will be easy to figure out using the Law of Total Expectation, Theorem 18.4.6. Specifically, we want to find $\text{Ex}[C]$ where C is the number of hours until the first crash. We'll do this by conditioning on whether or not the crash occurs in the first hour.

So let A to be the event that the system fails on the first step and $\bar{A}$ to be the complementary event that the system does not fail on the first step. Then the mean time to failure $\text{Ex}[C]$ is

$$\text{Ex}[C] = \text{Ex}[C \mid A] \Pr[A] + \text{Ex}[C \mid \bar{A}] \Pr[\bar{A}]. \quad (18.4)$$

Since A is the condition that the system crashes on the first step, we know that

$$\text{Ex}[C \mid A] = 1. \quad (18.5)$$

Since $\bar{A}$ is the condition that the system does *not* crash on the first step, conditioning on $\bar{A}$ is equivalent to taking a first step without failure and then starting over without conditioning. Hence,

$$\text{Ex}[C \mid \bar{A}] = 1 + \text{Ex}[C]. \quad (18.6)$$

Plugging (18.5) and (18.6) into (18.4):

$$\begin{aligned} \text{Ex}[C] &= 1 \cdot p + (1 + \text{Ex}[C])(1 - p) \\ &= p + 1 - p + (1 - p) \text{Ex}[C] \\ &= 1 + (1 - p) \text{Ex}[C]. \end{aligned}$$

Then, rearranging terms gives

$$1 = \text{Ex}[C] - (1 - p) \text{Ex}[C] = p \text{Ex}[C],$$

and thus

$$\text{Ex}[C] = 1/p.$$

The general principle here is well-worth remembering.

Mean Time to Failure

If a system independently fails at each time step with probability p , then the expected number of steps up to the first failure is $1/p$.

So, for example, if there is a 1% chance that the program crashes at the end of each hour, then the expected time until the program crashes is $1/0.01 = 100$ hours.

As a further example, suppose a couple wants to have a baby girl. For simplicity assume there is a 50% chance that each child they have is a girl, and the genders of their children are mutually independent. If the couple insists on having children until they get a girl, then how many baby boys should they expect first?

This is really a variant of the previous problem. The question, “How many hours until the program crashes?” is mathematically the same as the question, “How many children must the couple have until they get a girl?” In this case, a crash corresponds to having a girl, so we should set $p = 1/2$. By the preceding analysis, the couple should expect a baby girl after having $1/p = 2$ children. Since the last of these will be the girl, they should expect just one boy.

Something to think about: If every couple follows the strategy of having children until they get a girl, what will eventually happen to the fraction of girls born in this world?

Using the Law of Total Expectation to find expectations is a worthwhile approach to keep in mind, but it’s good review to derive the same formula directly from the definition of expectation. Namely, the probability that the first crash occurs in the i th hour for some $i > 0$ is the probability, $(1 - p)^{i-1}$, that it does not crash in each of the first $i - 1$ hours, times the probability, p , that it does crash in the i th hour. So

$$\begin{aligned} \text{Ex}[C] &= \sum_{i \in \mathbb{N}} i \cdot \Pr[C = i] && \text{(by (18.2))} \\ &= \sum_{i \in \mathbb{N}} i (1 - p)^{i-1} p \\ &= \frac{p}{1 - p} \cdot \sum_{i \in \mathbb{N}} i (1 - p)^i. \end{aligned} \tag{18.7}$$

But we’ve already seen a sum like this last one (you did remember this, right?), namely, equation (14.13):

$$\sum_{i \in \mathbb{N}} i x^i = \frac{x}{(1 - x)^2}.$$

Combining (14.13) with (18.7) gives

$$\text{Ex}[C] = \frac{p}{1 - p} \cdot \frac{1 - p}{(1 - (1 - p))^2} = \frac{1}{p}$$

as expected.

For the record, we’ll state a formal version of this result. A random variable like C that counts steps to first failure is said to have a *geometric distribution* with paramter p .

Definition 18.4.7. A random variable, C , has a *geometric distribution* with parameter p iff $\text{codomain}(C) = \mathbb{Z}^+$ and

$$\Pr[C = i] = (1 - p)^{i-1} p.$$

Lemma 18.4.8. If a random variable C has a geometric distribution with parameter p , then

$$\text{Ex}[C] = \frac{1}{p}. \quad (18.8)$$

18.4.7 Expected Returns in Gambling Games

Some of the most interesting examples of expectation can be explained in terms of gambling games. For straightforward games where you win w dollars with probability p and you lose x dollars with probability $1 - p$, it is easy to compute your *expected return* or *winnings*. It is simply

$$pw - (1 - p)x \text{ dollars.}$$

For example, if you are flipping a fair coin and you win \$1 for heads and you lose \$1 for tails, then your expected winnings are

$$\frac{1}{2} \cdot 1 - \left(1 - \frac{1}{2}\right) \cdot 1 = 0.$$

In such cases, the game is said to be *fair* since your expected return is zero.

Some gambling games are more complicated and thus more interesting. The following game where the winners split a pot is representative of many poker games, betting pools, and lotteries.

Splitting the Pot

After your last encounter with biker dude, one thing led to another and you have dropped out of school and become a Hell’s Angel. It’s late on a Friday night and, feeling nostalgic for the old days, you drop by your old hangout, where you encounter two of your former TAs, Eric and Nick. Eric and Nick propose that you join them in a simple wager. Each player will put \$2 on the bar and secretly write “heads” or “tails” on their napkin. Then one player will flip a fair coin. The \$6 on the bar will then be divided equally among the players who correctly predicted the outcome of the coin toss.

After your life-altering encounter with strange dice, you are more than a little skeptical. So Eric and Nick agree to let you be the one to flip the coin. This certainly seems fair. How can you lose?

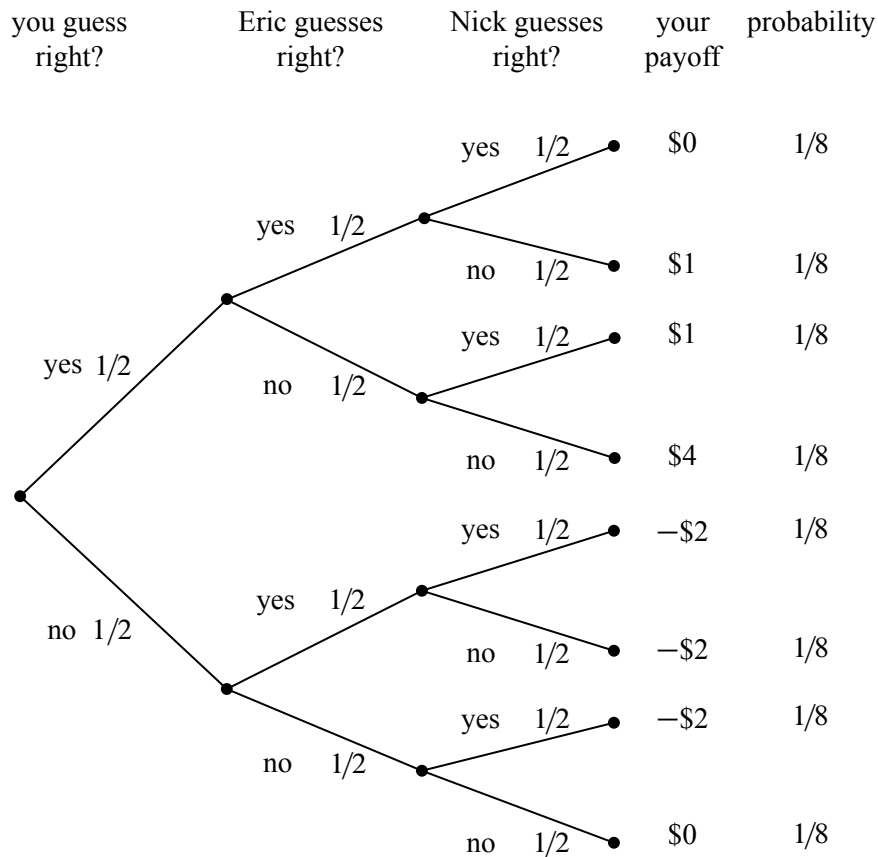


Figure 18.6 The tree diagram for the game where three players each wager \$2 and then guess the outcome of a fair coin toss. The winners split the pot.

But you have learned your lesson and so before agreeing, you go through the four-step method and write out the tree diagram to compute your expected return. The tree diagram is shown in Figure 18.6.

The “payoff” values in Figure 18.6 are computed by dividing the \$6 pot¹ among those players who guessed correctly and then subtracting the \$2 that you put into the pot at the beginning. For example, if all three players guessed correctly, then your payoff is \$0, since you just get back your \$2 wager. If you and Nick guess correctly and Eric guessed wrong, then your payoff is

$$\frac{6}{2} - 2 = 1.$$

¹The money invested in a wager is commonly referred to as the *pot*.

In the case that everyone is wrong, you all agree to split the pot and so, again, your payoff is zero.

To compute your expected return, you use equation (18.2):

$$\begin{aligned} \text{Ex}[\text{payoff}] &= 0 \cdot \frac{1}{8} + 1 \cdot \frac{1}{8} + 1 \cdot \frac{1}{8} + 4 \cdot \frac{1}{8} \\ &\quad + (-2) \cdot \frac{1}{8} + (-2) \cdot \frac{1}{8} + (-2) \cdot \frac{1}{8} + 0 \cdot \frac{1}{8} \\ &= 0. \end{aligned}$$

This confirms that the game is fair. So, for old time’s sake, you break your solemn vow to never ever engage in strange gambling games.

The Impact of Collusion

Needless to say, things are not turning out well for you. The more times you play the game, the more money you seem to be losing. After 1000 wagers, you have lost over \$500. As Nick and Eric are consoling you on your “bad luck,” you remember how rapidly the tails of the binomial distribute decrease, suggesting that the probability of losing \$500 in 1000 fair \$2 wagers is less than the probability of being struck by lightning while playing poker and being dealt four Aces. How can this be?

It is possible that you are truly very very unlucky. But it is more likely that something is wrong with the tree diagram in Figure 18.6 and that “something” just might have something to do with the possibility that Nick and Eric are colluding against you.

To be sure, Nick and Eric can only guess the outcome of the coin toss with probability $1/2$, but what if Nick and Eric always guess differently? In other words, what if Nick always guesses “tails” when Eric guesses “heads,” and vice-versa? This would result in a slightly different tree diagram, as shown in Figure 18.7.

The payoffs for each outcome are the same in Figures 18.6 and 18.7, but the probabilities of the outcomes are different. For example, it is no longer possible for all three players to guess correctly, since Nick and Eric are always guessing differently. More importantly, the outcome where your payoff is \$4 is also no longer possible. Since Nick and Eric are always guessing differently, one of them will always get a share of the pot. As you might imagine, this is not good for you!

When we use equation (18.2) to compute your expected return in the collusion

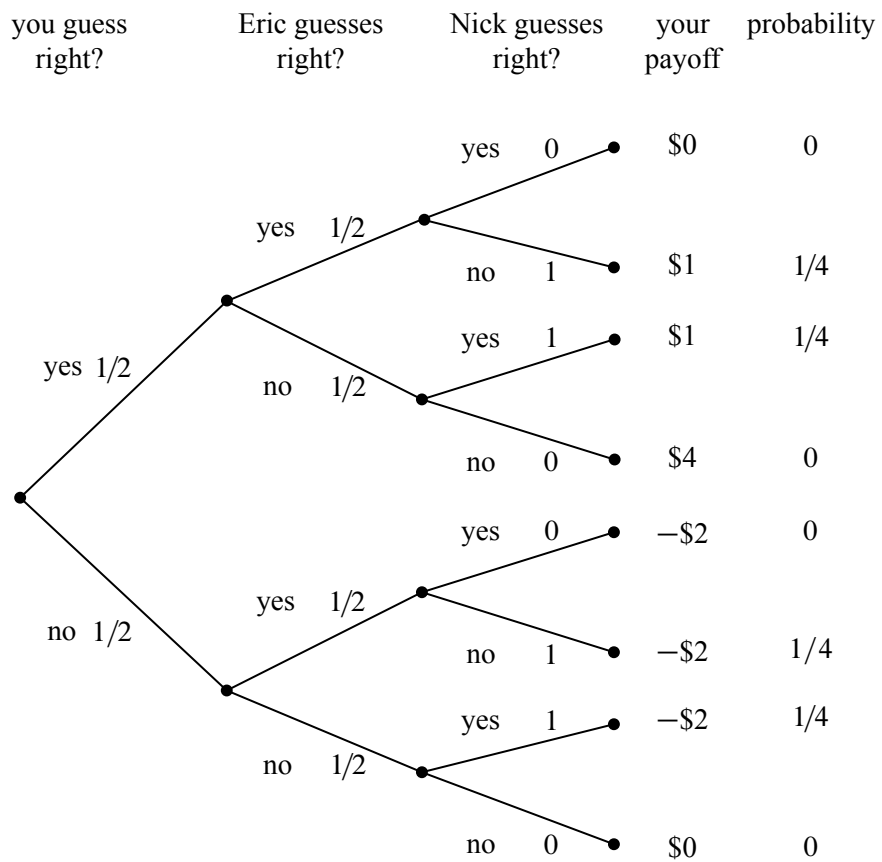


Figure 18.7 The revised tree diagram reflecting the scenario where Nick always guesses the opposite of Eric.

scenario, we find that

$$\begin{aligned}\text{Ex}[\text{payoff}] &= 0 \cdot 0 + 1 \cdot \frac{1}{4} + 1 \cdot \frac{1}{4} + 4 \cdot 0 \\ &\quad + (-2) \cdot 0 + (-2) \cdot \frac{1}{4} + (-2) \cdot \frac{1}{4} + 0 \cdot 0 \\ &= -\frac{1}{2}.\end{aligned}$$

This is very bad indeed. By colluding, Nick and Eric have made it so that you expect to lose \$.50 every time you play. No wonder you lost \$500 over the course of 1000 wagers.

Maybe it would be a good idea to go back to school —your Hell’s Angels buds may not be too happy that you just lost their \$500.

How to Win the Lottery

Similar opportunities to “collude” arise in many betting games. For example, consider the typical weekly football betting pool, where each participant wagers \$10 and the participants that pick the most games correctly split a large pot. The pool seems fair if you think of it as in Figure 18.6. But, in fact, if two or more players collude by guessing differently, they can get an “unfair” advantage at your expense!

In some cases, the collusion is inadvertent and you can profit from it. For example, many years ago, a former MIT Professor of Mathematics named Herman Chernoff figured out a way to make money by playing the state lottery. This was surprising since state lotteries typically have very poor expected returns. That’s because the state usually takes a large share of the wagers before distributing the rest of the pot among the winners. Hence, anyone who buys a lottery ticket is expected to *lose* money. So how did Chernoff find a way to make money? It turned out to be easy!

In a typical state lottery,

- all players pay \$1 to play and select 4 numbers from 1 to 36,
- the state draws 4 numbers from 1 to 36 uniformly at random,
- the states divides 1/2 of the money collected among the people who guessed correctly and spends the other half redecorating the governor’s residence.

This is a lot like the game you played with Nick and Eric, except that there are more players and more choices. Chernoff discovered that a small set of numbers was selected by a large fraction of the population. Apparently many people think the same way; they pick the same numbers not on purpose as in the previous game with Nick and Eric, but based on Manny’s batting average or today’s date.

It was as if the players were colluding to lose! If any one of them guessed correctly, then they’d have to split the pot with many other players. By selecting numbers uniformly at random, Chernoff was unlikely to get one of these favored sequences. So if he won, he’d likely get the whole pot! By analyzing actual state lottery data, he determined that he could win an average of 7 cents on the dollar. In other words, his expected return was not $-\$.50$ as you might think, but $+\$.07$.²

Inadvertent collusion often arises in betting pools and is a phenomenon that you can take advantage of. For example, suppose you enter a Super Bowl betting pool where the goal is to get closest to the total number of points scored in the game. Also suppose that the average Super Bowl has a total of 30 point scored and that everyone knows this. Then most people will guess around 30 points. Where should you guess? Well, you should guess just outside of this range because you get to cover a lot more ground and you don’t share the pot if you win. Of course, if you are in a pool with math students and they all know this strategy, then maybe you should guess 30 points after all.

18.5 Linearity of Expectation

Expected values obey a simple, very helpful rule called *Linearity of Expectation*. Its simplest form says that the expected value of a sum of random variables is the sum of the expected values of the variables.

Theorem 18.5.1. *For any random variables R_1 and R_2 ,*

$$\text{Ex}[R_1 + R_2] = \text{Ex}[R_1] + \text{Ex}[R_2].$$

Proof. Let $T ::= R_1 + R_2$. The proof follows straightforwardly by rearranging terms in equation (18.1) in the definition of expectation:

$$\begin{aligned} \text{Ex}[T] &::= \sum_{\omega \in \mathcal{S}} T(\omega) \cdot \text{Pr}[\omega] \\ &= \sum_{\omega \in \mathcal{S}} (R_1(\omega) + R_2(\omega)) \cdot \text{Pr}[\omega] && \text{(def of } T) \\ &= \sum_{\omega \in \mathcal{S}} R_1(\omega) \text{Pr}[\omega] + \sum_{\omega \in \mathcal{S}} R_2(\omega) \text{Pr}[\omega] && \text{(rearranging terms)} \\ &= \text{Ex}[R_1] + \text{Ex}[R_2]. && \text{(by (18.1))} \end{aligned}$$

■

²Most lotteries now offer randomized tickets to help smooth out the distribution of selected sequences.

A small extension of this proof, which we leave to the reader, implies

Theorem 18.5.2. *For random variables R_1, R_2 and constants $a_1, a_2 \in \mathbb{R}$,*

$$\text{Ex}[a_1 R_1 + a_2 R_2] = a_1 \text{Ex}[R_1] + a_2 \text{Ex}[R_2].$$

In other words, expectation is a linear function. A routine induction extends the result to more than two variables:

Corollary 18.5.3 (Linearity of Expectation). *For any random variables $R_1, \dots, R_k$ and constants $a_1, \dots, a_k \in \mathbb{R}$,*

$$\text{Ex} \left[\sum_{i=1}^k a_i R_i \right] = \sum_{i=1}^k a_i \text{Ex}[R_i].$$

The great thing about linearity of expectation is that *no independence is required*. This is really useful, because dealing with independence is a pain, and we often need to work with random variables that are not known to be independent.

As an example, let's compute the expected value of the sum of two fair dice.

18.5.1 Expected Value of Two Dice

What is the expected value of the sum of two fair dice?

Let the random variable R_1 be the number on the first die, and let R_2 be the number on the second die. We observed earlier that the expected value of one die is 3.5. We can find the expected value of the sum using linearity of expectation:

$$\text{Ex}[R_1 + R_2] = \text{Ex}[R_1] + \text{Ex}[R_2] = 3.5 + 3.5 = 7.$$

Notice that we did *not* have to assume that the two dice were independent. The expected sum of two dice is 7, even if they are glued together (provided each individual die remains fair after the gluing). Proving that this expected sum is 7 with a tree diagram would be a bother: there are 36 cases. And if we did not assume that the dice were independent, the job would be really tough!

18.5.2 Sums of Indicator Random Variables

Linearity of expectation is especially useful when you have a sum of indicator random variables. As an example, suppose there is a dinner party where n men check their hats. The hats are mixed up during dinner, so that afterward each man receives a random hat. In particular, each man gets his own hat with probability $1/n$. What is the expected number of men who get their own hat?

Letting G be the number of men that get their own hat, we want to find the expectation of G . But all we know about G is that the probability that a man gets his own hat back is $1/n$. There are many different probability distributions of hat permutations with this property, so we don't know enough about the distribution of G to calculate its expectation directly. But linearity of expectation makes the problem really easy.

The trick³ is to express G as a sum of indicator variables. In particular, let G_i be an indicator for the event that the i th man gets his own hat. That is, $G_i = 1$ if the i th man gets his own hat, and $G_i = 0$ otherwise. The number of men that get their own hat is then the sum of these indicator random variables:

$$G = G_1 + G_2 + \cdots + G_n. \quad (18.9)$$

These indicator variables are *not* mutually independent. For example, if $n - 1$ men all get their own hats, then the last man is certain to receive his own hat. But, since we plan to use linearity of expectation, we don't have worry about independence!

Since G_i is an indicator random variable, we know from Lemma 18.4.2 that

$$\text{Ex}[G_i] = \Pr[G_i = 1] = 1/n. \quad (18.10)$$

By Linearity of Expectation and equation (18.9), this means that

$$\begin{aligned} \text{Ex}[G] &= \text{Ex}[G_1 + G_2 + \cdots + G_n] \\ &= \text{Ex}[G_1] + \text{Ex}[G_2] + \cdots + \text{Ex}[G_n] \\ &= \overbrace{\frac{1}{n} + \frac{1}{n} + \cdots + \frac{1}{n}}^n \\ &= 1. \end{aligned}$$

So even though we don't know much about how hats are scrambled, we've figured out that on average, just one man gets his own hat back!

More generally, Linearity of Expectation provides a very good method for computing the expected number of events that will happen.

Theorem 18.5.4. *Given any collection of events $A_1, A_2, \dots, A_n$, the expected number of events that will occur is*

$$\sum_{i=1}^n \Pr[A_i].$$

³We are going to use this trick a lot so it is important to understand it.

For example, A_i could be the event that the i th man gets the right hat back. But in general, it could be any subset of the sample space, and we are asking for the expected number of events that will contain a random sample point.

Proof. Define R_i to be the indicator random variable for A_i , where $R_i(\omega) = 1$ if $w \in A_i$ and $R_i(\omega) = 0$ if $w \notin A_i$. Let $R = R_1 + R_2 + \cdots + R_n$. Then

$$\begin{aligned} \text{Ex}[R] &= \sum_{i=1}^n \text{Ex}[R_i] && \text{(by Linearity of Expectation)} \\ &= \sum_{i=1}^n \Pr[R_i = 1] && \text{(by Lemma 18.4.2)} \\ &= \sum_{i=1}^n \Pr[A_i]. && \text{(def of indicator variable)} \end{aligned}$$

So whenever you are asked for the expected number of events that occur, all you have to do is sum the probabilities that each event occurs. Independence is not needed.

18.5.3 Expectation of a Binomial Distribution

Suppose that we independently flip n biased coins, each with probability p of coming up heads. What is the expected number of heads?

Let J be the random variable denoting the number of heads. Then J has a binomial distribution with parameters n , p , and

$$\Pr[J = k] = \binom{n}{k} p^k (1-p)^{n-k}.$$

Applying equation (18.2), this means that

$$\text{Ex}[J] = \sum_{k=0}^n k \Pr[J = k] = \sum_{k=0}^n k \binom{n}{k} p^k (1-p)^{n-k}. \quad (18.11)$$

This sum looks a tad nasty, but linearity of expectation leads to an easy derivation of a simple closed form. We just express J as a sum of indicator random variables, which is easy. Namely, let J_i be the indicator random variable for the i th coin coming up heads, that is,

$$J_i ::= \begin{cases} 1 & \text{if the } i\text{th coin is heads} \\ 0 & \text{if the } i\text{th coin is tails.} \end{cases}$$

Then the number of heads is simply

$$J = J_1 + J_2 + \cdots + J_n.$$

By Theorem 18.5.4,

$$\text{Ex}[J] = \sum_{i=1}^n \text{Pr}[J_i] = pn. \quad (18.12)$$

That really was easy. If we flip n mutually independent coins, we expect to get pn heads. Hence the expected value of a binomial distribution with parameters n and p is simply pn .

But what if the coins are not mutually independent? It doesn't matter—the answer is still pn because Linearity of Expectation and Theorem 18.5.4 do not assume any independence.

If you are not yet convinced that Linearity of Expectation and Theorem 18.5.4 are powerful tools, consider this: without even trying, we have used them to prove a complicated looking identity, namely,

$$\sum_{k=0}^n k \binom{n}{k} p^k (1-p)^{n-k} = pn, \quad (18.13)$$

which follows by combining equations (18.11) and (18.12).⁴

The next section has an even more convincing illustration of the power of linearity to solve a challenging problem.

⁴Equation (18.13) may look daunting initially, but it is, after all, pretty similar to the binomial identity, and that connection leads to a simple derivation by algebra. Namely, starting with the binomial identity

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

we can differentiate with respect to x (as in Section 14.1.6) to get

$$n(x + y)^{n-1} = \sum_{k=0}^n k \binom{n}{k} x^{k-1} y^{n-k}.$$

Multiplying both sides by x gives

$$xn(x + y)^{n-1} = \sum_{k=0}^n k \binom{n}{k} x^k y^{n-k} \quad (18.14)$$

Plugging p for x and $1 - p$ for y in (18.14) then yields (18.13).

18.5.4 The Coupon Collector Problem

Every time we purchase a kid’s meal at Taco Bell, we are graciously presented with a miniature “Racin’ Rocket” car together with a launching device which enables us to project our new vehicle across any tabletop or smooth floor at high velocity. Truly, our delight knows no bounds.

There are n different types of Racin’ Rocket cars (blue, green, red, gray, etc.). The type of car awarded to us each day by the kind woman at the Taco Bell register appears to be selected uniformly and independently at random. What is the expected number of kid’s meals that we must purchase in order to acquire at least one of each type of Racin’ Rocket car?

The same mathematical question shows up in many guises: for example, what is the expected number of people you must poll in order to find at least one person with each possible birthday? Here, instead of collecting Racin’ Rocket cars, you’re collecting birthdays. The general question is commonly called the *coupon collector problem* after yet another interpretation.

A clever application of linearity of expectation leads to a simple solution to the coupon collector problem. Suppose there are five different types of Racin’ Rocket cars, and we receive this sequence:

blue green green red blue orange blue orange gray.

Let’s partition the sequence into 5 segments:

$\underbrace{\text{blue}}_{X_0}$
 $\underbrace{\text{green}}_{X_1}$
 $\underbrace{\text{green red}}_{X_2}$
 $\underbrace{\text{blue orange}}_{X_3}$
 $\underbrace{\text{blue orange gray}}_{X_4}$

The rule is that a segment ends whenever we get a new kind of car. For example, the middle segment ends when we get a red car for the first time. In this way, we can break the problem of collecting every type of car into stages. Then we can analyze each stage individually and assemble the results using linearity of expectation.

Let’s return to the general case where we’re collecting n Racin’ Rockets. Let X_k be the length of the k th segment. The total number of kid’s meals we must purchase to get all n Racin’ Rockets is the sum of the lengths of all these segments:

$$T = X_0 + X_1 + \cdots + X_{n-1}$$

Now let’s focus our attention on X_k , the length of the k th segment. At the beginning of segment k , we have k different types of car, and the segment ends when we acquire a new type. When we own k types, each kid’s meal contains a type that we already have with probability k/n . Therefore, each meal contains a new type of car with probability $1 - k/n = (n - k)/n$. Thus, the expected number

of meals until we get a new kind of car is $n/(n-k)$ by the Mean Time to Failure rule. This means that

$$\text{Ex}[X_k] = \frac{n}{n-k}.$$

Linearity of expectation, together with this observation, solves the coupon collector problem:

$$\begin{aligned} \text{Ex}[T] &= \text{Ex}[X_0 + X_1 + \cdots + X_{n-1}] \\ &= \text{Ex}[X_0] + \text{Ex}[X_1] + \cdots + \text{Ex}[X_{n-1}] \\ &= \frac{n}{n-0} + \frac{n}{n-1} + \cdots + \frac{n}{3} + \frac{n}{2} + \frac{n}{1} \\ &= n \left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{3} + \frac{1}{2} + \frac{1}{1} \right) \\ &= n \left(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1} + \frac{1}{n} \right) \\ &= nH_n \\ &\sim n \ln n. \end{aligned} \tag{18.15}$$

Wow! It's those Harmonic Numbers again!

We can use equation (18.15) to answer some concrete questions. For example, the expected number of die rolls required to see every number from 1 to 6 is:

$$6H_6 = 14.7 \dots$$

And the expected number of people you must poll to find at least one person with each possible birthday is:

$$365H_{365} = 2364.6 \dots$$

18.5.5 Infinite Sums

Linearity of expectation also works for an infinite number of random variables provided that the variables satisfy some stringent absolute convergence criteria.

Theorem 18.5.5 (Linearity of Expectation). *Let $R_0, R_1, \dots$, be random variables such that*

$$\sum_{i=0}^{\infty} \text{Ex}[|R_i|]$$

converges. Then

$$\text{Ex} \left[\sum_{i=0}^{\infty} R_i \right] = \sum_{i=0}^{\infty} \text{Ex}[R_i].$$

Proof. Let $T ::= \sum_{i=0}^{\infty} R_i$.

We leave it to the reader to verify that, under the given convergence hypothesis, all the sums in the following derivation are absolutely convergent, which justifies rearranging them as follows:

$$\begin{aligned} \sum_{i=0}^{\infty} \text{Ex}[R_i] &= \sum_{i=0}^{\infty} \sum_{s \in \mathcal{S}} R_i(s) \cdot \text{Pr}[s] && \text{(Def. 18.4.1)} \\ &= \sum_{s \in \mathcal{S}} \sum_{i=0}^{\infty} R_i(s) \cdot \text{Pr}[s] && \text{(exchanging order of summation)} \\ &= \sum_{s \in \mathcal{S}} \left[\sum_{i=0}^{\infty} R_i(s) \right] \cdot \text{Pr}[s] && \text{(factoring out Pr}[s]) \\ &= \sum_{s \in \mathcal{S}} T(s) \cdot \text{Pr}[s] && \text{(Def. of } T) \\ &= \text{Ex}[T] && \text{(Def. 18.4.1)} \\ &= \text{Ex} \left[\sum_{i=0}^{\infty} R_i \right]. && \text{(Def. of } T). \blacksquare \end{aligned}$$

18.5.6 Expectations of Products

While the expectation of a sum is the sum of the expectations, the same is usually not true for products. For example, suppose that we roll a fair 6-sided die and denote the outcome with the random variable R . Does $\text{Ex}[R \cdot R] = \text{Ex}[R] \cdot \text{Ex}[R]$?

We know that $\text{Ex}[R] = 3\frac{1}{2}$ and thus $\text{Ex}[R]^2 = 12\frac{1}{4}$. Let's compute $\text{Ex}[R^2]$ to see if we get the same result.

$$\begin{aligned} \text{Ex}[R^2] &= \sum_{\omega \in \mathcal{S}} R^2(\omega) \text{Pr}[\omega] = \sum_{i=1}^6 i^2 \cdot \text{Pr}[R_i = i] \\ &= \frac{1^2}{6} + \frac{2^2}{6} + \frac{3^2}{6} + \frac{4^2}{6} + \frac{5^2}{6} + \frac{6^2}{6} = 15\frac{1}{6} \neq 12\frac{1}{4}. \end{aligned}$$

That is,

$$\text{Ex}[R \cdot R] \neq \text{Ex}[R] \cdot \text{Ex}[R].$$

So the expectation of a product is not always equal to the product of the expectations.

There is a special case when such a relationship *does* hold however; namely, when the random variables in the product are *independent*.

Theorem 18.5.6. *For any two independent random variables R_1, R_2 ,*

$$\text{Ex}[R_1 \cdot R_2] = \text{Ex}[R_1] \cdot \text{Ex}[R_2].$$

The proof follows by judicious rearrangement of terms in the sum that defines $\text{Ex}[R_1 \cdot R_2]$. Details appear in Problem 18.17.

Theorem 18.5.6 extends routinely to a collection of mutually independent variables.

Corollary 18.5.7. *[Expectation of Independent Product]*

If random variables $R_1, R_2, \dots, R_k$ are mutually independent, then

$$\text{Ex} \left[\prod_{i=1}^k R_i \right] = \prod_{i=1}^k \text{Ex}[R_i].$$

Problems for Section 18.2

Practice Problems

Problem 18.1. (a) Prove that if A and B are independent events, then so are A and $\overline{B}$.

(b) Let I_A and I_B be the indicator variables for events A and B . Prove that I_A and I_B are independent iff A and B are independent.

Hint: For any event, E , let $E^1 ::= E$ and $E^0 ::= \overline{E}$. So the event $[I_E = a]$ is the same as E^a .

Homework Problems

Problem 18.2.

Let R, S , and T be random variables with the same codomain, V .

(a) Suppose R is uniform—that is,

$$\Pr[R = b] = \frac{1}{|V|},$$

for all $b \in V$ —and R is independent of S . Originally this text had the following argument:

The probability that $R = S$ is the same as the probability that R takes whatever value S happens to have, therefore

$$\Pr[R = S] = \frac{1}{|V|}. \quad (18.16)$$

Are you convinced by this argument? Write out a careful proof of (18.16).

Hint: The event $[R = S]$ is a disjoint union of events

$$[R = S] = \bigcup_{b \in V} [R = b \text{ AND } S = b].$$

(b) Let $S \times T$ be the random variable giving the values of S and T .⁵ Now suppose R has a uniform distribution, and R is independent of $S \times T$. How about this argument?

The probability that $R = S$ is the same as the probability that R equals the first coordinate of whatever value $S \times T$ happens to have, and this probability remains equal to $1/|V|$ by independence. Therefore the event $[R = S]$ is independent of $[S = T]$.

Write out a careful proof that $[R = S]$ is independent of $[S = T]$.

(c) Let $V = \{1, 2, 3\}$ and R, S, T take the following values with equal probability,

111, 211, 123, 223, 132, 232.

Verify that

1. R is independent of $S \times T$,
2. The event $[R = S]$ is not independent of $[S = T]$.
3. S and T have a uniform distribution,

Problem 18.3.

Let R, S , and T be mutually independent random variables with the same codomain, V . Problem 18.2 showed that if R is uniform—that is,

$$\Pr[R = b] = \frac{1}{|V|},$$

⁵That is, $S \times T : \mathcal{S} \rightarrow V \times V$ where

$$(S \times T)(\omega) ::= (S(\omega), T(\omega))$$

for every outcome $\omega \in \mathcal{S}$.

for all $b \in V$, then

the events $[R = S]$ and $[S = T]$ are independent.

This implies that these events are also independent if T is uniform, since R and T are symmetric in this assertion. Prove converssely that if neither R nor T is uniform, then these events are not independent.

Problems for Section 18.3

Practice Problems

Problem 18.4.

Suppose X_1 , X_2 , and X_3 are three mutually independent random variables, each having the uniform distribution

$$\forall k, k \in \{1, 2, 3\}. \Pr[X_i = k] = \frac{1}{3}.$$

Let M be another random variable giving the maximum of these three random variables. What is the probability density function of M ?

Class Problems

Guess the Bigger Number Game

Team 1:

- Write different integers between 0 and 7 on two pieces of paper.
- Put the papers face down on a table.

Team 2:

- Turn over one paper and look at the number on it.
- Either stick with this number or switch to the unseen other number.

Team 2 wins if it chooses the larger number; else, Team 1 wins.

Problem 18.5.

The analysis in section 18.3.3 implies that Team 2 has a strategy that wins 4/7 of the time no matter how Team 1 plays. Can Team 2 do better? The answer is “no,”

because Team 1 has a strategy that guarantees that it wins at least $3/7$ of the time, no matter how Team 2 plays. Describe such a strategy for Team 1 and explain why it works.

Problem 18.6.

Suppose you have a biased coin that has probability p of flipping heads. Let J be the number of heads in n independent coin flips. So J has the general binomial distribution:

$$\text{PDF}_J(k) = \binom{n}{k} p^k q^{n-k}$$

where $q ::= 1 - p$.

(a) Show that

$$\begin{aligned} \text{PDF}_J(k-1) &< \text{PDF}_J(k) && \text{for } k < np + p, \\ \text{PDF}_J(k-1) &> \text{PDF}_J(k) && \text{for } k > np + p. \end{aligned}$$

(b) Conclude that the maximum value of PDF_J is asymptotically equal to

$$\frac{1}{\sqrt{2\pi npq}}.$$

Hint: For the asymptotic estimate, it's ok to assume that np is an integer, so by part (a), the maximum value is $\text{PDF}_J(np)$. Use Stirling's formula (14.30).

Homework Problems

Problem 18.7.

A drunken sailor wanders along main street, which conveniently consists of the points along the x axis with integral coordinates. In each step, the sailor moves one unit left or right along the x axis. A particular *path* taken by the sailor can be described by a sequence of “left” and “right” steps. For example, (left,left,right) describes the walk that goes left twice then goes right.

We model this scenario with a random walk graph whose vertices are the integers and with edges going in each direction between consecutive integers. All edges are labelled $1/2$.

The sailor begins his random walk at the origin. This is described by an initial distribution which labels the origin with probability 1 and all other vertices with probability 0. After one step, the sailor is equally likely to be at location 1 or -1 , so the distribution after one step gives label $1/2$ to the vertices 1 and -1 and labels all other vertices with probability 0.

(a) Give the distributions after the 2nd, 3rd, and 4th step by filling in the table of probabilities below, where omitted entries are 0. For each row, write all the nonzero entries so they have the same denominator.

	location								
	-4	-3	-2	-1	0	1	2	3	4
initially					1				
after 1 step				1/2	0	1/2			
after 2 steps			?	?	?	?	?		
after 3 steps		?	?	?	?	?	?	?	
after 4 steps	?	?	?	?	?	?	?	?	?

(b)

1. What is the final location of a t -step path that moves right exactly i times?
2. How many different paths are there that end at that location?
3. What is the probability that the sailor ends at this location?

(c) Let L be the random variable giving the sailor’s location after t steps, and let $B ::= (L + t)/2$. Use the answer to part (b) to show that B has an unbiased binomial density function.

(d) Again let L be the random variable giving the sailor’s location after t steps, where t is even. Show that

$$\Pr[|L| < \frac{\sqrt{t}}{2}] < \frac{1}{2}.$$

So there is a better than even chance that the sailor ends up at least $\sqrt{t}/2$ steps from where he started.

Hint: Work in terms of B . Then you can use an estimate that bounds the binomial distribution. Alternatively, observe that the origin is the most likely final location and then use the asymptotic estimate

$$\Pr[L = 0] = \Pr[B = t/2] \sim \sqrt{\frac{2}{\pi t}}.$$

Problems for Section 18.5

Practice Problems

Problem 18.8.

The vast majority of people have an above average number of fingers. Which of the following statements accounts for this phenomenon? Explain your reasoning.

1. Most people have a super secret extra bonus finger of which they are unaware.
2. A pedantic minority don't count their thumbs as fingers, while the majority of people do.
3. Polydactyly is rarer than amputation.
4. When you add up the total number of fingers among the world's population and then divide by the size of the population, you get a number less than ten.
5. This follows from Markov's Theorem, since no one has a negative number of fingers.
6. Missing fingers are much more common than extra ones.
7. Missing fingers are at least slightly more common than extra ones.

Problem 18.9.

A news article reporting on the departure of a school official from California to Alabama dryly commented that this move would raise the average IQ in both states. Explain.

Problem 18.10.

MIT students sometimes delay laundry for a few days. Assume all random values described below are mutually independent.

(a) A *busy* student must complete 3 problem sets before doing laundry. Each problem set requires 1 day with probability $2/3$ and 2 days with probability $1/3$. Let B be the number of days a busy student delays laundry. What is $\text{Ex}[B]$?

Example: If the first problem set requires 1 day and the second and third problem sets each require 2 days, then the student delays for $B = 5$ days.

(b) A *relaxed* student rolls a fair, 6-sided die in the morning. If he rolls a 1, then he does his laundry immediately (with zero days of delay). Otherwise, he delays for one day and repeats the experiment the following morning. Let R be the number of days a relaxed student delays laundry. What is $\text{Ex}[R]$?

Example: If the student rolls a 2 the first morning, a 5 the second morning, and a 1 the third morning, then he delays for $R = 2$ days.

(c) Before doing laundry, an *unlucky* student must recover from illness for a number of days equal to the product of the numbers rolled on two fair, 6-sided dice. Let U be the expected number of days an unlucky student delays laundry. What is $\text{Ex}[U]$?

Example: If the rolls are 5 and 3, then the student delays for $U = 15$ days.

(d) A student is *busy* with probability $1/2$, *relaxed* with probability $1/3$, and *unlucky* with probability $1/6$. Let D be the number of days the student delays laundry. What is $\text{Ex}[D]$?

Problem 18.11.

Each Math for Computer Science final exam will be graded according to a rigorous procedure:

- With probability $\frac{4}{7}$ the exam is graded by a *TA*, with probability $\frac{2}{7}$ it is graded by a *lecturer*, and with probability $\frac{1}{7}$, it is accidentally dropped behind the radiator and arbitrarily given a score of 84.
- *TAs* score an exam by scoring each problem individually and then taking the sum.
 - There are ten true/false questions worth 2 points each. For each, full credit is given with probability $\frac{3}{4}$, and no credit is given with probability $\frac{1}{4}$.
 - There are four questions worth 15 points each. For each, the score is determined by rolling two fair dice, summing the results, and adding 3.
 - The single 20 point question is awarded either 12 or 18 points with equal probability.
- *Lecturers* score an exam by rolling a fair die twice, multiplying the results, and then adding a “general impression” score.
 - With probability $\frac{4}{10}$, the general impression score is 40.
 - With probability $\frac{3}{10}$, the general impression score is 50.
 - With probability $\frac{3}{10}$, the general impression score is 60.

Assume all random choices during the grading process are independent.

(a) What is the expected score on an exam graded by a *TA*?

- (b) What is the expected score on an exam graded by a lecturer?
- (c) What is the expected score on a Math for Computer Science final exam?

Class Problems

Problem 18.12.

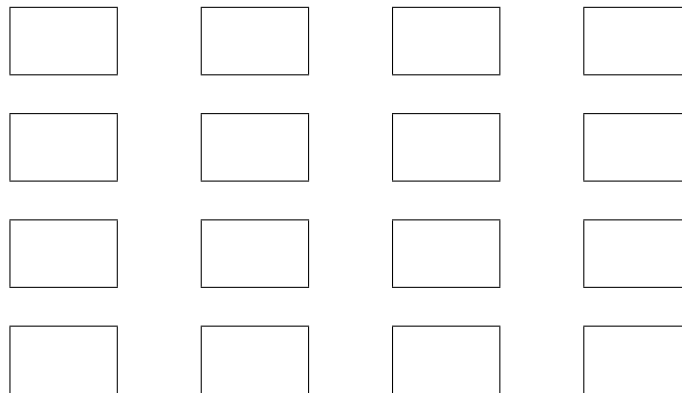
Let's see what it takes to make Carnival Dice fair. Here's the game with payoff parameter k : make three independent rolls of a fair die. If you roll a six

- no times, then you lose 1 dollar.
- exactly once, then you win 1 dollar.
- exactly twice, then you win two dollars.
- all three times, then you win k dollars.

For what value of k is this game fair?

Problem 18.13.

A classroom has sixteen desks in a 4×4 arrangement as shown below.



If there is a girl in front, behind, to the left, or to the right of a boy, then the two of them *flirt*. One student may be in multiple flirting couples; for example, a student in a corner of the classroom can flirt with up to two others, while a student in the center can flirt with as many as four others. Suppose that desks are occupied by boys and girls with equal probability and mutually independently. What is the expected number of flirting couples? *Hint*: Linearity.

Problem 18.14.

Here are seven propositions:

$$\begin{array}{cccc} x_1 & \text{OR} & x_3 & \text{OR} & \overline{x_7} \\ \overline{x_5} & \text{OR} & x_6 & \text{OR} & x_7 \\ x_2 & \text{OR} & \overline{x_4} & \text{OR} & x_6 \\ \overline{x_4} & \text{OR} & x_5 & \text{OR} & \overline{x_7} \\ x_3 & \text{OR} & \overline{x_5} & \text{OR} & \overline{x_8} \\ x_9 & \text{OR} & \overline{x_8} & \text{OR} & x_2 \\ \overline{x_3} & \text{OR} & x_9 & \text{OR} & x_4 \end{array}$$

Note that:

1. Each proposition is the disjunction (OR) of three terms of the form x_i or the form $\overline{x_i}$.
2. The variables in the three terms in each proposition are all different.

Suppose that we assign true/false values to the variables $x_1, \dots, x_9$ independently and with equal probability.

(a) What is the expected number of true propositions?

Hint: Let T_i be an indicator for the event that the i -th proposition is true.

(b) Use your answer to prove that for *any* set of 7 propositions satisfying the conditions 1. and 2., there is an assignment to the variables that makes all 7 of the propositions true.

Problem 18.15.

A *literal* is a propositional variable or its negation. A *k-clause* is an OR of k literals, with no variable occurring more than once in the clause. For example,

$$P \text{ OR } \overline{Q} \text{ OR } \overline{R} \text{ OR } V,$$

is a 4-clause, but

$$\overline{V} \text{ OR } \overline{Q} \text{ OR } \overline{X} \text{ OR } V,$$

is not, since V appears twice.

Let $\mathcal{S}$ be a set of n distinct k -clauses involving v variables. The variables in different k -clauses may overlap or be completely different, so $k \leq v \leq nk$.

A random assignment of true/false values will be made independently to each of the v variables, with true and false assignments equally likely. Write formulas in n , k , and v in answer to the first two parts below.

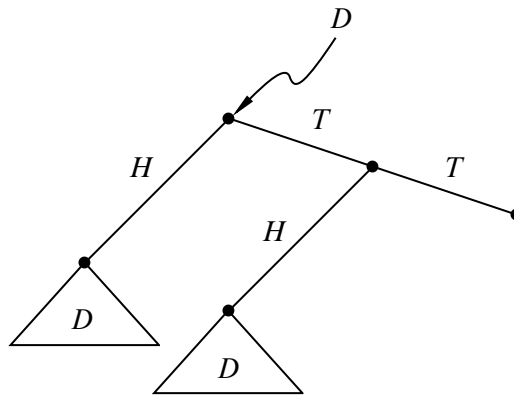


Figure 18.8 Sample space tree for coin toss until two consecutive heads.

(a) What is the probability that the last k -clause in $\mathcal{S}$ is true under the random assignment?

(b) What is the expected number of true k -clauses in $\mathcal{S}$?

(c) A set of propositions is *satisfiable* iff there is an assignment to the variables that makes all of the propositions true. Use your answer to part (b) to prove that if $n < 2^k$, then $\mathcal{S}$ is satisfiable.

Problem 18.16. (a) Suppose we flip a fair coin and let N_{TT} be the number of flips until the first time two Tails in a row appear. What is $\text{Ex}[N_{\text{TT}}]$?

Hint: Let D be the tree diagram for this process. Explain why D can be described by the tree in Figure 18.8

Use the **Law of Total Expectation** 18.4.6.

(b) Suppose we flip a fair coin until a Tail immediately followed by a Head come up. What is the expectation of the number N_{TH} of flips we perform?

(c) Suppose we now play a game: flip a fair coin until either TT or TH first occurs. You win if TT comes up first, lose if TH comes up first. Since TT takes 50% longer on average to turn up, your opponent agrees that he has the advantage. So you tell

him you’re willing to play if you pay him \$5 when he wins, but he merely pays you a 20% premium, that is, \$6, when you win.

If you do this, you’re sneakily taking advantage of your opponent’s untrained intuition, since you’ve gotten him to agree to unfair odds. What is your expected profit per game?

Problem 18.17.

Justify each line of the following proof that if R_1 and R_2 are *independent*, then

$$\text{Ex}[R_1 \cdot R_2] = \text{Ex}[R_1] \cdot \text{Ex}[R_2].$$

Proof.

$$\begin{aligned} & \text{Ex}[R_1 \cdot R_2] \\ &= \sum_{r \in \text{range}(R_1 \cdot R_2)} r \cdot \Pr[R_1 \cdot R_2 = r] \\ &= \sum_{r_i \in \text{range}(R_i)} r_1 r_2 \cdot \Pr[R_1 = r_1 \text{ and } R_2 = r_2] \\ &= \sum_{r_1 \in \text{range}(R_1)} \sum_{r_2 \in \text{range}(R_2)} r_1 r_2 \cdot \Pr[R_1 = r_1 \text{ and } R_2 = r_2] \\ &= \sum_{r_1 \in \text{range}(R_1)} \sum_{r_2 \in \text{range}(R_2)} r_1 r_2 \cdot \Pr[R_1 = r_1] \cdot \Pr[R_2 = r_2] \\ &= \sum_{r_1 \in \text{range}(R_1)} \left(r_1 \Pr[R_1 = r_1] \cdot \sum_{r_2 \in \text{range}(R_2)} r_2 \Pr[R_2 = r_2] \right) \\ &= \sum_{r_1 \in \text{range}(R_1)} r_1 \Pr[R_1 = r_1] \cdot \text{Ex}[R_2] \\ &= \text{Ex}[R_2] \cdot \sum_{r_1 \in \text{range}(R_1)} r_1 \Pr[R_1 = r_1] \\ &= \text{Ex}[R_2] \cdot \text{Ex}[R_1]. \end{aligned}$$

■

Problem 18.18.

A gambler bets \$10 on “red” at a roulette table (the odds of red are 18/38 which

slightly less than even) to win \$10. If he wins, he gets back twice the amount of his bet and he quits. Otherwise, he doubles his previous bet and continues.

- (a) What is the expected number of bets the gambler makes before he wins?
- (b) What is his probability of winning?
- (c) What is his expected final profit (amount won minus amount lost)?
- (d) The fact that the gambler’s expected profit is positive, despite the fact that the game is biased against him, is known as the *St. Petersburg paradox*. The paradox arises from an unrealistic, implicit assumption about the gambler’s money. Explain.
Hint: What is the expected size of his last bet?

Homework Problems

Problem 18.19.

A coin will be flipped repeatedly until the sequence tail/tail/head (TTH) comes up. Successive flips are independent, and the coin has probability p of coming up heads. Let N_{TTH} be the number of coin tosses until TTH first appears. What value of p minimizes $\text{Ex}[N_{\text{TTH}}]$?

Problem 18.20.

Let R and S be independent random variables, and f and g be any functions such that $\text{domain}(f) = \text{codomain}(R)$ and $\text{domain}(g) = \text{codomain}(S)$. Prove that $f(R)$ and $g(S)$ are independent random variables. *Hint:* The event $[f(R) = a]$ is the disjoint union of all the events $[R = r]$ for r such that $f(r) = a$.



19 Deviation from the Mean

19.1 Why the Mean?

In the previous chapter we took it for granted that expectation is important, and we developed a bunch of techniques for calculating expected values. But why should we care about this value? After all, a random variable may never take a value anywhere near its expected value.

The most important reason to care about the mean value comes from its connection to estimation by sampling. For example, suppose we want to estimate the average age, income, family size, or other measure of a population. To do this, we determine a random process for selecting people —say throwing darts at census lists. This process makes the selected person’s age, income, and so on into a random variable whose *mean* equals the *actual average* age or income of the population. So we can select a random sample of people and calculate the average of people in the sample to estimate the true average in the whole population. But when we make an estimate by repeated sampling, we need to know how much confidence we should have that our estimate is OK or how large a sample is needed to reach a given confidence level. The issue is also fundamental in all experimental science. Because of random errors —*noise* —repeated measurements of the same quantity rarely come out exactly the same. Determining how much confidence to put in experimental measurements is a fundamental and universal scientific issue. Technically, judging sampling or measurement accuracy reduces to finding the probability that an estimate *deviates* by a given amount from its expected value.

Another aspect of this issue comes up in engineering. When designing a sea wall, you need to know how strong to make it to withstand tsunamis for, say, at least a century. If you’re assembling a computer network, you need to know how many component failures it should tolerate to likely operate without maintenance for, say, at least a month. If your business is insurance, you need to know how large a financial reserve to maintain to be nearly certain of paying benefits for, say, the next three decades. Technically, such questions come down to finding the probability of *extreme* deviations from the mean.

This issue of *deviation from the mean* is the focus of this chapter.

19.2 Markov’s Theorem

Markov’s theorem gives a generally coarse estimate of the probability that a random variable takes a value *much larger* than its mean. It is an almost trivial result by itself, but it actually leads fairly directly to much stronger results.

The idea behind Markov’s Theorem can be explained with a simple example of *intelligence quotient*, IQ. This quantity was devised so that the average IQ measurement would be 100. Now from this fact alone we can conclude that at most 1/3 of the population can have an IQ of 300 or more, because if more than a third had an IQ of 300, then the average would have to be *more* than $(1/3) \cdot 300 = 100$, contradicting the fact that the average is 100. So the probability that a randomly chosen person has an IQ of 300 or more is at most 1/3. Of course this is not a very strong conclusion; in fact no IQ of over 300 has ever been recorded. But by the same logic, we can also conclude that at most 2/3 of the population can have an IQ of 150 or more. IQ’s of over 150 have certainly been recorded, though again, a much smaller fraction than 2/3 of the population actually has an IQ that high.

Although these conclusions about IQ are weak, they are actually the strongest general conclusions that can be reached about a random variable using *only* the fact that it is nonnegative and its mean is 100. For example, if we choose a random variable equal to 300 with probability 1/3, and 0 with probability 2/3, then its mean is 100, and the probability of a value of 300 or more really is 1/3. So we can’t hope to get a better upper bound based solely on this limited amount of information.

Theorem 19.2.1 (Markov’s Theorem). *If R is a nonnegative random variable, then for all $x > 0$*

$$\Pr[R \geq x] \leq \frac{\text{Ex}[R]}{x}. \quad (19.1)$$

Proof. Let y vary over the range of R . Then for any $x > 0$

$$\begin{aligned} \text{Ex}[R] &::= \sum_y y \Pr[R = y] \\ &\geq \sum_{y \geq x} y \Pr[R = y] \geq \sum_{y \geq x} x \Pr[R = y] = x \sum_{y \geq x} \Pr[R = y] \\ &= x \Pr[R \geq x], \end{aligned} \quad (19.2)$$

where the first inequality follows from the fact that $R \geq 0$.

Dividing the first and last expressions in (19.2) by x gives the desired result. ■

Our focus is deviation from the mean, so it's useful to rephrase Markov's Theorem this way:

Corollary 19.2.2. *If R is a nonnegative random variable, then for all $c \geq 1$*

$$\Pr[R \geq c \cdot \text{Ex}[R]] \leq \frac{1}{c}. \quad (19.3)$$

This Corollary follows immediately from Markov's Theorem(19.2.1) by letting x be $c \cdot \text{Ex}[R]$.

19.2.1 Applying Markov's Theorem

Let's go back to the Hat-Check problem of Section 18.5.2. Now we ask what the probability is that x or more men get the right hat, this is, what the value of $\Pr[G \geq x]$ is.

We can compute an upper bound with Markov's Theorem. Since we know $\text{Ex}[G] = 1$, Markov's Theorem implies

$$\Pr[G \geq x] \leq \frac{\text{Ex}[G]}{x} = \frac{1}{x}.$$

For example, there is no better than a 20% chance that 5 men get the right hat, regardless of the number of people at the dinner party.

The Chinese Appetizer problem is similar to the Hat-Check problem. In this case, n people are eating appetizers arranged on a circular, rotating Chinese banquet tray. Someone then spins the tray so that each person receives a random appetizer. What is the probability that everyone gets the same appetizer as before?

There are n equally likely orientations for the tray after it stops spinning. Everyone gets the right appetizer in just one of these n orientations. Therefore, the correct answer is $1/n$.

But what probability do we get from Markov's Theorem? Let the random variable, R , be the number of people that get the right appetizer. Then of course $\text{Ex}[R] = 1$ (right?), so applying Markov's Theorem, we find:

$$\Pr[R \geq n] \leq \frac{\text{Ex}[R]}{n} = \frac{1}{n}.$$

So for the Chinese appetizer problem, Markov's Theorem is tight!

On the other hand, Markov's Theorem gives the same $1/n$ bound in the Hat-Check problem where the probability of probability everyone gets their hat is $1/(n!)$. So for this case, Markov's Theorem gives a probability bound that is way too large.

19.2.2 Markov’s Theorem for Bounded Variables

Suppose we learn that the average IQ among MIT students is 150 (which is not true, by the way). What can we say about the probability that an MIT student has an IQ of more than 200? Markov’s theorem immediately tells us that no more than $150/200$ or $3/4$ of the students can have such a high IQ. Here we simply applied Markov’s Theorem to the random variable, R , equal to the IQ of a random MIT student to conclude:

$$\Pr[R > 200] \leq \frac{\text{Ex}[R]}{200} = \frac{150}{200} = \frac{3}{4}.$$

But let’s observe an additional fact (which may be true): no MIT student has an IQ less than 100. This means that if we let $T ::= R - 100$, then T is nonnegative and $\text{Ex}[T] = 50$, so we can apply Markov’s Theorem to T and conclude:

$$\Pr[R > 200] = \Pr[T > 100] \leq \frac{\text{Ex}[T]}{100} = \frac{50}{100} = \frac{1}{2}.$$

So only half, not $3/4$, of the students can be as amazing as they think they are. A bit of a relief!

In fact, we can get better bounds applying Markov’s Theorem to $R - b$ instead of R for any lower bound $b > 0$ on R (see Problem 19.2). Similarly, if we have any upper bound, u , on a random variable, S , then $u - S$ will be a nonnegative random variable, and applying Markov’s Theorem to $u - S$ will allow us to bound the probability that S is much *less* than its expectation.

19.3 Chebyshev’s Theorem

We’ve seen that Markov’s Theorem can give a better bound when applied to $R - b$ rather than R . More generally, a good trick for getting stronger bounds on a random variable R out of Markov’s Theorem is to apply some cleverly chosen function of R .

Choosing functions that are powers of $|R|$ turns out to be specially useful. In particular, since $|R|^\alpha$ is nonnegative, Markov’s inequality also applies to the event $[|R|^\alpha \geq x^\alpha]$. But this event is equivalent to the event $[|R| \geq x]$, so we have:

Lemma 19.3.1. *For any random variable R , $\alpha \in \mathbb{R}^+$, and $x > 0$,*

$$\Pr[|R| \geq x] \leq \frac{\text{Ex}[|R|^\alpha]}{x^\alpha}.$$

Rephrasing (19.3.1) in terms of the random variable, $|R - \text{Ex}[R]|$, that measures R 's deviation from its mean, we get

$$\Pr[|R - \text{Ex}[R]| \geq x] \leq \frac{\text{Ex}[(R - \text{Ex}[R])^\alpha]}{x^\alpha}. \quad (19.4)$$

The case when $\alpha = 2$ turns out to be so important that the numerator of the right hand side of (19.4) has been given a name:

Definition 19.3.2. The *variance*, $\text{Var}[R]$, of a random variable, R , is:

$$\text{Var}[R] ::= \text{Ex}[(R - \text{Ex}[R])^2].$$

The restatement of (19.4) for $\alpha = 2$ is known as *Chebyshev's Theorem*.

Theorem 19.3.3 (Chebyshev). *Let R be a random variable and $x \in \mathbb{R}^+$. Then*

$$\Pr[|R - \text{Ex}[R]| \geq x] \leq \frac{\text{Var}[R]}{x^2}.$$

The expression $\text{Ex}[(R - \text{Ex}[R])^2]$ for variance is a bit cryptic; the best approach is to work through it from the inside out. The innermost expression, $R - \text{Ex}[R]$, is precisely the deviation of R above its mean. Squaring this, we obtain, $(R - \text{Ex}[R])^2$. This is a random variable that is near 0 when R is close to the mean and is a large positive number when R deviates far above or below the mean. So if R is always close to the mean, then the variance will be small. If R is often far from the mean, then the variance will be large.

19.3.1 Variance in Two Gambling Games

The relevance of variance is apparent when we compare the following two gambling games.

Game A: We win \$2 with probability $2/3$ and lose \$1 with probability $1/3$.

Game B: We win \$1002 with probability $2/3$ and lose \$2001 with probability $1/3$.

Which game is better financially? We have the same probability, $2/3$, of winning each game, but that does not tell the whole story. What about the expected return for each game? Let random variables A and B be the payoffs for the two games. For example, A is 2 with probability $2/3$ and -1 with probability $1/3$. We can compute the expected payoff for each game as follows:

$$\begin{aligned} \text{Ex}[A] &= 2 \cdot \frac{2}{3} + (-1) \cdot \frac{1}{3} = 1, \\ \text{Ex}[B] &= 1002 \cdot \frac{2}{3} + (-2001) \cdot \frac{1}{3} = 1. \end{aligned}$$

The expected payoff is the same for both games, but they are obviously very different! This difference is not apparent in their expected value, but is captured by variance. We can compute the $\text{Var}[A]$ by working “from the inside out” as follows:

$$\begin{aligned} A - \text{Ex}[A] &= \begin{cases} 1 & \text{with probability } \frac{2}{3} \\ -2 & \text{with probability } \frac{1}{3} \end{cases} \\ (A - \text{Ex}[A])^2 &= \begin{cases} 1 & \text{with probability } \frac{2}{3} \\ 4 & \text{with probability } \frac{1}{3} \end{cases} \\ \text{Ex}[(A - \text{Ex}[A])^2] &= 1 \cdot \frac{2}{3} + 4 \cdot \frac{1}{3} \\ \text{Var}[A] &= 2. \end{aligned}$$

Similarly, we have for $\text{Var}[B]$:

$$\begin{aligned} B - \text{Ex}[B] &= \begin{cases} 1001 & \text{with probability } \frac{2}{3} \\ -2002 & \text{with probability } \frac{1}{3} \end{cases} \\ (B - \text{Ex}[B])^2 &= \begin{cases} 1,002,001 & \text{with probability } \frac{2}{3} \\ 4,008,004 & \text{with probability } \frac{1}{3} \end{cases} \\ \text{Ex}[(B - \text{Ex}[B])^2] &= 1,002,001 \cdot \frac{2}{3} + 4,008,004 \cdot \frac{1}{3} \\ \text{Var}[B] &= 2,004,002. \end{aligned}$$

The variance of Game A is 2 and the variance of Game B is more than two million! Intuitively, this means that the payoff in Game A is usually close to the expected value of \$1, but the payoff in Game B can deviate very far from this expected value.

High variance is often associated with high risk. For example, in ten rounds of Game A, we expect to make \$10, but could conceivably lose \$10 instead. On the other hand, in ten rounds of game B, we also expect to make \$10, but could actually lose more than \$20,000!

19.3.2 Standard Deviation

Because of its definition in terms of the square of a random variable, the variance of a random variable may be very far from a typical deviation from the mean. For example, in Game B above, the deviation from the mean is 1001 in one outcome and -2002 in the other. But the variance is a whopping 2,004,002. From a dimensional analysis viewpoint, the “units” of variance are wrong: if the random variable is in dollars, then the expectation is also in dollars, but the variance is in square dollars. For this reason, people often describe random variables using standard deviation instead of variance.

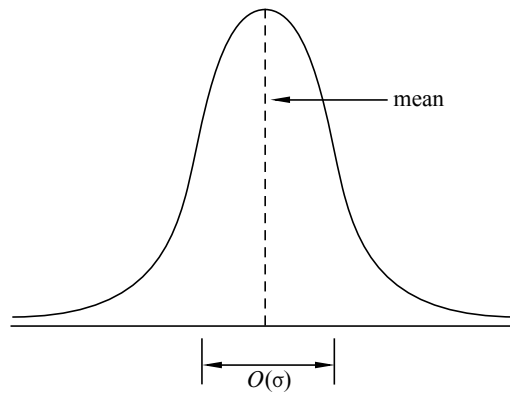


Figure 19.1 The standard deviation of a distribution indicates how wide the “main part” of it is.

Definition 19.3.4. The *standard deviation*, σ_R , of a random variable, R , is the square root of the variance:

$$\sigma_R ::= \sqrt{\text{Var}[R]} = \sqrt{\text{Ex}[(R - \text{Ex}[R])^2]}.$$

So the standard deviation is the square root of the mean of the square of the deviation, or the *root mean square* for short. It has the same units —dollars in our example—as the original random variable and as the mean. Intuitively, it measures the average deviation from the mean, since we can think of the square root on the outside as canceling the square on the inside.

Example 19.3.5. The standard deviation of the payoff in Game B is:

$$\sigma_B = \sqrt{\text{Var}[B]} = \sqrt{2,004,002} \approx 1416.$$

The random variable B actually deviates from the mean by either positive 1001 or negative 2002; therefore, the standard deviation of 1416 describes this situation reasonably well.

Intuitively, the standard deviation measures the “width” of the “main part” of the distribution graph, as illustrated in Figure 19.1.

It’s useful to rephrase Chebyshev’s Theorem in terms of standard deviation which we can do by substituting $x = c\sigma_R$ in (19.1):

Corollary 19.3.6. Let R be a random variable, and let c be a positive real number.

$$\Pr[|R - \text{Ex}[R]| \geq c\sigma_R] \leq \frac{1}{c^2}. \quad (19.5)$$

Here we see explicitly how the “likely” values of R are clustered in an $O(\sigma_R)$ -sized region around $\text{Ex}[R]$, confirming that the standard deviation measures how spread out the distribution of R is around its mean.

The IQ Example

Suppose that, in addition to the national average IQ being 100, we also know the standard deviation of IQ’s is 10. How rare is an IQ of 300 or more?

Let the random variable, R , be the IQ of a random person. So we are supposing that $\text{Ex}[R] = 100$, $\sigma_R = 10$, and R is nonnegative. We want to compute $\Pr[R \geq 300]$.

We have already seen that Markov’s Theorem 19.2.1 gives a coarse bound, namely,

$$\Pr[R \geq 300] \leq \frac{1}{3}.$$

Now we apply Chebyshev’s Theorem to the same problem:

$$\Pr[R \geq 300] = \Pr[|R - 100| \geq 200] \leq \frac{\text{Var}[R]}{200^2} = \frac{10^2}{200^2} = \frac{1}{400}.$$

So Chebyshev’s Theorem implies that at most one person in four hundred has an IQ of 300 or more. We have gotten a much tighter bound using the additional information, namely the variance of R , than we could get knowing only the expectation.

19.4 Properties of Variance

The definition of variance of R as $\text{Ex}[(R - \text{Ex}[R])^2]$ may seem rather arbitrary. A direct measure of average deviation would be $\text{Ex}[|R - \text{Ex}[R]|]$. But the direct measure doesn’t have the many useful properties that variance has, which is what this section is about.

19.4.1 A Formula for Variance

Applying linearity of expectation to the formula for variance yields a convenient alternative formula.

Lemma 19.4.1.

$$\text{Var}[R] = \text{Ex}[R^2] - \text{Ex}^2[R],$$

for any random variable, R .

Here we use the notation $\text{Ex}^2[R]$ as shorthand for $(\text{Ex}[R])^2$.

Proof. Let $\mu = \text{Ex}[R]$. Then

$$\begin{aligned}
 \text{Var}[R] &= \text{Ex}[(R - \text{Ex}[R])^2] && \text{(Def 19.3.2 of variance)} \\
 &= \text{Ex}[(R - \mu)^2] && \text{(def of } \mu) \\
 &= \text{Ex}[R^2 - 2\mu R + \mu^2] \\
 &= \text{Ex}[R^2] - 2\mu \text{Ex}[R] + \mu^2 && \text{(linearity of expectation)} \\
 &= \text{Ex}[R^2] - 2\mu^2 + \mu^2 && \text{(def of } \mu) \\
 &= \text{Ex}[R^2] - \mu^2 \\
 &= \text{Ex}[R^2] - \text{Ex}^2[R]. && \text{(def of } \mu)
 \end{aligned}$$

■

For example, if B is a Bernoulli variable where $p ::= \Pr[B = 1]$, then

Lemma 19.4.2.

$$\text{Var}[B] = p - p^2 = p(1 - p). \quad (19.6)$$

Proof. By Lemma 18.4.2, $\text{Ex}[B] = p$. But since B only takes values 0 and 1, $B^2 = B$. So Lemma 19.4.2 follows immediately from Lemma 19.4.1. ■

19.4.2 Variance of Time to Failure

According to section 18.4.6, the mean time to failure is $1/p$ for a process that fails during any given hour with probability p . What about the variance?

By Lemma 19.4.1,

$$\text{Var}[C] = \text{Ex}[C^2] - (1/p)^2 \quad (19.7)$$

so all we need is a formula for $\text{Ex}[C^2]$.

Reasoning about C using conditional expectation worked nicely in section 18.4.6 to find mean time to failure, and a similar approach works C^2 . Namely, the expected value of C^2 is the probability, p , of failure in the first hour times 1^2 , plus the probability, $(1 - p)$, of non-failure in the first hour times the expected value of

$(C + 1)^2$. So

$$\begin{aligned}
 \text{Ex}[C^2] &= p \cdot 1^2 + (1 - p) \text{Ex}[(C + 1)^2] \\
 &= p + (1 - p) \left(\text{Ex}[C^2] + \frac{2}{p} + 1 \right) \\
 &= p + (1 - p) \text{Ex}[C^2] + (1 - p) \left(\frac{2}{p} + 1 \right), \quad \text{so} \\
 p \text{Ex}[C^2] &= p + (1 - p) \left(\frac{2}{p} + 1 \right) \\
 &= \frac{p^2 + (1 - p)(2 + p)}{p} \quad \text{and} \\
 \text{Ex}[C^2] &= \frac{2 - p}{p^2}
 \end{aligned}$$

Combining this with (19.7) proves

Lemma 19.4.3. *If failures occur with probability p independently at each step, and C is the number of steps until the first failure¹, then*

$$\text{Var}[C] = \frac{1 - p}{p^2}. \quad (19.8)$$

19.4.3 Dealing with Constants

It helps to know how to calculate the variance of $aR + b$:

Theorem 19.4.4. *Let R be a random variable, and a a constant. Then*

$$\text{Var}[aR] = a^2 \text{Var}[R]. \quad (19.9)$$

Proof. Beginning with the definition of variance and repeatedly applying linearity of expectation, we have:

$$\begin{aligned}
 \text{Var}[aR] &::= \text{Ex}[(aR - \text{Ex}[aR])^2] \\
 &= \text{Ex}[(aR)^2 - 2aR \text{Ex}[aR] + \text{Ex}^2[aR]] \\
 &= \text{Ex}[(aR)^2] - \text{Ex}[2aR \text{Ex}[aR]] + \text{Ex}^2[aR] \\
 &= a^2 \text{Ex}[R^2] - 2 \text{Ex}[aR] \text{Ex}[aR] + \text{Ex}^2[aR] \\
 &= a^2 \text{Ex}[R^2] - a^2 \text{Ex}^2[R] \\
 &= a^2 (\text{Ex}[R^2] - \text{Ex}^2[R]) \\
 &= a^2 \text{Var}[R] \quad \quad \quad \text{(by Lemma 19.4.1)}
 \end{aligned}$$

¹That is, C has the geometric distribution with parameter p according to Definition 18.4.7.



It's even simpler to prove that adding a constant does not change the variance, as the reader can verify:

Theorem 19.4.5. *Let R be a random variable, and b a constant. Then*

$$\text{Var}[R + b] = \text{Var}[R]. \quad (19.10)$$

Recalling that the standard deviation is the square root of variance, this implies that the standard deviation of $aR + b$ is simply $|a|$ times the standard deviation of R :

Corollary 19.4.6.

$$\sigma_{(aR+b)} = |a| \sigma_R.$$

19.4.4 Variance of a Sum

In general, the variance of a sum is not equal to the sum of the variances, but variances do add for *independent* variables. In fact, *mutual* independence is not necessary: *pairwise* independence will do. This is useful to know because there are some important situations involving variables that are pairwise independent but not mutually independent.

Theorem 19.4.7. *If R_1 and R_2 are independent random variables, then*

$$\text{Var}[R_1 + R_2] = \text{Var}[R_1] + \text{Var}[R_2]. \quad (19.11)$$

Proof. We may assume that $\text{Ex}[R_i] = 0$ for $i = 1, 2$, since we could always replace R_i by $R_i - \text{Ex}[R_i]$ in equation (19.11). This substitution preserves the independence of the variables, and by Theorem 19.4.5, does not change the variances.

Now by Lemma 19.4.1, $\text{Var}[R_i] = \text{Ex}[R_i^2]$ and $\text{Var}[R_1 + R_2] = \text{Ex}[(R_1 + R_2)^2]$, so we need only prove

$$\text{Ex}[(R_1 + R_2)^2] = \text{Ex}[R_1^2] + \text{Ex}[R_2^2]. \quad (19.12)$$

But (19.12) follows from linearity of expectation and the fact that

$$\text{Ex}[R_1 R_2] = \text{Ex}[R_1] \text{Ex}[R_2] \quad (19.13)$$

since R_1 and R_2 are independent:

$$\begin{aligned} \text{Ex}[(R_1 + R_2)^2] &= \text{Ex}[R_1^2 + 2R_1 R_2 + R_2^2] \\ &= \text{Ex}[R_1^2] + 2\text{Ex}[R_1 R_2] + \text{Ex}[R_2^2] \\ &= \text{Ex}[R_1^2] + 2\text{Ex}[R_1] \text{Ex}[R_2] + \text{Ex}[R_2^2] \quad (\text{by (19.13)}) \\ &= \text{Ex}[R_1^2] + 2 \cdot 0 \cdot 0 + \text{Ex}[R_2^2] \\ &= \text{Ex}[R_1^2] + \text{Ex}[R_2^2] \end{aligned}$$



An independence condition is necessary. If we ignored independence, then we would conclude that $\text{Var}[R + R] = \text{Var}[R] + \text{Var}[R]$. However, by Theorem 19.4.4, the left side is equal to $4 \text{Var}[R]$, whereas the right side is $2 \text{Var}[R]$. This implies that $\text{Var}[R] = 0$, which, by the Lemma above, essentially only holds if R is constant.

The proof of Theorem 19.4.7 carries over straightforwardly to the sum of any finite number of variables. So we have:

Theorem 19.4.8. *[Pairwise Independent Additivity of Variance] If $R_1, R_2, \dots, R_n$ are pairwise independent random variables, then*

$$\text{Var}[R_1 + R_2 + \dots + R_n] = \text{Var}[R_1] + \text{Var}[R_2] + \dots + \text{Var}[R_n]. \quad (19.14)$$

Now we have a simple way of computing the variance of a variable, J , that has an (n, p) -binomial distribution. We know that $J = \sum_{k=1}^n I_k$ where the I_k are mutually independent indicator variables with $\Pr[I_k = 1] = p$. The variance of each I_k is $p(1 - p)$ by Lemma 19.4.2, so by linearity of variance, we have

Lemma (Variance of the Binomial Distribution). *If J has the (n, p) -binomial distribution, then*

$$\text{Var}[J] = n \text{Var}[I_k] = np(1 - p). \quad (19.15)$$

19.5 Estimation by Random Sampling

Democratic politicians were astonished in 2010 when their early polls of sample voters showed Republican Scott Brown was favored by a majority of voters and so would win the special election to fill the Senate seat Democrat Teddy Kennedy had occupied for over 40 years. Based on their poll results, they mounted an intense, but ultimately unsuccessful, effort to save the seat for their party.

19.5.1 A Voter Poll

How did polling give an advance estimate of the fraction of the Massachusetts voters who favored Scott Brown over his Democratic opponent?

Suppose at some time before the election that p was the fraction of voters favoring Scott Brown. We want to estimate this unknown fraction p . Suppose we have some random process—say throwing darts at voter registration lists—which will select each voter with equal probability. We can define a Bernoulli variable, K , by the rule that $K = 1$ if the random voter most prefers Brown, and $K = 0$ otherwise.

Now to estimate p , we take a large number, n , of random choices of voters² and count the fraction who favor Brown. That is, we define variables $K_1, K_2, \dots$, where K_i is interpreted to be the indicator variable for the event that the i th chosen voter prefers Brown. Since our choices are made independently, the K_i 's are independent. So formally, we model our estimation process by simply assuming we have mutually independent Bernoulli variables $K_1, K_2, \dots$, each with the same probability, p , of being equal to 1. Now let S_n be their sum, that is,

$$S_n ::= \sum_{i=1}^n K_i. \quad (19.16)$$

The variable S_n/n describes the fraction of sampled voters who favor Scott Brown. Most people intuitively expect this sample fraction to give a useful approximation to the unknown fraction, p —and they would be right. So we will use the sample value, S_n/n , as our *statistical estimate* of p . We know that S_n has the binomial distribution with parameters n and p , where we can choose n , but p is unknown.

How Large a Sample?

Suppose we want our estimate to be within 0.04 of the fraction, p , at least 95% of the time. This means we want

$$\Pr \left[\left| \frac{S_n}{n} - p \right| \leq 0.04 \right] \geq 0.95. \quad (19.17)$$

So we better determine the number, n , of times we must poll voters so that inequality (19.17) will hold. Chebyshev's Theorem offers a simple way to determine such a n .

Since S_n is binomially distributed, equation (19.15) gives

$$\text{Var}[S_n] = n(p(1-p)) \leq n \cdot \frac{1}{4} = \frac{n}{4}.$$

The bound of $1/4$ follows from the fact that $p(1-p)$ is maximized when $p = 1-p$, that is, when $p = 1/2$ (check this yourself!).

²We're choosing a random voter n times *with replacement*. That is, we don't remove a chosen voter from the set of voters eligible to be chosen later; so we might choose the same voter more than once in n tries! We would get a slightly better estimate if we required n *different* people to be chosen, but doing so complicates both the selection process and its analysis, with little gain in accuracy.

Next, we bound the variance of S_n/n :

$$\begin{aligned}\text{Var}\left[\frac{S_n}{n}\right] &= \left(\frac{1}{n}\right)^2 \text{Var}[S_n] && \text{(by (19.9))} \\ &\leq \left(\frac{1}{n}\right)^2 \frac{n}{4} && \text{(by (19.5.1))} \\ &= \frac{1}{4n} && (19.18)\end{aligned}$$

Using Chebyshev’s bound and (19.18) we have:

$$\Pr\left[\left|\frac{S_n}{n} - p\right| \geq 0.04\right] \leq \frac{\text{Var}[S_n/n]}{(0.04)^2} = \frac{1}{4n(0.04)^2} = \frac{156.25}{n} \quad (19.19)$$

To make our estimate with 95% confidence, we want the righthand side of (19.19) to be at most 1/20. So we choose n so that

$$\frac{156.25}{n} \leq \frac{1}{20},$$

that is,

$$n \geq 3,125.$$

Section 19.7.2 describes how to get tighter estimates of the tails of binomial distributions that lead to a bound on n that is about four times smaller than the one above. But working through this example using only the variance has the virtue of illustrating an approach to estimation that is applicable to arbitrary random variables, not just binomial variables, and it did lead to a feasible, though larger than necessary, sample size.

19.5.2 Matching Birthdays

There are important cases where the relevant distributions are not binomial because the mutual independence properties of the voter preference example do not hold. In these cases, estimation methods based on the Chebyshev bound may be the best approach. Birthday Matching is an example. We already saw in Section 17.6.6 that in a class of 85 students it is virtually certain that two or more students will have the same birthday. This suggests that quite a few pairs of students are likely to have the same birthday. How many?

So as before, suppose there are n students and d days in the year, and let D be the number of pairs of students with the same birthday. Now it will be easy to calculate the expected number of pairs of students with matching birthdays. Then we can

take the same approach as we did in estimating voter preferences to get an estimate of the probability of getting a number of pairs close to the expected number.

Unlike the situation with voter preferences, having matching birthdays for different pairs of students are not mutually independent events, but the matchings are *pairwise independent* —as explained in Section 17.6.6 (and proved in Problem 18.2). This will allow us to apply the same reasoning to Birthday Matching as we did for voter preference. Namely, let $B_1, B_2, \dots, B_n$ be the birthdays of n independently chosen people, and let $E_{i,j}$ be the indicator variable for the event that the i th and j th people chosen have the same birthdays, that is, the event $[B_i = B_j]$. So our probability model, the B_i 's are mutually independent variables, the $E_{i,j}$'s are pairwise independent. Also, the expectations of $E_{i,j}$ for $i \neq j$ equals the probability that $B_i = B_j$, namely, $1/d$.

Now, D , the number of matching pairs of birthdays among the n choices, is simply the sum of the $E_{i,j}$'s:

$$D ::= \sum_{1 \leq i < j \leq n} E_{i,j}. \quad (19.20)$$

So by linearity of expectation

$$\text{Ex}[D] = \text{Ex} \left[\sum_{1 \leq i < j \leq n} E_{i,j} \right] = \sum_{1 \leq i < j \leq n} \text{Ex}[E_{i,j}] = \binom{n}{2} \cdot \frac{1}{d}.$$

Similarly,

$$\begin{aligned} \text{Var}[D] &= \text{Var} \left[\sum_{1 \leq i < j \leq n} E_{i,j} \right] \\ &= \sum_{1 \leq i < j \leq n} \text{Var}[E_{i,j}] && \text{(by Theorem 19.4.8)} \\ &= \binom{n}{2} \cdot \frac{1}{d} \left(1 - \frac{1}{d} \right). && \text{(by Lemma 19.4.2)} \end{aligned}$$

In particular, for a class of $n = 95$ students with $d = 365$ possible birthdays, we have $\text{Ex}[D] \approx 12.23$ and $\text{Var}[D] \approx 12.23(1 - 1/365) < 12.2$. So by Chebyshev's Theorem

$$\Pr[|D - \text{Ex}[D]| \geq x] < \frac{12.2}{x^2}.$$

Letting $x = 7$, we conclude that there is a better than 75% chance that in a class of 95 students, the number of pairs of students with the same birthday will be within 7 of 12.23, namely will be between 6 and 20.

19.5.3 Pairwise Independent Sampling

The reasoning we used above to analyze voter polling and matching birthdays is very similar. We summarize it in slightly more general form with a basic result we call the Pairwise Independent Sampling Theorem. In particular, we do not need to restrict ourselves to sums of zero-one valued variables, or to variables with the same distribution. For simplicity, we state the Theorem for pairwise independent variables with possibly different distributions but with the same mean and variance.

Theorem 19.5.1 (Pairwise Independent Sampling). *Let $G_1, \dots, G_n$ be pairwise independent variables with the same mean, μ , and deviation, σ . Define*

$$S_n ::= \sum_{i=1}^n G_i. \quad (19.21)$$

Then

$$\Pr \left[\left| \frac{S_n}{n} - \mu \right| \geq x \right] \leq \frac{1}{n} \left(\frac{\sigma}{x} \right)^2.$$

Proof. We observe first that the expectation of S_n/n is μ :

$$\begin{aligned} \text{Ex} \left[\frac{S_n}{n} \right] &= \text{Ex} \left[\frac{\sum_{i=1}^n G_i}{n} \right] && \text{(def of } S_n) \\ &= \frac{\sum_{i=1}^n \text{Ex}[G_i]}{n} && \text{(linearity of expectation)} \\ &= \frac{\sum_{i=1}^n \mu}{n} \\ &= \frac{n\mu}{n} = \mu. \end{aligned}$$

The second important property of S_n/n is that its variance is the variance of G_i divided by n :

$$\begin{aligned} \text{Var} \left[\frac{S_n}{n} \right] &= \left(\frac{1}{n} \right)^2 \text{Var}[S_n] && \text{(by (19.9))} \\ &= \frac{1}{n^2} \text{Var} \left[\sum_{i=1}^n G_i \right] && \text{(def of } S_n) \\ &= \frac{1}{n^2} \sum_{i=1}^n \text{Var}[G_i] && \text{(pairwise independent additivity)} \\ &= \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}. && (19.22) \end{aligned}$$

This is enough to apply Chebyshev’s Theorem and conclude:

$$\begin{aligned} \Pr \left[\left| \frac{S_n}{n} - \mu \right| \geq x \right] &\leq \frac{\text{Var}[S_n/n]}{x^2}. && \text{(Chebyshev’s bound)} \\ &= \frac{\sigma^2/n}{x^2} && \text{(by (19.22))} \\ &= \frac{1}{n} \left(\frac{\sigma}{x} \right)^2. \end{aligned}$$

■

The Pairwise Independent Sampling Theorem provides a precise general statement about how the average of independent samples of a random variable approaches the mean. In particular, it proves what is known as the Law of Large Numbers³: by choosing a large enough sample size, we can get arbitrarily accurate estimates of the mean with confidence arbitrarily close to 100%.

Corollary 19.5.2. *[Weak Law of Large Numbers] Let $G_1, \dots, G_n$ be pairwise independent variables with the same mean, μ , and the same finite deviation, and let*

$$S_n ::= \frac{\sum_{i=1}^n G_i}{n}.$$

Then for every $\epsilon > 0$,

$$\lim_{n \rightarrow \infty} \Pr[|S_n - \mu| \leq \epsilon] = 1.$$

19.6 Confidence versus Probability

So Chebyshev’s Bound implies that sampling 3,125 voters will yield a fraction that, 95% of the time, is within 0.04 of the actual fraction of the voting population who prefer Brown.

Notice that the actual size of the voting population was never considered because *it did not matter*. People who have not studied probability theory often insist that the population size should matter. But our analysis shows that polling a little over 3000 people is always sufficient, whether there are ten thousand, or a million, or a billion ... voters. You should think about an intuitive explanation that might persuade someone who thinks population size matters.

³This is the *Weak Law of Large Numbers*. As you might suppose, there is also a *Strong Law*, but it’s outside the scope of 6.042.

Now suppose a pollster actually takes a sample of 3,125 random voters to estimate the fraction of voters who prefer Brown, and the pollster finds that 1250 of them prefer Brown. It’s tempting, **but sloppy**, to say that this means:

False Claim. *With probability 0.95, the fraction, p , of voters who prefer Brown is $1250/3125 \pm 0.04$. Since $1250/3125 - 0.04 > 1/3$, there is a 95% chance that more than a third of the voters prefer Brown to all other candidates.*

What’s objectionable about this statement is that it talks about the probability or “chance” that a real world fact is true, namely that the actual fraction, p , of voters favoring Brown is more than $1/3$. But p is what it is, and it simply makes no sense to talk about the probability that it is something else. For example, suppose p is actually 0.3; then it’s nonsense to ask about the probability that it is within 0.04 of $1250/3125$ —it simply isn’t.

This example of voter preference is typical: we want to estimate a fixed, unknown real-world quantity. But *being unknown does not make this quantity a random variable*, so it makes no sense to talk about the probability that it has some property.

A more careful summary of what we have accomplished goes this way:

We have described a probabilistic procedure for estimating the value of the actual fraction, p . The probability that *our estimation procedure* will yield a value within 0.04 of p is 0.95.

This is a bit of a mouthful, so special phrasing closer to the sloppy language is commonly used. The pollster would describe his conclusion by saying that

At the 95% *confidence level*, the fraction of voters who prefer Brown is $1250/3125 \pm 0.04$.

So confidence levels refer to the results of estimation procedures for real-world quantities. The phrase “confidence level” should be heard as a reminder that some statistical procedure was used to obtain an estimate, and in judging the credibility of the estimate, it may be important to learn just what this procedure was.

19.7 Sums of Random Variables

If all you know about a random variable is its mean and variance, then Chebyshev’s Theorem is the best you can do when it comes to bounding the probability that the random variable deviates from its mean. In some cases, however, we know

more—for example, that the random variable has a binomial distribution—and then it is possible to prove much stronger bounds. Instead of polynomially small bounds such as $1/c^2$, we can sometimes even obtain exponentially small bounds such as $1/e^c$. As we will soon discover, this is the case whenever the random variable T is the sum of n mutually independent random variables $T_1, T_2, \dots, T_n$ where $0 \leq T_i \leq 1$. A random variable with a binomial distribution is just one of many examples of such a T . Here is another.

19.7.1 A Motivating Example

Fussbook is a new social networking site oriented toward unpleasant people.

Like all major web services, Fussbook has a load balancing problem. Specifically, Fussbook receives 24,000 forum posts every 10 minutes. Each post is assigned to one of m computers for processing, and each computer works sequentially through its assigned tasks. Processing an average post takes a computer $1/4$ second. Some posts, such as pointless grammar critiques and snide witticisms, are easier. But the most protracted harangues require 1 full second.

Balancing the work load across the m computers is vital; if any computer is assigned more than 10 minutes of work in a 10-minute interval, then that computer is overloaded and system performance suffers. That would be bad, because Fussbook users are *not* a tolerant bunch.

An early idea was to assign each computer an alphabetic range of forum topics. (“That oughta work!”, one programmer said.) But after the computer handling the “*privacy*” and “*preferred text editor*” threads melted, the drawback of an ad hoc approach was clear: there are no guarantees.

If the length of every task were known in advance, then finding a balanced distribution would be a kind of “bin packing” problem. Such problems are hard to solve exactly, though approximation algorithms can come close. But in this case, task lengths are not known in advance, which is typical for workload problems in the real world.

So the load balancing problem seems sort of hopeless, because there is no data available to guide decisions. Heck, we might as well assign tasks to computers at random!

As it turns out, random assignment not only balances load reasonably well, but also permits provable performance guarantees in place of “That oughta work!” assertions. In general, a randomized approach to a problem is worth considering when a deterministic solution is hard to compute or requires unavailable information.

Some arithmetic shows that Fussbook’s traffic is sufficient to keep $m = 10$ computers running at 100% capacity with perfect load balancing. Surely, more than 10 servers are needed to cope with random fluctuations in task length and imperfect

load balance. But how many is enough? 11? 15? 20? 100? We’ll answer that question with a new mathematical tool.

19.7.2 The Chernoff Bound

The Chernoff⁴ bound is a hammer that you can use to nail a great many problems. Roughly, the Chernoff bound says that certain random variables are very unlikely to significantly exceed their expectation. For example, if the expected load on a computer is just a bit below its capacity, then that computer is unlikely to be overloaded, provided the conditions of the Chernoff bound are satisfied.

More precisely, the Chernoff Bound says that *the sum of lots of little, independent random variables is unlikely to significantly exceed the mean of the sum*. The Markov and Chebyshev bounds lead to the same kind of conclusion but typically provide much weaker bounds. In particular, the Markov and Chebyshev bounds are polynomial, while the Chernoff bound is exponential.

Here is the theorem. The proof will come later in Section 19.7.5.

Theorem 19.7.1 (Chernoff Bound). *Let $T_1, \dots, T_n$ be mutually independent random variables such that $0 \leq T_i \leq 1$ for all i . Let $T = T_1 + \dots + T_n$. Then for all $c \geq 1$,*

$$\Pr[T \geq c \operatorname{Ex}[T]] \leq e^{-\beta(c) \operatorname{Ex}[T]} \quad (19.23)$$

where $\beta(c) ::= c \ln c - c + 1$.

The Chernoff bound applies only to distributions of sums of independent random variables that take on values in the interval $[0, 1]$. The binomial distribution is of course such a distribution, but there are lots of other distributions because the Chernoff bound allows the variables in the sum to have differing, arbitrary, and even unknown distributions over the range $[0, 1]$. Furthermore, there is no direct dependence on the number of random variables in the sum or their expectations. In short, the Chernoff bound gives strong results for lots of problems based on little information —no wonder it is widely used!

19.7.3 Chernoff Bound for Binomial Tails

The Chernoff bound is pretty easy to apply, though the details can be daunting at first. Let’s walk through a simple example to get the hang of it: getting bounds on the tail of a binomial distribution, for example, bounding the probability that the number of heads that come up in 1000 independent tosses of a coin exceeds the

⁴Yes, this is the same Chernoff who figured out how to beat the state lottery —this guy knows a thing or two.

expectation by 20% or more? Let T_i be an indicator variable for the event that the i th coin is heads. Then the total number of heads is

$$T = T_1 + \cdots + T_{1000}.$$

The Chernoff bound requires that the random variables T_i be mutually independent and take on values in the range $[0, 1]$. Both conditions hold here. In this example the T_i 's only take the two values 0 and 1, since they're indicators.

The goal is to bound the probability that the number of heads exceeds its expectation by 20% or more; that is, to bound $\Pr[T \geq c \operatorname{Ex}[T]]$ where $c = 1.2$. To that end, we compute $\beta(c)$ as defined in the theorem:

$$\beta(c) = c \ln(c) - c + 1 = 0.0187 \dots$$

If we assume the coin is fair, then $\operatorname{Ex}[T] = 500$. Plugging these values into the Chernoff bound gives:

$$\begin{aligned} \Pr[T \geq 1.2 \operatorname{Ex}[T]] &\leq e^{-\beta(c) \cdot \operatorname{Ex}[T]} \\ &= e^{-(0.0187 \dots) \cdot 500} < 0.0000834. \end{aligned}$$

So the probability of getting 20% or more extra heads on 1000 coins is less than 1 in 10,000.

The bound becomes much stronger as the number of coins increases, because the expected number of heads appears in the exponent of the upper bound. For example, the probability of getting at least 20% extra heads on a million coins is at most

$$e^{-(0.0187 \dots) \cdot 500000} < e^{-9392},$$

which is an inconceivably small number.

Alternatively, the bound also becomes stronger for larger deviations. For example, suppose we're interested in the odds of getting 30% or more extra heads in 1000 tosses, rather than 20%. In that case, $c = 1.3$ instead of 1.2. Consequently, the parameter $\beta(c)$ rises from 0.0187 to about 0.0410, which may not seem significant, but because $\beta(c)$ appears in the exponent of the upper bound, the final probability decreases from around 1 in 10,000 to about 1 in a billion!

19.7.4 Chernoff Bound for a Lottery Game

Pick-4 is a lottery game where you pay \$1 to pick a 4-digit number between 0000 and 9999. If your number comes up in a random drawing, then you win \$5,000. Your chance of winning is 1 in 10,000. If 10 million people play, then the expected number of winners is 1000. When there are exactly 1000 winners, the lottery keeps

\$5 million of the \$10 million paid for tickets. The lottery operator’s nightmare is that the number of winners is much greater —say at the 2000 or greater point where the lottery has to pay out more than it received. What is the probability that will happen?

Let T_i be an indicator for the event that the i th player wins. Then $T = T_1 + \dots + T_n$ is the total number of winners. If we assume⁵ that the players’ picks and the winning number are random, independent and uniform, then the indicators T_i are independent, as required by the Chernoff bound.

Since 2000 winners would be twice the expected number, we choose $c = 2$, compute $\beta(c) = 0.386\dots$, and plug these values into the Chernoff bound:

$$\begin{aligned}\Pr[T \geq 2000] &= \Pr[T \geq 2 \operatorname{Ex}[T]] \\ &\leq e^{-k \operatorname{Ex}[T]} = e^{-(0.386\dots) \cdot 1000} \\ &< e^{-386}.\end{aligned}$$

So there is almost no chance that the lottery operator pays out double. In fact, the number of winners won’t even be 10% higher than expected very often. To prove that, let $c = 1.1$, compute $\beta(c) = 0.00484\dots$, and plug in again:

$$\begin{aligned}\Pr[T \geq 1.1 \operatorname{Ex}[T]] &\leq e^{-k \operatorname{Ex}[T]} \\ &= e^{-(0.00484) \cdot 1000} < 0.01.\end{aligned}$$

So the Pick-4 lottery may be exciting for the players, but the lottery operator has little doubt about the outcome!

Randomized Load Balancing

Now let’s return to Fussbook and its load balancing problem. Specifically, we need to determine how many machines suffice to ensure that no server is overloaded; that is, assigned to do more than 10 minutes of work in a 10-minute interval. So a server is overloaded if it gets assigned more than 600 seconds of work.

To begin, let’s find the probability that the first server is overloaded. Letting T be the number of seconds of work assigned to the first server, this means we want an upper bound on $\Pr[T \geq 600]$. Let T_i be the number of seconds that the first server spends on the i th task: then T_i is zero if the task is assigned to another machine, and otherwise T_i is the length of the task. So $T = \sum_{i=1}^n T_i$ is the total length of tasks assigned to the first server, where $n = 24,000$.

⁵As we noted in Chapter 18, human choices are often not uniform and they can be highly dependent. For example, lots of people will pick an important date. So the lottery folks should not get too much comfort from the analysis that follows, unless they assign random 4-digit numbers to each player.

The Chernoff bound is applicable only if the T_i are mutually independent and take on values in the range $[0, 1]$. The first condition is satisfied if we assume that task lengths and assignments are independent. And the second condition is satisfied because processing even the most interminable harangue takes at most 1 second.

In all, there are 24,000 tasks, each with an expected length of 1/4 second. Since tasks are assigned to computers at random, the expected load on the first server is:

$$\begin{aligned} \text{Ex}[T] &= \frac{24,000 \text{ tasks} \cdot 1/4 \text{ second per task}}{m \text{ machines}} \\ &= 6000/m \text{ seconds.} \end{aligned} \tag{19.24}$$

For example, if there are fewer than 10 machines, then the expected load on the first server is greater than its capacity, and we can expect it to be overloaded. If there are exactly 10 machines, then the server is expected to run for $6000/10 = 600$ seconds, which is 100% of its capacity.

Now we can use the Chernoff bound to upper bound the probability that the first server is overloaded. We have from (19.24)

$$600 = c \text{ Ex}[T] \quad \text{where } c ::= m/10,$$

so by the Chernoff bound

$$\Pr[T \geq 600] = \Pr[T \geq c \text{ Ex}[T]] \leq e^{-(c \ln(c) - c + 1) \cdot 6000/m},$$

The probability that *some* server is overloaded is at most m times the probability that the first server is overloaded, by the Union Bound in Section 17.4.2. So

$$\begin{aligned} \Pr[\text{some server is overloaded}] &\leq \sum_{i=1}^m \Pr[\text{server } i \text{ is overloaded}] \\ &= m \Pr[\text{the first server is overloaded}] \\ &\leq m e^{-(c \ln(c) - c + 1) \cdot 6000/m}, \end{aligned}$$

where $c = m/10$. Some values of this upper bound are tabulated below:

$$\begin{aligned} m &= 11 : 0.784 \dots \\ m &= 12 : 0.000999 \dots \\ m &= 13 : 0.0000000760 \dots \end{aligned}$$

These values suggest that a system with $m = 11$ machines might suffer immediate overload, $m = 12$ machines could fail in a few days, but $m = 13$ should be fine for a century or two!

19.7.5 Proof of the Chernoff Bound

The proof of the Chernoff bound is somewhat involved. Heck, even *Chernoff* didn’t come up with it! His friend, Herman Rubin, showed him the argument. Thinking the bound not very significant, Chernoff did not credit Rubin in print. He felt pretty bad when it became famous!⁶

Proof. (of Theorem 19.7.1)

For clarity, we’ll go through the proof “top down.” That is, we’ll use facts that are proved immediately afterward.

The key step is to exponentiate both sides of the inequality $T \geq c \operatorname{Ex}[T]$ and then apply the Markov bound:

$$\begin{aligned} \Pr[T \geq c \operatorname{Ex}[T]] &= \Pr[c^T \geq c^{c \operatorname{Ex}[T]}] \\ &\leq \frac{\operatorname{Ex}[c^T]}{c^{c \operatorname{Ex}[T]}} && \text{(by Markov)} \\ &\leq \frac{e^{(c-1) \operatorname{Ex}[T]}}{c^{c \operatorname{Ex}[T]}} && \text{(by Lemma 19.7.2 below)} \\ &= \frac{e^{(c-1) \operatorname{Ex}[T]}}{e^{c \ln(c) \operatorname{Ex}[T]}} = e^{-(c \ln(c) - c + 1) \operatorname{Ex}[T]}. \end{aligned}$$

■

Algebra aside, there is a brilliant idea in this proof: in this context, exponentiating somehow supercharges the Markov bound. This is not true in general! One unfortunate side-effect is that we have to bound some nasty expectations involving exponentials in order to complete the proof. This is done in the two lemmas below, where variables take on values as in Theorem 19.7.1.

Lemma 19.7.2.

$$\operatorname{Ex} \left[c^T \right] \leq e^{(c-1) \operatorname{Ex}[T]}.$$

⁶See “A Conversation with Herman Chernoff,” *Statistical Science* 1996, Vol. 11, No. 4, pp 335–350.

Proof.

$$\begin{aligned}
 \text{Ex}[c^T] &= \text{Ex}[c^{T_1 + \dots + T_n}] && \text{(def of } T) \\
 &= \text{Ex}[c^{T_1} \dots c^{T_n}] \\
 &= \text{Ex}[c^{T_1}] \dots \text{Ex}[c^{T_n}] && \text{(independent product Cor 18.5.7)} \\
 &\leq e^{(c-1)\text{Ex}[T_1]} \dots e^{(c-1)\text{Ex}[T_n]} && \text{(by Lemma 19.7.3 below)} \\
 &= e^{(c-1)(\text{Ex}[T_1] + \dots + \text{Ex}[T_n])} \\
 &= e^{(c-1)\text{Ex}[T_1 + \dots + T_n]} && \text{(linearity of Ex[.])} \\
 &= e^{(c-1)\text{Ex}[T]}.
 \end{aligned}$$

■

Lemma 19.7.3.

$$\text{Ex}[c^{T_i}] \leq e^{(c-1)\text{Ex}[T_i]}$$

Proof. All summations below range over values v taken by the random variable T_i , which are all required to be in the interval $[0, 1]$.

$$\begin{aligned}
 \text{Ex}[c^{T_i}] &= \sum c^v \Pr[T_i = v] && \text{(def of Ex[.])} \\
 &\leq \sum (1 + (c-1)v) \Pr[T_i = v] && \text{(convexity — see below)} \\
 &= \sum \Pr[T_i = v] + (c-1)v \Pr[T_i = v] \\
 &= \sum \Pr[T_i = v] + (c-1) \sum v \Pr[T_i = v] \\
 &= 1 + (c-1)\text{Ex}[T_i] \\
 &\leq e^{(c-1)\text{Ex}[T_i]} && \text{(since } 1 + z \leq e^z).
 \end{aligned}$$

The second step relies on the inequality

$$c^v \leq 1 + (c-1)v,$$

which holds for all v in $[0, 1]$ and $c \geq 1$. This follows from the general principle that a convex function, namely c^v , is less than the linear function, $1 + (c-1)v$, between their points of intersection, namely $v = 0$ and 1 . This inequality is why the variables T_i are restricted to the interval $[0, 1]$. ■

19.7.6 Comparing the Bounds

Suppose that we have a collection of mutually independent events $A_1, A_2, \dots, A_n$, and we want to know how many of the events are likely to occur.

Let T_i be the indicator random variable for A_i and define

$$p_i = \Pr[T_i = 1] = \Pr[A_i]$$

for $1 \leq i \leq n$. Define

$$T = T_1 + T_2 + \dots + T_n$$

to be the number of events that occur.

We know from Linearity of Expectation that

$$\begin{aligned} \text{Ex}[T] &= \text{Ex}[T_1] + \text{Ex}[T_2] + \dots + \text{Ex}[T_n] \\ &= \sum_{i=1}^n p_i. \end{aligned}$$

This is true even if the events are *not* independent.

By Theorem 19.4.8, we also know that

$$\begin{aligned} \text{Var}[T] &= \text{Var}[T_1] + \text{Var}[T_2] + \dots + \text{Var}[T_n] \\ &= \sum_{i=1}^n p_i(1 - p_i), \end{aligned}$$

and thus that

$$\sigma_T = \sqrt{\sum_{i=1}^n p_i(1 - p_i)}.$$

This is true even if the events are only pairwise independent.

Markov's Theorem tells us that for any $c > 1$,

$$\Pr[T \geq c \text{Ex}[T]] \leq \frac{1}{c}.$$

Chebyshev's Theorem gives us the stronger result that

$$\Pr[|T - \text{Ex}[T]| \geq c\sigma_T] \leq \frac{1}{c^2}.$$

The Chernoff Bound gives us an even stronger result, namely, that for any $c > 0$,

$$\Pr[T - \text{Ex}[T] \geq c \text{Ex}[T]] \leq e^{-(c \ln(c) - c + 1) \text{Ex}[T]}.$$

In this case, the probability of exceeding the mean by $c \text{Ex}[T]$ decreases as an exponentially small function of the deviation.

By considering the random variable $n - T$, we can also use the Chernoff Bound to prove that the probability that T is much lower than $\text{Ex}[T]$ is also exponentially small.

19.7.7 Murphy’s Law

If the expectation of a random variable is much less than 1, then Markov’s Theorem implies that there is only a small probability that the variable has a value of 1 or more. On the other hand, a result that we call *Murphy’s Law*⁷ says that if a random variable is an independent sum of 0-1-valued variables and has a large expectation, then there is a huge probability of getting a value of at least 1.

Theorem 19.7.4 (Murphy’s Law). *Let $A_1, A_2, \dots, A_n$ be mutually independent events. Let T_i be the indicator random variable for A_i and define*

$$T ::= T_1 + T_2 + \dots + T_n$$

to be the number of events that occur. Then

$$\Pr[T = 0] \leq e^{-\text{Ex}[T]}.$$

Proof.

$$\begin{aligned} \Pr[T = 0] &= \Pr[\bar{A}_1 \wedge \bar{A}_2 \wedge \dots \wedge \bar{A}_n] \\ &= \prod_{i=1}^n \Pr[\bar{A}_i] && \text{(by independence of } A_i) \\ &= \prod_{i=1}^n (1 - \Pr[A_i]) \\ &\leq \prod_{i=1}^n e^{-\Pr[A_i]} && \text{(since } 1 - x \leq e^{-x}) \\ &= e^{-\sum_{i=1}^n \Pr[A_i]} \\ &= e^{-\sum_{i=1}^n \text{Ex}[T_i]} && \text{(since } T_i \text{ is an indicator for } A_i) \\ &= e^{-\text{Ex}[T]} && \text{(linearity of expectation)} \quad \blacksquare \end{aligned}$$

⁷This is in reference and deference to the famous saying that “If something can go wrong, it will go wrong.”

For example, given any set of mutually independent events, if you expect 10 of them to happen, then at least one of them will happen with probability at least $1 - e^{-10}$. The probability that none of them happen is at most $e^{-10} < 1/22000$.

So if there are a lot of independent things that can go wrong and their probabilities sum to a number much greater than 1, then Theorem 19.7.4 proves that some of them surely will go wrong.

This result can help to explain “coincidences,” “miracles,” and crazy events that seem to have been very unlikely to happen. Such events do happen, in part, because there are so many possible unlikely events that the sum of their probabilities is greater than one. For example, someone *does* win the lottery.

In fact, if there are 100,000 random tickets in Pick-4, Theorem 19.7.4 says that the probability that there is no winner is less than $e^{-10} < 1/22000$. More generally, there are literally millions of one-in-a-million possible events and so some of them will surely occur.

19.8 Really Great Expectations

Making independent tosses of a fair coin until some desired pattern comes up is a simple process you should feel solidly in command of by now, right? So how about a bet about the simplest such process —tossing until a head comes up? Ok, you’re wary of betting with us, but how about this: we’ll let *you set the odds*.

19.8.1 Repeating Yourself

Here’s the bet: you make independent tosses of a fair coin until a head comes up. Then you will repeat the process. If a second head comes up in the same or fewer tosses than the first, you have to start over yet again. You keep starting over until you finally toss a run of tails longer than your first one. The payment rules are that you will pay me 1 cent each time you start over. When you win by finally getting a run of tails longer than your first one, I will pay you some generous amount. And by the way, you’re certain to win —whatever your initial run of tails happened to be, a longer run will occur again with probability 1!

For example, if your first tosses are TTTH, then you will keep tossing until you get a run of 4 tails. So your winning flips might be

TTTHTHTTHHTTHTHTTTHTHHHTTTT.

In this run there are 10 heads, which means you had to start over 9 times. So you would have paid me 9 cents by the time you finally won by tossing 4 tails. Now

you’ve won, and I’ll pay you generously —how does 25 cents sound? Maybe you’d rather have \$1? How about \$10?

Of course there’s a trap here. Let’s calculate your expected winnings.

Suppose your initial run of tails had length k . After that, each time a head comes up, you have to start over and try to get $k + 1$ tails in a row. If we regard your getting $k + 1$ tails in a row as a “failed” try, and regard your having to start over because a head came up too soon as a “successful” try, then the number of times you have to start over is the number of tries till the first failure. So the expected number of tries will be the mean time to failure, which is 2^{k+1} , because the probability of tossing $k + 1$ tails in a row is $2^{-(k+1)}$.

Let T be the length of your initial run of tails. So $T = k$ means that your initial tosses were $T^k H$. Let R be the number of times you repeat trying to beat your original run of tails. The number of cents you expect to finish with is the number of cents in my generous payment minus $\text{Ex}[R]$. It’s now easy to calculate $\text{Ex}[R]$ by conditioning on the value of T :

$$\text{Ex}[R] = \sum_{k \in \mathbb{N}} \text{Ex}[R \mid T = k] \cdot \Pr[T = k] = \sum_{k \in \mathbb{N}} 2^{k+1} \cdot 2^{-(k+1)} = \sum_{k \in \mathbb{N}} 1 = \infty.$$

So you can expect to pay me an infinite number of cents before winning my “generous” payment. No amount of generosity can make this bet fair!

We haven’t faced infinite expectations until now, but they just popped up in a very simple way. In fact this particular example is a special case of an astonishingly general one worked out in Problem 19.24: the expected waiting time for *any* random variable to achieve a larger value is infinite.

19.8.2 The St. Petersburg Paradox

One of the simplest casino bets is on “red” or “black” at the roulette table. In each play at roulette, a small ball is set spinning around a roulette wheel until it lands in a red, black, or green colored slot. The payoff for a bet on red or black matches the bet; for example, if you bet \$10 on red and the ball lands in a red slot, you get back your original \$10 bet plus another matching \$10.

In the US, a roulette wheel has two green slots among 18 black and 18 red slots, so the probability of red is $18/38 \approx 0.473$. In Europe, where roulette wheels have only one green slot, the odds for red are a little better —that is, $18/37 \approx 0.486$ —but still less than even.

There is a notorious gambling strategy allegedly used against the casino in St. Petersburg way back in czarist days: bet \$10 on red, and keep doubling the bet until a red comes up. This strategy implies that a player will leave the game as a net winner of \$10 as soon as the red first appears.

Suppose you had the good fortune to gamble against a fair roulette wheel. Then whatever your bet on a spin of the wheel, you are equally likely to win or lose, and your expected win is 0. This also means that the expected win after any given number of spins remains zero, so even playing the St. Petersburg strategy it seems your expected win would be 0.

But wait a minute. As long as there is a fixed, positive probability of red appearing on each spin of the wheel, it's *certain* that red will eventually come up. That is, you can be certain of leaving the casino having won \$10. This implies that even against an *unfair* roulette wheel, your expected win is \$10, contradicting the idea that you can't expect to win in a game that's biased against you.

This is paradoxical and something's obviously wrong here. In fact, there are two things wrong.

The first thing that's wrong is the argument claiming that the expectation is 0. It would be 0 if the number of bets had a fixed bound. If you could only make n bets, then your expectation in the fair game would be the sum of your expected wins on each of the bets, namely, $n \cdot 0 = 0$. But there is no such fixed bound, and that changes things.

To explain this carefully, let C_i be the number of dollars won on the i th spin. So $C_i = 2^{i-1}$ when red comes up for the first time on the i th spin, and $C_i = -2^{i-1}$, when the first red spin comes up after the i th spin. We can define C_i to be 0 if the first red comes up before the i th spin. This means

$$\text{Ex}[C_i] = 0.$$

Also, the total of your winnings is

$$C ::= \sum_{i \in \mathbb{Z}^+} C_i.$$

The conclusion that $\text{Ex}[C] = 10$ follows from Total Expectation, conditioning on the number of spins till a red first occurs. Namely, if the first red occurs on the i th spin, the amount won is

$$-10 \cdot (1 + 2 + 2^2 + \cdots + 2^{i-2}) + 10 \cdot 2^{i-1} = 10.$$

Then by Total Expectation,

$$\begin{aligned} \text{Ex}[C] &= \sum_{i \in \mathbb{Z}^+} \text{Ex}[C \mid \text{first red on } i \text{th spin}] \cdot \text{Pr}[\text{first red on } i \text{th spin}] \\ &= \sum_{i \in \mathbb{Z}^+} 10 \cdot 2^{-i} = 10 \cdot \sum_{i \in \mathbb{Z}^+} 2^{-i} = 10 \cdot 1 = 10. \end{aligned}$$

So sure enough,

$$\text{Ex}[C] ::= \text{Ex} \left[\sum_{i \in \mathbb{Z}^+} C_i \right] = 10. \quad (19.25)$$

But since $\text{Ex}[C_i] = 0$,

$$\sum_{i \in \mathbb{Z}^+} \text{Ex}[C_i] = \sum_{i \in \mathbb{Z}^+} 0 = 0. \quad (19.26)$$

It seems that (19.26) and (19.25) contradict each other, but they don't. The apparent contradiction comes from applying infinite linearity to conclude

False Claim.

$$\text{Ex} \left[\sum_{i \in \mathbb{Z}^+} C_i \right] = \sum_{i \in \mathbb{Z}^+} \text{Ex}[C_i].$$

But this is a case where the convergence conditions required for infinite linearity don't hold. Even though the left hand sum converges (to 10) and the right hand sum converges (to 0), the infinite linearity Theorem (18.5.5) requires that the sum of expectations of *absolute values* converges. That is, infinite linearity would follow if the sum

$$\sum_{i \in \mathbb{Z}^+} \text{Ex}[|C_i|] \quad (19.27)$$

converged. But

$$\begin{aligned} \text{Ex}[|C_i|] &= (|10 \cdot 2^{i-1}|) \cdot \text{Pr}[\text{1st red in } i \text{ th spin}] \\ &\quad + (|-10 \cdot 2^{i-1}|) \cdot \text{Pr}[\text{1st red after } i \text{ th spin}] \\ &\quad + 0 \cdot \text{Pr}[\text{1st red before the } i \text{ th spin}] \\ &= (10 \cdot 2^{i-1}) \cdot 2^{-(i)} + (10 \cdot 2^{i-1}) \cdot 2^{-(i)} + 0 = 10, \end{aligned}$$

so the sum (19.27) diverges—rapidly.

Probability theory truly leads to this absurd conclusion: a game entailing an unbounded number of fair bets may not be fair in the end. In fact, even against an *unfair* wheel, as long as there is some fixed positive probability of red on each spin, you are certain to win \$10 playing the St. Petersburg strategy!

This brings us to the second thing that's wrong here: you may wind up losing a lot of money before you catch up with your net win of \$10. Let L be the number of dollars you need to have in order to keep betting until the wheel finally spins red. If red first comes up on the i th spin, then L would equal

$$10(1 + 2 + 4 + \cdots + 2^i) = 10(2^{i+1} - 1)$$

By Total Expectation,

$$\begin{aligned} \text{Ex}[L] &= \sum_{i \in \mathbb{Z}^+} \text{Ex}[L \mid \text{1st red in } i \text{th spin}] \cdot \text{Pr}[\text{1st red in } i \text{th spin}] \\ &= \sum_{i \in \mathbb{Z}^+} (10 \cdot (2^{i+1} - 1)) \cdot 2^{-i} \geq \sum_{i \in \mathbb{Z}^+} 10 = \infty. \end{aligned}$$

That is, you can expect to lose an infinite amount of money before finally winning \$10 —giving you a 0% profit.

So yes, probability theory leads to the absurd conclusion that, even with the odds heavily against you, you’re certain to win playing roulette, but only if you make the absurd assumption that you have an infinite bankroll. We can’t fault the theory for reaching an absurd conclusion from an absurd assumption.

Problems for Section 19.2

Class Problems

Problem 19.1.

A herd of cows is stricken by an outbreak of *cold cow disease*. The disease lowers the normal body temperature of a cow, and a cow will die if its temperature goes below 90 degrees F. The disease epidemic is so intense that it lowered the average temperature of the herd to 85 degrees. Body temperatures as low as 70 degrees, **but no lower**, were actually found in the herd.

(a) Prove that at most 3/4 of the cows could have survived.

Hint: Let T be the temperature of a random cow. Make use of Markov’s bound.

(b) Suppose there are 400 cows in the herd. Show that the bound of part (a) is best possible by giving an example set of temperatures for the cows so that the average herd temperature is 85, and with probability 3/4, a randomly chosen cow will have a high enough temperature to survive.

Homework Problems

Problem 19.2.

If R is a nonnegative random variable, then Markov’s Theorem gives an upper bound on $\text{Pr}[R \geq x]$ for any real number $x > \text{Ex}[R]$. If a constant $b \geq 0$ is a lower bound on R , then Markov’s Theorem can also be applied to $R - b$ to obtain a possibly different bound on $\text{Pr}[R \geq x]$.

(a) Show that if $b > 0$, applying Markov’s Theorem to $R - b$ gives a smaller upper bound on $\text{Pr}[R \geq x]$ than simply applying Markov’s Theorem directly to R .

(b) What value of $b \geq 0$ in part (a) gives the best bound?

Problems for Section 19.4

Practice Problems

Problem 19.3.

A gambler plays 120 hands of draw poker, 60 hands of black jack, and 20 hands of stud poker per day. He wins a hand of draw poker with probability $1/6$, a hand of black jack with probability $1/2$, and a hand of stud poker with probability $1/5$.

(a) What is the expected number of hands the gambler wins in a day?

(b) What would the Markov bound be on the probability that the gambler will win at least 108 hands on a given day?

(c) Assume the outcomes of the card games are pairwise independent. What is the variance in the number of hands won per day?

(d) What would the Chebyshev bound be on the probability that the gambler will win at least 108 hands on a given day? You may answer with a numerical expression that is not completely evaluated.

Problem 19.4. (a) A computer program crashes at the end of each hour of use with probability $1/p$, if it has not crashed already. If H is the number of hours until the first crash, we know

$$\text{Ex}[H] = \frac{1}{p}, \quad (\text{Equation (18.8)})$$

$$\text{Var}[H] = \frac{q}{p^2} \quad (\text{Equation (19.8)}),$$

where $q ::= 1 - p$.

(b) What is the Chebyshev bound on

$$\Pr[|H - (1/p)| > x/p]$$

where $x > 0$?

(c) Conclude from part (b) that for $a \geq 2$,

$$\Pr[H > a/p] \leq \frac{1-p}{(a-1)^2}$$

Hint: Check that $|H - (1/p)| > (a-1)/p$ iff $H > a/p$.

(d) What actually is

$$\Pr[H > a/p]?$$

Conclude that for any fixed $p > 0$, the probability that $H > a/p$ is an asymptotically smaller function of a than the Chebyshev bound of part (c).

Class Problems

Problem 19.5.

The hat-check staff has had a long day serving at a party, and at the end of the party they simply return the n checked hats in a random way such that the probability that any particular person gets their own hat back is $1/n$.

Let X_i be the indicator variable for the i th person getting their own hat back. Let S_n be the total number of people who get their own hat back.

(a) What is the expected number of people who get their own hat back?

(b) Write a simple formula for $\text{Ex}[X_i X_j]$ for $i \neq j$.

Hint: What is $\Pr[X_j = 1 \mid X_i = 1]$?

(c) Explain why you cannot use the variance of sums formula to calculate $\text{Var}[S_n]$.

(d) Show that $\text{Ex}[S_n^2] = 2$. *Hint:* $X_i^2 = X_i$.

(e) What is the variance of S_n ?

(f) Show that there is at most a 1% chance that more than 10 people get their own hat back. Try to give an intuitive explanation of why the chance remains this small regardless of n .

Problem 19.6.

For any random variable, R , with mean, μ , and standard deviation, σ , the Chebyshev Bound says that for any real number $x > 0$,

$$\Pr[|R - \mu| \geq x] \leq \left(\frac{\sigma}{x}\right)^2.$$

Show that for any real number, μ , and real numbers $x \geq \sigma > 0$, there is an R for which the Chebyshev Bound is tight, that is,

$$\Pr[|R| \geq x] = \left(\frac{\sigma}{x}\right)^2. \quad (19.28)$$

Hint: First assume $\mu = 0$ and let R only take values 0, $-x$, and x .

Homework Problems

Problem 19.7.

There is a “one-sided” version of Chebyshev’s bound for deviation above the mean:

Lemma (One-sided Chebyshev bound).

$$\Pr[R - \text{Ex}[R] \geq x] \leq \frac{\text{Var}[R]}{x^2 + \text{Var}[R]}.$$

Hint: Let $S_a ::= (R - \text{Ex}[R] + a)^2$, for $0 \leq a \in \mathbb{R}$. So $R - \text{Ex}[R] \geq x$ implies $S_a \geq (x + a)^2$. Apply Markov’s bound to $\Pr[S_a \geq (x + a)^2]$. Choose a to minimize this last bound.

Problem 19.8.

A man has a set of n keys, one of which fits the door to his apartment. He tries the keys until he finds the correct one. Give the expectation and variance for the number of trials until success if

- (a) he tries the keys at random (possibly repeating a key tried earlier)
- (b) he chooses keys randomly from among those he has not yet tried.

Problems for Section 19.6

Practice Problems

Problem 19.9.

You work for the president and you want to estimate the fraction p of voters in the entire nation that will prefer him in the upcoming elections. You do this by random sampling. Specifically, you select a random voter and ask them who they are going to vote for. You do this n times, with each voter selected with uniform probability and independently of other selections. Finally, you use the fraction P of voters who said they will vote for the President as an estimate for p .

(a) Our theorems about sampling and distributions allow us to calculate how confident we can be that the random variable, P , takes a value near the constant, p . This calculation uses some facts about voters and the way they are chosen. Circle the true facts among the following:

1. Given a particular voter, the probability of that voter preferring the President is p .
2. The probability that some voter is chosen more than once in the random sample goes to one as n increases.

3. The probability that some voter is chosen more than once in the random sample goes to zero as the population of voters grows.
4. All voters are equally likely to be selected as the third in the random sample of n voters (assuming $n \geq 3$).
5. The probability that the second voter in the random sample will favor the President, given that the first voter prefers the President, is greater than p .
6. The probability that the second voter in the random sample will favor the President, given that the second voter is from the same state as the first, may not equal p .

(b) Suppose that according to your calculations, the following is true about your polling:

$$\Pr[|P - p| \leq 0.04] \geq 0.95.$$

You do the asking, you count how many said they will vote for the President, you divide by n , and find the fraction is 0.53. Among the following, circle the legitimate things you might say in a call to the President:

1. Mr. President, $p = 0.53$!
2. Mr. President, with probability at least 95 percent, p is within 0.04 of 0.53.
3. Mr. President, either p is within 0.04 of 0.53 or something very strange (5-in-100) has happened.
4. Mr. President, we can be 95% confident that p is within 0.04 of 0.53.

Class Problems

Problem 19.10.

A recent Gallup poll found that 35% of the adult population of the United States believes that the theory of evolution is “well-supported by the evidence.” Gallup polled 1928 Americans selected uniformly and independently at random. Of these, 675 asserted belief in evolution, leading to Gallup’s estimate that the fraction of Americans who believe in evolution is $675/1928 \approx 0.350$. Gallup claims a margin of error of 3 percentage points, that is, he claims to be confident that his estimate is within 0.03 of the actual percentage.

- (a) What is the largest variance an indicator variable can have?
- (b) Use the Pairwise Independent Sampling Theorem to determine a confidence level with which Gallup can make his claim.

(c) Gallup actually claims greater than 99% confidence in his estimate. How might he have arrived at this conclusion? (Just explain what quantity he could calculate; you do not need to carry out a calculation.)

(d) Accepting the accuracy of all of Gallup’s polling data and calculations, can you conclude that there is a high probability that the number of adult Americans who believe in evolution is 35 ± 3 percent?

Problem 19.11.

Let $B_1, B_2, \dots, B_n$ be mutually independent random variables with a uniform distribution on the integer interval $[1, d]$. Let D equal to the number of events $[B_i = B_j]$ that happen where $i \neq j$. It was observed in Section 17.6.6 (and proved in Problem 18.2) that $\Pr[B_i = B_j] = 1/d$ for $i \neq j$ and that the events $[B_i = B_j]$ are pairwise independent.

Let $E_{i,j}$ be the indicator variable for the event $[B_i = B_j]$.

(a) What are $\text{Ex}[E_{i,j}]$ and $\text{Var}[E_{i,j}]$ for $i \neq j$?

(b) What are $\text{Ex}[D]$ and $\text{Var}[D]$?

(c) In a 6.01 class of 500 students, the youngest student was born 15 years ago and the oldest 35 years ago. Let D be the number of students in the class who were born on exactly the same date. What is the probability that $4 \leq S \leq 32$? (For simplicity, assume that the distribution of birthdays is uniform over the 7305 days in the two decade interval from 35 years ago to 15 years ago.)

Problem 19.12.

A defendant in traffic court is trying to beat a speeding ticket on the grounds that—since virtually everybody speeds on the turnpike—the police have unconstitutional discretion in giving tickets to anyone they choose. (By the way, we don’t recommend this defense : -) .)

To support his argument, the defendant arranged to get a random sample of trips by 3,125 cars on the turnpike and found that 94% of them broke the speed limit at some point during their trip. He says that as a consequence of sampling theory (in particular, the Pairwise Independent Sampling Theorem), the court can be 95% confident that the actual percentage of all cars that were speeding is $94 \pm 4\%$.

The judge observes that the actual number of car trips on the turnpike was never considered in making this estimate. He is skeptical that, whether there were a thousand, a million, or 100,000,000 car trips on the turnpike, sampling only 3,125

is sufficient to be so confident.

Suppose you were the defendant. How would you explain to the judge why the number of randomly selected cars that have to be checked for speeding *does not depend on the number of recorded trips*? Remember that judges are not trained to understand formulas, so you have to provide an intuitive, nonquantitative explanation.

Problem 19.13.

The proof of the Pairwise Independent Sampling Theorem 19.5.1 was given for a sequence $R_1, R_2, \dots$ of pairwise independent random variables with the same mean and variance.

The theorem generalizes straightforwardly to sequences of pairwise independent random variables, possibly with *different* distributions, as long as all their variances are bounded by some constant.

Theorem (Generalized Pairwise Independent Sampling). *Let $X_1, X_2, \dots$ be a sequence of pairwise independent random variables such that $\text{Var}[X_i] \leq b$ for some $b \geq 0$ and all $i \geq 1$. Let*

$$A_n ::= \frac{X_1 + X_2 + \dots + X_n}{n},$$

$$\mu_n ::= \text{Ex}[A_n].$$

Then for every $\epsilon > 0$,

$$\Pr[|A_n - \mu_n| > \epsilon] \leq \frac{b}{\epsilon^2} \cdot \frac{1}{n}. \quad (19.29)$$

(a) Prove the Generalized Pairwise Independent Sampling Theorem.

(b) Conclude that the following holds:

Corollary (Generalized Weak Law of Large Numbers). *For every $\epsilon > 0$,*

$$\lim_{n \rightarrow \infty} \Pr[|A_n - \mu_n| \leq \epsilon] = 1.$$

Problem 19.14.

An *International Journal of Epidemiology* has a policy of publishing papers about drug trial results only if the conclusion about the drug’s effectiveness (or lack thereof) holds at the 95% confidence level. The editors and reviewers carefully check that any trial whose results they publish was *properly performed and accurately reported*. They are also careful to check that trials whose results they publish have been conducted independently of each other.

The editors of the Journal reason that under this policy, their readership can be confident that at most 5% of the published studies will be mistaken. Later, the editors are embarrassed —and astonished —to learn that *every one* of the 20 drug trial results they published during the year was wrong. The editors thought that because the trials were conducted independently, the probability of publishing 20 wrong results was negligible, namely, $(1/20)^{20} < 10^{-25}$.

Write a brief explanation to these befuddled editors explaining what’s wrong with their reasoning and how it could be that all 20 published studies were wrong.

Hint: [xkcd comic](#): “significant”

Exam Problems

Problem 19.15.

Yesterday, the programmers at a local company wrote a large program. To estimate the fraction, b , of lines of code in this program that are buggy, the QA team will take a small sample of lines chosen randomly and independently (so it is possible, though unlikely, that the same line of code might be chosen more than once). For each line chosen, they can run tests that determine whether that line of code is buggy, after which they will use the fraction of buggy lines in their sample as their estimate of the fraction b .

The company statistician can use estimates of a binomial distribution to calculate a value, s , for a number of lines of code to sample which ensures that with 97% confidence, the fraction of buggy lines in the sample will be within 0.006 of the actual fraction, b , of buggy lines in the program.

Mathematically, the *program* is an actual outcome that already happened. The *random sample* is a random variable defined by the process for randomly choosing s lines from the program. The justification for the statistician’s confidence depends on some properties of the program and how the random sample of s lines of code from the program are chosen. These properties are described in some of the statements below. Indicate which of these statements are true, and explain your answers.

1. The probability that the ninth line of code in the *program* is buggy is b .
2. The probability that the ninth line of code chosen for the *random sample* is defective, is b .
3. All lines of code in the program are equally likely to be the third line chosen in the *random sample*.
4. Given that the first line chosen for the *random sample* is buggy, the probability that the second line chosen will also be buggy is greater than b .

5. Given that the last line in the *program* is buggy, the probability that the next-to-last line in the program will also be buggy is greater than b .
6. The expectation of the indicator variable for the last line in the *random sample* being buggy is b .
7. Given that the first two lines of code selected in the *random sample* are the same kind of statement—they might both be assignment statements, or both be conditional statements, or both loop statements,...—the probability that the first line is buggy may be greater than b .
8. There is zero probability that all the lines in the *random sample* will be different.

Problem 19.16.

Let $G_1, G_2, G_3, \dots$, be an infinite sequence of pairwise independent random variables with the same expectation, μ , and the same finite variance. Let

$$f(n, \epsilon) ::= \Pr \left[\left| \frac{\sum_{i=1}^n G_i}{n} - \mu \right| \leq \epsilon \right].$$

The Weak Law of Large Numbers can be expressed as a logical formula of the form:

$$\forall \epsilon > 0 \, Q_1 Q_2 \dots [f(n, \epsilon) \geq 1 - \delta]$$

where $Q_1 Q_2 \dots$ is a sequence of quantifiers from among:

$$\begin{array}{cccccc} \forall n & \exists n & \forall n_0 & \exists n_0 & \forall n \geq n_0 & \exists n \geq n_0 \\ \forall \delta > 0 & \exists \delta > 0 & \forall \delta \geq 0 & \exists \delta \geq 0 & & \end{array}$$

Here the n and n_0 range over nonnegative integers, and δ and ϵ range over real numbers.

Write out the proper sequence $Q_1 Q_2 \dots$.

Problems for Section 19.7

Class Problems

Problem 19.17.

We want to store 2 billion records into a hash table that has 1 billion slots. Assuming the records are randomly and independently chosen with uniform probability

of being assigned to each slot, two records are expected to be stored in each slot. Of course under a random assignment, some slots may be assigned more than two records.

(a) Show that the probability that a given slot gets assigned more than 23 records is less than e^{-36} .

Hint: For $c = 12$, the value of $c \ln c - c + 1$ is greater than 18.

(b) Show that the probability that there is a slot that gets assigned more than 23 records is less than e^{-15} . This is less than $1/3,000,000$. *Hint:* $\ln 10^9 < 21$.

Problem 19.18.

Sometimes I forget a few items when I leave the house in the morning. For example, here are probabilities that I forget various pieces of footwear:

left sock	0.2
right sock	0.1
left shoe	0.1
right shoe	0.3

(a) Let X be the number of these that I forget. What is $\text{Ex}[X]$?

(b) Upper bound the probability that I forget one or more items. Make no independence assumptions.

(c) Use the Markov Inequality to upper bound the probability that I forget 3 or more items.

(d) Now suppose that I forget each item of footwear independently. Use Chebyshev’s Inequality to upper bound the probability that I forget two or more items.

(e) Use Theorem 19.7.4 to lower bound the probability that I forget one or more items.

(f) I’m supposed to remember many other items, of course: clothing, watch, backpack, notebook, pencil, kleenex, ID, keys, etc. Let X be the total number of items I remember. Suppose I remember items mutually independently and $\text{Ex}[X] = 36$. Use Chernoff’s Bound to give an upper bound on the probability that I remember 48 or more items.

(g) Give an upper bound on the probability that I remember 108 or more items.

Problem 19.19.

Reasoning based on the Chernoff bound goes a long way in explaining the recent subprime mortgage collapse. A bit of standard vocabulary about the mortgage market is needed:

- A **loan** is money lent to a borrower. If the borrower does not pay on the loan, the loan is said to be in **default**, and collateral is seized. In the case of mortgage loans, the borrower’s home is used as collateral.
- A **bond** is a collection of loans, packaged into one entity. A bond can be divided into **tranches**, in some ordering, which tell us how to assign losses from defaults. Suppose a bond contains 1000 loans, and is divided into 10 tranches of 100 bonds each. Then, all the defaults must fill up the lowest tranche before the affect others. For example, suppose 150 defaults happened. Then, the first 100 defaults would occur in tranche 1, and the next 50 defaults would happen in tranche 2.
- The lowest tranche of a bond is called the **mezzanine tranche**.
- We can make a “super bond” of tranches called a **collateralized debt obligation (CDO)** by collecting mezzanine tranches from different bonds. This super bond can then be itself separated into tranches, which are again ordered to indicate how to assign losses.

(a) Suppose that 1000 loans make up a bond, and the fail rate is 5% in a year. Assuming mutual independence, give an upper bound for the probability that there are one or more failures in the second-worst tranche. What is the probability that there are failures in the best Tranche?

(b) Now, do not assume that the loans are independent. Give an upper bound for the probability that there are one or more failures in the second tranche. What is an upper bound for the probability that the entire bond defaults? Show that it is a tight bound. *Hint:* Use Markov’s theorem.

(c) Given this setup (and assuming mutual independence between the loans), what is the expected failure rate in the mezzanine tranche?

(d) We take the mezzanine tranches from 100 bonds and create a CDO. What is the expected number of underlying failures to hit the CDO?

(e) We divide this CDO into 10 tranches of 1000 bonds each. Assuming mutual independence, give an upper bound on the probability of one or more failures in the best tranche. The third tranche?

(f) Repeat the previous question without the assumption of mutual independence.

Homework Problems

Problem 19.20.

An infinite version of Murphy’s Law is that if an infinite number of mutually independent events are expected to happen, then the probability that only finitely many happen is 0. This is known as the first *Borel-Cantelli Lemma*.

(a) Let $A_0, A_1, \dots$ be any infinite sequence of mutually independent events such that

$$\sum_{n \in \mathbb{N}} \Pr[A_n] = \infty. \quad (19.30)$$

Prove that $\Pr[\text{no } A_n \text{ occurs}] = 0$.

Hint: B_k the event that no A_n with $n \leq k$ occurs. So the event that no A_n occurs is

$$B ::= \bigcap_{k \in \mathbb{N}} B_k.$$

Apply Murphy’s Law, Theorem 19.7.4, to B_k .

(b) Conclude that $\Pr[\text{only finitely many } A_n \text{'s occur}] = 0$.

Hint: Let C_k be the event that no A_n with $n \geq k$ occurs. So the event that only finitely many A_n ’s occur is

$$C ::= \bigcup_{k \in \mathbb{N}} C_k.$$

Apply part (a) to C_k .

Problems for Section 19.8

Practice Problems

Problem 19.21.

Let R be a positive integer valued random variable such that

$$\text{PDF}_R(n) = \frac{1}{cn^3},$$

where

$$c ::= \sum_{n=1}^{\infty} \frac{1}{n^3}.$$

(a) Prove that $\text{Ex}[R]$ is finite.

(b) Prove that $\text{Var}[R]$ is infinite.

Problem 19.22.

Let T be a positive integer valued random variable such that

$$\text{PDF}_T(n) = \frac{1}{an^2},$$

where

$$a ::= \sum_{n \in \mathbb{Z}^+} \frac{1}{n^2}.$$

(a) Prove that $\text{Ex}[T]$ is infinite.

(b) Prove that $\text{Ex}[\sqrt{T}]$ is finite.

Class Problems

Problem 19.23.

You have a biased coin with nonzero probability $p < 1$ of coming up heads. You toss until a head comes up, and then, as in Section 19.8, you keep tossing until you get a long run of tails, but this time let “long run” mean a run of tails that is at least $k - 10$ when your initial run was length k . Prove that the expected number of times you toss a head and start over is still infinite.

Problem 19.24.

Let $T_0, T_1, \dots$ be a sequence of mutually independent random variables with the same distribution. Let

$$R ::= \min\{k > 0 \mid T_k > T_0\}.$$

(a) Suppose the range of the T_0 is the set $\{t_0 < t_1 < t_2 < \dots\}$. Explain why the following Theorem implies that $\text{Ex}[R] = \infty$.

Theorem 19.8.1. *If $p_0 + p_1 + p_2 + \dots = 1$ and all $p_i \geq 0$, then the sum*

$$\Omega ::= \sum_{k \in \mathbb{N}} \frac{p_k}{p_{k+1} + p_{k+2} + \dots}.$$

diverges.

(b) Let

$$S_k ::= p_k + p_{k+1} + \dots,$$

and

$$a_k ::= \frac{S_k}{S_{k+1}} - 1.$$

Prove that

$$\Omega = \sum_{k \in \mathbb{N}} a_k. \quad (19.31)$$

(c) Prove that

$$\prod_{k \leq n} (a_k + 1) = \frac{1}{S_{n+1}}.$$

(d) Conclude from part (c) that

$$\prod_{k \in \mathbb{N}} (a_k + 1) = \infty. \quad (19.32)$$

(e) Conclude that $e^\Omega = \infty$ and hence $\Omega = \infty$.

Exam Problems

Problem 19.25.

You have a process for generating a positive integer, K . The behavior of your process each time you use it is (mutually) independent of all its other uses. You use your process to generate a random integer, and then use your procedure repeatedly until you generate an integer as big as your first one. Let R be the number of additional integers you have to generate.

(a) State and briefly explain a simple closed formula for $\text{Ex}[R \mid K = k]$ in terms of $\text{Pr}[K \geq k]$.

Suppose $\text{Pr}[K = k] = \Theta(k^{-4})$.

(b) Show that $\text{Pr}[K \geq k] = \Theta(k^{-3})$.

(c) Show that $\text{Ex}[R]$ is infinite.



20 Random Processes

Random Walks are used to model situations in which an object moves in a sequence of steps in randomly chosen directions. For example in Physics, three-dimensional random walks are used to model Brownian motion and gas diffusion. In this chapter we’ll examine two examples of random walks. First, we’ll model gambling as a simple 1-dimensional random walk—a walk along a straight line. Then we’ll explain how the Google search engine used random walks through the graph of world-wide web links to determine the relative importance of websites.

20.1 Gamblers’ Ruin

Suppose a gambler starts with an initial stake of n dollars and makes a sequence of \$1 bets. If he wins an individual bet, he gets his money back plus another \$1. If he loses the bet, he loses the \$1.

We can model this scenario as a random walk between integer points on the real line. The position on the line at any time corresponds to the gambler’s cash-on-hand or *capital*. Walking one step to the right corresponds to winning a \$1 bet and thereby increasing his capital by \$1. Similarly, walking one step to the left corresponds to losing a \$1 bet.

The gambler plays until either he runs out of money or increases his capital to a target amount of T dollars. The amount $T - n$ is defined to be his *intended profit*. If he reaches his target, then he is called an overall *winner*, and he will have won his intended profit. If his capital reaches zero dollars before reaching his target, then we say that he is “ruined” or *goes broke*, and he will have lost n dollars. We’ll assume that the gambler has the same probability, p , of winning each individual \$1 bet and that the bets are mutually independent. We’d like to find the probability that the gambler wins.

The gambler’s situation as he proceeds with his \$1 bets is illustrated in Figure 20.1. The random walk has boundaries at 0 and T . If the random walk ever reaches either of these boundary values, then it terminates.

In a *fair game*, the gambler is equally likely to win or lose each bet, that is $p = 1/2$. The corresponding random walk is called *unbiased*. The gambler is more likely to win if $p > 1/2$ and less likely to win if $p < 1/2$; these random walks are called *biased*. We want to determine the probability that the walk terminates at boundary T , namely, the probability that the gambler wins. We’ll do this in

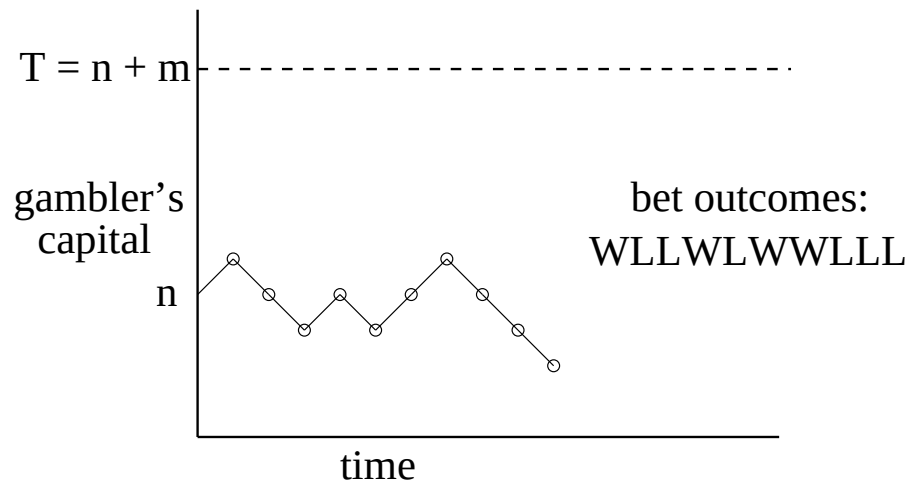


Figure 20.1 A graph of the gambler's capital versus time for one possible sequence of bet outcomes. At each time step, the graph goes up with probability p and down with probability $1 - p$. The gambler continues betting until the graph reaches either 0 or T . If he starts with $\$n$, his intended profit is $\$m$ where $T = n + m$.

Section 20.1.1, but before we derive the probability, let's just look at what it turns out to be.

Let's begin by supposing the coin is fair, the gambler starts with 100 dollars, and he wants to double his money. That is, he plays until he goes broke or reaches a target of 200 dollars. Since he starts equidistant from his target and bankruptcy, it's clear by symmetry that his probability of winning in this case is $1/2$.

We'll show below that starting with n dollars and aiming for a target of $T \geq n$ dollars, the probability the gambler reaches his target before going broke is n/T . For example, suppose he wants to win the same $\$100$, but instead starts out with $\$500$. Now his chances are pretty good: the probability of his making the 100 dollars is $5/6$. And if he started with one million dollars still aiming to win $\$100$ dollars he almost certain to win: the probability is $1M/(1M + 100) > .9999$.

So in the fair game, the larger the initial stake relative to the target, the higher the probability the gambler will win, which makes some intuitive sense. But note that although the gambler now wins nearly all the time, the game is still fair. When he wins, he only wins $\$100$; when he loses, he loses big: $\$1M$. So the gambler's average win is actually zero dollars.

Another way to describe this scenario is as a game between two players. Say Albert starts with $\$500$, and Eric starts with $\$100$. They flip a fair coin, and every

time a Head appears, Albert wins \$1 from Eric, and vice versa for Tails. They play this game until one person goes bankrupt. This problem is identical to the Gambler's Ruin problem with $n = 500$ and $T = 100 + 500 = 600$. So the probability of Albert winning is $500/600 = 5/6$.

Now suppose instead that the gambler chooses to play roulette in an American casino, always betting \$1 on red. This game is slightly biased against the gambler: the probability of winning a single bet is $p = 18/38 \approx 0.47$. (It's the two green numbers that slightly bias the bets and give the casino an edge.) Still, the bets are almost fair, and you might expect that starting with \$500, the gambler has a reasonable chance of winning \$100—the $5/6$ probability of winning in the unbiased game surely gets reduced, but perhaps not too drastically.

Not so! The gambler's odds of winning \$100 making one dollar bets against the “slightly” unfair roulette wheel are less than 1 in 37,000. If that seems surprising, listen to this: *no matter how much money* the gambler has to start —\$5000, \$50,000, $\$5 \cdot 10^{12}$ —his odds are still less than 1 in 37,000 of winning a mere 100 dollars!

Moral: Don't play!

The theory of random walks is filled with such fascinating and counter-intuitive conclusions.

20.1.1 The Probability of Avoiding Ruin

We will determine the probability that the gambler wins using an idea of Pascal's dating back to the beginnings of the subject of probability.

Pascal viewed the walk as a two-player game between Albert and Eric as described above. Albert starts with a stack of n chips and Eric starts with a stack of $m = T - n$ chips. At each bet, Albert wins Eric's top chip with probability p and loses his top chip to Eric with probability $q ::= 1 - p$. They play this game until one person goes bankrupt.

Pascal's ingenious idea was to alter the value of the chips to make the game fair. Namely, Albert's bottom chip will be given payoff value r where $r ::= q/p$, and the successive chips *up* his stack will be worth $r^2, r^3, \dots$ up to his top chip with payoff value r^n . Eric's top chip will be worth r^{n+1} and the successive chips *down* his stack will be worth $r^{n+2}, r^{n+3}, \dots$ down to his bottom chip worth r^{n+m} .

Now the expected change in Albert's chip values on the first bet is

$$r^{n+1} \cdot p - r^n \cdot q = \left(r^n \cdot \frac{q}{p} \right) \cdot p - r^n \cdot q = 0,$$

so this payoff makes the bet fair. Moreover, whether Albert wins or loses the bet, the successive chip values counting up Albert's stack and then down Eric's remain

$r, r^2, \dots, r^n, \dots, r^{n+m}$, ensuring by the same reasoning that every bet payoff remains fair. So Albert's expected payoff at the end of the game is the sum of the expectations of his payoffs of each bet, namely 0. Here we're legitimately appealing to infinite linearity, since the payoff amounts remain bounded independent of the number of bets.

When Albert wins all of Eric's chips his total payoff gain is $\sum_{i=n+1}^{n+m} r^i$, and when he loses all his chips to Eric, his total payoff loss is $\sum_{i=1}^n r^i$. Letting w_n be Albert's probability of winning, we now have

$$0 = \text{Ex}[\text{Albert's payoff}] = \left(\sum_{i=n+1}^{n+m} r^i \right) \cdot w_n - \left(\sum_{i=1}^n r^i \right) \cdot (1 - w_n).$$

In the truly fair game when $r = 1$, we have $0 = mw_n - n(1 - w_n)$, so $w_n = n/(n + m)$, as claimed above.

In the biased game with $r \neq 1$, we have

$$0 = r \cdot \frac{r^{n+m} - r^n}{r - 1} \cdot w_n - r \cdot \frac{r^n - 1}{r - 1} \cdot (1 - w_n).$$

Solving for w_n gives

$$w_n = \frac{r^n - 1}{r^{n+m} - 1} = \frac{r^n - 1}{r^T - 1} \quad (20.1)$$

We have now proved

Theorem 20.1.1. *In the Gambler's Ruin game with initial capital, n , target, T , and probability p of winning each individual bet,*

$$\Pr[\text{the gambler wins}] = \begin{cases} \frac{n}{T} & \text{for } p = \frac{1}{2}, \\ \frac{r^n - 1}{r^T - 1} & \text{for } p \neq \frac{1}{2}, \end{cases} \quad (20.2)$$

where $r ::= q/p$.

20.1.2 A Recurrence for the Probability of Winning

Pascal was obviously a clever fellow, but fortunately for the rest of us less ingenious folks, linear recurrences offer a methodical, if less inspiring, approach to Gambler's Ruin.

The probability that the gambler wins is a function of his initial capital, n , his target, $T \geq n$, and the probability, p , that he wins an individual one dollar bet.

For fixed p and T , let w_n be the gambler's probability of winning when his initial capital is n dollars. For example, w_0 is the probability that the gambler will win given that he starts off broke and w_T is the probability he will win if he starts off with his target amount, so clearly

$$w_0 = 0, \quad (20.3)$$

$$w_T = 1. \quad (20.4)$$

Otherwise, the gambler starts with n dollars, where $0 < n < T$. Now suppose the gambler wins his first bet. In this case, he is left with $n + 1$ dollars and becomes a winner with probability w_{n+1} . On the other hand, if he loses the first bet, he is left with $n - 1$ dollars and becomes a winner with probability w_{n-1} . By the Total Probability Rule, he wins with probability $w_n = pw_{n+1} + qw_{n-1}$. Solving for w_{n+1} we have

$$w_{n+1} = \frac{w_n}{p} - rw_{n-1} \quad (20.5)$$

where r is q/p as in section 20.1.1.

This recurrence holds only for $n + 1 \leq T$, but there's no harm in using (20.5) to define w_{n+1} for all $n + 1 > 1$. Now, letting

$$W(x) ::= w_0 + w_1x + w_2x^2 + \dots$$

be the generating function for the w_n , we derive from (20.5) and (20.3) using our generating function methods that

$$W(x) = \frac{w_1x}{rx^2 - x/p + 1}. \quad (20.6)$$

But it's easy to check that the denominator factors:

$$rx^2 - \frac{x}{p} + 1 = (1 - x)(1 - rx).$$

Now if $p \neq q$, then using partial fractions we conclude that

$$W(x) = \frac{A}{1 - x} + \frac{B}{1 - rx}, \quad (20.7)$$

for some constants A, B . To solve for A, B , note that by (20.6) and (20.7),

$$w_1x = A(1 - rx) + B(1 - x),$$

so letting $x = 1$, we get $A = w_1/(1 - r)$, and letting $x = 1/r$, we get $B = w_1/(r - 1)$. Therefore,

$$W(x) = \frac{w_1}{r - 1} \left(\frac{1}{1 - rx} - \frac{1}{1 - x} \right),$$

which implies

$$w_n = w_1 \frac{r^n - 1}{r - 1}. \quad (20.8)$$

Finally, we can use (20.8) to solve for w_1 by letting $n = T$ to get

$$w_1 = \frac{r - 1}{r^T - 1}.$$

Plugging this value of w_1 into (20.8), we arrive at the solution:

$$w_n = \frac{r^n - 1}{r^T - 1},$$

matching Pascal's result (20.1).

In the unbiased case where $p = q$, we get from (20.6) that

$$W(x) = \frac{w_1 x}{(1 - x)^2},$$

and again can use partial fractions to match Pascal's result (20.2).

A simpler expression for the biased case

The expression (20.1) for the probability that the Gambler wins in the biased game is a little hard to interpret. There is a simpler upper bound which is nearly tight when the gambler's starting capital is large and the game is biased *against* the gambler. Then $r > 1$, both the numerator and denominator in (20.1) are positive, and the numerator is smaller. This implies that

$$w_n < \frac{r^n}{r^T} = \left(\frac{1}{r}\right)^{T-n}$$

and gives:

Corollary 20.1.2. *In the Gambler's Ruin game with initial capital, n , target, T , and probability $p < 1/2$ of winning each individual bet,*

$$\Pr[\text{the gambler wins}] < \left(\frac{1}{r}\right)^{T-n} \quad (20.9)$$

where $r ::= q/p > 1$.

So the gambler gains his intended profit before going broke with probability at most $1/r$ raised to the intended profit power. Notice that this upper bound does not depend on the gambler's starting capital, but only on his intended profit. This

has the amazing consequence we announced above: *no matter how much money he starts with*, if he makes \$1 bets on red in roulette aiming to win \$100, the probability that he wins is less than

$$\left(\frac{18/38}{20/38}\right)^{100} = \left(\frac{9}{10}\right)^{100} < \frac{1}{37,648}.$$

The bound (20.9) decreases exponentially with the intended profit. So, for example, doubling his intended profit will square his probability of winning. In particular, the probability that the gambler's stake goes up 200 dollars before he goes broke playing roulette is at most

$$(9/10)^{200} = ((9/10)^{100})^2 < \left(\frac{1}{37,648}\right)^2,$$

which is about 1 in 1.4 billion.

20.1.3 Intuition

Why is the gambler so unlikely to make money when the game is slightly biased against him? Intuitively, there are two forces at work. First, the gambler's capital has random upward and downward *swings* due to runs of good and bad luck. Second, the gambler's capital will have a steady, downward *drift*, because the negative bias means an average loss of a few cents on each \$1 bet. The situation is shown in Figure 20.2.

Our intuition is that if the gambler starts with, say, a billion dollars, then he is sure to play for a very long time, so at some point there should be a lucky, upward swing that puts him \$100 ahead. The problem is that his capital is steadily drifting downward. If the gambler does not have a lucky, upward swing early on, then he is doomed. After his capital drifts downward a few hundred dollars, he needs a huge upward swing to save himself. And such a huge swing is extremely improbable. As a rule of thumb, *drift dominates swings* in the long term.

We can quantify these drifts and swings. After k rounds for $k \leq \min(m, n)$, the number of wins by our player has a binomial distribution with parameters $p < 1/2$ and k . His expected win on any single bet is $p - q = 2p - 1$ dollars, so his expected capital is $n - k(1 - 2p)$. Now to be a winner, his actual number of wins must exceed the expected number by $m + k(1 - 2p)$. But we saw before that the binomial distribution has a standard deviation of only $\sqrt{kp(1 - p)}$. So for the gambler to win, he needs his number of wins to deviate by

$$\frac{m + k(1 - 2p)}{\sqrt{kp(1 - p)}} = \Theta(\sqrt{k})$$

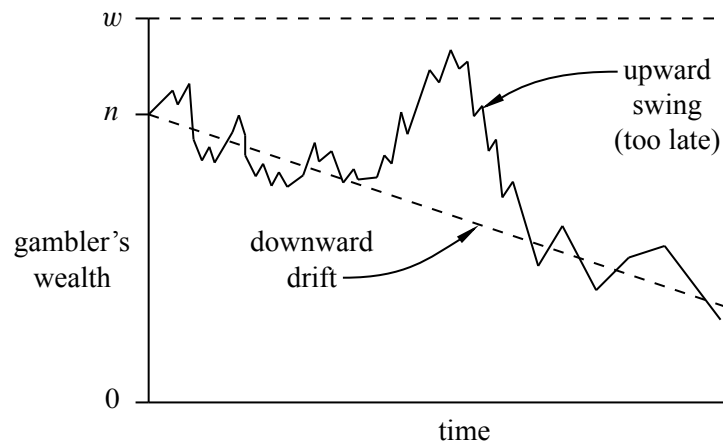


Figure 20.2 In a biased random walk, the downward drift usually dominates swings of good luck.

times its standard deviation. In our study of binomial tails, we saw that this was extremely unlikely.

In a fair game, there is no drift; swings are the only effect. In the absence of downward drift, our earlier intuition is correct. If the gambler starts with a trillion dollars then almost certainly there will eventually be a lucky swing that puts him \$100 ahead.

20.1.4 How Long a Walk?

Now that we know the probability, w_n , that the gambler is a winner in both fair and unfair games, we consider how many bets he needs on average to either win or go broke. A linear recurrence approach works here as well.

For fixed p and T , let e_n be the expected number of bets until the game ends when the gambler's initial capital is n dollars. Since the game is over in zero steps if $n = 0$ or T , the boundary conditions this time are $e_0 = e_T = 0$.

Otherwise, the gambler starts with n dollars, where $0 < n < T$. Now by the conditional expectation rule, the expected number of steps can be broken down into the expected number of steps given the outcome of the first bet weighted by the probability of that outcome. But after the gambler wins the first bet, his capital is $n + 1$, so he can expect to make another e_{n+1} bets. That is,

$$\text{Ex}[e_n \mid \text{gambler wins first bet}] = 1 + e_{n+1}.$$

Similarly, after the gambler loses his first bet, he can expect to make another e_{n-1}

bets:

$$\text{Ex}[e_n \mid \text{gambler loses first bet}] = 1 + e_{n-1}.$$

So we have

$$\begin{aligned} e_n &= p \text{Ex}[e_n \mid \text{gambler wins first bet}] + q \text{Ex}[e_n \mid \text{gambler loses first bet}] \\ &= p(1 + e_{n+1}) + q(1 + e_{n-1}) = pe_{n+1} + qe_{n-1} + 1. \end{aligned}$$

This yields the linear recurrence

$$e_{n+1} = \frac{1}{p}e_n - \frac{q}{p}e_{n-1} - \frac{1}{p}. \quad (20.10)$$

The routine solution of this linear recurrence yields:

Theorem 20.1.3. *In the Gambler's Ruin game with initial capital n , target T , and probability p of winning each bet,*

$$\text{Ex}[\text{number of bets}] = \begin{cases} n(T - n) & \text{for } p = \frac{1}{2}, \\ \frac{\frac{r^n - 1}{r^T - 1} \cdot T - n}{p - q} & \text{for } p \neq \frac{1}{2}. \end{cases} \quad (20.11)$$

In the unbiased case, (20.11) can be rephrased simply as

$$\text{Ex}[\text{number of fair bets}] = \text{initial capital} \cdot \text{intended profit}. \quad (20.12)$$

For example, if the gambler starts with \$10 dollars and plays until he is broke or ahead \$10, then $10 \cdot 10 = 100$ bets are required on average. If he starts with \$500 and plays until he is broke or ahead \$100, then the expected number of bets until the game is over is $500 \times 100 = 50,000$. This simple formula (20.12) cries out for an intuitive proof, but we have not found one (where are you, Pascal?).

20.1.5 Quit While You Are Ahead

Suppose that the gambler never quits while he is ahead. That is, he starts with $n > 0$ dollars, ignores any target T , but plays until he is flat broke. Call this the *unbounded Gambler's ruin* game. It turns out that if the game is not favorable, that is, $p \leq 1/2$, the gambler is sure to go broke. In particular, even in a “fair” game with $p = 1/2$, he is sure to go broke.

Lemma 20.1.4. *If the gambler starts with one or more dollars and plays a fair unbounded game, then he will go broke with probability 1.*

Proof. If the gambler has initial capital n and goes broke in a game without reaching a target T , then he would also go broke if he were playing and ignored the target. So the probability that he will lose if he keeps playing without stopping at any target T must be at least as large as the probability that he loses when he has a target $T > n$.

But we know that in a fair game, the probability that he loses is $1 - n/T$. This number can be made arbitrarily close to 1 by choosing a sufficiently large value of T . Hence, the probability of his losing while playing without any target has a lower bound arbitrarily close to 1, which means it must in fact be 1. ■

So even if the gambler starts with a million dollars and plays a perfectly fair game, he will eventually lose it all with probability 1. But there is good news: if the game is fair, he can “expect” to play forever:

Lemma 20.1.5. *If the gambler starts with one or more dollars and plays a fair unbounded game, then his expected number of plays is infinite.*

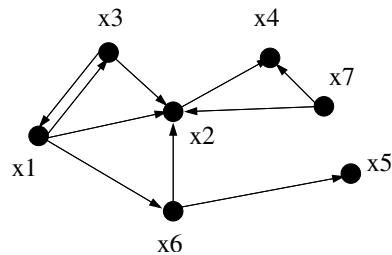
A proof appears in Problem 20.3.

So even starting with just one dollar, the expected number of plays before going broke is infinite! Of course, this does not mean that the gambler is *likely* to play for long—there is even a 50% chance he will lose the very first bet and go broke right away.

Lemma 20.1.5 says that the gambler can “expect” to play forever, while Lemma 20.1.4 says that he is certain to go broke. These facts sound contradictory, but they are sound consequences of the technical mathematical definition of expectation. The moral here, as in section 19.8, is that naive intuition is unreliable when it comes to infinite expectation.

20.2 Random Walks on Graphs

The hyperlink structure of the World Wide Web can be described as a digraph. The vertices are the web pages with a directed edge from vertex x to vertex y if x has a link to y . For example, in the following graph the vertices $x_1, \dots, x_n$ correspond to web pages and $\langle x_i \rightarrow x_j \rangle$ is a directed edge when page x_i contains a hyperlink to page x_j .



The web graph is an enormous graph with many billions and probably even trillions of vertices. At first glance, this graph wouldn’t seem to be very interesting. But in 1995, two students at Stanford, Larry Page and Sergey Brin realized that the structure of this graph could be very useful in building a search engine. Traditional document searching programs had been around for a long time and they worked in a fairly straightforward way. Basically, you would enter some search terms and the searching program would return all documents containing those terms. A relevance score might also be returned for each document based on the frequency or position that the search terms appeared in the document. For example, if the search term appeared in the title or appeared 100 times in a document, that document would get a higher score. So if an author wanted a document to get a higher score for certain keywords, he would put the keywords in the title and make it appear in lots of places. You can even see this today with some bogus web sites.

This approach works fine if you only have a few documents that match a search term. But on the web, there are billions of documents and millions of matches to a typical search.

For example, a few years ago a search on Google for “math for computer science notes” gave 378,000 hits! How does Google decide which 10 or 20 to show first? It wouldn’t be smart to pick a page that gets a high keyword score because it has “math math . . . math” across the front of the document.

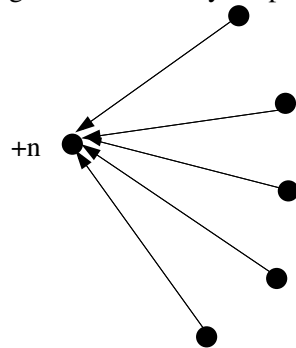
One way to get placed high on the list is to pay Google an advertising fees — and Google gets an enormous revenue stream from these fees. Of course an early listing is worth a fee only if an advertiser’s target audience is attracted to the listing. But an audience does get attracted to Google listings because its ranking method is really good at determining the most relevant web pages. For example, Google demonstrated its accuracy in our case by giving first rank to the Fall 2002 open courseware page for 6.042 : –) . So how did Google know to pick 6.042 to be first out of 378, 000?

Well back in 1995, Larry and Sergey got the idea to allow the digraph structure of the web to determine which pages are likely to be the most important.

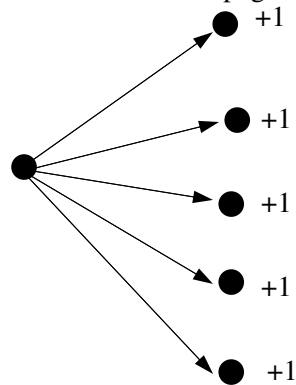
20.2.1 A First Crack at Page Rank

Looking at the web graph, any idea which vertex/page might be the best to rank 1st? Assume that all the pages match the search terms for now. Well, intuitively, we should choose x_2 , since lots of other pages point to it. This leads us to their first idea: try defining the *page rank* of x to be the number of links pointing to x , that is, $\text{indegree}(x)$. The idea is to think of web pages as voting for the most important page—the more votes, the better rank.

Of course, there are some problems with this idea. Suppose you wanted to have your page get a high ranking. One thing you could do is to create lots of dummy pages with links to your page.



There is another problem—a page could become unfairly influential by having lots of links to other pages it wanted to hype.



So this strategy for high ranking would amount to, “vote early, vote often,” which is no good if you want to build a search engine that’s worth paying fees for. So, admittedly, their original idea was not so great. It was better than nothing, but certainly not worth billions of dollars.

20.2.2 Random Walk on the Web Graph

But then Sergey and Larry thought some more and came up with a couple of improvements. Instead of just counting the indegree of a vertex, they considered the probability of being at each page after a long random walk on the web graph. In particular, they decided to model a user’s web experience as following each link on a page with uniform probability. That is, they assigned each edge $x \rightarrow y$ of the web graph with a probability conditioned on being on page x :

$$\Pr[\text{follow link } \langle x \rightarrow y \rangle \mid \text{at page } x] ::= \frac{1}{\text{outdegree}(x)}.$$

The user experience is then just a random walk on the web graph.

For example, if the user is at page x , and there are three links from page x , then each link is followed with probability $1/3$.

We can also compute the probability of arriving at a particular page, y , by summing over all edges pointing to y . We thus have

$$\begin{aligned} \Pr[\text{go to } y] &= \sum_{\text{edges } \langle x \rightarrow y \rangle} \Pr[\text{follow link } \langle x \rightarrow y \rangle \mid \text{at page } x] \cdot \Pr[\text{at page } x] \\ &= \sum_{\text{edges } \langle x \rightarrow y \rangle} \frac{\Pr[\text{at } x]}{\text{outdegree}(x)} \end{aligned} \quad (20.13)$$

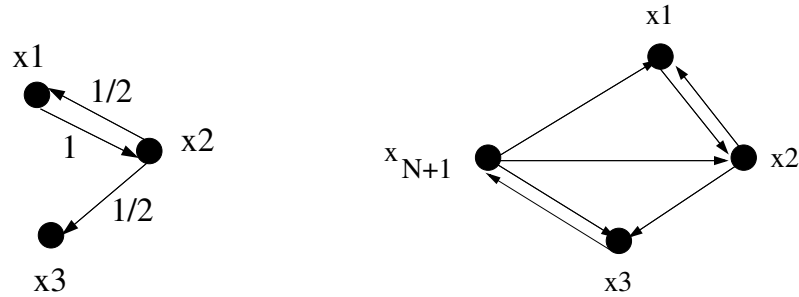
For example, in our web graph, we have

$$\Pr[\text{go to } x_4] = \frac{\Pr[\text{at } x_7]}{2} + \frac{\Pr[\text{at } x_2]}{1}.$$

One can think of this equation as x_7 sending half its probability to x_2 and the other half to x_4 . The page x_2 sends all of its probability to x_4 .

There’s one aspect of the web graph described thus far that doesn’t mesh with the user experience —some pages have no hyperlinks out. Under the current model, the user cannot escape these pages. In reality, however, the user doesn’t fall off the end of the web into a void of nothingness. Instead, he restarts his web journey.

To model this aspect of the web, Sergey and Larry added a supervertex to the web graph and had every page with no hyperlinks point to it. Moreover, the supervertex points to every other vertex in the graph, allowing you to restart the walk from a random place. For example, below left is a graph and below right is the same graph after adding the supervertex x_{N+1} .



The addition of the supervertex also removes the possibility that the value $1/\text{outdegree}(x)$ might involve a division by zero.

20.2.3 Stationary Distribution & Page Rank

The basic idea of page rank is just a stationary distribution over the web graph, so let's define a stationary distribution.

Suppose each vertex is assigned a probability that corresponds, intuitively, to the likelihood that a random walker is at that vertex at a randomly chosen time. We assume that the walk never leaves the vertices in the graph, so we require that

$$\sum_{\text{vertices } x} \Pr[\text{at } x] = 1. \quad (20.14)$$

Definition 20.2.1. An assignment of probabilities to vertices in a digraph is a *stationary distribution* if for all vertices x

$$\Pr[\text{at } x] = \Pr[\text{go to } x \text{ at next step}]$$

Sergey and Larry defined their page ranks to be a stationary distribution. They did this by solving the following system of linear equations: find a nonnegative number, $\text{PR}(x)$, for each vertex, x , such that

$$\text{PR}(x) = \sum_{\text{edges } \{y \rightarrow x\}} \frac{\text{PR}(y)}{\text{outdegree}(y)}, \quad (20.15)$$

corresponding to the intuitive equations given in (20.13). These numbers must also satisfy the additional constraint corresponding to (20.14):

$$\sum_{\text{vertices } x} \text{PR}(x) = 1. \quad (20.16)$$

So if there are n vertices, then equations (20.15) and (20.16) provide a system of $n + 1$ linear equations in the n variables, $\text{PR}(x)$. Note that constraint (20.16)

is needed because the remaining constraints (20.15) could be satisfied by letting $\text{PR}(x) ::= 0$ for all x , which is useless.

Sergey and Larry were smart fellows, and they set up their page rank algorithm so it would always have a meaningful solution. Their addition of a supervertex ensures there is always a *unique* stationary distribution. Moreover, starting from *any* vertex and taking a sufficiently long random walk on the graph, the probability of being at each page will get closer and closer to the stationary distribution. Note that general digraphs without supervertices may have neither of these properties: there may not be a unique stationary distribution, and even when there is, there may be starting points from which the probabilities of positions during a random walk do not converge to the stationary distribution. Examples of this appear in some problems below.

Now just keeping track of the digraph whose vertices are billions of web pages is a daunting task. That’s why Google is building power plants. Indeed, Larry and Sergey named their system Google after the number 10^{100} —which called a “googol” —to reflect the fact that the web graph is so enormous.

Anyway, now you can see how 6.042 ranked first out of 378,000 matches. Lots of other universities used our notes and presumably have links to the 6.042 open courseware site, and the university sites themselves are legitimate, which ultimately leads to 6.042 getting a high page rank in the web graph.

Problems for Section 20.1

Practice Problems

Problem 20.1. (a) In the unbiased Gambler’s Ruin game, if the Gambler’s initial capital is \$40, what is the probability that his capital at some point grows to be \$100?

(b) What is the expected total number of bets in the game (how many bets will the gambler make before he has lost everything)?

Problem 20.2.

Suppose that a gambler is playing a game in which he makes a series of \$1 bets. He wins each one with probability 0.49, and he keeps betting until he either runs out of money or reaches some fixed goal of T dollars.

Let $t(n)$ be the expected number of *bets* the gambler makes until the game ends, where n is the number of dollars the gambler has when he starts betting. Then the function t satisfies a linear recurrence of the form

$$t(n) = a \cdot t(n + 1) + b \cdot t(n - 1) + c$$

for real constants a, b, c and $0 < n < T$.

- (a) What are the values of a, b and c ?
- (b) What is $t(0)$?
- (c) What is $t(T)$?

Class Problems

Problem 20.3.

In gambler’s ruin scenario, the gambler makes independent \$1 bets, where the probability of winning a bet p and of losing is $q ::= 1 - p$. The gambler keeps betting until he goes broke or reaches a target of T dollars.

Suppose $T = \infty$, that is, the gambler keeps playing until he goes broke. Let r be the probability that starting with $n > 0$ dollars, the gambler’s stake ever gets reduced to $n - 1$ dollars.

- (a) Explain why

$$r = q + pr^2.$$

- (b) Conclude that if $p \leq 1/2$, then $r = 1$.

(c) Prove that even in a fair game, the gambler is sure to get ruined *no matter how much money he starts with!*

(d) Let t be the expected time for the gambler’s stake to go down by 1 dollar. Verify that

$$t = q + p(1 + 2t).$$

Conclude that starting with a 1 dollar stake in a fair game, the gambler can expect to play forever!

Problem 20.4.

A gambler is placing \$1 bets on the “1st dozen” in roulette. This bet wins when a number from one to twelve comes in, and then the gambler gets his \$1 back plus \$2 more. Recall that there are 38 numbers on the roulette wheel.

The gambler’s initial stake is n and his target is T . He will keep betting until he runs out of money (“goes broke”) or reaches his target. Let w_n be the probability of the gambler winning, that is, reaching target T before going broke.

- (a) Write a linear recurrence for w_n ; you need *not* solve the recurrence.

(b) Let e_n be the expected number of bets until the game ends. Write a linear recurrence for $\text{Ex}[e_n]$; you need *not* solve the recurrence.

Problem 20.5.

In the fair Gambler’s Ruin game with initial stake of n dollars and target of T dollars, let e_n be the number of \$1 bets the gambler makes until the game ends (because he reaches his target or goes broke).

(a) Describe constants a, b, c such that

$$e_n = ae_{n-1} + be_{n-2} + c. \quad (20.17)$$

for $1 < n < T$.

(b) Let e_n be defined by (20.17) for all $n > 1$, where $e_0 = 0$ and $e_1 = d$ for some constant d . Derive a closed form (involving d) for the generating function $E(x) ::= \sum_0^\infty e_n x^n$.

(c) Find a closed form (involving d) for e_n .

(d) Use part (c) to solve for d .

(e) Prove that $e_n = n(T - n)$.

Problems for Section 20.2

Practice Problems

Problem 20.6.

Consider the following random-walk graphs:

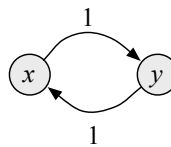


Figure 20.3

(a) Find $d(x)$ for a stationary distribution for graph 20.3.

(b) Find $d(y)$ for a stationary distribution for graph 20.3.

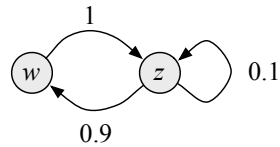


Figure 20.4

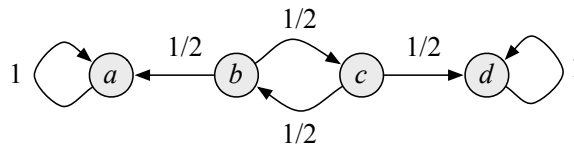


Figure 20.5

- (c) If you start at node x in graph 20.3 and take a (long) random walk, does the distribution over nodes ever get close to the stationary distribution?
- (d) Find $d(w)$ for a stationary distribution for graph 20.4.
- (e) Find $d(z)$ for a stationary distribution for graph 20.4.
- (f) If you start at node w in graph 20.4 and take a (long) random walk, does the distribution over nodes ever get close to the stationary distribution? (*Hint*: try a few steps and watch what is happening.)
- (g) How many stationary distributions are there for graph 20.5?
- (h) If you start at node b in graph 20.5 and take a (long) random walk, what will be the approximate probability that you are at node d ?

Class Problems

Problem 20.7. (a) Find a stationary distribution for the random walk graph in Figure 20.6.

(b) If you start at node x in Figure 20.6 and take a (long) random walk, does the distribution over nodes ever get close to the stationary distribution? Explain.

(c) Find a stationary distribution for the random walk graph in Figure 20.7.

(d) If you start at node w Figure 20.7 and take a (long) random walk, does the distribution over nodes ever get close to the stationary distribution? You needn't

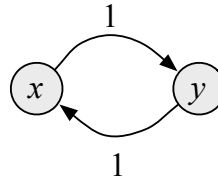


Figure 20.6

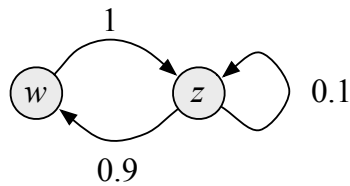


Figure 20.7

prove anything here, just write out a few steps and see what’s happening.

- (e) Find a stationary distribution for the random walk graph in Figure 20.8.

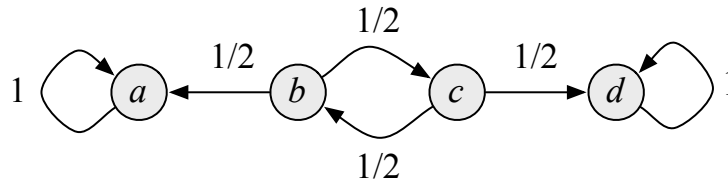


Figure 20.8

- (f) If you start at node b in Figure 20.8 and take a long random walk, the probability you are at node d will be close to what fraction? Explain.

Problem 20.8.

We use random walks on a digraph, G , to model the typical movement pattern of a Math for CS student right after the final exam.

The student comes out of the final exam located on a particular node of the graph, corresponding to the exam room. What happens next is unpredictable, as the student is in a total haze. At each step of the walk, if the student is at node u at the end of the previous step, they pick one of the edges $\langle u \rightarrow v \rangle$ uniformly at

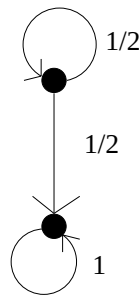
random from the set of all edges directed out of u , and then walk to the node v .

Let $n ::= |V(G)|$ and define the vector $P^{(j)}$ to be

$$P^{(j)} ::= (p_1^{(j)}, \dots, p_n^{(j)})$$

where $p_i^{(j)}$ is the probability of being at node i after j steps.

(a) We will start by looking at a simple graph. If the student starts at node 1 (the top node) in the following graph, what is $P^{(0)}$, $P^{(1)}$, $P^{(2)}$? Give a nice expression for $P^{(n)}$.



(b) Given an arbitrary graph, show how to write an expression for $p_i^{(j)}$ in terms of the $p_k^{(j-1)}$'s.

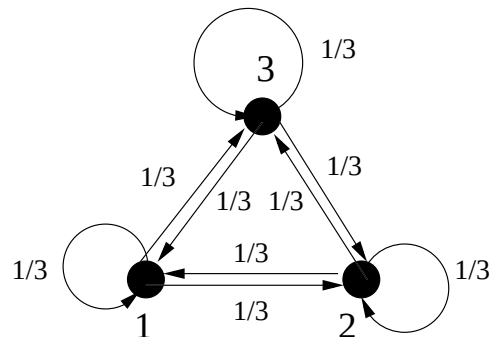
(c) Does your answer to the last part look like any other system of equations you've seen in this course?

(d) Let the *limiting distribution* vector, π , be

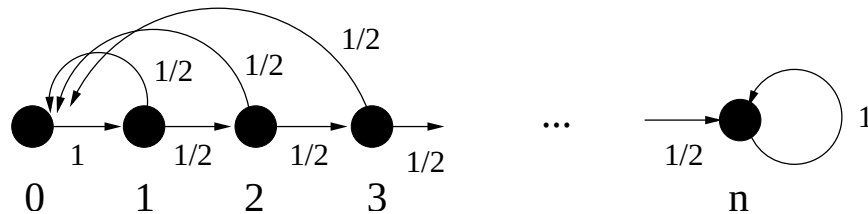
$$\lim_{k \rightarrow \infty} \frac{\sum_{i=1}^k P^{(i)}}{k}.$$

What is the limiting distribution of the graph from part a? Would it change if the start distribution were $P^{(0)} = (1/2, 1/2)$ or $P^{(0)} = (1/3, 2/3)$?

(e) Let's consider another directed graph. If the student starts at node 1 with probability 1/2 and node 2 with probability 1/2, what is $P^{(0)}$, $P^{(1)}$, $P^{(2)}$ in the following graph? What is the limiting distribution?



(f) Now we are ready for the real problem. In order to make it home, the poor Math for student is faced with n doors along a long hall way. Unbeknownst to him, the door that goes outside to paradise (that is, freedom from the class and more importantly, vacation!) is at the *very end*. At each step along the way, he passes by a door which he opens up and goes through with probability $1/2$. Every time he does this, he gets teleported back to the exam room. Let's figure out how long it will take the poor guy to escape from the class. What is $P^{(0)}$, $P^{(1)}$, $P^{(2)}$? What is the limiting distribution?



(g) Show that the expected number, $T(n)$, of teleportations you make back to the exam room before you escape to the outside world is $2^{n-1} - 1$.

Problem 20.9.

A Google-graph is a random-walk graph such that every edge leaving any given vertex has the same probability. That is, the probability of each edge $\langle v \rightarrow w \rangle$ is $1/\text{out-degree}(v)$.

A directed graph is *symmetric* if, whenever $\langle v \rightarrow w \rangle$ is an edge, so is $\langle w \rightarrow v \rangle$.

Given any finite, symmetric Google-graph, let

$$d(v) ::= \frac{\text{out-degree}(v)}{e},$$

where e is the total number of edges in the graph. Show that d is a stationary distribution.

Homework Problems

Problem 20.10.

A digraph is *strongly connected* iff there is a directed path between every pair of distinct vertices. In this problem we consider a finite random walk graph that is strongly connected.

(a) Let d_1 and d_2 be distinct distributions for the graph, and define the *maximum dilation*, γ , of d_1 over d_2 to be

$$\gamma ::= \max_{x \in V} \frac{d_1(x)}{d_2(x)}.$$

Call a vertex, x , *dilated* if $d_1(x)/d_2(x) = \gamma$. Show that there is an edge, $\langle y \rightarrow z \rangle$, from an undilated vertex y to a dilated vertex, z . *Hint:* Choose any dilated vertex, x , and consider the set, D , of dilated vertices connected to x by a directed path (going to x) that only uses dilated vertices. Explain why $D \neq V$, and then use the fact that the graph is strongly connected.

(b) Prove that the graph has *at most one* stationary distribution. (There always *is* a stationary distribution, but we’re not asking you prove this.) *Hint:* Let d_1 be a stationary distribution and d_2 be a different distribution. Let z be the vertex from part (a). Show that starting from d_2 , the probability of z changes at the next step. That is, $\hat{d}_2(z) \neq d_2(z)$.

Exam Problems

Problem 20.11.

For which of the graphs in Figure 20.9 is the uniform distribution over nodes a stationary distribution? The edges are labeled with transition probabilities. Explain your reasoning.

Problem 20.12.

(a) Circle all the properties below that are preserved under graph isomorphism.

- There is a cycle that includes all the vertices.
- Two edges are of equal length.
- The graph remains connected if any two edges are removed.
- There exists an edge that is an edge of every spanning tree.

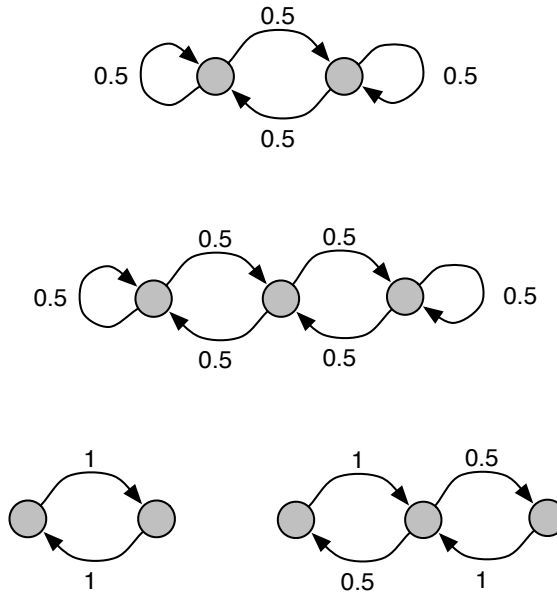


Figure 20.9 Which ones have uniform stationary distribution?

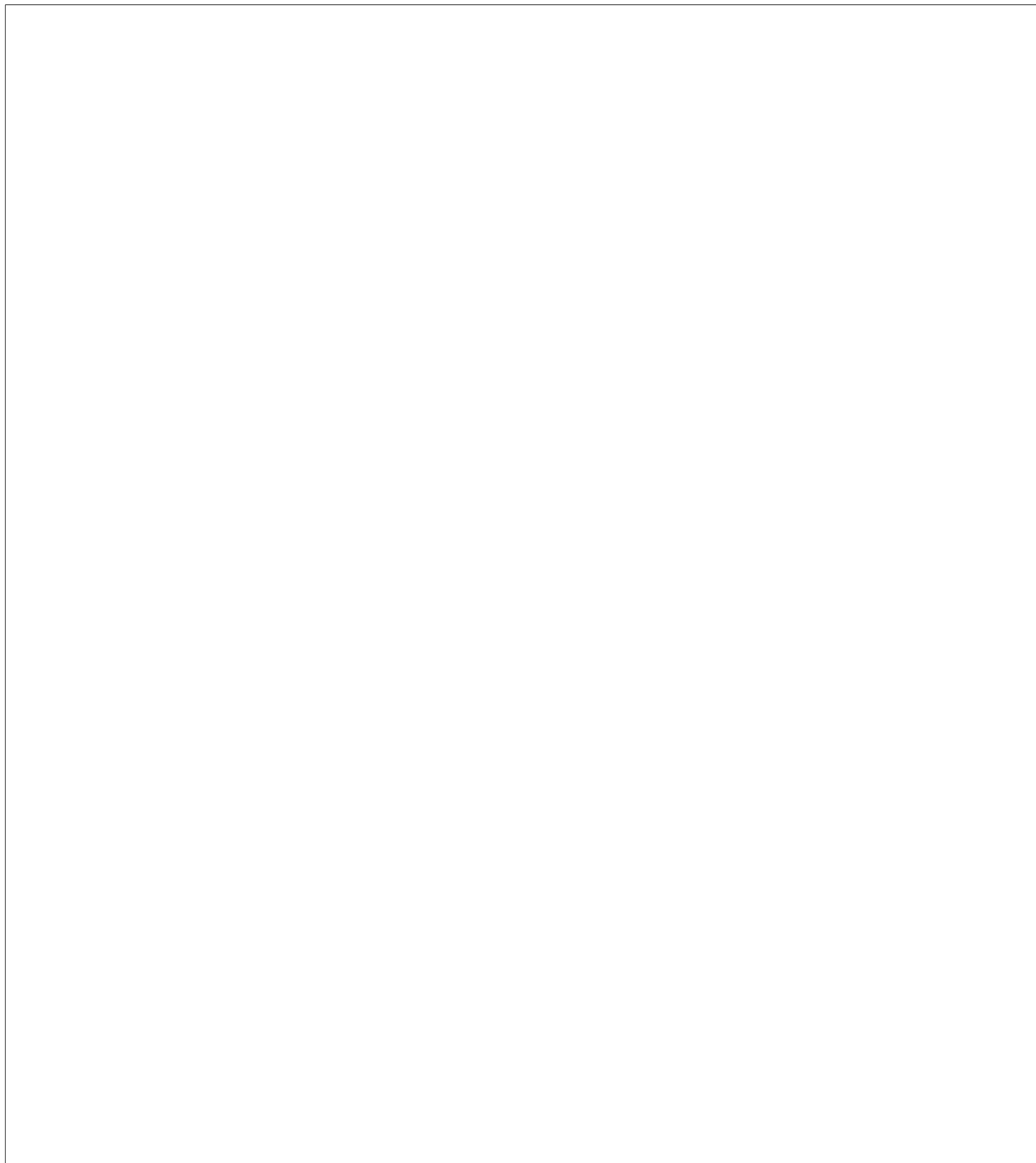
- The negation of a property that is preserved under isomorphism.

(b) For the following statements about **finite trees**, circle **true** or **false**, and *provide counterexamples* for those that are **false**.

- Any connected subgraph is a tree. **true** **false**
- Adding an edge between two nonadjacent vertices creates a cycle. **true**
false
- The number of vertices is one less than twice the number of leaves. **true**
false
- The number of vertices is one less than the number of edges. **true** **false**
- For every finite graph (not necessarily a tree), there is one (a finite tree) that spans it. **true** **false**

(c) What is the minimum number of **vertices** possible in a nonplanar graph?

(d) What is the minimum number of **edges** possible in a nonplanar graph that is 2-colorable?



V Recurrences



Introduction

A *recurrence* describes a sequence of numbers. Early terms are specified explicitly, and later terms are expressed as a function of their predecessors. As a trivial example, here is a recurrence describing the sequence 1, 2, 3, . . . :

$$\begin{aligned} T_1 &= 1 \\ T_n &= T_{n-1} + 1 \qquad \text{(for } n \geq 2\text{).} \end{aligned}$$

Here, the first term is defined to be 1 and each subsequent term is one more than its predecessor.

Recurrences turn out to be a powerful tool. In this chapter, we’ll emphasize using recurrences to analyze the performance of recursive algorithms. However, recurrences have other applications in computer science as well, such as enumeration of structures and analysis of random processes. And, as we saw in Section 14.4, they also arise in the analysis of problems in the physical sciences.

A recurrence in isolation is not a very useful description of a sequence. One can not easily answer simple questions such as, “What is the hundredth term?” or “What is the asymptotic growth rate?” So one typically wants to *solve* a recurrence; that is, to find a closed-form expression for the n th term.

We’ll first introduce two general solving techniques: *guess-and-verify* and *plug-and-chug*. These methods are applicable to every recurrence, but their success requires a flash of insight —sometimes an unrealistically brilliant flash. So we’ll also introduce two big classes of recurrences, linear and divide-and-conquer, that often come up in computer science. Essentially all recurrences in these two classes are solvable using cookbook techniques; you follow the recipe and get the answer. A drawback is that calculation replaces insight. The “Aha!” moment that is essential in the guess-and-verify and plug-and-chug methods is replaced by a “Huh” at the end of a cookbook procedure.

At the end of the chapter, we’ll develop rules of thumb to help you assess many recurrences without any calculation. These rules can help you distinguish promising approaches from bad ideas early in the process of designing an algorithm.

Recurrences are one aspect of a broad theme in computer science: reducing a big problem to progressively smaller problems until easy base cases are reached. This same idea underlies both induction proofs and recursive algorithms. As we’ll see, all three ideas snap together nicely. For example, the running time of a recursive algorithm could be described with a recurrence with induction used to verify the solution.

21 Recurrences

21.1 The Towers of Hanoi

According to legend, there is a temple in Hanoi with three posts and 64 gold disks of different sizes. Each disk has a hole through the center so that it fits on a post. In the misty past, all the disks were on the first post, with the largest on the bottom and the smallest on top, as shown in Figure 21.1.

Monks in the temple have labored through the years since to move all the disks to one of the other two posts according to the following rules:

- The only permitted action is removing the top disk from one post and dropping it onto another post.
- A larger disk can never lie above a smaller disk on any post.

So, for example, picking up the whole stack of disks at once and dropping them on another post is illegal. That’s good, because the legend says that when the monks complete the puzzle, the world will end!

To clarify the problem, suppose there were only 3 gold disks instead of 64. Then the puzzle could be solved in 7 steps as shown in Figure 21.2.

The questions we must answer are, “Given sufficient time, can the monks succeed?” If so, “How long until the world ends?” And, most importantly, “Will this happen before the final exam?”

21.1.1 A Recursive Solution

The Towers of Hanoi problem can be solved recursively. As we describe the procedure, we’ll also analyze the running time. To that end, let T_n be the minimum

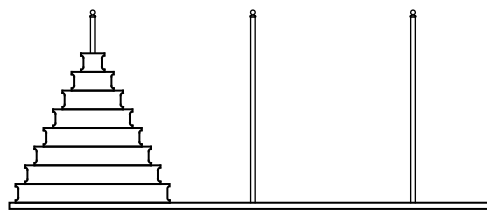


Figure 21.1 The initial configuration of the disks in the Towers of Hanoi problem.

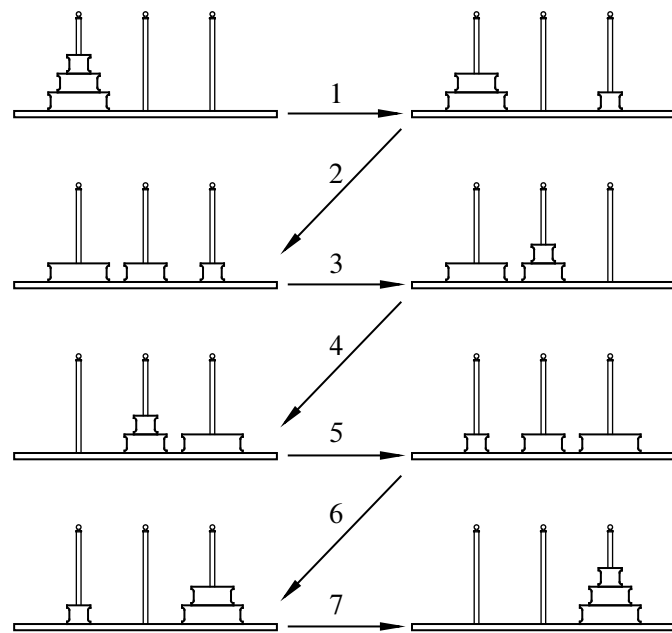


Figure 21.2 The 7-step solution to the Towers of Hanoi problem when there are $n = 3$ disks.

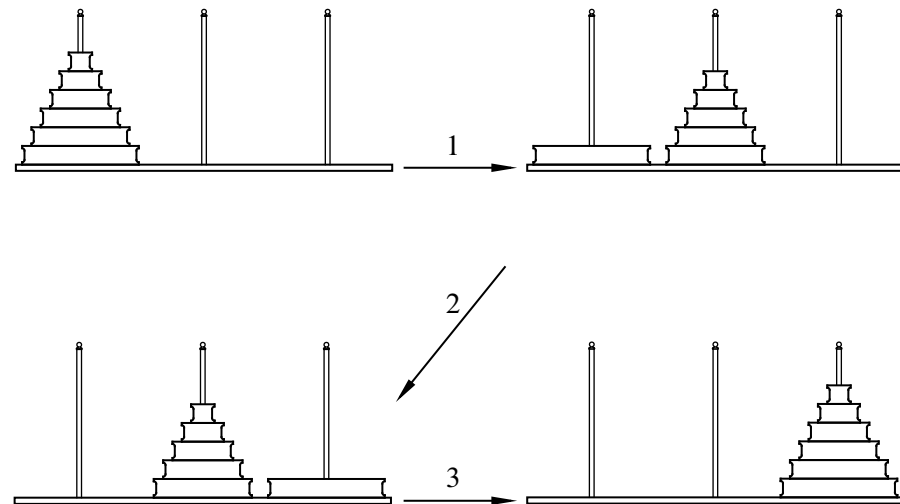


Figure 21.3 A recursive solution to the Towers of Hanoi problem.

number of steps required to solve the n -disk problem. For example, some experimentation shows that $T_1 = 1$ and $T_2 = 3$. The procedure illustrated above shows that T_3 is at most 7, though there might be a solution with fewer steps.

The recursive solution has three stages, which are described below and illustrated in Figure 21.3. For clarity, the largest disk is shaded in the figures.

Stage 1. Move the top $n - 1$ disks from the first post to the second using the solution for $n - 1$ disks. This can be done in T_{n-1} steps.

Stage 2. Move the largest disk from the first post to the third post. This takes just 1 step.

Stage 3. Move the $n - 1$ disks from the second post to the third post, again using the solution for $n - 1$ disks. This can also be done in T_{n-1} steps.

This algorithm shows that T_n , the minimum number of steps required to move n disks to a different post, is at most $T_{n-1} + 1 + T_{n-1} = 2T_{n-1} + 1$. We can use this fact to upper bound the number of operations required to move towers of various heights:

$$T_3 \leq 2 \cdot T_2 + 1 = 7$$

$$T_4 \leq 2 \cdot T_3 + 1 \leq 15$$

Continuing in this way, we could eventually compute an upper bound on T_{64} , the number of steps required to move 64 disks. So this algorithm answers our first

question: given sufficient time, the monks can finish their task and end the world. This is a shame. After all that effort, they’d probably want to smack a few high-fives and go out for burgers and ice cream, but nope —world’s over.

21.1.2 Finding a Recurrence

We can not yet compute the exact number of steps that the monks need to move the 64 disks, only an upper bound. Perhaps, having pondered the problem since the beginning of time, the monks have devised a better algorithm.

In fact, there is no better algorithm, and here is why. At some step, the monks must move the largest disk from the first post to a different post. For this to happen, the $n - 1$ smaller disks must all be stacked out of the way on the only remaining post. Arranging the $n - 1$ smaller disks this way requires at least T_{n-1} moves. After the largest disk is moved, at least another T_{n-1} moves are required to pile the $n - 1$ smaller disks on top.

This argument shows that the number of steps required is at least $2T_{n-1} + 1$. Since we gave an algorithm using exactly that number of steps, we can now write an expression for T_n , the number of moves required to complete the Towers of Hanoi problem with n disks:

$$\begin{aligned} T_1 &= 1 \\ T_n &= 2T_{n-1} + 1 \quad (\text{for } n \geq 2). \end{aligned}$$

This is a typical recurrence. These two lines define a sequence of values, $T_1, T_2, T_3, \dots$. The first line says that the first number in the sequence, T_1 , is equal to 1. The second line defines every other number in the sequence in terms of its predecessor. So we can use this recurrence to compute any number of terms in the sequence:

$$\begin{aligned} T_1 &= 1 \\ T_2 &= 2 \cdot T_1 + 1 = 3 \\ T_3 &= 2 \cdot T_2 + 1 = 7 \\ T_4 &= 2 \cdot T_3 + 1 = 15 \\ T_5 &= 2 \cdot T_4 + 1 = 31 \\ T_6 &= 2 \cdot T_5 + 1 = 63. \end{aligned}$$

21.1.3 Solving the Recurrence

We could determine the number of steps to move a 64-disk tower by computing T_7 , T_8 , and so on up to T_{64} . But that would take a lot of work. It would be nice to have a closed-form expression for T_n , so that we could quickly find the number of steps required for any given number of disks. (For example, we might want to know how

much sooner the world would end if the monks melted down one disk to purchase burgers and ice cream *before* the end of the world.)

There are several methods for solving recurrence equations. The simplest is to *guess* the solution and then *verify* that the guess is correct with an induction proof. As a basis for a good guess, let’s look for a pattern in the values of T_n computed above: 1, 3, 7, 15, 31, 63. A natural guess is $T_n = 2^n - 1$. But whenever you guess a solution to a recurrence, you should always verify it with a proof, typically by induction. After all, your guess might be wrong. (But why bother to verify in this case? After all, if we’re wrong, it’s not the end of the... no, let’s check.)

Claim 21.1.1. $T_n = 2^n - 1$ satisfies the recurrence:

$$\begin{aligned} T_1 &= 1 \\ T_n &= 2T_{n-1} + 1 \end{aligned} \quad (\text{for } n \geq 2).$$

Proof. The proof is by induction on n . The induction hypothesis is that $T_n = 2^n - 1$. This is true for $n = 1$ because $T_1 = 1 = 2^1 - 1$. Now assume that $T_{n-1} = 2^{n-1} - 1$ in order to prove that $T_n = 2^n - 1$, where $n \geq 2$:

$$\begin{aligned} T_n &= 2T_{n-1} + 1 \\ &= 2(2^{n-1} - 1) + 1 \\ &= 2^n - 1. \end{aligned}$$

The first equality is the recurrence equation, the second follows from the induction assumption, and the last step is simplification. ■

Such verification proofs are especially tidy because recurrence equations and induction proofs have analogous structures. In particular, the base case relies on the first line of the recurrence, which defines T_1 . And the inductive step uses the second line of the recurrence, which defines T_n as a function of preceding terms.

Our guess is verified. So we can now resolve our remaining questions about the 64-disk puzzle. Since $T_{64} = 2^{64} - 1$, the monks must complete more than 18 billion billion steps before the world ends. Better study for the final.

21.1.4 The Upper Bound Trap

When the solution to a recurrence is complicated, one might try to prove that some simpler expression is an upper bound on the solution. For example, the exact solution to the Towers of Hanoi recurrence is $T_n = 2^n - 1$. Let’s try to prove the “nicer” upper bound $T_n \leq 2^n$, proceeding exactly as before.

Proof. (Failed attempt.) The proof is by induction on n . The induction hypothesis is that $T_n \leq 2^n$. This is true for $n = 1$ because $T_1 = 1 \leq 2^1$. Now assume that $T_{n-1} \leq 2^{n-1}$ in order to prove that $T_n \leq 2^n$, where $n \geq 2$:

$$\begin{aligned} T_n &= 2T_{n-1} + 1 \\ &\leq 2(2^{n-1}) + 1 \\ &\not\leq 2^n \quad \leftarrow \text{Uh-oh!} \end{aligned}$$

The first equality is the recurrence relation, the second follows from the induction hypothesis, and the third step is a flaming train wreck. ■

The proof doesn’t work! As is so often the case with induction proofs, the argument only goes through with a *stronger* hypothesis. This isn’t to say that upper bounding the solution to a recurrence is hopeless, but this is a situation where induction and recurrences do not mix well.

21.1.5 Plug and Chug

Guess-and-verify is a simple and general way to solve recurrence equations. But there is one big drawback: you have to *guess right*. That was not hard for the Towers of Hanoi example. But sometimes the solution to a recurrence has a strange form that is quite difficult to guess. Practice helps, of course, but so can some other methods.

Plug-and-chug is another way to solve recurrences. This is also sometimes called “expansion” or “iteration.” As in guess-and-verify, the key step is identifying a pattern. But instead of looking at a sequence of *numbers*, you have to spot a pattern in a sequence of *expressions*, which is sometimes easier. The method consists of three steps, which are described below and illustrated with the Towers of Hanoi example.

Step 1: Plug and Chug Until a Pattern Appears

The first step is to expand the recurrence equation by alternately “plugging” (applying the recurrence) and “chugging” (simplifying the result) until a pattern appears. Be careful: too much simplification can make a pattern harder to spot. The rule to remember—indeed, a rule applicable to the whole of college life—is *chug in*

moderation.

$$\begin{aligned}
 T_n &= 2T_{n-1} + 1 \\
 &= 2(2T_{n-2} + 1) + 1 && \text{plug} \\
 &= 4T_{n-2} + 2 + 1 && \text{chug} \\
 &= 4(2T_{n-3} + 1) + 2 + 1 && \text{plug} \\
 &= 8T_{n-3} + 4 + 2 + 1 && \text{chug} \\
 &= 8(2T_{n-4} + 1) + 4 + 2 + 1 && \text{plug} \\
 &= 16T_{n-4} + 8 + 4 + 2 + 1 && \text{chug}
 \end{aligned}$$

Above, we started with the recurrence equation. Then we replaced T_{n-1} with $2T_{n-2} + 1$, since the recurrence says the two are equivalent. In the third step, we simplified a little—but not too much! After several similar rounds of plugging and chugging, a pattern is apparent. The following formula seems to hold:

$$\begin{aligned}
 T_n &= 2^k T_{n-k} + 2^{k-1} + 2^{k-2} + \cdots + 2^2 + 2^1 + 2^0 \\
 &= 2^k T_{n-k} + 2^k - 1
 \end{aligned}$$

Once the pattern is clear, simplifying is safe and convenient. In particular, we’ve collapsed the geometric sum to a closed form on the second line.

Step 2: Verify the Pattern

The next step is to verify the general formula with one more round of plug-and-chug.

$$\begin{aligned}
 T_n &= 2^k T_{n-k} + 2^k - 1 \\
 &= 2^k (2T_{n-(k+1)} + 1) + 2^k - 1 && \text{plug} \\
 &= 2^{k+1} T_{n-(k+1)} + 2^{k+1} - 1 && \text{chug}
 \end{aligned}$$

The final expression on the right is the same as the expression on the first line, except that k is replaced by $k + 1$. Surprisingly, this effectively *proves* that the formula is correct for all k . Here is why: we know the formula holds for $k = 1$, because that’s the original recurrence equation. And we’ve just shown that if the formula holds for some $k \geq 1$, then it also holds for $k + 1$. So the formula holds for all $k \geq 1$ by induction.

Step 3: Write T_n Using Early Terms with Known Values

The last step is to express T_n as a function of early terms whose values are known. Here, choosing $k = n - 1$ expresses T_n in terms of T_1 , which is equal to 1. Simplifying gives a closed-form expression for T_n :

$$\begin{aligned} T_n &= 2^{n-1}T_1 + 2^{n-1} - 1 \\ &= 2^{n-1} \cdot 1 + 2^{n-1} - 1 \\ &= 2^n - 1. \end{aligned}$$

We’re done! This is the same answer we got from guess-and-verify.

Let’s compare guess-and-verify with plug-and-chug. In the guess-and-verify method, we computed several terms at the beginning of the sequence, T_1, T_2, T_3 , etc., until a pattern appeared. We generalized to a formula for the n th term, T_n . In contrast, plug-and-chug works backward from the n th term. Specifically, we started with an expression for T_n involving the preceding term, T_{n-1} , and rewrote this using progressively earlier terms, T_{n-2}, T_{n-3} , etc. Eventually, we noticed a pattern, which allowed us to express T_n using the very first term, T_1 , whose value we knew. Substituting this value gave a closed-form expression for T_n . So guess-and-verify and plug-and-chug tackle the problem from opposite directions.

21.2 Merge Sort

Algorithms textbooks traditionally claim that sorting is an important, fundamental problem in computer science. Then they smack you with sorting algorithms until life as a disk-stacking monk in Hanoi sounds delightful. Here, we’ll cover just *one* well-known sorting algorithm, *Merge Sort*. The analysis introduces another kind of recurrence.

Here is how Merge Sort works. The input is a list of n numbers, and the output is those same numbers in nondecreasing order. There are two cases:

- If the input is a single number, then the algorithm does nothing, because the list is already sorted.
- Otherwise, the list contains two or more numbers. The first half and the second half of the list are each sorted recursively. Then the two halves are merged to form a sorted list with all n numbers.

Let’s work through an example. Suppose we want to sort this list:

10, 7, 23, 5, 2, 8, 6, 9.

Since there is more than one number, the first half (10, 7, 23, 5) and the second half (2, 8, 6, 9) are sorted recursively. The results are 5, 7, 10, 23 and 2, 6, 8, 9. All that remains is to merge these two lists. This is done by repeatedly emitting the smaller of the two leading terms. When one list is empty, the whole other list is emitted. The example is worked out below. In this table, underlined numbers are about to be emitted.

First Half	Second Half	Output
5, 7, 10, 23	<u>2</u> , 6, 8, 9	
<u>5</u> , 7, 10, 23	6, 8, 9	2
7, 10, 23	<u>6</u> , 8, 9	2, 5
<u>7</u> , 10, 23	8, 9	2, 5, 6
10, 23	<u>8</u> , 9	2, 5, 6, 7
10, 23	<u>9</u>	2, 5, 6, 7, 8
<u>10</u> , <u>23</u>		2, 5, 6, 7, 8, 9
		2, 5, 6, 7, 8, 9, 10, 23

The leading terms are initially 5 and 2. So we output 2. Then the leading terms are 5 and 6, so we output 5. Eventually, the second list becomes empty. At that point, we output the whole first list, which consists of 10 and 23. The complete output consists of all the numbers in sorted order.

21.2.1 Finding a Recurrence

A traditional question about sorting algorithms is, “What is the maximum number of comparisons used in sorting n items?” This is taken as an estimate of the running time. In the case of Merge Sort, we can express this quantity with a recurrence. Let T_n be the maximum number of comparisons used while Merge Sorting a list of n numbers. For now, assume that n is a power of 2. This ensures that the input can be divided in half at every stage of the recursion.

- If there is only one number in the list, then no comparisons are required, so $T_1 = 0$.
- Otherwise, T_n includes comparisons used in sorting the first half (at most $T_{n/2}$), in sorting the second half (also at most $T_{n/2}$), and in merging the two halves. The number of comparisons in the merging step is at most $n - 1$. This is because at least one number is emitted after each comparison and one more number is emitted at the end when one list becomes empty. Since n items are emitted in all, there can be at most $n - 1$ comparisons.

Therefore, the maximum number of comparisons needed to Merge Sort n items is given by this recurrence:

$$\begin{aligned} T_1 &= 0 \\ T_n &= 2T_{n/2} + n - 1 \quad (\text{for } n \geq 2 \text{ and a power of } 2). \end{aligned}$$

This fully describes the number of comparisons, but not in a very useful way; a closed-form expression would be much more helpful. To get that, we have to solve the recurrence.

21.2.2 Solving the Recurrence

Let’s first try to solve the Merge Sort recurrence with the guess-and-verify technique. Here are the first few values:

$$\begin{aligned} T_1 &= 0 \\ T_2 &= 2T_1 + 2 - 1 = 1 \\ T_4 &= 2T_2 + 4 - 1 = 5 \\ T_8 &= 2T_4 + 8 - 1 = 17 \\ T_{16} &= 2T_8 + 16 - 1 = 49. \end{aligned}$$

We’re in trouble! Guessing the solution to this recurrence is hard because there is no obvious pattern. So let’s try the plug-and-chug method instead.

Step 1: Plug and Chug Until a Pattern Appears

First, we expand the recurrence equation by alternately plugging and chugging until a pattern appears.

$$\begin{aligned} T_n &= 2T_{n/2} + n - 1 \\ &= 2(2T_{n/4} + n/2 - 1) + (n - 1) && \text{plug} \\ &= 4T_{n/4} + (n - 2) + (n - 1) && \text{chug} \\ &= 4(2T_{n/8} + n/4 - 1) + (n - 2) + (n - 1) && \text{plug} \\ &= 8T_{n/8} + (n - 4) + (n - 2) + (n - 1) && \text{chug} \\ &= 8(2T_{n/16} + n/8 - 1) + (n - 4) + (n - 2) + (n - 1) && \text{plug} \\ &= 16T_{n/16} + (n - 8) + (n - 4) + (n - 2) + (n - 1) && \text{chug} \end{aligned}$$

A pattern is emerging. In particular, this formula seems holds:

$$\begin{aligned} T_n &= 2^k T_{n/2^k} + (n - 2^{k-1}) + (n - 2^{k-2}) + \cdots + (n - 2^0) \\ &= 2^k T_{n/2^k} + kn - 2^{k-1} - 2^{k-2} \cdots - 2^0 \\ &= 2^k T_{n/2^k} + kn - 2^k + 1. \end{aligned}$$

On the second line, we grouped the n terms and powers of 2. On the third, we collapsed the geometric sum.

Step 2: Verify the Pattern

Next, we verify the pattern with one additional round of plug-and-chug. If we guessed the wrong pattern, then this is where we’ll discover the mistake.

$$\begin{aligned} T_n &= 2^k T_{n/2^k} + kn - 2^k + 1 \\ &= 2^k (2T_{n/2^{k+1}} + n/2^k - 1) + kn - 2^k + 1 && \text{plug} \\ &= 2^{k+1} T_{n/2^{k+1}} + (k+1)n - 2^{k+1} + 1 && \text{chug} \end{aligned}$$

The formula is unchanged except that k is replaced by $k+1$. This amounts to the induction step in a proof that the formula holds for all $k \geq 1$.

Step 3: Write T_n Using Early Terms with Known Values

Finally, we express T_n using early terms whose values are known. Specifically, if we let $k = \log n$, then $T_{n/2^k} = T_1$, which we know is 0:

$$\begin{aligned} T_n &= 2^k T_{n/2^k} + kn - 2^k + 1 \\ &= 2^{\log n} T_{n/2^{\log n}} + n \log n - 2^{\log n} + 1 \\ &= nT_1 + n \log n - n + 1 \\ &= n \log n - n + 1. \end{aligned}$$

We’re done! We have a closed-form expression for the maximum number of comparisons used in Merge Sorting a list of n numbers. In retrospect, it is easy to see why guess-and-verify failed: this formula is fairly complicated.

As a check, we can confirm that this formula gives the same values that we computed earlier:

n	T_n	$n \log n - n + 1$
1	0	$1 \log 1 - 1 + 1 = 0$
2	1	$2 \log 2 - 2 + 1 = 1$
4	5	$4 \log 4 - 4 + 1 = 5$
8	17	$8 \log 8 - 8 + 1 = 17$
16	49	$16 \log 16 - 16 + 1 = 49$

As a double-check, we could write out an explicit induction proof. This would be straightforward, because we already worked out the guts of the proof in step 2 of the plug-and-chug procedure.

21.3 Linear Recurrences

So far we’ve solved recurrences with two techniques: guess-and-verify and plug-and-chug. These methods require spotting a pattern in a sequence of numbers or expressions. In this section and the next, we’ll give cookbook solutions for two large classes of recurrences. These methods require no flash of insight; you just follow the recipe and get the answer.

21.3.1 Climbing Stairs

How many different ways are there to climb n stairs, if you can either step up one stair or hop up two? For example, there are five different ways to climb four stairs:

1. step, step, step, step
2. hop, hop
3. hop, step, step
4. step, hop step
5. step, step, hop

Working through this problem will demonstrate the major features of our first cookbook method for solving recurrences. We’ll fill in the details of the general solution afterward.

Finding a Recurrence

As special cases, there is 1 way to climb 0 stairs (do nothing) and 1 way to climb 1 stair (step up). In general, an ascent of n stairs consists of either a step followed by an ascent of the remaining $n - 1$ stairs or a hop followed by an ascent of $n - 2$ stairs. So the total number of ways to climb n stairs is equal to the number of ways to climb $n - 1$ plus the number of ways to climb $n - 2$. These observations define a recurrence:

$$\begin{aligned} f(0) &= 1 \\ f(1) &= 1 \\ f(n) &= f(n - 1) + f(n - 2) \quad \text{for } n \geq 2. \end{aligned}$$

Here, $f(n)$ denotes the number of ways to climb n stairs. Also, we’ve switched from subscript notation to functional notation, from T_n to f_n . Here the change is cosmetic, but the expressiveness of functions will be useful later.

This is the Fibonacci recurrence, the most famous of all recurrence equations. Fibonacci numbers arise in all sorts of applications and in nature. Fibonacci introduced the numbers in 1202 to study rabbit reproduction. Fibonacci numbers also appear, oddly enough, in the spiral patterns on the faces of sunflowers. And the input numbers that make Euclid’s GCD algorithm require the greatest number of steps are consecutive Fibonacci numbers.

Solving the Recurrence

The Fibonacci recurrence belongs to the class of linear recurrences, which are essentially all solvable with a technique that you can learn in an hour. This is somewhat amazing, since the Fibonacci recurrence remained unsolved for almost six centuries!

In general, a *homogeneous linear recurrence* has the form

$$f(n) = a_1 f(n-1) + a_2 f(n-2) + \cdots + a_d f(n-d)$$

where $a_1, a_2, \dots, a_d$ and d are constants. The *order* of the recurrence is d . Commonly, the value of the function f is also specified at a few points; these are called *boundary conditions*. For example, the Fibonacci recurrence has order $d = 2$ with coefficients $a_1 = a_2 = 1$ and $g(n) = 0$. The boundary conditions are $f(0) = 1$ and $f(1) = 1$. The word “homogeneous” sounds scary, but effectively means “the simpler kind.” We’ll consider linear recurrences with a more complicated form later.

Let’s try to solve the Fibonacci recurrence with the benefit centuries of hindsight. In general, linear recurrences tend to have exponential solutions. So let’s guess that

$$f(n) = x^n$$

where x is a parameter introduced to improve our odds of making a correct guess. We’ll figure out the best value for x later. To further improve our odds, let’s neglect the boundary conditions, $f(0) = 0$ and $f(1) = 1$, for now. Plugging this guess into the recurrence $f(n) = f(n-1) + f(n-2)$ gives

$$x^n = x^{n-1} + x^{n-2}.$$

Dividing both sides by x^{n-2} leaves a quadratic equation:

$$x^2 = x + 1.$$

Solving this equation gives *two* plausible values for the parameter x :

$$x = \frac{1 \pm \sqrt{5}}{2}.$$

This suggests that there are at least two different solutions to the recurrence, neglecting the boundary conditions.

$$f(n) = \left(\frac{1 + \sqrt{5}}{2}\right)^n \quad \text{or} \quad f(n) = \left(\frac{1 - \sqrt{5}}{2}\right)^n$$

A charming features of homogeneous linear recurrences is that any linear combination of solutions is another solution.

Theorem 21.3.1. *If $f(n)$ and $g(n)$ are both solutions to a homogeneous linear recurrence, then $h(n) = sf(n) + tg(n)$ is also a solution for all $s, t \in \mathbb{R}$.*

Proof.

$$\begin{aligned} h(n) &= sf(n) + tg(n) \\ &= s(a_1f(n-1) + \cdots + a_df(n-d)) + t(a_1g(n-1) + \cdots + a_dg(n-d)) \\ &= a_1(sf(n-1) + tg(n-1)) + \cdots + a_d(sf(n-d) + tg(n-d)) \\ &= a_1h(n-1) + \cdots + a_dh(n-d) \end{aligned}$$

The first step uses the definition of the function h , and the second uses the fact that f and g are solutions to the recurrence. In the last two steps, we rearrange terms and use the definition of h again. Since the first expression is equal to the last, h is also a solution to the recurrence. ■

The phenomenon described in this theorem—a linear combination of solutions is another solution—also holds for many differential equations and physical systems. In fact, linear recurrences are so similar to linear differential equations that you can safely snooze through that topic in some future math class.

Returning to the Fibonacci recurrence, this theorem implies that

$$f(n) = s \left(\frac{1 + \sqrt{5}}{2}\right)^n + t \left(\frac{1 - \sqrt{5}}{2}\right)^n$$

is a solution for all real numbers s and t . The theorem expanded two solutions to a whole spectrum of possibilities! Now, given all these options to choose from, we can find one solution that satisfies the boundary conditions, $f(0) = 1$ and $f(1) = 1$. Each boundary condition puts some constraints on the parameters s and t . In particular, the first boundary condition implies that

$$f(0) = s \left(\frac{1 + \sqrt{5}}{2}\right)^0 + t \left(\frac{1 - \sqrt{5}}{2}\right)^0 = s + t = 1.$$

Similarly, the second boundary condition implies that

$$f(1) = s \left(\frac{1 + \sqrt{5}}{2} \right)^1 + t \left(\frac{1 - \sqrt{5}}{2} \right)^1 = 1.$$

Now we have two linear equations in two unknowns. The system is not degenerate, so there is a unique solution:

$$s = \frac{1}{\sqrt{5}} \cdot \frac{1 + \sqrt{5}}{2} \quad t = -\frac{1}{\sqrt{5}} \cdot \frac{1 - \sqrt{5}}{2}.$$

These values of s and t identify a solution to the Fibonacci recurrence that also satisfies the boundary conditions:

$$\begin{aligned} f(n) &= \frac{1}{\sqrt{5}} \cdot \frac{1 + \sqrt{5}}{2} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \cdot \frac{1 - \sqrt{5}}{2} \left(\frac{1 - \sqrt{5}}{2} \right)^n \\ &= \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1}. \end{aligned}$$

It is easy to see why no one stumbled across this solution for almost six centuries. All Fibonacci numbers are integers, but this expression is full of square roots of five! Amazingly, the square roots always cancel out. This expression really does give the Fibonacci numbers if we plug in $n = 0, 1, 2$, etc.

This closed-form for Fibonacci numbers has some interesting corollaries. The first term tends to infinity because the base of the exponential, $(1 + \sqrt{5})/2 = 1.618\dots$ is greater than one. This value is often denoted ϕ and called the “golden ratio.” The second term tends to zero, because $(1 - \sqrt{5})/2 = -0.618033988\dots$ has absolute value less than 1. This implies that the n th Fibonacci number is:

$$f(n) = \frac{\phi^{n+1}}{\sqrt{5}} + o(1).$$

Remarkably, this expression involving irrational numbers is actually very close to an integer for all large n —namely, a Fibonacci number! For example:

$$\frac{\phi^{20}}{\sqrt{5}} = 6765.000029\dots \approx f(19).$$

This also implies that the ratio of consecutive Fibonacci numbers rapidly approaches the golden ratio. For example:

$$\frac{f(20)}{f(19)} = \frac{10946}{6765} = 1.618033998\dots$$

21.3.2 Solving Homogeneous Linear Recurrences

The method we used to solve the Fibonacci recurrence can be extended to solve any homogeneous linear recurrence; that is, a recurrence of the form

$$f(n) = a_1 f(n-1) + a_2 f(n-2) + \cdots + a_d f(n-d)$$

where $a_1, a_2, \dots, a_d$ and d are constants. Substituting the guess $f(n) = x^n$, as with the Fibonacci recurrence, gives

$$x^n = a_1 x^{n-1} + a_2 x^{n-2} + \cdots + a_d x^{n-d}.$$

Dividing by x^{n-d} gives

$$x^d = a_1 x^{d-1} + a_2 x^{d-2} + \cdots + a_{d-1} x + a_d.$$

This is called the *characteristic equation* of the recurrence. The characteristic equation can be read off quickly since the coefficients of the equation are the same as the coefficients of the recurrence.

The solutions to a linear recurrence are defined by the roots of the characteristic equation. Neglecting boundary conditions for the moment:

- If r is a nonrepeated root of the characteristic equation, then r^n is a solution to the recurrence.
- If r is a repeated root with multiplicity k then $r^n, nr^n, n^2r^n, \dots, n^{k-1}r^n$ are all solutions to the recurrence.

Theorem 21.3.1 implies that every linear combination of these solutions is also a solution.

For example, suppose that the characteristic equation of a recurrence has roots s , t , and u twice. These four roots imply four distinct solutions:

$$f(n) = s^n \quad f(n) = t^n \quad f(n) = u^n \quad f(n) = nu^n.$$

Furthermore, every linear combination

$$f(n) = a \cdot s^n + b \cdot t^n + c \cdot u^n + d \cdot nu^n \tag{21.1}$$

is also a solution.

All that remains is to select a solution consistent with the boundary conditions by choosing the constants appropriately. Each boundary condition implies a linear equation involving these constants. So we can determine the constants by solving a system of linear equations. For example, suppose our boundary conditions were

$f(0) = 0$, $f(1) = 1$, $f(2) = 4$, and $f(3) = 9$. Then we would obtain four equations in four unknowns:

$$\begin{array}{llll} f(0) = 0 & \text{implies} & a \cdot s^0 + b \cdot t^0 + c \cdot u^0 + d \cdot 0u^0 = 0 \\ f(1) = 1 & \text{implies} & a \cdot s^1 + b \cdot t^1 + c \cdot u^1 + d \cdot 1u^1 = 1 \\ f(2) = 4 & \text{implies} & a \cdot s^2 + b \cdot t^2 + c \cdot u^2 + d \cdot 2u^2 = 4 \\ f(3) = 9 & \text{implies} & a \cdot s^3 + b \cdot t^3 + c \cdot u^3 + d \cdot 3u^3 = 9 \end{array}$$

This looks nasty, but remember that s , t , and u are just constants. Solving this system gives values for a , b , c , and d that define a solution to the recurrence consistent with the boundary conditions.

21.3.3 Solving General Linear Recurrences

We can now solve all linear homogeneous recurrences, which have the form

$$f(n) = a_1 f(n-1) + a_2 f(n-2) + \cdots + a_d f(n-d).$$

Many recurrences that arise in practice do not quite fit this mold. For example, the Towers of Hanoi problem led to this recurrence:

$$\begin{aligned} f(1) &= 1 \\ f(n) &= 2f(n-1) + 1 \end{aligned} \quad (\text{for } n \geq 2).$$

The problem is the extra $+1$; that is not allowed in a homogeneous linear recurrence. In general, adding an extra function $g(n)$ to the right side of a linear recurrence gives an *inhomogeneous linear recurrence*:

$$f(n) = a_1 f(n-1) + a_2 f(n-2) + \cdots + a_d f(n-d) + g(n).$$

Solving inhomogeneous linear recurrences is neither very different nor very difficult. We can divide the whole job into five steps:

1. Replace $g(n)$ by 0, leaving a homogeneous recurrence. As before, find roots of the characteristic equation.
2. Write down the solution to the homogeneous recurrence, but do not yet use the boundary conditions to determine coefficients. This is called the *homogeneous solution*.
3. Now restore $g(n)$ and find a single solution to the recurrence, ignoring boundary conditions. This is called a *particular solution*. We'll explain how to find a particular solution shortly.

4. Add the homogeneous and particular solutions together to obtain the *general solution*.
5. Now use the boundary conditions to determine constants by the usual method of generating and solving a system of linear equations.

As an example, let’s consider a variation of the Towers of Hanoi problem. Suppose that moving a disk takes time proportional to its size. Specifically, moving the smallest disk takes 1 second, the next-smallest takes 2 seconds, and moving the n th disk then requires n seconds instead of 1. So, in this variation, the time to complete the job is given by a recurrence with a $+n$ term instead of a $+1$:

$$\begin{aligned} f(1) &= 1 \\ f(n) &= 2f(n-1) + n \quad \text{for } n \geq 2. \end{aligned}$$

Clearly, this will take longer, but how much longer? Let’s solve the recurrence with the method described above.

In Steps 1 and 2, dropping the $+n$ leaves the homogeneous recurrence $f(n) = 2f(n-1)$. The characteristic equation is $x = 2$. So the homogeneous solution is $f(n) = c2^n$.

In Step 3, we must find a solution to the full recurrence $f(n) = 2f(n-1) + n$, without regard to the boundary condition. Let’s guess that there is a solution of the form $f(n) = an + b$ for some constants a and b . Substituting this guess into the recurrence gives

$$\begin{aligned} an + b &= 2(a(n-1) + b) + n \\ 0 &= (a+1)n + (b-2a). \end{aligned}$$

The second equation is a simplification of the first. The second equation holds for all n if both $a+1 = 0$ (which implies $a = -1$) and $b-2a = 0$ (which implies that $b = -2$). So $f(n) = an + b = -n - 2$ is a particular solution.

In the Step 4, we add the homogeneous and particular solutions to obtain the general solution

$$f(n) = c2^n - n - 2.$$

Finally, in step 5, we use the boundary condition, $f(1) = 1$, determine the value of the constant c :

$$\begin{aligned} f(1) = 1 & \quad \text{IMPLIES} \quad c2^1 - 1 - 2 = 1 \\ & \quad \text{IMPLIES} \quad c = 2. \end{aligned}$$

Therefore, the function $f(n) = 2 \cdot 2^n - n - 2$ solves this variant of the Towers of Hanoi recurrence. For comparison, the solution to the original Towers of Hanoi problem was $2^n - 1$. So if moving disks takes time proportional to their size, then the monks will need about twice as much time to solve the whole puzzle.

21.3.4 How to Guess a Particular Solution

Finding a particular solution can be the hardest part of solving inhomogeneous recurrences. This involves guessing, and you might guess wrong.¹ However, some rules of thumb make this job fairly easy most of the time.

- Generally, look for a particular solution with the same form as the inhomogeneous term $g(n)$.
- If $g(n)$ is a constant, then guess a particular solution $f(n) = c$. If this doesn't work, try polynomials of progressively higher degree: $f(n) = bn + c$, then $f(n) = an^2 + bn + c$, etc.
- More generally, if $g(n)$ is a polynomial, try a polynomial of the same degree, then a polynomial of degree one higher, then two higher, etc. For example, if $g(n) = 6n + 5$, then try $f(n) = bn + c$ and then $f(n) = an^2 + bn + c$.
- If $g(n)$ is an exponential, such as 3^n , then first guess that $f(n) = c3^n$. Failing that, try $f(n) = bn3^n + c3^n$ and then $an^23^n + bn3^n + c3^n$, etc.

The entire process is summarized on the following page.

21.4 Divide-and-Conquer Recurrences

We now have a recipe for solving general linear recurrences. But the Merge Sort recurrence, which we encountered earlier, is not linear:

$$\begin{aligned} T(1) &= 0 \\ T(n) &= 2T(n/2) + n - 1 \quad (\text{for } n \geq 2). \end{aligned}$$

In particular, $T(n)$ is not a linear combination of a fixed number of immediately preceding terms; rather, $T(n)$ is a function of $T(n/2)$, a term halfway back in the sequence.

¹Chapter 16 explains how to solve linear recurrences with generating functions—it's a little more complicated, but it does not require guessing.

Short Guide to Solving Linear Recurrences

A linear recurrence is an equation

$$f(n) = \underbrace{a_1 f(n-1) + a_2 f(n-2) + \cdots + a_d f(n-d)}_{\text{homogeneous part}} + \underbrace{g(n)}_{\text{inhomogeneous part}}$$

together with boundary conditions such as $f(0) = b_0$, $f(1) = b_1$, etc. Linear recurrences are solved as follows:

1. Find the roots of the characteristic equation

$$x^n = a_1 x^{n-1} + a_2 x^{n-2} + \cdots + a_{k-1} x + a_k.$$

2. Write down the homogeneous solution. Each root generates one term and the homogeneous solution is their sum. A nonrepeated root r generates the term $c r^n$, where c is a constant to be determined later. A root r with multiplicity k generates the terms

$$d_1 r^n \quad d_2 n r^n \quad d_3 n^2 r^n \quad \dots \quad d_k n^{k-1} r^n$$

where $d_1, \dots, d_k$ are constants to be determined later.

3. Find a particular solution. This is a solution to the full recurrence that need not be consistent with the boundary conditions. Use guess-and-verify. If $g(n)$ is a constant or a polynomial, try a polynomial of the same degree, then of one higher degree, then two higher. For example, if $g(n) = n$, then try $f(n) = bn + c$ and then $an^2 + bn + c$. If $g(n)$ is an exponential, such as 3^n , then first guess $f(n) = c3^n$. Failing that, try $f(n) = (bn + c)3^n$ and then $(an^2 + bn + c)3^n$, etc.
4. Form the general solution, which is the sum of the homogeneous solution and the particular solution. Here is a typical general solution:

$$f(n) = \underbrace{c2^n + d(-1)^n}_{\text{homogeneous solution}} + \underbrace{3n + 1}_{\text{inhomogeneous solution}}.$$

5. Substitute the boundary conditions into the general solution. Each boundary condition gives a linear equation in the unknown constants. For example, substituting $f(1) = 2$ into the general solution above gives

$$2 = c \cdot 2^1 + d \cdot (-1)^1 + 3 \cdot 1 + 1$$

$$\text{IMPLIES} \quad -2 = 2c - d.$$

Determine the values of these constants by solving the resulting system of linear equations.

Merge Sort is an example of a divide-and-conquer algorithm: it divides the input, “conquers” the pieces, and combines the results. Analysis of such algorithms commonly leads to *divide-and-conquer* recurrences, which have this form:

$$T(n) = \sum_{i=1}^k a_i T(b_i n) + g(n)$$

Here $a_1, \dots, a_k$ are positive constants, $b_1, \dots, b_k$ are constants between 0 and 1, and $g(n)$ is a nonnegative function. For example, setting $a_1 = 2$, $b_1 = 1/2$, and $g(n) = n - 1$ gives the Merge Sort recurrence.

21.4.1 The Akra-Bazzi Formula

The solution to virtually all divide and conquer solutions is given by the amazing *Akra-Bazzi formula*. Quite simply, the asymptotic solution to the general divide-and-conquer recurrence

$$T(n) = \sum_{i=1}^k a_i T(b_i n) + g(n)$$

is

$$T(n) = \Theta \left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du \right) \right) \quad (21.2)$$

where p satisfies

$$\sum_{i=1}^k a_i b_i^p = 1. \quad (21.3)$$

A rarely-troublesome requirement is that the function $g(n)$ must not grow or oscillate too quickly. Specifically, $|g'(n)|$ must be bounded by some polynomial. So, for example, the Akra-Bazzi formula is valid when $g(n) = x^2 \log n$, but not when $g(n) = 2^n$.

Let’s solve the Merge Sort recurrence again, using the Akra-Bazzi formula instead of plug-and-chug. First, we find the value p that satisfies

$$2 \cdot (1/2)^p = 1.$$

Looks like $p = 1$ does the job. Then we compute the integral:

$$\begin{aligned} T(n) &= \Theta \left(n \left(1 + \int_1^n \frac{u-1}{u^2} du \right) \right) \\ &= \Theta \left(n \left(1 + \left[\log u + \frac{1}{u} \right]_1^n \right) \right) \\ &= \Theta \left(n \left(\log n + \frac{1}{n} \right) \right) \\ &= \Theta(n \log n). \end{aligned}$$

The first step is integration and the second is simplification. We can drop the $1/n$ term in the last step, because the $\log n$ term dominates. We’re done!

Let’s try a scary-looking recurrence:

$$T(n) = 2T(n/2) + (8/9)T(3n/4) + n^2.$$

Here, $a_1 = 2$, $b_1 = 1/2$, $a_2 = 8/9$, and $b_2 = 3/4$. So we find the value p that satisfies

$$2 \cdot (1/2)^p + (8/9)(3/4)^p = 1.$$

Equations of this form don’t always have closed-form solutions, so you may need to approximate p numerically sometimes. But in this case the solution is simple: $p = 2$. Then we integrate:

$$\begin{aligned} T(n) &= \Theta \left(n^2 \left(1 + \int_1^n \frac{u^2}{u^3} du \right) \right) \\ &= \Theta(n^2(1 + \log n)) \\ &= \Theta(n^2 \log n). \end{aligned}$$

That was easy!

21.4.2 Two Technical Issues

Until now, we’ve swept a couple issues related to divide-and-conquer recurrences under the rug. Let’s address those issues now.

First, the Akra-Bazzi formula makes no use of boundary conditions. To see why, let’s go back to Merge Sort. During the plug-and-chug analysis, we found that

$$T_n = nT_1 + n \log n - n + 1.$$

This expresses the n th term as a function of the first term, whose value is specified in a boundary condition. But notice that $T_n = \Theta(n \log n)$ for *every* value of T_1 . The boundary condition doesn’t matter!

This is the typical situation: *the asymptotic solution to a divide-and-conquer recurrence is independent of the boundary conditions*. Intuitively, if the bottom-level operation in a recursive algorithm takes, say, twice as long, then the overall running time will at most double. This matters in practice, but the factor of 2 is concealed by asymptotic notation. There are corner-case exceptions. For example, the solution to $T(n) = 2T(n/2)$ is either $\Theta(n)$ or zero, depending on whether $T(1)$ is zero. These cases are of little practical interest, so we won’t consider them further.

There is a second nagging issue with divide-and-conquer recurrences that does not arise with linear recurrences. Specifically, dividing a problem of size n may create subproblems of non-integer size. For example, the Merge Sort recurrence contains the term $T(n/2)$. So what if n is 15? How long does it take to sort seven-and-a-half items? Previously, we dodged this issue by analyzing Merge Sort only when the size of the input was a power of 2. But then we don’t know what happens for an input of size, say, 100.

Of course, a practical implementation of Merge Sort would split the input *approximately* in half, sort the halves recursively, and merge the results. For example, a list of 15 numbers would be split into lists of 7 and 8. More generally, a list of n numbers would be split into approximate halves of size $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. So the maximum number of comparisons is actually given by this recurrence:

$$\begin{aligned} T(1) &= 0 \\ T(n) &= T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n - 1 \quad (\text{for } n \geq 2). \end{aligned}$$

This may be rigorously correct, but the ceiling and floor operations make the recurrence hard to solve exactly.

Fortunately, *the asymptotic solution to a divide and conquer recurrence is unaffected by floors and ceilings*. More precisely, the solution is not changed by replacing a term $T(b_i n)$ with either $T(\lceil b_i n \rceil)$ or $T(\lfloor b_i n \rfloor)$. So leaving floors and ceilings out of divide-and-conquer recurrences makes sense in many contexts; those are complications that make no difference.

21.4.3 The Akra-Bazzi Theorem

The Akra-Bazzi formula together with our assertions about boundary conditions and integrality all follow from the *Akra-Bazzi Theorem*, which is stated below.

Theorem 21.4.1 (Akra-Bazzi). *Suppose that the function $T : \mathbb{R} \rightarrow \mathbb{R}$ satisfies the*

recurrence

$$T(x) \begin{cases} \text{is nonnegative and bounded} & \text{for } 0 \leq x \leq x_0, \\ = \sum_{i=1}^k a_i T(b_i x + h_i(x)) + g(x) & \text{for } x > x_0. \end{cases}$$

where:

1. $a_1, \dots, a_k$ are positive constants.
2. $b_1, \dots, b_k$ are constants between 0 and 1.
3. x_0 is large enough so that T is well-defined.
4. $g(x)$ is a nonnegative function such that $|g'(x)|$ is bounded by a polynomial.
5. $|h_i(x)| = O(x/\log^2 x)$.

Then

$$T(x) = \Theta \left(x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}} du \right) \right)$$

where p satisfies

$$\sum_{i=1}^k a_i b_i^p = 1.$$

The Akra-Bazzi theorem can be proved using a complicated induction argument, though we won't do that here. But let's at least go over the statement of the theorem.

All the recurrences we've considered were defined over the integers, and that is the common case. But the Akra-Bazzi theorem applies more generally to functions defined over the real numbers.

The Akra-Bazzi formula is lifted directly from the theorem statement, except that the recurrence in the theorem includes extra functions, h_i . These functions extend the theorem to address floors, ceilings, and other small adjustments to the sizes of subproblems. The trick is illustrated by this combination of parameters

$$\begin{array}{lll} a_1 = 1 & b_1 = 1/2 & h_1(x) = \left\lceil \frac{x}{2} \right\rceil - \frac{x}{2} \\ a_2 = 1 & b_2 = 1/2 & h_2(x) = \left\lfloor \frac{x}{2} \right\rfloor - \frac{x}{2} \\ & & g(x) = x - 1 \end{array}$$

which corresponds the recurrence

$$\begin{aligned} T(x) &= 1 \cdot T\left(\frac{x}{2} + \left(\left\lceil \frac{x}{2} \right\rceil - \frac{x}{2}\right)\right) + T\left(\frac{x}{2} + \left(\left\lfloor \frac{x}{2} \right\rfloor - \frac{x}{2}\right)\right) + x - 1 \\ &= T\left(\left\lceil \frac{x}{2} \right\rceil\right) + T\left(\left\lfloor \frac{x}{2} \right\rfloor\right) + x - 1. \end{aligned}$$

This is the rigorously correct Merge Sort recurrence valid for all input sizes, complete with floor and ceiling operators. In this case, the functions $h_1(x)$ and $h_2(x)$ are both at most 1, which is easily $O(x/\log^2 x)$ as required by the theorem statement. These functions h_i do not affect—or even appear in—the asymptotic solution to the recurrence. This justifies our earlier claim that applying floor and ceiling operators to the size of a subproblem does not alter the asymptotic solution to a divide-and-conquer recurrence.

21.4.4 The Master Theorem

There is a special case of the Akra-Bazzi formula known as the Master Theorem that handles some of the recurrences that commonly arise in computer science. It is called the *Master* Theorem because it was proved long before Akra and Bazzi arrived on the scene and, for many years, it was the final word on solving divide-and-conquer recurrences. We include the Master Theorem here because it is still widely referenced in algorithms courses and you can use it without having to know anything about integration.

Theorem 21.4.2 (Master Theorem). *Let T be a recurrence of the form*

$$T(n) = aT\left(\frac{n}{b}\right) + g(n).$$

Case 1: *If $g(n) = O\left(n^{\log_b(a)-\epsilon}\right)$ for some constant $\epsilon > 0$, then*

$$T(n) = \Theta\left(n^{\log_b(a)}\right).$$

Case 2: *If $g(n) = \Theta\left(n^{\log_b(a)} \log^k(n)\right)$ for some constant $k \geq 0$, then*

$$T(n) = \Theta\left(n^{\log_b(a)} \log^{k+1}(n)\right).$$

Case 3: *If $g(n) = \Omega\left(n^{\log_b(a)+\epsilon}\right)$ for some constant $\epsilon > 0$ and $ag(n/b) < cg(n)$ for some constant $c < 1$ and sufficiently large n , then*

$$T(n) = \Theta(g(n)).$$

The Master Theorem can be proved by induction on n or, more easily, as a corollary of Theorem 21.4.1. We will not include the details here.

21.5 A Feel for Recurrences

We’ve guessed and verified, plugged and chugged, found roots, computed integrals, and solved linear systems and exponential equations. Now let’s step back and look for some rules of thumb. What kinds of recurrences have what sorts of solutions?

Here are some recurrences we solved earlier:

	Recurrence	Solution
Towers of Hanoi	$T_n = 2T_{n-1} + 1$	$T_n \sim 2^n$
Merge Sort	$T_n = 2T_{n/2} + n - 1$	$T_n \sim n \log n$
Hanoi variation	$T_n = 2T_{n-1} + n$	$T_n \sim 2 \cdot 2^n$
Fibonacci	$T_n = T_{n-1} + T_{n-2}$	$T_n \sim (1.618\dots)^{n+1} / \sqrt{5}$

Notice that the recurrence equations for Towers of Hanoi and Merge Sort are somewhat similar, but the solutions are radically different. Merge Sorting $n = 64$ items takes a few hundred comparisons, while moving $n = 64$ disks takes more than 10^{19} steps!

Each recurrence has one strength and one weakness. In the Towers of Hanoi, we broke a problem of size n into two subproblem of size $n - 1$ (which is large), but needed only 1 additional step (which is small). In Merge Sort, we divided the problem of size n into two subproblems of size $n/2$ (which is small), but needed $(n - 1)$ additional steps (which is large). Yet, Merge Sort is faster by a mile!

This suggests that *generating smaller subproblems is far more important to algorithmic speed than reducing the additional steps per recursive call*. For example, shifting to the variation of Towers of Hanoi increased the last term from $+1$ to $+n$, but the solution only doubled. And one of the two subproblems in the Fibonacci recurrence is just *slightly* smaller than in Towers of Hanoi (size $n - 2$ instead of $n - 1$). Yet the solution is exponentially smaller! More generally, linear recurrences (which have big subproblems) typically have exponential solutions, while divide-and-conquer recurrences (which have small subproblems) usually have solutions bounded above by a polynomial.

All the examples listed above break a problem of size n into two smaller problems. How does the number of subproblems affect the solution? For example, suppose we increased the number of subproblems in Towers of Hanoi from 2 to 3, giving this recurrence:

$$T_n = 3T_{n-1} + 1$$

This increases the root of the characteristic equation from 2 to 3, which raises the solution exponentially, from $\Theta(2^n)$ to $\Theta(3^n)$.

Divide-and-conquer recurrences are also sensitive to the number of subproblems. For example, for this generalization of the Merge Sort recurrence:

$$\begin{aligned} T_1 &= 0 \\ T_n &= aT_{n/2} + n - 1. \end{aligned}$$

the Akra-Bazzi formula gives:

$$T_n = \begin{cases} \Theta(n) & \text{for } a < 2 \\ \Theta(n \log n) & \text{for } a = 2 \\ \Theta(n^{\log a}) & \text{for } a > 2. \end{cases}$$

So the solution takes on three completely different forms as a goes from 1.99 to 2.01!

How do boundary conditions affect the solution to a recurrence? We’ve seen that they are almost irrelevant for divide-and-conquer recurrences. For linear recurrences, the solution is usually dominated by an exponential whose base is determined by the number and size of subproblems. Boundary conditions matter greatly only when they give the dominant term a zero coefficient, which changes the asymptotic solution.

So now we have a rule of thumb! The performance of a recursive procedure is usually dictated by the size and number of subproblems, rather than the amount of work per recursive call or time spent at the base of the recursion. In particular, if subproblems are smaller than the original by an additive factor, the solution is most often exponential. But if the subproblems are only a fraction the size of the original, then the solution is typically bounded by a polynomial.

Index

- , set difference, 70
- $(k_1, k_2, \dots, k_m)$ -split of A , 484
- C_n , 320, 341
- I_E , indicator for event E , 636
- $K_{3,3}$, 381
- K_5 , 381
- big omega, 456
- $\Theta()$, 453
- bij, 90
- $\mathbb{C}$, 70
- $\emptyset$, 70
- $::=$, 5
- $\equiv \pmod{n}$, 205
- $\text{Ex}[R]$, expectation of R , 646
- $\text{Ex}^2[R]$, 686
- $\forall$, 6
- Done**, 410
- $\in$, 6
- inj, 86, 90
- $\mathbb{Z}$, 70
- $\mathbb{Z}^-$, 70
- $\cap$, 70
- λ , 73
- $\mathbb{N}$, 6, 70
- $\overline{A}$, 70
- $\phi(n)$, 216
- $\mathbb{Z}^+$, 6
- $\mathcal{P}(A)$, 71
- $\mathbb{Q}$, 70
- $\mathbb{R}$, 70
- $\mathbb{R}^+$, 70
- $\sim$, 451
- $\sim$ (asymptotic equality), 445
- strict, 90
- $\subset$, 70
- $\subseteq$, 70
- surj, 90
- $\cup$, 70
- k -combinations, 487
- k -edge connected, 342
- k -to-1 function, 479
- k -way independent, 610
- $n + 1$ -bit adder, 145
- r -permutation, 517
- IQ, 680, 686
- icr, 350
- while** programs, 410
- 2-D Array, 310
- 2-Layer Array, 310
- 2-dimensional array, 299
- absolute value, 709
- acyclic, 255
- Addition Rule (for generating functions), 543
- adjacency matrix, 251
- adjacent, 316
- Adleman, 213
- Agrawal, 189
- Akra-Bazzi formula, 773
- Akra-Bazzi Theorem, 775
- alphabet, 162
- annuity, 422
- antecedents, 9
- antichain, 268, 284
- antisymmetric, 259, 271
- antisymmetry, 259
- a posteriori, 601
- arrows, 245
- assignment statement, 139, 410
- asymmetric, 258, 282
- asymmetry, 258
- asymptotically equal, 445
- asymptotically smaller, 451

- asymptotic relations, [463](#)
- average, [646](#), [679](#)
- average degree, [318](#), [375](#)
- axiomatic method, [9](#)
- Axiom of Choice, [104](#)
- axioms, [4](#), [8](#)

- Banach-Tarski, [104](#)
- base case, [118](#)
- basis step, [118](#)
- Bayes’ Rule, [601](#)
- Beneš nets, [303](#)
- Bernoulli distribution, [640](#)
- Bernoulli variable, [687](#)
- Bernoulli variables, [636](#)
- biased, [725](#)
- bijection, [522](#)
- Bijection Rule, [471](#)
- bijective, [78](#)
- binary predicate, [53](#)
- binary relation, [76](#)
- Binary relations, [75](#)
- binary trees, [178](#)
- binomial, [485](#)
- binomial coefficient, [486](#)
- binomial coefficients, [518](#), [520](#)
- binomial distribution, [640](#), [644](#), [690](#)
- Binomial Theorem, [486](#), [551](#)
- bin packing, [697](#)
- bipartite graph, [323](#), [327](#), [362](#), [394](#)
 - degree-constrained, [327](#)
- birthday principle, [613](#)
- blocks, [267](#)
- body, [411](#)
- bogus proofs, [18](#)
- Bookkeeper Rule, [562](#)
- Boole’s inequality, [590](#)
- Boolean variables, [36](#)
- Borel-Cantelli Lemma, [721](#)
- bottleneck, [327](#)

- boundary conditions, [765](#)
- branches, [411](#)
- bridge, [390](#)
- Brin, Sergey, [245](#), [735](#)
- buildup error, [344](#)
- busy, [671](#), [672](#)
- butterfly, [301](#)
- butterfly net, [313](#)

- Cancellation, [210](#)
- Cantor’s paradise, [93](#), [105](#)
- cardinality, [90](#)
- carry bit, [56](#)
- CDO, [720](#)
- chain, [266](#), [284](#)
- chain of “iff”, [14](#)
- characteristic equation, [768](#)
- characters, [162](#)
- Chebyshev’s bound, [713](#)
- Chebyshev’s Theorem, [683](#), [695](#)
- Chebyshev bound, [711](#)
- Chernoff Bound, [698](#)
- Chinese Appetizer problem, [681](#)
- Chinese Remainder Theorem, [232](#)
- Choice axiom, [103](#)
- chromatic number, [336](#)
- Church-Turing thesis, [202](#)
- closed form, [542](#), [549](#)
- closed forms, [421](#)
- closed walk, [249](#), [340](#)
- CML, [312](#), [313](#)
- CNF, [45](#)
- codomain, [73](#), [76](#)
- Cohen, [104](#)
- collateralized debt obligation, [720](#)
- colorable, [336](#)
- coloring, [336](#)
 - solid, [352](#)
- combinatorial proof, [419](#), [509](#), [538](#), [539](#)

- common divisor, [193](#)
- communication nets, [245](#)
- compilation, [97](#)
- complement, [70](#)
- Complement Rule, [590](#)
- complete binary tree, [295](#)
- complete bipartite graph, [381](#)
- complete digraph, [273](#)
- complete graph, [319](#), [381](#)
- components, [72](#)
- composing, [75](#)
- composition, [75](#), [88](#), [254](#)
- concatenation, [162](#), [163](#), [250](#)
- conclusion, [9](#), [37](#)
- conditional, [411](#)
- conditional expectation, [649](#)
- conditional probability, [593](#)
- confidence, [717](#)
- confidence level, [696](#), [716](#)
- congestion, [298](#), [313](#)
- congestion for min-latency, [312](#), [313](#)
- congestion of the network, [299](#)
- congruence, [205](#)
- congruent, [205](#)
- conjunctive form, [45](#)
- conjunctive normal form, [45](#), [48](#)
- connected, [341](#), [343](#)
 - k -edge, [343](#)
 - edge, [343](#)
- connected components, [342](#)
- connects, [316](#)
- consequent, [9](#)
- consistent, [105](#)
- continuous faces, [385](#)
- Continuum Hypothesis, [104](#)
- contrapositive, [12](#), [42](#)
- convergence, [541](#)
- converges, [709](#)
- converse, [42](#)
- convex function, [703](#)
- convolution, [547](#)
- Convolution Counting Principle, [562](#)
- Convolution Rule, [552](#)
- corollary, [8](#)
- countable, [94](#), [95](#), [105](#), [107](#)
- countably infinite, [95](#)
- counter model, [54](#)
- coupon collector problem, [663](#)
- cover, [272](#), [326](#)
- covering edge, [272](#)
- critical path, [266](#), [267](#), [268](#)
- cumulative distribution function, [638](#)
- cut edge, [343](#)
- cycle, [249](#), [337](#), [340](#)
 - of length n , [320](#)
- cycle of a graph, [341](#)
- DAG, [243](#), [273](#)
- de Bruijn sequences, [278](#)
- degree, [316](#)
- degree-constrained, [327](#), [499](#), [529](#)
- degree sequence, [522](#)
- DeMorgan's Laws, [46](#)
- depth, [267](#)
- derivative, [544](#)
- Derivative Rule, [545](#)
- describable, [111](#)
- deviation from the mean, [679](#)
- diagonal argument, [97](#)
- diameter, [296](#)
- Die Hard, [191](#), [192](#)
- Difference Rule, [590](#)
- digraphs, [245](#)
- directed acyclic graph (DAG), [255](#)
- directed edge, [247](#)
- directed graph, [247](#)
- Directed graphs, [245](#)
- directed graphs, [243](#)
- discrete faces, [388](#)

- disjoint, [71](#)
- disjunctive form, [44](#)
- disjunctive normal form, [44](#), [48](#)
- distance
 - between vertices, [250](#)
- Distributive Law, [72](#)
- distributive law, [44](#)
- divide-and-conquer, [773](#)
- divides, [187](#)
- divisibility relation, [247](#)
- divisible, [188](#)
- Division Rule, [479](#)
- Division Theorem, [190](#)
- divisor, [188](#)
- DNF, [44](#)
- domain, [52](#), [73](#), [76](#)
- domain of discourse, [52](#), [532](#)
- Dongles, [390](#)
- double letter, [98](#)
- Double or nothing, [584](#)
- double summations, [448](#)
- drawing, [381](#)
- edge connected, [343](#)
- edge cover, [326](#)
- edges, [247](#), [316](#)
- efficient solution, [49](#)
- elements, [69](#)
- Elkies, [6](#)
- empty graph, [319](#), [337](#)
- empty relation, [279](#), [280](#), [282](#), [288](#), [292](#)
- empty sequence, [73](#)
- empty string, [65](#)
- end of chain, [266](#)
- endpoints, [316](#)
- end vertex, [247](#)
- Enigma, [207](#)
- environment, [411](#)
- equivalence class, [269](#), [292](#), [293](#)
- equivalence classes, [292](#)
- equivalence relation, [269](#), [272](#), [292](#), [293](#)
- equivalent, [40](#)
- erasable, [183](#)
- Euclid, [8](#), [188](#), [222](#)
- Euclid’s Algorithm, [193](#)
- Euler, [6](#), [222](#)
 - formula, [392](#)
- Euler’s ϕ function, [216](#)
- Euler’s constant, [445](#)
- Euler’s formula, [399](#)
- Euler’s Theorem, [217](#)
- Euler’s theorem, [235](#)
- Euler circuit, [276](#)
- Euler tour, [276](#)
- evaluation function, [172](#)
- event, [575](#), [589](#)
- events, [635](#)
- exclusive-or, [37](#)
- existential, [51](#)
- expectation, [646](#)
- expected return, [653](#)
- expected value, [570](#), [646](#), [647](#), [679](#)
- exponential backoff, [644](#)
- exponentially, [45](#), [49](#)
- extends F , [352](#)
- Extensionality, [102](#)
- face-down four-card trick, [529](#)
- factor, [188](#)
- factorial function, [422](#)
- factorials, [518](#), [520](#)
- Factoring, [189](#)
- fair, [653](#)
- fair game, [725](#)
- Fast Exponentiation, [139](#)
- father, [513](#)
- Fermat’s Last Theorem, [189](#)
- Fermat’s Little Theorem, [211](#)

- Fermat’s theorem, [231](#)
- Fibonacci, [547](#)
- Fibonacci recurrence, [765](#)
- Fifteen Puzzle, [155](#)
- Floyd’s Invariant Principle, [129](#)
- Foundation, [103](#)
- Four-Color Theorem, [7](#)
- Four Step Method, [616](#)
- four-step method, [620](#)
- Frege, [104](#)
- Frege, Gotlob, [101](#)
- function, [73](#), [77](#)
- Fundamental Theorem of Arithmetic, [199](#)

- Gödel, [104](#), [105](#)
- Gale, [334](#)
- Gauss, [189](#), [205](#)
- general binomial density function, [645](#)
- Generalized Pigeonhole Principle, [493](#)
- Generalized Product Rule, [476](#)
- generating function, [556](#), [561](#)
- Generating Functions, [541](#)
- geometric distribution, [653](#), [653](#)
- geometric series, [541](#)
- geometric sum, [421](#)
- Goldbach’s Conjecture, [51](#), [52](#)
- Goldbach Conjecture, [189](#)
- golden ratio, [195](#), [225](#)
- good count, [185](#), [559](#), [559](#)
- Google, [725](#)
- graph
 - bipartite, [323](#)
 - coloring problem, [336](#)
 - matching, [326](#)
 - perfect, [326](#)
 - shortest path, [253](#)
 - valid coloring, [336](#)
- graph coloring, [336](#)
- graph of R , [76](#)

- gray edge, [352](#)
- greatest common divisors, [187](#)
- grid, [299](#)
- grows unboundedly, [22](#)
- guess-and-verify, [751](#)

- half-adder, [56](#)
- Hall’s Matching Theorem, [324](#)
- Hall’s Theorem, [327](#), [529](#)
- Hall’s theorem, [362](#)
- Halting Problem, [97](#)
- Handshake Lemma, [319](#)
- Hardy, [187](#), [203](#)
- Harmonic number, [444](#)
- Hat-Check problem, [681](#)
- head, [247](#)
- Herman Rubin, [702](#)
- Hoare Logic, [415](#)
- homogeneous linear recurrence, [765](#)
- homogeneous solution, [769](#)
- hypothesis, [37](#)

- identity relation, [282](#), [292](#)
- image, [75](#), [78](#)
- implications, [11](#)
- incident, [316](#)
- Inclusion-Exclusion, [502](#), [504](#)
- inclusion-exclusion for probabilities, [590](#)
- Inclusion-Exclusion Rule, [502](#)
- increasing subsequence, [290](#)
- in-degree, [247](#)
- independence, [605](#)
- independent, [689](#)
- independent random variables, [637](#)
- indicator random variable, [636](#)
- indicator variable, [647](#), [712](#)
- indicator variables, [638](#)
- indirect proof, [16](#)
- Induction, [115](#)

- induction hypothesis, [118](#)
- inductive step, [118](#)
- inference rules, [9](#)
- infinite, [89](#)
- Infinity axiom, [102](#)
- infix notation, [76](#)
- inhomogeneous linear recurrence, [769](#)
- injection relation, [86](#)
- injective, [77](#)
- integer linear combination, [190](#)
- intended profit, [725](#)
- interest rate, [458](#)
- interpreters, [97](#)
- intersection, [70](#)
- Invariant, [191](#)
- invariant, [130](#)
- inverse, [79](#), [83](#)
- inverse image, [79](#)
- irrational, [13](#)
- irreducible, [227](#)
- irreflexive, [257](#), [270](#), [282](#)
- irreflexivity, [257](#)
- isomorphic, [260](#), [281](#), [404](#)

- Kayal, [189](#)
- King Chicken Theorem, [275](#)
- known-plaintext attack, [212](#)

- latency, [298](#)
- latency for min-congestion, [312](#), [313](#)
- Latin square, [360](#)
- lattice basis reduction, [496](#)
- Law of Large Numbers, [695](#)
- leaf, [347](#)
- lemma, [8](#)
- length- n cycle, [320](#)
- length- n walk relation, [255](#)
- length of a walk, [340](#)
- Let’s Make a Deal, [616](#)
- letters, [162](#)

- linear combination, [190](#)
- Linearity of Expectation, [658](#), [659](#)
- linear orders, [261](#)
- literal, [674](#)
- LMC, [312](#), [313](#)
- load balancing, [697](#), [700](#)
- logical deductions, [4](#)
- logical formulas, [4](#)
- lowest terms, [25](#)

- Mapping Rules, [471](#), [493](#)
- Markov’s bound, [713](#)
- Markov’s Theorem, [671](#), [680](#)
- Markov bound, [702](#)
- matched string, [165](#)
- matching, [324](#), [326](#)
- matching birthdays, [693](#)
- matching condition, [325](#)
- mathematical proof, [4](#)
- matrix multiplication, [453](#)
- maximal, [264](#)
- maximum, [264](#)
- maximum dilation, [746](#)
- mean, [14](#), [646](#)
- meaning, [411](#), [413](#)
- median, [649](#)
- Menger, [343](#)
- merge, [249](#), [250](#)
- Merge Sort, [760](#)
- merging vertices, [400](#)
- minimal, [114](#), [264](#), [264](#)
- minimum, [264](#)
- minimum weight spanning tree, [350](#)
- minor, [400](#)
- modulo, [205](#)
- modus ponens, [9](#)
- Monty Hall Problem, [571](#)
- multigraphs, [317](#)
- multinomial coefficient, [484](#)
- multinomials, [486](#)

- Multinomial Theorem, [538](#)
- multiple, [188](#)
- multiplicative, [233](#)
- multiplicative inverse, [208](#)
- Multiplicative Inverses, [208](#)
- multisets, [69](#)
- Murphy’s Law, [705](#)
- mutual independence, [689](#)
- mutually independent, [607](#), [630](#), [638](#), [693](#), [699](#)
- mutually recursive, [557](#)
- neighbors, [326](#), [357](#)
- network latency, [298](#)
- node, [247](#), [316](#)
- nodes, [317](#)
- nonconstant polynomial, [21](#)
- nonconstructive proof, [494](#)
- nondecreasing, [430](#)
- nonincreasing, [431](#)
- non-unique factorization, [227](#)
- norm, [227](#)
- not primes, [22](#)
- numbered tree, [513](#)
- numbered trees, [522](#)
- number of processors, [267](#)
- Number theory, [187](#)
- $o()$, asymptotically smaller, [451](#)
- $O()$, big oh, [452](#)
- $o()$, little oh, [451](#)
- one-sided Chebyshev bound, [713](#)
- optimal spouse, [333](#)
- order, [765](#)
- ordinary generating function, [541](#)
- ordinary induction, [116](#)
- outcome, [573](#), [589](#)
- out-degree, [247](#)
- outside face, [385](#)
- overhang, [434](#)
- packet, [295](#)
- Page, Larry, [245](#), [735](#)
- page rank, [736](#), [738](#)
- Pairing, [102](#)
- pairwise disjoint, [113](#)
- pairwise independence, [689](#)
- pairwise independent, [610](#), [612](#), [690](#), [693](#)
- Pairwise Independent Additivity, [690](#)
- Pairwise Independent Sampling, [694](#), [716](#)
- parallel schedule, [267](#)
- parallel time, [268](#)
- parity, [156](#)
- partial correctness, [138](#)
- partial correctness assertion, [415](#)
- partial fractions, [549](#)
- partial functions, [74](#)
- partial order, [281](#)
- particular solution, [769](#)
- partition, [267](#), [292](#), [323](#)
- partitions, [269](#)
- Pascal’s Identity, [509](#)
- path, [669](#)
- path-total, [271](#)
- perfect graph, [326](#)
- perfect number, [188](#), [222](#)
- permutation, [210](#), [406](#), [478](#), [518](#)
- Perturbation Method, [423](#)
- pessimial spouse, [333](#)
- Pick-4, [699](#)
- pigeonhole principle, [419](#)
- planar drawing, [381](#)
- planar embedding, [387](#), [388](#), [404](#)
- planar graph, [385](#)
- planar graphs, [339](#)
- planar subgraph, [395](#)
- plug-and-chug, [751](#)
- pointwise, [75](#)

- Polyhedra, [397](#)
- polyhedron, [397](#)
- polynomial growth, [49](#)
- polynomial time, [323](#)
- population size, [695](#)
- positive walk relation, [254](#)
- potential, [148](#)
- power set, [71](#), [81](#), [96](#)
- Power Set axiom, [103](#)
- Power sets, [96](#)
- precondition, [415](#)
- predicate, [7](#)
- pre-MST, [352](#)
- preserved, [206](#)
- preserved invariant, [134](#)
- preserved under isomorphism, [322](#)
- Primality Testing, [189](#)
- prime, [5](#), [188](#)
- prime factorization, [224](#)
- Prime Factorization Theorem, [28](#)
- prime number, [188](#)
- Prime Number Theorem, [214](#)
- private key, [215](#)
- probability density function, [638](#)
- probability density function,, [638](#)
- probability function, [589](#), [619](#)
- probability of an event, [589](#)
- probability space, [589](#)
- product of sets, [73](#)
- Product Rule, [473](#), [597](#)
- product rule, [630](#)
- Product Rule (for generating functions), [546](#)
- proof, [8](#)
- proof by contradiction, [16](#)
- proposition, [4](#), [5](#)
- propositional variables, [36](#)
- public key, [215](#)
- public key cryptography, [215](#)
- Pulverizer, [223](#), [231](#)
- Pythagoreans, [397](#)
- quicksort, [644](#)
- quotient, [191](#)
- Rabin cryptosystem, [238](#)
- randomized, [569](#)
- randomized algorithm, [644](#)
- random sample, [717](#)
- random sampling, [713](#)
- random variable, [635](#)
- random variables, [636](#)
- random walk, [669](#), [737](#)
- Random Walks, [725](#)
- range, [75](#)
- rank, [519](#)
- rational, [13](#), [16](#)
- reachability., [134](#)
- reachable states, [134](#)
- recognizable, [98](#)
- recognizes, [98](#)
- recurrence, [440](#), [751](#)
- Recursive data types, [161](#)
- recursive definitions, [161](#)
- reflexive, [254](#), [270](#)
- regular polyhedron, [397](#)
- relation on a set, [76](#)
- relatively prime, [215](#)
- relaxed, [671](#), [672](#)
- remainder, [191](#)
- Replacement axiom, [103](#)
- reversal, [176](#)
- Riemann Hypothesis, [214](#), [214](#)
- right-shift, [544](#)
- Right-Shift Rule, [544](#)
- ripple-carry, [57](#)
- ripple-carry circuit, [145](#)
- Rivest, [213](#)
- root mean square, [685](#)

- round-robin tournament, [274](#)
- routing, [296](#)
- routing problem, [296](#)
- RSA, [213](#), [237](#)
- RSA public key crypto-system, [187](#)
- RSA public key encryption scheme, [219](#)
- Russell, [101](#), [104](#)
- Russell’s Paradox, [101](#), [103](#)
- sample space, [573](#), [589](#)
- sampling, [713](#)
- SAT, [49](#)
- satisfiable, [43](#), [49](#), [60](#), [675](#)
- SAT-solvers, [49](#)
- Saxena, [189](#)
- Scaling Rule, [542](#)
- scheduled at step k , [267](#)
- Schröder-Bernstein, [93](#), [107](#)
- self-loop, [317](#)
- self-loops, [249](#)
- sequence, [72](#)
- sequencing, [411](#)
- set, [69](#)
 - covering, [326](#)
- set difference, [70](#), [80](#)
- Shamir, [213](#)
- Shapley, [334](#)
- simple graph, [316](#)
- Simple graphs, [315](#)
- simple graphs, [243](#)
- smallest counterexample, [27](#)
- solid coloring, [352](#)
- solves, [296](#)
- sound, [10](#)
- spanning subgraph, [350](#)
- spanning tree, [349](#)
- spread, [435](#)
- St. Petersburg paradox, [677](#)
- St. Petersburg Paradox, [707](#)
- stable matching, [329](#)
- standard deviation, [685](#), [686](#), [689](#)
- start vertex, [247](#)
- state graph, [130](#)
- state machines, [243](#)
- stationary distribution, [738](#)
- Stirling’s formula, [669](#)
- store, [412](#)
- strictly bigger, [96](#)
- strictly decreasing, [431](#)
- strictly increasing, [430](#)
- strict partial order, [257](#), [271](#)
- string procedure, [98](#)
- Strong Induction, [124](#)
- strongly connected, [746](#)
- Structural induction, [163](#)
- structural induction, [161](#), [181](#)
- subsequence, [290](#)
- subset, [70](#)
- subset relation, [281](#)
- substitution function, [173](#)
- suit, [519](#)
- summation notation, [27](#)
- Sum Rule, [474](#), [590](#)
- surjection relation, [85](#)
- surjective, [77](#)
- switches, [295](#)
- symbols, [162](#)
- symmetric, [243](#), [271](#), [315](#), [745](#)
- tail, [247](#)
- tails, [644](#)
- tails of the distribution, [644](#)
- terminals, [295](#)
- terms, [72](#)
- test, [411](#)
- tests, [411](#)
- theorems, [8](#)

- topological sort, [264](#)
- total, [77](#)
- total expectation, [651](#)
- total function, [74](#)
- totient function, [216](#)
- tournament digraph, [274](#)
- Towers of Hanoi, [556](#), [753](#)
- trail, [277](#)
- transition, [130](#)
- transition relation, [130](#)
- transitive, [254](#), [271](#), [282](#), [586](#)
- Traveling Salesman Problem, [277](#), [373](#)
- tree diagram, [573](#), [620](#)
- truth tables, [36](#)
- Turing, [202](#), [203](#), [213](#)
- Turing’s code, [203](#), [207](#), [212](#)
- Twin Prime Conjecture, [189](#)
- type-checking, [97](#), [99](#)

- unbiased, [725](#)
- unbiased binomial distribution, [644](#)
- unbounded Gambler’s ruin, [733](#)
- uncountable, [110](#), [113](#)
- undirected, [315](#)
- undirected edge, [316](#)
- uniform, [582](#), [591](#), [641](#)
- uniform distribution, [640](#), [641](#)
- union, [70](#)
- Union axiom, [102](#)
- Union Bound, [591](#)
- unique factorization, [224](#)
- unique factorizations, [226](#)
- Unique Factorization Theorem, [199](#)
- universal, [51](#)
- unlucky, [672](#)

- valid, [43](#)
- valid coloring, [336](#)
- value of an annuity, [424](#)
- variance, [683](#), [692](#), [712](#)

- Venn diagram, [630](#)
- vertex, [247](#), [316](#)
- vertex connected, [343](#)
- vertices, [247](#), [316](#)
- virtual machines, [97](#)

- walk, [373](#)
- walk counting matrix, [252](#)
- walk in a digraph, [248](#)
- walk in a simple graph, [340](#)
- walk relation, [254](#)
- Weak Law of Large Numbers, [695](#), [716](#)
- weakly connected, [277](#)
- weakly decreasing, [147](#), [199](#), [431](#)
- weakly increasing, [430](#)
- weak partial order, [272](#)
- well founded, [114](#)
- Well Ordering, [125](#)
- Well Ordering Principle, [25](#), [117](#), [129](#)
- while loop, [411](#)
- width, [367](#)
- winnings, [653](#)
- wrap, [559](#)

- Zermelo, [104](#)
- Zermelo-Frankel, [9](#)
- Zermelo-Frankel Set Theory, [102](#)
- ZFC, [9](#), [102](#), [104](#), [105](#)
- ZFC axioms, [104](#)

Glossary of Symbols

symbol	meaning
$::=$	is defined to be
$\wedge$	and
$\vee$	or
$\longrightarrow$	implies, if $\dots$, then $\dots$
$\longrightarrow$	state transition
$\neg P, \overline{P}$	not P
$\longleftrightarrow$	iff, equivalent
$\oplus$	xor, exclusive-or
$\exists$	exists
$\forall$	for all
$\in$	is a member of, is in
$\subseteq$	is a (possibly $=$) subset of
$\subset$	is a proper (not $=$) subset of
$\cup$	set union
$\cap$	set intersection
$\overline{A}$	complement of set A
$-$	set difference
$\mathcal{P}(A)$	powerset of set, A
$\emptyset$	the empty set, $\{ \}$
$\mathbb{Z}$	integers
$\mathbb{N}, \mathbb{Z}^{\geq 0}$	nonnegative integers
$\mathbb{Z}^+, \mathbb{N}^+$	positive integers
$\mathbb{Z}^-$	negative integers
$\mathbb{Q}$	rational numbers
$\mathbb{R}$	real numbers
$\mathbb{C}$	complex numbers
$R(X)$	image of set X under binary relation R
R^{-1}	inverse of binary relation R
$R^{-1}(X)$	inverse image of set X under relation R
surj	A surj B iff $\exists f : A \rightarrow B$. f is a surjective <i>function</i>
inj	A inj B iff $\exists R : A \rightarrow B$. R is an injective <i>relation</i>
bij	A bij B iff $\exists f : A \rightarrow B$. f is a bijection

symbol	meaning
$[\leq 1 \text{ in}]$	injective property of a relation
$[\geq 1 \text{ in}]$	surjective property of a relation
$[\leq 1 \text{ out}]$	function property of a relation
$[\geq 1 \text{ out}]$	total property of a relation
$[= 1 \text{ out}, = 1 \text{ in}]$	bijection relation
λ	the empty string/list
A^*	the finite strings over alphabet A
$\text{rev}(s)$	the reversal of string s
$s \cdot t$	concatenation of strings s, t ; $\text{append}(s, t)$
$\#_c(s)$	number of occurrences of character c in string s
$m \mid n$	integer m divides integer n ; m is a factor of n
gcd	greatest common divisor
(k, n)	$\{i \mid k < i < n\}$
$[k, n)$	$\{i \mid k \leq i < n\}$
$(k, n]$	$\{i \mid k < i \leq n\}$
$[k, n]$	$\{i \mid k \leq i \leq n\}$
$\langle u \rightarrow v \rangle$	directed edge from vertex u to vertex v
Id_A	identity relation on set A : $a \text{Id}_A a'$ iff $a = a'$
R^*	path relation of relation R ; reflexive transitive closure of R
R^+	positive path relation of R ; transitive closure of R
$\langle u - v \rangle$	undirected edge connecting vertices u and v
$E(G)$	the edges of graph G
$V(G)$	the vertices of graph G
C_n	the length- n undirected cycle
L_n	the length- n line graph
K_n	the n -vertex complete graph
$L(G)$	the “left” vertices of bipartite graph G
$R(G)$	the “right” vertices of bipartite graph G
$K_{n,m}$	the complete bipartite graph with n left and m right vertices
H_n	the n th Harmonic number $\sum_{i=1}^n 1/i$
$\sim$	asymptotic equality
$n!$	n factorial $::= n \cdot (n-1) \cdots 2 \cdot 1$
$o()$	asymptotic notation “little oh”
$O()$	asymptotic notation “big oh”
$\Theta()$	asymptotic notation “Theta”
$\Omega()$	asymptotic notation “big Omega”
$\omega()$	asymptotic notation “little omega”

symbol	meaning
$\Pr[A]$	probability of event A
$\Pr[A \mid B]$	conditional probability of A given B
I_E	indicator variable for event E
$\text{Ex}[R]$	expectation of random variable R
$\text{Ex}[R \mid A]$	conditional expectation of R given event A
$\text{Var}[R]$	variance of R
σ_R	standard deviation of R